

LÊ TRỌNG AN

KỸ THUẬT CƠ ĐIỆN TỬ

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG
QUẢN LÝ PHƯƠNG TIỆN
GIAO THÔNG TRONG LĨNH VỰC CHO THUÊ
XE TỰ LÁI**

Sinh viên: Lê Trọng An

Mã số sinh viên: 21010389

Khóa: K15

Ngành: Kỹ thuật Cơ Điện tử

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Hà Nội - 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG QUẢN LÝ
PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC
CHO THUÊ XE TỰ LÁI**

Sinh viên: Lê Trọng An

Mã số sinh viên: 21010389

Khóa: K15

Ngành: Kỹ thuật Cơ Điện tử

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Hà Nội – 2026

PHENIKAA UNIVERSITY
FACULTY OF MECHANICAL ENGINEERING AND
MECHATRONICS



MEM710071 – CAPSTONE DESIGN 2
DESIGN OF AN IOT SYSTEM FOR VEHICLE
MANAGEMENT
IN SELF-DRIVE CAR RENTAL SERVICES

Semester: I Academic year: 2025–2026

Project Team:

Student name	Student ID	Cohort
Le Trong An	21010389	K15

Course Instructor: Dr. Nguyen Duc Nam

This report is submitted to the Faculty of Mechanical Engineering and Mechatronics,
School of Engineering, Phenikaa University in partial fulfillment of the requirements
of the Graduation Thesis course MEM710071.

HANOI – 2026

ĐẠI HỌC PHENIKAA
KHOA CƠ KHÍ – CƠ ĐIỆN TỬ



MEM710071
ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG QUẢN LÝ
PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC CHO
THUÊ XE TỰ LÁI**

Học kỳ I Năm học 2025–2026

Họ và tên sinh viên	Mã số sinh viên	Lớp
Lê Trọng An	21010389	K15- KTCĐT2

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

HÀ NỘI, 2026

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN HƯỚNG DẪN

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Khoa: Cơ khí – Cơ Điện tử.

Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Sinh viên thực hiện: Lê Trọng An.

Lớp: K15-KTCĐT2.

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:

- Nhận xét về hình thức:
- Tính cấp thiết của đề tài:
- Mục tiêu của đề tài:
- Nội dung của đề tài:
- Tài liệu tham khảo:
- Phương pháp nghiên cứu:
- Tính sáng tạo và ứng dụng:

II. Nhận xét về tinh thần và thái độ làm việc của sinh viên:

III. Kết quả đạt được:

IV. Kết luận: Đồng ý cho bảo vệ: ☐ Không đồng ý cho bảo vệ: ☐

Hà Nội, ngày ..., tháng ... năm 20...

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

TS. Nguyễn Đức Nam

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN PHẢN BIỆN

Giảng viên phản biện: Theo quyết định phân công phản biện của Khoa

Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Sinh viên thực hiện: Lê Trọng An

Lớp: K15-KTCĐT2

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:

- Bố cục, hình thức trình bày:
- Đảm bảo tính cấp thiết, hiện đại, không trùng lặp:
- Nét»™i dung:
- Mức độ thực hiện:

II. Kết quả đạt được:

III. Ưu nhược điểm:

IV. Kết luận: Đồng ý cho bảo vệ: ☐ Không đồng ý cho bảo vệ: ☐

Hà Nội, ngày ..., tháng ...năm 20...

GIẢNG VIÊN PHẢN BIỆN

(Ký, ghi rõ họ tên)

Theo quyết định phân công phản biện của Khoa

BIÊN BẢN ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

I. Thông tin chung

- Thời gian: từ ...giờ ...đến ...giờ ...ngày .../.../20...
- Địa điểm:
- Họ và tên sinh viên: Lê Trọng An Mã số SV: 21010389
- Lớp: K15-KTCĐT2 Ngành: Kỹ thuật Cơ Điện tử Khóa: K15
- Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.
- Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

II. Thành phần hội đồng

- Chủ tịch
- Thư ký
- Giảng viên phản biện
- Ủy viên 1

III. Tổng hợp câu hỏi của thành viên hội đồng:

IV. Tổng hợp nội dung trả lời của sinh viên:

V. Nội dung đánh giá của Hội đồng

- Ý nghĩa của đồ án:
- Về nội dung, kết cấu của đồ án:
- Phương pháp nghiên cứu:
- Các kết quả nghiên cứu đạt được:

VI. Điểm kết luận: (Bằng chữ:)

CHỦ TỊCH HỘI ĐỒNG
(Ký, ghi rõ họ tên)

THƯ KÝ
(Ký, ghi rõ họ tên)

GIẢI TRÌNH CÁC CHỈNH SỬA

(Nếu có)

Chưa phát sinh nội dung cần giải trình.

LỜI CAM ĐOAN

Tên tôi là: Lê Trọng An.

Mã sinh viên: 21010389

Lớp: K15-KTCĐT2

Ngành: Kỹ thuật Cơ Điện tử.

Tôi đã thực hiện đồ án tốt nghiệp với đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của TS. Nguyễn Đức Nam.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào, tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

Hà Nội, ngày ..., tháng ...năm 20...

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

SINH VIÊN

(Ký, ghi rõ họ tên)

TS. Nguyễn Đức Nam

Lê Trọng An

TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT

Sự phát triển nhanh của dịch vụ cho thuê xe tự lái tại Việt Nam khiến nhu cầu giám sát phương tiện không còn dừng ở việc biết xe đang ở đâu, mà còn ở việc hiểu xe đang vận hành ra sao và có dấu hiệu bất thường hay không. Từ nhu cầu đó, đồ án này trình bày quá trình thiết kế và triển khai một hệ thống giám sát phương tiện theo hướng trọn gói, bao gồm cả thiết bị gắn trên xe, hạ tầng máy chủ tiếp nhận dữ liệu và giao diện để người quản lý theo dõi, nhằm hỗ trợ doanh nghiệp nắm vị trí xe, trạng thái kỹ thuật và phản ứng sớm với các rủi ro vận hành.

Ở tầng thiết bị, hệ thống được xây dựng trên bo mạch PCB tự thiết kế. Trong đó, ESP32-S3 giữ vai trò bộ điều khiển trung tâm; SIM7600CE-T đảm nhiệm truyền dữ liệu 4G và định vị vệ tinh GNSS để xác định vị trí xe; LIS3DH theo dõi rung động để nhận biết chuyển động khi xe đỗ; còn adapter vgate iCar Pro cho phép đọc dữ liệu OBD2 từ xe qua BLE. Kiến trúc nguồn được tổ chức theo nhiều nhánh để thiết bị vẫn duy trì hoạt động khi xe tắt máy, đồng thời hạn chế ảnh hưởng tới ắc quy chính.

Ở tầng phần mềm, dữ liệu từ thiết bị được gửi bằng MQTT 3.1.1 tới EMQX. MQTT đóng vai trò lớp nhắn tin nhẹ phù hợp với đường truyền 4G, còn EMQX là broker trung tâm tiếp nhận và phân phối bản tin. Từ đây, MQTT Bridge subscribe bốn nhóm topic `v1/{device_id}/{rawdata|status|events|firmware}`, chuẩn hóa payload rồi phân luồng dữ liệu tới PostgreSQL, VictoriaMetrics và VictoriaLogs. Song song với luồng ingest, backend còn lắng nghe nhánh `internal/events/#` để đẩy cập nhật thời gian thực ra giao diện qua Socket.IO.

Bảng điều khiển web được phát triển bằng Next.js 15 và React 19. Điều quan trọng ở đây không phải tên framework, mà là giao diện có thể hiển thị bản đồ, biểu đồ và trạng thái phương tiện gần như theo thời gian thực nhờ cơ chế đẩy dữ liệu mới từ máy chủ lên trình duyệt qua Socket.IO. Nhờ đó, người vận hành không chỉ quan sát vị trí xe mà còn có thể theo dõi lịch sử hành trình, cảnh báo và trạng thái thiết bị trên cùng một giao diện thống nhất.

Kết quả đạt được là một hệ thống IoT giám sát phương tiện hoàn chỉnh từ phần cứng đến phần mềm, có khả năng theo dõi vị trí thời gian thực, đọc dữ liệu chẩn đoán OBD2, thiết lập vùng cho phép, phát sinh cảnh báo tự động, cập nhật firmware từ xa và hỗ trợ quản lý đội xe. Quan trọng hơn, hệ thống cho thấy một hướng triển khai phù hợp với bối cảnh doanh nghiệp cho thuê xe tự lái trong nước: chi phí hợp lý, có thể tùy biến và đủ dư địa để mở rộng trong các giai đoạn sau.

Từ khóa: IoT, giám sát phương tiện, GPS, OBD2, MQTT, ESP32-S3, thời gian thực, vùng cho phép

Abstract (English)

As the self-drive car rental market in Vietnam grows, the need for remote vehicle monitoring and real-time fleet management becomes increasingly practical for operators. This thesis presents the design and development of an end-to-end IoT vehicle tracking system, covering the onboard device, cloud services, and operational dashboard for real-time tracking, diagnostic data collection, and alerting.

On the hardware side, the tracker is implemented as a custom integrated PCB rather than an assembly of off-the-shelf development boards. The main board integrates ESP32-S3 (MCU), SIMCom SIM7600CE-T (LTE+GNSS), LIS3DH (IMU), DS3231M (RTC), W25Q128 (SPI Flash), and power blocks based on MP2482, TPS54231, AP2112, SX1308, and TP5100. The modem is configured to Auto mode (AT+CNMP=2) so it can switch between LTE/UMTS/GSM automatically, uses the default APN internet, and delivers GNSS data via AT+CGNSINF (with optional NMEA streaming through AT+CGNSTST) on the same UART1 channel. The device reads engine diagnostic data via OBD2 through a vgate iCar Pro adapter over Bluetooth Low Energy (BLE), where this adapter is an external peripheral outside the main PCB. The power architecture uses a 5V main rail, an approximately 4V modem rail, a 3.3V logic rail, and a 18650 1S backup cell to maintain operation when the vehicle engine is off.

On the software side, device data is transmitted through MQTT 3.1.1 via EMQX with per-device ACL authorization. The MQTT Bridge subscribes to the device topics `v1/{deviceId}/{rawdata|status|events|firmware}`, validates the payloads, and routes them to PostgreSQL, VictoriaMetrics, and VictoriaLogs. The backend is built with Express.js and TypeScript in a domain-oriented structure, exposes REST/OpenAPI endpoints, and listens to internal MQTT events so Socket.IO channels can push realtime updates and OTA status back to operators.

The web interface is developed using Next.js 15 and React 19, providing a real-time dashboard with Leaflet maps, ECharts visualizations, and WebSocket connectivity via Socket.IO for instant vehicle position and status updates. The entire system is containerized and deployed using Docker, ensuring consistency between development and production environments.

The result is a complete end-to-end IoT vehicle tracking system, from hardware to software, capable of real-time location tracking, OBD2 diagnostic data retrieval, allowed-zone monitoring, automated alerts, remote firmware updates, and fleet-management support for self-drive car rental services.

Keywords: IoT, vehicle tracking, GPS, OBD2, MQTT, ESP32-S3, real-time, allowed-zone monitoring

LỜI CẢM ƠN - ACKNOWLEDGEMENTS

Trước tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Nguyễn Đức Nam — người đã trực tiếp hướng dẫn, định hướng và tận tình hỗ trợ tôi trong suốt quá trình thực hiện đề án tốt nghiệp này. Những góp ý chuyên môn và sự động viên của thầy là nguồn động lực quan trọng giúp tôi hoàn thành đề tài.

Tôi xin trân trọng cảm ơn quý thầy cô trong Khoa Cơ khí - Cơ Điện tử, Trường Kỹ thuật - Đại học Phenikaa đã truyền đạt những kiến thức nền tảng vững chắc trong suốt quá trình học tập, tạo điều kiện thuận lợi để tôi có thể áp dụng vào thực tiễn thông qua đề án này.

Tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các đồng nghiệp đã luôn ủng hộ, chia sẻ và đồng hành cùng tôi trong suốt thời gian thực hiện đề tài.

Mặc dù đã nỗ lực hoàn thiện đề án trong phạm vi thời gian và kiến thức cho phép, nội dung chắc chắn vẫn còn những thiếu sót. Tôi rất mong nhận được những ý kiến góp ý từ quý thầy cô và hội đồng để tiếp tục hoàn thiện đề tài.

Xin chân thành cảm ơn!

Hà Nội, ngày ...tháng ...năm 20...

SINH VIÊN THỰC HIỆN

Lê Trọng An

MỤC LỤC - TABLE OF CONTENT

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	i
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN	iii
BIÊN BẢN ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP	iv
GIẢI TRÌNH CÁC CHỈNH SỬA	v
LỜI CAM ĐOAN	vi
TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT	vii
LỜI CẢM ƠN - ACKNOWLEDGEMENTS	ix
MỤC LỤC - TABLE OF CONTENT	x
DANH MỤC BẢNG - LIST OF TABLES	xv
DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS	xx
DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION	xxiii
CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW	1
1.1. Đặt vấn đề / Bối cảnh của dự án	1
1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam	1
1.1.2. Thách thức kỹ thuật	2
1.1.3. Động lực thực hiện dự án	2
1.2. Mục tiêu và phạm vi của dự án	3
1.2.1. Mục tiêu tổng quát	3
1.2.2. Phạm vi dự án	4
1.3. Các tiêu chí cần đạt được của dự án	6
1.3.1. Tiêu chí phần cứng	6
1.3.2. Tiêu chí firmware	6
1.3.3. Tiêu chí máy chủ và dịch vụ xử lý dữ liệu	7
1.3.4. Tiêu chí giao diện web	7
1.4. Phương pháp tiếp cận thiết kế kỹ thuật	8
1.4.1. Phương pháp luận tổng thể	8
1.4.2. Kiến trúc hệ thống	9
1.4.3. Chiến lược quản lý năng lượng	12
1.4.4. Công nghệ sử dụng	13

1.5. Kết quả và khuyến nghị	15
1.5.1. Kết quả đạt được	15
1.5.2. Khuyến nghị và hướng phát triển	16
Kết luận chương 1	16
CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT	19
2.1. Mô tả vấn đề – Problem statement	19
2.1.1. Bối cảnh thực tế	19
2.1.2. Xác định vấn đề kỹ thuật	19
2.1.3. Mục tiêu kỹ thuật	20
2.2. Bối cảnh và cơ sở kỹ thuật – Background and Technical reviews	21
2.2.1. Tổng quan về hệ thống IoT trong giám sát phương tiện	21
2.2.2. Các giải pháp giám sát phương tiện hiện có	22
2.2.3. So sánh giao thức truyền thông IoT	22
2.2.4. So sánh vi điều khiển (MCU)	24
2.2.5. So sánh cơ sở dữ liệu	25
2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế – Design criteria and Constraints	26
2.3.1. Yêu cầu chức năng	26
2.3.2. Yêu cầu phi chức năng	28
2.3.3. Ràng buộc thiết kế	29
2.3.4. Tiêu chuẩn thiết kế áp dụng	30
2.4. Yêu cầu từ các bên liên quan – Stakeholder requirements	30
2.4.1. Xác định các bên liên quan	30
2.4.2. Ma trận yêu cầu các bên liên quan	31
2.4.3. Phân tích chi tiết yêu cầu	31
2.4.4. Ma trận truy xuất yêu cầu - chức năng	32
Kết luận chương 2	33
CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ – DESIGN SOLUTIONS	34
3.1. Phân tích và đề xuất giải pháp phần cứng – Hardware analysis and solution	34
3.1.1. Phân tích và lựa chọn phần cứng	34
3.1.2. Giải pháp phần cứng	44
3.2. Phân tích và đề xuất giải pháp Firmware – Firmware analysis and solution	56
3.2.1. Phân tích và lựa chọn giải pháp Firmware	56
3.2.2. Giải pháp Firmware	58
3.3. Phân tích và đề xuất kiến trúc máy chủ và xử lý dữ liệu – Server and data-processing architecture	82
3.3.1. Phân tích và lựa chọn kiến trúc máy chủ	82
3.3.2. Giải pháp dịch vụ máy chủ và xử lý nghiệp vụ	84

3.4. Phân tích và đề xuất công nghệ giao diện web – Web interface analysis and solution	99
3.4.1. Phân tích và lựa chọn công nghệ giao diện web	99
3.4.2. Giải pháp giao diện web	101
3.5. Phân tích, đánh giá và lựa chọn phương án khả thi – Analysis, Evaluation and Selection	119
3.6. Tối ưu phương án thiết kế – The optimal solution	120
Kết luận chương 3	120

CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ – IMPLEMENTATION AND RESULTS 123

4.1. Thiết kế chi tiết giải pháp – Detailed design solution	123
4.1.1. Thiết kế chi tiết phần cứng	123
4.1.2. Triển khai firmware, hạ tầng máy chủ và giao diện điều khiển . .	141
4.2. Chế tạo và lắp ráp hệ thống – Manufacture and Assembly	141
4.2.1. Lắp ráp mạch điện tử (Electronics Assembly)	141
4.2.2. Lắp đặt trong xe (Vehicle Installation)	144
4.2.3. Triển khai Firmware	149
4.2.4. Triển khai hệ thống máy chủ	170
4.2.5. Triển khai giao diện web quản trị	188
4.2.6. Cấu hình giám sát hệ thống	195
4.2.7. Checklist làm cứng vận hành trước khi đưa vào môi trường thật .	199
Tóm tắt	201
4.3. Đo lường và kết quả – Measurements and Results	202
4.3.1. Thiết lập môi trường thử nghiệm	202
4.3.2. Kết quả kiểm thử phần cứng	204
4.3.3. Kết quả kiểm thử firmware	214
4.3.4. Kết quả kiểm thử hệ thống máy chủ»§	221
4.3.5. Kiểm thử tích hợp toàn hệ thống (End-to-End Integration Test) .	226
4.3.6. Tổng hợp và so sánh với chỉ tiêu thiết kế	234
4.3.7. Đánh giá độ ổn định vận hành thực tế	237
4.3.8. Kết luận chương 4	239
Tài liệu tham khảo Chương 4 (Phần D)	239

CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ – EVALUATION AND RECOMMENDATIONS 241

5.1. Đánh giá hiệu năng	241
5.1.1. Đánh giá hiệu năng phần cứng	241
5.1.2. Đánh giá hiệu năng firmware	243
5.1.3. Đánh giá hiệu năng hệ thống máy chủ và dịch vụ	244

5.1.4. Đánh giá hiệu năng giao diện người dùng (giao diện web)	245
5.2. Đánh giá kinh tế và môi trường	246
5.2.1. Phân tích chi phí phần cứng	246
5.2.2. Phân tích chi phí hạ tầng đám mây	248
5.2.3. Lợi thế từ công nghệ mã nguồn mở	248
5.2.4. Đánh giá tác động môi trường	249
5.3. Đánh giá rủi ro và biện pháp giảm thiểu	249
5.3.1. Ma trận rủi ro	249
5.3.2. Phân tích chi tiết các rủi ro chính	252
5.3.3. Tổng hợp mức độ rủi ro	253
5.4. Khuyến nghị cho tương lai	253
5.4.1. Giai đoạn 2: Mở rộng tính năng (giai đoạn 2)	253
5.4.2. Ứng dụng trí tuệ nhân tạo và học máy	254
5.4.3. Ứng dụng di động (Mobile Application)	255
5.4.4. Tính toán biên (Edge Computing)	255
5.4.5. Triển khai vận hành thực tế	255
5.4.6. Phần cứng phiên bản 2 (Hardware v2)	256
5.4.7. Tóm tắt lộ trình phát triển	257
Kết luận chương 5	257

CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM – REFLECTION AND LESSONS LEARNED

	258
6.1. Ứng dụng kiến thức kỹ thuật – Application of prior coursework	258
6.1.1. Vi xử lý và Vi điều khiển	258
6.1.2. Mạng máy tính và IoT	259
6.1.3. Cơ sở dữ liệu	259
6.1.4. Lập trình hướng đối tượng	260
6.1.5. Điện tử số và Analog	261
6.1.6. Kỹ thuật phần mềm	261
6.2. Giải quyết các vấn đề kỹ thuật phức tạp – Complex engineering problems	262
6.2.1. Vấn đề 1: Phân tích bản tin OBD2 đa khung qua BLE	262
6.2.2. Vấn đề 2: Đường ống dữ liệu thời gian thực với đảm bảo phân phối	263
6.2.3. Vấn đề 3: Quản lý năng lượng với nhiều nguồn cấp	264
6.2.4. Vấn đề 4: Kiến trúc đám mây có khả năng mở rộng cho IoT	265
6.3. Tác động đạo đức và xã hội – Ethical and social impacts	266
6.3.1. Quyền riêng tư và bảo vệ dữ liệu cá nhân	266
6.3.2. An toàn phương tiện và trách nhiệm kỹ thuật	267
6.3.3. Tác động xã hội tích cực	267
6.3.4. Những lo ngại cần xem xét	268

6.4. Tổng kết và bài học kinh nghiệm – Reflection and Lessons Learned . . .	268
6.4.1. Tầm quan trọng của thiết kế kiến trúc trước khi lập trình	269
6.4.2. Độ phức tạp của hệ thống IoT nhiều tầng	269
6.4.3. Sức mạnh của hệ sinh thái mã nguồn mở	270
6.4.4. Kiểm thử ở mọi tầng là bắt buộc»™c	270
6.4.5. Tổng kết cá nhân	271
Kết luận chương 6	271
Tài liệu tham khảo Chương 6	272
TÀI LIỆU TRÍCH DẪN - REFERENCES	273
[A] Tài liệu tiếng Việt	273
[B] Tài liệu tiếng Anh (Books & Papers)	273
[C] Tài liệu kỹ thuật (Datasheets & Technical Documents)	274
[D] Tài liệu trực tuyến (Online Resources)	275
PHỤ LỤC - APPENDICES	279
PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT	280
1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)	280
1.2. Chi phí hạ tầng cloud (ước tính hàng tháng)	282
1.3. Tổng hợp chi phí dự án	282
PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS	283
2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031–5)	283
2.2. MQTT 3.1.1 trong triển khai thiết bị	284
2.3. Bảo mật thông tin (ISO 27001 - tham khảo)	285
2.4. Thiết kế REST API (RFC 7231 và best practices)	285
PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES	287
3.1. Phân chia giai đoạn dự án	287
3.2. Kế hoạch thực hiện dự án theo giai đoạn	287
3.3. Chi tiết công việc theo giai đoạn	287
PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS	291
4.1. Kho mã nguồn của dự án	291
4.2. Tham chiếu cơ sở dữ liệu (Database Schema)	291
4.3. Tài liệu API (API Documentation)	292
4.4. Tài liệu cấu hình môi trường (Environment Configuration)	293
4.5. Hướng dẫn cài đặt và chạy hệ thống	295

DANH MỤC BẢNG - LIST OF TABLES

1.1	Tóm tắt phạm vi dự án theo từng tầng hệ thống	5
1.3.1A	Tổng hợp thông tin về STT, Tiêu chí, Mục tiêu cụ thể	6
1.2A	Tiêu chí firmware	7
1.2B	Tiêu chí máy chủ và dịch vụ xử lý dữ liệu	7
1.2C	Tiêu chí giao diện web	8
1.3	Tổng hợp công nghệ sử dụng trong dự án	13
2.1	So sánh các giải pháp giám sát phương tiện	22
2.2	So sánh các giao thức truyền thông IoT	23
2.3	So sánh các vi điều khiển cho ứng dụng thiết bị theo dõi IoT	24
2.4	So sánh các giải pháp cơ sở dữ liệu cho hệ thống IoT	25
2.3.4A	Tổng hợp chuẩn và quy ước kỹ thuật áp dụng trong hệ thống	30
2.5	Ma trận yêu cầu các bên liên quan	31
2.6	Ma trận truy xuất yêu cầu các bên liên quan với chức năng hệ thống	33
3.1	Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển	38
3.2	Thông số kỹ thuật ESP32-S3 được chọn	40
3.3	Đối chiếu các phương án LTE/GNSS theo tác động tích hợp	41
3.4	Thông số kỹ thuật phương án SIM7600CE-T	43
3.5	So sánh phương pháp kết nối OBD2 theo thông số triển khai	44
3.6	Chiến lược kết nối BLE OBD2 theo chế độ hoạt động	47
3.7	Thông số kỹ thuật cảm biến LIS3DSH	48
3.8	Thông số khối LDO 3.3V	51
3.9	Thông số khối Buck 5V	51
3.10	Thông số khối Buck 3.8V/4V cho modem	52
3.11	Thông số khối Boost 5V dự phòng	52
3.12	Bảng trạng thái chuyển nguồn và cảnh báo	52
3.13	Thông số khối sạc pin	53
3.14	Thông số kỹ thuật pin dự phòng 18650 1S	53
3.14A	Bảng tổng hợp linh kiện hệ thống thiết bị theo dõi (BOM)	55
3.15	Tổng hợp chi phí theo nhóm	56
3.5A	So sánh các phương án nền tảng firmware	57
3.2.2A	Tổng hợp thông tin về Lớp, Thành phần chính, Vai trò	59
3.2.2B	Tổng hợp thông tin về Tình huống lỗi, Xử lý, Thời gian timeout	67
3.2.2C	Tổng hợp thông tin về Bước, Lệnh AT, Mục đích chính	67
3.2.2D	Tổng hợp thông tin về Tình huống, Biện pháp	69
3.2.2E	Tổng hợp thông tin về GPIO, Chức năng, Hướng	69
3.2.2F	Tổng hợp thông tin về Điều kiện, Trạng thái charger, Lý do	72
3.2.2G	Tổng hợp thông tin về Chế độ, Trạng thái ESP32, Trạng thái modem	73

3.2.2H	Tổng hợp thông tin về Topic, Hướng, QoS	75
3.2.2I	Tổng hợp thông tin về Lệnh, Mục đích, Tham số chính	76
3.2.2J	Tổng hợp thông tin về Trạng thái, Mã, Mô tả	77
3.2.2K	Tổng hợp thông tin về Cảm biến, Phương pháp hiệu chuẩn, Lưu trữ	82
3.15A	So sánh các phương án kiến trúc hệ thống máy chủ	83
3.16	Phân bổ tài nguyên Docker cho các dịch vụ	87
3.17	So sánh các MQTT Broker phổ biến	88
3.18A	So sánh các phương án kiến trúc lưu trữ dữ liệu	89
3.18	So sánh các cơ sở dữ liệu trong hệ thống	91
3.19	Các nhóm bảng chính trong PostgreSQL	92
3.20	So sánh VictoriaMetrics và InfluxDB	94
3.21	So sánh các khung phát triển cho API phía máy chủ	95
3.22	Tóm tắt các nhóm API Endpoints chính	96
3.23	So sánh JWT và cơ chế xác thực theo phiên làm việc	99
3.23A	So sánh các phương án công nghệ giao diện web tổng thể	100
3.24	So sánh các khung phát triển giao diện web	101
3.25	Chiến lược quản lý trạng thái	103
3.26	So sánh thư viện bản đồ	104
3.2.4A	Tổng hợp thông tin về Cột, Mô tả, Chức năng	109
3.27	Ma trận các trang chức năng chính	112
3.2.4B	Tổng hợp thông tin về thành phần bản đồ, file và chức năng	115
3.28	Ngưỡng chiều rộng màn hình và cách điều chỉnh giao diện	117
3.29	Ma trận đánh giá tổng hợp các phương án thiết kế	119
4.1.1A	Tổng hợp thông tin về Thông số, Giá trị	123
4.1	Phân công chân GPIO của ESP32-S3	127
4.2	Khối LDO 3.3V	131
4.3	Khối Buck 5V	134
4.4	Khối Buck 3.8V/4V cho modem	134
4.5	Khối Boost 5V dự phòng	135
4.6	Logic chuyển nguồn tự động	135
4.7	Khối sạc TP5100	138
4.6	Danh sách vật liệu đầy đủ cho hệ thống thiết bị theo dõi	142
4.7	Tổng chi phí ước tính	143
4.5	Công cụ phát triển firmware	151
4.6	Kiến trúc phân tầng firmware	152
4.7	Bảng ánh xạ chân GPIO	153
4.8	Các PID OBD2 được đọc định kỳ (Mode 0x01 - Current Data)	157
4.8A	Bảng bổ sung chỉ mục (4.2.3. Triển khai Firmware)	160
4.9	So sánh các chế độ ngủ	165

4.10	Các trạng thái của firmware	167
4.11	Chuyển đổi trạng thái	167
4.5	Danh sách các dịch vụ được đóng gói bằng Docker và cổng truy cập	176
4.5-cloud	Ánh xạ tên miền công khai, cổng truy cập và dịch vụ nội bộ trong môi trường thử nghiệm	176
4.6	So sánh các khung phát triển API phía máy chủ cho hệ thống IoT .	182
4.7	Danh sách các module nghiệp vụ chính	183
4.8	Tổng hợp các nhóm API endpoint chính	186
4.2.5A	Tổng hợp thông tin về Công nghệ, Phiên bản, Vai trò	189
4.9	Các giai đoạn triển khai giao diện web	193
4.10	Ba trụ cột giám sát hệ thống	195
4.11	Checklist làm cứng vận hành theo mức ưu tiên	199
4.11A	Tổng hợp công nghệ sử dụng trong hệ thống máy chủ	201
4.12	Thiết bị đo lường sử dụng trong thử nghiệm phần cứng	202
4.13	Cấu hình môi trường thử nghiệm phần mềm	203
4.14	Kết quả đo dòng tiêu thụ năng lượng theo chế độ hoạt động	207
4.15	Thời lượng pin dự phòng theo kịch bản sử dụng (quy đổi theo bộ pin tham chiếu 18650)	210
4.16	Kết quả kiểm thử nhiệt độ hoạt động	211
4.17	Kết quả kiểm thử tương thích OBD2 trên các thương hiệu xe	212
4.18	Kết quả kiểm thử mạch LVD	213
4.19	Kết quả đo thời gian kết nối BLE OBD2	214
4.20	Thời gian phản hồi OBD2 PID	215
4.21	Thời gian bắt vệ tinh GPS (TTFF)	216
4.22	Độ chính xác vị trí GPS	217
4.23	Độ trễ truyền dữ liệu MQTT qua mạng 4G	217
4.24	Kết quả kiểm thử máy trạng thái firmware	218
4.25	Kết quả kiểm thử kết nối lại MQTT	219
4.25A	Đối chiếu luồng OTA theo mã nguồn triển khai	220
4.26	Kết quả đo hiệu suất API phía máy chủ	221
4.27	Độ trễ toàn tuyến (thiết bị -> máy chủ -> trình duyệt)	222
4.28	Tốc độ ghi dữ liệu của cơ sở dữ liệu	223
4.29	Kết quả kiểm thử tải đồng thời	223
4.30	Kết quả đánh giá hiệu suất giao diện web bằng Lighthouse	225
4.31	Kết quả kiểm thử luồng dữ liệu chính	226
4.32	Kết quả kiểm thử cảnh báo vượt vùng cho phép	229
4.33	Kết quả kiểm thử lệnh điều khiển từ xa	229
4.34	Kết quả kiểm thử mất kết nối và phục hồi	230
4.35	Tổng hợp kết quả đo lường phần cứng so với chỉ tiêu thiết kế	234

4.36	Tổng hợp kết quả đo lường firmware so với chỉ tiêu thiết kế	235
4.37	Tổng hợp kết quả đo lường máy chủ và dịch vụ xử lý dữ liệu so với chỉ tiêu thiết kế	236
4.38	Tổng hợp kết quả đo lường giao diện web so với chỉ tiêu thiết kế	236
4.39	Bảng tổng hợp tổng thể — So sánh kết quả với chỉ tiêu thiết kế	237
4.40	Chỉ số ổn định vận hành thực tế	238
5.1	Đánh giá hiệu năng quản lý năng lượng	242
5.2	Đánh giá hiệu năng firmware	244
5.3	Đánh giá hiệu năng tổng hợp phần máy chủ và giao diện web	246
5.4	Chi phí BOM thiết bị theo dõi	247
5.5	So sánh chi phí với giải pháp thương mại	247
5.6	Chi phí hạ tầng đám mây theo quy mô	248
5.7	Thang đo xác suất và tác động	249
5.8	Ma trận đánh giá rủi ro và biện pháp giảm thiểu	250
5.9	Tổng hợp mức độ rủi ro theo phân loại	253
5.10	Lộ trình phát triển khuyến nghị	257
6.3.1A	Tổng hợp thông tin về STT, Biện pháp, Mô tả chi tiết	266
6.4.2A	Tổng hợp thông tin về Tầng, Ngôn ngữ / Công nghệ, Thách thức chính	269
PL-1.1	Bảng kê chi phí linh kiện (BOM)	280
PL-1.2	Chi phí hạ tầng cloud	282
PL-1.3	Tổng hợp chi phí dự án	282
2.1A	Tổng hợp thông tin về Chuẩn cụ thể, Nội dung, Áp dụng	283
2.1B	Tổng hợp thông tin về PID (hex), Mô tả, Đơn vị	283
2.2A	Tổng hợp thông tin về Đặc tính, Mô tả, Áp dụng thực tế	284
2.3A	Tổng hợp thông tin về Nguyên tắc, Hiện thực trong dự án	285
2.4A	Tổng hợp thông tin về Nguyên tắc, Mô tả, Ví dụ trong dự án	285
PL-3.1	Phân chia giai đoạn và thời gian thực hiện	287
3.3A	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	289
3.3B	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	289
3.3C	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	289
3.3D	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	289
3.3E	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	290
3.3F	Tổng hợp thông tin về Tuần, Công việc, Sản phẩm	290
4.2A	Tổng hợp thông tin về STT, Tên bảng, Mô tả	291
4.2B	Tổng hợp thông tin về Metric name mẫu, Labels, Mô tả	292
4.3A	Tổng hợp thông tin về Nhóm, Method, Endpoint	292
4.4A	Tổng hợp thông tin về Biến, Dịch vụ, Mô tả	294
4.5A	Tổng hợp thông tin về Yêu cầu, Phiên bản tối thiểu	295

4.5B	Tổng hợp thông tin về Dịch vụ, URL, Tài khoản mặc định	296
------	--	-----

DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS

1.1	Sơ đồ tổng quan vấn đề và giải pháp đề xuất	3
1.2	Quy trình phát triển dự án theo các giai đoạn	10
1.3	Kiến trúc tổng thể hệ thống IoT Vehicle Tracking	11
1.4	Sơ đồ chuyển đổi giữa các chế độ năng lượng	13
1.5	Lộ trình phát triển dự án theo các giai đoạn	17
2.1	Sơ đồ kiến trúc dữ liệu của hệ thống	27
3.1	Sơ đồ khối tổng thể hệ thống thiết bị theo dõi	35
3.1a	Phân rã chi tiết các khối chức năng của thiết bị theo dõi	36
3.2	Sơ đồ kết nối BLE giữa ESP32-S3 và vgate iCar Pro	46
3.3	Sơ đồ kết nối LIS3DSH với ESP32-S3 qua I2C	48
3.3a	Sơ đồ chi tiết chân kết nối LIS3DSH với ESP32-S3	49
3.4	Sơ đồ khối hệ thống quản lý nguồn	50
3.4a	Kiến trúc nguồn và phân phối điện áp trong hệ thống	51
3.5	Sơ đồ kiến trúc phân lớp của firmware	60
3.6	Sơ đồ tương tác giữa các FreeRTOS task	62
3.7	Lưu đồ thuật toán luồng hoạt động chính của firmware	63
3.8	Lưu đồ thuật toán quy trình kết nối và đọc dữ liệu BLE OBD2	66
3.9	Lưu đồ điều phối modem SIM7600CE-T theo trạng thái mạng và chế độ vận hành	71
3.10	Lưu đồ thuật toán điều khiển chuyển nguồn	72
3.11	Sơ đồ máy trạng thái của thiết bị theo dõi	78
3.12	Sơ đồ kiến trúc tổng quan hệ thống máy chủ	85
3.12a	Luồng dữ liệu chi tiết từ thiết bị đến giao diện quản trị	85
3.13	Sơ đồ chiến lược lưu trữ kép	91
3.14	Sơ đồ quan hệ cơ sở dữ liệu (ER)	93
3.14a	Luồng điều phối OTA giữa API phía máy chủ, EMQX, thiết bị và MQTT Bridge	97
3.15	Sơ đồ kiến trúc chia theo từng nhóm tính năng của ứng dụng giao diện web	105
3.15a	Luồng dữ liệu một chiều giữa Next.js, Query, Zustand và API phía máy chủ	107
3.16	Biểu đồ trình tự luồng xác thực người dùng	108
3.17	Giao diện trang tổng quan hệ thống	109
3.18	Giao diện trang quản lý phương tiện	111
3.19	Kiến trúc tích hợp bản đồ thời gian thực	113
3.19a	Phân rã các lớp của bản đồ React Leaflet và kênh Socket.IO	114

3.20	Giao diện trang bản đồ thời gian thực với các marker xe	116
3.21	Wireframe bố cục giao diện tổng quan	118
3.22	Các mẫu thiết kế giao diện của hệ thống - bảng dữ liệu, biểu mẫu, biểu đồ	118
3.23	Sơ đồ kiến trúc tổng thể phương án tối ưu	121
4.1	Sơ đồ khối tổng thể hệ thống thiết bị theo dõi IoT	124
4.1a	Bố trí chi tiết các khối phần cứng trong thiết bị theo dõi	125
4.2	Sơ đồ kết nối giữa ESP32-S3 và modem SIM7600CE-T qua UART	126
4.2a	Sơ đồ chân kết nối SIM7600CE-T với ESP32-S3	127
4.3	Sơ đồ mạch đo điện áp ắc quy bằng voltage divider và ADC ESP32-S3	129
4.4	Sơ đồ nguyên lý mạch điện tổng hợp của hệ thống thiết bị theo dõi	130
4.5	Sơ đồ đấu nối tổng thể giữa ESP32-S3 và các khối phần cứng tích hợp	131
4.6	Kiến trúc tổng thể mạch quản lý nguồn	132
4.6a	Kiến trúc chuyển nguồn giữa nguồn xe và pin dự phòng	133
4.7	Kiến trúc nhánh buck 5V chính dùng MP2482	134
4.8	Kiến trúc nhánh nguồn dự phòng 5V từ pin 18650 qua SX1308 . .	135
4.9	Sơ đồ chuyển nguồn giữa nhánh chính và nhánh dự phòng	137
4.10	Kiến trúc sạc pin TP5100 và đường cấp nguồn dự phòng	139
4.11	Sơ đồ bố cục bên trong vỏ hộp bảo vệ	140
4.12	Quy trình lắp ráp phần cứng trên PCB	145
4.13	Sơ đồ bố trí linh kiện sau khi lắp ráp	146
4.14	Minh họa vị trí cổng OBD2 trên xe và cách kết nối	147
4.15	Minh họa lắp đặt thiết bị theo dõi trong xe và đi dây	149
4.16	Checklist kiểm tra hệ thống sau khi lắp đặt trong xe	150
4.16a	Chuỗi bước OTA trên firmware từ nhận lệnh đến xác nhận sau reboot	164
4.17	Lưu đồ thuật toán chính của firmware thiết bị theo dõi xe	169
4.18	Kiến trúc tổng thể hệ thống máy chủ và luồng dữ liệu	171
4.19	Cấu trúc thư mục hệ thống theo mô hình triển khai	173
4.20	Luồng xử lý dữ liệu của MQTT Bridge	178
4.21	Cấu trúc thư mục API phía máy chủ theo kiến trúcDDD	185
4.22	Cấu trúc thư mục giao diện web theo kiến trúc tách theo từng nhóm tính năng	190
4.23	Giao diện trang tổng quan hệ thống	191
4.24	Giao diện trang quản lý xe	191
4.25	Giao diện bản đồ thời gian thực với vị trí các xe	192
4.26	Giao diện trang quản lý cảnh báo	193
4.27	Màn hình Grafana hiển thị tổng quan hiệu năng hệ thống	197
4.28	Giao diện quản trị EMQX hiển thị trạng thái kết nối thiết bị	198

4.29	Bố trí thiết bị đo lường trong phòng thí nghiệm	203
4.30	Thiết bị theo dõi được lắp đặt trên xe thử nghiệm	205
4.31	Sơ đồ môi trường thử nghiệm tổng thể	206
4.32	Đồ thị dòng tiêu thụ theo thời gian trong một chu kỳ hoạt động hoàn chỉnh (xe chạy -> xe đỗ -> cảnh báo -> xe đỗ)	208
4.33	Đồ thị điện áp pin dự phòng theo thời gian khi quy đổi theo bộ pin tham chiếu 18650 trong kịch bản theo dõi liên tục khi xe chạy . . .	211
4.34	Đồ thị dòng tiêu thụ theo nhiệt độ môi trường	213
4.35	Biểu đồ phân bố thời gian kết nối BLE OBD2 (20 lần đo)	215
4.36	Đồ thị thời gian bắt vệ tinh GPS trong các kịch bản (Cold/Warm/Hot Start)	216
4.37	Biểu đồ phân bố độ trễ MQTT trong điều kiện mạng 4G ổn định .	218
4.38	Biểu đồ thời gian phản hồi API (P50, P95, P99) cho các endpoint chính	222
4.39	Đồ thị phân bố độ trễ end-to-end (1000 bản tin mẫu)	222
4.40	Đồ thị hiệu suất hệ thống theo số lượng thiết bị đồng thời	224
4.41	Kết quả chấm điểm Lighthouse cho trang tổng quan	226
4.42	Mình họa giao diện Dashboard hiển thị vị trí xe đang di chuyển trên bản đồ Leaflet	227
4.43	Mình họa khung dữ liệu OBD2 thời gian thực trên giao diện (RPM, tốc độ, nhiệt độ nước làm mát)	228
4.44	Mình họa màn hình cảnh báo vượt vùng cho phép với bản đồ và vùng theo dõi	231
4.45	Mình họa luồng gửi lệnh điều khiển từ Dashboard và phản hồi từ thiết bị	232
4.46	Mình họa quá trình bản đồ cập nhật lại hành trình sau khi phục hồi kết nối	233
4.47	Biểu đồ so sánh chỉ tiêu thiết kế và kết quả đạt được	238
PL-3.1	Kế hoạch thực hiện dự án theo giai đoạn (24 tuần)	288

DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION

Từ viết tắt	Tiếng Anh	Tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
BOM	Bill of Materials	Danh sách linh kiện
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên nguồn chéo
DDD	Domain-Driven Design	Thiết kế hướng miền
Docker	Docker Container Platform	Nền tảng container hóa ứng dụng
EMQX	Erlang MQTT Broker	Máy chủ MQTT dựa trên Erlang
ESP32	Espressif System-on-Chip 32-bit	Vi điều khiển SoC 32-bit cá»§a Espressif
FreeRTOS	Free Real-Time Operating System	Hệ điều hành thời gian thực mã nguồn mở
GPIO	General Purpose Input/Output	Cổng vào/ra đa năng
GNSS	Global Navigation Satellite System	Hệ thống vệ tinh dẫn đường toàn cầu
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
IMU	Inertial Measurement Unit	Đơn vị đo lường quán tính
IoT	Internet of Things	Internet vạn vật
JWT	JSON Web Token	Mã thông báo web dạng JSON
LVD	Low Voltage Disconnect	Ngắt điện áp thấp
MCU	Microcontroller Unit	Vi điều khiển
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông tin hàng đợi

Từ viết tắt	Tiếng Anh	Tiếng Việt
OBD2	On-Board Diagnostics II	Chẩn đoán trên xe thể hệ 2
ORM	Object-Relational Mapping	Ánh xạ đối tượng - quan hệ
PCB	Printed Circuit Board	Bảng mạch in
REST	Representational State Transfer	Kiến trúc truyền trạng thái đại diện
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
SoC	System on Chip	Hệ thống trên chip
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TLS	Transport Layer Security	Bảo mật tầng vận chuyển
UART	Universal Asynchronous Receiver-Transmitter	Bộ thu phát bất đồng bộ đa năng
WebSocket	WebSocket Protocol	Giao thức kết nối hai chiều thời gian thực

CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW

1.1. Đặt vấn đề / Bối cảnh của dự án

1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam

Những năm gần đây, thị trường cho thuê xe tự lái tại Việt Nam tăng trưởng rõ rệt, đặc biệt tại TP. Hồ Chí Minh, Hà Nội và Đà Nẵng. Theo báo cáo ngành vận tải [1], nhu cầu thuê xe tăng trung bình 15–20% mỗi năm nhờ sự phục hồi của du lịch nội địa, nhu cầu di chuyển linh hoạt và xu hướng chia sẻ phương tiện.

Với doanh nghiệp vận hành đội xe, tốc độ tăng trưởng này mang lại cơ hội kinh doanh nhưng cũng kéo theo một yêu cầu rất thực tế: phải biết xe đang ở đâu, đang được sử dụng như thế nào và có an toàn hay không, thay vì chỉ dựa vào các cuộc gọi xác nhận. Khi quy mô đội xe mở rộng, áp lực giám sát vận hành và bảo toàn tài sản tăng lên nhanh chóng. Trong thực tế, các doanh nghiệp cho thuê xe tự lái hiện nay thường gặp bốn nhóm khó khăn sau:

- **Quản lý thủ công kém hiệu quả:** Nhiều doanh nghiệp quy mô vừa và nhỏ vẫn quản lý xe bằng sổ sách, gọi điện xác nhận hoặc dựa vào sự chủ động của khách hàng. Cách làm này không cung cấp thông tin thời gian thực về vị trí và trạng thái phương tiện [2].
- **Rủi ro mất cắp và sử dụng sai mục đích:** Khi không có hệ thống giám sát, xe có thể bị sử dụng vượt phạm vi địa lý đã thỏa thuận, chạy quá số km quy định, hoặc trong trường hợp xấu nhất là bị chiếm đoạt. Việc phát hiện các tình huống này thường bị trễ, gây thiệt hại lớn về tài sản [3].
- **Thiếu dữ liệu chẩn đoán kỹ thuật:** Doanh nghiệp khó theo dõi tình trạng kỹ thuật của xe từ xa, nên việc bảo trì thường mang tính bị động và chỉ được thực hiện khi sự cố đã xuất hiện. Điều này làm tăng chi phí sửa chữa và rút ngắn tuổi thọ phương tiện.
- **Giải pháp thương mại đắt đỏ:** Các hệ thống định vị và giám sát xe thương mại hiện có trên thị trường (như Vietmap, iTracking) thường có chi phí cao — bao gồm phí thiết bị, phí dịch vụ hàng tháng, và phí tích hợp — không phù hợp với các doanh nghiệp quy mô nhỏ với ngân sách hạn chế [4].

Những khó khăn trên cho thấy nhu cầu giám sát là rất rõ, nhưng chưa tự động dẫn tới một lời giải khả thi. Muốn biến nhu cầu vận hành thành một hệ thống thật sự dùng được, bài toán phải được chuyển thành các thách thức kỹ thuật cụ thể.

1.1.2. Thách thức kỹ thuật

Để giải quyết những khó khăn trên, hệ thống không thể chỉ là một thiết bị định vị đơn giản. Bài toán thực tế là phải dung hòa nhiều yêu cầu kỹ thuật vốn dễ xung đột với nhau: thiết bị phải tiết kiệm điện nhưng vẫn luôn sẵn sàng, phải truyền dữ liệu liên tục nhưng vẫn chịu được mất sóng, và phải đủ đơn giản để triển khai thực tế với chi phí hợp lý. Bốn thách thức nổi bật là:

Thứ nhất, bài toán tiêu thụ năng lượng. Nếu thiết bị theo dõi hoạt động liên tục ở trạng thái công suất cao, ắc quy xe có thể bị hao đáng kể chỉ sau vài tuần, từ đó ảnh hưởng trực tiếp đến khả năng khởi động [5]. Vì vậy, hệ thống phải biết khi nào cần hoạt động đầy đủ và khi nào có thể chuyển sang chế độ tiết kiệm điện.

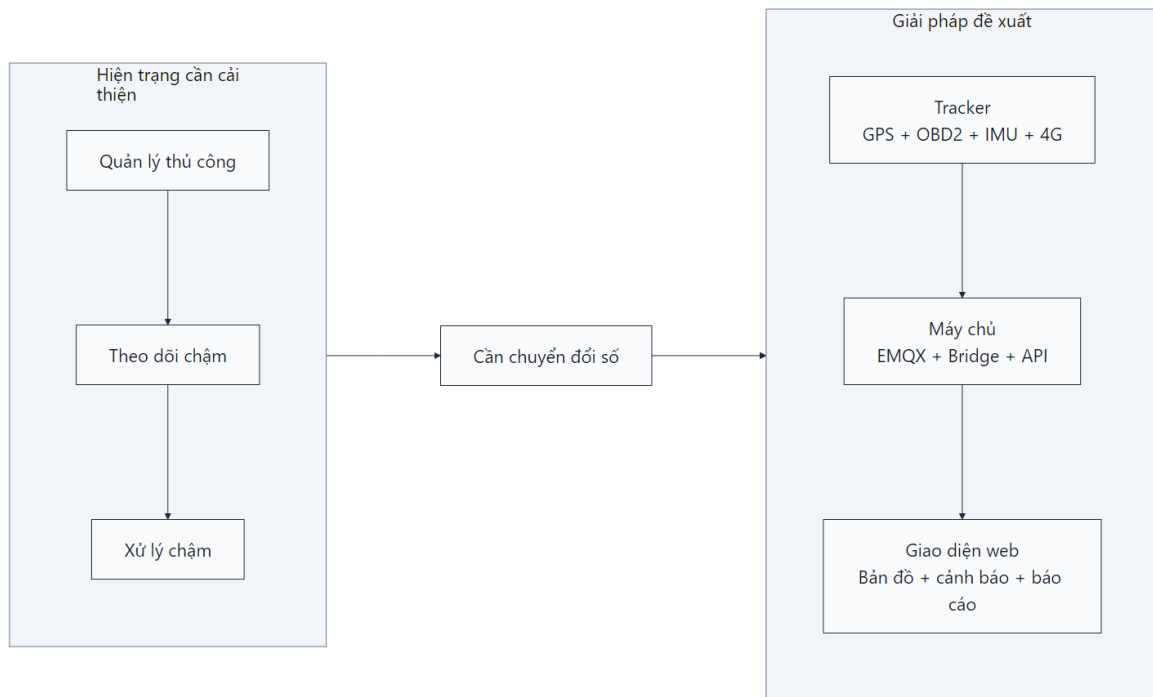
Thứ hai, giám sát khi xe đỗ. Khi xe tắt máy (IGN OFF), thiết bị không thể duy trì mọi khối chức năng như lúc xe đang chạy. Tuy nhiên, nó vẫn phải phát hiện được các chuyển động bất thường, chẳng hạn xe bị dắt đi hoặc bị cẩu kéo. Đây là bài toán cân bằng giữa độ nhạy phát hiện và mức tiêu thụ năng lượng [6].

Thứ ba, độ tin cậy của kết nối. Hệ thống hoạt động trong môi trường di động nên kết nối 4G/LTE có thể gián đoạn bất kỳ lúc nào. Điều người quản lý cần không chỉ là "có mạng thì gửi được", mà là khi mất mạng thiết bị vẫn không làm mất dữ liệu quan trọng và có thể đồng bộ lại khi kết nối được phục hồi.

Thứ tư, xử lý dữ liệu thời gian thực. Khi số lượng xe tăng lên hàng chục hoặc hàng trăm, dịch vụ phía máy chủ phải tiếp nhận liên tục dữ liệu vị trí, thông số OBD2 và trạng thái cảm biến với độ trễ thấp, đồng thời vẫn giữ giao diện giám sát dễ quan sát và dễ thao tác cho người quản lý.

1.1.3. Động lực thực hiện dự án

Mục tiêu của đề tài không chỉ là làm ra một thiết bị biết gửi tọa độ GPS. Điều cần xây dựng là một hệ thống đủ trọn vẹn để người vận hành đội xe có thể nhìn thấy trạng thái phương tiện, hiểu nguyên nhân của các cảnh báo và đưa ra quyết định kịp thời. Từ nhu cầu thực tiễn đó, dự án "IoT Vehicle Tracking System", một hệ thống IoT theo dõi phương tiện, được triển khai nhằm xây dựng một giải pháp đồng bộ, bao gồm thiết bị phần cứng trên xe, phần mềm điều khiển thiết bị, hạ tầng máy chủ tiếp nhận và xử lý dữ liệu, cùng giao diện web để khai thác thông tin. Mục tiêu là tạo ra một hệ thống có chi phí hợp lý, dễ tùy biến và có thể mở rộng theo quy mô doanh nghiệp cho thuê xe tự lái tại Việt Nam.



Hình 1.1: Sơ đồ tổng quan vấn đề và giải pháp đề xuất

1.2. Mục tiêu và phạm vi của dự án

1.2.1. Mục tiêu tổng quát

Mục tiêu chính của dự án là thiết kế và hiện thực một hệ thống IoT theo dõi phương tiện hoàn chỉnh, bao gồm cả phần cứng lẫn phần mềm, phục vụ cho bài toán quản lý đội xe cho thuê tự lái. Các mục tiêu dưới đây được xác định từ góc nhìn của đơn vị vận hành đội xe: họ cần thông tin đủ nhanh để can thiệp, nhưng cũng cần hệ thống đủ bền bỉ để chạy lâu dài trên xe. Hệ thống cần đáp ứng các yêu cầu sau:

1. **Theo dõi vị trí thời gian thực:** Cung cấp vị trí định vị vệ tinh GPS/GNSS của xe theo chu kỳ 5–30 giây khi xe đang di chuyển, để người quản lý nhìn thấy hành trình gần như liên tục trên bản đồ thay vì chỉ nhận các mốc vị trí rời rạc.
2. **Giám sát trạng thái kỹ thuật qua OBD2:** Kết nối với cổng chẩn đoán OBD-II của xe, nơi cung cấp các thông tin vận hành và lỗi cơ bản, thông qua BLE để đọc các thông số như tốc độ, vòng tua máy, nhiệt độ động cơ và mã lỗi chẩn đoán (DTC).
3. **Cảnh báo thông minh:** Tự động phát hiện và gửi thông báo khi có các sự kiện bất thường như xe vượt khỏi vùng giám sát đã vẽ trên bản đồ, vượt tốc độ quy định, chuyển động bất thường khi xe đang đỗ, hoặc mất kết nối thiết bị.
4. **Tối ưu hóa năng lượng:** Đảm bảo thiết bị hoạt động liên tục mà không làm cạn ắc quy xe, với thời lượng pin dự phòng đủ để hoạt động độc lập trong trường hợp mất nguồn chính.
5. **Giao diện quản lý trực quan:** Cung cấp giao diện web dạng bảng điều khiển tổng

hợp để người quản lý đội xe theo dõi vị trí, xem lịch sử hành trình, quản lý cảnh báo và tạo báo cáo.

Năm mục tiêu trên cho biết hệ thống cần đạt được điều gì. Phần tiếp theo sẽ chốt rõ phạm vi nào được hiện thực trong đồ án và phạm vi nào được giữ lại cho các giai đoạn phát triển sau.

1.2.2. Phạm vi dự án

Để đề tài không dàn trải, phạm vi nghiên cứu được giới hạn theo từng tầng của hệ thống, từ đối tượng ứng dụng đến phần cứng, firmware và hạ tầng máy chủ. Cách xác định phạm vi này giúp người đọc thấy rõ đâu là phần đã được triển khai trong đồ án và đâu là nội dung được giữ lại cho các giai đoạn phát triển sau.

Đối tượng ứng dụng:

- Loại xe: Ô tô con (xe 4 chỗ, sedan, SUV) với hệ thống điện 12V hoặc 24V DC
- Ắc quy phổ thông: 40–70 Ah (trung bình 45–60 Ah)
- Đối tượng sử dụng: Các công ty cho thuê xe tự lái quy mô vừa và nhỏ tại Việt Nam

Phạm vi phần cứng:

- Thiết bị theo dõi IoT sử dụng vi điều khiển ESP32-S3
- Modem LTE + GNSS SIMCom SIM7600CE-T (Auto mode LTE/UMTS/GSM, APN mặc định internet) kết nối trực tiếp qua UART1 để cung cấp cả dữ liệu 4G và GNSS
- Adapter OBD2 BLE vgate iCar Pro (đọc dữ liệu chẩn đoán xe qua Bluetooth)
- Cảm biến gia tốc LIS3DH (IMU) để phát hiện chuyển động và rung
- Pin dự phòng 18650 1S với mạch sạc và bảo vệ

Phạm vi firmware:

- Xây dựng phần mềm trên thiết bị bằng ESP-IDF kết hợp FreeRTOS để điều phối các tác vụ nền theo thời gian thực
- Giao tiếp BLE với adapter OBD2 để đọc dữ liệu chẩn đoán của xe
- Truyền dữ liệu lên máy chủ bằng MQTT 3.1.1 qua EMQX, phân tầng QoS 0/1 theo loại bản tin
- Quản lý năng lượng theo nhiều trạng thái vận hành, gồm lúc xe đang chạy, lúc xe đỗ và lúc cần phản ứng khẩn

Phạm vi máy chủ và dịch vụ xử lý dữ liệu:

- EMQX làm broker MQTT, nhận và chuyển tiếp dữ liệu từ thiết bị
- MQTT Bridge (dịch vụ trung gian độc lập) xử lý và phân phối dữ liệu tới đúng nơi lưu trữ
- PostgreSQL lưu trữ dữ liệu quan hệ (người dùng, xe, khách hàng, cảnh báo)

- VictoriaMetrics lưu trữ dữ liệu thay đổi liên tục theo thời gian
- VictoriaLogs lưu trữ nhật ký sự kiện
- API phía máy chủ hiện thực bằng Express.js và được tổ chức theo từng nhóm nghiệp vụ để dễ mở rộng
- Giao diện web hiện thực bằng Next.js 15, kết hợp bản đồ và biểu đồ để người dùng theo dõi vận hành

Giới hạn phạm vi nghiên cứu:

- Không xử lý các chức năng chẩn đoán OBD-II chuyên sâu (chỉ đọc các thông số cơ bản)
- Không phát hiện va chạm (IMU chỉ dùng cho phát hiện chuyển động/rung)
- Không điều khiển động cơ xe (tắt bơm xăng, khóa xe) — chỉ gửi cảnh báo
- Không bao gồm ứng dụng di động ở giai đoạn đầu (dự kiến làm ở giai đoạn 2 bằng Flutter và tận dụng lại giao diện web hiện có)

Bảng 1.1: Tóm tắt phạm vi dự án theo từng tầng hệ thống

Tầng hệ thống	Công nghệ chính	Phạm vi
Phần cứng	ESP32-S3, SIMCom SIM7600CE-T (LTE + GNSS), vgate iCar Pro, LIS3DH	Thiết kế PCB, chế tạo và tích hợp thiết bị hoàn chỉnh
Firmware	ESP-IDF, FreeRTOS, MQTT 3.1.1, OTA qua HTTPS	Phần mềm trên thiết bị: đọc dữ liệu, quản lý năng lượng và gửi các gói dữ liệu nhỏ lên máy chủ
MQTT Broker	EMQX 5.x	Broker MQTT nhận và chuyển tiếp dữ liệu từ thiết bị
Dịch vụ máy chủ	Express.js, TypeScript, PostgreSQL, VictoriaMetrics	API phía máy chủ và xử lý dữ liệu
Giao diện web	Next.js 15, React 19, Leaflet, ECharts	Bản đồ, biểu đồ và cảnh báo cho người vận hành
Hạ tầng triển khai	Docker, Nginx Proxy Manager, Grafana	Đóng gói dịch vụ, công bố truy cập và giám sát vận hành

1.3. Các tiêu chí cần đạt được của dự án

Dự án đặt ra các tiêu chí cụ thể cho từng tầng của hệ thống. Các tiêu chí này không chỉ dùng để đánh giá kết quả cuối cùng, mà còn đóng vai trò như một cam kết thiết kế xuyên suốt quá trình thực hiện: mỗi quyết định chọn linh kiện, tổ chức firmware hay xây dựng hạ tầng đều phải quay lại trả lời câu hỏi liệu nó có giúp hệ thống vận hành đủ ổn định, đủ tiết kiệm năng lượng và đủ hữu ích cho bài toán quản lý xe hay không.

1.3.1. Tiêu chí phần cứng

Ở tầng phần cứng, điều quan trọng nhất không phải là "gắn được nhiều khối chức năng", mà là thiết bị có thể tồn tại bền bỉ trong môi trường trên xe: đủ tiết kiệm điện, đủ ổn định về nhiệt và đủ an toàn với ắc quy.

Bảng 1.3.1A: Tổng hợp thông tin về STT, Tiêu chí, Mục tiêu cụ thể

STT	Tiêu chí	Mục tiêu cụ thể
1	Tiêu thụ điện ở chế độ ngủ sâu (deep sleep, tức gần như tắt toàn bộ khối không cần thiết)	$< 500 \mu A$
2	Tiêu thụ điện chế độ hoạt động	$< 250 \text{ mA}$ (trung bình)
3	Thời gian thức dậy từ chế độ ngủ sâu	< 3 giây
4	Thời lượng pin dự phòng (18650 1S)	≥ 2.5 giờ theo dõi liên tục; ≥ 24 giờ cảnh báo gián đoạn (mục tiêu lấy theo mức pin tham chiếu của cell 18650 đã chọn)
5	Ngưỡng chuyển nguồn bảo vệ ắc quy	Cấu hình cho hệ 12V: $OFF = 12.0 \text{ V}$, $ON = 12.2 \text{ V}$; Profile 24V: $OFF = 24.0 \text{ V}$, $ON = 24.4 \text{ V}$
6	Nhiệt độ hoạt động	-10 C đến $+60 \text{ C}$

1.3.2. Tiêu chí firmware

Ở tầng firmware, tiêu chí không chỉ dừng ở việc đọc được cảm biến hay gửi được dữ liệu, mà nằm ở khả năng điều phối đúng nhịp giữa các khối để thiết bị phản ứng phù hợp với từng trạng thái vận hành của xe.

Bảng 1.2A: Tiêu chí firmware

STT	Tiêu chí	Mục tiêu cụ thể
1	Tần suất gửi GPS khi lái xe	5–30 giây (cấu hình được)
2	Tần suất heartbeat khi đỗ xe	10–30 phút
3	Thời gian kết nối OBD2 BLE	< 10 giây
4	Độ chính xác vị trí GNSS	< 5 mét (điều kiện thoáng)
5	Xử lý mất kết nối	Tự động kết nối lại; có phương án bổ sung buffer cục bộ
6	Phát hiện chuyển động bất thường	Độ nhạy IMU cấu hình được

1.3.3. Tiêu chí máy chủ và dịch vụ xử lý dữ liệu

Khi chuyển sang tầng máy chủ và các dịch vụ xử lý dữ liệu, trọng tâm đánh giá đổi sang khả năng tiếp nhận dữ liệu liên tục, phản hồi nhanh và bảo vệ dữ liệu khi số lượng thiết bị tăng lên theo quy mô đội xe.

Bảng 1.2B: Tiêu chí máy chủ và dịch vụ xử lý dữ liệu

STT	Tiêu chí	Mục tiêu cụ thể
1	Độ trễ xử lý bản tin MQTT	< 500 ms từ thiết bị tới lớp xử lý chính
2	Số lượng thiết bị đồng thời	≥ 50 thiết bị (đã kiểm thử), dư địa tới khoảng 100 thiết bị
3	Độ sẵn sàng hệ thống (thời gian hoạt động ổn định)	$\geq 99.5\%$
4	Thời gian phản hồi API	< 200 ms (p95)
5	Lưu trữ dữ liệu vận hành gửi liên tục như vị trí, tốc độ, trạng thái nguồn	≥ 90 ngày
6	Bảo mật xác thực	Phiên đăng nhập lưu ở máy chủ; token ngẫu nhiên chỉ dùng làm mã tham chiếu và được băm SHA-256

1.3.4. Tiêu chí giao diện web

Cuối cùng, giao diện web là nơi toàn bộ nỗ lực kỹ thuật được chuyển thành trải nghiệm quan sát và ra quyết định cho người vận hành, nên tiêu chí ở đây tập trung vào tính cập nhật, dễ theo dõi và dễ sử dụng trong bối cảnh thực tế.

Bảng 1.2C: Tiêu chí giao diện web

STT	Tiêu chí	Mục tiêu cụ thể
1	Cập nhật bản đồ thời gian thực	< 2 giây độ trễ
2	Hỗ trợ trình duyệt	Chrome, Firefox, Edge (phiên bản mới nhất)
3	Bố cục thích ứng theo màn hình	Desktop và tablet
4	Hiển thị cảnh báo	Thông báo tức thời qua WebSocket, tức kênh để máy chủ đẩy dữ liệu mới lên giao diện

Các tiêu chí trên vì thế đóng vai trò như một thước đo chung cho toàn bộ đồ án: mọi lựa chọn về phần cứng, firmware, máy chủ hay giao diện ở các chương sau đều phải quay lại đối chiếu với bộ mục tiêu này. Từ nền đó, phần tiếp theo trình bày cách nhóm tiếp cận bài toán theo từng bước.

1.4. Phương pháp tiếp cận thiết kế kỹ thuật

1.4.1. Phương pháp luận tổng thể

Vì đề tài đi qua cả mạch nguồn, phần mềm nhúng trên thiết bị, hạ tầng máy chủ và màn hình quản trị tổng hợp, rủi ro lớn nhất không nằm ở việc từng khối có chạy riêng lẻ hay không, mà ở việc chúng có còn phối hợp đúng khi ghép thành một hệ thống hoàn chỉnh hay không.

Do đó, dự án chọn cách làm theo hai nguyên tắc. Thứ nhất là **thiết kế từ dưới lên (bottom-up design)**: kiểm chứng trước các giả định rủi ro nhất ở tầng thấp rồi mới ghép dần lên các tầng trên. Thứ hai là **phát triển lặp (iterative development)**: làm từng vòng nhỏ, đo được và sửa được, thay vì để đến cuối dự án mới phát hiện xung đột về nguồn, giao tiếp hay hiệu năng. Cụ thể, tiến trình được tổ chức như sau:

1. **Giai đoạn 1 — Nghiên cứu và thiết kế phần cứng**: Xây nền cho hệ thống bằng cách khảo sát linh kiện, đối chiếu datasheet, thiết kế sơ đồ nguyên lý và tính toán nguồn. Mục tiêu của giai đoạn này là bảo đảm thiết bị có thể tồn tại bền bỉ trong môi trường trên xe trước khi nói đến các tính năng ở tầng trên.
2. **Giai đoạn 2 — Phát triển firmware**: Biến sơ đồ phần cứng thành một thiết bị thực sự biết “làm gì khi nào”. Ở giai đoạn này, ESP32-S3 được lập trình bằng ESP-IDF để đọc dữ liệu xe qua BLE OBD2, giao tiếp với modem qua UART, nhận tín hiệu

chuyển động từ IMU qua I2C, quyết định khi nào cần thức hoặc ngủ, rồi gửi dữ liệu lên máy chủ bằng MQTT. Đây là giai đoạn quyết định thiết bị theo dõi có phản ứng đúng lúc và tiết kiệm điện đúng chỗ hay không.

3. **Giai đoạn 3 — Xây dựng hạ tầng máy chủ:** Dựng “nơi dữ liệu sẽ đi tới” sau khi rời chiếc xe. Giai đoạn này chuẩn bị các dịch vụ nhận bản tin, lưu dữ liệu quản trị, lưu dữ liệu thay đổi liên tục theo thời gian và lưu nhật ký vận hành, đồng thời chốt cách các dịch vụ kết nối với nhau.
4. **Giai đoạn 4 — Phát triển API phía máy chủ:** Biến dữ liệu kỹ thuật thành những chức năng quản trị có thể dùng được, như đăng nhập, quản lý xe, xem lịch sử, tạo cảnh báo và phát cập nhật thời gian thực ra giao diện quản trị.
5. **Giai đoạn 5 — Phát triển giao diện web:** Trình bày dữ liệu dưới dạng bản đồ, bảng thông tin và biểu đồ để người vận hành nhìn nhanh, hiểu nhanh và thao tác ít nhất có thể.
6. **Giai đoạn 6 — Tích hợp và kiểm thử:** Ghép toàn bộ các lớp lại thành một chuỗi hoàn chỉnh từ xe đến giao diện quản trị, sau đó kiểm thử, đo đạc và tối ưu những điểm còn ngẽn.

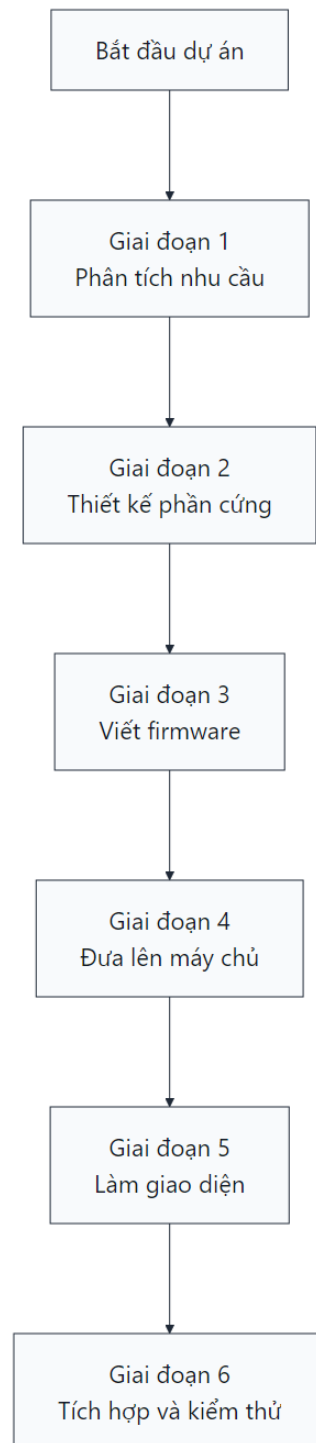
1.4.2. Kiến trúc hệ thống

Nếu nhìn theo đường đi của thông tin từ xe đến màn hình quản lý, hệ thống có thể hình dung qua ba lớp. Lớp thứ nhất nằm trên xe, có nhiệm vụ thu dữ liệu và gửi dữ liệu đi. Lớp thứ hai nằm trên hạ tầng hệ thống máy chủ, có nhiệm vụ tiếp nhận, phân luồng và lưu dữ liệu vào đúng nơi. Lớp thứ ba là lớp ứng dụng, nơi dữ liệu được biến thành bản đồ, bảng thông tin, cảnh báo và công cụ quản trị để con người sử dụng. Cách nhìn này giải thích vì sao kiến trúc được tổ chức theo dạng phân tầng thay vì để mọi chức năng dồn vào một chỗ.

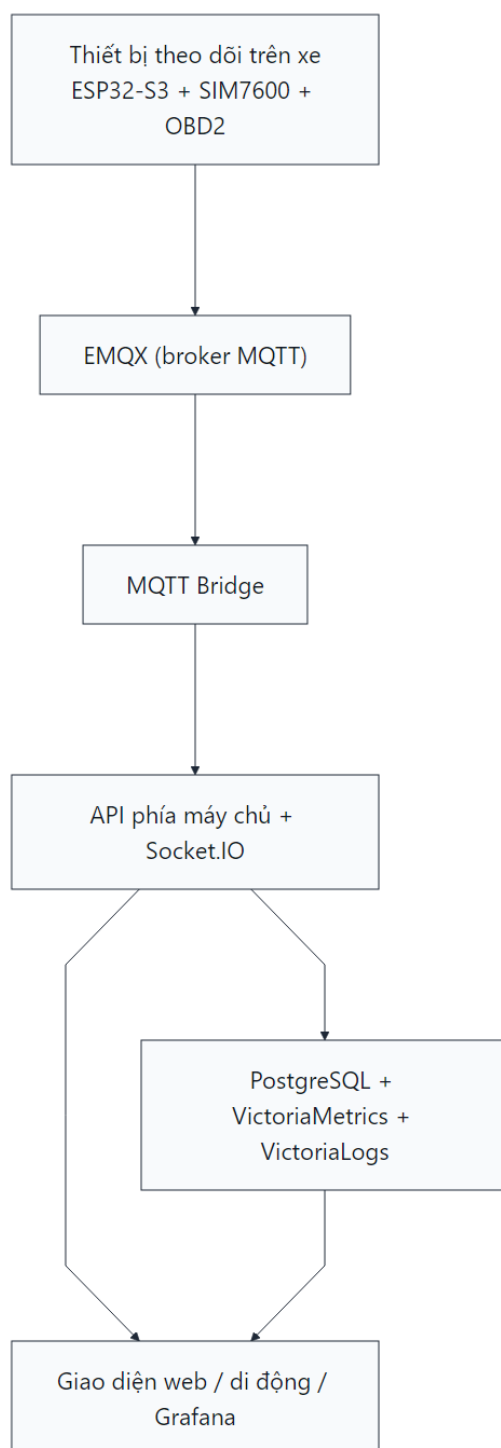
Trong triển khai thực tế, lớp trên xe gồm các thành phần chuyên đi lấy dữ liệu và giữ thiết bị hoạt động ổn định, tiêu biểu là ESP32-S3, modem SIM7600CE-T, cảm biến rung LIS3DH và adapter OBD2. Lớp giữa làm nhiệm vụ nhận bản tin, kiểm tra rồi chuyển dữ liệu sang nơi lưu phù hợp như VictoriaMetrics, VictoriaLogs và API phía máy chủ thông qua EMQX và MQTT Bridge. Lớp trên cùng là nơi người quản lý làm việc, gồm giao diện web quản trị và công cụ giám sát. Ứng dụng di động được giữ cho giai đoạn sau để đồ án tập trung trước vào chuỗi giá trị cốt lõi từ xe đến màn hình quản trị.

Chiến lược dữ liệu (Data Strategy):

Chiến lược dữ liệu được xây theo cùng logic phân tầng này vì dữ liệu trong hệ thống không đồng nhất: có loại cần tính nhất quán và quan hệ chặt chẽ, có loại cần ghi liên tục với tần suất cao, và có loại chủ yếu phục vụ truy vết sự cố. Mỗi nhóm dữ liệu vì thế được đưa về đúng nơi phù hợp với đặc tính truy vấn và mục đích sử dụng.



Hình 1.2: Quy trình phát triển dự án theo các giai đoạn



Hình 1.3: Kiến trúc tổng thể hệ thống IoT Vehicle Tracking

- **PostgreSQL:** Lưu trữ dữ liệu quan hệ có cấu trúc như người dùng, xe, khách hàng, hành trình, cảnh báo, vùng giám sát trên bản đồ và lệnh điều khiển. Đây là lớp dữ liệu cần quan hệ rõ ràng và ràng buộc chặt chẽ.
- **VictoriaMetrics:** Lưu trữ dữ liệu chuỗi thời gian như tọa độ GPS, dữ liệu OBD2 và chỉ số cảm biến. Nhóm dữ liệu này cần ghi nhiều, đọc nhanh theo trục thời gian và lưu dài hạn với chi phí hợp lý.
- **VictoriaLogs:** Lưu trữ nhật ký sự kiện và dấu vết ai làm gì và khi nào, cùng các bản ghi hoạt động của thiết bị để phục vụ quan sát hệ thống và truy vết sự cố.

1.4.3. Chiến lược quản lý năng lượng

Thiết kế năng lượng của thiết bị theo dõi thực chất là bài toán quyết định lúc nào thiết bị cần hoạt động đầy đủ, lúc nào chỉ nên “canh chừng”, và lúc nào phải ưu tiên bảo vệ nguồn xe. Nếu xử lý không khéo, hệ thống sẽ rơi vào hai cực đoan: theo dõi rất sát nhưng làm hao ắc quy, hoặc tiết kiệm điện quá mức nên bỏ lỡ sự kiện quan trọng. Vì vậy, chiến lược quản lý năng lượng đa chế độ (multi-mode power management) được xây quanh các trạng thái vận hành thật của xe, để mỗi thời điểm chỉ bật những khối thật sự cần thiết:

Chế độ 1 — Khi xe đang chạy (IGN ON, ưu tiên độ đầy đủ dữ liệu):

- Hệ thống hoạt động toàn bộ công suất
- Gửi vị trí GPS định kỳ (5–30 giây)
- Sạc pin dự phòng từ nguồn xe
- Kết nối máy chủ liên tục để theo dõi thời gian thực

Chế độ 2 — Parking Mode (IGN OFF, ưu tiên bảo toàn năng lượng):

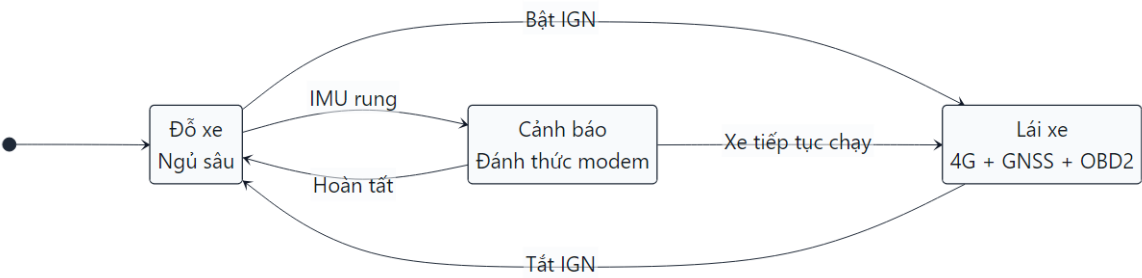
- Vi điều khiển ESP32-S3 vào chế độ ngủ sâu (deep sleep)
- Tiêu thụ cực thấp ($< 500 \mu A$)
- Thức dậy định kỳ (10–30 phút) để gửi heartbeat
- IMU (LIS3DH) hoạt động độc lập, cảnh rung ở ngưỡng đã cấu hình

Chế độ 3 — Cảnh báo khẩn (ưu tiên phản ứng nhanh khi có sự cố):

- IMU đánh thức ESP32 ngay lập tức qua interrupt
- Bật 4G + GPS, gửi cảnh báo ưu tiên lên máy chủ
- Tiếp tục theo dõi liên tục cho đến khi được xác nhận an toàn
- Sử dụng pin dự phòng nếu nguồn chính bị cắt

Bên cạnh đó, hệ thống còn có mạch chuyển nguồn dự phòng với cơ chế hysteresis để tránh đóng/ngắt liên tục khi điện áp dao động quanh ngưỡng. Thiết bị tự động chuyển sang pin dự phòng khi điện áp ắc quy tụt xuống dưới mức an toàn theo từng profile: 12V dùng $Switch_{OFF} = 12.0 V$, $Switch_{ON} = 12.2 V$; 24V dùng $Switch_{OFF} = 24.0 V$, $Switch_{ON} = 24.4 V$, qua đó bảo vệ ắc quy không bị rút cạn quá mức [7].

Nhờ cách tổ chức này, năng lượng không còn là bài toán tối ưu từng linh kiện riêng lẻ, mà trở thành một phần của logic vận hành toàn hệ thống.



Hình 1.4: Sơ đồ chuyển đổi giữa các chế độ năng lượng

1.4.4. Công nghệ sử dụng

Phần công nghệ dưới đây không nhằm chứng minh dự án dùng nhiều công cụ, mà để chỉ ra mỗi công cụ giải quyết đúng một việc trong chuỗi vận hành. Nếu bỏ cách nhìn “vai trò trước, tên công nghệ sau”, bảng này rất dễ trở thành danh sách thuật ngữ rời rạc.

Có thể đọc toàn bộ hệ thống theo ba cụm vai trò thay vì học thuộc từng tên công nghệ. Cụm thứ nhất là **khối trên xe**: vi điều khiển, modem, cảm biến và adapter OBD2 chịu trách nhiệm thu dữ liệu và giữ thiết bị hoạt động ổn định. Cụm thứ hai là **khối tiếp nhận và lưu trữ**: firmware điều phối phần cứng, còn hạ tầng máy chủ nhận bản tin, phân loại, lưu dữ liệu và lưu dấu vết vận hành. Cụm cuối là **khối khai thác**: giao diện web, bản đồ, biểu đồ và công cụ giám sát giúp người vận hành đọc hiểu dữ liệu nhanh. Với cách nhìn này, các tên như ESP32-S3, EMQX hay Next.js chỉ còn là ví dụ cụ thể cho từng vai trò, không phải một danh sách thuật ngữ rời rạc.

Bảng 1.3: Tổng hợp công nghệ sử dụng trong dự án

Thành phần	Công nghệ	Phiên bản	Vai trò
Vi điều khiển	ESP32-S3	—	Vi điều khiển chính, điều phối toàn bộ thiết bị
LTE + GNSS	SIMCom SIM7600CE-T (LTE + GNSS tích hợp)	—	Truyền dữ liệu 4G và lấy vị trí từ vệ tinh
OBD2 Adapter	vgate iCar Pro	BLE 4.0	Đọc dữ liệu chẩn đoán xe
Cảm biến gia tốc	LIS3DH	—	Phát hiện chuyển động và rung

Bảng 1.3: Tổng hợp công nghệ sử dụng trong dự án (tiếp theo)

Thành phần	Công nghệ	Phiên bản	Vai trò
Pin dự phòng	18650 1S Li-ion	3500 mAh	Nguồn điện dự phòng
Framework firmware	ESP-IDF	5.x	Bộ công cụ để phát triển phần mềm nhúng cho ESP32
RTOS	FreeRTOS	—	Lớp điều phối các tác vụ nền theo thời gian thực
MQTT Broker	EMQX	5.x	Trạm trung chuyển bản tin từ thiết bị
API phía máy chủ	Express.js + TypeScript	—	Lớp giao tiếp để giao diện gọi lấy dữ liệu và gửi lệnh
Cơ sở dữ liệu quan hệ	PostgreSQL	16	Lưu trữ dữ liệu có cấu trúc
Cơ sở dữ liệu chuỗi thời gian	VictoriaMetrics	—	Lưu trữ dữ liệu vận hành gửi liên tục như vị trí, trạng thái nguồn và cảm biến
Nhật ký sự kiện	VictoriaLogs	—	Lưu trữ nhật ký vận hành và lỗi
Giao diện web	Next.js 15 + React 19	—	Giao diện quản lý cho người vận hành
Bản đồ	Leaflet	—	Hiển thị bản đồ thời gian thực
Biểu đồ	ECharts	—	Trực quan hóa dữ liệu
WebSocket	Socket.IO	4.8	Kênh để máy chủ đẩy dữ liệu mới lên giao diện
Container hóa	Docker + Docker Compose	—	Triển khai dịch vụ
Giám sát	Grafana + VictoriaMetrics, VictoriaLogs	—	Màn hình giám sát hạ tầng và nhật ký vận hành

1.5. Kết quả và khuyến nghị

1.5.1. Kết quả đạt được

Nhìn theo cách đó, các kết quả đạt được không nên hiểu như một danh sách các khối đã lắp xong, mà như bằng chứng cho thấy chuỗi giá trị từ thiết bị đến giao diện quản trị đã được hình thành. Có thể tóm lược theo bốn tầng chính như sau:

Về phần cứng:

- Hoàn thiện thiết bị theo dõi IoT với bo mạch riêng do nhóm thiết kế, sử dụng ESP32-S3-WROOM-1 làm vi điều khiển trung tâm, tích hợp modem LTE + GNSS SIMCom SIM7600CE-T, cảm biến gia tốc LIS3DH, khối nguồn đa rail, pin dự phòng 18650 1S và kết nối OBD2 BLE qua adapter vgate iCar Pro.
- Quá trình lựa chọn linh kiện bám theo datasheet, độ ổn định chất lượng và khả năng cung ứng trong nước để thuận lợi cho việc chế tạo, kiểm thử và thay thế khi cần.
- Hệ thống quản lý năng lượng đa chế độ hoạt động hiệu quả, với mức tiêu thụ điện ngủ sâu đạt yêu cầu ($< 500 \mu A$), đảm bảo không làm cạn ắc quy xe trong quá trình sử dụng bình thường.
- Mạch Low Voltage Disconnect (LVD) bảo vệ ắc quy xe hiệu quả, tự động ngắt khi điện áp tụt dưới ngưỡng an toàn.

Về firmware:

- Phát triển thành công phần mềm trên thiết bị bằng ESP-IDF kết hợp FreeRTOS, giúp thiết bị theo dõi đọc dữ liệu xe qua BLE OBD2, điều khiển modem qua UART, theo dõi chuyển động bằng IMU, quản lý năng lượng và gửi dữ liệu MQTT lên hệ thống máy chủ.
- Cơ chế phát hiện chuyển động bất thường qua IMU hoạt động chính xác, có khả năng đánh thức hệ thống từ chế độ ngủ sâu trong vòng < 3 giây.

Về hạ tầng máy chủ và xử lý dữ liệu:

- Xây dựng thành công hạ tầng máy chủ hoàn chỉnh để nhận dữ liệu từ thiết bị, lưu dữ liệu nghiệp vụ, lưu dữ liệu theo thời gian và lưu nhật ký vận hành; toàn bộ được đóng gói bằng Docker để dễ triển khai và lặp lại môi trường.
- Hoàn thiện API quản trị phía máy chủ cho các nhu cầu cốt lõi như đăng nhập, quản lý xe, quản lý thiết bị, xem dữ liệu vận hành, xử lý cảnh báo và quản lý vùng giám sát.
- Hệ thống đã kiểm thử ở mức 50+ thiết bị đồng thời với độ trễ từ lúc dữ liệu rời thiết bị đến lúc tới nơi xử lý dưới 500 ms, đồng thời cho thấy còn dư địa mở rộng ổn định tới khoảng 100 thiết bị.

Về giao diện web:

- Giao diện web quản lý (Next.js 15) với bản đồ thời gian thực (Leaflet), biểu đồ phân

tích (ECharts), hệ thống cảnh báo trực tuyến và màn hình tổng quan.

- Cập nhật vị trí xe trên bản đồ với độ trễ < 2 giây thông qua WebSocket (Socket.IO), tức kênh để máy chủ chủ động đẩy dữ liệu mới lên trình duyệt.

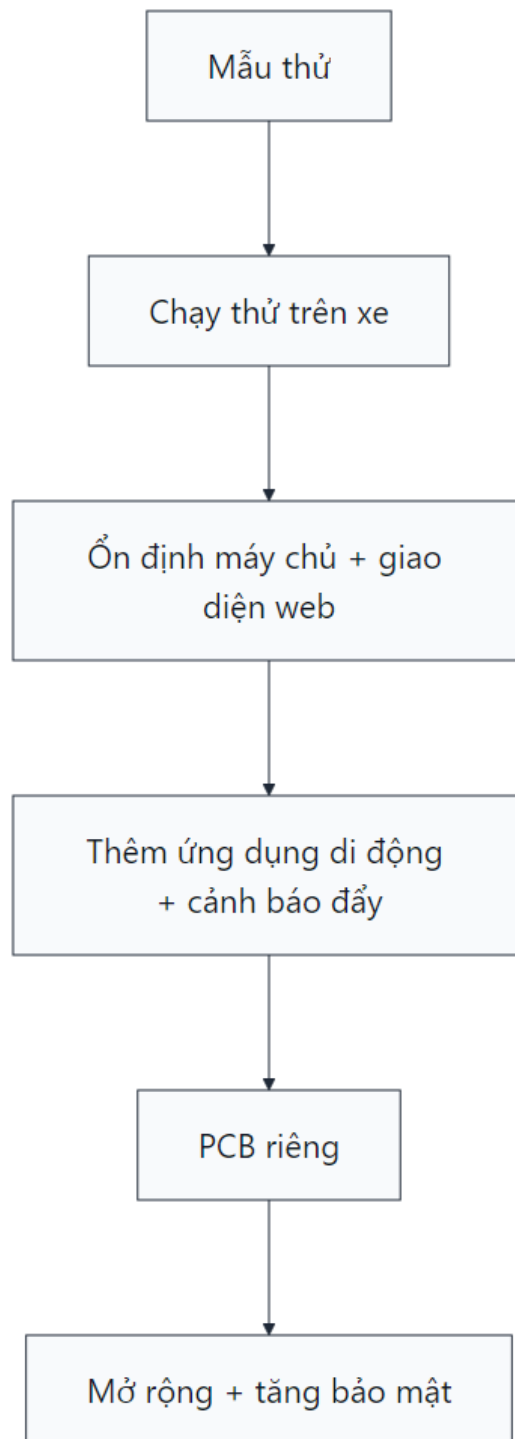
1.5.2. Khuyến nghị và hướng phát triển

Trên cơ sở các kết quả đã đạt được và các giới hạn hiện hữu, các hướng phát triển tiếp theo được sắp xếp theo logic mở rộng từ việc làm chắc hệ lõi đến việc mở rộng giá trị sử dụng của hệ thống:

1. **Ứng dụng di động (giai đoạn 2):** Phát triển ứng dụng Flutter theo hướng vỏ ứng dụng mỏng, chủ yếu nhúng lại giao diện web hiện có, để người quản lý giám sát đội xe trên điện thoại và nhận thông báo đẩy khi có cảnh báo [8].
2. **Tăng cường bảo mật:** Triển khai TLS/SSL cho kết nối MQTT trong môi trường vận hành thật, áp dụng danh sách quyền MQTT chi tiết hơn để mỗi thiết bị chỉ đi đúng kênh dữ liệu của mình, và bổ sung cơ chế mã hóa dữ liệu từ thiết bị tới máy chủ.
3. **Tối ưu hóa hiệu năng:** Nghiên cứu và áp dụng các kỹ thuật nén dữ liệu vận hành gửi liên tục, chẳng hạn chỉ gửi phần sai khác hoặc dùng định dạng gói gọn hơn, để giảm băng thông 4G, đặc biệt khi vận hành đội xe lớn (> 500 xe).
4. **Tích hợp trí tuệ nhân tạo:** Áp dụng các mô hình học máy để phân tích hành vi lái xe, dự đoán thời điểm cần bảo trì và phát hiện các dấu hiệu bất thường từ dữ liệu vận hành đã thu thập.
5. **Mở rộng phạm vi OBD2:** Hỗ trợ đọc thêm nhiều thông số chẩn đoán xe, bao gồm mã lỗi DTC (Diagnostic Trouble Codes), dữ liệu động cơ nâng cao, và tích hợp với các loại xe điện (EV).
6. **Cải thiện khả năng mở rộng:** Nghiên cứu kiến trúc tách nhiều dịch vụ nhỏ, có thêm hàng đợi bản tin chuyên dụng (chẳng hạn Kafka hoặc RabbitMQ), để xử lý luồng dữ liệu lớn hơn khi số lượng thiết bị tăng lên hàng ngàn.
7. **Tối ưu PCB cho sản xuất:** Nâng thiết kế PCB hiện tại theo hướng dễ sản xuất hơn và ít nhiễu điện từ hơn, giảm kích thước, tăng độ bền cơ khí và chuẩn bị cho sản xuất hàng loạt.

Kết luận chương 1

Chương 1 đã trả lời câu hỏi vì sao đề tài này cần được thực hiện và hệ thống cần đạt được điều gì trong bối cảnh thực tế của doanh nghiệp cho thuê xe tự lái. Từ bối cảnh thị trường, nhu cầu vận hành đến các thách thức về năng lượng, kết nối và dữ liệu, chương đã thiết lập bộ mục tiêu, phạm vi và tiêu chí đánh giá cho toàn bộ đề án. Nếu Chương 1 là phần đặt vấn đề, thì Chương 2 sẽ chuyển sang câu hỏi quan trọng hơn: cần giải quyết



Hình 1.5: Lộ trình phát triển dự án theo các giai đoạn

chính xác những bài toán kỹ thuật nào để biến mục tiêu đó thành một hệ thống khả thi.

CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT

Nếu Chương 1 đặt ra bối cảnh và mục tiêu của đề tài, thì Chương 2 đi sâu vào phần cốt lõi hơn: bóc tách bài toán kỹ thuật thật sự của hệ thống. Trọng tâm của chương không chỉ là liệt kê công nghệ, mà là làm rõ vì sao mỗi quyết định về phần cứng, truyền thông, lưu trữ và kiến trúc đều phải xuất phát từ nhu cầu vận hành thực tế của một hệ thống giám sát phương tiện.

2.1. Mô tả vấn đề – Problem statement

2.1.1. Bối cảnh thực tế

Trong bối cảnh ngành cho thuê ô tô và quản lý đội xe tại Việt Nam tiếp tục mở rộng, nhu cầu giám sát phương tiện theo thời gian thực đã trở thành yêu cầu thiết yếu đối với doanh nghiệp vận tải. Theo thống kê của Bộ Giao thông Vận tải, số lượng ô tô cá nhân và thương mại tăng trung bình 10–12% mỗi năm trong giai đoạn 2020–2025, kéo theo áp lực ngày càng lớn về quản lý, an ninh và tối ưu vận hành [1].

Tuy nhiên, điều doanh nghiệp cần không chỉ là các điểm vị trí rời rạc, mà là khả năng nhìn thấy trạng thái phương tiện đủ sớm để đưa ra quyết định vận hành. Khoảng cách giữa nhu cầu này và cách làm hiện tại thể hiện ở các hạn chế sau:

- **Giám sát thủ công:** Tài xế báo cáo vị trí qua điện thoại, không có dữ liệu liên tục, không thể xác minh chính xác hành trình. Phương pháp này phụ thuộc hoàn toàn vào yếu tố con người, dễ xảy ra sai sót và gian lận.
- **Thiết bị GPS đơn giản:** Chỉ cung cấp tọa độ vị trí, không đọc được dữ liệu động cơ (tốc độ, vòng tua, nhiên liệu), không hỗ trợ phát hiện bất thường khi xe đậu. Thiếu khả năng tích hợp sâu với hệ thống quản lý.
- **Giải pháp thương mại (fleet management):** Chi phí cao (50–200 USD/thiết bị + phí dịch vụ hàng tháng), phụ thuộc vào nhà cung cấp nước ngoài, khó tùy biến theo nhu cầu cụ thể của doanh nghiệp Việt Nam.

2.1.2. Xác định vấn đề kỹ thuật

Từ bối cảnh thực tiễn đó, bài toán của đề tài có thể quy về bốn câu hỏi kỹ thuật nền tảng. Đây cũng là bốn câu hỏi chi phối gần như mọi quyết định ở các chương sau, từ lựa chọn phần cứng đến thiết kế hạ tầng máy chủ»§:

Vấn đề 1: Quản lý năng lượng trong môi trường ô tô

Thiết bị theo dõi phải hoạt động liên tục 24/7 trong môi trường ô tô, nơi nguồn điện luôn bị ràng buộc bởi dung lượng ắc quy. Khi xe tắt máy (IGN OFF), thiết bị vẫn lấy điện từ ắc quy 12V hoặc 24V của xe; nếu duy trì chế độ theo dõi liên tục, ắc quy có thể cạn sau vài tuần và làm xe không khởi động được [2]. Vì vậy, câu hỏi không đơn thuần là ”thiết

bị có hoạt động được không”, mà là ”thiết bị có hoạt động đủ thông minh để không gây phiền toái cho chính chiếc xe mà nó đang giám sát hay không”.

Vấn đề 2: Giám sát khi xe đậu

Khi xe tắt máy, hệ thống cần tiết kiệm điện nhưng vẫn phát hiện được các chuyển động bất thường như bị trộm hoặc bị kéo cầu. Vì vậy, thiết bị phải có cơ chế đánh thức nhanh khi phát hiện sự kiện, đồng thời duy trì dòng tiêu thụ ở mức rất thấp. Khi xe ”im lặng”, thiết bị cũng phải ”ngủ”, nhưng vẫn phải tỉnh dậy đủ nhanh khi có sự cố.

Vấn đề 3: Truyền dữ liệu qua mạng di động

Hệ thống phải gửi đều dữ liệu vị trí và trạng thái xe qua mạng di động, ngay cả khi chất lượng sóng thay đổi liên tục. Vì vậy, bài toán không chỉ là “gửi được hay không”, mà là khi mất mạng thiết bị có tự nối lại được không, có giữ được dữ liệu quan trọng không, và có mở rộng được khi số xe tăng lên không. Vì thế, việc chọn giữa MQTT, HTTP hay CoAP chính là chọn “cách gửi tin” phù hợp nhất với một thiết bị phải truyền nhiều gói nhỏ, đều đặn và đôi khi cần nhận lệnh ngược từ máy chủ. MQTT phù hợp khi thiết bị cần gửi các gói dữ liệu nhỏ thường xuyên; HTTP gần với kiểu “gửi yêu cầu rồi chờ trả lời” quen thuộc của web; còn CoAP là giao thức gọn hơn, thường dùng cho thiết bị tài nguyên rất hạn chế.

Vấn đề 4: Kiến trúc máy chủ có khả năng mở rộng

Hệ thống cần hỗ trợ nhiều phương tiện đồng thời, cập nhật vị trí theo thời gian thực qua WebSocket, bảo đảm an toàn bằng cơ chế xác thực phiên, và xử lý được luồng dữ liệu lớn đến từ nhiều thiết bị IoT. Với người sử dụng cuối, điều này được cảm nhận rất đơn giản: bản đồ phải cập nhật kịp thời, cảnh báo phải đến đúng lúc, và dữ liệu phải nhất quán ngay cả khi số lượng xe tăng lên.

2.1.3. Mục tiêu kỹ thuật

Từ bốn vấn đề nền tảng trên, đồ án rút ra bốn nhóm mục tiêu kỹ thuật. Mỗi nhóm vừa trả lời một nhu cầu vận hành cụ thể của doanh nghiệp cho thuê xe, vừa trở thành tiêu chí để sàng lọc và lựa chọn giải pháp ở Chương 3:

1. Thiết kế thiết bị theo dõi có thể vừa xác định vị trí bằng GNSS, vừa đọc dữ liệu chẩn đoán từ xe qua OBD2, dựa trên vi điều khiển ESP32-S3; đồng thời tối ưu tiêu thụ năng lượng khi xe đậu (mục tiêu dòng deep sleep toàn thiết bị dưới 500 μ A trong phiên bản phần cứng hiện tại và tiếp tục giảm ở các vòng tối ưu PCB tiếp theo).
2. Xây dựng cơ chế quản lý nguồn thông minh với pin dự phòng và mạch Low Voltage Disconnect (LVD), tức mạch tự ngắt khi điện áp xuống thấp, để thiết bị bám được trạng thái xe nhưng không làm ảnh hưởng đến khả năng khởi động của phương tiện.
3. Thiết kế đường truyền dữ liệu sử dụng MQTT qua mạng 4G/LTE để gửi các gói dữ

liệu nhỏ qua mạng di động, ưu tiên tự phục hồi kết nối và chờ sẵn phương án lưu đệm khi mất mạng kéo dài.

4. Phát triển nền tảng máy chủ có khả năng mở rộng, bao gồm API phía máy chủ, cơ sở dữ liệu quan hệ cho phần quản trị, cơ sở dữ liệu chuỗi thời gian cho dữ liệu phát sinh liên tục, và giao diện web theo dõi thời gian thả±c.

Bốn nhóm mục tiêu trên cho biết đích đến của thiết kế. Tuy nhiên, để chọn đúng công nghệ và kiến trúc, vẫn cần đặt chúng vào bối cảnh kỹ thuật rộng hơn. Vì vậy, mục 2.2 chuyển sang rà lại các nền tảng kỹ thuật liên quan trực tiếp đến bài toán.

2.2. Bối cảnh và cơ sở kỹ thuật – Background and Technical reviews

2.2.1. Tổng quan về hệ thống IoT trong giám sát phương tiện

Internet of Things (IoT) là mô hình cho phép các thiết bị vật lý kết nối với nhau và với hệ thống máy chủ thông qua Internet. Ở góc nhìn đơn giản, một hệ thống IoT giám sát phương tiện luôn phải đi qua ba bước: thu dữ liệu trên xe, truyền dữ liệu lên mạng, rồi biến dữ liệu đó thành thông tin mà người quản lý có thể đọc và hành động. Tương ứng với ba bước đó, hệ thống thường được tổ chức thành ba lớp chính [3]:

- **Lớp cảm biến và thu thập dữ liệu (Perception Layer):** Đây là phần nằm gần chiếc xe nhất. Nó gồm GPS/GNSS để biết xe đang ở đâu, IMU để biết xe có rung hoặc di chuyển không, OBD2 để đọc trạng thái vận hành của xe, và các cảm biến khác nếu cần. Nhiệm vụ của lớp này là “nhìn thấy” những gì đang diễn ra trên xe và biến chúng thành dữ liệu thô.
- **Lớp mạng và truyền thông (Network Layer):** Đây là con đường đưa dữ liệu rời khỏi chiếc xe. Lớp này dùng các giao thức như MQTT, HTTP hoặc CoAP qua mạng di động để chuyển dữ liệu từ thiết bị lên máy chủ. Nói gọn hơn, đây là phần quyết định dữ liệu sẽ được gửi theo kiểu “gói dữ liệu nhỏ gửi liên tục”, “yêu cầu rồi chờ trả lời” hay một chuẩn gọn hơn cho thiết bị nhỏ. Nó cũng phải xử lý các tình huống mất sóng hoặc kết nối lại.
- **Lớp ứng dụng và xử lý (Application Layer):** Đây là nơi dữ liệu được biến thành thứ con người có thể dùng. Lớp này bao gồm các dịch vụ máy chủ như broker MQTT, API phía máy chủ, cơ sở dữ liệu và giao diện quản trị như màn hình web tổng hợp hoặc ứng dụng di động.

Trong triển khai của đề tài, kiến trúc ba lớp trên được bổ sung thêm một lớp đệm dữ liệu cục bộ tại thiết bị bằng **microSD SDMMC 4-bit + FATFS**. Đây là một thẻ nhớ cục bộ nối với ESP32-S3 bằng chuẩn giao tiếp tốc độ cao và được quản lý bằng hệ thống tệp quen thuộc. Có thể xem nó như một “kho chờ ngắn hạn”: khi mạng di động gián đoạn, dữ liệu vận hành gửi lên liên tục sẽ tạm nằm ở đây; khi kết nối phục hồi, hệ thống phát lại theo đúng thứ tự. Nhờ vậy, độ liên tục dữ liệu tăng lên mà không cần thay đổi lớn ở kiến trúc máy chủ phía trên.

2.2.2. Các giải pháp giám sát phương tiện hiện có

Bảng 2.1: So sánh các giải pháp giám sát phương tiện

Tiêu chí	Giám sát thủ công	GPS Tracker đơn giản	Fleet Management thương mại	Hệ thống đề xuất
Chi phí thiết bị	Không	500.000–1.500.000 VND	1.000.000–5.000.000 VND	1.514.000 VND
Phí dịch vụ hàng tháng	Không	50.000–100.000 VND	200.000–500.000 VND	Chi phí 4G SIM (70.000 VND)
Độ chính xác vị trí	Thấp (báo cáo thủ công)	Trung bình (GPS)	Cao (GPS + A-GPS)	Cao (GNSS đa hệ thống)
Dữ liệu động cơ (OBD2)	Không	Không	Có (tùy model)	Có (BLE OBD2)
Phát hiện bất thường khi đậu	Không	Hạn chế	Có	Có (IMU + deep sleep)
Khả năng tùy biến	Không áp dụng	Thấp	Thấp (phụ thuộc nhà cung cấp)	Cao (mã nguồn mở)
Quản lý năng lượng	Không áp dụng	Cơ bản	Tốt	Tốt (đa chế độ + pin dự phòng)
Tích hợp hệ thống	Không	Hạn chế (API riêng)	API do nhà cung cấp	API mở (REST + WebSocket)

Bảng so sánh cho thấy phương án đề xuất giữ được các năng lực quan trọng của hệ thống thương mại như độ chính xác vị trí, khả năng đọc OBD2 và phát hiện bất thường, nhưng chi phí đầu tư thấp hơn và mức độ tùy biến cao hơn. Với bài toán của doanh nghiệp cho thuê xe trong nước, đây là hướng lựa chọn phù hợp hơn về cả kỹ thuật lẫn chi phí vận hành.

2.2.3. So sánh giao thức truyền thông IoT

Việc lựa chọn giao thức truyền thông không thể chỉ hiểu là chọn một “chuẩn phổ biến”. Với người không chuyên, có thể hình dung ba lựa chọn chính như sau: HTTP giống cách “gửi yêu cầu rồi chờ trả lời”; MQTT giống cách “đưa bản tin vào một kênh chung để bên cần nhận lấy ra”; còn CoAP là giao thức nhẹ cho thiết bị tài nguyên rất hạn chế, thường đi trên UDP.

Với hệ thống theo dõi phương tiện, quyết định này chi phối trực tiếp lượng dữ liệu phải truyền, khả năng chịu mất sóng và cách dịch vụ máy chủ mở rộng về sau [4]. Vì thế, việc so sánh không nhằm tìm giao thức “tốt nhất nói chung”, mà tìm giao thức phù hợp nhất với nhịp gửi dữ liệu nhỏ, liên tục và có lúc cần nhận lệnh ngược từ máy chủ.

Trong bảng dưới đây, “overhead” là phần dữ liệu phụ bắt buộc đi kèm mỗi gói tin; còn “QoS” là mức cam kết giao bản tin thành công. Hai tiêu chí này quan trọng vì chúng ảnh hưởng trực tiếp đến băng thông, độ tin cậy và tuổi thọ pin của thiết bị.

Bảng 2.2: So sánh các giao thức truyền thông IoT

Tiêu chí	MQTT	HTTP / REST	CoAP
Mô hình truyền thông	Phát bản tin vào kênh chung, bên nhận đăng ký theo dõi	Gửi yêu cầu rồi chờ phản hồi	Gửi yêu cầu rồi chờ phản hồi
Giao thức truyền tải bên dưới	TCP	TCP	UDP
Phần dữ liệu phụ đi kèm mỗi gói tin	2 byte (tối thiểu)	Hàng trăm byte	4 byte
Chất lượng dịch vụ (QoS)	3 mức (0, 1, 2)	Không có	2 mức (có xác nhận / không xác nhận)
Tiêu thụ băng thông	Thấp	Cao	Thấp
Hỗ trợ trao đổi hai chiều	Có (đăng ký nhận bản tin)	Không, trừ khi bổ sung cơ chế hỏi lặp hoặc kênh đẩy	Có (Observe)
Phù hợp cho IoT	Rất tốt	Trung bình	Tốt
Độ phổ biến	Rất cao	Rất cao	Trung bình
Mức phù hợp với EMQX	Có sẵn	Có, nhưng phải bổ sung thành phần mở rộng	Hạn chế

Phân tích lựa chọn: Kết quả so sánh cho thấy MQTT phù hợp hơn không phải vì nó “cao cấp” hơn HTTP hay CoAP, mà vì cách nó trao đổi dữ liệu gần với nhịp vận hành của hệ thống: thiết bị gửi nhiều bản tin nhỏ qua mạng di động không phải lúc nào cũng ổn định, đồng thời đôi lúc cần nhận lệnh ngược từ máy chủ. Cụ thể:

- **Phần dữ liệu phụ đi kèm rất nhỏ:** Header tối thiểu 2 byte, giảm băng thông và tiết kiệm năng lượng cho thiết bị IoT chạy bằng pin.

- **Mô hình phát bản tin vào kênh chung rồi đăng ký nhận:** Cho phép nhiều thành phần nhận dữ liệu cùng lúc mà không làm thiết bị trên xe phải gửi lặp đi lặp lại cho từng nơi nhận. Điều này phù hợp với kiến trúc có giao diện quản trị, dịch vụ xử lý và các kênh cảnh báo cùng khai thác một nguồn dữ liệu.
- **QoS linh hoạt:** Hệ thống không cần mọi bản tin đều được bảo đảm ở cùng một mức. QoS 0 phù hợp với dữ liệu vận hành gửi dày theo chu kỳ như vị trí GPS, trong khi QoS 1 phù hợp hơn với cảnh báo và lệnh điều khiển vì các bản tin này cần được đảm bảo tới nơi ít nhất một lần.
- **Lưu trạng thái cuối và thông báo khi thiết bị mất kết nối:** Hai cơ chế này giúp hệ thống phản ánh trạng thái online/offline của thiết bị rõ ràng hơn, từ đó người vận hành có thể phân biệt giữa "xe không có dữ liệu mới" và "thiết bị vừa mất kết nối".

2.2.4. So sánh vi điều khiển (MCU)

Bảng 2.3: So sánh các vi điều khiển cho ứng dụng thiết bị theo dõi IoT

Tiêu chí	ESP32-S3	STM32L4	Raspberry Pi Zero 2W
Kiến trúc	Xtensa LX7, dual-core, 240 MHz	ARM Cortex-M4, 80 MHz	ARM Cortex-A53, quad-core, 1 GHz
RAM	512 KB SRAM + 8 MB PSRAM	256 KB SRAM	512 MB DRAM
BLE	BLE 5.0 (tích hợp)	Không (cần module ngoài)	BLE 5.0 (tích hợp)
Wi-Fi	802.11 b/g/n (tích hợp)	Không	802.11 b/g/n (tích hợp)
Tiêu thụ deep sleep	10–15 μ A	1–2 μ A	Không hỗ trợ deep sleep
Giá thành (VND)	80.000–150.000	150.000–300.000	400.000–600.000
Framework phát triển	Arduino / ESP-IDF	STM32CubeIDE / Mbed	Linux / Python
Cộng đồng hỗ trợ	Rất lớn	Lớn	Rất lớn
Độ phù hợp cho thiết bị theo dõi	Cao	Trung bình	Thấp

Phân tích lựa chọn: Nếu chỉ so riêng mức tiết kiệm điện của bản thân vi điều khiển,

STM32L4 có lợi thế rõ rệt. Nhưng thiết bị theo dõi của đề tài không chỉ cần “ngủ tiết kiệm”, mà còn phải đồng thời nói chuyện với OBD2 qua BLE, modem qua UART, cảm biến qua I2C và xử lý logic trạng thái của toàn thiết bị. Khi nhìn theo bài toán tổng thể đó, ESP32-S3 cho điểm cân bằng thực tế hơn giữa tích hợp, chi phí và công sức triển khai. Các lý do chính là:

- **BLE tích hợp sẵn:** Có thể kết nối trực tiếp với adapter OBD2 vgate iCar Pro mà không cần gắn thêm module BLE riêng. Điều này làm mạch gọn hơn, ít linh kiện hơn và giảm chi phí tích hợp.
- **Chi phí phù hợp:** Giá thành thấp hơn đáng kể so với STM32L4 trong khi vẫn đủ năng lực xử lý cho bài toán của đề án.
- **Dễ phát triển và thử nghiệm:** Hệ sinh thái ESP-IDF, tài liệu và cộng đồng hỗ trợ cho MQTT, BLE và GNSS rất phong phú. Với nhóm làm đề án, đây là lợi thế lớn vì giảm thời gian thử sai và giảm rủi ro tắc ở các phần tích hợp.
- **Mức tiết kiệm điện vẫn đủ cho mục tiêu hệ thống:** Dòng deep sleep riêng của ESP32-S3 cao hơn STM32L4, nhưng tuổi thọ pin thực tế còn phụ thuộc cả mạch nguồn, cảm biến và linh kiện phụ trợ. Vì vậy, chênh lệch ở mức riêng MCU không phải yếu tố duy nhất quyết định hiệu quả năng lượng của toàn thiết bị.

2.2.5. So sánh cơ sở dữ liệu

Bảng 2.4: So sánh các giải pháp cơ sở dữ liệu cho hệ thống IoT

Tiêu chí	PostgreSQL + VictoriaMetrics	MongoDB + InfluxDB	TimescaleDB
Dữ liệu quan hệ (users, vehicles)	Rất tốt (PostgreSQL)	Trung bình (MongoDB)	Tốt (PostgreSQL core)
Dữ liệu chuỗi thời gian (vị trí, cảm biến, trạng thái gửi liên tục)	Rất tốt (VictoriaMetrics)	Tốt (InfluxDB)	Tốt (TimescaleDB extension)
Hiệu suất ghi	Cao (VictoriaMetrics: hàng triệu điểm/giây)	Cao (InfluxDB: hàng triệu điểm/giây)	Trung bình
Nén dữ liệu	Rất tốt (VictoriaMetrics: 10–70x nén)	Tốt (InfluxDB: TSM)	Tốt (PostgreSQL compression)
Tài nguyên tiêu thụ	Thấp (VictoriaMetrics: 1–2 GB RAM)	Cao (InfluxDB: 4+ GB RAM)	Trung bình

Bảng 2.4: So sánh các giải pháp cơ sở dữ liệu cho hệ thống IoT (tiếp theo)

Tiêu chí	PostgreSQL + VictoriaMetrics	MongoDB + InfluxDB	TimescaleDB
Ngôn ngữ truy vấn	SQL (PostgreSQL) + MetricsQL (VictoriaMetrics)	MongoDB Query + Flux/InfluxQL	SQL
Tích hợp Grafana	Có (native)	Có	Có
Giấy phép	Apache 2.0	SSPL (MongoDB) + MIT (InfluxDB OSS)	Apache 2.0
Độ phức tạp vận hành	Trung bình (2 hệ thống riêng)	Cao (2 hệ thống khác nhau)	Thấp (1 hệ thống)

Phân tích lựa chọn: Với hệ thống này, không phải mọi dữ liệu đều có cùng bản chất. Dữ liệu người dùng, xe, chuyến đi hay cảnh báo cần quan hệ rõ ràng giữa các bảng; trong khi dữ liệu vị trí và cảm biến lại là những bản ghi nhỏ, phát sinh liên tục theo thời gian. Vì vậy, thay vì ép tất cả vào một chỗ, đề tài chọn mỗi loại dữ liệu vào đúng nơi phù hợp: PostgreSQL cho dữ liệu nghiệp vụ và VictoriaMetrics cho dữ liệu chuỗi thời gian.

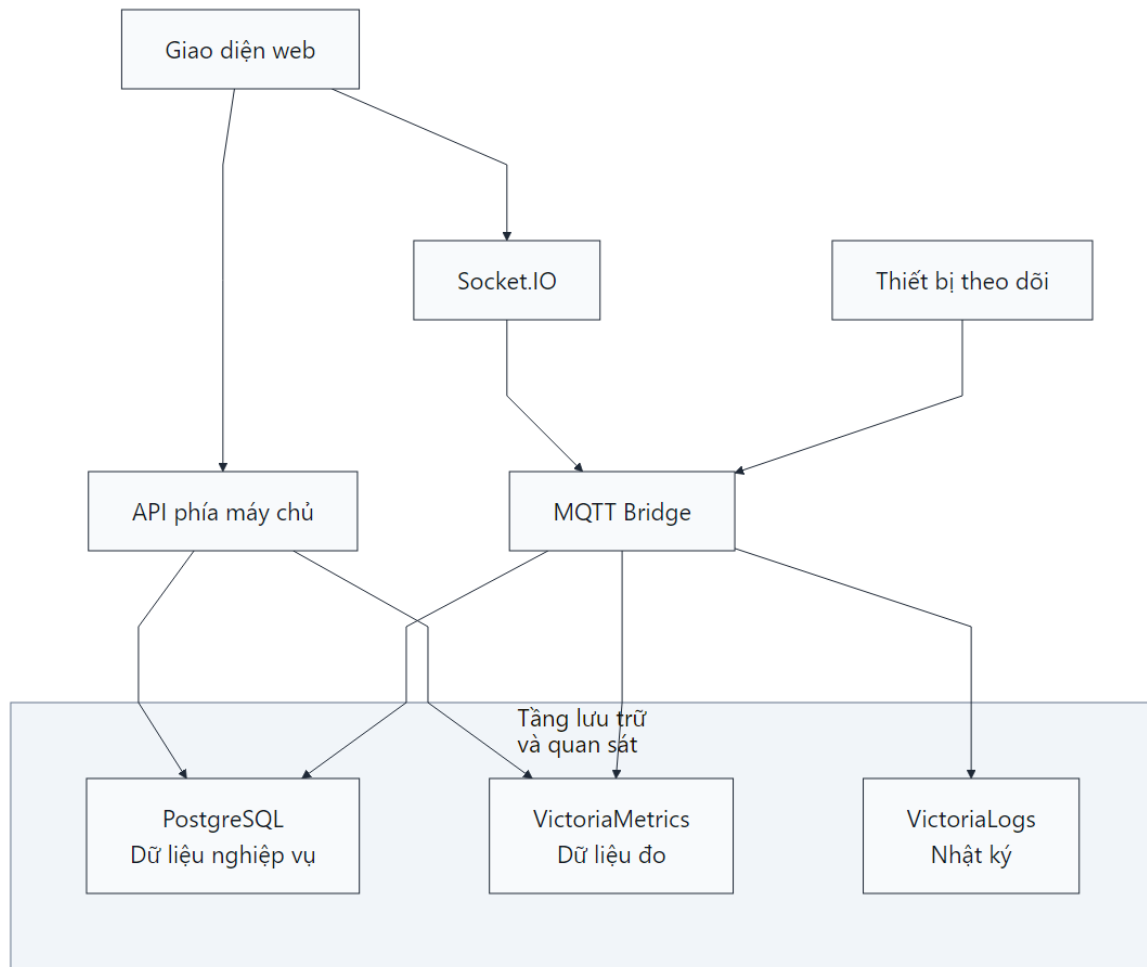
- **Phân tách trách nhiệm rõ ràng:** PostgreSQL lưu những dữ liệu cần liên kết chặt như người dùng, phương tiện, cảnh báo và vùng giám sát; VictoriaMetrics lưu những dòng dữ liệu tăng liên tục như GPS, OBD2 và dữ liệu cảm biến gửi theo chu kỳ. Mỗi hệ thống chỉ phải làm đúng phần việc mà nó phù hợp nhất.
- **Hiệu suất và tài nguyên:** VictoriaMetrics nén dữ liệu tốt và dùng ít RAM hơn nhiều phương án cùng loại, phù hợp với hạ tầng máy chủ của đồ án.
- **Thuận lợi cho giám sát:** Cả hai đều tích hợp tốt với Grafana, nên việc dựng màn hình giám sát kỹ thuật và theo dõi vận hành không cần thêm lớp chuyển đổi phức tạp.

2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế – Design criteria and Constraints

2.3.1. Yêu cầu chức năng

Những yêu cầu chức năng dưới đây đánh dấu bước chuyển từ “bài toán” sang “năng lực hệ thống”. Có thể xem chúng như năm lời hứa vận hành tối thiểu mà hệ thống phải thực hiện được để người quản lý thật sự dùng được trong bối cảnh đội xe vận hành ngoài thực tế.

FC-01: Theo dõi vị trí thời gian thực



Hình 2.1: Sơ đồ kiến trúc dữ liệu của hệ thống

- Gửi vị trí GPS định kỳ (5–30 giây khi lái xe, 10–30 phút khi đậu xe)
- Độ chính xác vị trí: dưới 5 mét trong điều kiện trời quang
- Hỗ trợ đa hệ thống định vị (GPS, GLONASS, BeiDou)

FC-02: Đọc dữ liệu động cơ qua OBD2

- Kết nối BLE với OBD2 adapter (vgate iCar Pro)
- Đọc trạng thái IGN (bật/tắt máy), RPM, tốc độ, nhiên liệu
- Tần suất đọc: 5–30 giây khi xe chạy (mặc định 5 giây)

FC-03: Phát hiện bất thường khi đậu xe

- IMU (LIS3DH) phát hiện chuyển động/rung bất thường
- Đánh thức ESP32 từ deep sleep trong vòng 100ms
- Gửi cảnh báo ưu tiên cao lên máy chủ

FC-04: Quản lý năng lượng thông minh

- Chuyển đổi giữa 3 chế độ: xe đang chạy (hoạt động đầy đủ), xe đang đỗ (tiết kiệm điện) và cảnh báo bất thường

- Pin dự phòng tự động tiếp quản theo profile nguồn: hệ 12V tại ngưỡng $Switch_{OFF} = 12.0\text{ V}$, hệ 24V tại ngưỡng $Switch_{OFF} = 24.0\text{ V}$
- Có phương án dự phòng suy luận trạng thái IGN qua ADC khi kết nối OBD2 BLE không sẵn sàng

FC-05: Giao diện web giám sát

- Bản đồ thời gian thực (Leaflet) hiển thị vị trí tất cả phương tiện
- Biểu đồ dữ liệu OBD2 (ECharts) - tốc độ, RPM, nhiên liệu
- Hệ thống cảnh báo và thông báo tự động
- Quản lý phương tiện, tài xế, khách hàng, hành trình

Năm nhóm chức năng trên bao quát trọn chuỗi nhu cầu cốt lõi của hệ thống: biết xe đang ở đâu, hiểu xe đang như thế nào, phát hiện khi có bất thường, không làm phiền hệ thống điện của xe và trình bày thông tin ở dạng mà người vận hành có thể đọc nhanh, xử lý nhanh.

2.3.2. Yêu cầu phi chức năng

Nếu yêu cầu chức năng trả lời “hệ thống phải làm được gì”, thì yêu cầu phi chức năng trả lời “hệ thống phải làm tốt tới mức nào”. Đây là lớp tiêu chí cho biết nguyên mẫu có đủ độ chín để bước ra môi trường vận hành thật hay chưa, đặc biệt khi phải theo dõi nhiều xe và chạy liên tục.

NFR-01: Hiệu suất

- Độ trễ toàn tuyến (từ thiết bị đến giao diện quản trị): dưới 3 giây trong điều kiện mạng bình thường
- Thời gian phản hồi API: dưới 200 ms cho 95% số yêu cầu
- Hỗ trợ đồng thời tối thiểu 50 phương tiện kết nối MQTT, với dư địa mở rộng ổn định tới khoảng 100 thiết bị

NFR-02: Độ tin cậy

- Tự động kết nối lại khi mạng phục hồi
- Giữ được cấu hình và trạng thái quan trọng qua NVS; lưu đệm dữ liệu vận hành cục bộ là hướng mở rộng của thiết bị
- MQTT QoS 1 cho các tin nhắn cảnh báo (đảm bảo gửi ít nhất một lần)

NFR-03: Bảo mật

- Xác thực phiên bằng token ngẫu nhiên không tự mang thông tin người dùng; phía máy chủ mới là nơi lưu và kiểm tra trạng thái phiên, còn trong cơ sở dữ liệu chỉ lưu bản băm SHA-256 của token
- MQTT ACL (Access Control List), tức danh sách quyền truy cập theo từng thiết bị
- Mã hóa TLS cho kết nối MQTT và HTTPS trong môi trường sản xuất

NFR-04: Khả năng mở rộng

- Kiến trúc nhiều dịch vụ tách rời, mỗi dịch vụ được đóng gói thành một gói chạy riêng bằng Docker
- MQTT broker (EMQX) hỗ trợ chạy thành cụm, tức nhiều máy có thể cùng phục vụ khi số thiết bị tăng
- Cơ sở dữ liệu chuỗi thời gian (VictoriaMetrics) tối ưu cho luồng dữ liệu lớn

NFR-05: Khả năng bảo trì

- Mã nguồn tổ chức theo Domain-Driven Design (DDD), nghĩa là chia theo từng nhóm nghiệp vụ như xe, thiết bị hay cảnh báo thay vì gom theo loại file kỹ thuật
- API theo kiểu REST với tài liệu tự sinh bằng Swagger/OpenAPI, tức trang mô tả sẵn các API để người phát triển và người kiểm thử cùng tra cứu
- Ghi log tập trung với VictoriaLogs và theo dõi trên Grafana, tức màn hình giám sát chung của hệ thống

Nhóm yêu cầu trên mô tả hệ thống cần làm được gì và cần đạt tới mức nào. Phần tiếp theo thu hẹp thêm một bước: trong khuôn khổ đề án, nhóm thực sự bị giới hạn bởi những điều kiện nào về phần cứng, phần mềm và chi phí.

2.3.3. Ràng buộc thiết kế

Các ràng buộc sau đây giới hạn không gian thiết kế của đề tài. Chúng buộc nhóm phải chọn phương án cân bằng giữa tính khả thi học thuật, khả năng triển khai thực tế trên xe và ngân sách sẵn có.

Ràng buộc phần cứng:

- Điện áp đầu vào: 12V hoặc 24V DC từ ắc quy xe (dao động phụ thuộc cấu hình hệ thống điện)
- Nhiệt độ hoạt động: -10 độ C đến 70 độ C (môi trường trong xe ô tô)
- Rung động và sốc: Chịu được rung động liên tục khi xe vận hành trên đường xá
- Kích thước: Đủ nhỏ để lắp đặt kín đáo trong xe (không lớn hơn 120x80x40 mm)

Ràng buộc phần mềm:

- Framework API phía máy chủ: ưu tiên Express.js + TypeScript vì đây là nền tảng nhóm đã có sẵn kinh nghiệm, giúp rút ngắn thời gian học lại từ đầu
- Framework giao diện web: chọn Next.js 15 + React 19 để phù hợp định hướng học thuật và thuận lợi cho màn hình quản trị có nhiều khu vực hiển thị
- MQTT Broker: chọn EMQX vì là giải pháp mã nguồn mở ổn định, đủ đáp ứng vai trò broker MQTT cho giai đoạn hiện tại
- Triển khai: dùng Docker Compose trên máy chủ Linux để mỗi dịch vụ có thể chạy tách riêng nhưng vẫn dễ dựng lại môi trường

Ràng buộc về chi phí:

- Tổng chi phí phần cứng: dưới 2.000.000 VND cho một bộ thiết bị theo dõi
- Ưu tiên các linh kiện có datasheet rõ ràng, chất lượng ổn định và nguồn cung trong nước để tiếp cận
- Ưu tiên các giải pháp mã nguồn mở để giảm chi phí giấy phép

2.3.4. Tiêu chuẩn thiết kế áp dụng

Bảng dưới đây tổng hợp các chuẩn kỹ thuật và quy ước triển khai được dùng làm nền cho hệ thống. Điểm quan trọng không phải là gom thật nhiều chuẩn, mà là chọn đúng những chuẩn thực sự chi phối khả năng tương thích giữa thiết bị, mạng truyền và ứng dụng.

Bảng 2.3.4A: Tổng hợp chuẩn và quy ước kỹ thuật áp dụng trong hệ thống

Chuẩn / quy ước	Mô tả	Áp dụng trong hệ thống
IEEE 802.15.1 (Bluetooth)	Chuẩn BLE 4.0/5.0	Kết nối OBD2 adapter
3GPP LTE Cat-4	Chuẩn 4G/LTE	Truyền dữ liệu qua modem SIM7600CE-T (Cat-4)
NMEA 0183	Chuẩn dữ liệu GPS	Phân tích tọa độ từ khối GNSS tích hợp
SAE J1979 (OBD2)	Chuẩn chẩn đoán động cơ	Đọc dữ liệu qua OBD2 BLE adapter
MQTT v5.0 (OASIS)	Giao thức nhắn tin cho thiết bị IoT	Truyền dữ liệu thiết bị - máy chủ
REST (RFC 7231)	Cách tổ chức API theo tài nguyên	API phía máy chủ
Mã phiên ngẫu nhiên + ACL MQTT	Cơ chế xác thực và phân quyền	Xác thực phiên người dùng, giới hạn quyền truy cập thiết bị theo ACL MQTT

2.4. Yêu cầu từ các bên liên quan – Stakeholder requirements

2.4.1. Xác định các bên liên quan

Hệ thống IoT Vehicle Tracking System liên quan đến nhiều nhóm sử dụng với nhu cầu khác nhau. Nếu không tách rõ ngay từ đầu, hệ thống rất dễ rơi vào tình trạng mạnh về mặt kỹ thuật nhưng lệch khỏi nhu cầu vận hành. Vì vậy, phần này xác định ai là người hưởng lợi trực tiếp, ai là người vận hành, ai là người bị tác động, từ đó giữ trọng tâm

thiết kế vào những nhu cầu thực sự cần giải quyết.

2.4.2. Ma trận yêu cầu các bên liên quan

Bảng 2.5: Ma trận yêu cầu các bên liên quan

STT	Bên liên quan	Vai trò	Yêu cầu chính	Mức độ ưu tiên
1	Công ty cho thuê xe	Chủ sở hữu hệ thống	Giám sát vị trí tất cả xe theo thời gian thực; Nhận cảnh báo khi xe đi ra khỏi vùng cho phép; Báo cáo hành trình, quãng đường, nhiên liệu; Giảm thiểu thời gian xe đậu không sử dụng	Cao
2	Tài xế / Người thuê xe	Người vận hành xe	Không bị giám sát quá mức (quyền riêng tư); Thiết bị không ảnh hưởng đến vận hành xe; Không rút cạn ắc quy xe	Trung bình
3	Quản trị viên hệ thống	Người quản lý kỹ thuật	Giao diện quản lý dễ sử dụng; Hệ thống ổn định, ít lỗi; Có khả năng theo dõi trạng thái thiết bị (online/offline); Quản lý người dùng và phân quyền	Cao
4	Đội bảo trì	Kỹ thuật viên	Thiết bị dễ lắp đặt và bảo trì; Có thể kiểm tra tình trạng thiết bị từ xa mà không cần tháo xuống; Cảnh báo bảo trì định kỳ (pin yếu, thiết bị ngoại tuyến)	Trung bình
5	Khách hàng (người thuê xe cuối)	Người dùng gián tiếp	Quá trình thuê xe nhanh gọn; Tin tưởng vào độ bảo mật của dữ liệu cá nhân; Nhận thông báo về trạng thái đơn thuê	Thấp (đưa vào giai đoạn 2)

2.4.3. Phân tích chi tiết yêu cầu

Từ ma trận trên, có thể thấy không phải mọi bên liên quan đều đòi hỏi cùng một loại giá trị. Nhóm sử dụng chính quan tâm đến hiệu quả vận hành và an ninh; nhóm quản trị quan tâm đến tính ổn định và khả năng kiểm soát; còn nhóm bảo trì cần hệ thống dễ chẩn đoán và dễ can thiệp khi có sự cố.

Công ty cho thuê xe (Stakeholder chính)

Đây là nhóm đối tượng trọng tâm của hệ thống. Các yêu cầu của nhóm này tập trung vào ba khía cạnh:

1. *Giám sát và an ninh*: Theo dõi vị trí phương tiện 24/7, phát hiện và cảnh báo ngay khi có bất thường như xe bị di chuyển trái phép, vượt khỏi vùng giám sát đã vẽ trên bản đồ hoặc chạy quá tốc độ. Hệ thống cần gửi thông báo qua nhiều kênh như giao diện web quản trị, Telegram bot và email.
2. *Báo cáo và phân tích*: Xuất báo cáo hành trình chi tiết (quãng đường, thời gian, điểm dừng), thống kê nhiên liệu tiêu thụ, đánh giá hành vi lái xe (tốc độ trung bình, số lần phanh gấp). Các báo cáo này giúp tối ưu hóa chi phí vận hành và bảo trì.
3. *Quản lý đội xe*: Quản lý trạng thái từng phương tiện (đang cho thuê, đang bảo trì, sẵn sàng), lịch bảo trì định kỳ dựa trên số km hoặc thời gian, lịch sử cho thuê và doanh thu theo phương tiện.

Quản trị viên hệ thống

Yêu cầu của quản trị viên tập trung vào khả năng vận hành và giám sát hệ thống:

1. *Trang tổng quan*: Hiển thị trạng thái tất cả thiết bị (online/offline, mức pin, cường độ tín hiệu), số lượng phương tiện đang hoạt động, cảnh báo chưa xử lý.
2. *Quản lý thiết bị*: Thêm/xóa/sửa thông tin thiết bị theo dõi, cấp phát thiết bị cho phương tiện, gửi lệnh điều khiển từ xa (reset, cập nhật cấu hình, yêu cầu vị trí).
3. *Quản lý người dùng*: Tạo tài khoản, phân quyền (admin, manager, viewer), theo dõi lịch sử đăng nhập và thao tác thông qua audit log, tức bản ghi ai đã làm gì trong hệ thống.

Đội bảo trì

Yêu cầu của đội bảo trì hướng đến việc giảm thời gian và chi phí bảo trì:

1. *Chẩn đoán từ xa*: Xem trạng thái thiết bị (mức pin, nhiệt độ, cường độ tín hiệu 4G) từ xa, phát hiện và cảnh báo khi thiết bị gặp lỗi (mất kết nối kéo dài, pin yếu).
2. *Hướng dẫn lắp đặt*: Tài liệu hướng dẫn lắp đặt chi tiết, danh sách kiểm tra (checklist) khi lắp đặt mới hoặc bảo trì.

Nhìn tổng thể, ba nhóm yêu cầu này kéo hệ thống về một điểm cân bằng khá rõ: phải đủ mạnh để quản lý đội xe, nhưng cũng phải đủ gọn và ổn định để lắp đặt, vận hành và bảo trì trên thực địa. Đây cũng là cơ sở để ma trận truy xuất ở mục tiếp theo ánh xạ yêu cầu định tính sang các chức năng cụ thể của hệ thống.

2.4.4. Ma trận truy xuất yêu cầu - chức năng

Bảng 2.6: Ma trận truy xuất yêu cầu các bên liên quan với chức năng hệ thống

Yêu cầu bên liên quan	FC-01 (Vị trí)	FC-02 (OBD2)	FC-03 (Bắt thường)	FC-04 (Năng lượng)	FC-05 (Web)
Giám sát vị trí 24/7	X			X	X
Cảnh báo vượt vùng cho phép	X				X
Báo cáo nhiên liệu		X			X
Phát hiện trộm xe			X	X	X
Không rút cạn ắc quy				X	
Quản lý thiết bị từ xa					X
Chẩn đoán lỗi từ xa	X	X		X	X

Ma trận truy xuất trên cho thấy các chức năng hệ thống (FC-01 đến FC-05) bao phủ toàn bộ yêu cầu của các bên liên quan chính. Trong đó, FC-04 (Quản lý năng lượng) và FC-05 (Giao diện web) là hai chức năng được nhắc đến nhiều nhất, qua đó khẳng định tầm quan trọng của thiết kế quản lý năng lượng thông minh và giao diện giám sát thân thiện.

Kết luận chương 2

Chương 2 đã chuyển bài toán ở mức "ý tưởng" thành các ràng buộc kỹ thuật cụ thể: thiết bị phải tiết kiệm điện ra sao, truyền dữ liệu thế nào, lưu trữ ở đâu và kiến trúc cần mở rộng theo hướng nào. Thông qua quá trình khảo sát và đối sánh các giải pháp hiện có, chương đã làm rõ tiêu chí lựa chọn thay vì chỉ liệt kê công nghệ. Đây chính là cơ sở để Chương 3 bước sang phần quyết định quan trọng tiếp theo: chọn phương án thiết kế nào là phù hợp nhất với bài toán đã được phân tích.

CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ – DESIGN SOLUTIONS

Sau khi xác định rõ yêu cầu và ràng buộc ở Chương 2, Chương 3 tập trung vào câu hỏi ”chọn gì và vì sao chọn như vậy”. Chương này không chỉ nêu phương án cuối cùng, mà còn trình bày quá trình cân nhắc giữa nhiều lựa chọn ở bốn tầng: phần cứng thiết bị theo dõi, phần mềm nhúng trên vi điều khiển, hạ tầng máy chủ xử lý dữ liệu và giao diện người dùng.

Nhờ cách trình bày đó, người đọc có thể theo dõi logic thiết kế theo từng bước thay vì chỉ nhìn thấy kết quả cuối cùng.

3.1. Phân tích và đề xuất giải pháp phần cứng – Hardware analysis and solution

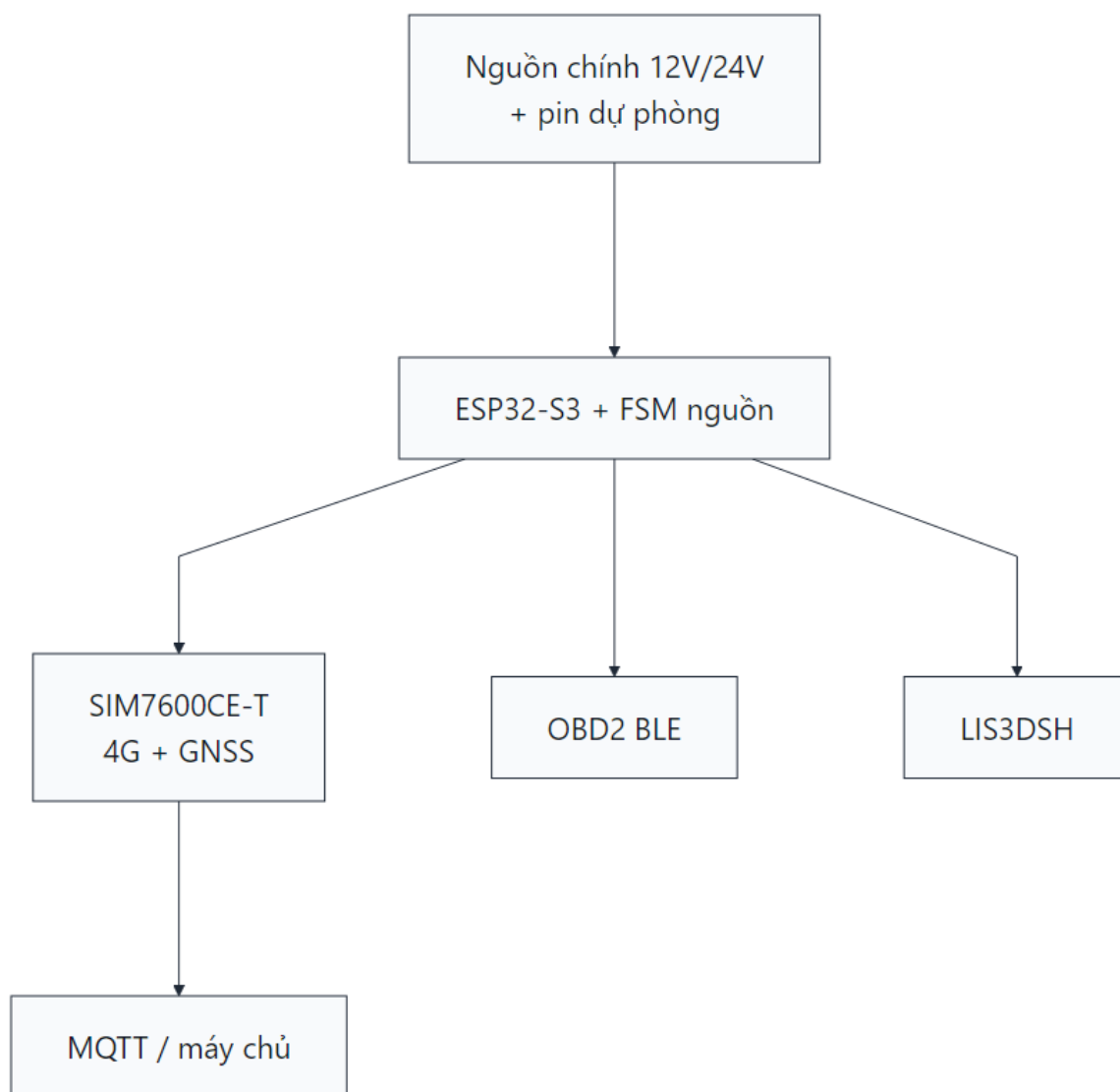
3.1.1. Phân tích và lựa chọn phần cứng

Phần này trình bày quá trình phân tích, lựa chọn và thiết kế các thành phần phần cứng cho thiết bị theo dõi xe IoT. Mục tiêu không chỉ là tạo ra một thiết bị theo dõi ”đủ chức năng” trên lý thuyết, mà là một thiết bị có thể lắp lên xe, vận hành ổn định và còn chừa dư địa cho bảo trì, mở rộng. Vì vậy, mỗi quyết định kỹ thuật đều được soi chiếu dưới bốn góc nhìn: độ tin cậy, khả năng hoạt động liên tục với nguồn dự phòng, mức độ tích hợp và mức độ phù hợp với ngân sách.

Quy ước thuật ngữ trong phần cứng: tài liệu dùng cụm ”khối phần cứng/khối chức năng” cho các thành phần tích hợp trên PCB; từ ”module” chỉ dùng khi nói về module phần mềm, như firmware trên thiết bị hoặc dịch vụ phía máy chủ, hoặc khi trích dẫn nguyên văn tên sản phẩm của nhà sản xuất.

3.1.1.1. Sơ đồ khối hệ thống Hệ thống thiết bị theo dõi được tổ chức theo kiến trúc khối chức năng với năm khối chính, trong đó ESP32-S3 đóng vai trò điều phối trung tâm. Nếu nhìn dưới góc độ người sử dụng, năm khối này tương ứng với năm câu hỏi rất thực tế: thiết bị lấy dữ liệu xe ở đâu, xác định vị trí bằng cách nào, phát hiện rung/chuyển động ra sao, gửi dữ liệu về máy chủ như thế nào và lấy điện từ đâu để duy trì hoạt động. Sơ đồ khối tổng thể được trình bày như sau:

Sơ đồ khối cho thấy thiết bị theo dõi thực chất đang làm bốn việc liên tiếp: đọc trạng thái của xe, theo dõi chuyển động, gửi dữ liệu ra ngoài và tự bảo vệ nguồn điện. Trong đó, OBD2 là cổng chẩn đoán của xe, cho biết xe đang vận hành ra sao; khối LTE/GNSS phụ trách liên lạc qua mạng di động và định vị bằng vệ tinh; IMU là cảm biến quán tính giúp hệ thống vẫn ”canh xe” khi động cơ đã tắt; còn khối nguồn đảm bảo thiết bị đổi giữa nguồn chính và pin dự phòng một cách an toàn. Vi điều khiển đứng ở giữa để gom mọi tín hiệu này và quyết định thiết bị phải ở trạng thái nào.

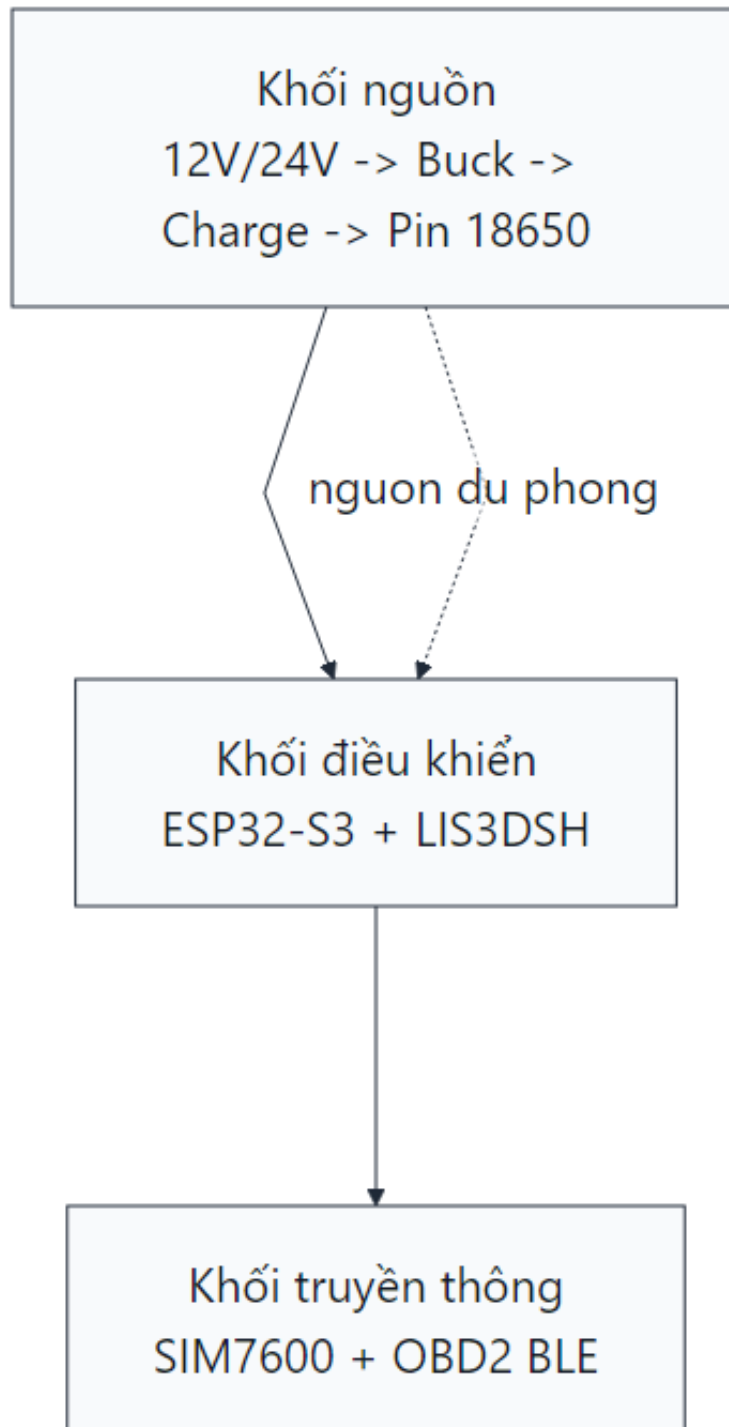


Hình 3.1: Sơ đồ khối tổng thể hệ thống thiết bị theo dõi

Hệ thống vận hành theo ba chế độ chính: (1) chế độ lái xe — khi động cơ bật (IGN ON), các khối chức năng cần thiết được kích hoạt; (2) chế độ đỗ xe — khi động cơ tắt (IGN OFF), ESP32 chuyển sang deep sleep, tức trạng thái ngủ sâu để giảm tiêu thụ điện, và chỉ IMU LIS3DH duy trì giám sát chuyển động; (3) chế độ cảnh báo — khi IMU ghi nhận chuyển động bất thường, hệ thống tự đánh thức và gửi cảnh báo qua 4G.

Luồng dữ liệu vì thế cũng rất dễ hình dung. Dữ liệu vận hành của xe (như RPM, tốc độ, nhiên liệu) được lấy qua BLE, tức Bluetooth tiết kiệm năng lượng, từ adapter OBD2; vị trí được lấy từ GNSS tích hợp trong modem; còn rung động được đọc từ IMU qua I2C, tức đường giao tiếp ngắn thường dùng cho cảm biến. ESP32-S3 tổng hợp ba nguồn này, đóng gói thành bản tin rồi gửi lên máy chủ bằng MQTT qua kết nối 4G/LTE của modem.

Để tăng độ liên tục dữ liệu khi mạng gián đoạn, luồng dữ liệu vận hành gửi liên tục được bổ sung thêm nhánh lưu đệm cục bộ trên microSD. Thẻ nhớ này được nối bằng SDMMC



Hình 3.1a: Phân rã chi tiết các khối chức năng của thiết bị theo dõi

4-bit, tức chuẩn giao tiếp nhanh cho thẻ nhớ, và được quản lý bằng FATFS, tức hệ thống tệp để đọc ghi ổn định. Khi mất mạng, dữ liệu sẽ được giữ lại theo đúng thứ tự; khi có mạng trở lại, thiết bị gửi bù phần còn thiếu lên MQTT.

Sau khi chốt được bức tranh tổng thể và đường đi của dữ liệu, bước kế tiếp là chọn phần tử điều phối trung tâm đủ sức gánh toàn bộ các khối đó. Vì vậy, phần sau tập trung vào việc lựa chọn vi điều khiển.

3.1.1.2. Phân tích và lựa chọn vi điều khiển (MCU) Việc lựa chọn vi điều khiển (MCU) là quyết định thiết kế quan trọng nhất, ảnh hưởng trực tiếp đến khả năng tích hợp các khối ngoại vi, mức tiêu thụ năng lượng, và độ phức tạp phát triển firmware. Ba ứng viên được đưa vào quá trình đánh giá là ESP32-S3 của Espressif, STM32L4 của STMicroelectronics, và nRF52840 của Nordic Semiconductor.

a) Các ứng viên **ESP32-S3** là vi điều khiển hai nhân dùng kiến trúc Xtensa LX7, hoạt động tới 240 MHz và tích hợp sẵn WiFi cùng Bluetooth LE 5.0 [19]. Điểm quan trọng với đồ án không chỉ là xung nhịp, mà là chip này có đủ tài nguyên để vừa xử lý logic điều khiển, vừa giao tiếp với nhiều phần cứng khác nhau. Thiết bị có 512 KB SRAM trên chip, dùng flash ngoài, hỗ trợ nhiều cổng giao tiếp như UART, I2C, SPI và ADC, tức các đường nối cơ bản để làm việc với modem, cảm biến và mạch đo điện áp, đồng thời có chế độ ngủ sâu rất tiết kiệm điện [19], [20].

STM32L4 (đại diện bởi STM32L476 trong so sánh) là dòng vi điều khiển tiết kiệm điện của ST, dùng nhân ARM Cortex-M4 tới 80 MHz và không tích hợp sẵn BLE hay Wi-Fi [53], [54]. Điểm mạnh của dòng này là các chế độ ngủ sâu có mức tiêu thụ rất thấp theo từng cấu hình nguồn [53].

nRF52840 là vi điều khiển của Nordic với nhân ARM Cortex-M4F 64 MHz, hỗ trợ mạnh về Bluetooth LE 5.x và 802.15.4 [55], [56]. Thiết bị có 1 MB Flash, 256 KB RAM, hỗ trợ cổng nối tiếp UART/UARTE và cũng có chế độ tắt sâu với dòng tiêu thụ rất thấp [55], [56].

b) Đối chiếu yêu cầu thiết kế với thông số phần cứng Thay vì chấm điểm tổng quát, Bảng 3.1 đối chiếu trực tiếp các ràng buộc kỹ thuật của thiết bị theo dõi với thông số và tác động tích hợp của từng MCU.

Bảng 3.1: Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển

Yêu cầu của thiết bị theo dõi	ESP32-S3	STM32L4	nRF52840
Kết nối OBD2 BLE với vgate iCar Pro	BLE 5.0 tích hợp, tương thích ngược BLE 4.0	Không tích hợp BLE, phải thêm module ngoài	BLE 5.0 tích hợp
Cấu hình UART của hệ thống	3 UART controllers (đủ cho modem + debug, còn dư cho mở rộng) [19], [20]	Nhiều serial instance trong dòng STM32L476 (USART, UART, LPUART), đáp ứng yêu cầu số cổng [53], [54]	UARTE hoặc UART thường dùng 2 instance, ít dư địa nếu cần giữ debug UART riêng [55], [56]
Tài nguyên xử lý cho OBD2 + modem + IMU	Dual-core 240 MHz, 512 KB SRAM [19]	Cortex-M4 tới 80 MHz, 128 KB SRAM trên STM32L476RE [54]	Cortex-M4F 64 MHz, 256 KB RAM [55], [56]
Dòng ngủ sâu/siêu thấp công suất	Deep-sleep mức μA tùy cấu hình RTC/IO [19], [20]	STOP2 cỡ 1.1 μA (theo điều kiện datasheet), standby/shutdown cỡ nA [53], [54]	System OFF mức dưới μA tùy RAM retention/GPIO [55]
Framework triển khai	Arduino IDE + ESP-IDF [47]	STM32Cube/HAL [53]	nRF Connect SDK [57]
Phần cứng bổ sung để đạt đủ chức năng	Không cần thêm radio BLE	Cần thêm BLE module nếu vẫn giữ OBD2 BLE (STM32L476 không tích hợp BLE) [53], [54]	Không cần BLE module, nhưng cần xử lý bài toán thiếu serial cho debug
Chi phí triển khai phần cứng tham chiếu	100.000–200.000 VND (mặt bằng BOM dự án)	150.000–300.000 VND (mặt bằng BOM dự án)	Thường cao hơn ESP32-S3 và ít phổ biến hơn trong bối cảnh dự án

Bảng 3.1: Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển (tiếp theo)

Yêu cầu của thiết bị theo dõi	ESP32-S3	STM32L4	nRF52840
Tác động tới sơ đồ hiện tại	Giữ nguyên sơ đồ BLE + modem UART, thuận lợi cho tích hợp GNSS trong modem	Tăng phần cứng BLE ngoài và công đoạn tích hợp ban đầu	Có thể phải chuyển debug sang USB/SWO để nhường UART cho chức năng chính

c) Phân tích các yếu tố quan trọng BLE tích hợp: Đây là ràng buộc rất thực tế vì thiết bị theo dõi dùng adapter OBD2 vgate iCar Pro theo chuẩn BLE 4.0. Nếu vi điều khiển không có BLE tích hợp, thiết kế sẽ phải bổ sung thêm một chip hoặc module Bluetooth riêng. Điều đó không chỉ làm tăng BOM, mà còn khiến sơ đồ mạch, nguồn cấp và phần mềm điều khiển phức tạp hơn ngay từ đầu. Ở khía cạnh này, ESP32-S3 và nRF52840 thuận lợi hơn STM32L4.

Số cổng UART khả dụng: UART ở đây là cổng nối tiếp cơ bản để vi điều khiển giao tiếp với modem hoặc mở cổng debug. Trong kiến trúc hiện tại, UART không phải tài nguyên dư dả. Hệ thống cần ít nhất một kênh cho modem và một kênh cho debug, đồng thời vẫn nên chừa sẵn một cổng cho mở rộng hoặc chẩn đoán khi triển khai thực tế. ESP32-S3 đáp ứng tốt nhu cầu này với 3 UART phần cứng; nRF52840 hạn chế hơn; còn STM32L4 có thể đáp ứng nhưng phải đánh đổi bằng việc bổ sung BLE ngoài.

Tiêu thụ deep sleep: Deep sleep là chế độ ngủ sâu, nơi vi điều khiển gần như tắt hết hoạt động để tiết kiệm điện tối đa. STM32L4 và nRF52840 có lợi thế rõ ràng về dòng ngủ sâu theo tài liệu hãng. Tuy nhiên, trong một thiết bị theo dõi hoàn chỉnh, vi điều khiển không phải thành phần duy nhất tiêu thụ năng lượng. Modem LTE, khối GNSS, chu kỳ đánh thức và các linh kiện phụ trợ cũng góp phần đáng kể vào mức tiêu thụ điện của toàn hệ thống. Vì vậy, chênh lệch ở mức riêng MCU có ý nghĩa, nhưng không đủ để bù cho việc tăng độ phức tạp tích hợp [19], [53], [54], [55].

Độ phức tạp triển khai: Xét theo công sức hiện thực, ESP32-S3 thuận lợi hơn vì đã có BLE tích hợp sẵn và hệ sinh thái phát triển quen thuộc với nhóm làm đồ án. Điều này đặc biệt quan trọng khi firmware phải đồng thời xử lý OBD2 qua BLE, gửi lệnh AT cho modem, đọc IMU qua I2C và duy trì logic ngủ sâu. STM32L4 tiết kiệm điện tốt hơn nhưng kéo theo thêm việc tích hợp BLE; còn nRF52840 mạnh về BLE nhưng ít dư địa UART hơn cho kịch bản thiết bị theo dõi hiện tại.

d) Kết luận lựa chọn Trên cơ sở đối chiếu trực tiếp các yêu cầu của hệ thống, **ESP32-S3** được chọn làm vi điều khiển chính cho thiết bị theo dõi. Nói ngắn gọn, đây là phương án cân bằng tốt nhất giữa ”đủ mạnh để làm được việc” và ”đủ gọn để triển khai thực tế”. Quyết định này dựa trên các điểm chốt sau:

- Đáp ứng đầy đủ kiến trúc hiện tại: 1 kết nối BLE cho OBD2, UART cho modem và một đường debug riêng, đồng thời vẫn còn dư địa UART cho mở rã»TMng.
- BLE 5.0 tích hợp giúp bỏ hẳn module BLE ngoài, giảm BOM và rút ngắn công đoạn tích hợp.
- Tài nguyên xử lý 240 MHz dual-core và 512 KB SRAM đủ để chạy đồng thời BLE, modem, IMU và máy trạng thái.
- Chi phí triển khai phần cứng với ESP32-S3 thấp hơn STM32L4 khoảng 30–50% theo mặt bằng linh kiện đang dùng trong đồ án.
- Dòng deep-sleep của ESP32-S3 cao hơn STM32L4 theo tài liệu hãng, nhưng vẫn nằm trong giới hạn chấp nhận được của bài toán pin dự phòng khi xét toàn bộ duty-cycle của modem, GNSS và các chu kỳ wake-up [19], [53], [54].

Bảng 3.2: Thông số kỹ thuật ESP32-S3 được chọn

Thông số	Giá trị
CPU	Dual-core Xtensa LX7, tối đa 240 MHz [19]
RAM on-chip	512 KB SRAM [19]
Flash	Flash ngoài (biến thể WROOM thường 4–16 MB) [19], [20]
Bluetooth	Bluetooth LE 5.0, tương thích ngược BLE 4.0 cho OBD2 adapter [19]
WiFi	802.11 b/g/n (không dùng trong luồng chính của thiết bị theo dõi) [19]
UART	3 UART controllers [19], [20]
I2C	2 I2C controllers [19], [20]
ADC	SAR ADC 12-bit, tối đa 20 kênh (theo package) [19]
GPIO	Tối đa 45 GPIO (theo package) [19]
Dòng deep-sleep	Mức μ A tùy cấu hình RTC domain và IO [19], [20]
Biến thể linh kiện	ESP32-S3-WROOM-1
Phương án tích hợp	Tích hợp trực tiếp vào bo mạch chính

3.1.1.3. Phân tích và lựa chọn modem truyền thông Modem truyền thông đóng vai trò then chốt trong hệ thống vì nó đồng thời giải quyết hai nhu cầu thiết yếu nhất của thiết bị theo dõi: gửi dữ liệu về máy chủ qua mạng di động và cung cấp vị trí của xe

thông qua GNSS. Với bài toán này, lựa chọn modem không chỉ ảnh hưởng đến tốc độ hay thông số kỹ thuật, mà còn tác động trực tiếp đến cách đi dây, cách tổ chức firmware và mức độ phức tạp của toàn bộ bo mạch. Trong kiến trúc hiện tại, hệ thống sử dụng **SIM7600CE-T tích hợp LTE + GNSS** nhằm giảm số lượng phần cứng rời, đơn giản hóa đi dây và đồng bộ với firmware thực tế.

a) Các ứng viên Ba phương án được đưa vào đánh giá gồm hai phương án tách rời (**SIMCom A7670C + u-blox NEO-M8N**, **Quectel EC200U-CN + NEO-M8N**) và một phương án tích hợp (**SIMCom SIM7600CE-T**).

SIMCom A7670C + u-blox NEO-M8N là phương án tách rời: A7670C phụ trách LTE/AT command, NEO-M8N phụ trách GNSS/NMEA. Theo trang sản phẩm chính thức của SIMCom, A7670C nằm trong nhóm LTE Cat-1 và dùng nguồn 3.4–4.2 V [58], [62].

Quectel EC200U-CN + NEO-M8N có cấu trúc gần giống phương án A7670C + NEO-M8N. Tuy nhiên dòng EC200U được hãng mô tả là Cat-1 bis, GNSS là tính năng tùy chọn theo biến thể/cấu hình và I/O logic có ràng buộc riêng, nên mức độ phù hợp với cấu hình đang triển khai thấp hơn [59], [61].

SIMCom SIM7600CE-T là phương án tích hợp GNSS ngay trong modem, tức cùng một khối phần cứng vừa lo kết nối mạng di động vừa lo lấy vị trí từ vệ tinh. Cách làm này giúp đi dây gọn hơn và giảm số lượng phần cứng rời [60].

b) Đối chiếu kiến trúc LTE/GNSS theo ràng buộc tích hợp Vấn đề cốt lõi của thiết bị theo dõi không chỉ là modem có lên mạng được hay không, mà còn là phương án LTE/GNSS, tức phương án kết hợp giữa kết nối di động và định vị vệ tinh, ảnh hưởng thế nào tới sơ đồ UART, hồ sơ tiêu thụ nguồn và mức độ thuận lợi khi phát triển firmware. Bảng 3.3 dưới đây đối chiếu trực tiếp các điểm khác biệt này.

Bảng 3.3: Đối chiếu các phương án LTE/GNSS theo tác động tích hợp

Yêu cầu tích hợp	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T
Số khối phần cứng rời	2 khối rời: A7670C (LTE) + NEO-M8N (GNSS) [58], [62]	2 khối rời: EC200U-CN (LTE) + NEO-M8N (GNSS) [59], [61]	1 khối tích hợp LTE + GNSS (SIM7600CE-T) [60]

Bảng 3.3: Đối chiếu các phương án LTE/GNSS theo tác động tích hợp (tiếp theo)

Yêu cầu tích hợp	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T
LTE category / tốc độ	Cat-1, tối đa 10 Mbps DL / 5 Mbps UL [58]	Cat-1, tối đa 10 Mbps DL / 5 Mbps UL [59], [61]	Cat-4, tối đa 150 Mbps DL / 50 Mbps UL [60]
GNSS tích hợp trong modem	Không tích hợp GNSS trên A7670C, cần GNSS ngoài [58], [62]	GNSS tùy chọn theo variant/cấu hình dòng EC200U [59], [61]	Có GNSS tích hợp trong module [60]
Điện áp cấp nguồn modem	3.4–4.2 V (typ. 3.8 V) [58]	3.3–4.3 V (typ. 3.8 V) [61]	3.4–4.2 V (typ. 3.8 V) [60]
Đường dữ liệu vị trí trong kiến trúc đề tài	NMEA/UBX từ NEO-M8N qua UART riêng [62], [63]	NMEA/UBX từ NEO-M8N qua UART riêng [62], [63]	GNSS AT tích hợp trong cùng modem [60]
Bật/tắt LTE và GNSS độc lập	Có, do tách 2 module/2 rail nguồn	Có, nếu triển khai tách tương tự A7670C+NEO-M8N	Không tách hoàn toàn do GNSS đi cùng modem
Tác động khi reset modem LTE	Có thể reset modem và giữ logic GNSS riêng	Tương tự phương án tách nếu GNSS dùng module riêng	Reset modem thường kéo theo gián đoạn GNSS
Mức phụ thuộc giữa các chức năng firmware	Thấp, tách rõ modem_lte và gnss	Thấp, nhưng phải đổi tập lệnh modem	Cao hơn do LTE/GNSS chung khối tích hợp
Tác động tới BOM phần cứng thực tế	Bộ modem + GNSS đang chốt 330.000–500.000 VND (BOM dự án)	Chưa phải cấu hình BOM mục tiêu của phương án đề xuất	Giảm số khối phần cứng rời, thuận lợi cho đi dây và lắp ráp

c) Kết luận lựa chọn SIMCom SIM7600CE-T được lựa chọn làm kiến trúc truyền thông chính vì đây là phương án giúp thiết kế tổng thể gọn hơn và dễ triển khai hơn, dù không phải lúc nào cũng linh hoạt nhất ở từng thành phần riêng lẻ. Các lý do cụ thể gồm:

- Tích hợp LTE và GNSS trong cùng một module giúp giảm số lượng phần cứng rời và rút gọn đường kết nối UART.
- Giữ nguyên sơ đồ chân đang triển khai (UART1 trên GPIO16/17, PWR-KEY trên

GPIO26), không cần thay đổi pin mapping phần cứng.

- Luồng AT được chuẩn hóa với Auto mode (AT+CNMP=2), kiểm tra AT+CEREG? trước AT+CGACT=1, 1, và APN mặc định internet, phù hợp với firmware hiện tại.
- GNSS tích hợp được điều khiển trực tiếp bằng AT+CGNSPWR và đọc dữ liệu qua AT+CGNSINF, thống nhất với module firmware đang vận hành.
- Phù hợp với phương án BOM phần cứng đang áp dụng cho bo mạch đã chế tạo, thuận lợi cho triển khai và đối chiếu tài liệu kỹ thuật.

Bảng 3.4: Thông số kỹ thuật phương án SIM7600CE-T

Thông số	Giá trị
Kiến trúc	SIM7600CE-T tích hợp LTE Cat-4 và GNSS trong cùng module [60]
LTE category / tốc độ	Cat-4, tối đa 150 Mbps downlink / 50 Mbps uplink [60]
GNSS	GNSS tích hợp (GPS/GLONASS/BeiDou/Galileo), điều khiển bằng AT+CGNSPWR, đọc dữ liệu bằng AT+CGNSINF [60]
GNSS tích hợp trong modem	Có [60]
Giao tiếp với MCU	1 UART (GPIO16/17) cho toàn bộ AT LTE/GNSS + GPIO26 điều khiển PWR-KEY [60]
Giao thức dữ liệu	LTE dùng AT command cho PDP/MQTT; GNSS lấy dữ liệu qua AT+CGNSINF và có thể stream NMEA bằng AT+CGNSTST khi cần [60]
Điện áp modem	3.4–4.2 V, điển hình 3.8 V [60]
Luồng attach LTE	AT+CNMP=2 (auto mode), cấu hình AT+CGDCONT=1, "IP", "internet", kiểm tra AT+CEREG? trước AT+CGACT=1, 1
Đồng bộ GNSS	Bật/tắt GNSS bằng AT+CGNSPWR=1/0, đọc dữ liệu bằng AT+CGNSINF
Điện áp cấp nguồn	3.4–4.2 V, điển hình 3.8 V [60]
Ý nghĩa tích hợp	Giảm số linh kiện rời, đơn giản hóa đi dây và đồng bộ với firmware thực tế
Ghi chú	APN mặc định sử dụng trong hệ thống là internet; pin mapping phần cứng giữ nguyên

Đến đây, hai khối điều phối và liên lạc của thiết bị theo dõi đã được chốt. Phần còn lại

của thiết kế phần cứng là trả lời ba câu hỏi thực dụng hơn: lấy dữ liệu xe bằng cách nào, canh chuyển động khi xe đỗ ra sao và nuôi toàn bộ hệ thống bằng nguồn nào. Vì vậy, mục 3.1.2 chuyển sang cách ghép các lựa chọn đó thành phần cứng hoàn chỉnh.

3.1.2. Giải pháp phần cứng

Trên cơ sở các lựa chọn phần cứng đã phân tích ở mục 3.1.1, phần này đi vào cách nhóm hiện thực chúng thành kiến trúc thiết bị theo dõi hoàn chỉnh, từ kết nối OBD2 và IMU đến khối nguồn và BOM tham chiếu.

3.1.2.1. Thiết kế khối thu thập dữ liệu OBD2

a) Lựa chọn phương pháp kết nối OBD2 Thu thập dữ liệu từ ECU, tức hộp điều khiển điện tử của xe, qua cổng OBD2 (On-Board Diagnostics II) là một yêu cầu cốt lõi của thiết kế thiết bị theo dõi vì đây là nguồn thông tin giúp hệ thống đi xa hơn một thiết bị định vị thông thường. Có hai hướng tiếp cận chính: sử dụng adapter OBD2 có dây, tức bộ chuyển đổi kiểu ELM327 nối qua UART; hoặc adapter OBD2 không dây, tức trao đổi qua BLE. Nhóm lựa chọn kết nối không dây qua BLE vì cách làm này phù hợp hơn với cách bố trí thiết bị thực tế trên xe và với tài nguyên phần cứng của thiết bị theo dõi:

Bảng 3.5: So sánh phương pháp kết nối OBD2 theo thông số triển khai

Tiêu chí kỹ thuật	OBD2 có dây (ELM327 UART)	OBD2 không dây (BLE ELM327 / vgate iCar Pro)
Kết nối vật lý thiết bị theo dõi ↔ OBD2	Cần dây kéo cố định từ cổng OBD2 đến thiết bị theo dõi	Không cần dây giữa thiết bị theo dõi và cổng OBD2
Giao thức tới MCU	UART TTL	BLE 4.0 GATT; vgate iCar Pro là bản Bluetooth 4.0 BLE [29], [64]
Tài nguyên MCU tiêu tốn	Chiếm thêm 1 UART phần cứng trong lúc kết nối	Tận dụng BLE tích hợp của ESP32-S3, giữ UART cho modem/GNSS/debug
Giao thức OBD-II hỗ trợ	Phụ thuộc adapter ELM327 cụ thể	Vgate công bố hỗ trợ J1850 PWM/VPW, ISO9141-2, ISO14230-4, ISO15765-4 CAN, SAE J1939 CAN [64]
Hành vi sleep của adapter	Phụ thuộc adapter có dây cụ thể	Vendor công bố tự sleep sau khoảng 30 phút khi xe tắt máy [64]

Bảng 3.5: So sánh phương pháp kết nối OBD2 theo thông số triển khai (tiếp theo)

Tiêu chí kỹ thuật	OBD2 có dây (ELM327 UART)	OBD2 không dây (BLE ELM327 / vgate iCar Pro)
Hành vi wake	Phụ thuộc adapter và đường cấp nguồn	Vendor quảng bá tự khởi động khi ignition ON; cần hiểu là hành vi phụ thuộc xe [64]
Dòng tiêu thụ adapter	Phụ thuộc adapter; không dùng 1 số cứng nếu không có datasheet đúng mẫu	Trang hãng được kiểm tra không công bố dòng active/standby, nên không chốt số mA trong luận văn [64]
Hành vi khi MCU deep sleep	UART không duy trì truyền dữ liệu khi MCU ngủ sâu	BLE link cũng không duy trì qua deep sleep; cần kết nối lại sau khi đánh thức
Tác động lắp đặt thực địa	Tracker thường bị ràng buộc gần cổng OBD2	Tracker có thể đặt lệch vị trí OBD2 trong cabin nếu vẫn trong vùng BLE
Bảo trì/thay adapter giữa các xe	Cần tháo dây liên quan đến thiết bị theo dõi	Chỉ cần rút/cắm adapter và cấu hình lại kết nối nếu cần

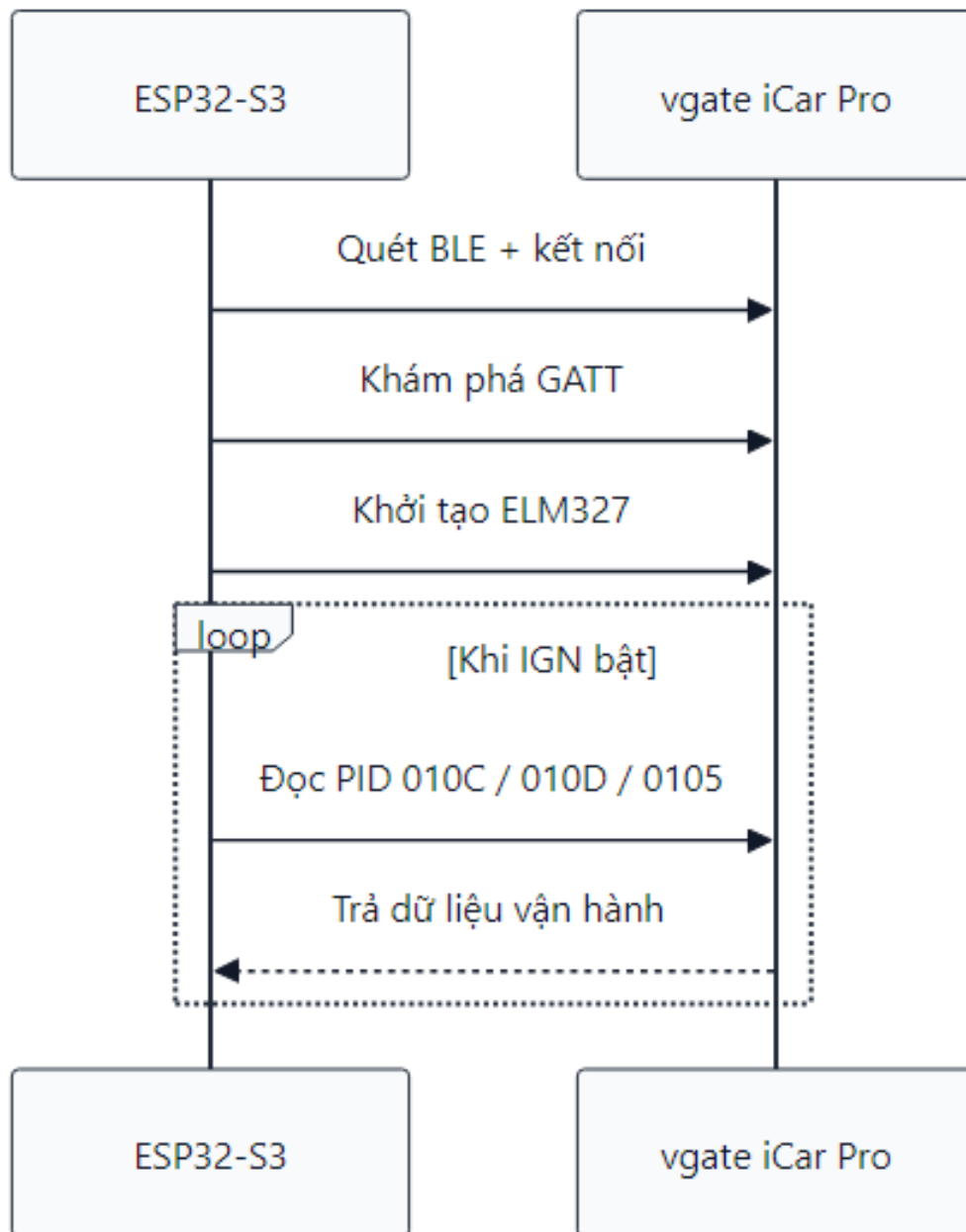
Kết luận: Phương pháp BLE được chọn vì phù hợp hơn với kiến trúc hiện tại của thiết bị theo dõi: tận dụng BLE tích hợp của ESP32-S3, không chiếm thêm UART phần cứng, giảm ràng buộc đi dây, đồng thời vgate iCar Pro đã có công bố rõ về BLE 4.0, auto-sleep và danh sách giao thức OBD-II. Với các thông số hãng chưa công bố (như dòng active/standby), báo cáo chỉ ghi nhận là chưa có số liệu chính thức thay vì đưa ra giá trị ước lượng.

b) Lựa chọn adapter OBD2 BLE: vgate iCar Pro vgate iCar Pro là adapter OBD2 sử dụng Bluetooth Low Energy (BLE) 4.0, tương thích với BLE 5.0 của ESP32-S3. Adapter này được chọn dựa trên các tiêu chí sau:

- **Tương thích giao thức:** Hỗ trợ tập lệnh ELM327 qua BLE GATT characteristics, cho phép đọc đa dạng dữ liệu từ ECU bao gồm trạng thái động cơ (IGN), số vòng quay (RPM), tốc độ xe, mức nhiên liệu, nhiệt độ động cơ, và mã lỗi chẩn đoán (DTC) [7].
- **Khả năng kết nối:** BLE 4.0 của adapter tương thích ngược với BLE 5.0 của ESP32-S3 [19], [64]. Vendor cũng công bố cơ chế tự sleep sau khoảng 30 phút khi xe tắt máy và tự khởi động lại theo ignition, phù hợp cho kịch bản thiết bị theo dõi trên xe [64].
- **Tiêu thụ năng lượng:** Ở thời điểm rà soát nguồn, trang hãng được kiểm tra không

công bố dòng active/standby chính thức cho vgate iCar Pro BLE [64]. Vì vậy báo cáo chỉ ghi nhận đây là thông số chưa được công bố đầy đủ, không đưa ra giá trị mA cố định khi chưa có datasheet chính thức đúng mẫu.

- **Phổ biến và giá hợp lý:** Khảo sát thị trường Việt Nam ngày 12/04/2026 cho thấy các mức giá tham khảo của vgate iCar Pro BLE nằm quanh khoảng 367.000–609.000 VND tùy kênh bán lẻ và phiên bản.



Hình 3.2: Sơ đồ kết nối BLE giữa ESP32-S3 và vgate iCar Pro

c) Chiến lược kết nối theo chế độ hoạt động Một ràng buộc thiết kế quan trọng là ESP32-S3 không thể duy trì kết nối BLE trong trạng thái deep sleep. Vì vậy, kết nối OBD2 không được xem là một kênh ”luôn bật”, mà chỉ hoạt động khi nó thực sự tạo ra giá trị dữ liệu cho từng trạng thái vận hành của xe:

Bảng 3.6: Chiến lược kết nối BLE OBD2 theo chế độ hoạt động

Chế độ	Cần BLE OBD2?	Lý do
Lái xe (IGN ON)	Có	Đọc dữ liệu xe (RPM, tốc độ, nhiên liệu) liên tục
Đỗ xe (IGN OFF)	Không	Không cần dữ liệu OBD2, chỉ cần IMU phát hiện chuyển động
Cảnh báo	Tùy chọn	Có thể kết nối để xác nhận IGN, hoặc chỉ dùng IMU + GPS

Khi xe chạy (IGN ON), ESP32-S3 duy trì kết nối BLE liên tục với adapter để đọc dữ liệu OBD2 định kỳ mỗi 5–30 giây. Khi xe đỗ (IGN OFF), kết nối BLE được ngắt trước khi ESP32 vào deep sleep, vì ở thời điểm này mục tiêu ưu tiên không còn là lấy dữ liệu động cơ, mà là giữ mức tiêu thụ thấp nhất trong khi vẫn duy trì khả năng phát hiện xâm nhập. IMU LIS3DH đảm nhiệm việc giám sát đó độc lập và chỉ đánh thức hệ thống khi có tín hiệu đáng ngờ. Cách tổ chức này giúp giảm đáng kể năng lượng tiêu thụ ở pha đỗ xe mà không làm mất đi chức năng cảnh báo cốt lõi.

Trong trường hợp adapter OBD2 không kết nối được (timeout 10 giây, retry 2–3 lần), hệ thống tự động chuyển sang phương pháp dự phòng (fallback) là đo điện áp ắc quy qua ADC để phát hiện trạng thái IGN. Phương pháp này kém chính xác hơn nhưng đảm bảo hệ thống vẫn hoạt động bình thường.

Như vậy, OBD2 giải quyết tốt bài toán “xe đang chạy ra sao” nhưng chưa đủ cho trạng thái đỗ xe, khi mục tiêu chính là phát hiện tác động bất thường với mức tiêu thụ điện rất thấp. Vì vậy, phần tiếp theo chuyển sang khối IMU.

3.1.2.2. Thiết kế khối cảm biến chuyển động IMU

a) Vai trò của IMU trong hệ thống Cảm biến đo quán tính (IMU - Inertial Measurement Unit) đóng vai trò then chốt trong việc phát hiện chuyển động của xe khi đang đỗ, cho phép hệ thống phát hiện các tình huống bất thường như rung xe (cố gắng mở cửa), kéo xe, hoặc cầu xe. Đặc biệt, IMU cho phép ESP32-S3 duy trì light sleep cho nhánh canh rung và đánh thức gần như tức thời khi có chuyển động thực sự, trong khi heartbeat

định kỳ vẫn quay về deep sleep theo timer để tiết kiệm năng lượng.

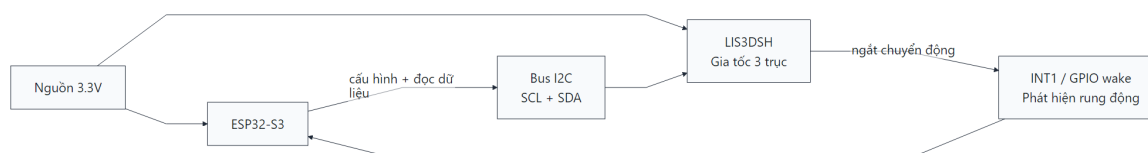
b) Lựa chọn cảm biến: LIS3DSH LIS3DSH của STMicroelectronics là cảm biến gia tốc 3 trục (3-axis accelerometer) low-power được lựa chọn cho hệ thống. Các đặc tính kỹ thuật chính được tóm tắt trong bảng sau:

Bảng 3.7: Thông số kỹ thuật cảm biến LIS3DSH

Thông số	Giá trị
Loại	3-axis Digital Accelerometer
Điện áp hoạt động	1.8–3.6 V
Giao tiếp	I2C (400 kHz) hoặc SPI (10 MHz)
Phạm vi đo	+/-2g, +/-4g, +/-8g, +/-16g (có thể điều chỉnh)
Độ phân giải	16-bit
Tiêu thụ (Active)	10–50 μ A
Tiêu thụ (Low-power)	2–5 μ A
Wake-up interrupt	Có (ngưỡng có thể cấu hình)
Package	LGA-16 (3x3x1 mm)

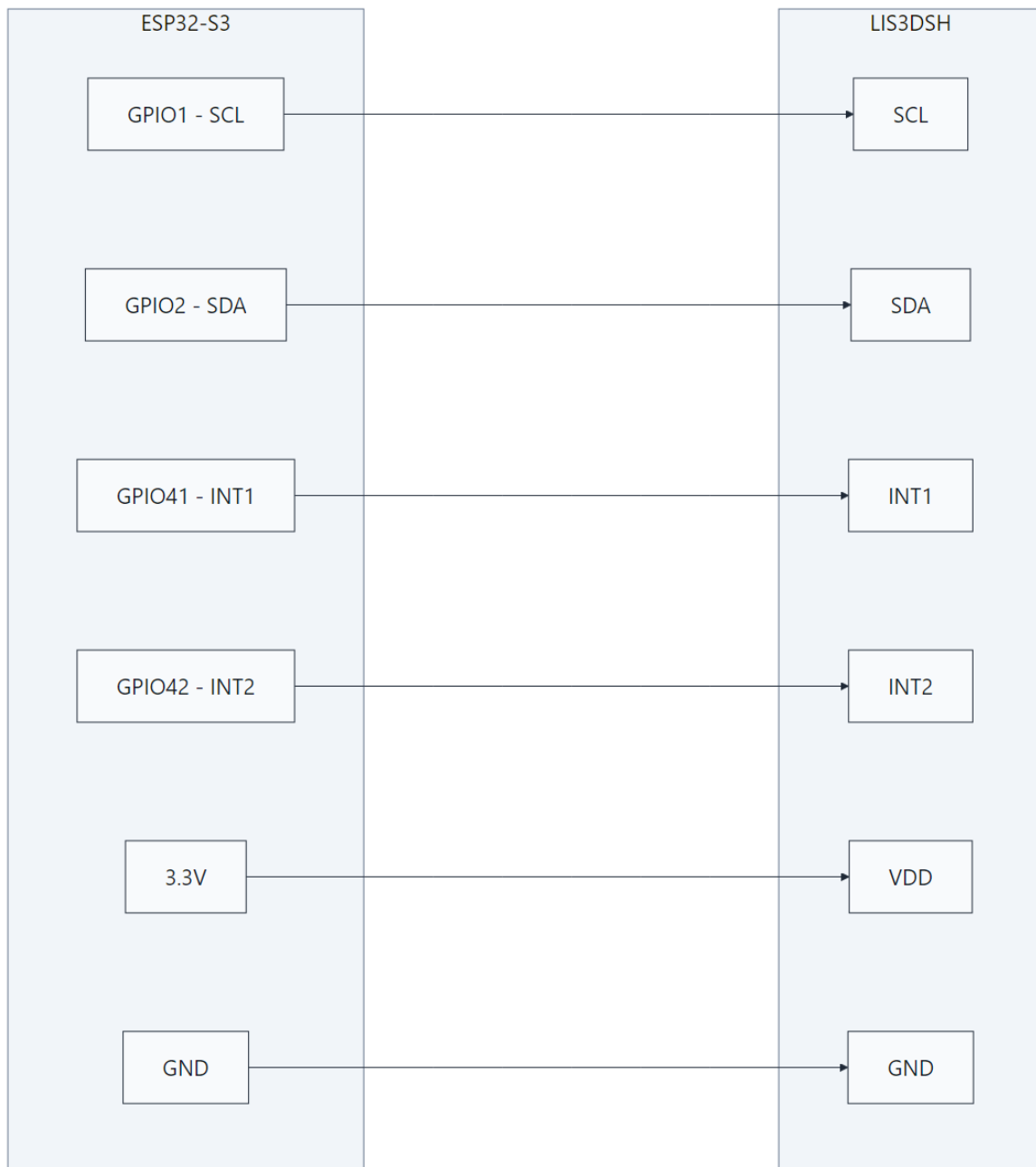
c) Nguyên lý hoạt động trong hệ thống LIS3DSH được cấu hình ở chế độ low-power với tần suất lấy mẫu 10 Hz và chỉ bật chức năng motion detection. Khi phát hiện gia tốc vượt ngưỡng 0.2g trên bất kỳ trục nào (X, Y, Z), cảm biến phát tín hiệu interrupt đến chân INT1 nối với GPIO41 của ESP32-S3. Vì GPIO41 không thuộc nhóm RTC GPIO, firmware dùng GPIO wake trong light sleep cho cảnh báo rung, còn heartbeat định kỳ vẫn dùng timer wake [9].

Ngưỡng gia tốc 0.2g được chọn để cân bằng giữa độ nhạy và khả năng chống báo giả. Giá trị này đủ lớn để loại bỏ rung nhẹ từ môi trường (gió, xe cộ đi ngang) nhưng đủ nhỏ để phát hiện các chuyển động thực sự như rung của xe hoặc di chuyển.



Hình 3.3: Sơ đồ kết nối LIS3DSH với ESP32-S3 qua I2C

Cấu hình I2C sử dụng địa chỉ 0x18, tốc độ 400 kHz, với hai điện trở kéo lên 10 k Ω trên các net IMU-SCL và IMU-SDA. Ở runtime hiện tại, bus I2C được ánh xạ về GPIO1 (SCL) và GPIO2 (SDA), chân INT1 đi vào GPIO41, còn INT2 đi vào GPIO42. Cơ chế wake do IMU kích hoạt dùng *light sleep* qua GPIO wake thay vì deep sleep EXT0/EXT1.



Hình 3.3a: Sơ đồ chi tiết chân kết nối LIS3DSH với ESP32-S3

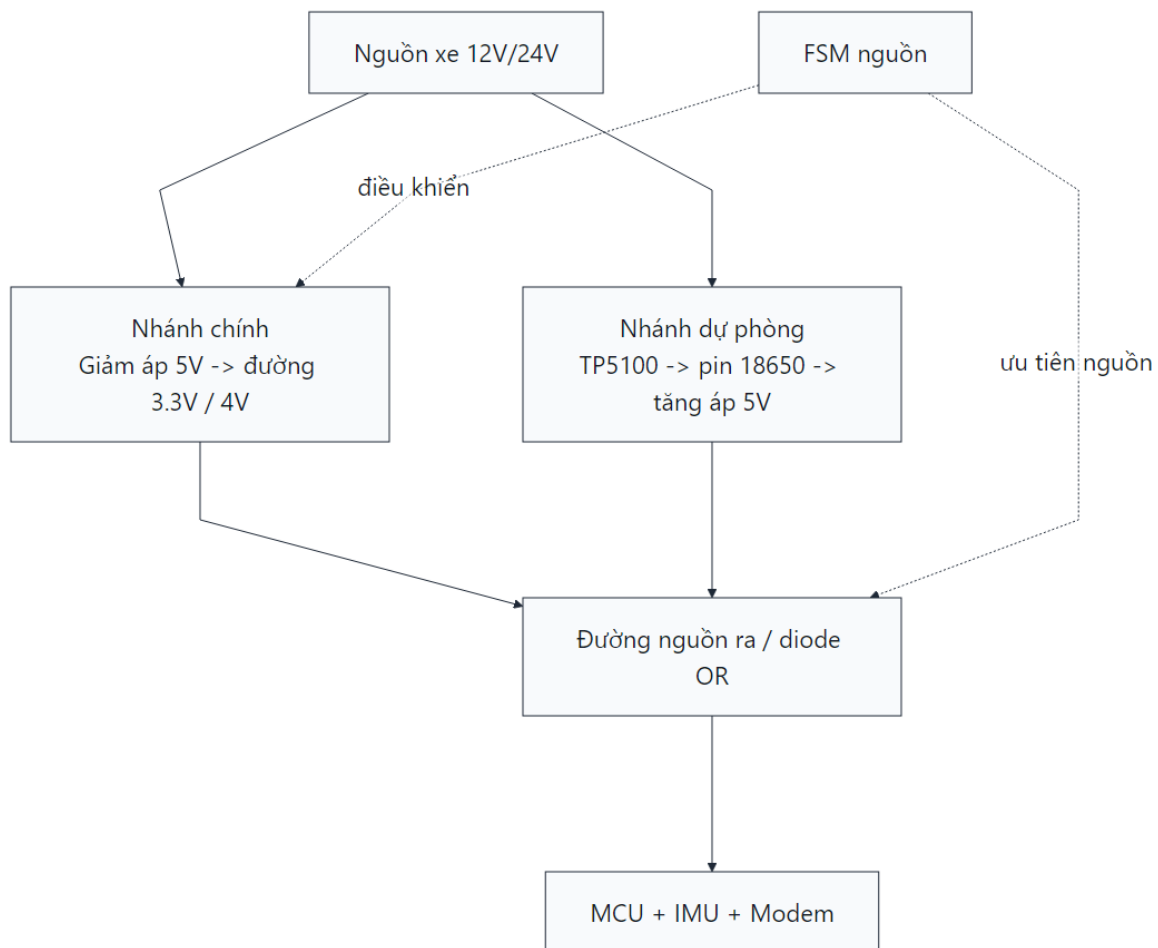
d) Ưu điểm của giải pháp

- **Tiết kiệm năng lượng tốt nhất:** Dòng tiêu thụ chỉ 2–5 μA ở chế độ low-power, trong khi vẫn duy trì khả năng phát hiện chuyển động. ESP32-S3 không cần chạy liên tục để kiểm tra cảm biến.
- **Phát hiện chuyển động độc lập:** LIS3DSH tự xử lý việc phát hiện chuyển động bằng phần cứng, chỉ gửi interrupt khi có sự kiện — không phụ thuộc vào CPU của ESP32.
- **Hệ sinh thái phát triển phong phú:** Nhiều thư viện sẵn có cho Arduino và ESP-

IDF; IC LIS3DSH hoặc cảm biến tương đương đều dễ tích hợp vào sơ đồ phần cứng của thiết bị.

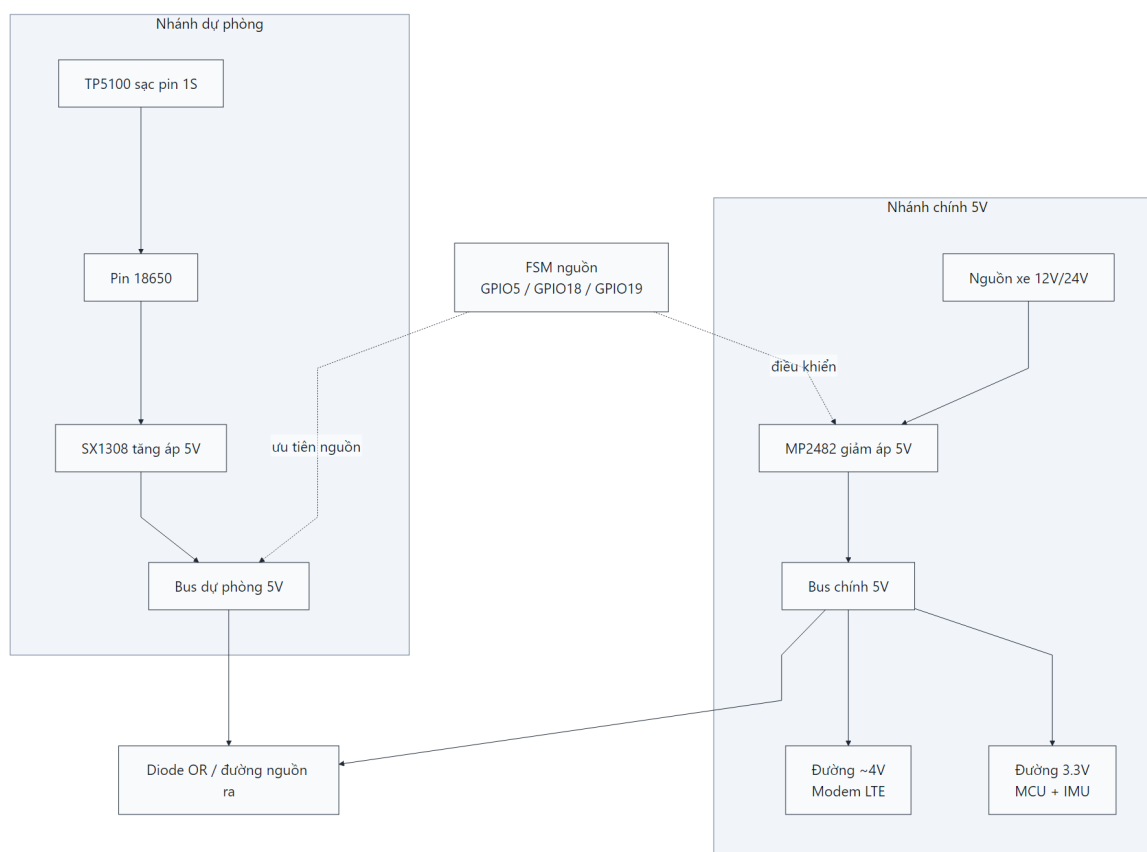
Nếu OBD2 cung cấp bức tranh vận hành khi xe đang nổ máy và IMU đảm nhận việc canh xe khi đỗ, thì khối còn lại phải bảo đảm hai cơ chế đó luôn có điện đúng lúc. Phần tiếp theo vì thế đi vào hệ thống quản lý nguồn.

3.1.2.3. Thiết kế hệ thống quản lý nguồn Hệ thống quản lý nguồn là khối phức tạp nhất trong thiết kế phần cứng, vì nó phải đồng thời giải quyết ba bài toán vốn dễ xung đột với nhau: tạo đúng các mức điện áp cho từng tải, duy trì đường nguồn dự phòng khi cần và bảo vệ ắc quy xe khỏi bị rút cạn quá mức. Vì vậy, khối này được tách thành năm nhánh chức năng để người đọc dễ theo dõi vai trò của từng đường nguồn thay vì chỉ nhìn một sơ đồ điện phức tạp.



Hình 3.4: Sơ đồ khối hệ thống quản lý nguồn

a) LDO 3.3V cho ESP32-S3 (AP2112-3.3) Nhánh 3.3V dùng AP2112-3.3 như một LDO để hạ từ **bus 5V chính** xuống 3.3V, cấp cho ESP32-S3 và các tải logic 3.3V. Cách tổ chức này phù hợp với kiến trúc nguồn thực tế của bo mạch: MP2482 đảm nhiệm bước



Hình 3.4a: Kiến trúc nguồn và phân phối điện áp trong hệ thống

hạ áp đầu tiên từ 12–24V xuống 5V, còn AP2112-3.3 đảm nhiệm bước ổn áp cuối cho miền logic.

Bảng 3.8: Thông số khối LDO 3.3V

Khối	IC	Input	Output	Tải
LDO 3.3V	AP2112-3.3	5V	3.3V	ESP32-S3

b) Buck 5V bus chính (MP2482) Nhánh buck 5V dùng **MP2482** để tạo **bus 5V chính** từ nguồn 12–24V của xe. Bus 5V này là nguồn trung gian cho các khối phụ trợ và đầu vào mạch sạc pin.

Bảng 3.9: Thông số khối Buck 5V

Khối	IC	Input	Output	Tải
Buck 5V	MP2482	12–24V	5V	Bus 5V chính

c) Buck 3.8V/4V cho modem (TPS54231) Do modem SIM7600CE-T làm việc trên miền nguồn thấp áp, hệ thống bố trí nhánh buck riêng dùng **TPS54231** để hạ 12–24V xuống khoảng **4V** cấp cho modem.

Bảng 3.10: Thông số khối Buck 3.8V/4V cho modem

Khối	IC	Input	Output	Tải
Buck 3.8V	TPS54231	12–24V	4V	SIM7600CE-T

d) Tăng áp 5V từ pin dự phòng (SX1308) và mạch chuyển nguồn Khi ắc quy xe yếu, nguồn dự phòng lấy từ pin 18650 1S (danh định 3.7V). Khối **SX1308** tăng áp lên 5V để duy trì cấp nguồn cho hệ thống.

Bảng 3.11: Thông số khối Boost 5V dự phòng

Khối	IC	Input	Output	Tải
Boost 5V	SX1308	Battery 3.7V	5V	Nguồn dự phòng từ pin

Mạch chuyển nguồn khi vận hành sử dụng **diode OR** giữa nhánh 5V chính (MP2482) và nhánh 5V dự phòng (SX1308), kết hợp điều khiển GPIO để bật/tắt các nhánh liên quan. Mục tiêu là chuyển nguồn mượt, không reset hệ thống khi đổi nguồn.

e) Giám sát điện áp và LVD LVD dùng ngưỡng profile kép theo firmware:

- **Profile 12V:** $Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$
- **Profile 24V:** $Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$

Bảng 3.12: Bảng trạng thái chuyển nguồn và cảnh báo

Chế độ	IGN	U_{batt}	Nguồn thiết bị	Sạc pin	Cảnh báo
Xe chạy	ON	Profile 12V: $U_{batt} \geq 13.0\text{ V}$; Profile 24V: $U_{batt} \geq 26.0\text{ V}$	Ắc quy	Có	Không
Xe đỗ	OFF	Profile 12V: $U_{batt} > 12.0\text{ V}$; Profile 24V: $U_{batt} > 24.0\text{ V}$	Ắc quy	Không	Không
Ắc quy yếu	OFF	Profile 12V: $U_{batt} \leq 12.0\text{ V}$; Profile 24V: $U_{batt} \leq 24.0\text{ V}$	Pin 18650 1S	Không	Cảnh báo chuyển nguồn

Bảng 3.12: Bảng trạng thái chuyển nguồn và cảnh báo (tiếp theo)

Chế độ	IGN	U_{batt}	Nguồn thiết bị	Sạc pin	Cảnh báo
Ắc quy phục hồi	OFF	Profile 12V: $U_{batt} \geq 12.2\text{ V}$; Profile 24V: $U_{batt} \geq 24.4\text{ V}$	Ắc quy	Không	Cảnh báo phục hồi

Ngoài kênh ADC, tín hiệu trạng thái LVD từ khối LVD phần cứng được đưa về **GPIO19 (LVD_STATUS)** để giám sát nhanh. Quy ước vận hành: **GPIO19 HIGH = low-voltage**, **GPIO19 LOW = bình thường**.

f) Mạch sạc pin 1S (TP5100) Khối sạc dùng **TP5100** ở cấu hình **1 cell**, nhận **5V** từ nhánh **MP2482** và sạc pin 18650 1S tới mức **4.2V/1S**. So với phương án TP4056 trước đây, TP5100 phù hợp hơn với nhánh nguồn dự phòng vì đây là bộ sạc chuyên mạch, giảm tổn hao nhiệt khi nạp.

Bảng 3.13: Thông số khối sạc pin

Khối	IC	Input	Output	Tải
Sạc pin	TP5100	5V từ MP2482	4.2V	Pin 18650 1S

Dòng sạc TP5100 được cấu hình bằng điện trở thiết lập dòng sạc và còn phụ thuộc điều kiện nhiệt của mạch, vì vậy không dùng một giá trị dòng cố định cho mọi điều kiện vận hành.

g) Pin dự phòng 18650 1S Li-ion Pin 18650 1S Li-ion được chọn làm nguồn dự phòng với cấu hình 1 cell đơn giản, kết hợp khối sạc TP5100, khối boost SX1308 và mạch bảo vệ pin 1S trong kiến trúc nguồn của hệ thống.

Bảng 3.14: Thông số kỹ thuật pin dự phòng 18650 1S

Thông số	Giá trị
Loại	Li-ion 18650 1S
Dung lượng	3500 mAh @ 3.7V
Năng lượng	12.95 Wh
Điện áp	3.0–4.2V (nominal 3.7V)
Số chu kỳ	500–1000 chu kỳ (80% capacity)

Tính toán thời gian hoạt động từ pin dự phòng:

Các giá trị dưới đây là ước tính theo cùng giả định tổn hao của phương án gốc, với dung lượng hiệu quả xấp xỉ 3150 mAh (tương đương 90% dung lượng danh định), không phải số đo thực nghiệm mới trên cell 18650.

Chế độ heartbeat (đỗ xe, chu kỳ 15 phút):

- Dòng trung bình: $I_{\text{avg}} = \frac{250 \text{ mA} \times 15 \text{ s} + 3 \text{ mA} \times 885 \text{ s}}{900 \text{ s}} = 7.1 \text{ mA}$
- Dung lượng hiệu quả (sau tổn hao): 3150 mAh
- Thời gian hoạt động: $T = \frac{3150}{7.1} = 444 \text{ giờ} \approx 18.5 \text{ ngày}$

Chế độ heartbeat (đỗ xe, chu kỳ 30 phút):

- Dòng trung bình: $\approx 5 \text{ mA}$
- Thời gian hoạt động: $T = \frac{3150}{5} = 630 \text{ giờ} \approx 26 \text{ ngày}$

Chế độ cảnh báo (track liên tục):

- Dòng trung bình: $\approx 250 \text{ mA}$
- Thời gian hoạt động: $T = \frac{3150}{250} = 12.6 \text{ giờ}$

Giá trị 12.6 giờ ở đây là phép tính lý tưởng theo tải giả định 250 mA tại mức cell; khi quy đổi sang tải hệ thống thực tế có hao hụt boost và các pha truyền dữ liệu đỉnh cao hơn, thời lượng tracking liên tục trong Chương 4 được lấy theo khoảng bảo thủ hơn là 2.6–3.0 giờ.

Kết quả quy đổi cho thấy bộ pin tham chiếu 18650 1S vẫn đủ khả năng duy trì hoạt động thiết bị trong nhiều ngày ở chế độ heartbeat và trên 12 giờ ở chế độ cảnh báo liên tục theo mô hình lý tưởng; trong vận hành toàn hệ thống, các giá trị bảo thủ hơn ở Chương 4 phù hợp hơn cho mục tiêu duy trì giám sát tạm thời khi ắc quy xe yếu, nhưng vẫn cần được kiểm thử thực nghiệm lại nếu chuyển sang mẫu pin mới trong pha chế tạo tiếp theo [14].

3.1.2.4. Bảng tổng hợp linh kiện (Bill of Materials)

*Bảng 3.14A: Bảng tổng hợp linh kiện hệ thống thiết bị theo dõi
(BOM)*

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
1	ESP32-S3-WROOM-1 (N16R8)	Cái	1	95.000	MCU trung tâm của bo mạch
2	IC cảm biến LIS3DH	Cái	1	40.000	Cảm biến gia tốc 3 trục
3	vgate iCar Pro (OBD2 BLE)	Cái	1	367.000	BLE 4.0, ELM327 compatible
4	SIMCom SIM7600CE-T	Bá»™	1	650.000	Modem LTE + GNSS tích hợp
5	Pin 18650 1S Li-ion 3500mAh	Cái	1	79.000	Loại có protection board
6	IC sạc TP5100 + mạch phụ trợ	Cái	1	25.000	Input 5V, output 4.2V, dòng theo điện trở thiết lập
7	Mạch LDO AP2112-3.3	Cái	1	5.000	5V -> 3.3V cấp ESP32-S3
8	Mạch buck MP2482 5V	Cái	1	17.000	Bus 5V chính
9	Mạch buck TPS54231 4V	Cái	1	9.000	12–24V -> 4V cấp modem SIM7600CE-T
10	Mạch boost SX1308 5V dự phòng	Cái	1	12.000	Nguồn dự phòng từ pin 18650 1S
11	Khối giám sát LVD + ADC	Cái	1	8.000	Giám sát LVD (GPIO19)
12	Diode Schottky (SS54/SS34)	Cái	2	2.500	Diode-OR power path
13	Mạch bảo vệ pin 1S (BMS)	Cái	1	12.000	Bảo vệ pin 18650 1S
14	Điện trở (10kOhm, 2.2kOhm)	Gói	1	5.000	Voltage divider, pull-up
15	Tụ điện (100uF, 220uF)	Gói	1	5.000	Lọc nhiễu, decoupling

Bảng 3.14A: Bảng tổng hợp linh kiện hệ thống thiết bị theo dõi (BOM) (tiếp theo)

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
16	Connector, header pin	Gói	1	55.000	Header, jack OBD2 và đầu nối nguồn
17	PCB 2 lớp (50x50 mm)	Cái	1	20.000	Bo mạch theo thiết kế của đề tài
18	Vỏ bảo vệ (tùy chọn)	Cái	1	25.000	Nhựa hoặc kim loại
19	Dây nối, cáp, phụ kiện	-	-	60.000	Dây điện, anten 4G/GNSS, khay SIM
20	Linh kiện phụ trợ khác	-	-	20.000	Fuse, switch, LED

Bảng 3.15: Tổng hợp chi phí theo nhóm

Nhóm	Hạng mục	Chi phí (VND)
A	Thành phần chính (MCU, cảm biến, modem, OBD2, pin)	1.231.000
B	Quản lý nguồn (AP2112-3.3, MP2482, TPS54231, SX1308, TP5100, diode OR, khối LVD, pin 1S)	93.000
C	Kết nối, RF và linh kiện phụ trợ	145.000
D	PCB và vỏ (tùy chọn)	45.000
	Tổng cộng	1.514.000

Theo mặt bằng giá tham chiếu thị trường Việt Nam được rà soát ngày 12/04/2026, tổng chi phí vật tư cho một bộ thiết bị theo dõi hoàn chỉnh vào khoảng **1.514.000 VND**. Mức này vẫn nằm dưới ngưỡng chi phí mục tiêu của đồ án, đồng thời phản ánh thực tế hơn so với mặt bằng giá linh kiện ở thời điểm đầu năm.

3.2. Phân tích và đề xuất giải pháp Firmware – Firmware analysis and solution

3.2.1. Phân tích và lựa chọn giải pháp Firmware

3.2.1.1. Đặt vấn đề cho giải pháp Firmware Nếu xem phần cứng là “cơ thể” của thiết bị, thì firmware là lớp biến tập linh kiện đó thành một hệ thống biết *phản ứng đúng lúc*. Người dùng không trực tiếp nhìn thấy firmware, nhưng việc vị trí có cập nhật đều, cảnh

báo có đến kịp hay không, hay ắc quy xe có bị ảnh hưởng hay không đều do lớp này quyết định. Vì nằm ở điểm giao nhau giữa phần cứng, truyền thông và yêu cầu nghiệp vụ, bài toán firmware không dừng ở chuyện “cho thiết bị chạy được”, mà phải bảo đảm thiết bị chạy đúng, ổn định và tiết kiệm năng lượng trong môi trường thực tế.

- **Đa nhiệm thời gian thực:** thu thập cảm biến, điều khiển modem, truyền dữ liệu và xử lý cảnh báo phải chạy song song, độ trễ thấp.
- **Quản lý năng lượng nghiêm ngặt:** chuyển trạng thái linh hoạt giữa active/sleep/deep sleep theo điều kiện vận hành xe.
- **Giao tiếp đa giao thức:** BLE (OBD2), UART (modem), MQTT/HTTP (máy chủ), I2C/ADC/GPIO (ngoại vi phần cứng).
- **Độ bền vận hành cao:** có cơ chế retry, fallback và phục hồi sau mất kết nối mạng hoặc lỗi ngoại vi.

3.2.1.2. So sánh các phương án nền tảng Firmware

Bảng 3.5A: So sánh các phương án nền tảng firmware

Phương án	Mô tả	Ưu điểm	Hạn chế	Mức phù hợp
PA-FW1: Arduino Core + Superloop	Vòng lặp chính tuần tự, xử lý tác vụ theo kiểu hỏi lần lượt từng phần trong vòng lặp	Dễ bắt đầu, ít cấu hình	Khó mở rộng đa nhiệm thực sự, khó tối ưu deep sleep phức tạp, quản lý lỗi hạn chế	Trung bình
PA-FW2: ESP-IDF + FreeRTOS (Đã chọn)	Kiến trúc task và event, driver chính thức Espressif	Đa nhiệm tốt, hỗ trợ power management sâu, tích hợp NimBLE / modem / UART / I2C ổn định	Độ phức tạp cao hơn Arduino	Cao
PA-FW3: Zephyr RTOS trên ESP32	RTOS đa nền tảng, kiến trúc phân lớp	Tính chuẩn hóa tốt, khả năng mở rộng dài hạn	Hệ sinh thái ESP32 chuyên biệt và tài liệu thực chiến BLE-OBD2/modem ít hơn ESP-IDF	Trung bình

3.2.1.3. Chọn giải pháp Firmware Từ kết quả so sánh, đề án chọn **PA-FW2: ESP-IDF + FreeRTOS** làm nền tảng firmware chính. Ở đây, ESP-IDF có thể hiểu là bộ công

cụ chính thức để lập trình ESP32-S3; FreeRTOS là lớp điều phối giúp firmware chia việc theo task và sự kiện thay vì dồn mọi xử lý vào một vòng lặp lớn. Đây không phải là lựa chọn “mạnh nhất trên giấy”, mà là lựa chọn bám sát nhất cách thiết bị phải vận hành ngoài thực tế: nhiều ngoại vi, nhiều trạng thái, yêu cầu tiết kiệm điện và cần dễ truy vết khi có lỗi.

Nền tảng này thuyết phục hơn ở bốn điểm chính:

- **Khớp yêu cầu điều phối thời gian thực của hệ thống:** ESP-IDF + FreeRTOS cho phép tổ chức máy trạng thái trung tâm, các driver chạy không đồng thời tuyệt đối và các cơ chế khóa/chờ ở mức khối chức năng. Nói cách khác, đây là nền tảng đủ linh hoạt để firmware vừa theo dõi nhiều tín hiệu cùng lúc vừa không làm luồng xử lý chính trở nên rối và khó kiểm soát.
- **Tối ưu cho ESP32-S3:** driver và power feature chính thức giúp giảm rủi ro tích hợp phần cứng, đồng thời rút ngắn đáng kể thời gian xử lý các lỗi phát sinh trong giai đoạn chế tạo và kiểm thử.
- **Phù hợp bài toán năng lượng:** hỗ trợ deep sleep/wakeup và điều khiển ngoại vi theo trạng thái, nên phù hợp với một thiết bị phải luôn cân bằng giữa khả năng giám sát và mức tiêu thụ điện.
- **Dễ kiểm soát chất lượng vận hành:** thuận lợi xây dựng máy trạng thái, cơ chế thử lại và phương án dự phòng cho BLE/modem. Điều này đặc biệt quan trọng vì thiết bị không hoạt động trong phòng thí nghiệm lý tưởng mà trong môi trường xe có nhiễu, rung động và mất sóng bất chợt.

Phân so sánh trên trả lời vì sao ESP-IDF + FreeRTOS là nền tảng phù hợp nhất cho đề án. Từ đây, mục 3.2.2 chuyển từ câu hỏi “chọn gì” sang câu hỏi “tổ chức ra sao” trong kiến trúc firmware thực tế.

3.2.2. Giải pháp Firmware

Firmware là nơi toàn bộ quyết định vận hành của thiết bị được hiện thực hóa thành hành vi cụ thể: đọc cảm biến, đánh giá trạng thái xe, điều khiển modem, quản lý năng lượng và gửi dữ liệu lên máy chủ. Trên cơ sở phương án đã chọn ở mục 3.2.1, phần này trình bày cách nhóm biến các yêu cầu đó thành một kiến trúc firmware đủ rõ ràng để phát triển, đủ gọn để vận hành ổn định và đủ linh hoạt để mở rộng sau này.

3.2.2.1. Kiến trúc firmware và luồng hoạt động

a) Kiến trúc phân lớp (Layered Architecture) Để người đọc dễ hình dung, firmware được tách theo từng lớp trách nhiệm thay vì trình bày như một khối mã nguồn lớn. Có thể hiểu đơn giản: lớp dưới cùng “nói chuyện với phần cứng”, lớp giữa giữ trạng thái và quyết định hệ thống đang ở chế độ nào, còn lớp trên cùng đóng gói dữ liệu và giao tiếp

với hệ thống máy chủ. Cách chia này giúp trả lời rõ ba câu hỏi quan trọng: lớp nào làm việc với phần cứng, lớp nào giữ trạng thái vận hành, và lớp nào chịu trách nhiệm giao tiếp với hệ thống bên ngoài.

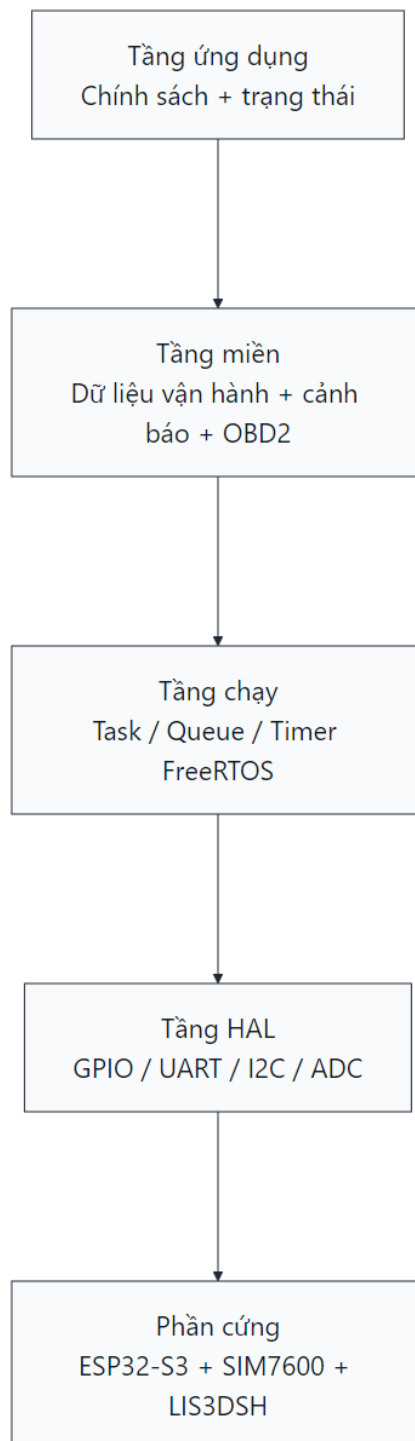
Bảng 3.2.2A: Tổng hợp thông tin về Lớp, Thành phần chính, Vai trò

Lớp	Thành phần chính	Vai trò
1	adc_reader, imu_lis3dh, modem_at, modem_lte, modem_gnss, ble_mgr, ble_obd	Trừu tượng hóa phần cứng và giao thức ngoại vi
2	power_mgr, nvs_config, app_state	Quản lý nguồn, lưu cấu hình NVS và duy trì ngữ cảnh RTC qua deep sleep
3	state_machine, command_handler	Điều phối trạng thái vận hành, lệnh điều khiển và luồng OTA
4	data_formatter, mqtt_client	Đóng gói dữ liệu JSON, publish/subscribe MQTT theo topic chuẩn của hệ thống
5	offline_queue, sd_log_store	Hàng đợi cục bộ trên microSD (đưa vào hàng đợi, phát lại theo xác nhận/QoS, dọn dữ liệu theo hạn mức, metadata phiên ghi)

Trong triển khai hiện tại, firmware không bị chia nhỏ thành quá nhiều tác vụ ngang hàng. Phần lớn quyết định vận hành vẫn đi qua một máy trạng thái trung tâm, tức là một bộ điều phối quyết định thiết bị đang ở chế độ nào và bước tiếp theo phải làm gì. Các cơ chế của FreeRTOS chỉ được dùng ở những nơi thật sự cần phối hợp nhiều tiến trình như BLE, lớp AT command của modem hoặc các hàng đợi dữ liệu.

b) Cơ chế điều phối với FreeRTOS Điểm quan trọng ở đây là FreeRTOS được dùng có chọn lọc, không dùng theo kiểu “càng nhiều task càng tốt”. Luồng chính trong `app_main()` sẽ đọc cấu hình, xác định vì sao thiết bị vừa thức dậy, rồi lập nhanh `state_machine_run()` để điều phối hành vi kế tiếp. Cách tổ chức này giữ cho logic điều khiển nằm ở một chỗ, nhờ đó người đọc mã nguồn dễ bám hơn và quá trình debug cũng bớt rối hơn.

```
void app_main(void) {
    ESP_ERROR_CHECK(nvs_config_init());
```



Hình 3.5: Sơ đồ kiến trúc phân lớp của firmware


```

config_t config = {0};
ESP_ERROR_CHECK(nvs_config_load(&config));

app_state_t state = APP_STATE_INIT;
esp_sleep_wakeup_cause_t wakeup = esp_sleep_get_wakeup_cause();
if (wakeup == ESP_SLEEP_WAKEUP_TIMER) {
    state = APP_STATE_HEARTBEAT;
} else if (wakeup == ESP_SLEEP_WAKEUP_EXT0) {
    state = APP_STATE_ALARM;
}

ESP_ERROR_CHECK(state_machine_init(&config));

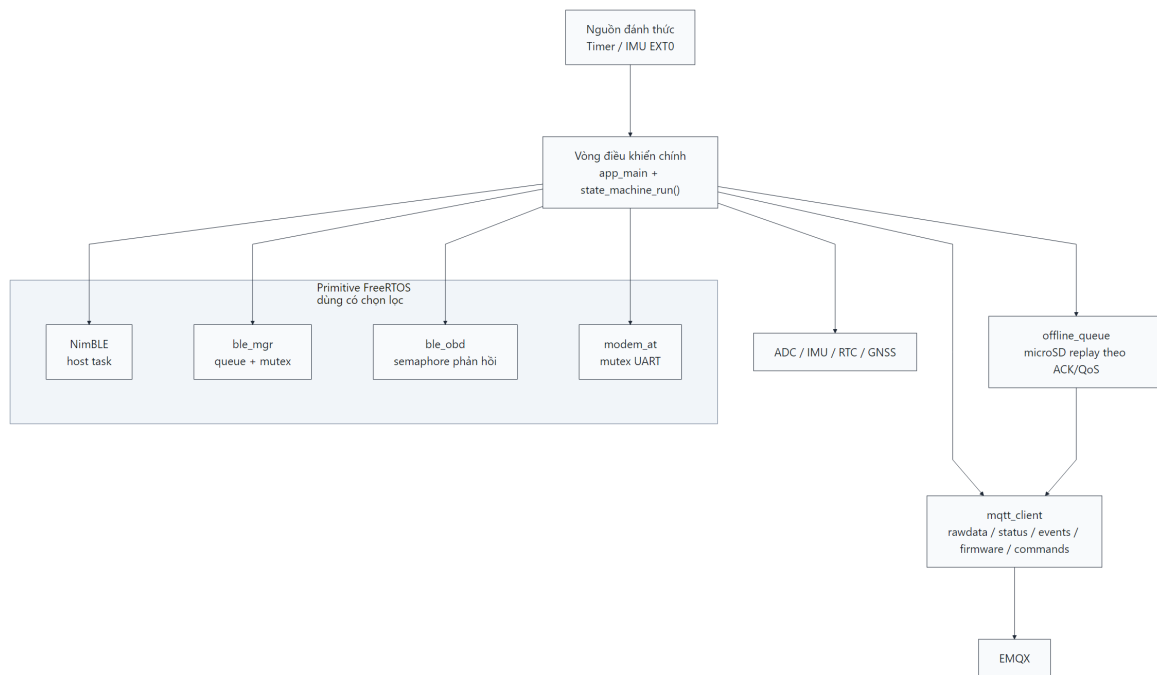
while (true) {
    state = state_machine_run(state);
    vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

Các primitive FreeRTOS vẫn xuất hiện ở những nơi cần “giữ trật tự” khi nhiều phần cùng dùng chung tài nguyên. Cụ thể, NimBLE host chạy ở một task nền riêng; ble_mgr dùng queue và mutex để điều phối kết quả quét và kết nối; ble_obd dùng semaphore để chờ phản hồi từ adapter; còn modem_at dùng mutex để bảo đảm các lệnh AT đi qua UART theo đúng thứ tự.

c) Luồng hoạt động cơ bản Nếu nhìn từ góc độ vận hành, firmware lặp lại một chu trình rất rõ: khởi tạo, nhận biết trạng thái xe, chọn chế độ phù hợp, thực hiện tác vụ và quay về trạng thái tiết kiệm năng lượng khi có thể. Luồng tổng thể được tổ chức theo trình tự sau:

1. **Khởi tạo ngoại vi:** Cấu hình và khởi động các peripheral gồm IMU (LIS3DH qua I2C), modem SIM7600CE-T (UART1, Auto mode AT+CNMP=2, GNSS qua AT+CGNSINF), ADC (đọc điện áp), và BLE stack (NimBLE), tức lớp phần mềm nền để xử lý kết nối Bluetooth Low Energy.
2. **Đọc trạng thái IGN và điện áp ắc quy:** Hệ thống ưu tiên đọc trạng thái động cơ (IGN) trực tiếp từ ECU qua OBD2 BLE. Nếu không kết nối được OBD2, hệ thống fallback sang đo điện áp ắc quy qua ADC theo profile: profile 12V dùng $IGN_{ON} \geq 13.0\text{ V}$ và $IGN_{OFF} \leq 12.0\text{ V}$; profile 24V dùng $IGN_{ON} \geq 26.0\text{ V}$ và $IGN_{OFF} \leq 24.0\text{ V}$.



Hình 3.6: Sơ đồ tương tác giữa các FreeRTOS task

3. **Quyết định chế độ hoạt động:** Dựa trên trạng thái IGN và dữ liệu cảm biến, firmware chuyển sang chế độ phù hợp (lái xe, đỗ xe, hoặc cảnh báo).

4. **Thực thi tác vụ trong từng chế độ:**

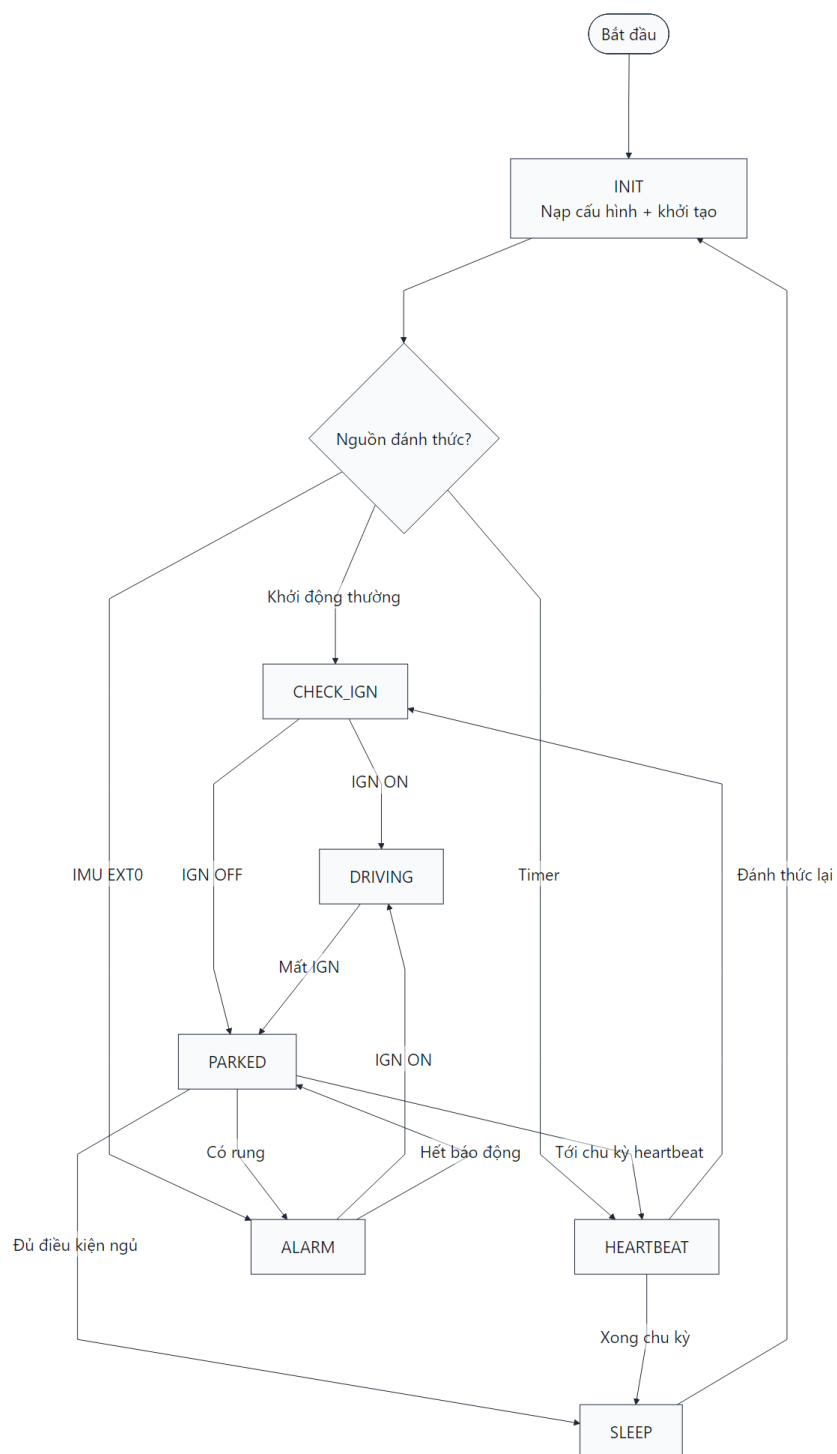
- **Chế độ lái xe (Driving):** Kết nối OBD2, theo dõi liên tục, giữ BLE hoạt động, gửi dữ liệu vận hành định kỳ.
- **Chế độ đỗ xe (Parked):** Không kết nối OBD2, chuyển sang deep sleep, gửi heartbeat định kỳ.
- **Chế độ cảnh báo (Alarm):** Gửi cảnh báo ngay lập tức, theo dõi liên tục, có thể kết nối OBD2 tùy chọn.

5. **Xử lý sau tác vụ:** Nếu IGN ON, giữ kết nối BLE và không deep sleep. Nếu IGN OFF, ngắt BLE và chuyển sang deep sleep để tiết kiệm năng lượng.

3.2.2.2. Module giao tiếp Bluetooth OBD2

a) Tổng quan giao tiếp BLE OBD2 Nói ngắn gọn, module này là phần giúp thiết bị theo dõi “hỏi xe đang ở trạng thái nào” thông qua cổng OBD2 mà không cần dây nối trực tiếp tới vi điều khiển. Adapter vgate iCar Pro được cắm vào cổng chẩn đoán OBD-II của xe, sau đó giao tiếp không dây với thiết bị theo dõi bằng BLE. Trong firmware, NimBLE là lớp phần mềm lo phần Bluetooth; còn GATT là cách hai thiết bị tổ chức dịch vụ và trường dữ liệu để đọc/ghi thông tin với nhau.

Kiến trúc module BLE OBD2 được chia thành bốn tầng để mỗi tầng lo một việc riêng.



Hình 3.7: Lưu đồ thuật toán luồng hoạt động chính của firmware

state_machine.c quyết định khi nào cần đọc dữ liệu xe; ble_obd.c xử lý phân lệnh OBD2; ble_mgr.c lo kết nối Bluetooth; còn NimBLE đảm nhiệm lớp BLE nền tảng. Cách chia này giúp việc đọc dữ liệu xe không bị lẫn với chi tiết kỹ thuật của GATT/BLE, nên dễ debug và mở rộng hơn.

b) Quy trình kết nối BLE OBD2 Quy trình kết nối BLE với adapter OBD2 diễn ra theo các bước sau:

1. **Kiểm tra địa chỉ BLE đã lưu:** Đọc địa chỉ MAC của vgate iCar Pro từ bộ nhớ flash (NVS). Nếu đã có địa chỉ, chuyển trực tiếp sang bước kết nối (kết nối lại nhanh trong 1–3 giây).
2. **Quét tìm thiết bị (Scan):** Nếu chưa có địa chỉ, thực hiện BLE scan để tìm adapter. Quá trình scan lọc thiết bị theo tên (device name) hoặc service UUID đặc trưng của OBD2 adapter.
3. **Kết nối GATT:** Thiết lập kết nối BLE với adapter, thực hiện GATT service discovery để tìm các characteristic cần thiết (TX và RX characteristic).
4. **Gửi lệnh khởi tạo ELM327:** Gửi chuỗi lệnh khởi tạo giao thức ELM327 qua BLE GATT write:
 - ATZ — Reset adapter
 - ATE0 — Tắt echo
 - ATL0 — Tắt line feed
 - ATS0 — Tắt khoảng trắng
 - ATSP0 — Tự động phát hiện giao thức OBD2
5. **Đọc dữ liệu OBD2:** Gửi các lệnh PID (Parameter ID) để đọc thông số xe:
 - 010C — RPM động cơ
 - 010D — Tốc độ xe
 - 0105 — Nhiệt độ nước làm mát
 - 012F — Mức nhiên liệu
 - AT IGN — Trạng thái động cơ (ignition)

```
/* Quy trình kết nối và đọc dữ liệu OBD2 */
```

```
void ble_obd2_task(void *param) {  
    /* Bước 1: Đọc BLE address từ NVS */  
    uint8_t saved_addr[6];  
    bool has_saved = nvs_read_ble_address(saved_addr);  
  
    /* Bước 2: Kết nối hoặc scan */  
    if (has_saved) {
```

```

        ble_connect_direct(saved_addr); /* Reconnect nhanh */
    } else {
        ble_start_scan(BLE_SCAN_TIMEOUT_MS);
        ble_connect_first_obd2_device();
        nvs_save_ble_address(connected_addr);
    }

    /* Bước 3: Khởi tạo ELM327 */
    ble_obd_send_command("ATZ");
    ble_obd_send_command("ATE0");
    ble_obd_send_command("ATSP0");

    /* Bước 4: Vòng lặp đọc dữ liệu */
    while (device_state == STATE_DRIVING) {
        obd2_data_t data;
        data.rpm    = ble_obd_read_pid(PID_RPM);
        data.speed  = ble_obd_read_pid(PID_SPEED);
        data.temp   = ble_obd_read_pid(PID_COOLANT_TEMP);
        data.fuel   = ble_obd_read_pid(PID_FUEL_LEVEL);

        xQueueSend(telemetry_queue, &data, pdMS_TO_TICKS(100));
        vTaskDelay(pdMS_TO_TICKS(TRACKING_INTERVAL_MS));
    }
}

```

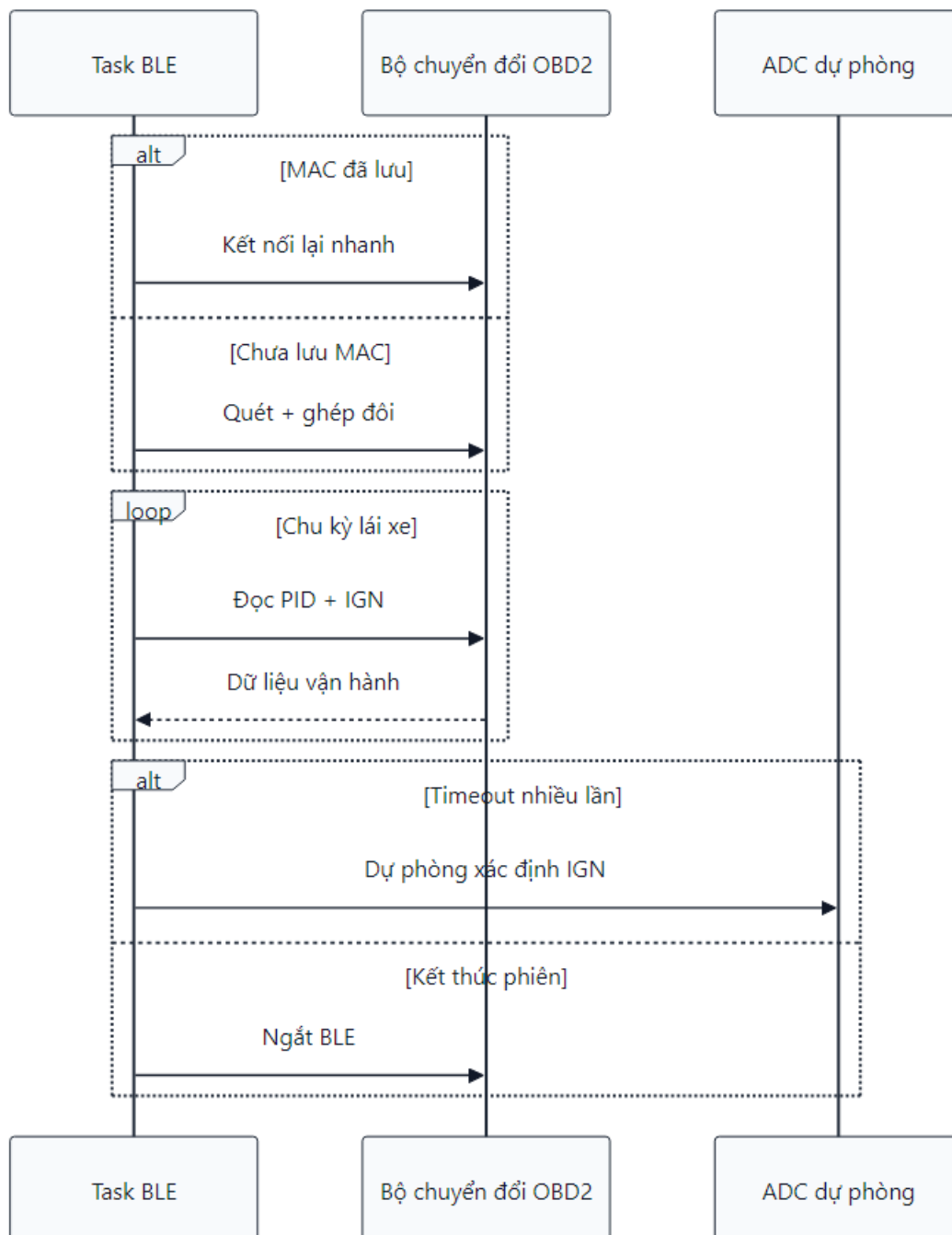
c) Chiến lược kết nối theo chế độ hoạt động Việc kết nối BLE OBD2 được tối ưu hóa theo từng chế độ hoạt động của thiết bị để tiết kiệm năng lượng:

Chế độ lái xe (IGN ON): ESP32-S3 duy trì kết nối BLE liên tục với OBD2 adapter. Dữ liệu OBD2 được đọc định kỳ (mỗi 5–10 giây). Hệ thống chỉ sử dụng light sleep (giữ BLE active) thay vì deep sleep.

Chế độ đỗ xe (IGN OFF): Không kết nối OBD2 adapter. Trạng thái IGN đã được xác nhận trước khi chuyển sang deep sleep. Việc không kết nối BLE khi đỗ xe giúp tiết kiệm đáng kể thời gian wake-up và năng lượng tiêu thụ.

Chế độ cảnh báo (Motion Detected): Kết nối OBD2 là tùy chọn. Hệ thống có thể chỉ sử dụng IMU và GPS để xác nhận chuyển động mà không cần dữ liệu OBD2.

Đặc biệt, khi ESP32-S3 chuyển sang deep sleep, kết nối BLE bị ngắt hoàn toàn. Mỗi lần wake-up, firmware cần thực hiện kết nối lại với adapter. Nhờ có việc lưu địa chỉ MAC vào flash, thời gian kết nối lại chỉ mất khoảng 1–3 giây, nhanh hơn đáng kể so với pairing



Hình 3.8: Lưu đồ thuật toán quy trình kết nối và đọc dữ liệu BLE OBD2

lần đầu (3–10 giây).

d) Xử lý lỗi và cơ chế dự phòng (Fallback) Module BLE OBD2 được thiết kế với nhiều lớp xử lý lỗi:

Bảng 3.2.2B: Tổng hợp thông tin về Tình huống lỗi, Xử lý, Thời gian timeout

Tình huống lỗi	Xử lý	Thời gian timeout
Không kết nối được adapter	Retry 2–3 lần với delay 2 giây, sau đó fallback	10 giây
Mất kết nối giữa chừng	Reconnect 2–3 lần, nếu thất bại thì fallback	5 giây mỗi lần
Adapter không phản hồi lệnh	Retry 1–2 lần, nếu thất bại thì fallback	5 giây

Cơ chế dự phòng (Fallback): Khi không thể giao tiếp với OBD2 adapter, hệ thống chuyển sang phát hiện trạng thái động cơ bằng phương pháp đo điện áp ắc quy. Phương pháp này có độ chính xác thấp hơn OBD2 nhưng vẫn đảm bảo hệ thống hoạt động liên tục. Tiêu chí phân biệt theo profile: profile 12V dùng $IGN_{ON} \geq 13.0\text{ V}$ và $IGN_{OFF} \leq 12.0\text{ V}$; profile 24V dùng $IGN_{ON} \geq 26.0\text{ V}$ và $IGN_{OFF} \leq 24.0\text{ V}$.

Mọi lỗi kết nối OBD2 đều được ghi lại vào log để phục vụ việc giám sát và khắc phục sự cố từ xa.

3.2.2.3. Module điều khiển modem SIMCom

a) Kiến trúc modem SIMCom SIM7600CE-T (LTE + GNSS tích hợp) Trong kiến trúc hiện tại, modem SIMCom SIM7600CE-T đảm nhận toàn bộ luồng LTE và GNSS. ESP32-S3 giao tiếp với modem qua UART1 (GPIO16 TX, GPIO17 RX). Firmware sử dụng một chuỗi lệnh AT chính xác để đảm bảo modem sẵn sàng trước khi bật PDP context và khởi động GNSS.

Bảng 3.2.2C: Tổng hợp thông tin về Bước, Lệnh AT, Mục đích chính

Bước	Lệnh AT	Mục đích chính
1	AT	Kiểm tra modem còn phản hồi
2	AT+CPIN?	Kiểm tra SIM card đã sẵn sàng

Bảng 3.2.2C: Tổng hợp thông tin về Bước, Lệnh AT, Mục đích chính (tiếp theo)

Bước	Lệnh AT	Mục đích chính
3	AT+CNMP=2	Đặt chế độ tự động LTE/UMTS/GSM
4	AT+CGDCONT=1 ,"IP","internet"	Cấu hình PDP context với APN mặc định internet
5	AT+CEREG?	Poll đến khi +CEREG: 0,1 (hoặc stat=5) trước khi kích hoạt PDP context
6	AT+CGACT=1,1	Bật PDP context sau khi đăng ký mạng thành công
7	AT+CGPADDR=1	Ghi nhận địa chỉ IP được cấp (dùng để log và kiểm tra)
8	AT+CGNSPWR=1	Bật GNSS tích hợp và đợi modem trả về fix vị trí
9	AT+CGNSINF	Đọc tọa độ GNSS hiện tại (vị trí, tốc độ, số vệ tinh); AT+CGNSTST chỉ dùng nếu cần dữ liệu NMEA hoàn toàn

Toàn bộ lệnh AT cho LTE và GNSS được thực hiện trên cùng UART1. Trình tự kiểm tra AT+CEREG? trước khi gọi AT+CGACT=1,1 giúp tránh kích hoạt PDP context khi modem chưa đăng ký mạng.

b) Trình tự khởi tạo LTE/GNSS trong firmware Ở tầng firmware, chuỗi lệnh trên không được gửi dồn trong một lần, mà được tách thành các bước kiểm tra có điều kiện: xác nhận modem phản hồi, xác nhận SIM sẵn sàng, chờ modem đăng ký mạng, sau đó mới bật PDP context và GNSS. Cách tổ chức này giúp quá trình khởi tạo dễ retry hơn và hạn chế lỗi dây chuyền khi điều kiện sóng di động thay đổi.

c) Quản lý kết nối và tiết kiệm điện

- **Khi thiết bị hoạt động liên tục:** Giữ PDP context mở và GNSS bật liên tục để gửi dữ liệu vận hành định kỳ và bản tin báo vẫn đang hoạt động.
- **Khi dừng (parking mode):** Firmware đóng PDP context (AT+CGACT=0,1), đưa modem vào AT+CSCLK=1 hoặc AT+CFUN=0, rồi chuyển ESP32-S3 sang deep sleep. Khi wake-up, modem được đánh thức qua UART và thực hiện lại các lệnh AT+CFUN=1 + AT+CGACT=1,1.
- **GNSS:** Sau khi AT+CGNSPWR=1, firmware tuần tự gọi AT+CGNSINF để lấy dữ liệu; AT+CGNSTST chỉ dùng khi cần stream NMEA để hỗ trợ debug hoặc phân tích phụ trợ.

d) Xử lý sự cố

Bảng 3.2.2D: Tổng hợp thông tin về Tình huống, Biện pháp

Tình huống	Biện pháp
Modem không phản hồi	Reset nhanh qua PWR-KEY (GPIO26) rồi bắt đầu lại chuỗi AT
Mất kết nối 4G	Đợi AT+CEREG? thành 1, gọi AT+CGACT=0,1 rồi AT+CGACT=1,1 để tái khởi động PDP
GNSS không fix dù đã bật	Gọi lại AT+CGNSINF, nếu vẫn thất bại thì bật lại GNSS (AT+CGNSPWR=0/1) rồi thử lại
Modem quá nóng	Đưa AT+CFUN=0 để tạm thời tắt RF và chờ nhiệt độ giảm trước khi tiếp tục

Các chế độ ngủ (AT+CSCLK=1, AT+CFUN=0) và wake-up bằng UART giữ cho modem tiêu thụ thấp mà vẫn có thể khởi động lại nhanh khi ESP32-S3 tỉnh dậy.

3.2.2.4. Module quản lý nguồn và GPIO

a) Sơ đồ chân GPIO Module quản lý nguồn sử dụng các chân GPIO của ESP32-S3 để điều khiển và giám sát hệ thống nguồn điện. Bảng sau liệt kê đầy đủ các chân GPIO được sử dụng:

Bảng 3.2.2E: Tổng hợp thông tin về GPIO, Chức năng, Hướng

GPIO	Chức năng	Hướng	Mô tả
2	IGN_IN	Input	Đọc trạng thái động cơ (GPIO hoặc OBD2)
4	U_BATT_ADC	Input	Đọc điện áp ắc quy (ADC 12-bit)
5	CHARGER_EN	Output	Điều khiển khối sạc TP5100
18	POWER_PATH_EN	Output	Chọn nguồn cấp (ắc quy hoặc pin dự phòng)
19	LVD_STATUS	Input	Đọc trạng thái LVD từ khối LVD phần cứng
41	LIS3DH_INT	Input	Ngắt từ cảm biến gia tốc IMU
2	LIS3DH_SDA	I/O	Đường dữ liệu I2C
1	LIS3DH_SCL	I/O	Đường clock I2C

Bảng 3.2.2E: Tổng hợp thông tin về GPIO, Chức năng, Hướng (tiếp theo)

GPIO	Chức năng	Hướng	Mô tả
16	MODEM_UART_TX	Output	UART TX đến modem
17	MODEM_UART_RX	Input	UART RX từ modem
26	MODEM_PWR-KEY	Output	Điều khiển nguồn modem

b) Điều khiển mạch chuyển nguồn (Diode OR + EN) Hệ thống dùng mạch chuyển nguồn theo **diode OR** giữa nhánh 5V chính (MP2482) và nhánh 5V dự phòng (SX1308). Firmware điều khiển qua GPIO18 để ưu tiên nhánh nguồn phù hợp theo profile điện áp.

- **GPIO18 = LOW (0):** Ưu tiên nguồn ắc quy (nhánh MP2482 hoạt động)
- **GPIO18 = HIGH (1):** Giảm/khóa nhánh chính và cho phép nhánh dự phòng qua SX1308 + diode OR

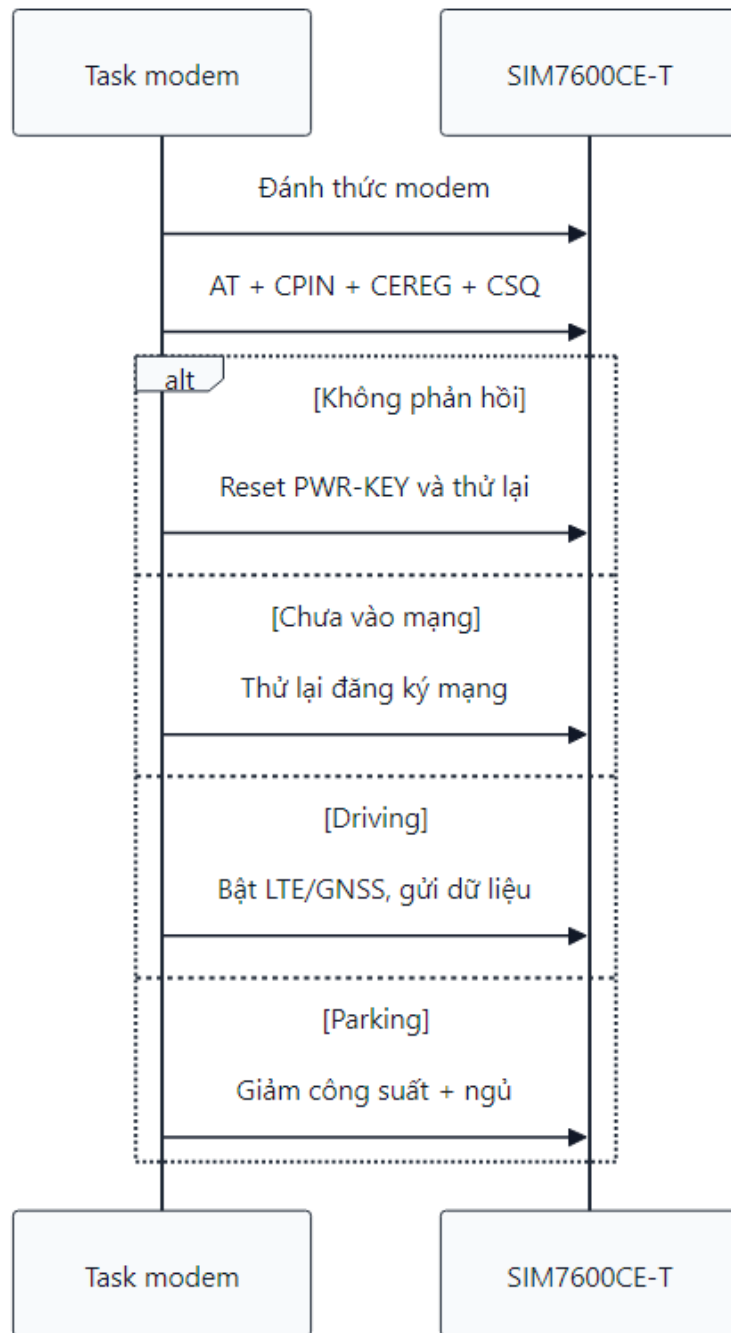
Logic điều khiển nguồn bám theo profile 12V/24V:

```
power_status_t power_get_status(float lvd_threshold_v, float lvd_hysteresis_v) {
    power_status_t status = {
        .voltage = adc_read_battery_voltage(),
        .is_low = false,
        .charger_on = gpio_get_level(PIN_CHARGER_EN) == 1,
    };

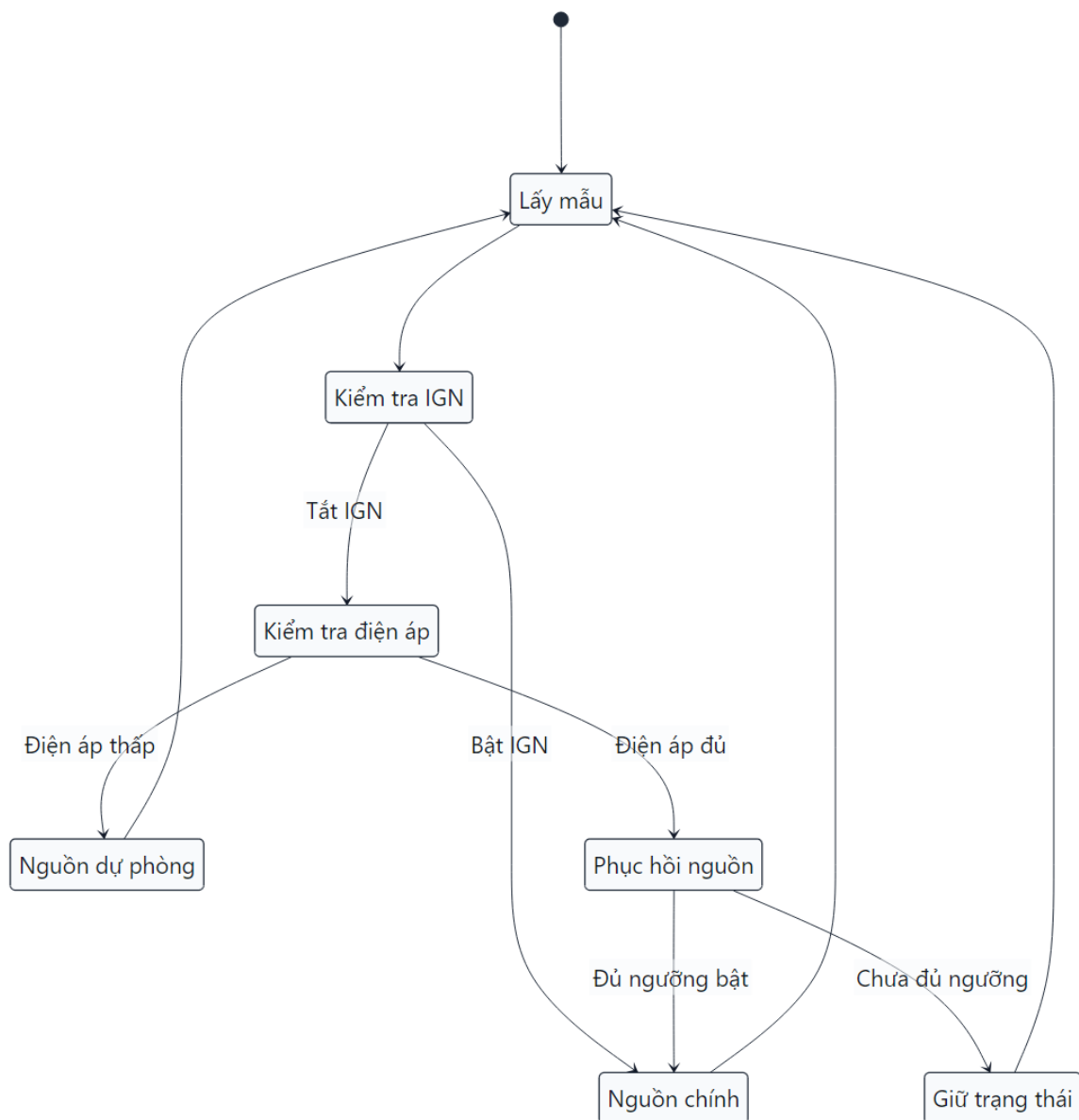
    if (status.voltage <= lvd_threshold_v) {
        s_voltage_low_latched = true;
    } else if (status.voltage >= lvd_hysteresis_v) {
        s_voltage_low_latched = false;
    }

    status.is_low = s_voltage_low_latched || power_is_low_voltage();
    return status;
}
```

c) Điều khiển khối sạc (TP5100) Khối sạc TP5100 được điều khiển qua chân GPIO5 (CHARGER_EN). Logic sạc được thiết kế để bảo vệ cả ắc quy xe lẫn pin dự phòng:



Hình 3.9: Lưu đồ điều phối modem SIM7600CE-T theo trạng thái mạng và chế độ vận hành



Hình 3.10: Lưu đồ thuật toán điều khiển chuyển nguồn

Bảng 3.2.2F: Tổng hợp thông tin về Điều kiện, Trạng thái charger, Lý do

Điều kiện	Trạng thái charger	Lý do
IGN ON và U_{batt} vượt ngưỡng profile (ví dụ $> 12\text{ V}$ cho hệ 12V)	Enable (GPIO5 = HIGH)	Máy phát điện đang nạp, có thể sạc pin dự phòng
IGN OFF	Disable (GPIO5 = LOW)	Bảo vệ ắc quy không bị hao pin khi xe tắt máy

*Bảng 3.2.2F: Tổng hợp thông tin về Điều kiện, Trạng thái charger,
Lý do (tiếp theo)*

Điều kiện	Trạng thái charger	Lý do
U_{batt} dưới ngưỡng profile (ví dụ $< 12\text{ V}$ cho hệ 12V)	Disable (GPIO5 = LOW)	Ắc quy yếu, không đủ năng lượng để sạc pin dự phòng

d) Giám sát điện áp và LVD Firmware đọc điện áp ắc quy qua kênh ADC 12-bit (GPIO4) với bộ chia áp để đưa điện áp ắc quy 12V hoặc 24V về dải đo của ADC (0–3.3V). Giá trị ADC được chuyển đổi sang điện áp thực thông qua công thức hiệu chuẩn (calibration).

Ngoài ra, hệ thống có thêm kênh giám sát LVD độc lập từ khối LVD phần cứng (GPIO19). Đây là kênh dự phòng cho ADC, cho phép kiểm tra nhanh trạng thái nguồn với cơ chế trễ (hysteresis): profile 12V dùng $Switch_{\text{OFF}} = 12.0\text{ V}$, $Switch_{\text{ON}} = 12.2\text{ V}$; profile 24V dùng $Switch_{\text{OFF}} = 24.0\text{ V}$, $Switch_{\text{ON}} = 24.4\text{ V}$. Khối LVD xử lý hysteresis ở mức phần cứng, tránh hiện tượng dao động (oscillation) quanh ngưỡng.

e) Các chế độ quản lý nguồn Firmware hỗ trợ ba chế độ năng lượng chính:

*Bảng 3.2.2G: Tổng hợp thông tin về Chế độ, Trạng thái ESP32,
Trạng thái modem*

Chế độ	Trạng thái ESP32	Trạng thái modem	Trạng thái BLE	Dòng tiêu thụ
Xe đang chạy	Hoạt động đầy đủ	4G + GNSS hoạt động	Đang kết nối	150–250 mA
Ngủ nhẹ	Ngủ nhẹ	Ngủ nhẹ của modem (CSCLK)	Giữ kết nối	10–20 mA
Ngủ sâu	Ngủ sâu	Ngủ sâu của modem (CFUN=0)	Ngắt kết nối	$< 2\text{ mA}$

Chuyển đổi giữa các chế độ dựa trên trạng thái IGN và dữ liệu cảm biến, được điều phối bởi máy trạng thái chính (trình bày tại mục 3.2.2.5).

3.2.2.5. Định dạng dữ liệu và máy trạng thái

a) Định dạng dữ liệu MQTT Dữ liệu từ thiết bị được đóng gói theo định dạng JSON và truyền lên broker qua nhóm topic `v1/{device_id}/...` Trong hệ thống hiện tại,

thiết bị phát ba nhóm gói dữ liệu chính là rawdata, status và events; ngoài ra còn có luồng firmware để báo trạng thái cập nhật từ xa và luồng commands theo chiều ngược lại từ máy chủ.

Thông điệp Rawdata (gói dữ liệu vận hành chính)

Đây là gói dữ liệu được gửi định kỳ khi thiết bị đang hoạt động, phản ánh trạng thái GNSS, nguồn và cảm biến rung:

```
{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "timestamp": 1710000000000,
  "uptime": 452130,
  "data": {
    "vibration": 128,
    "battery_top": 12.54,
    "battery_bot": 3.96,
    "latitude": 10.77653,
    "longitude": 106.70098,
    "speed": 42.5,
    "course": 181.2,
    "satellites": 11,
    "ignition": true,
    "error_code": 0
  }
}
```

Thông điệp Event (sự kiện/cảnh báo)

Được gửi khi firmware phát hiện một sự kiện bất thường hoặc muốn ghi nhận một mốc vận hành đáng chú ý:

```
{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "event_type": "high_vibration",
  "code": 201,
  "message": "Vehicle movement detected while parked",
  "timestamp": 1710000005123
}
```

Thông điệp Firmware (trạng thái OTA)

Luồng firmware được dùng để ghi nhận vòng đời cập nhật firmware từ lúc gán tác vụ đến khi xác nhận thành công sau reboot hoặc rollback:

```
{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "jobId": "ota_1710000000_ab12cd",
  "status": "confirming",
  "progress": 99,
  "targetVersion": "1.2.0",
  "currentVersion": "1.1.3",
  "partition": "ota_0"
}
```

Thông điệp Status (trạng thái hoạt động)

Được gửi khi thiết bị đổi trạng thái phiên làm việc hoặc phát heartbeat tại các mốc cần theo dõi:

```
{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "status": "running",
  "session_id": 1024,
  "timestamp": 1710000010000
}
```

b) Thiết kế topic MQTT Hệ thống sử dụng cấu trúc phân cấp topic MQTT để tách rõ luồng dữ liệu vận hành gửi đều, trạng thái, sự kiện và cập nhật từ xa:

Bảng 3.2.2H: Tổng hợp thông tin về Topic, Hướng, QoS

Topic	Hướng	QoS	Mô tả
v1/{device_id}/rawdata	Device → Server	0	Gói dữ liệu vận hành chính; gửi dày, chấp nhận mất vài mẫu
v1/{device_id}/status	Device → Server	1	Heartbeat, trạng thái phiên, online/offline
v1/{device_id}/events	Device → Server	1	Cảnh báo và sự kiện quan trọng
v1/{device_id}/firmware	Device → Server	1	Tiến trình cập nhật từ xa và kết quả cập nhật firmware

Bảng 3.2.2H: Tổng hợp thông tin về Topic, Hướng, QoS (tiếp theo)

Topic	Hướng	QoS	Mô tả
v1/{device_id}/commands	Server → Device	1	Lệnh điều khiển và cập nhật cấu hình từ máy chủ

Các lệnh điều khiển từ máy chủ hiện được firmware xử lý gồm:

Bảng 3.2.2I: Tổng hợp thông tin về Lệnh, Mục đích, Tham số chính

Lệnh	Mục đích	Tham số chính
update_config	Cập nhật cấu hình thiết bị	heartbeat_interval_s, tracking_interval_s
request_location	Yêu cầu gửi vị trí ngay ở chu kỳ kế tiếp	Không
enable_tracking	Bật hoặc tắt phát dữ liệu vận hành định kỳ	enabled (boolean)
reboot	Khởi động lại thiết bị	Không
ota_update	Kích hoạt cập nhật firmware OTA	jobId, version, url, size, sha256, force, confirmTimeoutSec
manual_rollback / ota_rollback	Yêu cầu quay về firmware trước đó	Không bắt buộc

c) Chiến lược xử lý mất kết nối Ở phạm vi triển khai hiện tại, firmware kết hợp hai cơ chế song song: (1) tự phục hồi kết nối MQTT và (2) hàng đợi cục bộ trên **microSD SDMMC 4-bit** để giữ dữ liệu khi mất mạng. Luồng dữ liệu được hiện thực qua cặp module `offline_queue` và `sd_log_store`: bản ghi được đưa vào hàng đợi khi dữ liệu vận hành phát sinh (`offline_queue_enqueue()`), ghi xuống FATFS (`sd_log_store_append()`), sau đó replay theo thứ tự sequence khi mạng phục hồi (`offline_queue_replay_tick()`) và chốt ACK cho bản ghi quan trọng (`offline_queue_handle_publish_ack()`).

Thiết kế này cho phép tách bản ghi thành hai nhóm: **quan trọng** (QoS 1, yêu cầu xác nhận đã nhận) và **ít quan trọng hơn** (QoS 0, ưu tiên tốc độ gửi/ghi), đồng thời áp dụng cơ chế giới hạn dung lượng + dọn bản ghi cũ (`sd_log_store_gc_if_needed()`) để tránh đầy thẻ. Trong phạm vi vận hành hiện tại, hệ thống giả định thẻ SD ở trạng thái **đã cắm sẵn từ lúc khởi động**; luồng rút/cắm nóng trong lúc đang chạy không phải phạm vi mục

tiêu.

```
/* Chiến lược hiện tại: enqueue local + replay khi MQTT sẵn sàng */  
offline_queue_enqueue(type, payload, gps_fix, net_up);
```

```
if (tracker_mqtt_is_connected()) {  
    offline_queue_replay_tick();  
}
```

d) Máy trạng thái thiết bị (Device State Machine) Máy trạng thái là cơ chế điều phối trung tâm của firmware, quyết định hành vi thiết bị tại từng thời điểm vận hành. Hệ thống định nghĩa bảy trạng thái chính:

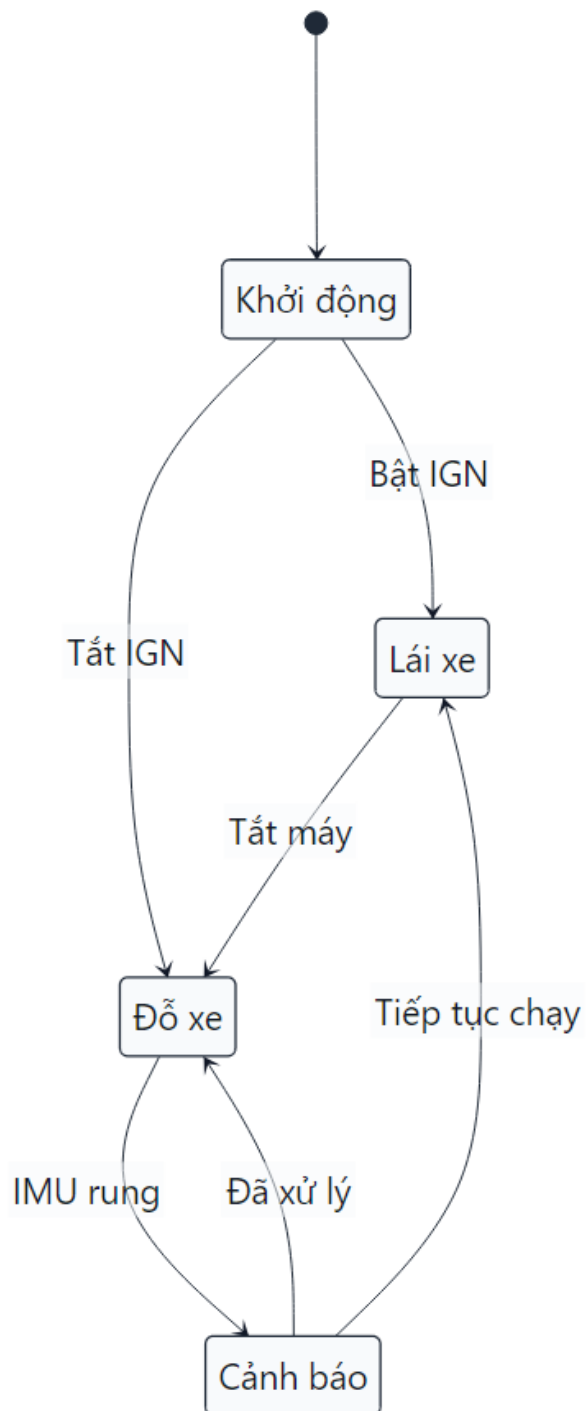
Bảng 3.2.2J: Tổng hợp thông tin về Trạng thái, Mã, Mô tả

Trạng thái	Mã	Mô tả
INIT	0	Khởi tạo hệ thống, cấu hình ngoại vi
CHECK_IGN	1	Kiểm tra trạng thái động cơ
DRIVING	2	Chế độ lái xe - theo dõi liên tục
PARKED	3	Chế độ đỗ xe - tiết kiệm năng lượng
ALARM	4	Chế độ cảnh báo - phát hiện chuyển động bất thường
HEARTBEAT	5	Gửi tín hiệu heartbeat
SLEEP	6	Ngủ sâu - tiêu thụ năng lượng tối thiểu

Bảng chuyển đổi trạng thái:

Trạng thái hiện tại	Sự kiện kích hoạt	Trạng thái tiếp theo
INIT	Khởi tạo hoàn tất	CHECK_IGN
CHECK_IGN	IGN = ON	DRIVING
CHECK_IGN	IGN = OFF	PARKED
DRIVING	IGN = OFF được phát hiện	PARKED
PARKED	IMU interrupt (motion detected)	ALARM
PARKED	Timer wake-up	HEARTBEAT
ALARM	Motion dừng, IGN = OFF	PARKED
ALARM	IGN = ON	DRIVING
HEARTBEAT	Gửi heartbeat xong	SLEEP

Trạng thái hiện tại	Sự kiện kích hoạt	Trạng thái tiếp theo
SLEEP	Timer wake-up hoặc IMU interrupt	CHECK_IGN



Hình 3.11: Sơ đồ máy trạng thái của thiết bị theo dõi

```

/* Cấu trúc máy trạng thái */
typedef enum {

```

```

STATE_INIT,
STATE_CHECK_IGN,
STATE_DRIVING,
STATE_PARKED,
STATE_ALARM,
STATE_HEARTBEAT,
STATE_SLEEP
} device_state_t;

void state_machine_task(void *param) {
    device_state_t state = STATE_INIT;

    while (1) {
        switch (state) {
            case STATE_INIT:
                init_peripherals();
                state = STATE_CHECK_IGN;
                break;

            case STATE_CHECK_IGN:
                if (check_ignition_status()) {
                    state = STATE_DRIVING;
                } else {
                    state = STATE_PARKED;
                }
                break;

            case STATE_DRIVING:
                start_continuous_tracking();
                /* Chuyển sang PARKED khi phát hiện IGN OFF */
                EventBits_t bits = xEventGroupWaitBits(
                    system_event_group, EVT_IGN_OFF,
                    pdTRUE, pdFALSE, pdMS_TO_TICKS(1000));
                if (bits & EVT_IGN_OFF) {
                    stop_ble_connection();
                    state = STATE_PARKED;
                }
                break;
        }
    }
}

```

```

    case STATE_PARKED:
        prepare_for_sleep();
        state = STATE_SLEEP;
        break;

    case STATE_ALARM:
        send_alert_immediate();
        start_continuous_tracking();
        /* Đợi cho đến khi hết chuyển động */
        if (!is_motion_detected() && !check_ignition_status()) {
            state = STATE_PARKED;
        }
        break;

    case STATE_HEARTBEAT:
        wakeup_modem();
        tracker_mqtt_connect();
        send_heartbeat();
        state = STATE_SLEEP;
        break;

    case STATE_SLEEP:
        shutdown_modem();
        configure_wakeup_sources();
        enter_deep_sleep();
        /* Sau khi wake-up, bắt đầu lại từ CHECK_IGN */
        state = STATE_CHECK_IGN;
        break;
    }
}
}

```

Lưu trạng thái qua deep sleep: Trước khi vào deep sleep, firmware lưu trạng thái hiện tại vào vùng nhớ RTC (RTC memory) — vùng nhớ đặc biệt của ESP32-S3 được giữ nguyên nội dung trong suốt quá trình deep sleep. Khi wake-up, firmware đọc trạng thái từ RTC memory để biết chế độ hoạt động trước đó và xử lý phù hợp.

3.2.2.6. Cấu hình hệ thống

a) Cấu trúc dữ liệu cấu hình Toàn bộ cấu hình của thiết bị được lưu trữ trong bộ nhớ flash (NVS — Non-Volatile Storage) của ESP32-S3 dưới dạng cấu trúc dữ liệu cố định:

```
typedef struct {
    char    device_id[16];           /* Mã định danh thiết bị */
    char    mqtt_broker[64];         /* Địa chỉ MQTT broker */
    uint16_t mqtt_port;              /* Cổng MQTT (mặc định 1883) */
    char    mqtt_username[32];       /* Tên đăng nhập MQTT */
    char    mqtt_password[32];       /* Mật khẩu MQTT */
    uint16_t heartbeat_interval;     /* Chu kỳ gửi heartbeat (giây) */
    uint16_t tracking_interval;       /* Chu kỳ gửi telemetry (giây) */
    char    obd2_ble_address[18];     /* Địa chỉ MAC của OBD2 adapter */
    float    switch_off;              /* Ngưỡng chuyển sang pin dự phòng: 12.0V (12V) hoặc 24.0V (24V) */
    float    switch_on;               /* Ngưỡng quay lại ắc quy: 12.2V (12V) hoặc 24.4V (24V) */
    float    ign_on_threshold;        /* Ngưỡng suy luận IGN ON: >=13.0V (12V) hoặc >=26.0V (24V) */
    float    ign_off_threshold;       /* Ngưỡng suy luận IGN OFF: <=12.0V (12V) hoặc <=24.0V (24V) */
} config_t;
```

b) Quản lý cấu hình Cấu hình được quản lý theo ba cơ chế:

Cấu hình mặc định (Default Configuration): Được định nghĩa trong mã nguồn firmware, áp dụng khi thiết bị khởi động lần đầu hoặc khi NVS bị xóa. Bao gồm các giá trị an toàn như heartbeat_interval = 900 (15 phút), tracking_interval = 10 (10 giây), và profile nguồn kép: profile 12V (switch_off=12.0, switch_on=12.2, ign_on_threshold=13.0, ign_off_threshold=12.0) hoặc profile 24V (switch_off=24.0, switch_on=24.4, ign_on_threshold=26.0, ign_off_threshold=24.0).

Cấu hình lưu trữ (Persistent Configuration): Lưu trong NVS, được tải khi thiết bị khởi động. Các thay đổi cấu hình từ máy chủ được lưu vào NVS để giữ nguyên sau khi reset hoặc mất điện.

Cập nhật cấu hình từ xa (Remote Configuration Update): Máy chủ gửi lệnh update_config qua topic MQTT v1/{device_id}/commands. Firmware phân tích lệnh, cập nhật các tham số tương ứng và lưu vào NVS. Thiết bị gửi xác nhận (ACK) về máy chủ sau khi cập nhật thành công.

c) Hiệu chuẩn cảm biến (Sensor Calibration) Để đảm bảo độ chính xác của phép đo, firmware hỗ trợ hiệu chuẩn ba loại cảm biến:

Bảng 3.2.2K: Tổng hợp thông tin về Cảm biến, Phương pháp hiệu chuẩn, Lưu trữ

Cảm biến	Phương pháp hiệu chuẩn	Lưu trữ
ADC (điện áp áp quy)	So sánh với đồng hồ vạn năng, tính hệ số hiệu chỉnh	NVS
IMU (LIS3DH)	Đặt thiết bị trên mặt phẳng, đo offset các trục X/Y/Z	NVS
GNSS	Hiệu chỉnh offset vị trí (nếu cần)	NVS

Các giá trị hiệu chuẩn được lưu trữ trong NVS và được tải khi thiết bị khởi động, đảm bảo tính nhất quán của phép đo giữa các lần reset.

3.3. Phân tích và đề xuất kiến trúc máy chủ và xử lý dữ liệu – Server and data-processing architecture

3.3.1. Phân tích và lựa chọn kiến trúc máy chủ

3.3.1.1. Đặt vấn đề cho kiến trúc máy chủ»§ Khi dữ liệu rời khỏi thiết bị, trọng tâm không còn là “đọc được dữ liệu” mà là “biến bản tin đó thành thông tin hữu ích”. Ở tầng máy chủ và xử lý dữ liệu, một bản tin chỉ thật sự có giá trị khi nó được tiếp nhận ổn định, lưu đúng nơi, đẩy tới giao diện quản trị đúng lúc và vẫn truy vết được khi có sự cố. Vì vậy, kiến trúc máy chủ phải đồng thời giải quyết các bài toán sau:

- **Tiếp nhận dữ liệu liên tục từ nhiều thiết bị:** dữ liệu vận hành gửi theo chu kỳ ngắn, có thể dồn thành cụm khi thiết bị kết nối lại.
- **Cập nhật thời gian thực cho giao diện quản trị:** dữ liệu vị trí/trạng thái cần phản ánh gần như tức thời cho người vận hành.
- **Lưu trữ dữ liệu hỗn hợp:** vừa có dữ liệu chuỗi thời gian (GPS, nguồn, IMU; sẵn sàng mở rộng thêm OBD2), vừa có dữ liệu quan hệ nghiệp vụ (xe, người dùng, cảnh báo, vùng cho phép).
- **Chi phí và năng lực vận hành phù hợp đồ án:** ưu tiên tự triển khai trên máy chủ của nhóm, dễ dựng và dễ mở rộng theo từng dịch vụ.

3.3.1.2. So sánh các phương án kiến trúc hệ thống máy chủ

Bảng 3.15A: So sánh các phương án kiến trúc hệ thống máy chủ

Phương án	Mô tả ngắn	Ưu điểm	Hạn chế	Mức phù hợp
PA-A: Một API lớn + một cơ sở dữ liệu chung	Thiết bị gửi HTTP trực tiếp vào API phía máy chủ và mọi dữ liệu dồn vào cùng một nơi lưu	Triển khai ban đầu đơn giản	Không tối ưu cho dữ liệu gửi dày theo thời gian, khó mở rộng cập nhật thời gian thực, tải dễ dồn lên một dịch vụ	Trung bình
PA-B: Dùng dịch vụ máy chủ do nhà cung cấp quản lý sẵn	AWS IoT Core + Lambda + cơ sở dữ liệu do nhà cung cấp vận hành	Khả năng mở rộng cao, nhiều dịch vụ sẵn có	Chi phí vận hành cao, phụ thuộc nhà cung cấp, độ phức tạp hạ tầng vượt phạm vi đề án	Trung bình
PA-C: Kiến trúc hướng sự kiện, tự triển khai	MQTT Broker + MQTT Bridge + hai lớp lưu trữ dữ liệu + API + WebSocket	Phù hợp với IoT cần cập nhật nhanh, tách tải tốt, mở rộng theo từng dịch vụ và tự chủ chi phí	Cần chuẩn hóa luồng bản tin và quản lý nhiều dịch vụ Docker	Cao (Đã chọn)

3.3.1.3. Chọn giải pháp kiến trúc hệ thống máy chủ Đề án chọn **PA-C: kiến trúc phân tầng kết hợp phản ứng theo sự kiện**, tức khi có bản tin mới thì từng lớp liên quan xử lý ngay, với chuỗi xử lý:

Device -> EMQX -> MQTT Bridge -> PostgreSQL/

VictoriaMetrics/VictoriaLogs -> API phía máy chủ -> giao diện web

Lựa chọn này xuất phát từ cách dữ liệu thực sự di chuyển trong hệ thống: thiết bị gửi bản tin liên tục, lớp tiếp nhận dữ liệu đầu vào tiếp nhận và phân luồng, lớp lưu trữ ghi theo đúng đặc tính dữ liệu, còn lớp ứng dụng phục vụ người dùng và nghiệp vụ. Nói cách khác, kiến trúc được chọn vì nó bám sát hành trình tự nhiên của dữ liệu thay vì dồn mọi trách nhiệm về một dịch vụ duy nhất.

Các lý do lựa chọn chính gồm:

- **Khớp bản chất dữ liệu IoT:** MQTT xử lý tốt mô hình pub/sub, tức mô hình “một bên phát bản tin vào kênh chung, nhiều bên có thể nhận”, nên giảm mức phụ thuộc

trực tiếp giữa thiết bị và tầng ứng dụng. Điều này giúp thiết bị chỉ tập trung gửi dữ liệu, thay vì phải biết chi tiết ai sẽ nhận và xử lý dữ liệu đó ở phía sau.

- **Tối ưu hiệu năng theo vai trò:** Bridge xử lý luồng dữ liệu đi vào đầu tiên, API phía máy chủ tập trung vào nghiệp vụ, còn cơ sở dữ liệu được tách theo loại dữ liệu. Mỗi lớp chỉ làm đúng việc mình mạnh nhất, nhờ đó hệ thống dễ giữ ổn định hơn khi tải tăng.
- **Dễ mở rộng theo số lượng dịch vụ:** có thể tăng thêm bản sao riêng cho broker, bridge hoặc API khi số thiết bị tăng, thay vì phải mở rộng toàn bộ hệ thống một cách đồng loạt và tốn kém.
- **Phù hợp chi phí và phạm vi thực hiện:** tự dựng trên máy chủ bằng Docker Compose, không phụ thuộc nền tảng máy chủ thương mại, nên phù hợp với tính chất của một đồ án vừa cần triển khai thật, vừa cần kiểm soát tốt nguồn lực.

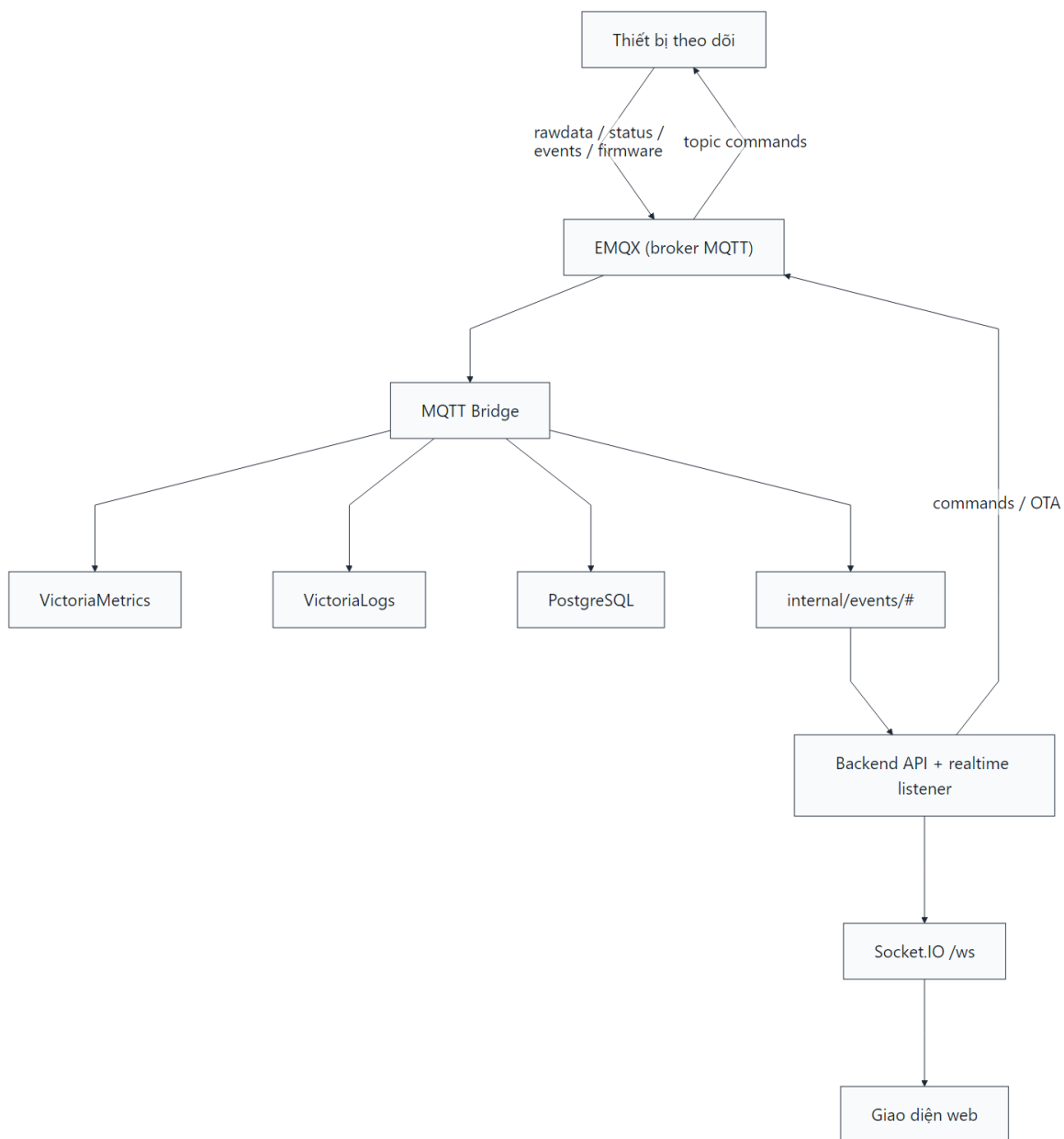
3.3.2. Giải pháp dịch vụ máy chủ và xử lý nghiệp vụ

Sau khi xác định phương án kiến trúc phù hợp, phần này đi vào cách triển khai cụ thể từng lớp của hệ thống máy chủ và dịch vụ xử lý. Cách trình bày được sắp xếp theo hành trình dữ liệu, từ lúc bản tin đi vào broker cho tới khi thông tin xuất hiện trên giao diện của người vận hành.

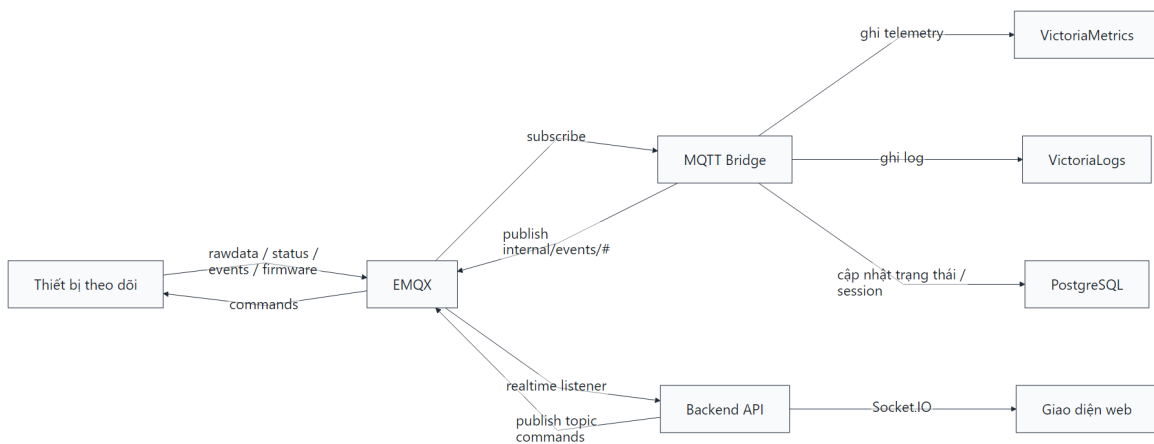
3.3.2.1. Kiến trúc tổng quan hệ thống máy chủ (System Architecture Overview)

a) Mô hình kiến trúc tổng thể Để người đọc dễ theo dõi, có thể hình dung hệ thống máy chủ như một chuỗi xử lý nhiều lớp: lớp đầu nhận dữ liệu, lớp giữa lưu và xử lý, lớp cuối cung cấp thông tin cho người dùng. Kiến trúc này vừa tận dụng được ưu thế của mô hình phản ứng theo sự kiện trong IoT, vừa giữ cho trách nhiệm của từng thành phần không bị chồng lấn. Cụ thể, hệ thống gồm các tầng chính sau:

- **Tầng tiếp nhận dữ liệu:** Đây là lớp đón dữ liệu đầu tiên từ thiết bị, nhận các bản tin MQTT rồi chuyển chúng đến đúng nơi cần xử lý.
- **Tầng lưu trữ dữ liệu:** Dữ liệu được tách ra hai nơi: một nơi lưu dữ liệu quản trị có quan hệ rõ ràng, một nơi lưu dữ liệu thay đổi liên tục theo thời gian. Cách chia này tránh ép mọi thứ vào cùng một cơ sở dữ liệu.
- **Tầng xử lý nghiệp vụ:** Đây là nơi xử lý các yêu cầu từ giao diện, áp quy tắc nghiệp vụ và quản lý trạng thái chung của hệ thống.
- **Tầng cập nhật thời gian thực:** Đây là nơi đẩy dữ liệu mới trực tiếp đến người dùng thông qua WebSocket, tức kênh để máy chủ chủ động gửi thông tin mới lên trình duyệt.



Hình 3.12: Sơ đồ kiến trúc tổng quan hệ thống máy chủ



Hình 3.12a: Luồng dữ liệu chi tiết từ thiết bị đến giao diện quản trị

b) Luồng dữ liệu chính trong hệ thống Nếu mô tả bằng một “câu chuyện dữ liệu”, luồng chính của hệ thống diễn ra theo trình tự sau:

1. **Thiết bị IoT** gửi dòng dữ liệu vận hành chính như vị trí GNSS, trạng thái nguồn, mức rung, ignition và uptime lên EMQX Broker thông qua MQTT 3.1.1 theo topic có cấu trúc `v1/{device_id}/rawdata`. Ở đây, EMQX Broker có thể hiểu là broker MQTT, tức nơi nhận và chuyển tiếp bản tin.
2. **EMQX Broker** tiếp nhận và phân phối bản tin đến các bên cần dùng dữ liệu. Rules Engine của EMQX chỉ đóng vai trò lọc hoặc bỏ trợ nhẹ; phần lớn nghiệp vụ cảnh báo vẫn được xử lý tại MQTT Bridge.
3. **MQTT Bridge** là dịch vụ trung gian đứng giữa broker và các nơi lưu dữ liệu. Dịch vụ này lắng nghe các topic từ EMQX, kiểm tra dữ liệu đầu vào bằng Zod schema, tức bộ quy tắc mô tả dữ liệu nào hợp lệ, sau đó ghi đồng thời vào nhiều nơi, nghĩa là một lần nhận dữ liệu nhưng lưu cho nhiều mục đích khác nhau:
 - Ghi dữ liệu thay đổi liên tục theo thời gian vào **VictoriaMetrics** (vị trí, tốc độ, điện áp, rung, ignition, uptime).
 - Ghi nhật ký sự kiện vào **VictoriaLogs** (kết nối/ngắt kết nối, lỗi, sự kiện MQTT).
 - Cập nhật trạng thái thiết bị vào **PostgreSQL** (trạng thái trực tuyến, phiên làm việc).
 - Phát sự kiện đến **Socket.IO** để cập nhật thời gian thực cho giao diện quản trị.
4. **API phía máy chủ** (Express.js) là lớp để giao diện gọi vào khi cần lấy dữ liệu lịch sử, dữ liệu quản trị hoặc gửi lệnh. Dịch vụ này truy vấn dữ liệu từ PostgreSQL và VictoriaMetrics, đồng thời gửi lệnh điều khiển ngược về thiết bị thông qua EMQX.
5. **Giao diện web** (Next.js) nhận dữ liệu cập nhật thời gian thực qua Socket.IO và dữ liệu lịch sử qua REST API.

c) Mô hình triển khai Docker Ở mức triển khai, kiến trúc trên được hiện thực theo cách mỗi dịch vụ có một tệp Docker Compose riêng và cùng tham gia một mạng Docker dùng chung. Cách tổ chức này giúp cấu trúc vận hành của đồ án bám sát cách các hệ thống thật được chia dịch vụ. Hiểu đơn giản, mỗi dịch vụ được đặt trong một “hộp chạy riêng” gọi là container; các hộp này có thể nói chuyện với nhau qua mạng dùng chung, nhưng vẫn tách lỗi và tài nguyên tương đối rõ ràng. Nhờ vậy, cấu trúc vận hành của đồ án bám sát cách các hệ thống thật được chia dịch vụ trong thực tế:

- **Độc lập khi triển khai:** Mỗi dịch vụ có thể khởi động, dừng hoặc cập nhật độc lập mà không ảnh hưởng đến các dịch vụ khác.
- **Quản lý tài nguyên riêng:** Mỗi container, tức mỗi gói chạy tách biệt, được giới hạn CPU và RAM cụ thể (ví dụ: API phía máy chủ 512MB/1 CPU, MQTT Bridge 256MB/0.5 CPU).
- **Dễ dàng mở rộng:** Có thể nhân thêm bản sao cho từng dịch vụ riêng lẻ theo nhu

cầu.

Bảng 3.16: Phân bổ tài nguyên Docker cho các dịch vụ

Dịch vụ	Container	Port	Tài nguyên
Tracking_Backend	tracking-backend	4000	512MB, 1 CPU
Tracking_Frontend	tracking-frontend	4001	512MB, 1 CPU
Tracking_MqttBridge	tracking-mqtt-bridge	-	256MB, 0.5 CPU
Tracking_PostgreSQL	tracking-postgres	5432	1GB, 2 CPU
Tracking_EMQX	tracking-emqx	1883, 8883, 8083, 8084, 18083	1GB, 2 CPU
Tracking_VictoriaMetrics	tracking-victoriametrics	8428	512MB, 1 CPU
Tracking_VictoriaLogs	tracking-victorialogs	9428	512MB, 1 CPU
Tracking_Grafana	tracking-grafana	4002	Chưa cấu hình cứng

3.3.2.2. MQTT Broker - EMQX (lựa chọn và cấu hình)

a) Vai trò của MQTT Broker MQTT Broker là điểm vào đầu tiên của dữ liệu ở phía máy chủ. Vai trò của nó có thể hiểu đơn giản là nơi nhận bản tin từ thiết bị rồi chuyển tiếp cho các thành phần cần xử lý tiếp theo. Nhờ mô hình **phát bản tin vào kênh chung rồi đăng ký nhận**, các thành phần trong hệ thống có thể giao tiếp bất đồng bộ mà không cần ghép chặt với nhau.

Trong hệ thống theo dõi phương tiện, cách tổ chức này đặc biệt quan trọng vì số lượng thiết bị có thể tăng dần theo thời gian, trong khi nhịp gửi dữ liệu lại thay đổi theo chế độ hoạt động của từng xe.

b) So sánh và lựa chọn MQTT Broker Để lựa chọn MQTT Broker phù hợp, đồ án đã đánh giá bốn giải pháp phổ biến trên thị trường:

Bảng 3.17: So sánh các MQTT Broker phổ biến

Tiêu chí	Mosquitto	EMQX	HiveMQ	VerneMQ
Kết nối đồng thời	1.000	100.000.000	200.000.000	10.000.000
Thông lượng bản tin	40K msg/s	100K msg/s	200K msg/s	50K msg/s
Độ trễ xử lý	0,25 ms	0,27 ms	<1 ms	2,1 ms
Sử dụng CPU	Rất thấp	Thấp (2%)	Thấp	Cao (10%)
Sử dụng RAM	254 MB	495 MB	Trung bình	1,2 GB
Hỗ trợ chạy nhiều nút	Không	Có (20+ node)	Có	Có
Khả dụng cao (HA)	Không	Có	Có	Có
Độ phức tạp cài đặt	Rất dễ	Trung bình	Khó	Trung bình
Chi phí	Miễn phí	Miễn phí/Trả phí	Trả phí	Miễn phí/Trả phí
Cộng đồng	Lớn	Lớn	Trung bình	Nhỏ

Kết luận lựa chọn: EMQX

EMQX được lựa chọn làm MQTT Broker cho hệ thống vì đây là phương án giữ được thể cân bằng tốt giữa hiệu năng, khả năng mở rộng và mức độ phù hợp với nguồn lực triển khai của đồ án. Các lý do chính gồm:

- **Hiệu năng phù hợp:** Theo tài liệu công bố của EMQX, broker đáp ứng tốt tải bản tin lớn với độ trễ thấp, đủ cho quy mô triển khai hiện tại và còn dư địa mở rộng.
- **Khả năng mở rộng:** Hỗ trợ chạy nhiều nút cùng lúc, thuận lợi nếu hệ thống tăng số lượng thiết bị trong giai đoạn sau.
- **Bộ lọc bản tin tích hợp:** Thuận tiện cho các tác vụ lọc hoặc chuyển đổi dữ liệu đơn giản ngay tại broker; tuy nhiên trong kiến trúc hiện tại, đây không phải lớp xử lý nghiệp vụ chính.
- **Chi phí phù hợp và hệ sinh thái phong phú:** Phiên bản mã nguồn mở đáp ứng đủ nhu cầu của đồ án và có cộng đồng hỗ trợ rộng.
- **Màn hình quản lý:** Cung cấp giao diện web quản lý tại port 18083, hỗ trợ giám sát kết nối, topic và bản tin theo thời gian thực.

c) Cấu hình bộ lọc dữ liệu của EMQX Trong hệ thống này, bộ lọc dữ liệu của EMQX chỉ được dùng cho các phép xử lý nhẹ ngay tại broker, chẳng hạn nhận biết sớm một số điều kiện bất thường trước khi chuyển bản tin tới lớp xử lý phía sau. Phần lớn quyết định nghiệp vụ, đặc biệt là ghi nhận cảnh báo và đồng bộ giao diện, vẫn do MQTT Bridge và

API phía máy chủ đảm nhiệm.

Các tình huống phù hợp để cấu hình tại lớp này gồm:

- **Phát hiện rung mạnh:** nhận biết sớm các bản tin cảnh báo rung để chuyển sang luồng xử lý cảnh báo.
- **Phát hiện vượt tốc độ:** đánh dấu các bản tin có tốc độ vượt ngưỡng cấu hình để lớp phía sau tiếp tục xác minh và lưu vết.
- **Phát hiện pin dự phòng thấp:** gắn cờ các bản tin cho thấy điện áp pin dự phòng xuống thấp để ưu tiên theo dõi.

Luồng xử lý tổng quát:

1. Thiết bị phát bản tin dữ liệu vận hành và bản tin sự kiện lên broker MQTT.
2. Bộ lọc tại broker chỉ thực hiện bước nhận biết điều kiện ban đầu và chuyển tiếp bản tin phù hợp sang lớp xử lý kế tiếp.
3. MQTT Bridge đánh giá nghiệp vụ chi tiết, chuẩn hóa dữ liệu và phát sự kiện cho API phía máy chủ.
4. API phía máy chủ ghi dữ liệu vào PostgreSQL và phát Socket.IO để cập nhật giao diện quản trị theo thời gian thực.

Ngoài ra, hệ thống có thể mở rộng thêm quy tắc vùng cho phép ở mức cơ bản, thống kê traffic theo topic, hoặc phát hiện thiết bị im lặng bất thường để phục vụ vận hành.

3.3.2.3. Thiết kế cơ sở dữ liệu (Database Architecture Design) Đặt vấn đề lưu trữ:

Khi đi sâu hơn vào tầng dữ liệu, có thể thấy hệ thống đang phục vụ hai nhu cầu rất khác nhau. Một mặt, dữ liệu vận hành gửi liên tục cần được ghi nhanh và truy vấn theo thời gian. Mặt khác, dữ liệu nghiệp vụ đòi hỏi tính toàn vẹn, quan hệ rõ ràng và khả năng quản lý dài hạn. Nếu dùng một công cụ cho cả hai loại bài toán, hệ thống thường sẽ phải đánh đổi hiệu quả ở một trong hai phía.

Bảng 3.18A: So sánh các phương án kiến trúc lưu trữ dữ liệu

Phương án	Mô tả	Ưu điểm	Hạn chế	Mức phù hợp
PA-DB1: PostgreSQL duy nhất	Dùng PostgreSQL cho cả nghiệp vụ và dữ liệu vận hành gửi liên tục	Đơn giản vận hành	Ghi lượng dữ liệu lớn dễ ảnh hưởng truy vấn nghiệp vụ, chi phí index/partition cao	Trung bình

Bảng 3.18A: So sánh các phương án kiến trúc lưu trữ dữ liệu (tiếp theo)

Phương án	Mô tả	Ưu điểm	Hạn chế	Mức phù hợp
PA-DB2: Time-series DB duy nhất	Dùng CSDL chuỗi thời gian cho mọi loại dữ liệu	Tối ưu ghi dữ liệu cảm biến	Kém phù hợp dữ liệu quan hệ phức tạp, khó đảm bảo ràng buộc nghiệp vụ	Thấp
PA-DB3: Hybrid DB (Đã chọn)	PostgreSQL + VictoriaMetrics, VictoriaLogs	Mỗi loại dữ liệu dùng đúng công cụ, cân bằng hiệu năng và khả năng truy vấn	Tăng số thành phần cần vận hành	Cao

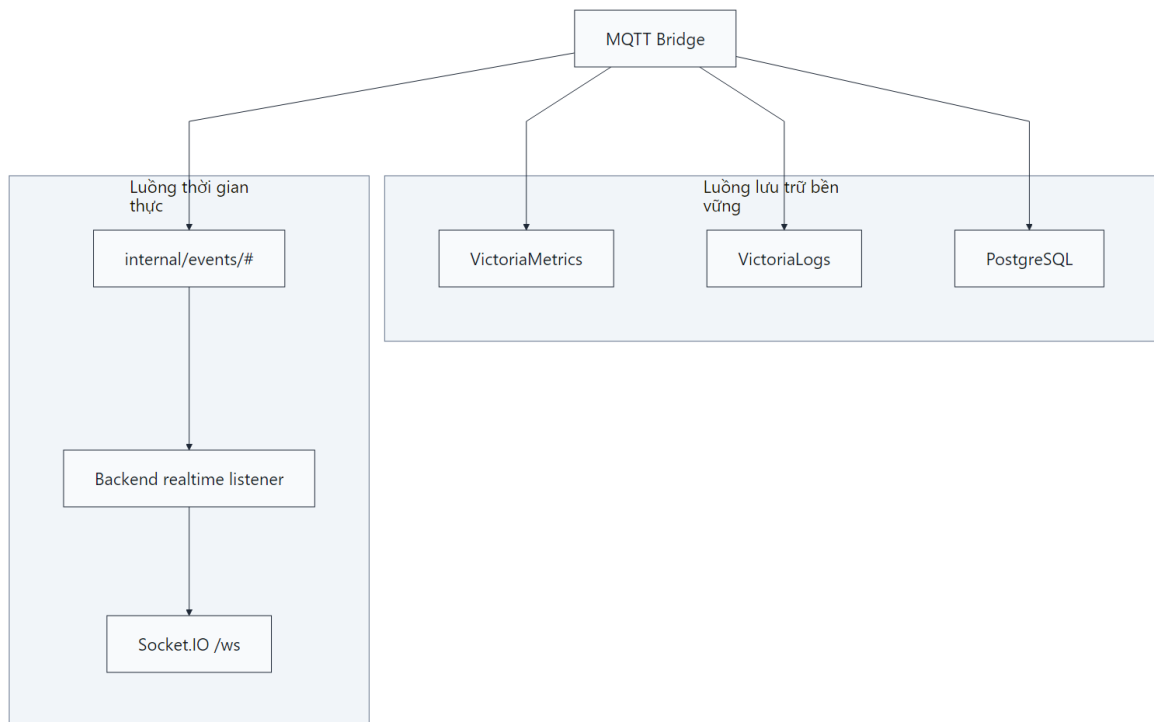
Kết luận lựa chọn: Chọn **PA-DB3 (hybrid database)** để tách trách nhiệm dữ liệu, giữ ổn định cho nghiệp vụ và tối ưu luồng dữ liệu vận hành theo thời gian thực.

a) Chiến lược lưu trữ kép (Dual Database Strategy) Hệ thống theo dõi phương tiện cần lưu trữ hai loại dữ liệu có đặc tính khác nhau căn bản. Có thể hình dung rất đơn giản: một bên là dữ liệu “chảy liên tục” từ xe, bên còn lại là dữ liệu “quản lý nghiệp vụ”. Nếu cố dồn cả hai vào cùng một kiểu cơ sở dữ liệu, hệ thống sẽ sớm gặp xung đột giữa nhu cầu ghi nhanh và nhu cầu truy vấn quan hệ chặt chẽ. Vì vậy, việc tách lớp lưu trữ ở đây không phải để làm kiến trúc phức tạp hơn, mà để mỗi loại dữ liệu được đưa về đúng nơi phù hợp:

- Dữ liệu thô (dữ liệu vận hành nguyên bản):** Bao gồm vị trí GPS, tốc độ, mức pin, dữ liệu OBD2, dữ liệu cảm biến gia tốc. Loại dữ liệu này có tần suất ghi rất cao (mỗi giây khi xe đang di chuyển), khối lượng lớn, nhưng chỉ cần lưu trữ trong thời gian ngắn (7–30 ngày) và chủ yếu phục vụ truy vấn theo chuỗi thời gian.
- Dữ liệu nghiệp vụ:** Bao gồm thông tin xe, khách hàng, chuyến đi, cảnh báo, vi phạm, lệnh điều khiển. Loại dữ liệu này có tần suất ghi thấp, khối lượng nhỏ, nhưng cần lưu trữ lâu dài (6–12 tháng trở lên) và yêu cầu tính toàn vẹn dữ liệu quan hệ (ACID).

Do sự khác biệt cơ bản về đặc tính, hệ thống áp dụng chiến lược lưu trữ kép sử dụng ba cơ sở dữ liệu chuyên biệt. Trong đó, VictoriaLogs được thêm như một lớp riêng cho nhật ký vận hành, giúp việc truy vết lỗi không làm ảnh hưởng tới đường lưu dữ liệu nghiệp vụ hay dữ liệu vận hành gửi liên tục:

- **PostgreSQL (quan hệ):** lưu dữ liệu nghiệp vụ như vehicles, customers, trips, alerts, violations, commands, geofences.



Hình 3.13: Sơ đồ chiến lược lưu trữ kép

- **VictoriaMetrics (chuỗi thời gian):** lưu dữ liệu vận hành mật độ cao như vị trí GPS, tốc độ, mức pin, trạng thái bật/tắt máy, thời gian hoạt động liên tục và dữ liệu IMU (retention khoảng 30 ngày).
- **VictoriaLogs (nhật ký):** lưu sự kiện thiết bị, lỗi, phiên làm việc, bản tin MQTT và nhật ký kiểm toán (retention khoảng 7 ngày).

Bảng 3.18: So sánh các cơ sở dữ liệu trong hệ thống

Cơ sở dữ liệu	Mục đích	Ưu điểm chính
PostgreSQL 16	Dữ liệu quan hệ (nghệ vụ)	Bảo toàn tính nhất quán, liên kết tốt giữa các bảng và truy vấn phức tạp
VictoriaMetrics	Dữ liệu chuỗi thời gian (dữ liệu vận hành gửi liên tục)	Ghi nhanh, truy vấn theo mốc thời gian tốt, tiêu thụ RAM thấp
VictoriaLogs	Nhật ký tập trung	Tìm kiếm nhanh, lưu trữ nhỏ gọn, phù hợp truy vết sự cố

b) Cơ sở dữ liệu PostgreSQL - Schema quan hệ PostgreSQL 16 lưu trữ toàn bộ dữ liệu nghiệp vụ của hệ thống. Schema được chuẩn hóa đến dạng chuẩn thứ ba (3NF) và hỗ trợ soft delete cho các bảng quan trọng.

Các nhóm bảng chính (Phase 1 - khoảng 20 bảng):

Bảng 3.19: Các nhóm bảng chính trong PostgreSQL

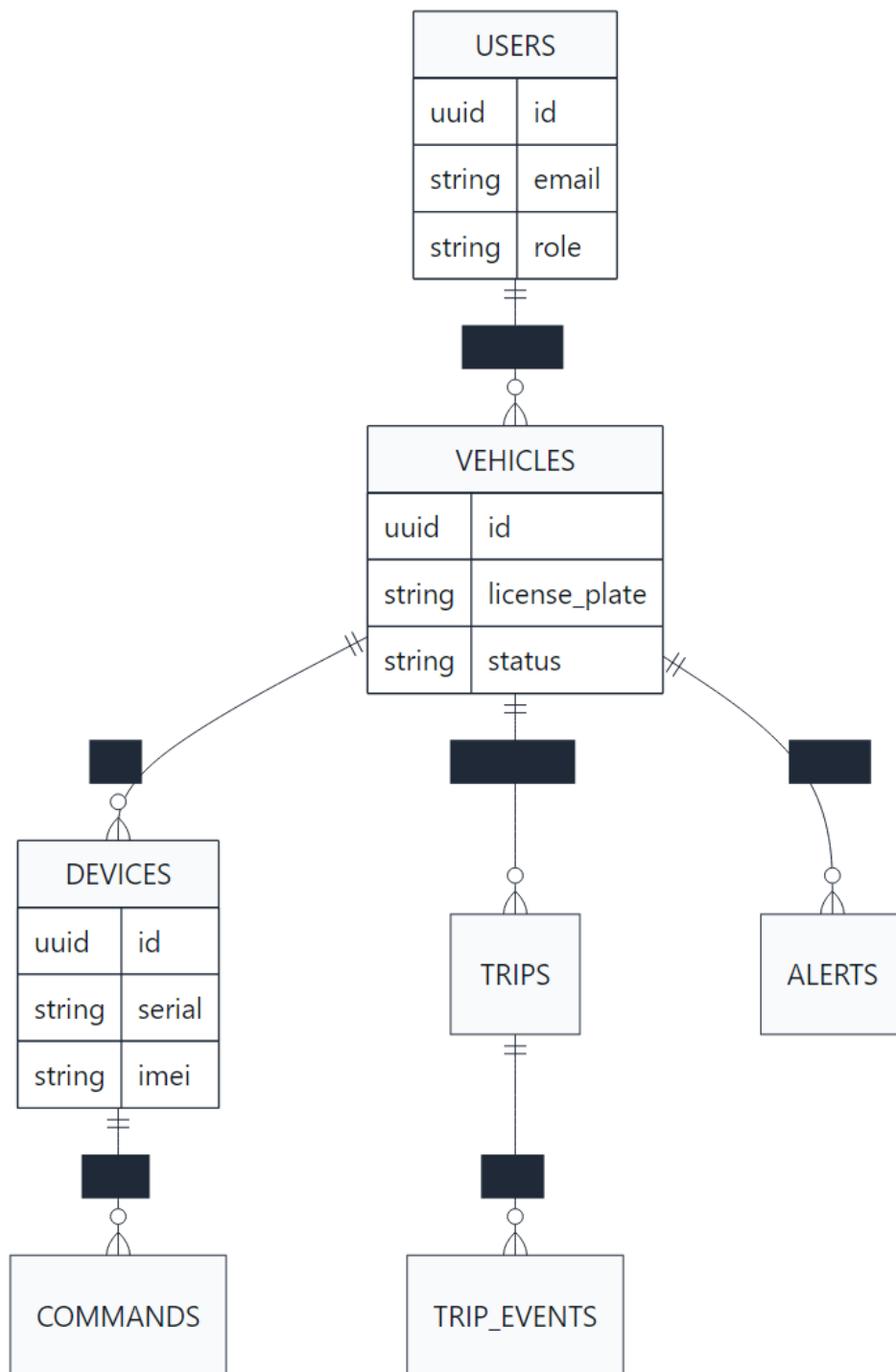
Nhóm bảng	Các bảng chính	Mô tả
Người dùng & Xác thực	users, user_sessions	Quản lý tài khoản, phiên đăng nhập
Phương tiện & Thiết bị	vehicles, devices, device_configurations	Thông tin xe và thiết bị theo dõi
Khách hàng	customers	Thông tin khách hàng thuê xe
Chuyến đi	trips, trip_events, stops	Hành trình, sự kiện và điểm dừng
Cảnh báo & Vi phạm	alerts, violations	Cảnh báo hệ thống và vi phạm giao thông
Vùng địa lý	geofences, vehicle_geofences	Vùng địa lý và liên kết với xe
Lệnh điều khiển	commands	Lệnh gửi đến thiết bị
Bảo trì	maintenance_records	Lịch sử bảo trì phương tiện
Nhật ký	connection_logs, device_status_history	Nhật ký kết nối và lịch sử trạng thái
Thông báo	notifications	Thông báo hệ thống
Nhiên liệu	fuel_records	Quản lý nhiên liệu

Sơ đồ quan hệ chính (ER rút gọn):

- users 1:N vehicles, customers, notifications.
- vehicles 1:1 devices; 1:N trips, alerts, maintenance_records; N:N geofences (qua vehicle_geofences).
- customers 1:N trips.
- trips 1:N trip_events, stops, violations.
- devices 1:N commands, device_status_history, connection_logs.

Schema được thiết kế với khả năng mở rộng, các khóa ngoại và chỉ mục (index) đã được chuẩn bị sẵn để tích hợp thêm các bảng của giai đoạn 2 bao gồm: bookings (đặt xe), rental_contracts (hợp đồng thuê), payments (thanh toán), damage_reports (báo cáo hư hỏng) và reviews (đánh giá).

c) Cơ sở dữ liệu VictoriaMetrics - Schema chuỗi thời gian VictoriaMetrics là cơ sở dữ liệu chuỗi thời gian tương thích Prometheus, được lựa chọn thay thế InfluxDB nhờ hiệu năng vượt trội (nhanh hơn 10 lần), mức tiêu thụ RAM thấp hơn và tích hợp tốt với



Hình 3.14: Sơ đồ quan hệ cơ sở dữ liệu (ER)

Grafana.

Bảng 3.20: So sánh VictoriaMetrics và InfluxDB

Tiêu chí	InfluxDB	VictoriaMetrics (Đã chọn)
Ngôn ngữ truy vấn	Flux (phức tạp)	PromQL (đơn giản)
Hiệu năng	Tốt	Nhanh gấp 10 lần
Sử dụng RAM	Cao	Thấp
Nén dữ liệu	Tốt	Tốt hơn (gấp 10 lần)
Tương thích Prometheus	Không	Có
Độ khó học	Cao (Flux)	Thấp (PromQL)
Tích hợp Grafana	Qua plugin	Hỗ trợ sẵn (Native)

Chính sách lưu trữ (Retention Policy): Dữ liệu thô được lưu trữ trong 30 ngày (-retentionPeriod=30d). Hệ thống hỗ trợ cấu hình downsampling thông qua vmagent với recording rules để tạo dữ liệu tổng hợp theo giờ/ngày, phục vụ báo cáo dài hạn mà không tốn dung lượng lưu trữ lớn.

d) VictoriaLogs - Cơ sở dữ liệu nhật ký VictoriaLogs được sử dụng làm hệ thống nhật ký tập trung, tức nơi gom log từ nhiều dịch vụ về một chỗ để dễ theo dõi, thay thế bộ công cụ ELK Stack truyền thống với ưu điểm nhẹ hơn và dễ triển khai. VictoriaLogs lưu trữ các loại nhật ký: sự kiện thiết bị, nhật ký lỗi, phiên làm việc, bản tin MQTT, và nhật ký kiểm toán, tức dấu vết ai đã làm gì và vào lúc nào. Chính sách lưu trữ: 7 ngày cho nhật ký thông thường.

Đến đây, kiến trúc lưu trữ đã trả lời câu hỏi dữ liệu sẽ được cất ở đâu. Tuy nhiên, để người dùng khai thác được số dữ liệu đó, hệ thống vẫn cần một lớp nghiệp vụ đứng ra tổ chức API, quyền truy cập và các luồng điều khiển. Vì vậy, phần tiếp theo chuyển sang API phía máy chủ.

3.3.2.4. API phía máy chủ và kiến trúc phần mềm (API Server & Software Architecture)

a) Lựa chọn công nghệ API phía máy chủ là lớp làm việc trực tiếp với người vận hành và các quy tắc nghiệp vụ. Nếu MQTT Bridge chuyên nhận dòng dữ liệu thô từ thiết bị, thì API phía máy chủ phụ trách những gì người dùng nhìn thấy hằng ngày: đăng nhập, quản lý xe, xem lịch sử, tạo cảnh báo, quản lý vùng cho phép và điều phối OTA. Vì vai trò khác nhau, tiêu chí chọn framework ở đây cũng khác: quan trọng nhất là mã nguồn

phải rõ ràng, linh hoạt và dễ bảo trì, chứ không chỉ có kết quả đo tốc độ đẹp trên các bài thử chuẩn.

Bảng 3.21: So sánh các khung phát triển cho API phía máy chủ

Tiêu chí	Express + TypeScript	NestJS	Go + Gin
Độ khó học	Rất dễ	Trung bình	Khó
Linh hoạt	Rất cao	Trung bình	Cao
Hiệu năng	Cao	Cao	Rất cao
Lượng mã khung lặp lại	Ít	Nhiều	Trung bình
Hệ sinh thái IoT	Rất tốt	Tốt	Tốt
Hỗ trợ thời gian thực	Rất tốt	Rất tốt	Tốt

Kết luận lựa chọn: Node.js + Express + TypeScript

Lựa chọn này phù hợp vì ba lý do dễ thấy. Thứ nhất, Express nhẹ và ít ràng buộc, nên nhóm có thể tổ chức mã nguồn bám theo nghiệp vụ thay vì phải đi theo khuôn sẵn có của framework. Thứ hai, TypeScript giúp phát hiện sớm sai khác kiểu dữ liệu giữa lớp nhận yêu cầu, lớp xử lý nghiệp vụ và cơ sở dữ liệu, nhờ đó giảm lỗi lan xuống sâu. Thứ ba, Zod kiểm tra dữ liệu đầu vào ngay khi chương trình chạy, tức là dữ liệu sai cấu trúc sẽ bị chặn từ cửa vào. Với một hệ thống phải nhận dữ liệu từ giao diện quản trị, thiết bị và luồng OTA, ba yếu tố này thực dụng hơn việc chọn một framework nhiều tính năng nhưng nặng khuôn mẫu.

b) Kiến trúc Domain-Driven Design (DDD) API phía máy chủ được tổ chức theo kiến trúc Domain-Driven Design với ba tầng rõ ràng. Trong phạm vi đề án, DDD có thể hiểu đơn giản là chia mã nguồn theo từng bài toán nghiệp vụ thay vì gom theo loại file kỹ thuật. Nhờ đó, phần quản lý xe, thiết bị, dữ liệu vận hành gửi liên tục hay cảnh báo được tách thành các miền tương đối độc lập; khi thay đổi một nghiệp vụ cụ thể, phạm vi ảnh hưởng cũng dễ kiểm soát hơn.

Tầng 1 - Lớp nhận yêu cầu (API Layer / Routes): Định nghĩa các đường gọi API, nơi ánh xạ từng yêu cầu HTTP tới bộ xử lý tương ứng.

Tầng 2 - Lớp nghiệp vụ (Service Layer): Chứa toàn bộ logic nghiệp vụ, điều phối giữa các nơi truy cập dữ liệu và các dịch vụ bên ngoài.

Tầng 3 - Lớp truy cập dữ liệu (Repository Layer): Truy cập cơ sở dữ liệu, thực hiện các truy vấn SQL và trả về dữ liệu đã được kiểm tra kiểu.

Hệ thống vì thế được chia thành hơn 20 domain module, mỗi module giữ trọn phần việc của mình với các thư mục `services/`, `repositories/`, `types/` riêng. Cách chia này

giúp người đọc lần theo từng nghiệp vụ rõ hơn và giảm việc sửa chéo giữa các phần không liên quan.

c) MQTT Bridge - Dịch vụ độc lập MQTT Bridge đã được tách thành một dịch vụ Node.js độc lập với cơ chế ghi log riêng, nhóm kết nối cơ sở dữ liệu dùng lại nhiều lần và cấu hình riêng. Cách tách này giúp dịch vụ dễ mở rộng hơn, cô lập lỗi tốt hơn và giữ kiến trúc tổng thể gọn hơn.

Khi cấu trúc dịch vụ đã rõ, bước tiếp theo là chốt “hợp đồng giao tiếp” mà giao diện và các công cụ vận hành sẽ dùng hằng ngày. Vì vậy, mục sau tập trung vào thiết kế API endpoint.

3.3.2.5. Thiết kế API Endpoints (REST API Design)

a) Nguyên tắc thiết kế API được thiết kế theo kiểu REST, nghĩa là tổ chức theo từng tài nguyên như xe, thiết bị hay cảnh báo: URL dạng số nhiều, gắn phiên bản ở đầu đường dẫn (/api/v1/), có phân trang, định dạng lỗi thống nhất và dùng HTTP Status Code chuẩn. Ở góc nhìn vận hành, API chính là “hợp đồng giao tiếp” giữa giao diện quản trị, API phía máy chủ và một số luồng thiết bị. Vì vậy, cách đặt tên và phân nhóm endpoint phải giúp người đọc đoán được ngay endpoint đó phục vụ ai, thao tác gì và có cần đăng nhập hay không.

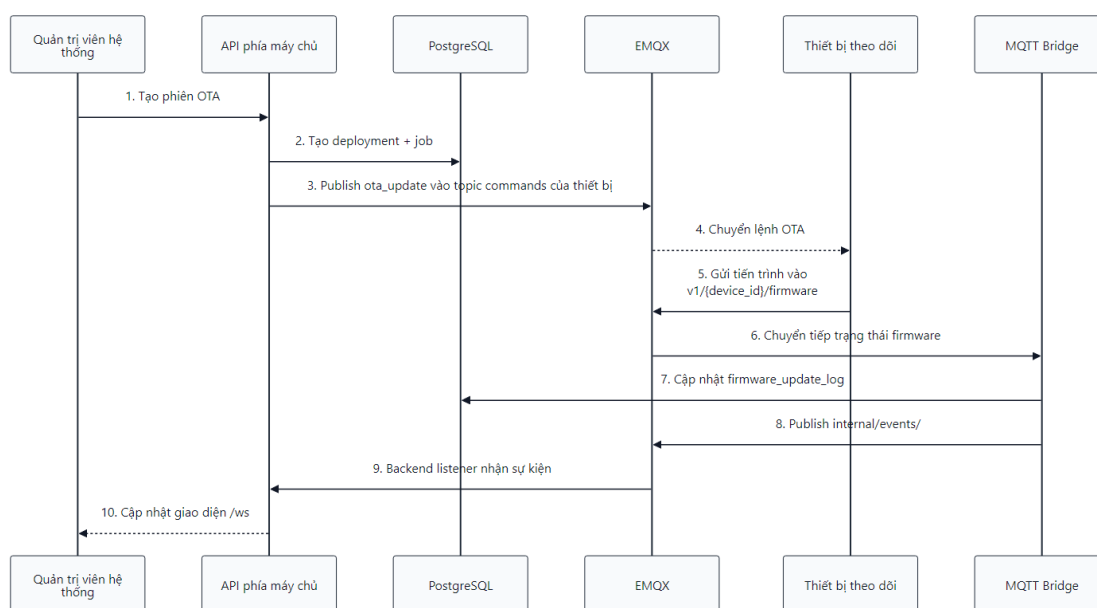
b) Bảng tóm tắt API Endpoints

Bảng 3.22: Tóm tắt các nhóm API Endpoints chính

Nhóm	Phương thức	Endpoint	Mô tả	Xác thực
Đăng nhập	POST	/api/v1/auth/login	Đăng nhập	Không
	GET	/api/v1/auth/me	Thông tin người dùng hiện tại	Có
	POST	/api/v1/auth/logout	Đăng xuất	Có
Thiết bị	GET	/api/v1/devices	Danh sách thiết bị	Có
	POST	/api/v1/devices	Tạo thiết bị mới	Có
	POST	/api/v1/devices/:id/ota	Gửi lệnh OTA đến thiết bị	Có

Bảng 3.22: Tóm tắt các nhóm API Endpoints chính (tiếp theo)

Nhóm	Phương thức	Endpoint	Mô tả	Xác thực
	POST	/api/v1/devices/:id/ota/rollback	Yêu cầu rollback firmware	Có
Phương tiện	GET	/api/v1/vehicles	Danh sách xe	Có
	POST	/api/v1/vehicles	Tạo xe mới	Có
Dữ liệu IoT	POST	/api/v1/iot/data	Gửi dữ liệu cảm biến	Không
Dữ liệu vận hành	GET	/api/v1/devices/:id/telemetry	Dữ liệu vận hành theo từng thiết bị	Có
	GET	/api/v1/telemetry/history	Truy vấn lịch sử dữ liệu vận hành	Có
Tổng quan	GET	/api/v1/dashboard/stats	Thống kê tổng quan	Có
Firmware	POST	/api/v1/firmware/upload	Tải lên firmware	Có
	GET	/api/v1/firmware/:id/download	Tải file firmware để OTA	Có



Hình 3.14a: Luồng điều phối OTA giữa API phía máy chủ, EMQX, thiết bị và MQTT Bridge

API phía máy chủ vẫn duy trì một số alias tương thích như `/api/v1/device` hoặc `/api/v1/export`, nhưng tài liệu ưu tiên dạng tài nguyên số nhiều (`/devices`, `/vehicles`, `/customers`, `/trips`, `/alerts`, `/geofences`) để thống nhất cách đặt tên. Cách trình bày này giúp người không quen với kiểu API dạng tài nguyên vẫn có thể hiểu nhanh nhóm endpoint nào phục vụ đăng nhập, nhóm nào phục vụ quản lý xe, nhóm nào dành cho dữ liệu vận hành hay OTA.

Khi “cửa vào” của hệ thống đã rõ bằng các endpoint cụ thể, phần còn lại là cách dữ liệu mới được đẩy ngược ra giao diện mà không buộc người dùng phải tải lại trang liên tục. Vì vậy, mục tiếp theo chuyển sang kênh WebSocket thời gian thực.

3.3.2.6. WebSocket và truyền dữ liệu thời gian thực (Cập nhật Communication)

Hệ thống sử dụng **Socket.IO 4.8** như một kênh để máy chủ chủ động gửi dữ liệu mới ra giao diện web ngay khi có thay đổi, thay vì bắt giao diện quản trị hỏi lại liên tục. Trong triển khai hiện tại, kênh này được chia thành năm nhóm chính: `/dashboard` cho số liệu tổng quan, `/devices` cho trạng thái theo từng xe, `/notifications` cho cảnh báo, `/exports` cho tiến trình xuất báo cáo và `/firmware` cho luồng OTA. Tất cả các nhóm đều đi qua xác thực; riêng `/devices` cho phép vào “phòng” theo từng thiết bị để chỉ nhận những cập nhật liên quan.

Các sự kiện tiêu biểu gồm cập nhật trạng thái xe, vị trí xe, xác nhận lệnh, số liệu tổng quan, cảnh báo vượt vùng cho phép, tiến trình xuất báo cáo và phân công firmware. Riêng luồng OTA được tách khá rõ: API phía máy chủ phát các sự kiện mức nghiệp vụ để giao diện biết có nhiệm vụ mới, còn trạng thái chi tiết trong quá trình cập nhật được MQTT Bridge ghi vào `firmware_update_log` và `VictoriaLogs` để thuận tiện cho việc theo dõi và truy vết khi có lỗi.

Khi dữ liệu đã đi được đến màn hình quản trị theo thời gian thực, phần còn lại là bảo đảm chỉ đúng người, đúng thiết bị và đúng luồng dữ liệu được phép đi qua. Vì vậy, mục kế tiếp chuyển sang thiết kế bảo mật toàn hệ thống.

3.3.2.7. Bảo mật hệ thống

a) Xác thực theo phiên làm việc (không dùng JWT) Hệ thống sử dụng cơ chế xác thực dựa trên phiên làm việc, nghĩa là trạng thái đăng nhập được giữ ở phía máy chủ. Mỗi lần đăng nhập thành công, máy chủ tạo ra một mã phiên ngẫu nhiên và lưu thông tin phiên tương ứng trong cơ sở dữ liệu, thay vì dùng JWT tự mang toàn bộ thông tin phiên. Token chỉ đóng vai trò khóa tham chiếu; API phía máy chủ mới là nơi lưu và kiểm tra trạng thái phiên, còn giá trị token được băm SHA-256 trước khi ghi xuống cơ sở dữ liệu.

Bảng 3.23: So sánh JWT và cơ chế xác thực theo phiên làm việc

Tiêu chí	JWT	Theo phiên làm việc (đã chọn)
Thu hồi token	Khó (cần blacklist)	Dễ (xóa khỏi DB)
Kiểm soát phiên	Hạn chế	Toàn quyền (xem, thu hồi mọi phiên)
Bảo mật	Token chứa thông tin nhạy cảm	Chỉ chứa ID ngẫu nhiên
Kích thước	Lớn (gồm phần đầu, phần dữ liệu và chữ ký số)	Nhỏ (chỉ mã phiên)

b) Bảo mật tầng truyền tải Hệ thống bảo vệ tầng truyền tải bằng nhiều lớp bảo trợ: Helmet để thiết lập các header bảo mật sẵn trên phản hồi web, CORS để giới hạn website nào được phép gọi sang hệ thống, giới hạn tần suất yêu cầu để chặn việc gửi quá nhiều yêu cầu trong thời gian ngắn từ cùng một địa chỉ IP, và cơ chế gắn mã nhận diện từng yêu cầu để truy vết luồng xử lý khi cần dò lỗi.

c) Bảo mật MQTT Ở tầng MQTT, mỗi thiết bị được cấp ACL riêng và Client ID duy nhất, kết hợp xác thực bằng username/password. Cách làm này giúp cô lập phạm vi publish/subscribe của từng thiết bị theo dõi và giảm rủi ro một thiết bị truy cập nhầm sang luồng dữ liệu của thiết bị khác.

d) Hệ thống thông báo cảnh báo Hệ thống hỗ trợ gửi thông báo qua Telegram Bot (thời gian thực) và Email SMTP (chi tiết). Các loại cảnh báo: xe di chuyển bất thường, vi phạm tốc độ, pin thấp, thiết bị mất kết nối, lịch bảo trì sắp đến.

Đến đây, tầng máy chủ đã chốt cách nhận dữ liệu, lưu dữ liệu, bảo vệ truy cập và phát thông tin ra ngoài. Phần tiếp theo vì thế chuyển sang tầng giao diện web, nơi toàn bộ dữ liệu đó được biến thành màn hình vận hành mà người dùng thực sự tương tác.

3.4. Phân tích và đề xuất công nghệ giao diện web – Web interface analysis and solution

3.4.1. Phân tích và lựa chọn công nghệ giao diện web

3.4.1.1. Đặt vấn đề cho giải pháp giao diện web Nếu tầng máy chủ là nơi xử lý dữ liệu, thì giao diện web là nơi người vận hành thực sự “nhìn thấy” hệ thống đang hoạt động ra sao. Một màn hình tổng quan tốt không chỉ hiển thị đủ thông tin, mà còn phải giúp người dùng nhận ra điều gì quan trọng, điều gì bất thường và cần hành động ở đâu. Vì vậy, bài toán thiết kế giao diện web cần đồng thời đáp ứng các yêu cầu sau:

- **Hiển thị thời gian thực ổn định:** bản đồ, trạng thái thiết bị và cảnh báo phải cập

nhật liên tục với độ trễ thấp.

- **Khả năng mở rộng theo tính năng nghiệp vụ:** quản lý xe, chuyển đi, vùng cho phép, tức vùng giám sát được vẽ trên bản đồ, cảnh báo và báo cáo cần để mở rộng.
- **Hiệu năng và trải nghiệm người dùng:** tải trang nhanh, điều hướng mượt, bố cục thích ứng tốt trên desktop/mobile.
- **Dễ triển khai vận hành:** quy trình đóng gói và chạy bằng Docker phải thuận lợi cho mô hình nhiều dịch vụ.

3.4.1.2. So sánh các phương án công nghệ giao diện web

Bảng 3.23A: So sánh các phương án công nghệ giao diện web tổng thể

Phương án	Mô tả bộ công cụ	Ưu điểm	Hạn chế	Mức phù hợp
PA-FE1: Vite + React chỉ dựng ở trình duyệt	React + Vite + định tuyến chạy phía trình duyệt	Build nhanh, cấu hình linh hoạt	Khả năng hiện nội dung sớm kém hơn cách dựng một phần ở máy chủ, cần tự lắp ghép nhiều thành phần	Trung bình
PA-FE2: Nuxt (Vue)	Vue + Nuxt với khả năng dựng ở máy chủ và dựng sẵn	Cấu trúc rõ, hỗ trợ hiển thị sớm tốt	Khác hệ sinh thái React đang dùng ở dự án, chi phí chuyển đổi cao	Trung bình
PA-FE3: Next.js + React (Đã chọn)	Next.js 15 + React 19 + App Router	Hỗ trợ tốt việc dựng trước một phần giao diện, hệ sinh thái lớn, tối ưu cho môi trường vận hành và tích hợp kênh thời gian thực thuận lợi	Độ phức tạp framework cao hơn cách dựng thuần ở trình duyệt	Cao

3.4.1.3. Chọn giải pháp giao diện web Đồ án chọn **PA-FE3: Next.js 15 + React 19** làm nền tảng giao diện web chính. Đây không phải là lựa chọn chạy theo xu hướng framework, mà là lựa chọn bám sát nhu cầu của một giao diện tổng quan phải vừa “đọc nhanh” vừa “phản ứng nhanh” khi dữ liệu và cảnh báo thay đổi liên tục.

Các lý do lựa chọn gồm:

- **Phù hợp yêu cầu dữ liệu lịch sử và dữ liệu thời gian thực cùng lúc:** kết hợp tốt giữa dữ liệu lấy theo yêu cầu rồi lưu tạm để dùng lại và kênh WebSocket để máy chủ chủ động gửi dữ liệu mới.
- **Hiệu năng tải trang tốt:** Cơ chế App Router của Next.js và khả năng dựng một phần giao diện từ phía máy chủ giúp người dùng thấy nội dung ban đầu sớm hơn.
- **Mở rộng tính năng thuận lợi:** dễ tổ chức theo từng nhóm tính năng người dùng thực sự thao tác, thay vì dồn tất cả mã nguồn vào một chỗ.
- **Sẵn sàng triển khai vận hành:** hỗ trợ xuất gói chạy độc lập và đóng gói Docker rõ ràng.

3.4.2. Giải pháp giao diện web

Phần 3.4.1 đã chốt bộ công cụ nền cho giao diện web. Từ điểm đó, mục này đi tiếp vào cách biến lựa chọn công nghệ thành một giao diện vận hành cụ thể: mã nguồn được tổ chức ra sao, dữ liệu được lấy và cập nhật thế nào, và từng màn hình phục vụ nhu cầu gì của người dùng.

3.4.2.1. Lựa chọn công nghệ giao diện web

a) Framework ứng dụng web Để lựa chọn framework phù hợp, nhóm không dừng ở so sánh tên công nghệ, mà đặt câu hỏi thực tế hơn: framework nào giúp giao diện quản trị tải nhanh, tổ chức rõ và vẫn dễ mở rộng khi số lượng trang quản trị tăng lên. Từ đó, nhóm đối chiếu Next.js, Nuxt.js và Vite + React theo các tiêu chí hiệu năng, khả năng mở rộng và trải nghiệm phát triển.

Bảng 3.24: So sánh các khung phát triển giao diện web

Tiêu chí	Next.js 15	Nuxt.js 3	Vite + React
Ngôn ngữ cơ sở	React 19 + TypeScript	Vue 3 + TypeScript	React + TypeScript
Cách dựng trang	Có thể dựng ở máy chủ, dựng sẵn khi build hoặc kết hợp cả hai	Có thể dựng ở máy chủ hoặc dựng sẵn	Chủ yếu dựng ở phía trình duyệt
Định tuyến và bố cục nhiều tầng	Có sẵn App Router của Next.js	Có sẵn định tuyến theo file	Không có sẵn, cần thư viện bổ sung
Khả năng hiện nội dung sớm / hỗ trợ tìm kiếm	Tốt	Tốt	Kém hơn vì phần lớn nội dung phải chờ trình duyệt dựng xong

Bảng 3.24: So sánh các khung phát triển giao diện web (tiếp theo)

Tiêu chí	Next.js 15	Nuxt.js 3	Vite + React
Hệ sinh thái	Rất lớn (React ecosystem)	Lớn (Vue ecosystem)	Lớn (React ecosystem)
Hỗ trợ kênh thời gian thực	Tốt (Socket.IO, Server Actions)	Tốt	Tốt
Hiệu năng phát triển và build	Cao (công cụ dựng nhanh, hỗ trợ hiện sớm một phần trang)	Cao (bộ máy dựng của Nuxt)	Cao (công cụ đóng gói nhanh Vite)
Đóng gói để triển khai	Có sẵn Dockerfile, gói chạy độc lập	Có sẵn	Cần cấu hình thủ công
Cộng đồng & tài liệu	Rất lớn	Lớn	Lớn

Kết luận lựa chọn: Next.js 15 được chọn làm framework chính vì nó tạo được thể cân bằng tốt giữa trải nghiệm của người dùng cuối và khả năng tổ chức mã nguồn của đội phát triển. Các lý do cụ thể gồm:

- **Dựng một phần giao diện ngay từ máy chủ:** React Server Components (RSC), tức cơ chế cho phép xử lý và dựng sẵn một phần giao diện ở phía máy chủ, giúp giảm lượng JavaScript phải gửi về trình duyệt và tăng tốc độ tải trang đầu tiên.
- **Cơ chế chia trang theo thư mục:** App Router, tức cách Next.js tổ chức đường dẫn và bố cục theo cấu trúc thư mục, giúp quản lý trang, bố cục lồng nhau và khu vực được bảo vệ trực quan hơn.
- **Hệ sinh thái React:** Tận dụng được hệ sinh thái cực kỳ phong phú của React, bao gồm các thư viện cho lấy dữ liệu từ máy chủ (TanStack Query), quản lý trạng thái giao diện (Zustand), bản đồ (Leaflet) và biểu đồ (ECharts). Nói ngắn gọn, đây là nhóm công cụ đủ để giao diện vừa lấy dữ liệu lịch sử, vừa nhận dữ liệu mới, vừa hiển thị trực quan mà không phải tự viết mọi thứ từ đầu.
- **TypeScript tích hợp:** Hỗ trợ TypeScript từ gốc, đảm bảo an toàn kiểu dữ liệu xuyên suốt ứng dụng.
- **Sẵn sàng cho triển khai:** Có sẵn các cơ chế tối ưu cho môi trường vận hành thật như tối ưu ảnh, tách nhỏ mã nguồn khi tải và xuất gói chạy độc lập cho Docker.

b) Thư viện giao diện Hệ thống sử dụng shadcn/ui kết hợp với Radix UI làm nền tảng xây dựng giao diện. Có thể hiểu đây là một bộ khung gồm các thành phần giao diện cơ bản đã được chuẩn bị sẵn về khả năng dùng bàn phím, cách lấy nét và độ nhất quán. Điểm đáng giá của lựa chọn này không nằm ở số lượng thành phần dựng sẵn, mà ở khả

năng giữ giao diện nhất quán trong khi nhóm vẫn kiểm soát được cách từng thành phần được triển khai trong dự án.

Đặc điểm nổi bật:

- **Radix UI Primitives:** Cung cấp các thành phần cơ bản đã xử lý sẵn khả năng truy cập, điều hướng bằng bàn phím và quản lý điểm lấy nét.
- **Tailwind CSS v4:** Hệ thống lớp tiện ích giúp thay đổi giao diện nhanh mà không phải viết nhiều CSS thủ công.
- **Class Variance Authority (CVA):** Giúp quản lý các biến thể của cùng một thành phần giao diện, ví dụ nút chính, nút phụ hay nút cảnh báo.
- **Hệ màu OKLCH:** Giúp điều chỉnh màu dễ kiểm soát hơn khi phải hỗ trợ cả giao diện sáng và tối.

c) Quản lý trạng thái (State Management) Với một giao diện quản trị vừa có dữ liệu tĩnh, dữ liệu thời gian thực và trạng thái giao diện, dồn mọi thứ vào cùng một nơi quản lý sẽ làm mã nguồn nhanh chóng rối lên. Vì vậy, hệ thống áp dụng chiến lược tách biệt giữa trạng thái cục bộ của trình duyệt và trạng thái lấy từ máy chủ:

Bảng 3.25: Chiến lược quản lý trạng thái

Loại trạng thái	Thư viện	Mục đích	Ví dụ
Trạng thái giao diện cục bộ	Zustand 5	Những gì chỉ thuộc về màn hình đang mở	Token phiên đăng nhập, trạng thái sidebar, theme
Trạng thái dữ liệu từ máy chủ	TanStack Query 5	Dữ liệu lấy từ API phía máy chủ và cần tái sử dụng	Danh sách xe, chuyến đi, cảnh báo
Trạng thái gắn với URL	nuqs	Bộ lọc hoặc tab cần giữ lại ngay trên đường dẫn	Bộ lọc, phân trang, tab đang chọn
Trạng thái thời gian thực	Socket.IO Client	Dữ liệu mới được máy chủ đẩy xuống ngay lập tức	Vị trí xe, dữ liệu vận hành gửi liên tục

Kết luận: Zustand được chọn vì API đơn giản, kích thước cực nhỏ (1.1 KB) và phù hợp với yêu cầu của hệ thống nơi token phiên đăng nhập chỉ lưu trong bộ nhớ tạm thời, không ghi ra vùng nhớ lâu trên trình duyệt như localStorage, nhằm tăng tính bảo mật.

d) Thư viện bản đồ

Bảng 3.26: So sánh thư viện bản đồ

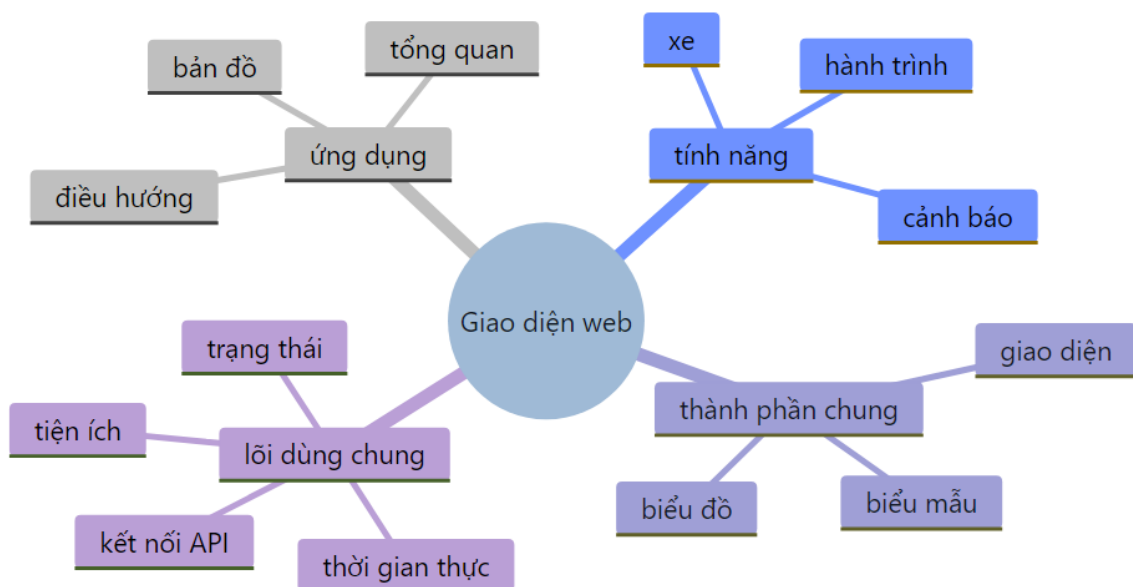
Tiêu chí	Leaflet.js	Mapbox GL JS	Google Maps JS API
Giấy phép	MIT (mã nguồn mở)	Sở hữu riêng (có gói miễn phí giới hạn)	Sở hữu riêng (trả phí)
Chi phí	Miễn phí	Miễn phí đến 50K loads/tháng	0.007 USD/load sau 28K
Hiệu năng với nhiều điểm xe	Tốt (với MarkerCluster, tức gom cụm các điểm gần nhau)	Rất tốt (WebGL rendering, tức dựng hình bằng bộ xử lý đồ họa)	Tốt
Tùy chỉnh điểm đánh dấu	Cao	Cao	Trung bình
Tự lưu và tự phục vụ bản đồ nền	Hỗ trợ	Không	Không
Tích hợp với React	react-leaflet (trưởng thành)	react-map-gl	@react-google-maps/api
Vẽ vùng cho phép	@geoman-io/leaflet-geoman-free	mapbox-gl-draw	Drawing Manager

Kết luận: Leaflet.js được chọn vì hoàn toàn miễn phí, hỗ trợ tự lưu và phân phối bản đồ nền, và có hệ sinh thái plugin phong phú (MarkerCluster để gom nhiều điểm xe gần nhau, Geoman để vẽ và sửa vùng giám sát). Điều này đặc biệt phù hợp với hệ thống IoT có thể triển khai trong mạng nội bộ hoặc môi trường không muốn phụ thuộc dịch vụ bản đồ bên ngoài.

3.4.2.2. Kiến trúc ứng dụng web

a) Mô hình kiến trúc chia theo từng nhóm tính năng Ứng dụng giao diện web được tổ chức theo cách chia mã nguồn bám theo từng nhóm tính năng nghiệp vụ, thường gọi là Feature-Sliced Design (FSD). Mỗi tính năng vì thế được đóng gói thành một module tương đối độc lập gồm thành phần giao diện, hook, kiểu dữ liệu và tiện ích liên quan. Nói gọn hơn, FSD chia mã nguồn theo “thứ người dùng đang làm” thay vì chỉ chia theo loại file. Ví dụ, phần quản lý xe sẽ gom mọi phần liên quan đến xe vào cùng một cụm; nhờ đó người mới đọc mã nguồn dễ lần theo nghiệp vụ hơn, còn việc bổ sung tính năng mới cũng ít làm rối cấu trúc cũ.

- `app/`: khung chia trang của Next.js, bao gồm bố cục gốc và các khu vực quản trị được bảo vệ.
- `features/`: các module nghiệp vụ theo FSD (vehicles, trips, alerts, map), mỗi module tách rõ thành phần giao diện, hook và kiểu dữ liệu.
- `components/`: thư viện thành phần dùng chung (UI, bố cục, bản đồ, biểu đồ, biểu mẫu).
- `lib/`: hạ tầng ứng dụng (lớp gọi API, kênh thời gian thực, nơi giữ trạng thái, tiện ích dùng chung).
- `types/`: các kiểu TypeScript dùng toàn cục.



Hình 3.15: Sơ đồ kiến trúc chia theo từng nhóm tính năng của ứng dụng giao diện web

Nguyên tắc tổ chức:

- **Đặt gần theo tính năng:** Các file liên quan đến một tính năng được đặt cùng thư mục, giảm thời gian tìm kiếm và bảo trì.

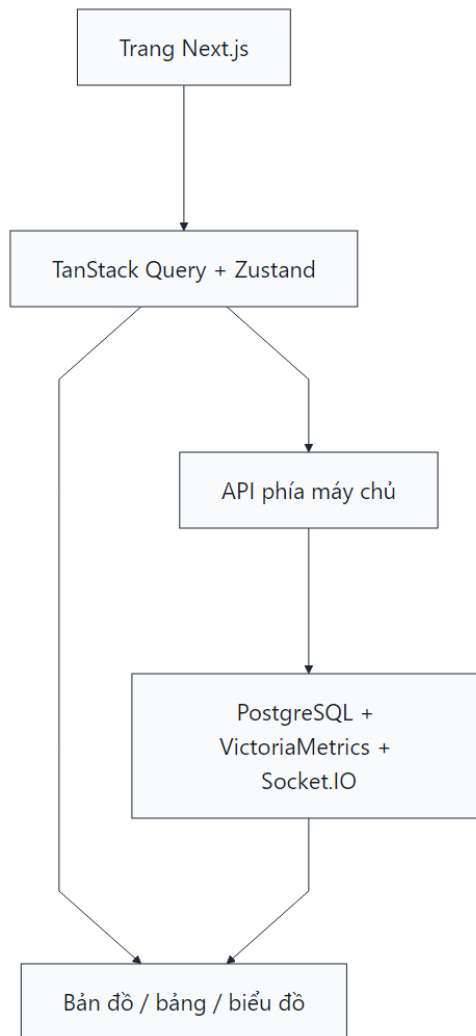
- **Bao gói:** Mỗi module nghiệp vụ chỉ công bố ra ngoài phần thật sự cần dùng; các phần còn lại được giữ bên trong để tránh phụ thuộc chéo.
- **Tách rõ trách nhiệm:** Tách rõ nơi điều hướng trang, nơi chứa logic nghiệp vụ, nơi chứa giao diện dùng lại và nơi chứa hạ tầng kỹ thuật.

b) Luồng dữ liệu trong ứng dụng Luồng dữ liệu trong ứng dụng giao diện web tuân theo mô hình một chiều: dữ liệu đi từ nơi lấy dữ liệu đến nơi hiển thị theo một hướng rõ ràng, đồng thời kết hợp ba nguồn dữ liệu chính:

1. **REST API (TanStack Query):** Dữ liệu CRUD, gồm các thao tác tạo, đọc, sửa, xóa cơ bản như danh sách xe, chuyển đi hay cảnh báo, được lấy từ API phía máy chủ qua HTTP. TanStack Query giúp lưu tạm và làm mới dữ liệu theo cơ chế “hiển thị ngay dữ liệu đang có rồi âm thầm hỏi bản mới”, nhờ đó người dùng không phải chờ trắng màn hình mỗi lần tải lại.
2. **Trạng thái chỉ phục vụ hiển thị (Zustand):** Các trạng thái chỉ liên quan đến giao diện như phiên đăng nhập hiện tại, sidebar hay theme được quản lý ngay trên trình duyệt bằng Zustand mà không cần gọi API.
3. **Dữ liệu cập nhật tức thời (Socket.IO):** Dữ liệu vị trí xe và dữ liệu vận hành gửi liên tục được nhận theo thời gian thực qua WebSocket. Socket.IO là lớp giúp mở và giữ kênh để máy chủ chủ động gửi dữ liệu mới lên trình duyệt. Khi kết nối WebSocket đang hoạt động, TanStack Query sẽ tự động tắt cơ chế hỏi lặp lại, tránh nhận trùng một dữ liệu theo hai đường.

c) Cơ chế xác thực và bảo vệ đường dẫn truy cập Hệ thống giao diện web sử dụng một mã phiên ngẫu nhiên do API phía máy chủ cấp; bản thân mã này không chứa thông tin người dùng. Cách làm này khác với JWT tự mang thông tin phiên. Việc trình duyệt gửi token trong header Authorization: Bearer chỉ là cách mang token theo từng yêu cầu; về bản chất, đây vẫn là cơ chế giữ phiên trên máy chủ vì mọi quyền truy cập đều chỉ được quyết định sau khi máy chủ tra cứu phiên tương ứng:

1. **Đăng nhập:** Người dùng nhập email và mật khẩu, API phía máy chủ tạo bản ghi phiên và trả về một mã phiên ngẫu nhiên.
2. **Lưu trữ token:** Token chỉ được giữ trong bộ nhớ tạm của Zustand, không lưu vào localStorage hay sessionStorage, là hai nơi lưu dữ liệu bền hơn trên trình duyệt, nhằm giảm nguy cơ mã độc chèn vào trang rồi lấy cắp dữ liệu.
3. **Gửi yêu cầu:** Mọi yêu cầu HTTP tự động đính kèm token vào header Authorization: Bearer <token>.
4. **Lớp chặn truy cập:** Thành phần AuthGuard bao bọc toàn bộ khu vực /dashboard/*, tự động chuyển hướng về trang đăng nhập nếu chưa xác thực hoặc token hết hạn.



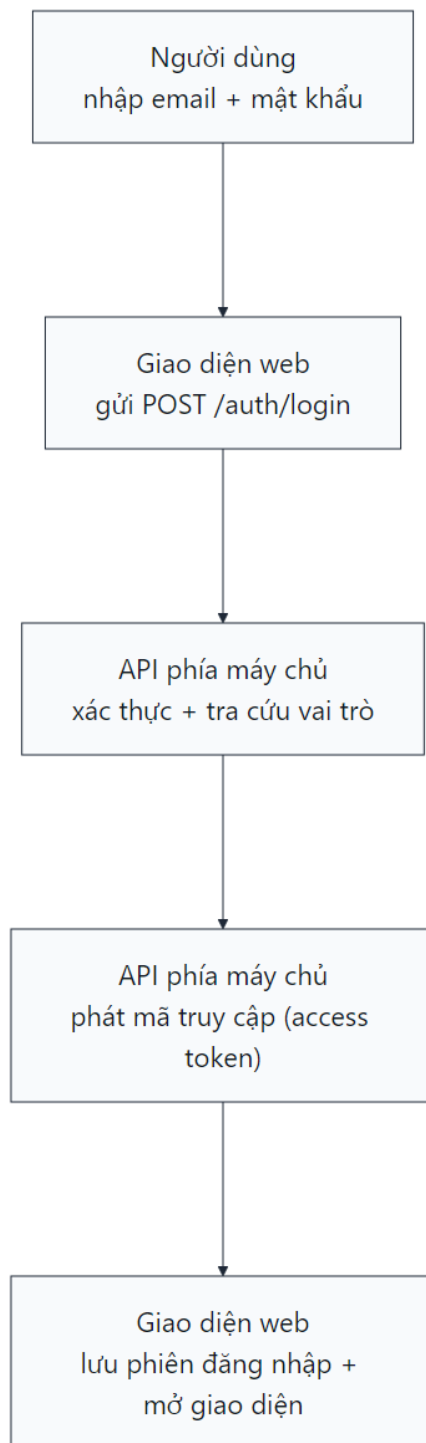
Hình 3.15a: Luồng dữ liệu một chiều giữa Next.js, Query, Zustand và API phía máy chủ

5. **Gia hạn phiên:** Khi phiên sắp hết hạn, giao diện web chủ động gọi cơ chế gia hạn để nhận token mới, nhờ đó người dùng không bị đăng nhập lại đột ngột trong lúc thao tác.

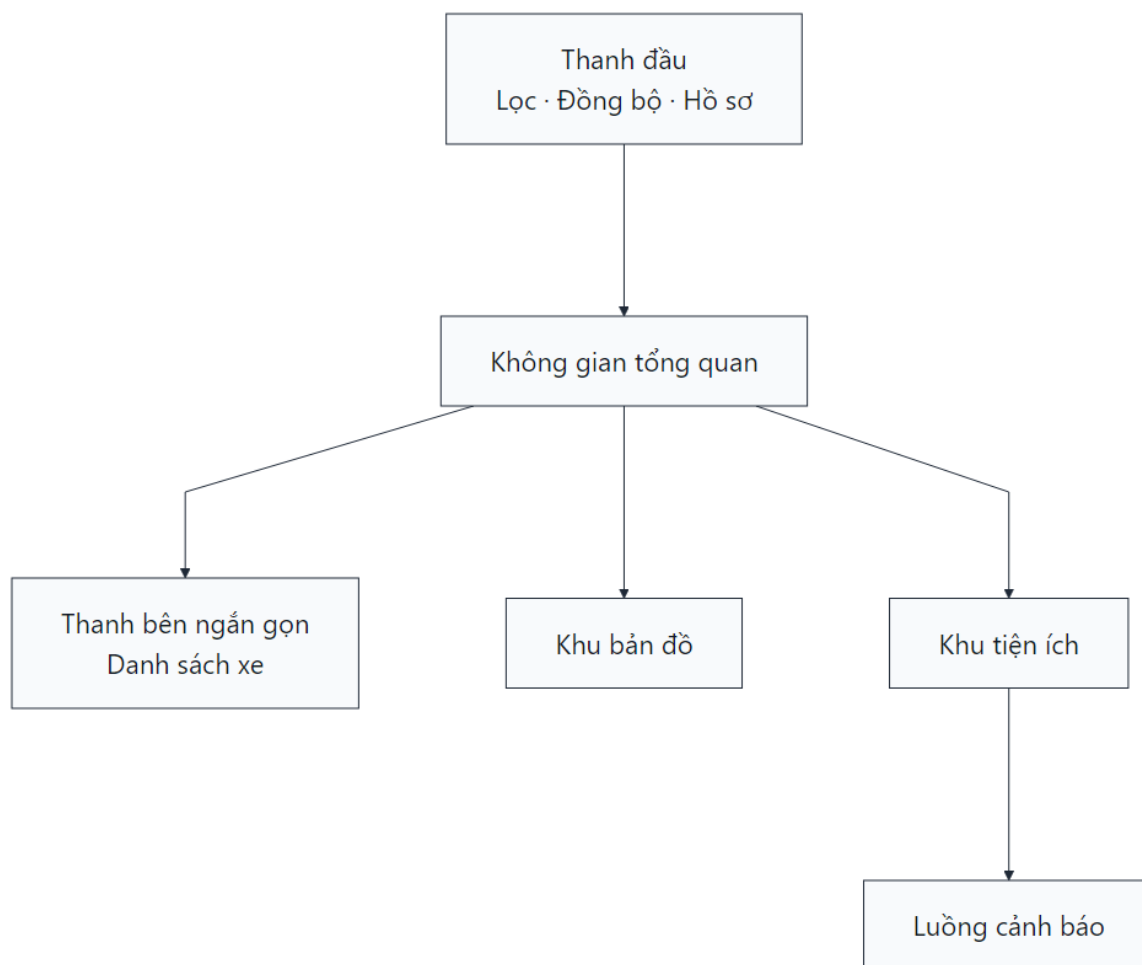
3.4.2.3. Thiết kế các trang chức năng chính

a) Trang tổng quan Trang tổng quan là điểm vào chính sau khi đăng nhập, cung cấp cái nhìn tổng thể về trạng thái hệ thống:

- **Các thẻ chỉ số nhanh:** Hiển thị các chỉ số quan trọng: tổng số xe đang hoạt động, số chuyến đi trong ngày, số cảnh báo chưa xử lý, số vi phạm má»i.
- **Bản đồ mini:** Hiển thị vị trí tất cả xe đang hoạt động trên bản đồ thu nhỏ.
- **Bảng cảnh báo gần đây:** Danh sách 10 cảnh báo mới nhất với nhãn mức độ nghiêm trọng (thấp, trung bình, cao, rất cao).
- **Biểu đồ thống kê:** Biểu đồ số chuyến đi theo ngày và phân loại cảnh báo theo loại.



Hình 3.16: Biểu đồ trình tự luồng xác thực người dùng



Hình 3.17: Giao diện trang tổng quan hệ thống

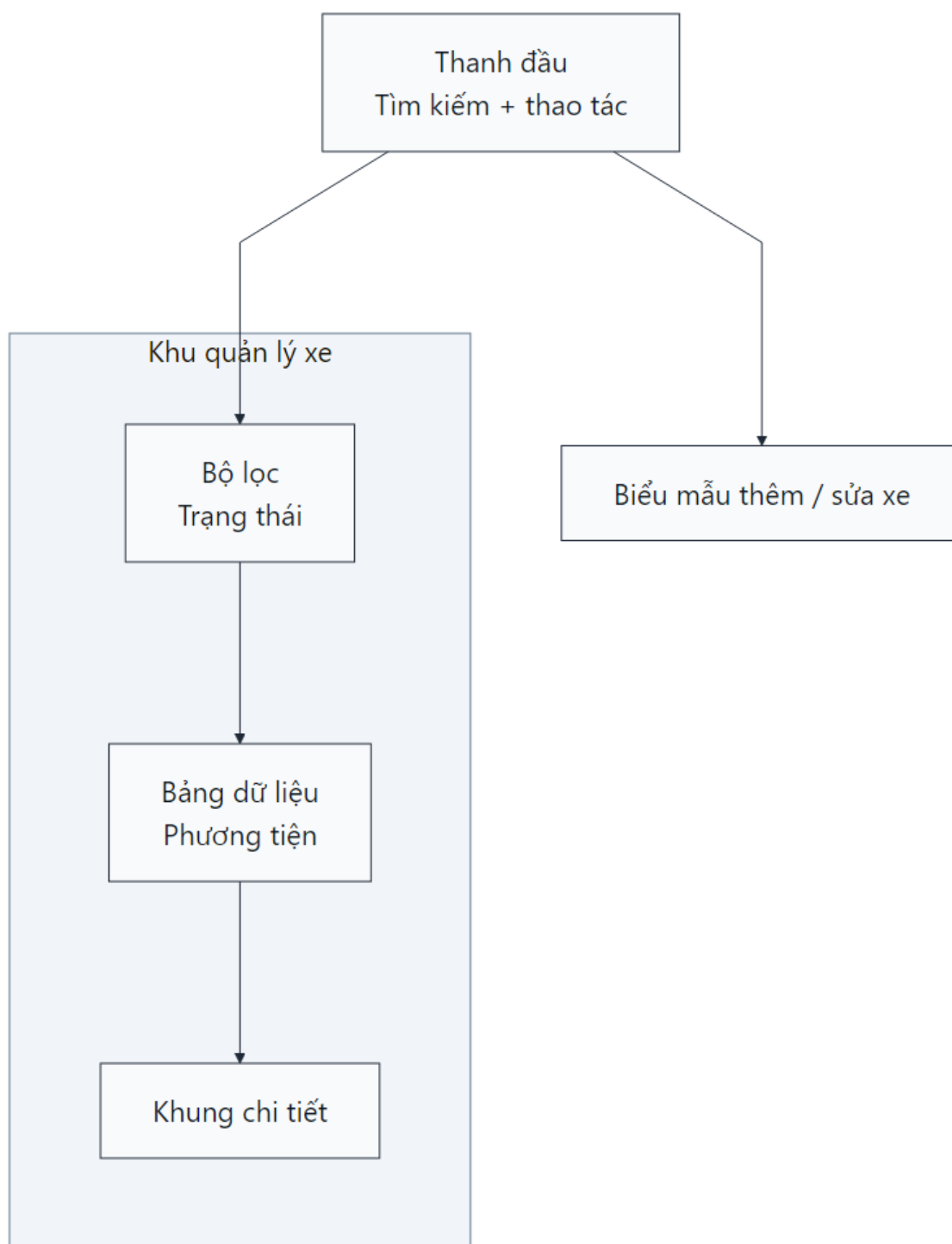
b) Quản lý phương tiện Trang danh sách (/dashboard/vehicles):

Bảng 3.2.4A: Tổng hợp thông tin về Cột, Mô tả, Chức năng

Cột	Mô tả	Chức năng
Biển số xe	Số đăng ký phương tiện	Tìm kiếm
Hãng/Model	Thương hiệu và đời xe	Lọc
Trạng thái	Đang hoạt động, tạm ngưng, bảo trì, ngừng sử dụng	Lọc theo trạng thái
Thiết bị	Mã thiết bị IoT gắn kèm	Liên kết
Lần cuối hoạt động	Thời điểm nhận dữ liệu gần nhất	Sắp xếp
Thao tác	Xem, Sửa, Xóa, Xem trên bản đồ	Nút hành động

Trang chi tiết (/dashboard/vehicles/[id]): Hiện thị thông tin đầy đủ của phương

tiện bao gồm thông tin cơ bản, vị trí hiện tại trên bản đồ, trạng thái thiết bị IoT, các cảnh báo đang hoạt động, lịch sử chuyến đi và lịch sử bảo trì.



Hình 3.18: Giao diện trang quản lý phương tiện

c) Quản lý chuyển đi, cảnh báo và vùng địa lý Trang quản lý chuyển đi cung cấp lịch sử và chi tiết từng chuyến, gồm tuyến đường trên bản đồ, biểu đồ tốc độ theo thời gian và timeline sự kiện. Trang cảnh báo tổng hợp cảnh báo theo nhãn mức độ nghiêm trọng và hỗ trợ xử lý hàng loạt. Trang quản lý vùng cho phép hỗ trợ vẽ hình tròn, đa giác hoặc hình chữ nhật trực tiếp trên bản đồ bằng Leaflet Geoman.

Bảng 3.27: Ma trận các trang chức năng chính

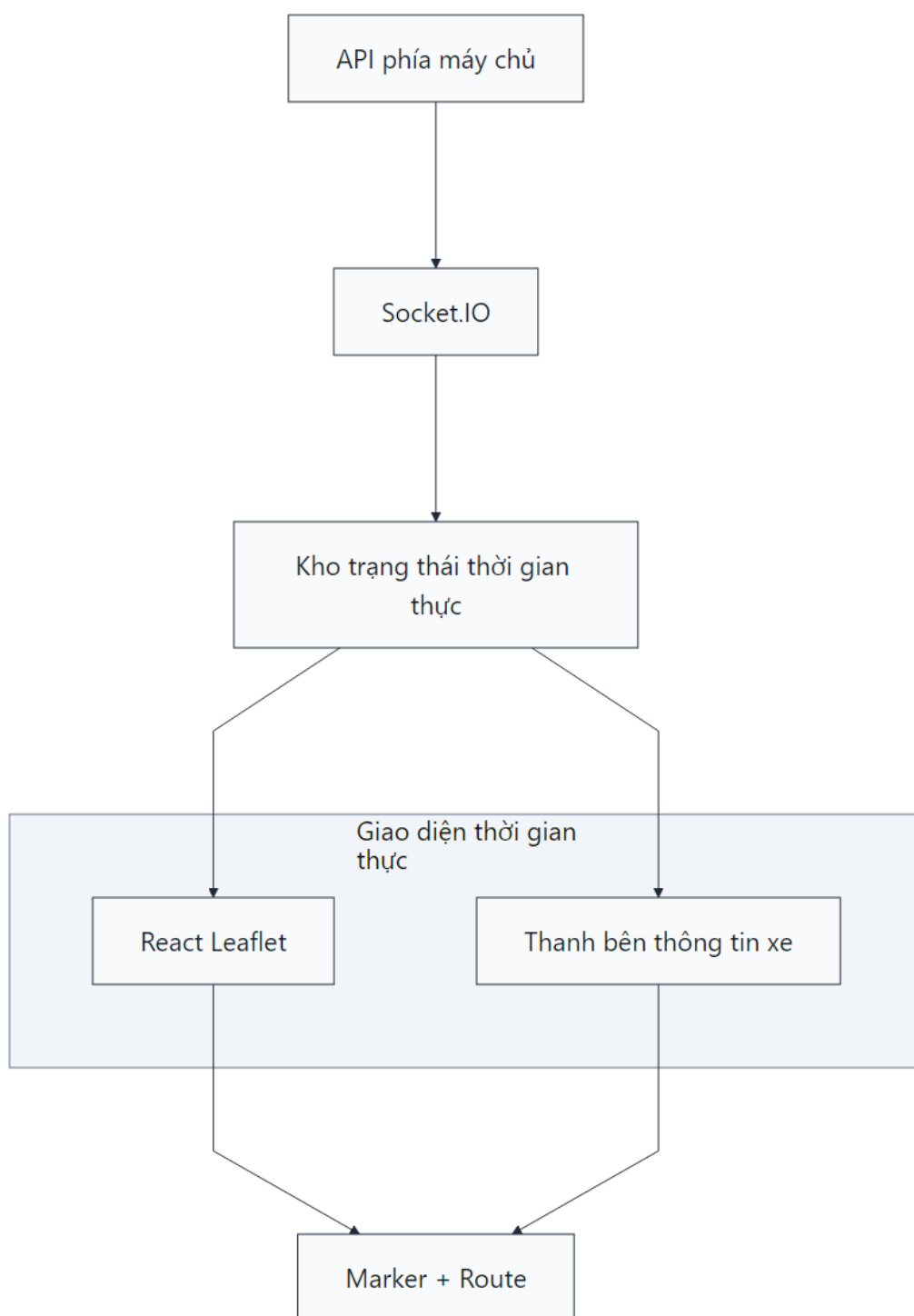
Trang	Đường dẫn	Chức năng chính	Cập nhật tức thời
Tổng quan	/dashboard	Tổng quan hệ thống, các thẻ chỉ số chính, biểu đồ	Có (cập nhật số liệu)
Bản đồ	/dashboard/map	Theo dõi tất cả xe thời gian thực	Có (vị trí xe)
Phương tiện	/dashboard/vehicles	CRUD xe, trạng thái, lịch sử	Có (trạng thái xe)
Chuyến đi	/dashboard/trips	Lịch sử chuyến đi, tuyến đường	Không
Cảnh báo	/dashboard/alerts	Xem, xử lý cảnh báo	Có (cảnh báo mới)
Vi phạm	/dashboard/violations	Quản lý vi phạm tốc độ	Không
Thiết bị	/dashboard/devices	Quản lý thiết bị IoT	Có (trạng thái thiết bị)
Vùng địa lý	/dashboard/geofences	Tạo, sửa vùng cho phép trên bản đồ	Không
Bảo trì	/dashboard/maintenance	Lịch sử bảo trì phương tiện	Không
Thông báo	/dashboard/notifications	Cấu hình Telegram, Email	Không
Cài đặt	/dashboard/settings	Hồ sơ, quản lý người dùng	Không

3.4.2.4. Tích hợp bản đồ thời gian thực

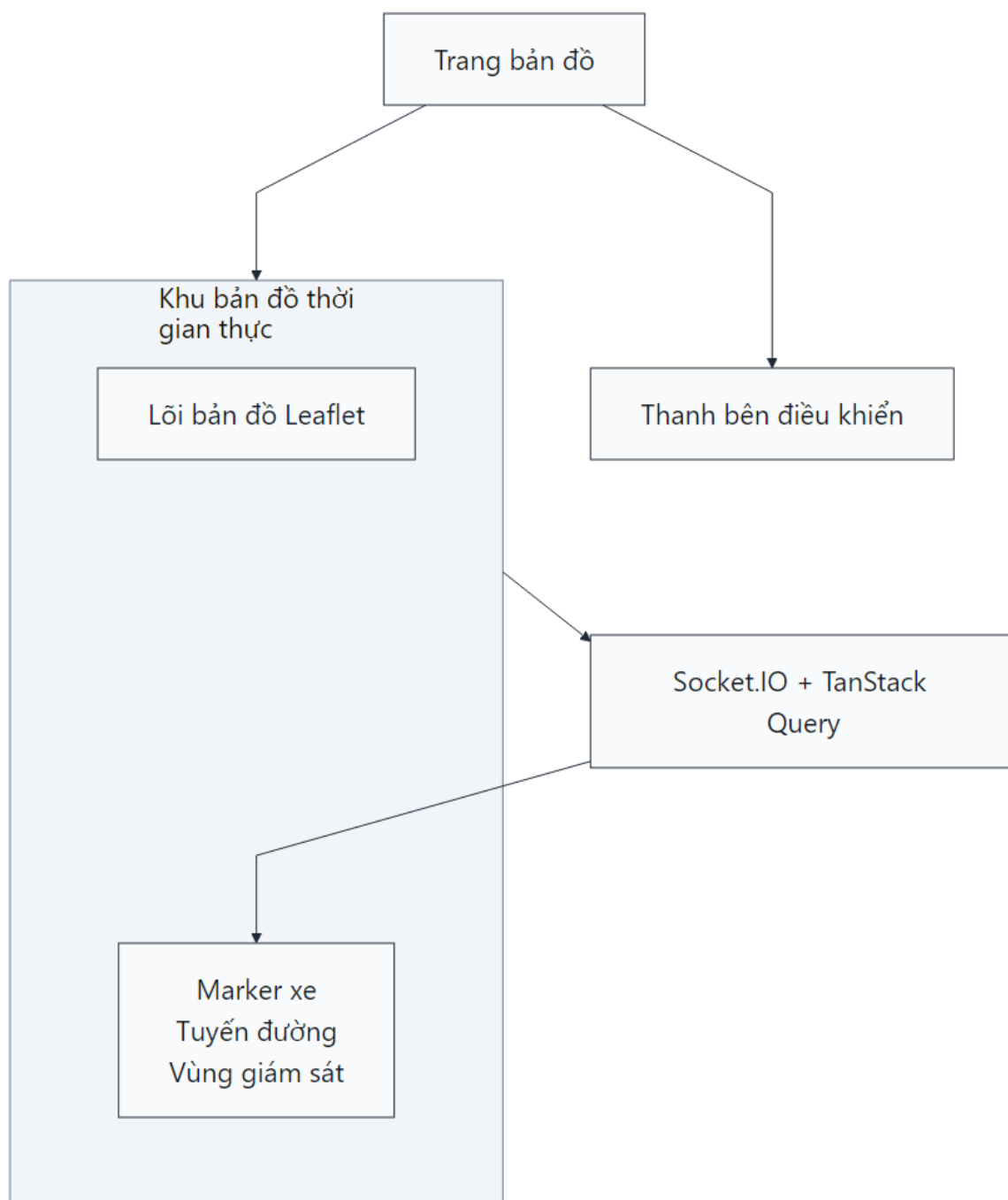
a) Kiến trúc tích hợp bản đồ Trang bản đồ thời gian thực (/dashboard/map) là tính năng cốt lõi của hệ thống giám sát phương tiện. Kiến trúc tích hợp bản đồ bao gồm ba lớp chính:

b) Cơ chế cập nhật vị trí thời gian thực Quy trình cập nhật vị trí xe trên bản đồ diễn ra như sau:

1. **Thiết bị IoT** gửi dữ liệu GPS qua MQTT đến EMQX Broker.
2. **MQTT Bridge** nhận và xử lý dữ liệu, lưu vào VictoriaMetrics, đồng thời chuyển tiếp đến API phía máy chủ.



Hình 3.19: Kiến trúc tích hợp bản đồ thời gian thực



Hình 3.19a: Phân rã các lớp của bản đồ React Leaflet và kênh Socket.IO

3. **API phía máy chủ** phát sự kiện `vehicle:{id}:location` qua Socket.IO đến tất cả trình duyệt đang đăng ký nhận dữ liệu.
4. **Giao diện web** nhận sự kiện, cập nhật tọa độ điểm đánh dấu trên bản đồ. Sử dụng kỹ thuật nội suy (interpolation) để điểm đánh dấu di chuyển mượt mà giữa hai điểm tọa độ liên tiếp.

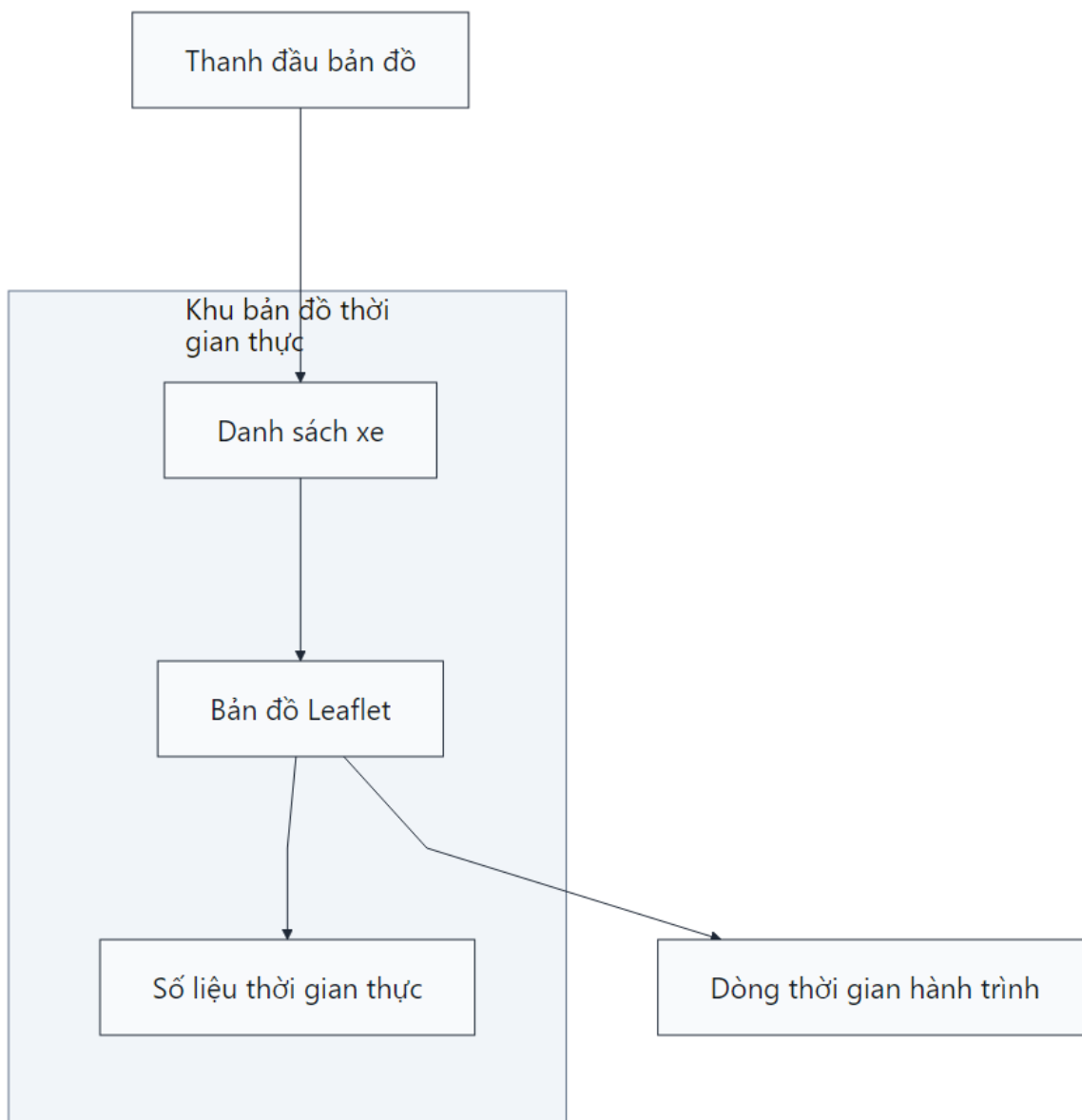
Tối ưu hiệu năng bản đồ:

- **Gom cụm điểm xe:** Sử dụng `react-leaflet-cluster` để gom nhóm các điểm xe gần nhau khi thu nhỏ bản đồ, giảm số lượng phần tử phải vẽ cùng lúc.
- **Chỉ vẽ phần đang nhìn thấy:** Chỉ vẽ điểm đánh dấu của các xe nằm trong vùng nhìn hiện tại của bản đồ.
- **Gom các cập nhật gần nhau:** Gom nhóm các cập nhật vị trí trong khoảng 100ms để tránh re-render quá nhiều lần.

c) **Các thành phần bản đồ** Hệ thống bao gồm bốn thành phần bản đồ chuyên biệt:

Bảng 3.2.4B: Tổng hợp thông tin về thành phần bản đồ, file và chức năng

Thành phần	File	Chức năng
VehicleMap	<code>components /map/vehicle- map.tsx</code>	Hiển thị tất cả xe trên bản đồ với điểm đánh dấu và ô thông tin
RouteMap	<code>components /map/route- map.tsx</code>	Hiển thị tuyến đường chuyển đi với đường nối và các điểm dừng
GeofenceMap	<code>components /map/geofenc e-map.tsx</code>	Vẽ và chỉnh sửa vùng địa lý (đa giác, hình tròn, hình chữ nhật)
MarkerPopup	<code>components /map/marker- popup.tsx</code>	Ô thông tin hiển thị chi tiết xe khi bấm vào điểm đánh dấu (biển số, tốc độ, trạng thái)



Hình 3.20: Giao diện trang bản đồ thời gian thực với các marker xe

3.4.2.5. Thiết kế giao diện thích ứng và trải nghiệm người dùng

a) Chiến lược thích ứng theo màn hình Ở giai đoạn hiện tại, giao diện được ưu tiên cho desktop và tablet, nhưng vẫn giữ ba mức ngưỡng chiều rộng màn hình chính để không phá vỡ trải nghiệm trên màn hình nhỏ:

Bảng 3.28: Ngưỡng chiều rộng màn hình và cách điều chỉnh giao diện

Ngưỡng	Kích thước	Giao diện	Điều chỉnh chính
Mobile	< 640px	Giao diện tối giản, một cột	Sidebar thành drawer, bảng thành danh sách card, bản đồ toàn màn hình
Tablet	640px - 1024px	Giao diện trung gian	Sidebar thu gọn (icon-only), bảng giữ nguyên với cuộn ngang
Desktop	> 1024px	Giao diện đầy đủ	Sidebar mở rộng, bảng đầy đủ cột, bản đồ chia đôi màn hình

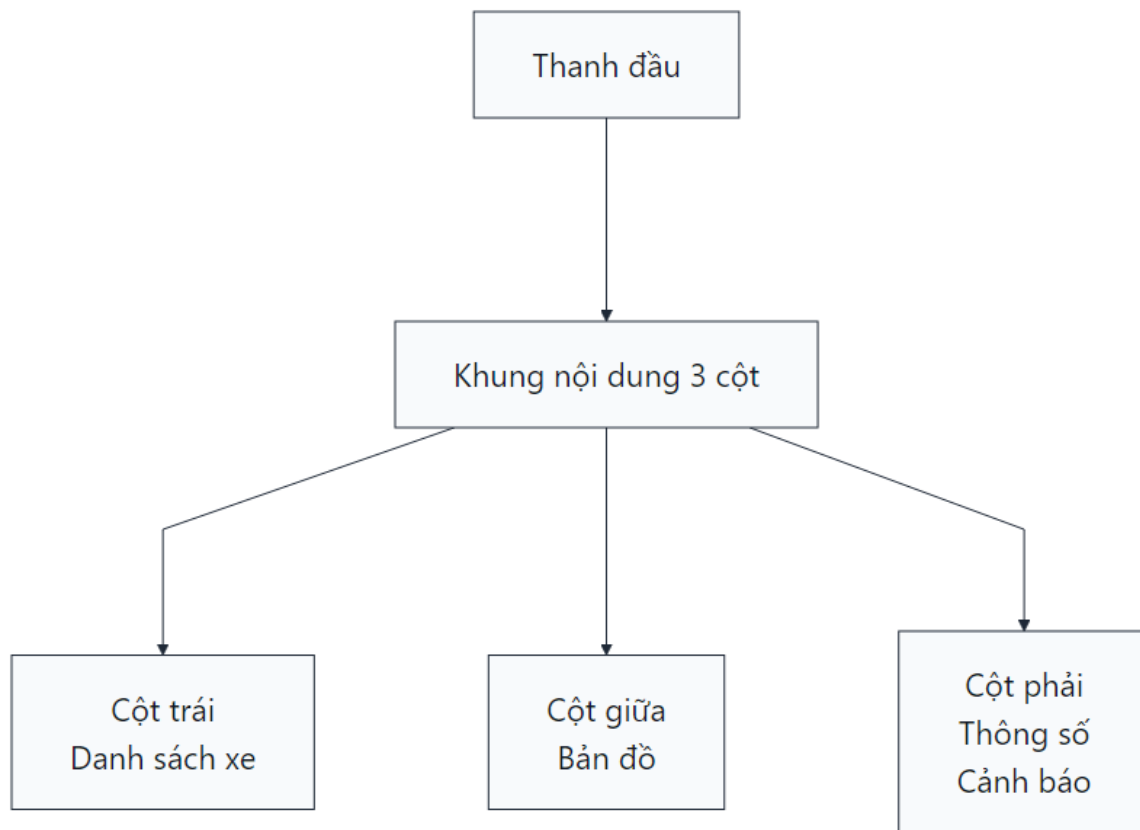
b) Bố cục chính Giao diện tổng quan sử dụng bố cục gồm ba vùng chính: Header (đường dẫn đang đứng, ô tìm kiếm, menu người dùng, nút đổi giao diện sáng/tối), Sidebar (menu điều hướng bên trái), và vùng nội dung chính (nội dung trang, bộ lọc, bảng dữ liệu).

c) Hệ thống theme Light/Dark Hệ thống hỗ trợ hai chế độ giao diện sáng và tối, sử dụng CSS Variables với hệ màu oklch. Chuyển đổi được thực hiện qua thư viện next-themes và lưu trong cookie.

d) Các mẫu thiết kế giao diện (UI Patterns) Hệ thống áp dụng các mẫu thiết kế giao diện nhất quán xuyên suốt ứng dụng:

- **Bảng dữ liệu:** Sắp xếp, lọc, phân trang ở phía máy chủ, chọn nhiều dòng, xuất dữ liệu CSV/Excel.
- **Biểu mẫu nhập liệu:** React Hook Form kết hợp Zod để kiểm tra dữ liệu nhập ngay tại chỗ và báo lỗi ngay dưới trường nhập.
- **Bản đồ:** Điểm đánh dấu đổi màu theo trạng thái, cửa sổ thông tin khi bấm vào xe và bộ công cụ vẽ vùng cho phép.
- **Biểu đồ:** ECharts/Recharts cho dữ liệu vận hành, gồm biểu đồ đường cho tốc độ, đồng hồ hiển thị RPM và biểu đồ vùng cho nhiên liệu/nhiệt độ.

e) Tối ưu trải nghiệm người dùng Để nâng cao trải nghiệm người dùng, hệ thống áp dụng một số kỹ thuật:



Hình 3.21: Wireframe bố cục giao diện tổng quan



Hình 3.22: Các mẫu thiết kế giao diện của hệ thống - bảng dữ liệu, biểu mẫu, biểu đồ

- **Cập nhật tạm ngay trên giao diện:** Giao diện hiển thị kết quả trước khi nhận phản hồi cuối cùng từ máy chủ để thao tác không bị khựng.
- **Khung chờ:** Hiển thị khung chờ thay vì vòng quay spinner khi đang tải dữ liệu.
- **Thanh báo đang chuyển trang:** Thanh tiến trình ở đầu trang (sử dụng nextjs-toploader) khi chuyển trang.
- **Phím tắt:** Hỗ trợ phím tắt cho các thao tác phổ biến (Cmd+K cho tìm kiếm).
- **Vùng chặn lỗi riêng:** Mỗi khu vực chức năng được bao bọc bởi một lớp chặn lỗi riêng để hỏng chỗ nào thì cô lập chỗ đó.
- **Trạng thái chưa có dữ liệu:** Hiển thị trạng thái trống có hướng dẫn hành động khi chưa có dữ liệu.

3.5. Phân tích, đánh giá và lựa chọn phương án khả thi – Analysis, Evaluation and Selection

Dựa trên phân tích và đề xuất ở các mục 3.1 đến 3.4, bảng tổng hợp dưới đây đánh giá và so sánh các phương án đã xem xét:

Bảng 3.29: Ma trận đánh giá tổng hợp các phương án thiết kế

Tầng hệ thống	Phương án được chọn	Lý do chính
Vi điều khiển	ESP32-S3	BLE 5.0 tích hợp, hai lõi xử lý, tài nguyên đủ cho nhiều tác vụ đồng thời
LTE + GNSS	SIMCom SIM7600CE-T	Modem tích hợp LTE + GNSS, giảm số lượng khối phần cứng rời
OBD2 Adapter	vgate iCar Pro (BLE)	Không cần dây, tương thích rộng
Cảm biến IMU	LIS3DH	Tiết kiệm điện và có thể tự đánh thức khi phát hiện chuyển động
Buffer cục bộ	microSD SDMMC 4-bit + FATFS + phát lại dữ liệu	Duy trì liên tục dữ liệu vận hành khi mất mạng, rồi gửi bù theo đúng thứ tự khi có mạng lại
MQTT Broker	EMQX	Có lớp lọc bản tin, phân quyền theo từng thiết bị và có thể mở rộng nếu số xe tăng
Database quan hệ	PostgreSQL 16	Ổn định, quen thuộc, phù hợp dữ liệu có quan hệ chặt chẽ
Database time-series	VictoriaMetrics	Ghi dữ liệu theo thời gian rất nhanh và truy vấn tốt theo mốc thời gian

Bảng 3.29: Ma trận đánh giá tổng hợp các phương án thiết kế (tiếp theo)

Tầng hệ thống	Phương án được chọn	Lý do chính
API phía máy chủ	Express.js + TypeScript	Nhẹ, linh hoạt, dễ tổ chức theo từng bài toán nghiệp vụ
Giao diện web	Next.js 15 + React 19	Hiện nội dung sớm, tổ chức trang rõ và hệ sinh thái lớn
Bản đồ	Leaflet.js	Mã nguồn mở, có thể tự lưu và phân phối bản đồ nền

3.6. Tối ưu phương án thiết kế – The optimal solution

Từ kết quả phân tích và đánh giá ở mục 3.5, phương án thiết kế tối ưu cho hệ thống IoT giám sát phương tiện được tổng hợp như sau:

Kiến trúc phân tầng:

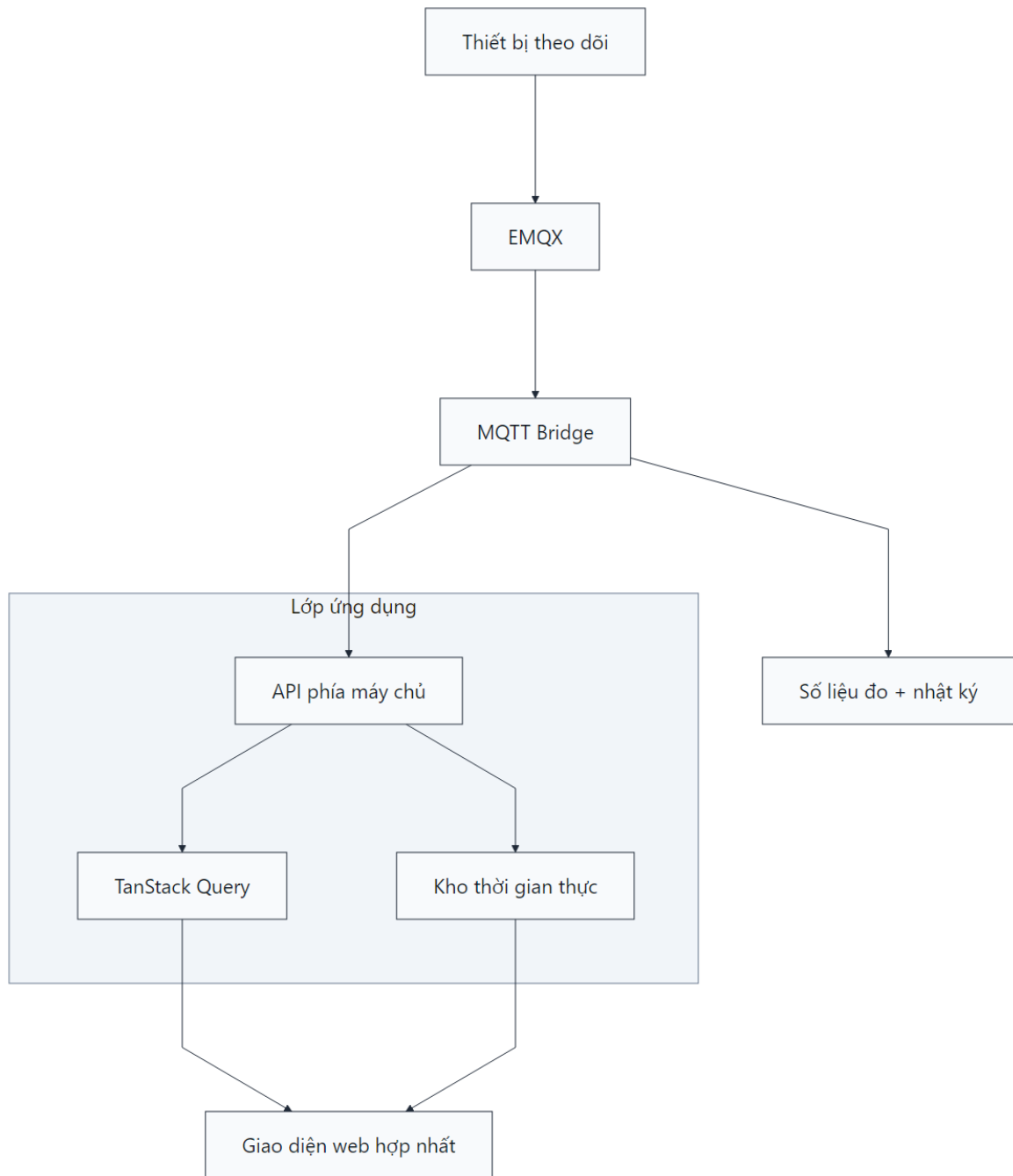
- **Tầng thiết bị:** ESP32-S3 + SIMCom SIM7600CE-T + vgate iCar Pro + LIS3DH, phối hợp để lấy dữ liệu xe, lấy vị trí và quản lý nguồn với pin dự phòng 18650 1S
- **Tầng truyền thông:** MQTT 3.1.1 qua 4G LTE, dùng mức cam kết gửi khác nhau cho từng loại bản tin, kết hợp vùng lưu tạm microSD và cơ chế gửi bù khi kết nối phục hồi
- **Tầng xử lý:** MQTT Bridge nhận bản tin rồi ghi ra nhiều nơi phù hợp như PostgreSQL và VictoriaMetrics, sau đó API phía máy chủ xử lý nghiệp vụ cho người dùng
- **Tầng trình bày:** Next.js 15, bản đồ Leaflet, kênh thời gian thực Socket.IO và biểu đồ ECharts để người vận hành quan sát dữ liệu

Phương án này đáp ứng các yêu cầu kỹ thuật đã đặt ra ở Chương 2, đồng thời giữ chi phí phần cứng ở mức khoảng 1.514.000 VND cho mỗi thiết bị theo mặt bằng giá tham chiếu tháng 4/2026.

Kết luận chương 3

Chương 3 đã trả lời câu hỏi ”thiết kế hệ thống này như thế nào để vừa đáp ứng yêu cầu kỹ thuật, vừa khả thi khi triển khai”. Từ phần cứng, phần mềm trên thiết bị, hạ tầng máy chủ xử lý dữ liệu đến giao diện web, các phương án đều đã được cân nhắc theo tiêu chí hiệu năng, chi phí, độ phức tạp và khả năng mở rộng.

Vì vậy, phương án cuối cùng ở Chương 3 không xuất hiện như một lựa chọn cảm tính, mà như kết quả của quá trình sàng lọc có cơ sở. Trên nền tảng đó, Chương 4 chuyển sang



Hình 3.23: Sơ đồ kiến trúc tổng thể phương án tối ưu

phần hiện thực: biến các lựa chọn thiết kế thành hệ thống vận hành được trong thực tế.

CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ

– IMPLEMENTATION AND RESULTS

Nếu Chương 3 dừng ở mức lựa chọn phương án, thì Chương 4 trả lời câu hỏi liệu những lựa chọn đó có thực sự hoạt động khi đưa vào chế tạo, lập trình và tích hợp hay không. Nội dung chương đi từ phần cứng thiết bị, firmware, hạ tầng máy chủ, giao diện giám sát đến các kết quả đo lường sau khi lắp đặt hệ thống trên xe, qua đó giúp người đọc đối chiếu giữa ý tưởng thiết kế và kết quả triển khai thực tế.

4.1. Thiết kế chi tiết giải pháp – Detailed design solution

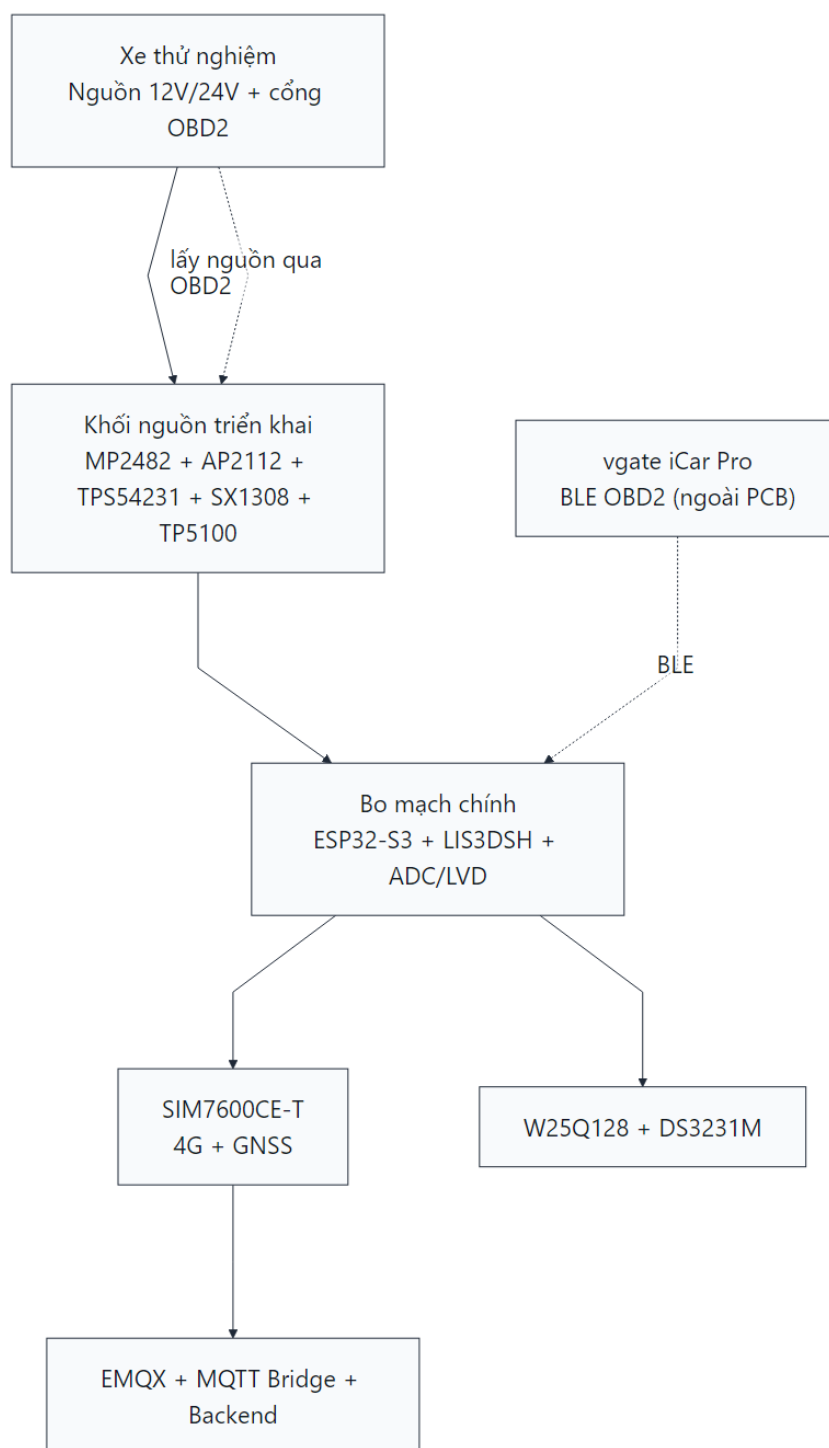
4.1.1. Thiết kế chi tiết phần cứng

a) Kiến trúc tổng thể Xét theo chức năng, ESP32-S3 là bộ điều phối trung tâm của thiết bị theo dõi. Nó thu tín hiệu từ các khối khác, quyết định thiết bị đang ở chế độ nào và ra lệnh cho từng phần cứng hoạt động đúng lúc. Cụ thể hơn, ESP32-S3 nói chuyện với modem qua UART, tức cổng nối tiếp cơ bản; lấy dữ liệu xe qua BLE, tức Bluetooth tiết kiệm năng lượng; đọc cảm biến rung qua I2C, tức đường giao tiếp ngắn cho cảm biến; và đo điện áp ắc quy qua ADC, tức bộ chuyển đổi từ tín hiệu điện áp analog sang số. Toàn bộ thiết bị được nuôi bởi mạch quản lý năng lượng có khả năng tự chuyển giữa ắc quy xe và pin dự phòng.

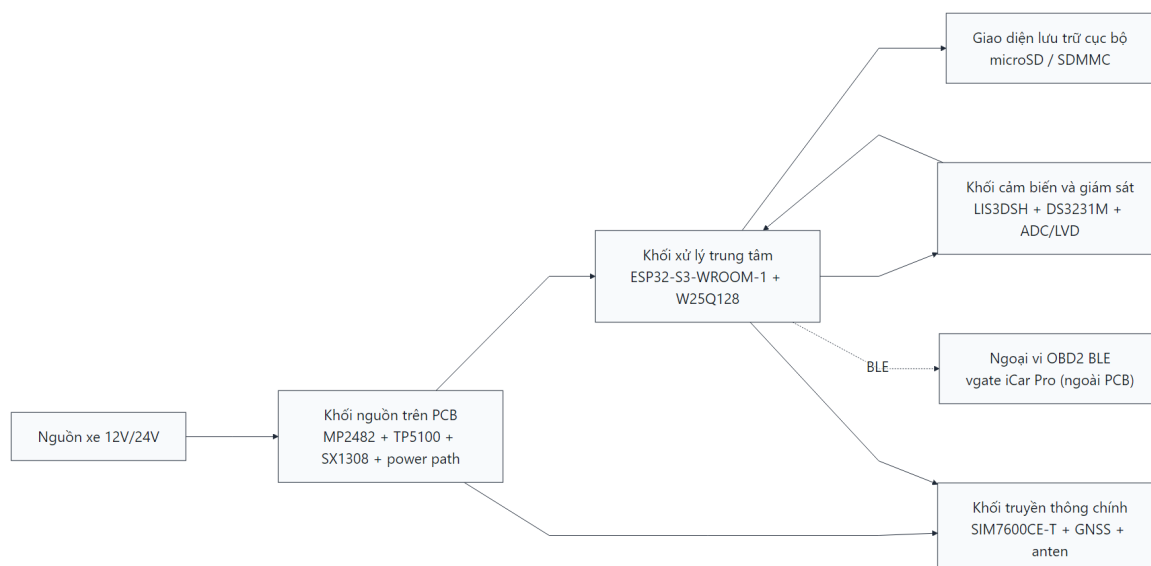
b) Khối vi điều khiển và giao tiếp Vi điều khiển ESP32-S3-WROOM-1 được lựa chọn làm nhân xử lý trung tâm của hệ thống với các thông số kỹ thuật nổi bật:

Bảng 4.1.1A: Tổng hợp thông tin về Thông số, Giá trị

Thông số	Giá trị
CPU	Dual-core Xtensa LX7 @ 240 MHz (32-bit)
RAM	512 KB SRAM
Flash	4–16 MB (tùy variant)
Bluetooth	BLE 5.0 (tương thích ngược BLE 4.0)
GPIO	45 chân
UART	3 cổng
I2C	2 cổng
ADC	12-bit, 20 kênh
Deep Sleep	10–15 μ A (với external wakeup)
Active (BLE)	20–40 mA



Hình 4.1: Sơ đồ khối tổng thể hệ thống thiết bị theo dõi IoT



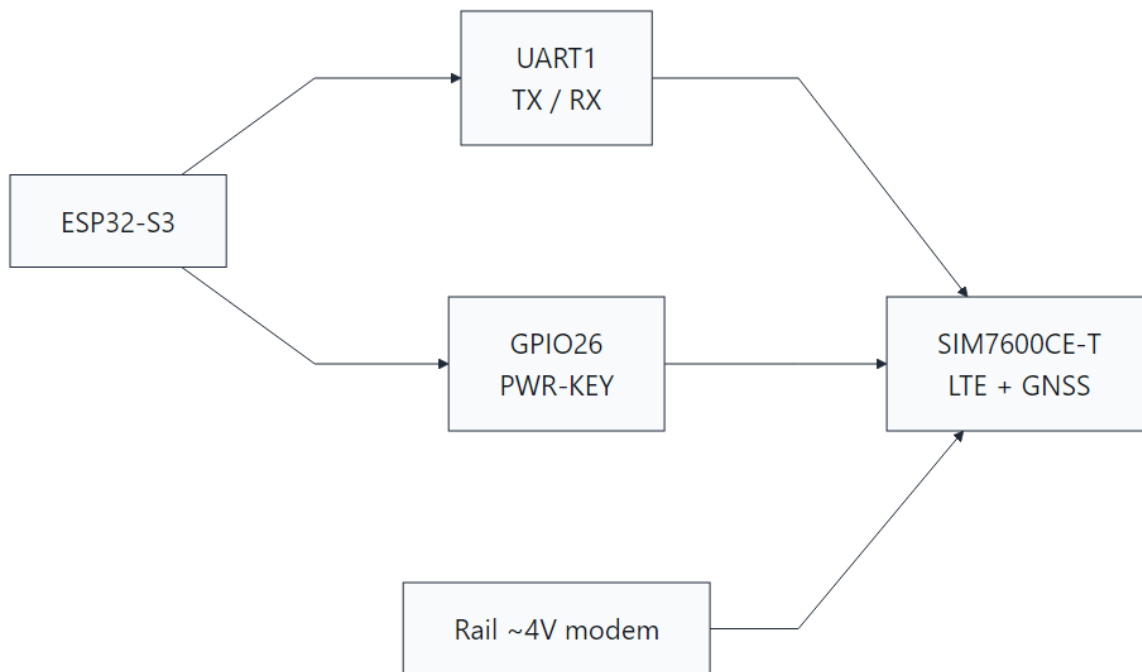
Hình 4.1a: Bố trí chi tiết các khối phần cứng trong thiết bị theo dõi

Giao tiếp UART với modem SIM7600CE-T (LTE + GNSS tích hợp): Khối modem gói hai nhiệm vụ quan trọng vào cùng một phần cứng: đưa dữ liệu lên mạng di động và cung cấp vị trí GNSS của xe. ESP32-S3 trao đổi với modem qua UART1 bằng tập lệnh AT, tức là các lệnh văn bản để yêu cầu modem đăng ký mạng, mở kết nối dữ liệu và đọc thông tin định vị. Nhờ GNSS đã nằm sẵn trong modem, thiết kế không cần thêm một module định vị riêng, nên đi dây gọn hơn và việc tích hợp firmware cũng đơn giản hơn.

Giao tiếp BLE với OBD2 adapter: ESP32-S3 sử dụng BLE 5.0 tích hợp để kết nối với adapter vgate iCar Pro (BLE 4.0). Nhờ kết nối này, thiết bị theo dõi không chỉ biết xe đang ở đâu mà còn biết xe đang ở trạng thái nào thông qua các dữ liệu OBD-II như trạng thái khóa điện (IGN), tốc độ động cơ (RPM), vận tốc xe, mức nhiên liệu và mã lỗi chẩn đoán (DTC).

Giao tiếp I2C với cảm biến LIS3DSH: Cảm biến gia tốc 3 trục LIS3DSH được kết nối qua bus I2C (GPIO2 SDA, GPIO1 SCL). Vai trò chính của cảm biến này là giúp hệ thống "ngủ mà vẫn canh", tức là phát hiện chuyển động khi xe đang đỗ và đánh thức ESP32-S3 từ light sleep thông qua GPIO wake khi có rung động bất thường; heartbeat định kỳ vẫn dùng timer wake.

c) Khối đo điện áp ắc quy Khối này dùng mạch chia áp để đưa điện áp ắc quy xe về dải đo an toàn cho ADC của ESP32-S3. Firmware sẽ quy đổi giá trị ADC về điện áp thực tế và áp dụng profile 12V hoặc 24V để nhận diện trạng thái nguồn, ngưỡng chuyển nguồn, và điều kiện IGN ON/OFF.



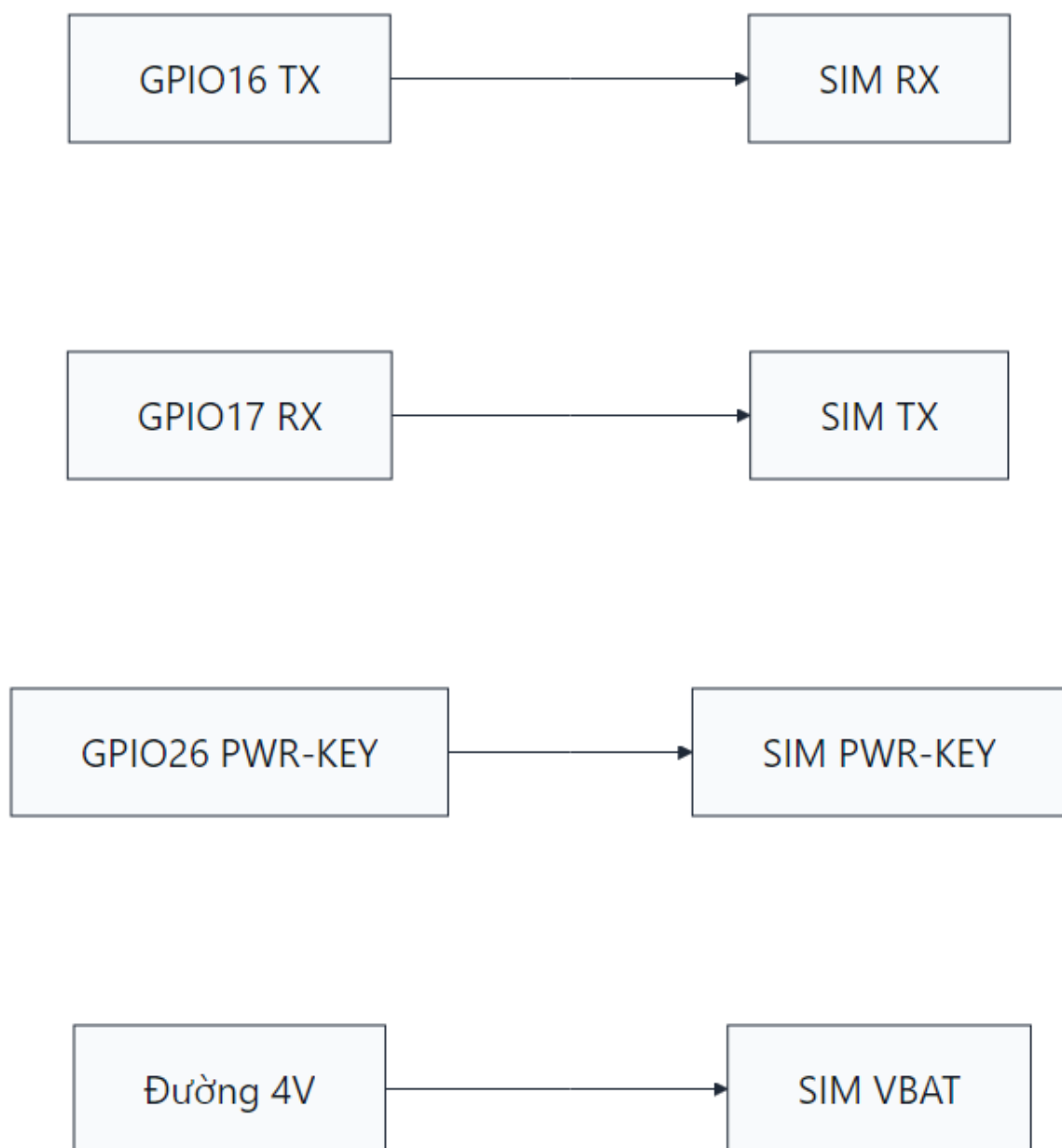
Hình 4.2: Sơ đồ kết nối giữa ESP32-S3 và modem SIM7600CE-T qua UART

d) Sơ đồ nguyên lý tổng hợp Sơ đồ nguyên lý tổng hợp của hệ thống thể hiện toàn bộ các khối mạch đã được tích hợp trên PCB do nhóm tự thiết kế. Các kết nối nội bộ chính bao gồm:

- **UART1** (2 dây tín hiệu TX/RX): Kết nối với modem SIM7600CE-T để truyền nhận cả dữ liệu 4G và GNSS (AT + AT+CGNSINF/AT+CGNSTST).
- **I2C** (2 dây tín hiệu SDA/SCL): Kết nối với cảm biến gia tốc LIS3DH
- **BLE** (không dây): Kết nối với adapter OBD2 vgate iCar Pro
- **ADC** (1 kênh): Đọc điện áp ắc quy qua voltage divider
- **GPIO** (3 chân output): Điều khiển chân EN chuyển nguồn, chân EN sạc, Modem PWR-KEY
- **GPIO** (2 chân input): Đọc trạng thái LVD, ngắt từ IMU

e) Sơ đồ đấu nối và GPIO mapping (Wiring & GPIO Pinout)

a) Bảng phân công chân GPIO Việc phân công chân GPIO của ESP32-S3 được thiết kế đảm bảo không xung đột giữa các chức năng, đồng thời tối ưu hóa cho khả năng deep sleep và external wakeup. Bảng 4.1 trình bày chi tiết phân công chân GPIO cho toàn bộ hệ thống.



Hình 4.2a: Sơ đồ chân kết nối SIM7600CE-T với ESP32-S3

Bảng 4.1: Phân công chân GPIO của ESP32-S3

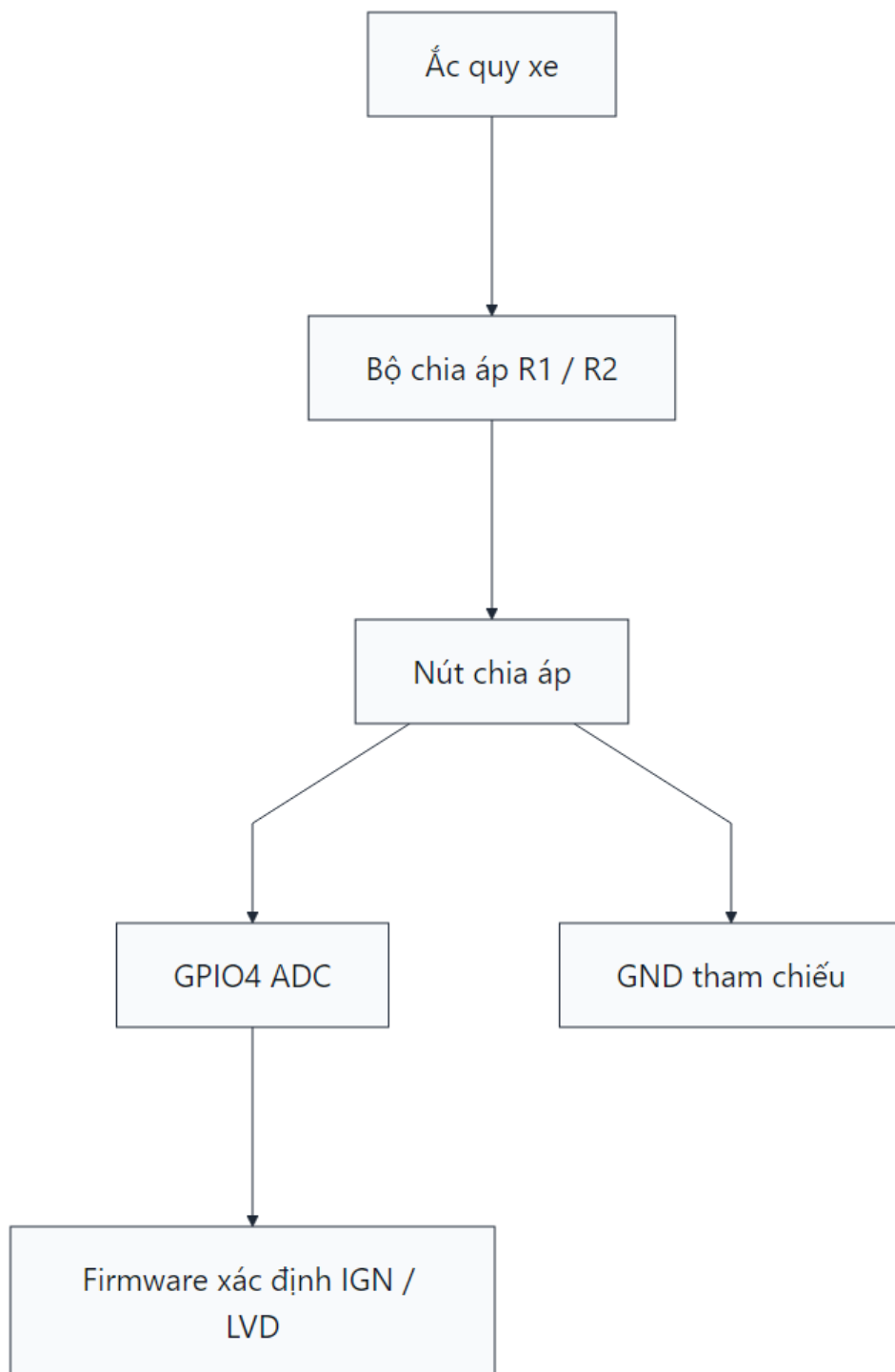
Chân GPIO	Chức năng	Hướng	Mô tả chi tiết
GPIO 2	IGN_IN	Input	Đọc trạng thái khóa điện (IGN) từ xe hoặc qua OBD2
GPIO 4	U_BATT_A DC	Input (ADC)	Đọc điện áp ắc quy xe qua voltage divider (R1=100k, R2=10k)

Bảng 4.1: Phân công chân GPIO của ESP32-S3 (tiếp theo)

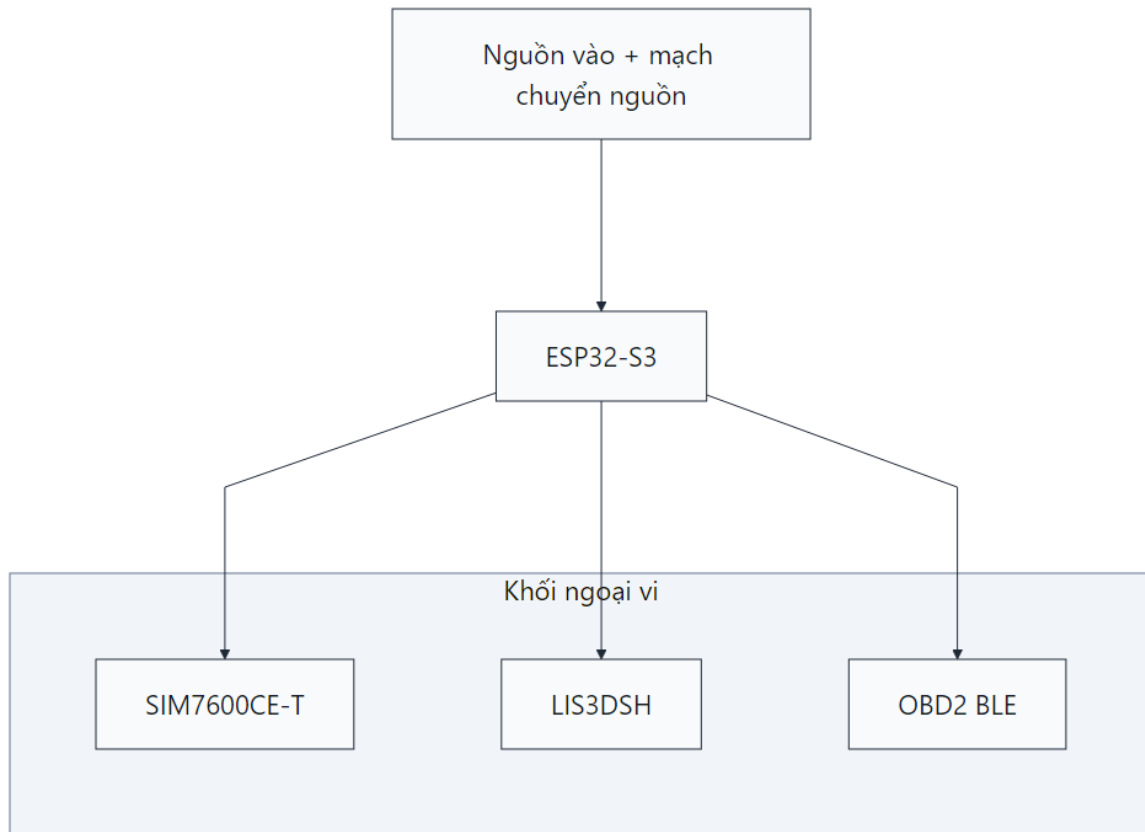
Chân GPIO	Chức năng	Hướng	Mô tả chi tiết
GPIO 5	CHARGER_EN	Output	Điều khiển bật/tắt khối sạc TP5100 (HIGH = sạc, LOW = không sạc)
GPIO 16	MODEM_UART_TX	Output	Truyền dữ liệu UART đến modem SIM7600CE-T
GPIO 17	MODEM_UART_RX	Input	Nhận dữ liệu UART từ modem SIM7600CE-T
GPIO 18	POWER_PATH_EN	Output	Điều khiển nhánh nguồn: LOW = ưu tiên MP2482, HIGH = ưu tiên SX1308 dự phòng
GPIO 19	LVD_STATUS	Input	Đọc trạng thái Low Voltage Disconnect (HIGH = low-voltage, LOW = bình thường)
GPIO 41	LIS3DH_INT	Input (Interrupt)	Nhận tín hiệu ngắt từ cảm biến gia tốc LIS3DH khi phát hiện chuyển động
GPIO 2	LIS3DH_SDA	I/O (I2C)	Đường dữ liệu I2C kết nối với cảm biến LIS3DH
GPIO 1	LIS3DH_SCL	I/O (I2C)	Đường xung nhịp I2C kết nối với cảm biến LIS3DH
GPIO 26	MODEM_PWR-KEY	Output	Điều khiển bật/tắt nguồn modem SIM7600CE-T

b) Các lưu ý về phân công chân Việc phân công chân GPIO cần tuân thủ một số ràng buộc kỹ thuật của ESP32-S3:

- **GPIO 34–39** chỉ hỗ trợ chế độ input, không có điện trở pull-up/pull-down nội. Do đó, các chân này không được sử dụng cho các tín hiệu output trong thiết kế này.
- **Các chân ADC** nằm trong dải GPIO 0–15 và GPIO 25–27, hỗ trợ độ phân giải 12-bit. Chân GPIO 4 được chọn làm kênh ADC đo điện áp vì nằm trong vùng ADC1, cho phép đọc đồng thời với WiFi/BLE.
- **Wake-up cho nhánh IMU** của hệ thống dùng *GPIO wake* trong *light sleep*. Với wiring production hiện tại, chân GPIO 41 (LIS3DH_INT) không thuộc nhóm RTC GPIO nên không thể dùng EXT0/EXT1 cho deep sleep wake.



Hình 4.3: Sơ đồ mạch đo điện áp ắc quy bằng voltage divider và ADC ESP32-S3



Hình 4.4: Sơ đồ nguyên lý mạch điện tổng hợp của hệ thống thiết bị theo dõi

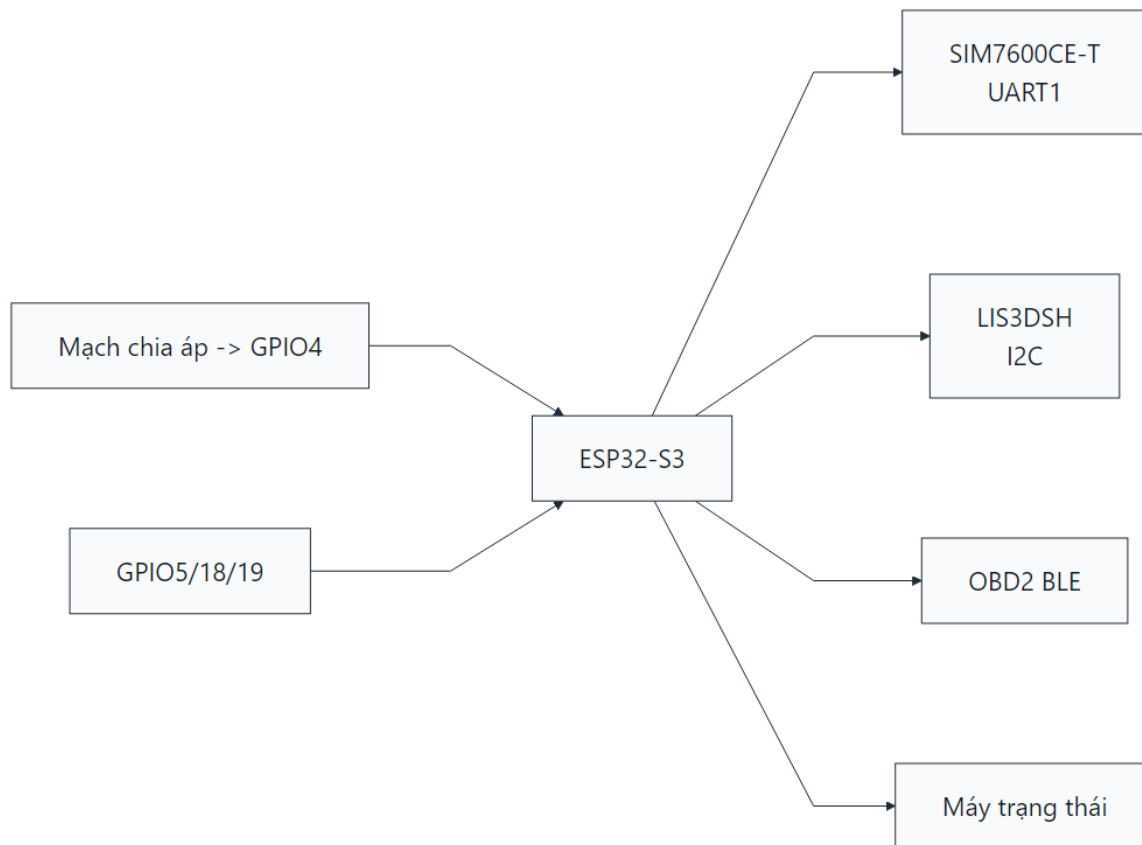
c) Sơ đồ đấu nối tổng thể Sơ đồ đấu nối mô tả cách ánh xạ chân giữa ESP32-S3-WROOM-1 trên PCB chính và các khối phần cứng tích hợp trên bo mạch. Hình 4.5 đã được chuẩn hóa theo pin mapping firmware hiện tại, trong đó GPIO2/1 dành cho I2C IMU, GPIO41/42 nhận ngắt IMU, GPIO26 điều khiển PWR-KEY, GPIO19 nhận LVD_STATUS, và GPIO4 đo U_{batt} .

Bên cạnh các chân đã đưa vào `pin_map.h`, netlist phần cứng còn thể hiện đầy đủ cụm microSD J4 theo chuẩn SDMMC 4-bit (SD-DAT0...3, SD-CMD, SD-CLK, SD-CD) với pull-up ngoài trên các đường dữ liệu/lệnh. Điều này là nền tảng để firmware triển khai `sd_log_store` và `offline_queue` cho cơ chế lưu đệm cục bộ.

f) Thiết kế mạch quản lý nguồn (Power Management Circuit)

a) Tổng quan kiến trúc nguồn Mạch quản lý nguồn là thành phần thiết yếu của hệ thống thiết bị theo dõi, bảo đảm thiết bị hoạt động liên tục ngay cả khi ắc quy xe yếu hoặc mất điện. Kiến trúc nguồn gồm sáu khối chức năng chính:

1. **LDO 3.3V (AP2112-3.3):** Hạ từ bus 5V chính xuống 3.3V cấp ESP32-S3



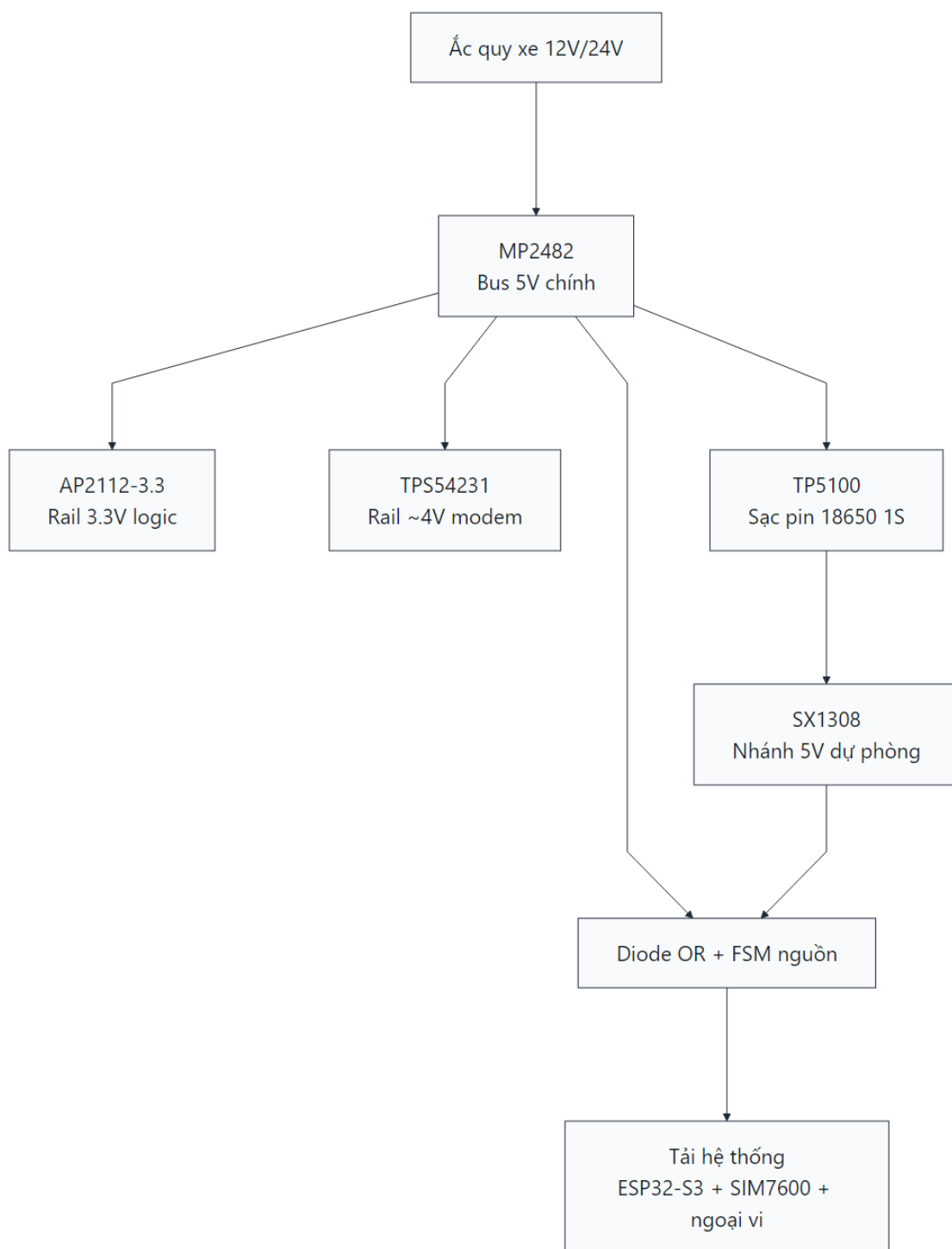
Hình 4.5: Sơ đồ đầu nối tổng thể giữa ESP32-S3 và các khối phần cứng tích hợp

2. **Buck 5V (MP2482):** Chuyển đổi 12V/24V xuống 5V bus chính
3. **Buck 4V (TPS54231):** Chuyển đổi 12V/24V xuống 4V cấp modem SIM7600CE-T
4. **Boost 5V (SX1308):** Tăng áp từ pin 18650 1S (3.7V) lên 5V dự phòng
5. **Mạch chuyển nguồn Diode-OR + điều khiển EN:** Tự động duy trì nguồn liên tục giữa nhánh chính và nhánh dự phòng
6. **Mạch sạc TP5100 + LVD (khối LVD/ADC):** Sạc pin 1S và giám sát ngưỡng điện áp bảo vệ ắc quy

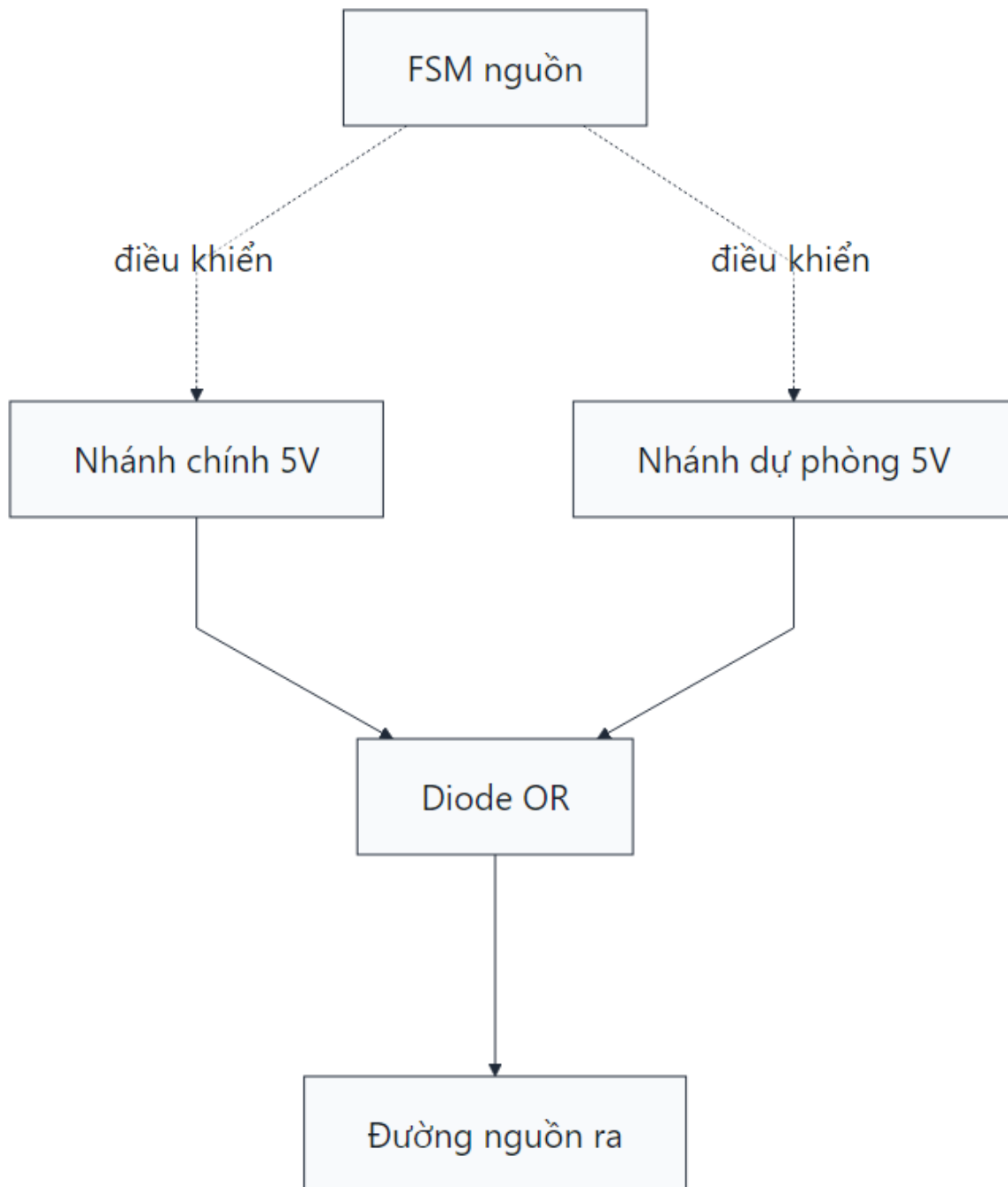
b) LDO 3.3V cho ESP32-S3 (AP2112-3.3) Khối 3.3V dùng AP2112-3.3 như một LDO để hạ từ bus 5V chính xuống 3.3V cấp cho ESP32-S3.

Bảng 4.2: Khối LDO 3.3V

Khối	IC	Input	Output	Tải
LDO 3.3V	AP2112-3.3	5V	3.3V	ESP32-S3



Hình 4.6: Kiến trúc tổng thể mạch quản lý nguồn

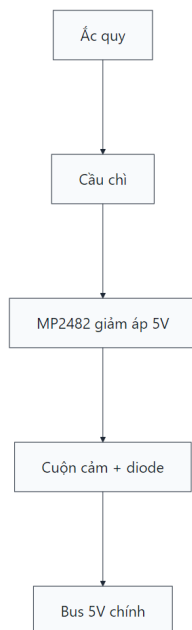


Hình 4.6a: Kiến trúc chuyển nguồn giữa nguồn xe và pin dự phòng

c) Buck 5V bus chính (MP2482) Khối buck 5V dùng **MP2482** tạo bus 5V chính từ 12V/24V.

Bảng 4.3: Khối Buck 5V

Khối	IC	Input	Output	Tải
Buck 5V	MP2482	12–24V	5V	Bus 5V chính



Hình 4.7: Kiến trúc nhánh buck 5V chính dùng MP2482

d) Buck 3.8V/4V cho modem (TPS54231) Khối buck modem dùng **TPS54231** để hạ 12V/24V xuống khoảng 4V cấp cho SIM7600CE-T.

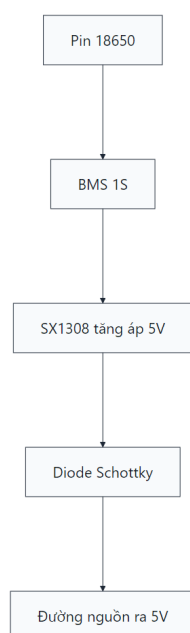
Bảng 4.4: Khối Buck 3.8V/4V cho modem

Khối	IC	Input	Output	Tải
Buck 3.8V	TPS54231	12–24V	4V	SIM7600CE-T

e) Boost 5V dự phòng và Power Path Khối boost dùng **SX1308** để nâng áp từ pin 18650 1S (3.7V) lên 5V khi chạy nguồn dự phòng.

Bảng 4.5: Khối Boost 5V dự phòng

Khối	IC	Input	Output	Tải
Boost 5V	SX1308	Battery 3.7V	5V	Nguồn dự phòng từ pin



Hình 4.8: Kiến trúc nhánh nguồn dự phòng 5V từ pin 18650 qua SX1308

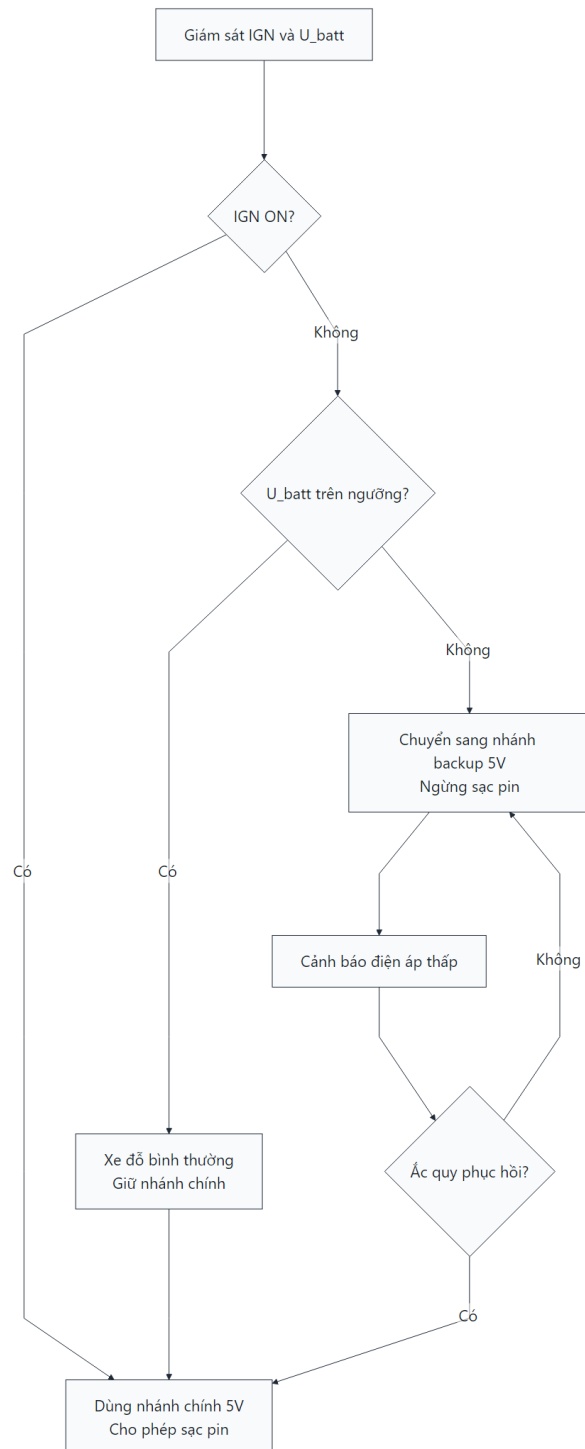
Mạch chuyển nguồn khi vận hành được triển khai theo **diode OR** giữa nhánh 5V chính (MP2482) và nhánh 5V dự phòng (SX1308), phối hợp điều khiển GPIO để đảm bảo chuyển nguồn liên tục.

Bảng 4.6: Logic chuyển nguồn tự động

Chế độ	IGN	U_{batt}	Nguồn Tracker	Sạc Pin	Cảnh báo
Xe chạy	ON	Profile 12V: $U_{\text{batt}} \geq 13.0 \text{ V}$; Profile 24V: $U_{\text{batt}} \geq 26.0 \text{ V}$	Ắc quy	Có	Không
Xe đỗ	OFF	Profile 12V: $U_{\text{batt}} > 12.0 \text{ V}$; Profile 24V: $U_{\text{batt}} > 24.0 \text{ V}$	Ắc quy	Không	Không
Ắc quy yếu	OFF	Profile 12V: $U_{\text{batt}} \leq 12.0 \text{ V}$; Profile 24V: $U_{\text{batt}} \leq 24.0 \text{ V}$	Pin 18650 1S	Không	Có

Bảng 4.6: Logic chuyển nguồn tự động (tiếp theo)

Chế độ	IGN	U_{batt}	Nguồn Tracker	Sạc Pin	Cảnh báo
Ắc quy phục hồi	OFF	Profile 12V: $U_{\text{batt}} \geq 12.2 \text{ V}$; Profile 24V: $U_{\text{batt}} \geq 24.4 \text{ V}$	Ắc quy	Không	Có



Hình 4.9: Sơ đồ chuyển nguồn giữa nhánh chính và nhánh dự phòng

f) Mạch sạc pin TP5100 Khối sạc pin dùng **TP5100** ở cấu hình **1 cell**, nhận **5V** từ **MP2482** và sạc pin 18650 1S ở mức 4.2V.

Bảng 4.7: Khối sạc TP5100

Khối	IC	Input	Output	Tải
Sạc pin	TP5100	5V từ MP2482	4.2V	Pin 18650 1S

Dòng sạc của TP5100 được cấu hình bằng điện trở thiết lập dòng sạc và còn phụ thuộc điều kiện nhiệt của mạch, không cố định một giá trị trong mọi điều kiện.

g) Mạch giám sát điện áp Low Voltage Disconnect (LVD) Hệ thống dùng ADC ($R1 = 100\text{ k}\Omega$, $R2 = 10\text{ k}\Omega$) kết hợp khối LVD phần cứng để giám sát điện áp.

Ngưỡng profile:

- Profile 12V: $OFF = 12.0\text{ V}$, $ON = 12.2\text{ V}$
- Profile 24V: $OFF = 24.0\text{ V}$, $ON = 24.4\text{ V}$

Tín hiệu LVD_STATUS đọc tại GPIO19 với quy ước vận hành: **HIGH** = low-voltage, **LOW** = bình thường.

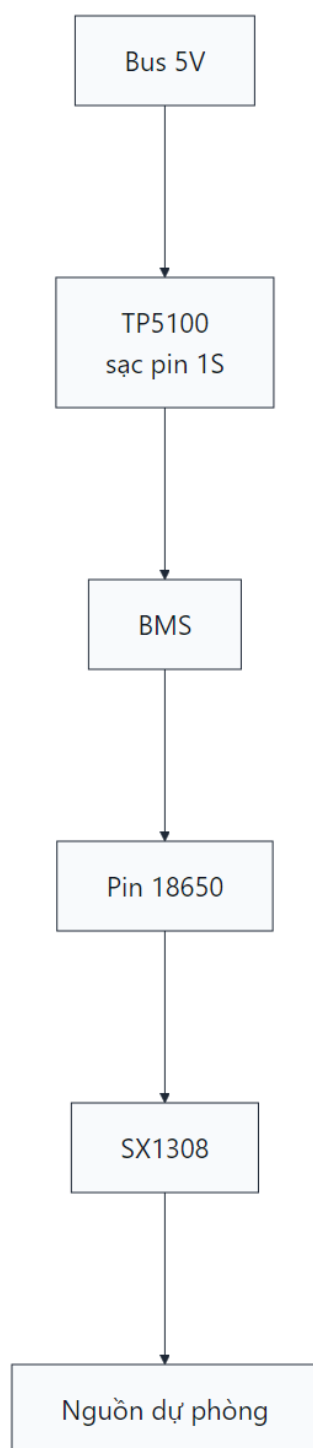
g) Thiết kế vỏ hộp bảo vệ (Enclosure Design)

a) Yêu cầu thiết kế Vỏ hộp bảo vệ cần đáp ứng các yêu cầu sau:

- **Kích thước nhỏ gọn:** Phù hợp lắp đặt dưới táp-lô hoặc gần cổng OBD2, kích thước tối đa khoảng $100 \times 70 \times 35\text{ mm}$
- **Tản nhiệt:** Đảm bảo thông gió cho các IC công suất (MP2482, SX1308) với tổn hao nhiệt tổng cộng có thể lên đến 3–4W
- **Bảo vệ:** Chống bụi, chống nước cơ bản (IP54 hoặc tương đương) cho môi trường bên trong xe
- **Tiếp cận anten:** Bố trí vị trí phù hợp cho anten 4G/LTE, anten GNSS và anten BLE để đảm bảo chất lượng thu phát tín hiệu

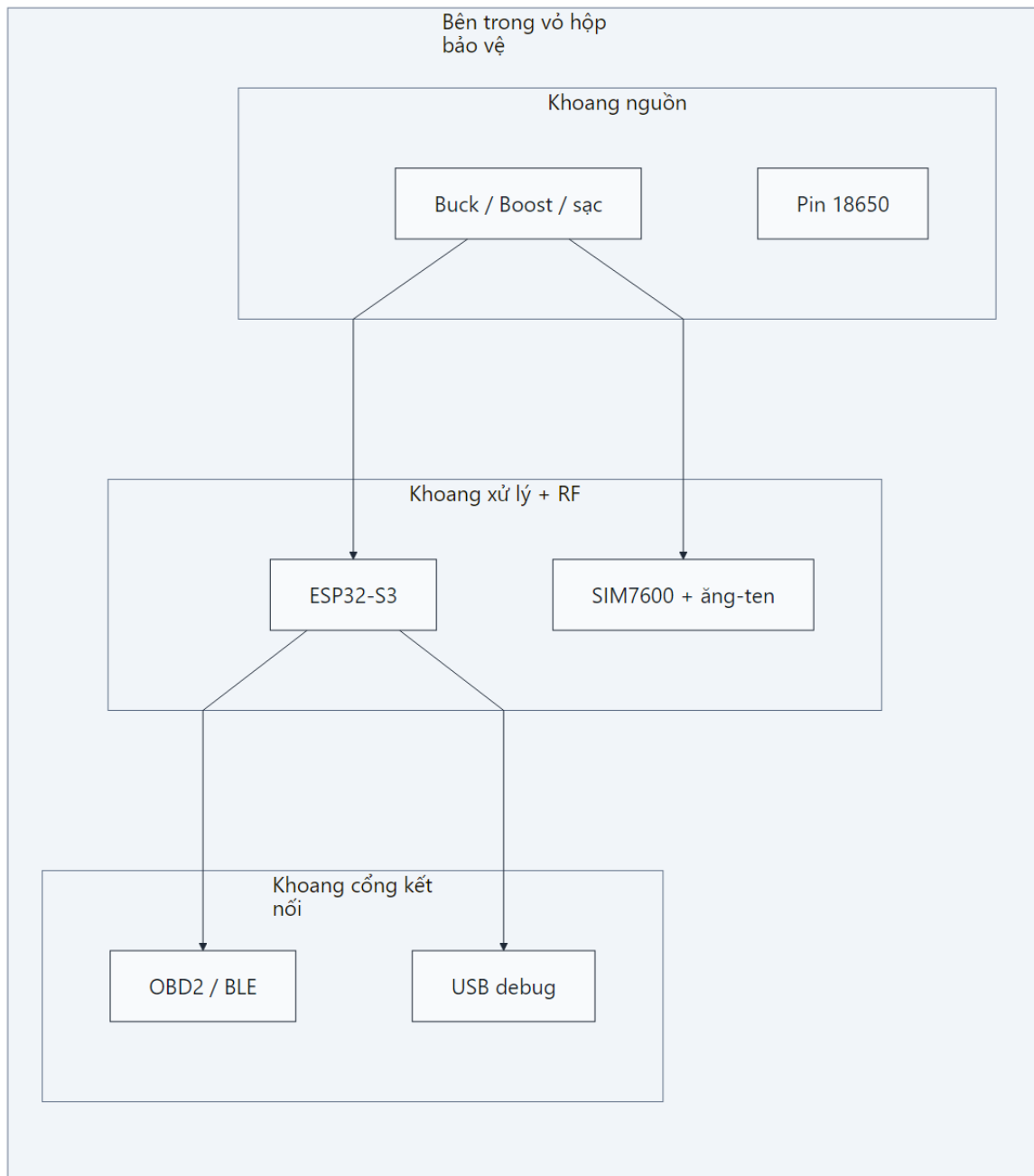
b) Bố cục bên trong Bố cục bên trong vỏ hộp được thiết kế theo nguyên tắc phân vùng chức năng:

Bố cục bên trong được chia thành ba vùng rõ ràng: cụm xử lý và truyền thông (ESP32-S3, SIM7600CE-T), cụm nguồn (AP2112-3.3, MP2482, TPS54231, SX1308, TP5100, khối



Hình 4.10: Kiến trúc sạc pin TP5100 và đường cấp nguồn dự phòng

LVD) và cụm lưu trữ năng lượng (pin 18650 1S + mạch bảo vệ 1S). Hình 4.11 thể hiện cách nhóm các khối phần cứng này trong vỏ hộp cùng vị trí anten và cổng OBD2.



Hình 4.11: Sơ đồ bố cục bên trong vỏ hộp bảo vệ

c) Các lưu ý thiết kế Tản nhiệt: Các khối công suất (MP2482, SX1308) được bố trí tại vị trí thông thoáng, tách khỏi cảm biến nhiệt độ; khe thông gió đặt ở mặt bên và mặt dưới vỏ hộp. IC MP2482 được gắn heatsink khi vận hành công suất cao.

Anten: Anten 4G/LTE và GNSS đặt ở mặt trên vỏ hộp, hướng lên trên để tối ưu thu sóng. Anten BLE (tích hợp trong ESP32-S3) hướng về cổng OBD2 để rút ngắn khoảng

cách kết nối với adapter vgate iCar Pro. Các anten cách nhau tối thiểu 30 mm nhằm giảm nhiễu tương hỗ.

Chống nhiễu điện từ (EMI): Mạch công suất (Buck, Boost) được bố trí tách xa mạch tín hiệu (UART, I2C, ADC); dây nguồn và dây tín hiệu đi riêng, không chạy song song.

Kết nối: Cổng kết nối OBD2, cổng SIM card và cổng USB (cho nạp firmware/debug) được bố trí ở mặt bên vỏ hộp, dễ tiếp cận khi lắp đặt và bảo trì.

4.1.2. Triển khai firmware, hạ tầng máy chủ và giao diện điều khiển

Ngoài các mô-đun truyền thông và điều phối trạng thái, firmware đã triển khai luồng **SD card log** theo chu trình vận hành rõ ràng:

1. **Khởi tạo và mount thẻ SD:** `sd_log_store_init()` và `sd_log_store_mount()` khởi tạo host SDMMC 4-bit, mount FATFS vào `/sdcard`, tạo cây thư mục làm việc và nạp metadata phiên ghi.
2. **Ghi bản ghi cục bộ:** `offline_queue_enqueue()` chuẩn hóa bản ghi dữ liệu vận hành/sự kiện, sau đó `sd_log_store_append()` ghi xuống hàng đợi file theo thứ tự seq.
3. **Phát lại khi mạng phục hồi:** `offline_queue_replay_tick()` đọc bản ghi chờ, gửi lại theo topic phù hợp; bản ghi quan trọng đi QoS 1 và được chốt bằng `offline_queue_handle_publish_ack()`.
4. **Kiểm soát dung lượng:** `sd_log_store_gc_if_needed()` dọn dữ liệu ít quan trọng hơn đã ACK khi vượt ngưỡng quota, giữ ổn định không gian lưu trữ.
5. **Kết thúc phiên ghi an toàn:** firmware gọi `sd_log_store_stop_session()` và `sd_log_store_unmount()` khi dừng phiên để giảm nguy cơ lỗi hệ thống tệp.

Trong phạm vi đồ án, luồng này được đánh giá theo giả định thẻ đã được gắn sẵn ngay từ lúc khởi động để ưu tiên độ ổn định. Kịch bản cắm hoặc rút thẻ trong khi thiết bị đang chạy chưa phải mục tiêu chính của phiên bản hiện tại.

4.2. Chế tạo và lắp ráp hệ thống – Manufacture and Assembly

4.2.1. Lắp ráp mạch điện tử (Electronics Assembly)

Phần lắp ráp không chỉ nhằm ghép các linh kiện thành một bo mạch hoàn chỉnh, mà còn là bước kiểm chứng xem các giả định thiết kế ở Chương 3 có đúng vững khi đi vào hiện thực hay không. Vì vậy, phần này được trình bày theo đúng thứ tự mà một người triển khai thực tế sẽ quan tâm: cần có những vật tư gì, lắp theo thứ tự nào, và kiểm tra ra sao để hạn chế lỗi ngay từ đầu.

a) Danh sách vật liệu (Bill of Materials)

Bảng 4.6: Danh sách vật liệu đầy đủ cho hệ thống thiết bị theo dõi

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
1	ESP32-S3-WROOM-1 (N16R8)	Cái	1	95.000	MCU trung tâm của bo mạch
2	IC cảm biến LIS3DH	Cái	1	40.000	Cảm biến gia tốc 3 trục, giao tiếp I2C
3	vgate iCar Pro (OBD2 BLE)	Cái	1	367.000	Adapter OBD2 BLE 4.0, tương thích ESP32-S3
4	SIMCom SIM7600CE-T	Bá»™	1	650.000	Modem LTE Cat-4 tích hợp GNSS
5	Pin 18650 1S Li-ion 3500mAh	Cái	1	79.000	Loại có protection board
6	IC sạc TP5100 + mạch phụ trợ	Cái	1	25.000	Input 5V từ MP2482, output 4.2V, dòng theo điện trở thiết lập
7	Mạch LDO AP2112-3.3	Cái	1	5.000	5V -> 3.3V cho ESP32-S3
8	Mạch buck MP2482 5V	Cái	1	17.000	Bus 5V chính
9	Mạch buck TPS54231 4V	Cái	1	9.000	12–24V -> 4V cấp modem SIM7600CE-T
10	Mạch boost SX1308 5V dự phòng	Cái	1	12.000	Nguồn dự phòng từ pin 18650 1S
11	Khối giám sát LVD + ADC	Cái	1	8.000	Giám sát LVD (GPIO19)
12	Diode Schottky (SS54/SS34)	Cái	2	2.500	Diode OR dự phòng
13	Mạch bảo vệ pin 1S (BMS)	Cái	1	12.000	Bảo vệ pin 18650 1S
14	Điện trở (10k, 2.2k, v.v.)	Gói	1	5.000	Voltage divider, pull-up/pull-down

*Bảng 4.6: Danh sách vật liệu đầy đủ cho hệ thống thiết bị theo dõi
(tiếp theo)*

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
15	Tụ điện (100uF, 220uF, v.v.)	Gói	1	5.000	Lọc nhiễu, decoupling
16	Connector, Header Pin	Gói	1	55.000	Header, jack OBD2 và đầu nối nguồn
17	PCB (nếu tự thiết kế)	Cái	1	20.000	PCB 2 lớp, kích thước 50x50 mm
18	Vỏ bảo vệ	Cái	1	25.000	Vỏ nhựa hoặc kim loại
19	Dây nối, cáp điện	Mét	—	60.000	Dây điện, anten 4G/GNSS, khay SIM
20	Linh kiện phụ trợ khác	—	—	20.000	Cầu chì, công tắc, LED chỉ thị

Bảng 4.7: Tổng chi phí ước tính

Hạng mục	Chi phí (VND)
Thành phần chính (MCU, cảm biến, modem, OBD2, pin)	1.231.000
Hệ thống quản lý nguồn	93.000
Kết nối, RF và linh kiện phụ trợ	145.000
PCB và vỏ bảo vệ	45.000
Tổng cộng	1.514.000

b) Quy trình lắp ráp Để giảm rủi ro lỗi dây chuyền, quy trình lắp ráp không đi theo hướng hàn toàn bộ rồi mới kiểm tra, mà chia thành từng cụm chức năng và kiểm tra ngay sau mỗi bước. Trình tự thực hiện như sau:

Bước 1 - Kiểm tra linh kiện: Kiểm tra toàn bộ IC, linh kiện và bán thành phẩm trước khi lắp ráp. Test riêng từng khối chức năng chính (ESP32-S3-WROOM-1, AP2112-3.3, MP2482, TPS54231, SX1308, TP5100, SIM7600CE-T, khối LVD) để bảo đảm hoạt động đúng.

Bước 2 - Lắp ráp khối nguồn trên PCB: Hàn và kiểm tra MP2482 để tạo bus 5V từ nguồn ắc quy xe (12V hoặc 24V). Lắp AP2112-3.3 cấp riêng cho ESP32-S3, TPS54231 tạo rail 4V cho modem SIM7600CE-T, SX1308 tạo 5V dự phòng từ pin 18650 1S, diode OR giữa nhánh 5V chính và nhánh 5V dự phòng, cùng TP5100 kết nối với pin và mạch bảo vệ 1S.

Bước 3 - Lắp ráp khối xử lý và điều khiển: Hàn ESP32-S3-WROOM-1, LIS3DH và các linh kiện liên quan lên PCB chính. Kiểm tra các chân GPIO theo bảng phân công (Bảng 4.1) và cấp nguồn theo từng rail chức năng (3.3V logic, 4V modem, 5V bus/backup).

Bước 4 - Tích hợp các thành phần còn lại: Lắp modem SIM7600CE-T lên bo mạch, kiểm tra UART1 (GPIO16, GPIO17) và chân PWR-KEY (GPIO26), cấu hình lấy dữ liệu GNSS qua AT+CGNSINF/AT+CGNSTST trên cùng UART, kiểm tra cảm biến LIS3DH trên I2C (GPIO2, GPIO1), và hiệu chuẩn mạch đo điện áp ắc quy tại GPIO4 (ADC).

Bước 5 - Kiểm tra tích hợp: Nạp firmware cơ bản để kiểm tra từng chức năng: đọc ADC, điều khiển GPIO, giao tiếp UART với modem, quét BLE, đọc I2C từ LIS3DH. Kiểm tra chuyển nguồn tự động bằng cách thay đổi điện áp đầu vào.

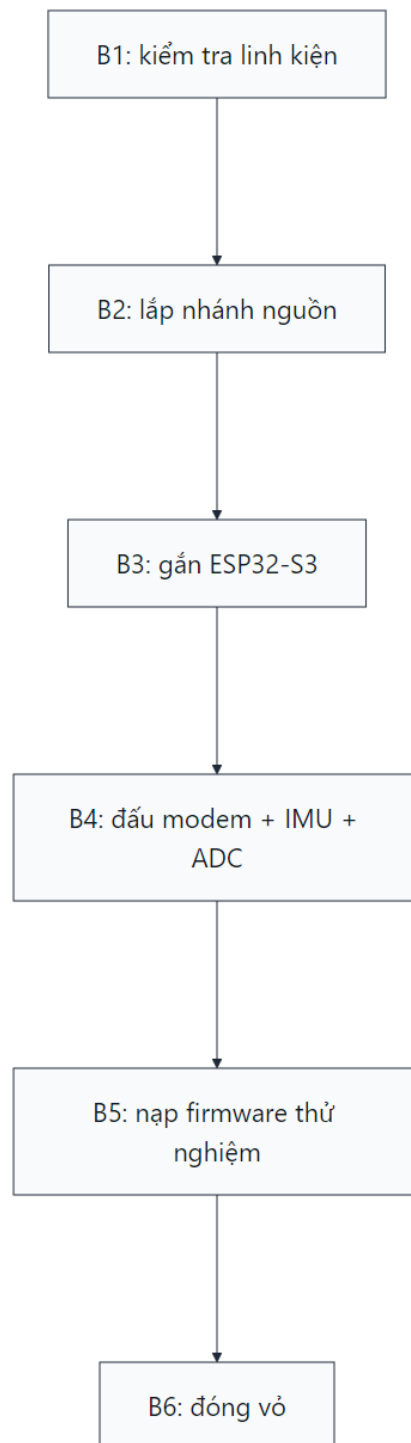
Bước 6 - Lắp ráp vào vỏ: Định vị bo mạch chính, cụm anten và pin dự phòng bên trong vỏ hộp theo bố cục đã thiết kế. Cố định bằng ốc vít hoặc keo nhiệt. Kết nối anten 4G/LTE và GNSS. Đóng nắp vỏ hộp và kiểm tra tổng thể.

c) Kiểm tra và hiệu chuẩn Sau khi hoàn tất lắp ráp cơ khí và điện, hệ thống cần được hiệu chuẩn để đảm bảo các giá trị đo và cơ chế chuyển nguồn phản ánh đúng điều kiện vận hành thực tế của xe. Các hạng mục kiểm tra chính gồm:

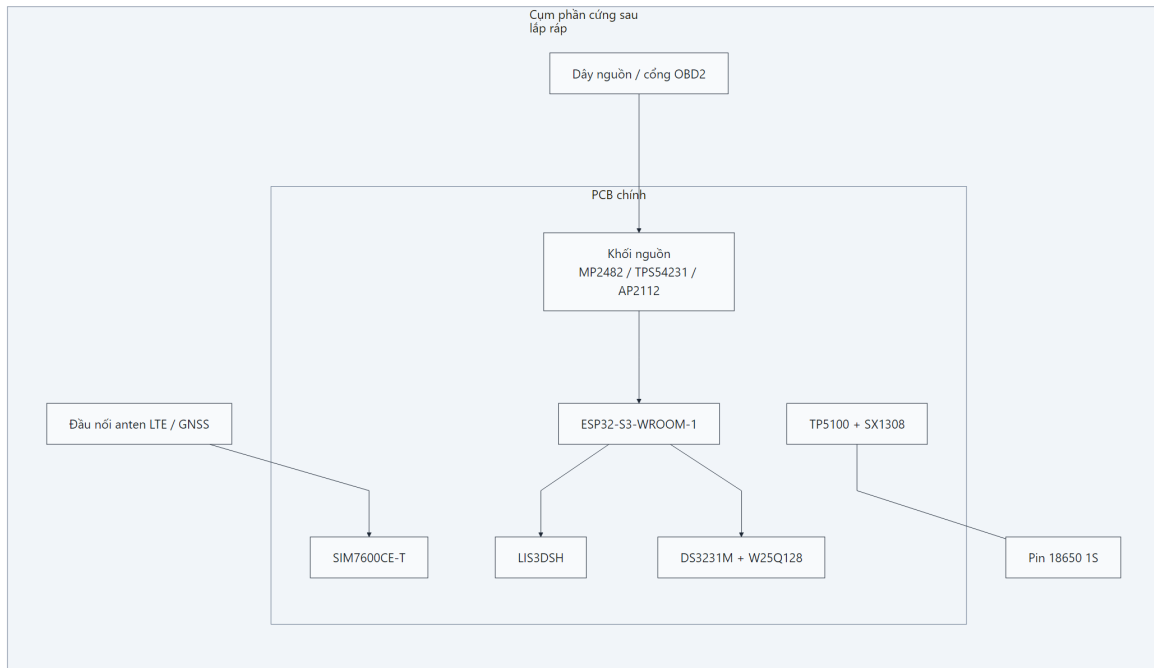
- **Hiệu chuẩn ADC:** So sánh giá trị điện áp đọc từ ADC với giá trị đo từ đồng hồ vạn năng (multimeter). Điều chỉnh hệ số hiệu chỉnh trong firmware nếu cần.
- **Kiểm tra chuyển nguồn:** Mô phỏng tình huống ắc quy yếu theo profile cấu hình (ví dụ hệ 12V giảm từ 12V xuống dưới 12V), xác nhận hệ thống tự động chuyển sang pin dự phòng.
- **Kiểm tra sạc pin:** Xác nhận khối sạc TP5100 hoạt động đúng: sạc khi IGN_{ON} , ngừng sạc khi IGN_{OFF} hoặc U_{batt} thấp.
- **Kiểm tra giao tiếp:** Xác nhận modem phản hồi lệnh AT qua UART, cảm biến LIS3DH trả về dữ liệu qua I2C, kết nối BLE với OBD2 adapter thành công.

4.2.2. Lắp đặt trong xe (Vehicle Installation)

Nếu bước lắp ráp trả lời câu hỏi “thiết bị có hoạt động được không”, thì bước lắp đặt trong xe trả lời câu hỏi “thiết bị có hoạt động ổn định trong môi trường thật hay không”.



Hình 4.12: Quy trình lắp ráp phần cứng trên PCB



Hình 4.13: Sơ đồ bố trí linh kiện sau khi lắp ráp

Phần này vì thế tập trung vào những quyết định ảnh hưởng trực tiếp đến chất lượng thu sóng, độ an toàn điện và khả năng bảo trì sau này.

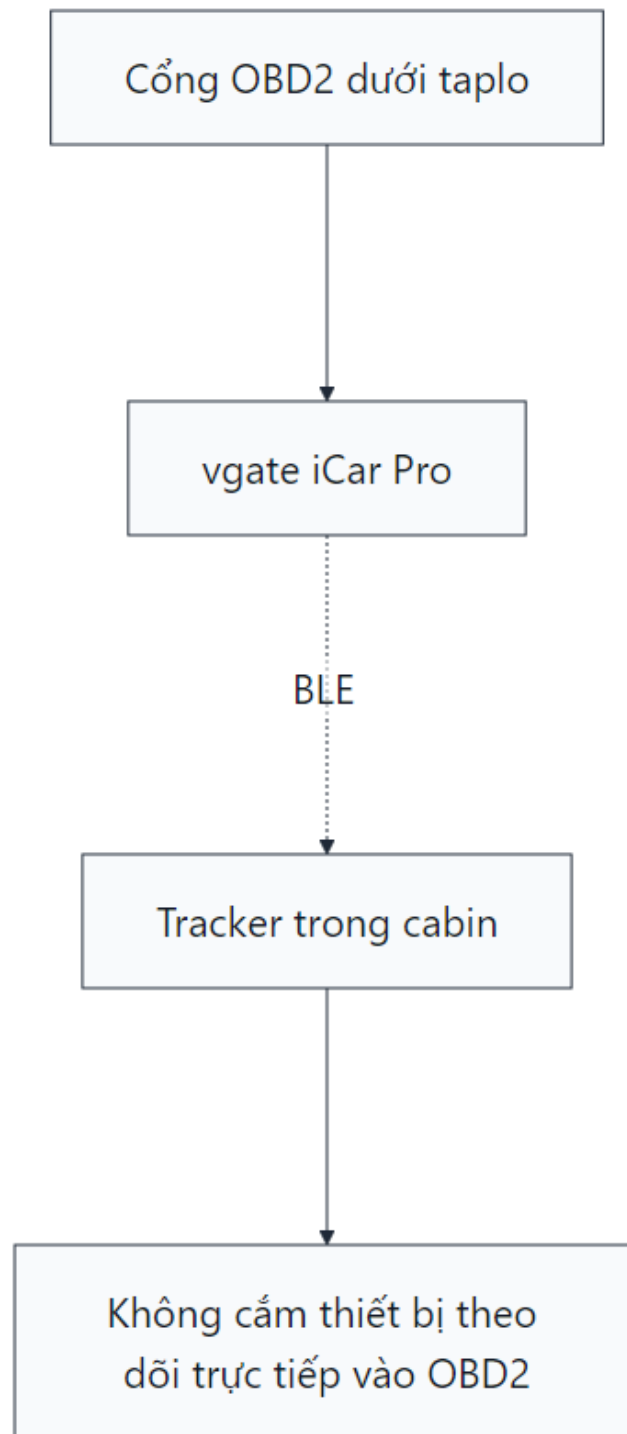
a) Vị trí lắp đặt Thiết bị theo dõi có thể được lắp ở nhiều vị trí khác nhau trong xe, tuy nhiên việc chọn vị trí không chỉ phụ thuộc vào chỗ trống. Người triển khai cần đồng thời cân nhắc độ kín đáo, độ thuận tiện khi đi dây, khoảng cách tới cổng OBD2 và điều kiện thu sóng cho GNSS/4G.

- **Dưới bảng táp-lô:** Vị trí khuyến nghị, gần cổng OBD2 (thường nằm dưới vô-lăng bên trái), dễ dàng kết nối nguồn và OBD2. Không gian đủ cho vỏ hộp và không can thiệp đến vùng hoạt động của tài xế.
- **Dưới ghế lái hoặc ghế phụ:** Vị trí thay thế khi không gian dưới táp-lô không đủ. Cần kéo dài cáp kết nối OBD2.
- **Trong hộp cầu chì (fuse box):** Vị trí kín đáo, phù hợp cho ứng dụng theo dõi không cần người dùng can thiệp.

b) Kết nối với cổng OBD2 Cổng OBD2 (16 chân) cung cấp cả nguồn điện và giao tiếp chẩn đoán. Trong cấu hình hiện tại, thiết bị theo dõi chỉ lấy Battery Power ở chân 16 và mass tại chân 4/5, còn dữ liệu OBD2 đi qua adapter vgate iCar Pro cắm trực tiếp vào cổng xe rồi truyền không dây về ESP32-S3 qua BLE.

Lưu ý: Adapter vgate iCar Pro được cắm trực tiếp vào cổng OBD2 của xe. ESP32-S3 giao tiếp với adapter này qua BLE, không cần kết nối dây vật lý cho phần dữ liệu OBD2.

Chỉ có nguồn điện từ ắc quy xe (12V hoặc 24V) và GND được lấy từ cổng OBD2 thông qua dây nối riêng.



Hình 4.14: Minh họa vị trí cổng OBD2 trên xe và cách kết nối

c) Đi dây và kết nối anten **Dây nguồn:** Dây nguồn từ cổng OBD2 đến thiết bị theo dõi (12V hoặc 24V tùy hệ xe) sử dụng dây điện tiết diện tối thiểu 0.5 mm² (AWG 22), có bọc cách điện tốt. Cầu chì (fuse) 5A được đặt tại đầu dây nguồn gần cổng OBD2 để

bảo vệ mạch.

Anten 4G/LTE và GNSS: Anten được đặt hướng lên mặt trên táp-lô hoặc gần kính chắn gió phía trước để đảm bảo thu sóng tốt nhất. Dây anten (cáp đồng trục) được đi dọc theo khung xe, tránh chạy song song với dây điện nguồn.

Anten BLE: Do BLE sử dụng anten PCB tích hợp trên ESP32-S3, cần đảm bảo khoảng cách từ thiết bị theo dõi đến adapter vgate iCar Pro (cắm ở cổng OBD2) không quá 3 mét và không bị che chắn bởi vật liệu kim loại dày.

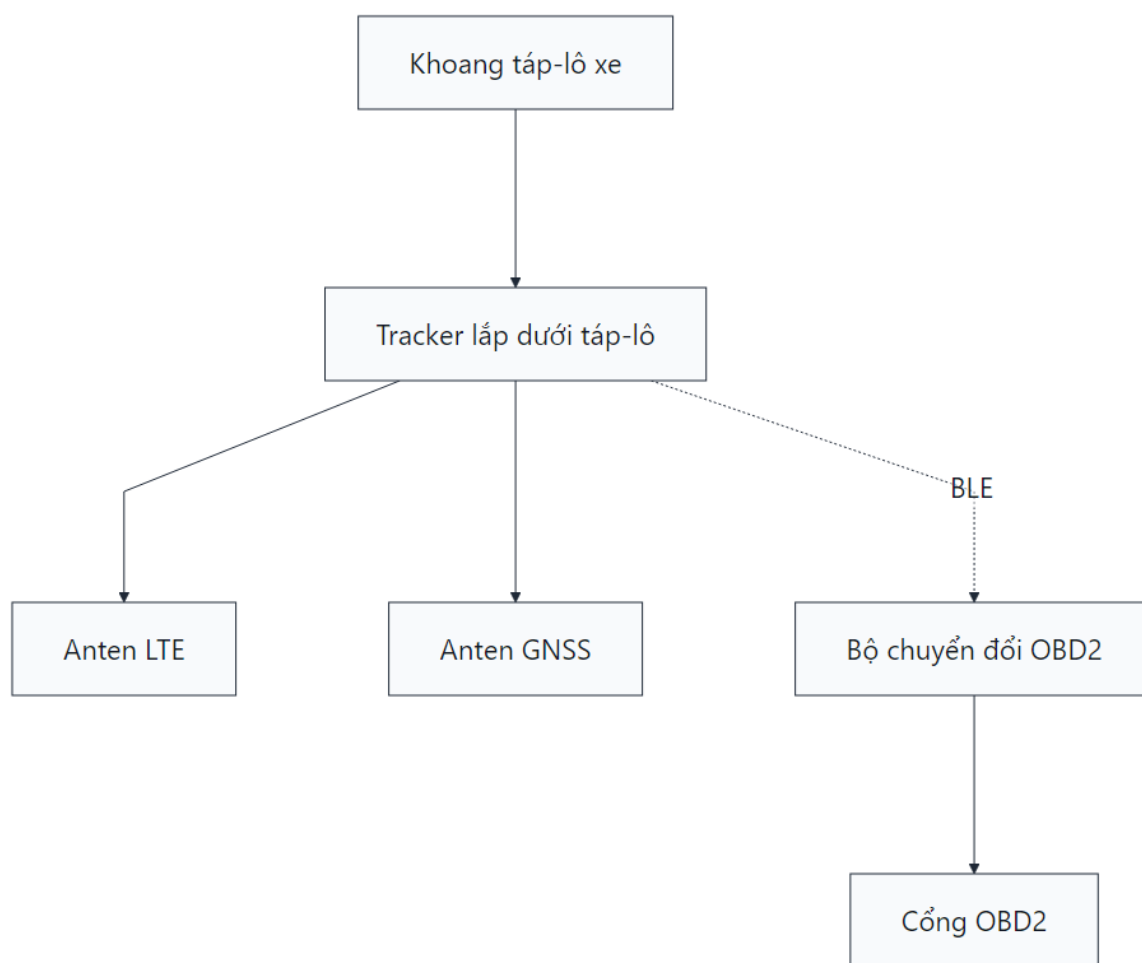
d) Quản lý cáp và chống nhiễu»...u

- **Cố định cáp:** Tất cả các dây nối được cố định bằng dây rút nhựa (cable tie) và kẹp cáp, tránh các dây bị rung lắc gây tiếng ồn hoặc hư hỏng tiếp xúc.
- **Chống nhiễu EMI:** Dây tín hiệu UART (từ ESP32 đến modem) được sử dụng loại cáp xoắn đôi (twisted pair) hoặc cáp có bọc chống nhiễu. Dây nguồn và dây tín hiệu được đi riêng, không chạy song song để giảm nhiễu điện từ từ hệ thống điện của xe.
- **Chống ẩm:** Các mối nối điện được bọc bằng keo nhiệt hoặc ống co nhiệt (heat shrink tube) để chống ẩm và chống oxy hóa trong môi trường xe.

e) Kiểm tra sau lắp đặt Sau khi lắp đặt xong, cần kiểm tra lại toàn bộ theo góc nhìn vận hành thực tế, tức là không chỉ xem thiết bị có lên nguồn hay không mà còn đánh giá được nó có giao tiếp tốt, chuyển chế độ đúng và duy trì kết nối ổn định trong xe hay không.

1. **Kiểm tra nguồn điện:** Đo điện áp tại đầu vào thiết bị, xác nhận điện áp ắc quy xe (12V hoặc 24V) được cấp đầy đủ
2. **Kiểm tra kết nối BLE:** Xác nhận ESP32-S3 kết nối được với adapter vgate iCar Pro và đọc dữ liệu OBD2 (IGN, RPM, tốc độ)
3. **Kiểm tra GNSS:** Xác nhận modem SIM7600CE-T bắt được vệ tinh và trả về tọa độ GPS chính xác (sai số < 5 mét)
4. **Kiểm tra 4G/LTE:** Xác nhận modem đăng ký mạng thành công, gửi được dữ liệu lên máy chủ qua MQTT
5. **Kiểm tra chuyển nguồn:** Tắt máy xe (IGN OFF), xác nhận thiết bị chuyển sang chế độ tiết kiệm năng lượng và sử dụng pin dự phòng khi cần
6. **Kiểm tra deep sleep:** Xác nhận ESP32-S3 vào chế độ deep sleep khi xe đỗ, và đánh thức đúng khi phát hiện rung động (qua LIS3DH) hoặc đến chu kỳ heartbeat

Sau khi phần cứng đã được đặt vào xe và các kết nối cơ bản đã được xác nhận, lớp quyết định thiết bị phản ứng như thế nào trong từng tình huống chính là firmware. Vì vậy, phần tiếp theo chuyển từ lắp đặt cơ khí - điện sang triển khai phần mềm nhúng.



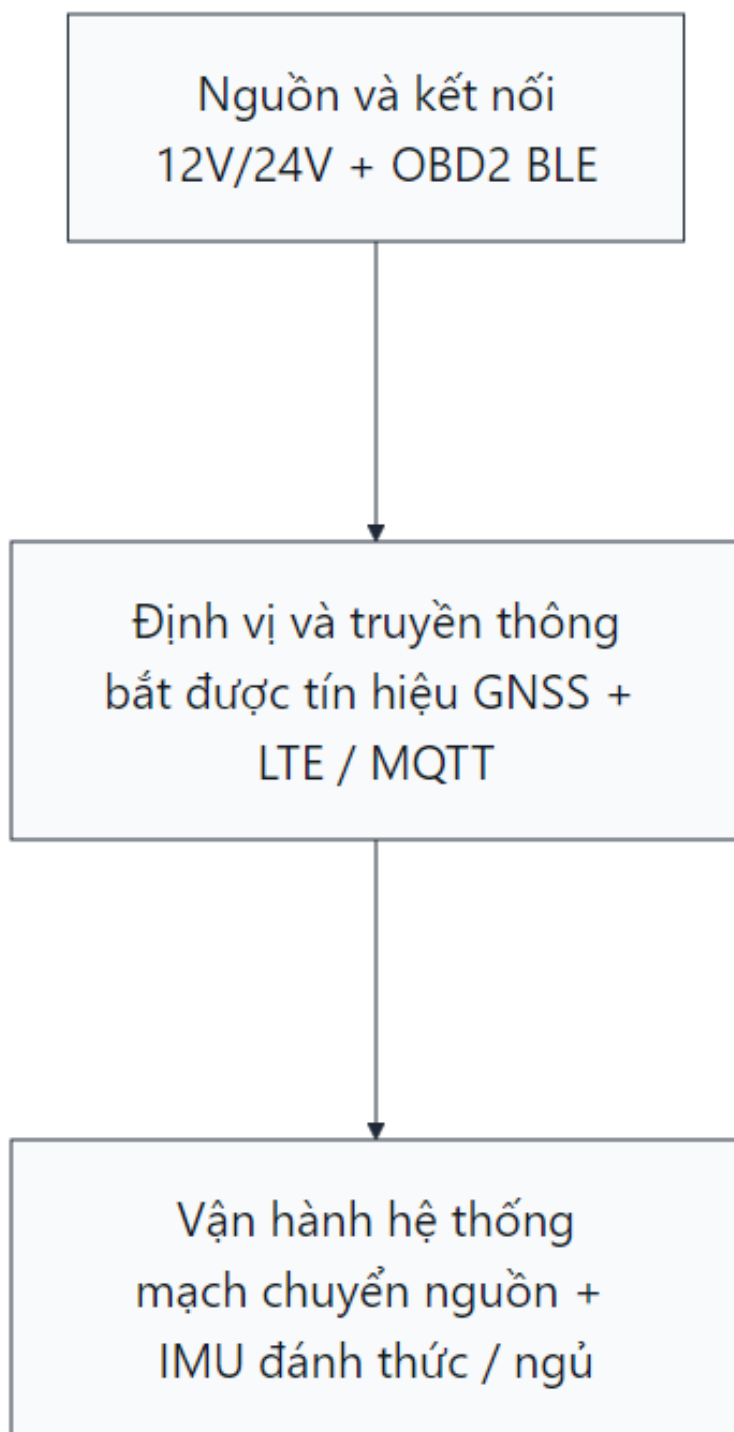
Hình 4.15: Minh họa lắp đặt thiết bị theo dõi trong xe và đi dây

4.2.3. Triển khai Firmware

Phần này trình bày quá trình triển khai firmware cho thiết bị theo dõi xe IoT, từ nền tảng phát triển đến cơ chế vận hành. Nội dung bao gồm môi trường phát triển, cấu trúc mã nguồn, các module chức năng chính và lưu đồ thuật toán điều khiển toàn hệ thống.

4.2.3.1. Môi trường phát triển và công cụ

a) Framework và toolchain Firmware được phát triển trên nền tảng ESP-IDF phiên bản 5.4, tức bộ công cụ chính thức của Espressif dành cho ESP32-S3. Có thể xem ESP-IDF như “bộ nền” để viết firmware: nó cung cấp FreeRTOS để điều phối tác vụ, các driver ngoại vi để làm việc với phần cứng, NimBLE để xử lý Bluetooth và hệ thống build để biên dịch toàn bộ dự án.



Hình 4.16: Checklist kiểm tra hệ thống sau khi lắp đặt trong xe

Bảng 4.5: Công cụ phát triển firmware

Thành phần	Công cụ / Phiên bản	Mục đích
Framework	ESP-IDF v5.4	SDK chính thức cho ESP32-S3
IDE	VS Code + ESP-IDF Extension	Biên tập, debug, flash firmware
Build system	CMake + Ninja	Biên dịch và liên kết mã nguồn
BLE stack	NimBLE (tích hợp ESP-IDF)	Giao tiếp Bluetooth Low Energy
RTOS	FreeRTOS (tích hợp ESP-IDF)	Quản lý tác vụ đa luồng
Debug	JTAG / UART Monitor	Giám sát log và debug thời gian thực
Version control	Git	Quản lý phiên bản mã nguồn

b) Quy trình build và nạp firmware Quy trình biên dịch firmware đi theo ba bước quen thuộc: cấu hình dự án, biên dịch toàn bộ mã nguồn, rồi nạp firmware xuống vi điều khiển. Trong ESP-IDF, bước cấu hình được thực hiện qua tiện ích cấu hình của bộ SDK; sau đó hệ thống build dựa trên CMake và Ninja sẽ tạo file firmware để nạp qua UART.

Cấu hình dự án (chọn target ESP32-S3)

```
idf.py set-target esp32s3
```

Cấu hình tham số (BLE, UART, GPIO, ...)

```
idf.py menuconfig
```

Biên dịch dự án

```
idf.py build
```

Nạp firmware và giám sát log qua cổng nối tiếp

```
idf.py -p SERIAL_PORT flash monitor
```

c) Cấu hình NimBLE BLE stack NimBLE được chọn thay cho Bluedroid mặc định của ESP-IDF vì nó gọn hơn về bộ nhớ nhưng vẫn đủ chức năng BLE Central để kết nối với adapter OBD2. Nói cách khác, đây là lựa chọn thực dụng: tiết kiệm RAM cho firmware mà không làm mất các khả năng BLE cần thiết của đề tài.

Cấu hình NimBLE được lưu trong tệp cấu hình dự án của ESP-IDF (`sdkconfig`), thường được thiết lập qua tiện ích cấu hình của bộ SDK:

```
CONFIG_BT_ENABLED=y
```

```
CONFIG_BT_NIMBLE_ENABLED=y
```

```

CONFIG_BT_NIMBLE_MAX_CONNECTIONS=1
CONFIG_BT_NIMBLE_ROLE_CENTRAL=y
CONFIG_BT_NIMBLE_ROLE_PERIPHERAL=n

```

4.2.3.2. Cấu trúc mã nguồn firmware

a) Tổ chức thư mục Cấu trúc mã nguồn firmware hiện tại bám sát mã nguồn ESP-IDF đang chạy trên thiết bị, tập trung vào các mô-đun theo chức năng thay vì chia thành nhiều task nghiệp vụ riêng biệt:

- `main/main.c`: điểm vào chính của firmware.
- `main/inc/`: khai báo module chức năng (`app_config`, `app_state`, `command_handler`, `data_formatter`, `mqtt_client`, `modem_*`, `ble_*`, `power_mgr`, `util`).
- `main/src/`: hiện thực nghiệp vụ lõi (`state_machine.c`, `command_handler.c`, `data_formatter.c`, `mqtt_client.c`, `nvs_config.c`, `power_mgr.c`, `modem_*`, `ble_*`, `adc_reader.c`, `imu_lis3dh.c`, `util.c`).
- `sdkconfig`, `CMakeLists.txt`: tệp cấu hình build và tham số runtime chính của dự án ESP-IDF.

b) Kiến trúc phân tầng Firmware được tổ chức quanh bốn nhóm chức năng chính, phản ánh đúng cách chia mô-đun trong mã nguồn hiện tại:

Bảng 4.6: Kiến trúc phân tầng firmware

Nhóm	Thành phần	Chức năng
1	<code>adc_reader</code> , <code>imu_lis3dh</code> , <code>modem_at</code> , <code>modem_lte</code> , <code>modem_gnss</code> , <code>ble_mgr</code> , <code>ble_obd</code>	Truy cập phần cứng và giao thức ngoại vi
2	<code>power_mgr</code> , <code>nvs_config</code> , <code>app_state</code>	Quản lý nguồn, ngưỡng LVD, cấu hình lưu bền và ngữ cảnh qua deep sleep
3	<code>state_machine</code> , <code>command_handler</code>	Điều phối trạng thái vận hành, lệnh điều khiển, OTA và rollback
4	<code>data_formatter</code> , <code>mqtt_client</code> , <code>util</code>	Đóng gói dữ liệu, giao tiếp MQTT, tải firmware OTA và kiểm tra SHA-256

Việc đồng bộ giữa các mô-đun được thực hiện có chọn lọc bằng mutex, semaphore và

queue ở cấp driver. Cách tiếp cận này giúp firmware vẫn tận dụng được FreeRTOS nhưng tránh phân mảnh logic sang quá nhiều task ứng dụng khó kiểm soát.

c) Định nghĩa chân GPIO

Bảng 4.7: Bảng ánh xạ chân GPIO

GPIO	Chức năng	Hướng	Mô tả
2	IGN_IN	Input	Đọc trạng thái khóa điện (hoặc qua OBD2)
4	U_BATT_ADC	Input	Đọc điện áp ắc quy (ADC 12-bit)
5	CHARGER_EN	Output	Điều khiển khối sạc TP5100
16	MODEM_UART_TX	Output	UART TX đến modem SIM7600CE-T
17	MODEM_UART_RX	Input	UART RX từ modem SIM7600CE-T
18	POWER_PATH_EN	Output	Chọn nguồn cấp (ắc quy/pin dự phòng)
19	LVD_STATUS	Input	Trạng thái từ khối LVD phần cứng
41	LIS3DH_INT	Input	Ngắt từ cảm biến gia tốc IMU
2	LIS3DH_SDA	I/O	I2C data line
1	LIS3DH_SCL	I/O	I2C clock line
26	MODEM_PWR-KEY	Output	Điều khiển nguồn modem

4.2.3.3. Triển khai module BLE-OBD2

a) Tổng quan giao tiếp BLE với adapter OBD2 Module BLE-OBD2 thiết lập kết nối Bluetooth Low Energy với adapter vgate iCar Pro tại cổng OBD2 để gửi yêu cầu đọc dữ liệu theo chuẩn OBD2 (Mode 0x01 - Current Data) và phân tích phản hồi, từ đó trích xuất các thông số vận hành động cơ.

Chu trình giao tiếp BLE-OBD2 gồm các giai đoạn: quét thiết bị (scan) — kết nối (connect) — khám phá dịch vụ GATT (service discovery) — gửi lệnh OBD2 (write characteristic) — nhận phản hồi (notification callback).

b) Khởi tạo NimBLE stack Việc khởi tạo BLE stack được thực hiện trong hàm `ble_init_stack()`. NimBLE chạy trong một FreeRTOS task riêng biệt, xử lý toàn bộ các sự kiện BLE thông qua vòng lặp sự kiện (event loop).

```
void ble_init_stack(ble_init_config_t const *config)
{
    // Khởi tạo cổng NimBLE
    esp_err_t ret = nimble_port_init();
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Khởi tạo NimBLE thất bại");
        return;
    }

    // Cấu hình callback cho host
    ble_hs_cfg.reset_cb      = config->reset_cb;
    ble_hs_cfg.sync_cb       = config->sync_cb;
    ble_hs_cfg.store_status_cb = ble_store_util_status_rr;
    ble_store_config_init();

    // Tạo task chạy NimBLE event loop
    xTaskCreate(ble_task, "NimBLE_task", 4096, NULL, 5, NULL);
}

static void ble_task(void *param)
{
    nimble_port_run(); // Blocking - chạy event loop BLE
}
```

c) Quy trình quét và kết nối thiết bị Firmware thực hiện quét BLE để tìm adapter OBD2 dựa trên tên thiết bị (chứa chuỗi "iCar") hoặc UUID dịch vụ. Khi phát hiện thiết bị phù hợp, hệ thống sẽ kết nối và khám phá dịch vụ GATT để xác định characteristic dùng cho truyền/nhận dữ liệu.

```
// Tham số quét BLE - chế độ thụ động (passive) để tiết kiệm năng lượng
static const struct ble_gap_disc_params disc_params = {
    .passive          = 1,
    .itvl             = 0x0010, // Khoảng cách quét
    .window           = 0x0010, // Cửa sổ quét
    .filter_duplicates = 1,      // Lọc quảng cáo trùng lặp
};
```

```

// Callback xử lý sự kiện GAP (scan, connect, disconnect)
static int ble_mgr_gap_event_cb(struct ble_gap_event *event, void *arg)
{
    switch (event->type) {
    case BLE_GAP_EVENT_DISC:
        // Phân tích dữ liệu quảng cáo
        ble_hs_adv_parse_fields(&adv_fields,
                               event->disc.data,
                               event->disc.length_data);
        // Kiểm tra UUID dịch vụ có trùng khớp không
        if (ble_mgr_adv_contains_service(&adv_fields,
                                         disc_cfg->svc_def->service_uuid)) {
            // Kết nối đến thiết bị
            ble_gap_connect(BLE_OWN_ADDR_PUBLIC,
                           &event->disc.addr,
                           30000,           // Timeout 30 giây
                           &conn_params,
                           ble_mgr_gap_event_cb, mgr_ctx);
        }
        break;

    case BLE_GAP_EVENT_CONNECT:
        if (event->connect.status == 0) {
            // Kết nối thành công - khám phá dịch vụ GATT
            ble_gattc_disc_all_svcs(conn_handle,
                                    ble_mgr_gatt_svc_discovered_cb,
                                    mgr_ctx);
        }
        break;
    }
    return 0;
}

```

Để tối ưu thời gian kết nối lại sau khi thức dậy từ deep sleep, firmware lưu địa chỉ MAC của adapter OBD2 vào bộ nhớ flash. Khi thức dậy, firmware ưu tiên kết nối trực tiếp đến địa chỉ đã lưu thay vì quét lại từ đầu, giảm thời gian kết nối lại xuống còn 1–3 giây so với 3–10 giây khi quét mới.

d) Gửi lệnh và phân tích phản hồi OBD2 Giao thức OBD2 sử dụng định dạng lệnh ELM327 truyền qua BLE GATT characteristic. Mỗi lệnh có dạng "MMPP\r", trong đó MM là mode (01 cho dữ liệu hiện tại) và PP là PID (Parameter ID) cần đọc.

```
// Gửi Lệnh OBD2 và đợi phản hồi
int ble_obd_rxtx(ble_obd_ctx_t *obd, uint8_t mode, uint8_t pid,
                 uint32_t timeout_ms)
{
    // Định dạng Lệnh: "MMPP\r"
    snprintf(obd->tx_data.buf, sizeof(obd->tx_data.buf),
             "%02X%02X\r", mode, pid);

    // Gửi qua GATT write characteristic
    ble_mgr_send(obd->mgr_ctx, obd_tx_char->handle,
                 obd->tx_data.buf, strlen(obd->tx_data.buf));

    // Đợi phản hồi (semaphore với timeout)
    if (xSemaphoreTake(obd->api.response_sem,
                      pdMS_TO_TICKS(timeout_ms)) != pdTRUE) {
        return -1; // Hết thời gian chờ
    }
    return 0;
}
```

Phản hồi OBD2 được xử lý trong notification callback. Dữ liệu trả về có dạng "41 0C 1F 40\r", trong đó byte đầu là mode + 0x40 (chỉ thị phản hồi), byte thứ hai là PID, và các byte còn lại là dữ liệu cần giải mã.

```
// Phân tích phản hồi OBD2
static void ble_obd_process_obd_data(ble_obd_ctx_t *obd,
                                     char *data, size_t len)
{
    char *saveptr;
    char *tok = strtok_r(data, " \r", &saveptr);
    uint8_t values[BLE_OBD_MAX_DATA_LEN];
    int count = 0;

    while (tok && count < BLE_OBD_MAX_DATA_LEN) {
        long val = strtol(tok, NULL, 16); // Chuyển hex sang số nguyên
        values[count++] = (uint8_t)val;
        tok = strtok_r(NULL, " \r", &saveptr);
    }
}
```



```

    }

    // Xác thực phản hồi: kiểm tra mode và PID trùng khớp
    if (values[0] == (obd->tx_data.mode + 0x40) &&
        values[1] == obd->tx_data.pid) {
        // Gọi callback với dữ liệu đã trích xuất
        obd->response_cb(obd->tx_data.pid,
                        values + 2,      // Bỏ qua mode và PID
                        count - 2,      // Độ dài dữ liệu thực
                        obd->usr_ctx);
    }
}

```

e) Bảng PID OBD2 sử dụng trong hệ thống

Bảng 4.8: Các PID OBD2 được đọc định kỳ (Mode 0x01 - Current Data)

PID (Hex)	Thông số	Độ dài	Công thức tính	Đơn vị
0x0C	Tốc độ động cơ (RPM)	2 byte	$\frac{A \times 256 + B}{4}$	vòng/phút
0x0D	Tốc độ xe	1 byte	A	km/h
0x04	Tải động cơ	1 byte	$\frac{A \times 100}{255}$	%
0x05	Nhiệt độ nước làm mát	1 byte	A - 40	độ C
0x2F	Mức nhiên liệu	1 byte	$\frac{A \times 100}{255}$	%
0x0F	Nhiệt độ khí nạp	1 byte	A - 40	độ C
0x11	Vị trí bướm ga	1 byte	$\frac{A \times 100}{255}$	%
0x42	Điện áp module điều khiển	2 byte	$\frac{A \times 256 + B}{1000}$	V

Các hàm chuyển đổi tương ứng được định nghĩa trong component obd2_parser:

```

// Chuyển đổi RPM: (A * 256 + B) / 4
static int obd_conv_rpm(int32_t *value, uint8_t const *data, size_t len)
{
    *value = ((data[0] << 8) | data[1]) / 4;
    return 0;
}

```

```
// Chuyển đổi phần trăm: (A * 100) / 255
static int obd_conv_percent(int32_t *value, uint8_t const *data, size_t len)
{
    *value = (data[0] * 100) / 255;
    return 0;
}

// Chuyển đổi nhiệt độ: A - 40
static int obd_conv_temperature(int32_t *value, uint8_t const *data, size_t len)
{
    *value = data[0] - 40;
    return 0;
}
```

f) Xử lý lỗi và fallback Khi thiết bị theo dõi không thể kết nối BLE với adapter OBD2 trong thời gian chờ cho phép, firmware không dừng lại mà chuyển sang cơ chế dự phòng. Lúc này, thiết bị đọc điện áp ắc quy qua ADC để ước lượng trạng thái khóa điện theo profile 12V hoặc 24V. Cách suy luận này không chính xác bằng việc đọc trực tiếp qua OBD2, nhưng vẫn đủ để hệ thống tiếp tục theo dõi và đưa ra quyết định cơ bản khi đường BLE gặp sự cố.

4.2.3.4. Triển khai module modem MQTT

a) Khởi tạo và kiểm tra modem Module modem quản lý toàn bộ giao tiếp với modem SIMCom SIM7600CE-T thông qua giao tiếp UART và tập lệnh AT. Quy trình khởi tạo bao gồm kiểm tra phản hồi modem, trạng thái SIM, đăng ký mạng và chất lượng tín hiệu.

```
// Trình tự khởi tạo modem SIM7600CE-T
int modem_init(modem_ctx_t *ctx)
{
    // Bước 1: Kiểm tra modem phản hồi
    if (modem_send_at(ctx, "AT", "OK", 5000) != 0) {
        // Modem không phản hồi - reset bằng GPIO PWR-KEY
        gpio_set_level(MODEM_PWR-KEY, 0);
        vTaskDelay(pdMS_TO_TICKS(1500));
        gpio_set_level(MODEM_PWR-KEY, 1);
        vTaskDelay(pdMS_TO_TICKS(10000)); // Đợi khởi động lại
        if (modem_send_at(ctx, "AT", "OK", 5000) != 0) {
```

```

        return -1; // Khởi tạo thất bại
    }
}

// Bước 2: Kiểm tra SIM card
if (modem_send_at(ctx, "AT+CPIN?", "+CPIN: READY", 5000) != 0) {
    return -2; // SIM chưa sẵn sàng
}

// Bước 3: Đợi đăng ký mạng (tối đa 60 giây)
for (int i = 0; i < 12; i++) {
    if (modem_send_at(ctx, "AT+CEREG?", "+CEREG: 0,1", 5000) == 0) {
        break; // Đã đăng ký mạng
    }
    vTaskDelay(pdMS_TO_TICKS(5000));
}

// Bước 4: Kiểm tra chất lượng tín hiệu
modem_send_at(ctx, "AT+CSQ", "+CSQ:", 3000);

// Bước 5: Thiết lập APN và kích hoạt PDP context
modem_send_at(ctx, "AT+CGDCONT=1,\"IP\", \"internet\"", "OK", 5000);
modem_send_at(ctx, "AT+CGACT=1,1", "OK", 10000);

return 0; // Khởi tạo thành công
}

```

b) Kết nối và gửi dữ liệu MQTT Trong triển khai hiện tại, modem SIM7600CE-T đảm nhiệm việc đăng ký mạng và kích hoạt PDP context bằng AT command; sau khi đường truyền dữ liệu sẵn sàng, firmware sử dụng `esp_mqtt_client` của ESP-IDF để mở phiên MQTT, gửi dữ liệu vận hành và đăng ký nhận topic lệnh điều khiển. Cách làm này giúp phân truyền MQTT tách biệt rõ khỏi phần điều khiển modem, dễ debug hơn và tận dụng trực tiếp event loop của ESP-IDF.

```

static void tracker_mqtt_build_topics(void) {
    snprintf(s_topic_rawdata, sizeof(s_topic_rawdata), "v1/%s/rawdata", s_cfg.dev);
    snprintf(s_topic_status, sizeof(s_topic_status), "v1/%s/status", s_cfg.device);
    snprintf(s_topic_events, sizeof(s_topic_events), "v1/%s/events", s_cfg.device);
    snprintf(s_topic_firmware, sizeof(s_topic_firmware), "v1/%s/firmware", s_cfg.device);
    snprintf(s_topic_commands, sizeof(s_topic_commands), "v1/%s/commands", s_cfg.device);
}

```

```
}
```

```
static void tracker_mqtt_event_handler(..., int32_t event_id, void *event_data) {  
    switch ((esp_mqtt_event_id_t)event_id) {  
        case MQTT_EVENT_CONNECTED:  
            s_connected = true;  
            if (s_cfg.command_subscribe_enabled) {  
                esp_mqtt_client_subscribe(s_client, s_topic_commands, 1);  
            }  
            break;  
        case MQTT_EVENT_DATA:  
            if (s_command_callback != NULL) {  
                s_command_callback(s_topic_commands, payload);  
            }  
            break;  
        default:  
            break;  
    }  
}
```

Trong đó, rawdata được publish với QoS 0 để tối ưu băng thông cho luồng dữ liệu vận hành gửi dày; còn status, events, firmware và commands dùng QoS 1 để tăng độ tin cậy cho heartbeat, cảnh báo, OTA và lệnh điều khiển.

Cấu trúc topic MQTT:

Bảng 4.8A: Bảng bổ sung chỉ mục (4.2.3. Triển khai Firmware)

Topic	Hướng	Mục đích
v1/{device_id}/rawdata	Device → Server	Dữ liệu vận hành chính: GNSS, nguồn, rung
v1/{device_id}/events	Device → Server	Sự kiện và cảnh báo quan trọng
v1/{device_id}/status	Device → Server	Heartbeat và trạng thái phiên
v1/{device_id}/firmware	Device → Server	Tiến trình OTA
v1/{device_id}/commands	Server → Device	Lệnh điều khiển từ xa

c) Đọc dữ liệu GNSS GNSS tích hợp trong SIM7600CE-T được điều khiển qua cổng UART. Firmware bật GNSS, đợi fix vị trí, parse bản tin NMEA/CGNSINF và đọc tọa độ định kỳ.

```

// Bật và đọc dữ liệu GNSS từ modem
int modem_gnss_read(modem_ctx_t *ctx, gnss_data_t *gnss)
{
    // Bật nguồn GNSS (nếu chưa bật)
    modem_send_at(ctx, "AT+CGNSPWR=1", "OK", 3000);

    // Đọc thông tin GNSS
    // Phản hồi: +CGNSINF: <mode>,<sat_gps>,<sat_glo>,<sat_bds>,
    //           <lat>,<N/S>,<lon>,<E/W>,<date>,<time>,<alt>,
    //           <speed>,<course>,<hdop>,<vdop>
    char response[256];
    if (modem_send_at_get_response(ctx, "AT+CGNSINF",
                                    response, sizeof(response),
                                    3000) == 0) {
        // Phân tích phản hồi
        gnss_parse_cgnsinf(response, gnss);
        return 0;
    }
    return -1; // Chưa có fix
}

// Cấu trúc dữ liệu GNSS
typedef struct {
    double    latitude;    // Vĩ độ (độ thập phân)
    double    longitude;   // Kinh độ (độ thập phân)
    float     altitude;    // Độ cao (mét)
    float     speed;       // Tốc độ (km/h)
    float     course;      // Hướng đi (độ)
    uint8_t   satellites;  // Số vệ tinh
    bool      fix_valid;   // Trạng thái fix hợp lệ
} gnss_data_t;

```

Thời gian fix GNSS phụ thuộc vào trạng thái trước đó của modem: hot start khoảng 1–3 giây khi module chỉ mất tín hiệu trong thời gian ngắn, warm start khoảng 5–15 giây khi còn dữ liệu ephemeris, và cold start khoảng 30–60 giây khi khởi tạo lại hoàn toàn.

d) Xử lý lệnh điều khiển từ xa Firmware lắng nghe lệnh từ máy chủ thông qua topic v1/{device_id}/commands. Gói lệnh được command_handler.c phân tích theo cấu trúc JSON, sau đó cập nhật cấu hình hoặc đặt cờ hành động để state_machine.c xử lý ở chu kỳ kế tiếp.

```

void command_handler_process(const char *command_json) {
    cJSON *root = cJSON_Parse(command_json);
    const cJSON *command = cJSON_GetObjectItemCaseSensitive(root, "command");
    const cJSON *params = cJSON_GetObjectItemCaseSensitive(root, "params");

    if (strcmp(command->valuestring, "update_config") == 0) {
        command_apply_update_config(params);
    } else if (strcmp(command->valuestring, "request_location") == 0) {
        s_location_requested = true;
        s_pending_action = COMMAND_ACTION_REQUEST_LOCATION;
    } else if (strcmp(command->valuestring, "enable_tracking") == 0) {
        const cJSON *enabled = cJSON_GetObjectItemCaseSensitive(params, "enabled");
        if (cJSON_IsBool(enabled)) {
            s_tracking_enabled = cJSON_IsTrue(enabled);
        }
    } else if (strcmp(command->valuestring, "reboot") == 0) {
        s_pending_action = COMMAND_ACTION_REBOOT;
    } else if (strcmp(command->valuestring, "ota_update") == 0) {
        s_pending_action = COMMAND_ACTION_OTA_UPDATE;
    } else if (strcmp(command->valuestring, "manual_rollback") == 0 ||
               strcmp(command->valuestring, "ota_rollback") == 0) {
        s_pending_action = COMMAND_ACTION_OTA_ROLLBACK;
    }
}

```

e) Luồng cập nhật firmware OTA Tính năng OTA mới được tích hợp xuyên suốt từ API phía máy chủ đến firmware. API phía máy chủ tạo jobId, tra cứu firmware theo firmwareVersion, sinh URL tải file dạng /api/v1/firmware/{id}/download, rồi gửi lệnh ota_update qua topic v1/{device_id}/commands. Firmware kiểm tra đầy đủ các tham số jobId, version, url, size, sha256, force và confirmTimeoutSec trước khi bắt đầu cập nhật.

```

state_machine_publish_firmware_status("assigned", 0, cmd.version, cmd.job_id, "",

if (util_ota_apply_update(&s_config, s_current_version, &cmd, &report) == ESP_OK)
    util_copy_string(g_rtc_context.ota_job_id, sizeof(g_rtc_context.ota_job_id),
    util_copy_string(g_rtc_context.ota_target_version, sizeof(g_rtc_context.ota_t
    util_copy_string(g_rtc_context.ota_previous_version, sizeof(g_rtc_context.ota
    g_rtc_context.ota_pending_confirm = true;
    g_rtc_context.ota_confirm_timeout_sec = cmd.confirm_timeout_sec;

```

```

    state_machine_publish_firmware_payload(&report);
    esp_restart();
}

```

Hàm `util_ota_apply_update()` sử dụng HTTPS để tải file firmware, ghi trực tiếp xuống OTA partition, tính và đối chiếu SHA-256, sau đó đặt phân vùng boot mới bằng `esp_ota_set_boot_partition()`. Sau lần khởi động kế tiếp, firmware phát trạng thái `confirming`, gọi `esp_ota_mark_app_valid_cancel_rollback()` để xác nhận image mới, rồi mới phát `success`. Nếu có lỗi trong quá trình xác nhận hoặc người vận hành gửi lệnh `manual_rollback`, hệ thống sẽ quay về phân vùng trước đó và phát trạng thái `failed` hoặc `rolled_back` tương ứng.

4.2.3.5. Triển khai quản lý nguồn và chế độ ngủ»§

a) Đọc điện áp và điều khiển nguồn Module quản lý nguồn thực hiện đọc điện áp ắc quy qua ADC, điều khiển nhánh nguồn theo kiến trúc diode OR + EN, và điều khiển khối sạc TP5100.

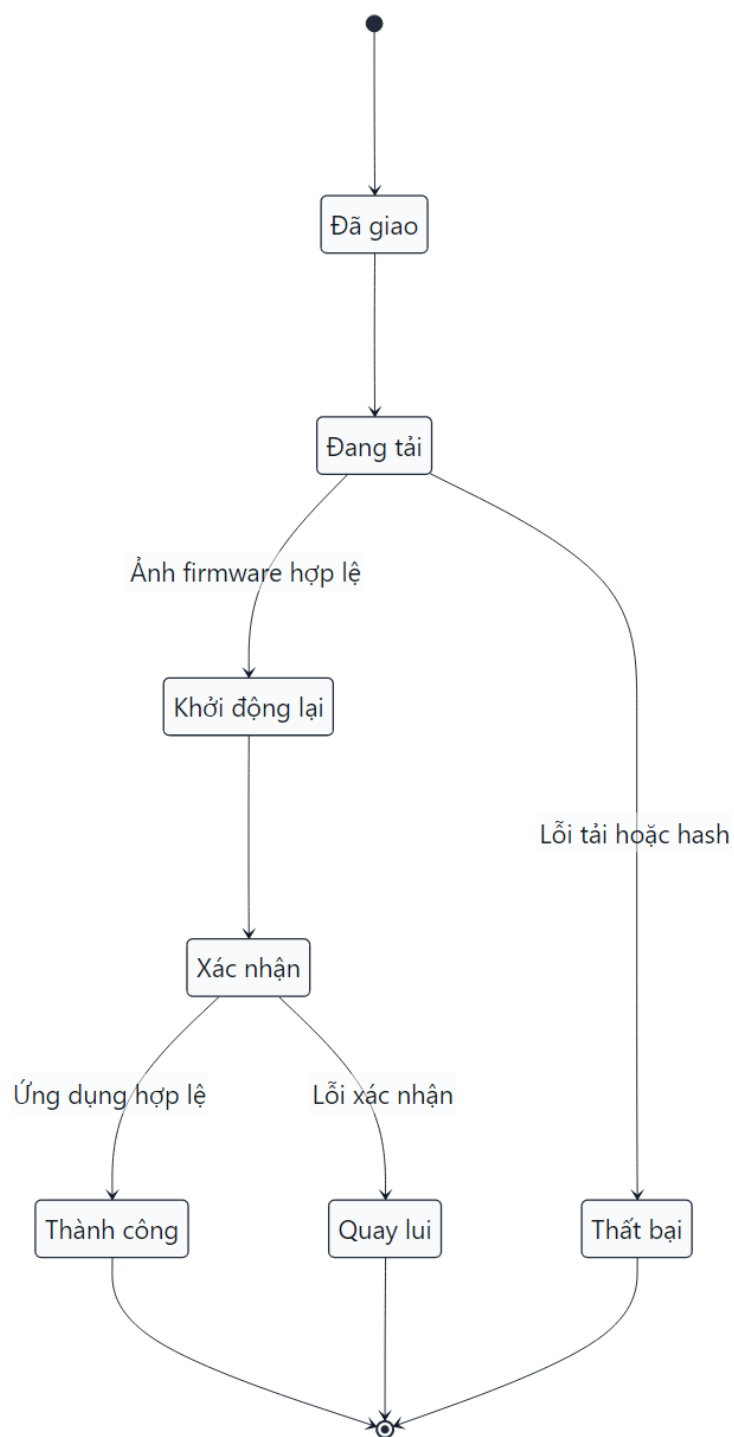
```

// Đọc điện áp ắc quy qua ADC (12-bit, chia áp)
float read_battery_voltage(void)
{
    int adc_raw = adc1_get_raw(ADC1_CHANNEL_4); // GPIO4
    float voltage = (adc_raw / 4095.0f) * 3.3f; // Chuyển sang volt
    voltage *= VOLTAGE_DIVIDER_RATIO; // Bù tỷ lệ chia áp
    return voltage;
}

// Logic quản lý nguồn chính
void power_management_task(void *param)
{
    while (true) {
        float u_batt = read_battery_voltage();
        bool ign_on = get_ignition_status();
        bool lvd_ok = gpio_get_level(LVD_STATUS);
        float switch_off = cfg.power_profile_24v ? 24.0f : 12.0f;
        float switch_on = cfg.power_profile_24v ? 24.4f : 12.2f;

        if (ign_on) {

```



Hình 4.16a: Chuỗi bước OTA trên firmware từ nhận lệnh đến xác nhận sau reboot


```

        // IGN ON: dùng nguồn ắc quy, bật sạc pin dự phòng
        gpio_set_level(POWER_PATH_EN, 0);    // Chọn ắc quy
        gpio_set_level(CHARGER_EN, 1);        // Bật sạc
    } else if (u_batt <= switch_off) {
        // Điện áp thấp: chuyển sang pin dự phòng theo profile
        gpio_set_level(POWER_PATH_EN, 1);    // Chọn pin dự phòng
        gpio_set_level(CHARGER_EN, 0);        // Tắt sạc
        send_low_battery_alert(u_batt);
    } else if (u_batt >= switch_on) {
        // Điện áp phục hồi: quay lại ắc quy theo profile
        gpio_set_level(POWER_PATH_EN, 0);    // Chọn ắc quy
        gpio_set_level(CHARGER_EN, 0);        // Tắt sạc (IGN OFF)
    }
    // Giữ nguyên trạng thái nếu trong vùng trễ của profile (12V: 12.0-12.2V;

    vTaskDelay(pdMS_TO_TICKS(5000)); // Kiểm tra mỗi 5 giây
}
}

```

Ngưỡng điện áp sử dụng cơ chế trễ (hysteresis) theo profile cấu hình để tránh hiện tượng chuyển đổi liên tục khi điện áp dao động quanh ngưỡng: profile 12V dùng 12.0 V (OFF) và 12.2 V (ON), profile 24V dùng 24.0 V (OFF) và 24.4 V (ON). Ngưỡng suy luận trạng thái động cơ cũng được tách profile: $IGN_{ON} \geq 13.0\text{ V}$ và $IGN_{OFF} \leq 12.0\text{ V}$ (12V), hoặc $IGN_{ON} \geq 26.0\text{ V}$ và $IGN_{OFF} \leq 24.0\text{ V}$ (24V).

b) Chế độ ngủ và đánh thức Firmware sử dụng hai chế độ ngủ của ESP32-S3 tùy theo tình huống:

Bảng 4.9: So sánh các chế độ ngủ

Chế độ	Dòng tiêu thụ	Thời gian thức dậy	BLE	Điều kiện sử dụng
Light sleep	0.8 mA	< 1 ms	Giữ active	IGN ON, đợi dữ liệu giữa các chu kỳ
Deep sleep	10 μ A	100 ms	Mất kết nối	IGN OFF, chế độ đỗ xe

Trước khi vào trạng thái ngủ đỗ xe, firmware thực hiện trình tự tắt các ngoại vi để tiết kiệm năng lượng tối đa:

```

// Trình tự trước khi vào parked sleep

```

```

void enter_parked_sleep(uint32_t sleep_duration_sec)
{
    // 1. Ngắt kết nối BLE (nếu đang kết nối)
    ble_obd_disconnect();

    // 2. Tắt GNSS
    modem_send_at(&modem_ctx, "AT+CGNSPWR=0", "OK", 3000);

    // 3. Tắt 4G và đưa modem vào chế độ ngủ
    modem_send_at(&modem_ctx, "AT+CGACT=0,1", "OK", 5000);
    modem_send_at(&modem_ctx, "AT+CSCLK=1", "OK", 3000);

    // 4. Tắt sạc pin
    gpio_set_level(CHARGER_EN, 0);

    // 5. Lưu trạng thái hiện tại vào RTC memory
    rtc_data.last_mode      = current_mode;
    rtc_data.last_ign_status = ign_on;
    rtc_data.sleep_count++;

    // 6. Cấu hình nguồn đánh thức
    //    - Timer: heartbeat định kỳ
    esp_sleep_enable_timer_wakeup(sleep_duration_sec * 1000000ULL);
    //    - IMU interrupt trên GPIO41: dùng GPIO wake ở light sleep
    gpio_wakeup_enable(LIS3DH_INT, GPIO_INTR_HIGH_LEVEL);
    esp_sleep_enable_gpio_wakeup();
    // 7. Vào light sleep
    esp_light_sleep_start();
}

```

Khi thức dậy từ parked sleep, firmware đọc nguyên nhân đánh thức và trạng thái đã lưu trong RTC memory để quyết định chế độ hoạt động tiếp theo mà không cần khởi tạo lại toàn bộ.

c) Quản lý nguồn modem theo chế độ Modem SIM7600CE-T hỗ trợ nhiều chế độ ngủ với mức tiêu thụ khác nhau:

- **UART sleep** (AT+CSCLK=1): Modem tự động ngủ khi không có dữ liệu UART, tiêu thụ 1–5 mA, đánh thức bằng bất kỳ ký tự UART nào.
- **Minimum functionality** (AT+CFUN=0): Tắt RF, giữ UART, tiêu thụ < 1 mA, đánh

thức bằng lệnh AT hoặc GPIO.

- **Flight mode** (AT+CFUN=4): Tắt RF nhưng giữ chức năng khác, phù hợp cho heart-beat ngắn.

Thời gian đánh thức modem phụ thuộc chế độ: từ sleep là 100–500 ms, từ chế độ tối thiểu là 1–3 giây, và kết nối lại 4G mất thêm 5–15 giây.

4.2.3.6. Lưu đồ thuật toán chính

a) Máy trạng thái Toàn bộ logic điều khiển firmware được tổ chức theo mô hình máy trạng thái, tức bộ điều phối trạng thái hữu hạn, gồm bảy trạng thái chính:

Bảng 4.10: Các trạng thái của firmware

Trạng thái	Mã	Mô tả
INIT	STATE_INIT	Khởi tạo hệ thống, ngoại vi
CHECK_IGN	STATE_CHECK_IGN	Kiểm tra trạng thái khóa điện
DRIVING	STATE_DRIVING	Chế độ lái xe - theo dõi liên tục
PARKED	STATE_PARKED	Chế độ đỗ xe - heartbeat định kỳ
ALARM	STATE_ALARM	Chế độ cảnh báo - phát hiện chuyển động
HEARTBEAT	STATE_HEARTBEAT	Gửi tín hiệu heartbeat
SLEEP	STATE_SLEEP	Deep sleep tiết kiệm năng lượng

Bảng 4.11: Chuyển đổi trạng thái

Trạng thái hiện tại	Điều kiện	Trạng thái mới
INIT	Khởi tạo xong	CHECK_IGN
CHECK_IGN	IGN = ON	DRIVING
CHECK_IGN	IGN = OFF	PARKED
DRIVING	IGN = OFF (phát hiện)	PARKED
PARKED	IMU interrupt (chuyển động)	ALARM
PARKED	Timer wake-up	HEARTBEAT
ALARM	Chuyển động dừng, IGN = OFF	PARKED
HEARTBEAT	Gửi xong heartbeat	SLEEP
SLEEP	Timer hoặc IMU wake-up	CHECK_IGN

```

typedef enum {
    APP_STATE_INIT = 0,
    APP_STATE_CHECK_IGN,
    APP_STATE_DRIVING,
    APP_STATE_PARKED,
    APP_STATE_ALARM,
    APP_STATE_HEARTBEAT,
    APP_STATE_SLEEP,
} app_state_t;

void app_main(void) {
    config_t config = {0};
    ESP_ERROR_CHECK(nvs_config_load(&config));
    ESP_ERROR_CHECK(state_machine_init(&config));

    app_state_t state = APP_STATE_INIT;
    while (true) {
        state = state_machine_run(state);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

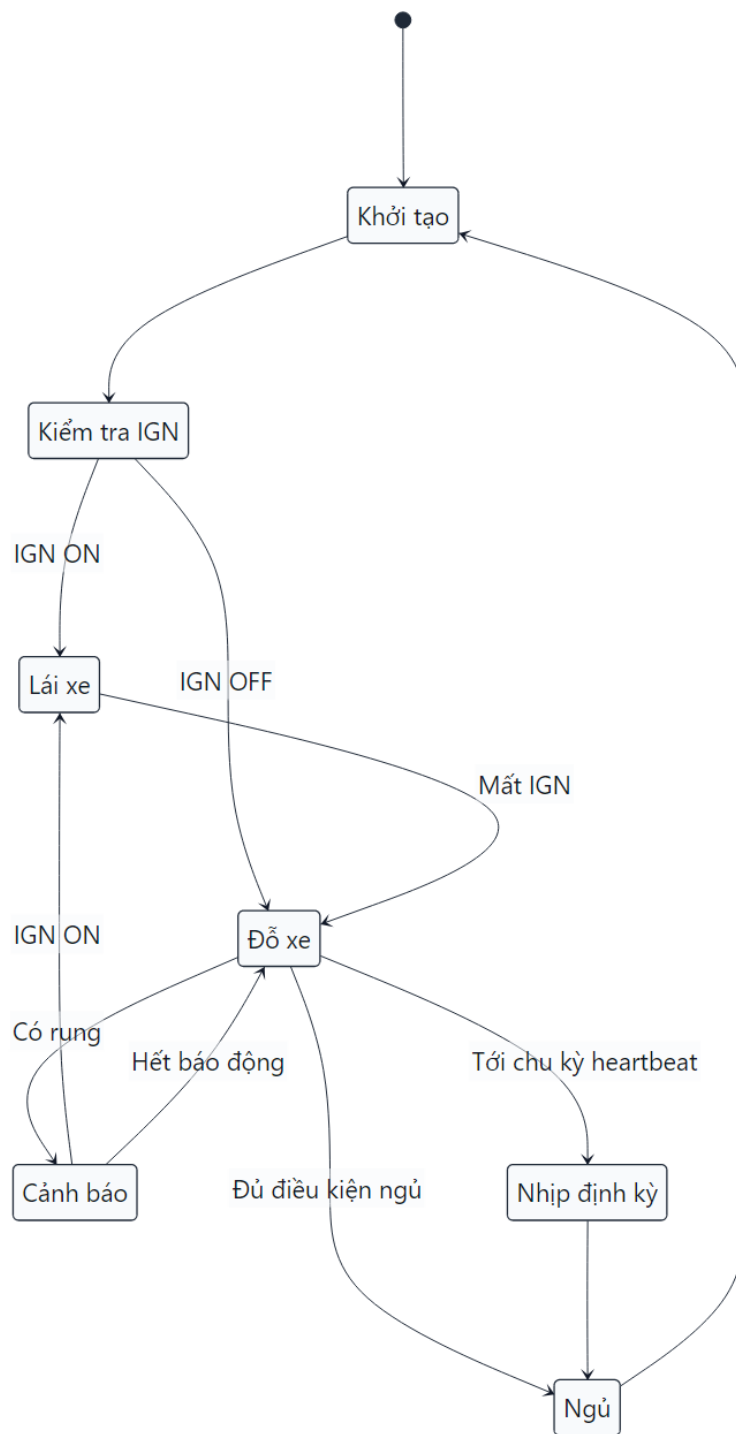
b) Lưu đồ thuật toán tổng thể Lưu đồ được tổng hợp từ luồng xử lý trong mã firmware triển khai thực tế và đối chiếu tài liệu kỹ thuật ESP32-S3 [7], vì vậy nó phản ánh cách thiết bị vận hành ngoài thực tế chứ không chỉ là sơ đồ ý tưởng.

c) Định dạng dữ liệu vận hành Gói dữ liệu vận hành của thiết bị được tổ chức theo kiểu “vỏ ngoài gọn, ruột rõ ràng”: thông tin nhận dạng và thời điểm gửi nằm ở lớp ngoài, còn các giá trị đo thực tế nằm trong trường data. Cách tổ chức này giúp phía máy chủ nhìn nhanh được bản tin đến từ ai, gửi lúc nào và cần đọc nhóm dữ liệu nào, từ đó MQTT Bridge kiểm tra và phân luồng dữ liệu thuận tiện hơn.

```

{
    "device_id": "TRACKER_001",
    "auth_token": "device_secret_token",
    "timestamp": 1710000000000,
    "uptime": 452130,
    "data": {
        "vibration": 128,
        "battery_top": 12.54,

```



Hình 4.17: Lưu đồ thuật toán chính của firmware thiết bị theo dõi xe

```
"battery_bot": 3.96,  
"latitude": 10.77653,  
"longitude": 106.70098,  
"speed": 42.5,  
"course": 181.2,  
"satellites": 11,  
"ignition": true,  
"error_code": 0  
}  
}
```

Dữ liệu được gửi định kỳ theo cấu hình vận hành hiện tại: nhịp nhanh hơn ở chế độ lái xe, thưa hơn ở các nhịp heartbeat khi đỗ. Tần suất gửi có thể được điều chỉnh từ xa thông qua lệnh `update_config` từ máy chủ.

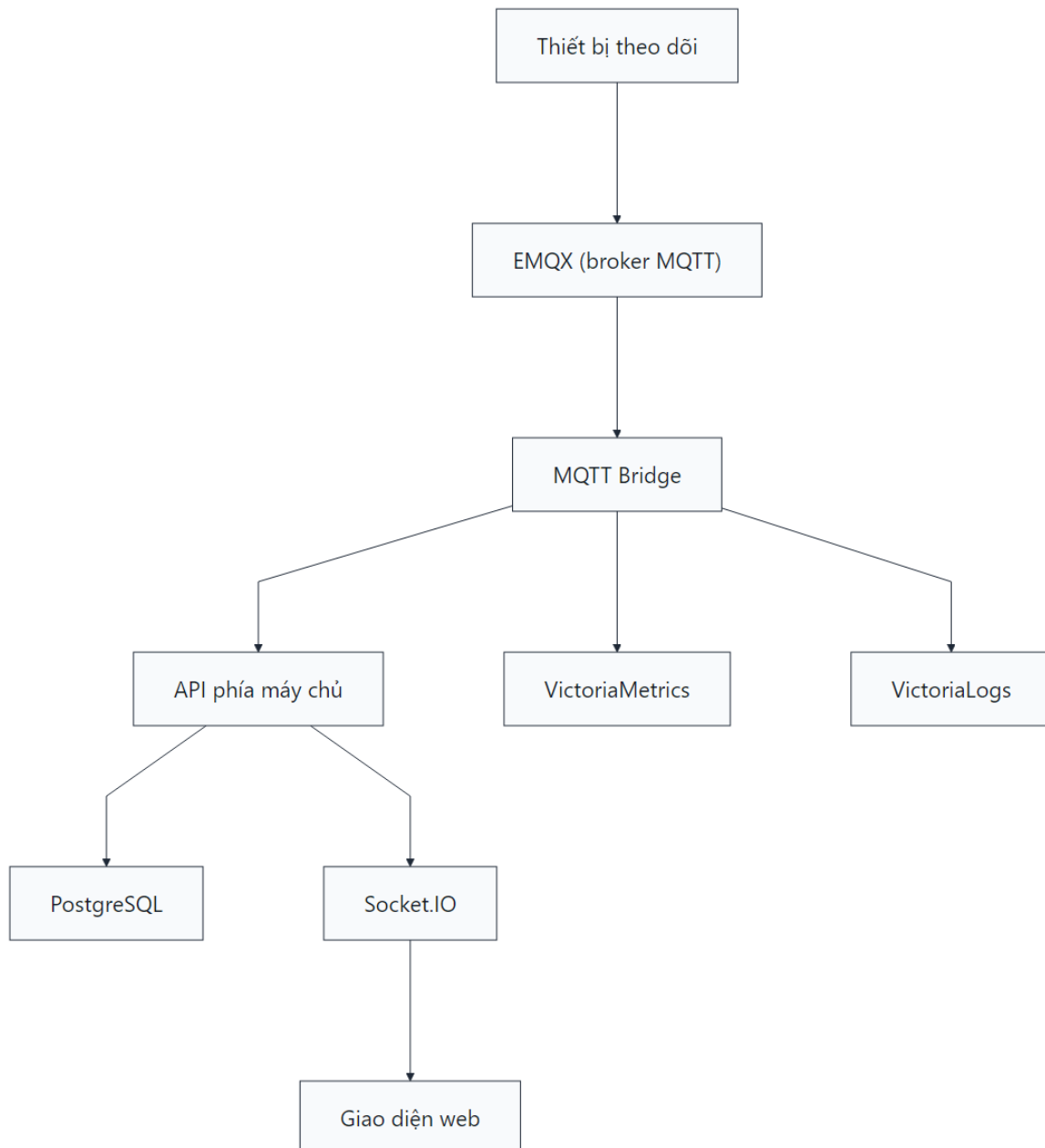
4.2.4. Triển khai hệ thống máy chủ

Hệ thống máy chủ là nơi biên dữ liệu từ thiết bị thành khả năng quan sát và điều hành ở phía người quản lý. Trên cơ sở phương án đã chọn ở Chương 3, phần này trình bày quá trình hiện thực hóa tầng máy chủ theo từng khối: hạ tầng Docker, MQTT Bridge, API quản trị phía máy chủ, giao diện quản trị và lớp giám sát vận hành.

Nếu nhìn theo hành trình dữ liệu, triển khai thực tế có thể tóm gọn như sau: thiết bị gửi bản tin lên lớp tiếp nhận; lớp này kiểm tra và đẩy dữ liệu sang từng nơi lưu phù hợp; sau đó API phía máy chủ và giao diện web dùng phần dữ liệu đã được chuẩn hóa để truy vấn, điều khiển và hiển thị thời gian thực. Nói cụ thể hơn, chuỗi đó lần lượt đi qua EMQX, MQTT Bridge, VictoriaMetrics, VictoriaLogs, PostgreSQL, API phía máy chủ và giao diện quản trị. Cách mô tả này giúp người đọc nối được kiến trúc ở Chương 3 với các cấu hình triển khai cụ thể ở phần sau.

4.2.4.1. Cấu hình Docker và hạ tầng đóng gói dịch vụ

a) Mô hình triển khai tách theo từng dịch vụ Ở giai đoạn triển khai, nhóm chọn cách tách mỗi dịch vụ thành một đơn vị cấu hình Docker Compose riêng. Cách tổ chức này phù hợp với đồ án vì vẫn đủ đơn giản để cài đặt, nhưng đã phản ánh khá rõ tư duy chia dịch vụ trong một hệ thống thật. Theo đó, mỗi dịch vụ có tệp cấu hình riêng thay vì gom toàn bộ vào một tệp lớn. Có thể hiểu ngắn gọn: mỗi dịch vụ chạy trong một container, tức một môi trường thực thi tách biệt, nên dễ quản lý hơn khi cần sửa một phần mà không



Hình 4.18: Kiến trúc tổng thể hệ thống máy chủ và luồng dữ liệu

muốn làm ảnh hưởng cả hệ thống. Cách tổ chức này có một số lợi điểm rõ ràng:

- **Độc lập triển khai:** Mỗi dịch vụ có thể được khởi động, dừng lại, hoặc cập nhật riêng rẽ mà không ảnh hưởng đến các dịch vụ khác. Điều này đặc biệt hữu ích trong quá trình phát triển và giai đoạn dò lỗi.
- **Quản lý tài nguyên riêng biệt:** Mỗi dịch vụ được cấu hình giới hạn tài nguyên (CPU, memory) phù hợp với nhu cầu thực tế, tránh tình trạng một dịch vụ chiếm dụng tài nguyên của dịch vụ khác.
- **Dễ dàng mở rộng:** Có thể tăng thêm bản sao từng dịch vụ độc lập dựa trên nhu cầu tải thực tế.

- **Đơn giản hóa quy trình kiểm tra và triển khai tự động:** Mỗi dịch vụ có thể có quy trình kiểm tra, đóng gói và triển khai tự động riêng.

Cấu trúc thư mục tổng thể của hệ thống được tổ chức như sau:

- Tracking_Backend/: API phía máy chủ viết bằng Express + TypeScript.
- Tracking_Frontend/: giao diện quản trị web bằng Next.js.
- Tracking_MqttBridge/: dịch vụ nhận luồng MQTT đầu vào một cách độc lập.
- Tracking_PostgreSQL/: PostgreSQL và script khởi tạo.
- Tracking_EMQX/: MQTT broker EMQX và cấu hình.
- Tracking_VictoriaMetrics/, Tracking_VictoriaLogs/, Tracking_Grafana/: khối quan sát vận hành gồm số liệu, nhật ký và bảng theo dõi.
- Tracking_NPM/: cổng chuyển tiếp truy cập từ bên ngoài vào các dịch vụ nội bộ, đồng thời quản lý chứng chỉ số.
- Tracking_Data/: dữ liệu vận hành thực tế (không đưa vào Git), gồm postgres-data/, emqx-data/, victoria-metrics-data/, victoria-logs-data/, grafana-data/.

b) Mạng chia sẻ (Shared Network) Một chi tiết tưởng nhỏ nhưng ảnh hưởng lớn đến tính ổn định khi triển khai là cách các dịch vụ “nhìn thấy nhau” trên Docker. Toàn bộ dịch vụ được đưa vào một mạng Docker dùng chung; nhờ vậy, dù nằm ở các file compose khác nhau, các container vẫn có thể giao tiếp qua tên dịch vụ mà không cần cấu hình thủ công phức tạp.

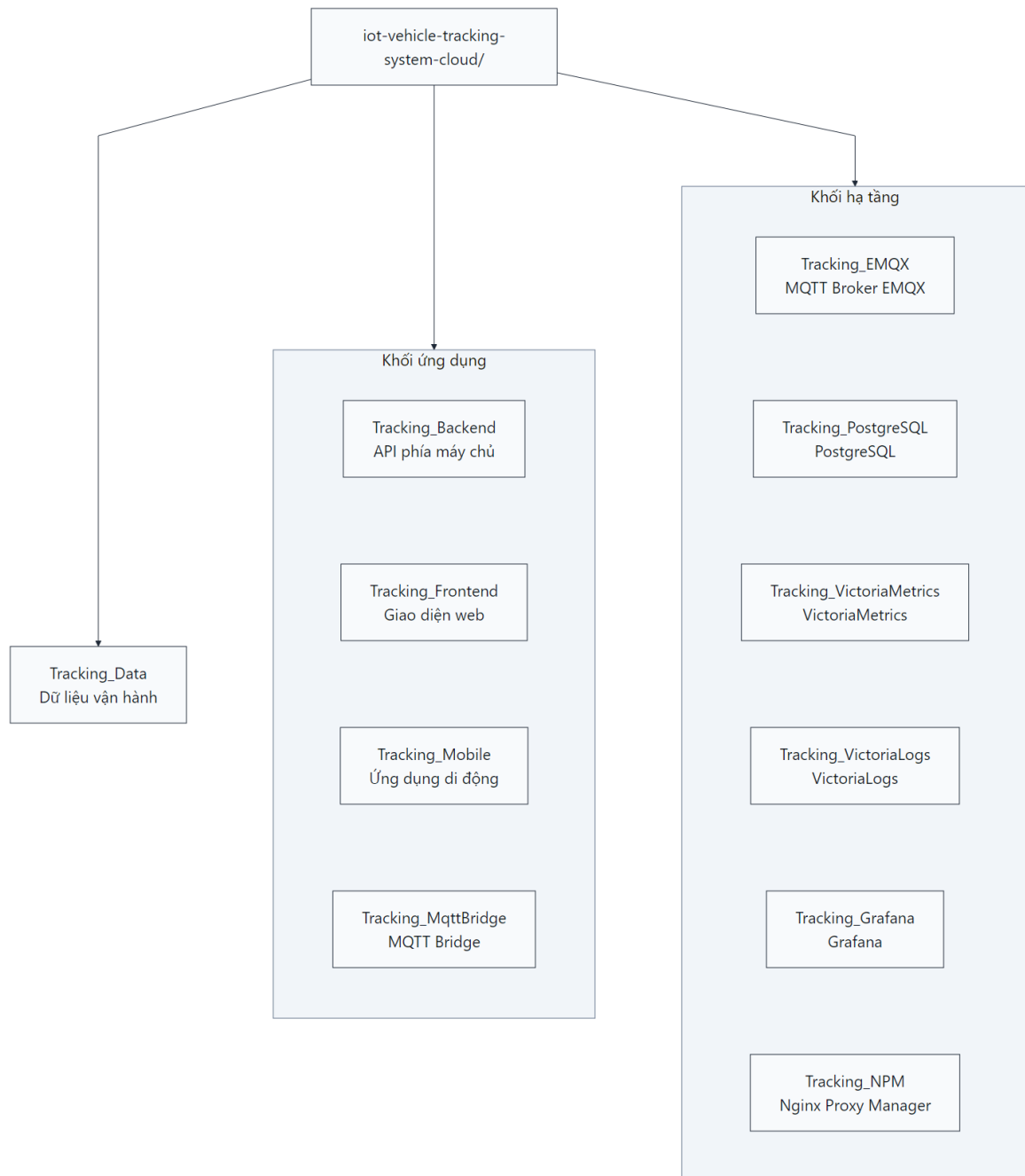
Tạo mạng chia sẻ (chỉ chạy một lần đầu tiên)

```
docker network create tracking-network
```

c) Cấu hình Docker Compose cho các dịch vụ hạ tầng Mỗi dịch vụ hạ tầng (infrastructure service) được cấu hình trong một file docker-compose.yml riêng. Dưới đây là ví dụ cấu hình cho dịch vụ PostgreSQL và EMQX:

PostgreSQL (Tracking_PostgreSQL/docker-compose.yml):

```
version: "3.8"
services:
  postgres:
    image: postgres:16-alpine
    container_name: tracking-postgres
    ports:
      - "5432:5432"
    environment:
```

Hình 4.19: Cấu trúc thư mục hệ thống theo mô hình triển khai

```

    POSTGRES_DB: vehicle_tracking
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ${POSTGRESQL_PASSWORD}
volumes:
  - ../Tracking_Data/postgres-data:/var/lib/postgresql/data
  - ./init:/docker-entrypoint-initdb.d
restart: unless-stopped
networks:
  - tracking-network
deploy:
  resources:
    limits:
      memory: 512M
      cpus: "1.0"

networks:
  tracking-network:
    external: true

```

EMQX MQTT Broker (Tracking_EMQX/docker-compose.yml):

```

version: "3.8"
services:
  emqx:
    image: emqx/emqx:5.4.0
    container_name: tracking-emqx
    ports:
      - "1883:1883" # MQTT
      - "8883:8883" # MQTT TLS
      - "8083:8083" # MQTT qua WebSocket
      - "8084:8084" # MQTT qua WebSocket Secure
      - "18083:18083" # EMQX Dashboard
    volumes:
      - emqx-data:/opt/emqx/data
      - emqx-log:/opt/emqx/log
    environment:
      EMQX_DASHBOARD__DEFAULT_PASSWORD: ${EMQX_DASHBOARD__DEFAULT_PASSWORD}
      EMQX_NODE__COOKIE: ${EMQX_NODE__COOKIE}
    restart: unless-stopped
    networks:

```

```

- tracking-network
deploy:
  resources:
    limits:
      memory: 1G
      cpus: "2.0"

volumes:
  emqx-data:
  emqx-log:

networks:
  tracking-network:
    external: true

```

Cấu hình tương tự được áp dụng cho VictoriaMetrics (port 8428), VictoriaLogs (port 9428), và Grafana (port 4002). Các dịch vụ đều chia sẻ cùng mạng Docker dùng chung; riêng các thành phần lưu trữ như PostgreSQL, VictoriaMetrics, VictoriaLogs và Grafana còn gắn volume trong Tracking_Data/ để bảo đảm dữ liệu vẫn còn sau khi container dừng rồi chạy lại.

d) Thư mục dữ liệu và quản lý volume Thư mục Tracking_Data/ được sử dụng làm nơi lưu trữ dữ liệu vận hành cho tất cả các dịch vụ. Thư mục này được loại khỏi phạm vi theo dõi của Git để tránh đưa dữ liệu chạy thật vào kho mã nguồn. Cấu trúc này đảm bảo:

- Dữ liệu không bị mất khi container bị xóa và tạo lại
- Dễ dàng sao lưu toàn bộ dữ liệu hệ thống bằng cách sao chép thư mục Tracking_Data/
- Tách biệt giữa mã nguồn và dữ liệu vận hành

e) Quy trình khởi động hệ thống Hệ thống được khởi động theo thứ tự phụ thuộc: dịch vụ hạ tầng trước, sau đó đến các dịch vụ ứng dụng:

```

# Bước 1: Khởi động dịch vụ hạ tầng
cd Tracking_PostgreSQL && docker compose up -d
cd Tracking_EMQX && docker compose up -d
cd Tracking_VictoriaMetrics && docker compose up -d
cd Tracking_VictoriaLogs && docker compose up -d
cd Tracking_Grafana && docker compose up -d

```

Bước 2: Khởi động dịch vụ ứng dụng

```
cd Tracking_MqttBridge && docker compose up -d --build
```

```
cd Tracking_Backend && docker compose up -d --build
```

```
cd Tracking_Frontend && docker compose up -d --build
```

Bảng 4.5: Danh sách các dịch vụ được đóng gói bằng Docker và cổng truy cập

Dịch vụ	Tên container	Cổng	Mô tả
PostgreSQL	tracking-postgres	5432	Cơ sở dữ liệu quan hệ
EMQX	tracking-emqx	1883, 8883, 8083, 8084, 18083	Trạm trung chuyển MQTT và giao diện quản lý
VictoriaMetrics	tracking-victoriametrics	8428	Dữ liệu chuỗi thời gian
VictoriaLogs	tracking-victorialogs	9428	Nhật ký tập trung
Grafana	tracking-grafana	4002	Màn hình giám sát
MQTT Bridge	tracking-mqtt-bridge	—	Dịch vụ trung gian xử lý bản tin MQTT (không mở cổng ra ngoài)
API phía máy chủ	tracking-backend	4000	REST API và kênh WebSocket
Giao diện web	tracking-frontend	4001	Giao diện web
Nginx Proxy Manager	tracking-npm	80, 81, 443	Cổng chuyển tiếp truy cập từ Internet vào các dịch vụ nội bộ

Ảnh xạ truy cập môi trường thử nghiệm:

Bảng 4.5-cloud: Ảnh xạ tên miền công khai, cổng truy cập và dịch vụ nội bộ trong môi trường thử nghiệm

Tên miền công khai	Cổng công khai	Dịch vụ nội bộ	Cổng nội bộ	Vai trò
thingdock.de v	443	tracking-frontend	4001	Giao diện web đi qua lớp chuyển tiếp Nginx Proxy Manager

Bảng 4.5-cloud: Ánh xạ tên miền công khai, cổng truy cập và dịch vụ nội bộ trong môi trường thử nghiệm (tiếp theo)

Tên miền công khai	Cổng công khai	Dịch vụ nội bộ	Cổng nội bộ	Vai trò
api.thingdock.k.dev	443	tracking-backend	4000	API phía máy chủ, Swagger và Socket.IO đi qua lớp chuyển tiếp Nginx Proxy Manager
mqtt.thingdock.k.dev	443	tracking-emqx	8083	MQTT qua WebSocket đi qua lớp chuyển tiếp Nginx Proxy Manager
mqtts.thingdock.k.dev	8883	tracking-emqx	8883	MQTT TLS trực tiếp cho thiết bị
grafana.thingdock.k.dev	443	tracking-grafana	4002	Grafana đi qua lớp chuyển tiếp Nginx Proxy Manager
emqx.thingdock.k.dev	443	tracking-emqx	18083	Giao diện quản lý EMQX đi qua lớp chuyển tiếp Nginx Proxy Manager
npm.thingdock.k.dev	443	tracking-npm	81	Trang quản trị NPM
(nội bộ)	8428	tracking-victoriametrics	8428	VictoriaMetrics, chỉ dùng nội bộ

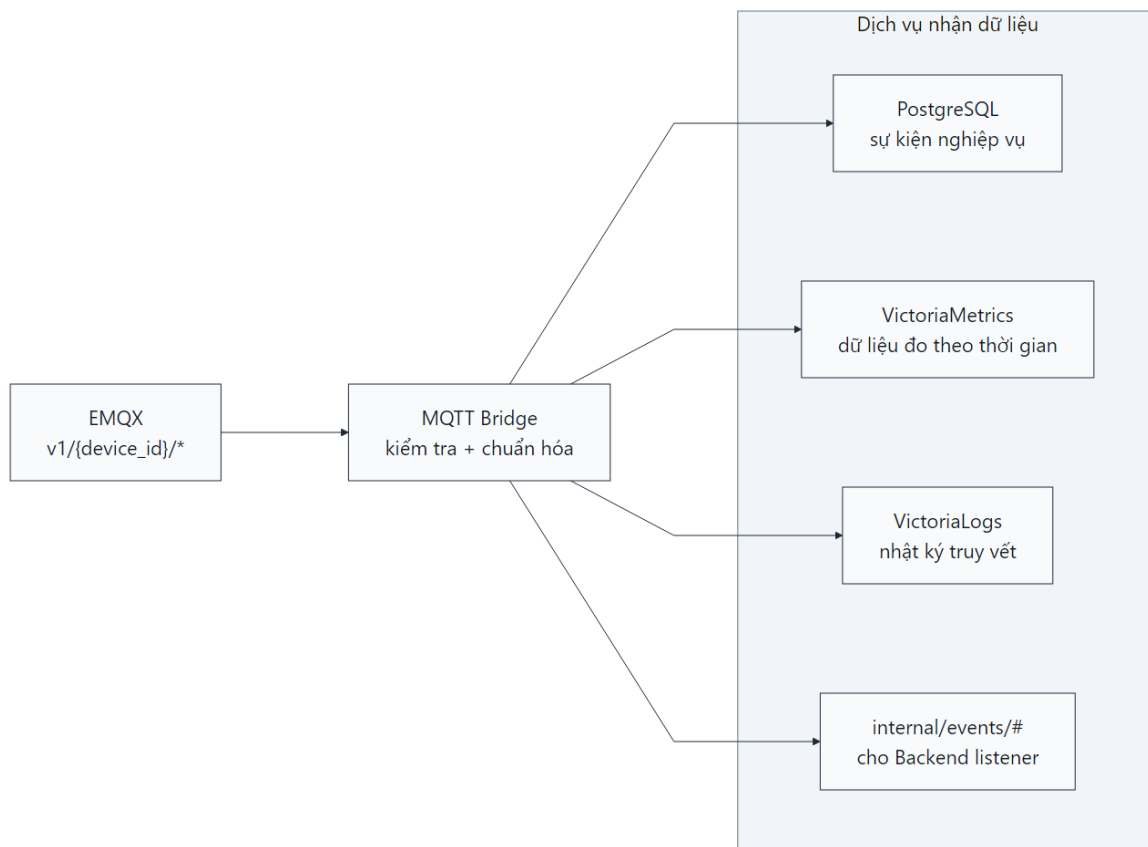
Ghi chú: Trong triển khai môi trường thử nghiệm hiện tại, Cloudflare chỉ giữ vai trò lớp phân giải tên miền, tức giúp tên miền trở về địa chỉ IP công khai của VPS; bản thân Cloudflare không giữ vai trò xử lý nghiệp vụ. Sau đó Nginx Proxy Manager mới làm lớp chuyển tiếp truy cập đến các container nội bộ. Vì vậy, người dùng chỉ truy cập <https://thingdock.dev>, <https://api.thingdock.dev>, <wss://mqtt.thingdock.dev>, <https://grafana.thingdock.dev>, <https://emqx.thingdock.dev> và <https://npm.thingdock.dev> qua cổng HTTPS mặc định. Các port 4001, 4000, 8083, 4002, 18083 và 81 là port nội bộ phía sau NPM, không xuất hiện trong URL public. Riêng MQTT TLS cho thiết bị đi trực tiếp tới EMQX qua mqtts.thingdock.dev:8883. VictoriaMetrics (8428) được giữ ở chế độ nội bộ, không công khai qua subdomain (tên miền phụ).

4.2.4.2. Triển khai MQTT Bridge (MQTT Bridge Implementation)

a) Vai trò và kiến trúc MQTT Bridge (`Tracking_MqttBridge/`) là lớp trung chuyển giữa thiết bị và phần còn lại của hệ thống. Thiết bị gửi lên các bản tin MQTT gọn để tiết kiệm băng thông, nhưng phía sau lại cần dữ liệu có cấu trúc rõ ràng để lưu vào cơ sở dữ liệu, dựng giao diện quản trị và truy vết lỗi. Vì vậy, MQTT Bridge làm nhiệm vụ nhận bản tin thô, kiểm tra nó, rồi chuyển nó thành các bản ghi phù hợp cho từng nơi sử dụng. Việc tách thành dịch vụ riêng cũng giúp đường vào dữ liệu vận hành gửi liên tục gọn hơn và không trộn lẫn với nghiệp vụ web của API phía máy chủ.

Luồng xử lý của MQTT Bridge có thể hình dung theo bốn bước:

1. Thiết bị IoT gửi bản tin MQTT vào topic `v1/{device_id}/rawdata` qua EMQX.
2. MQTT Bridge nhận bản tin, kiểm tra cấu trúc dữ liệu bằng Zod, tức là kiểm tra xem bản tin có đủ và đúng các trường bắt buộc hay không.
3. Sau khi hợp lệ, dữ liệu được gửi tới đúng nơi: dữ liệu vận hành gửi liên tục vào VictoriaMetrics, sự kiện hoặc lỗi vào VictoriaLogs, còn trạng thái phiên vào PostgreSQL.
4. Cuối cùng, MQTT Bridge phát sự kiện nội bộ để API phía máy chủ nhận và đẩy cập nhật mới ra giao diện quản trị theo thời gian thực.



Hình 4.20: Luồng xử lý dữ liệu của MQTT Bridge

b) Cấu trúc mã nguồn

- `src/index.ts`: điểm khởi động dịch vụ.
- `src/handlers/`: xử lý bản tin theo nhóm topic (`rawdata`, `status`, `events`, `firmware`).
- `src/infrastructure/`: kết nối PostgreSQL, logger, VictoriaMetrics và Victoria-Logs.
- `src/mqtt/`: client MQTT và cấu hình đăng ký QoS.
- `src/services/`: nghiệp vụ nền như batch writer, xác thực thiết bị, logic kiểm tra vùng cho phép.
- `src/validators/`, `src/types/`, `src/utils/`: chuẩn hóa khuôn dữ liệu hợp lệ, kiểu dữ liệu và tiện ích hỗ trợ.

c) Kết nối MQTT và xử lý bản tin MQTT Bridge kết nối tới EMQX Broker và subscribe vào các topic tương ứng với dữ liệu từ thiết bị IoT. Mỗi topic được định tuyến (route) tới handler phù hợp theo suffix của topic:

```
// index.ts + mqtt/subscriptions.ts
await subscribeToDeviceTopics(client);

client.on("message", (topic, message) => {
  const deviceId = extractDeviceId(topic);
  const suffix = topic.split("/").slice(2).join("/");

  switch (suffix) {
    case "rawdata":
      void handleRawData(deviceId!, message);
      break;
    case "status":
      void handleStatus(deviceId!, message);
      break;
    case "events":
      void handleEvent(deviceId!, message);
      break;
    case "firmware":
      void handleFirmware(deviceId!, message);
      break;
  }
});
```

Trong các handler này, `handleFirmware()` là mắt xích mới đáng chú ý sau khi bổ sung OTA. Handler này không chỉ kiểm tra gói dữ liệu trạng thái firmware mà còn ghi tiến

trình cập nhật vào `firmware_update_log`, đồng thời phát bản ghi sang VictoriaLogs để thuận tiện cho việc truy vết khi cập nhật thất bại hoặc rollback.

d) Xác thực dữ liệu đầu vào (kiểm tra cấu trúc gói dữ liệu) Mỗi bản tin MQTT nhận được đều được xác thực bằng Zod schema trước khi xử lý, giúp chặn sớm gói dữ liệu sai cấu trúc và giảm rủi ro ghi dữ liệu bẩn vào luồng xử lý:

```
// validators/payload.validator.ts
import { z } from "zod";

export const rawDataSchema = z.object({
  device_id: z.string().min(1),
  auth_token: z.string().min(1),
  timestamp: z.number().positive(),
  uptime: z.number().nonnegative().optional(),
  data: z.object({
    vibration: z.number().optional(),
    battery_top: z.number().optional(),
    battery_bot: z.number().optional(),
    latitude: z.number().min(-90).max(90).optional(),
    longitude: z.number().min(-180).max(180).optional(),
    speed: z.number().nonnegative().optional(),
    course: z.number().min(0).max(360).optional(),
    satellites: z.number().int().nonnegative().optional(),
    ignition: z.boolean().optional(),
    error_code: z.number().int().optional(),
  }),
});
```

Ngoài schema cho dữ liệu vận hành gửi liên tục, MQTT Bridge còn dùng `firmwareStatusSchema` để kiểm tra chặt các trường `jobId`, `status`, `progress`, `targetVersion`, `currentVersion`, `partition` và `error`. Nhờ vậy, luồng OTA có một lớp bảo vệ riêng trước khi dữ liệu được ghi vào cơ sở dữ liệu.

e) Ghi dữ liệu kép MQTT Bridge thực hiện cơ chế ghi dữ liệu kép, tức một lần nhận bản tin nhưng ghi ra hai nơi phục vụ hai mục đích khác nhau: dữ liệu vận hành gửi liên tục được ghi đồng thời vào VictoriaMetrics (cơ sở dữ liệu chuỗi thời gian) và VictoriaLogs (nhật ký sự kiện), trong khi thông tin trạng thái thiết bị được cập nhật vào PostgreSQL:

- **VictoriaMetrics:** Lưu trữ các metric theo thời gian như `vehicle_latitude`, `vehicle_longitude`, `vehicle_speed`, `vehicle_vibration`,

vehicle_battery_top, vehicle_battery_bot, vehicle_ignition, vehicle_uptime. Dữ liệu này phục vụ cho truy vấn lịch sử và biểu đồ vận hành.

- **VictoriaLogs**: Lưu trữ các sự kiện hệ thống như kết nối/ngắt kết nối thiết bị, cảnh báo, lỗi xác thực gói dữ liệu. Dữ liệu này phục vụ cho phân tích và debug.
- **PostgreSQL**: Cập nhật trạng thái hiện tại của thiết bị (last_seen_at, current_status), tạo bản ghi phiên hoạt động (session), và lưu các thực thể nghiệp vụ như alert, trip và vùng cho phép.

f) Cấu hình Docker cho MQTT Bridge

```
# Tracking_MqttBridge/docker-compose.yml
version: "3.8"
services:
  mqtt-bridge:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tracking-mqtt-bridge
    env_file: .env
    restart: unless-stopped
    networks:
      - tracking-network
    deploy:
      resources:
        limits:
          memory: 256M
          cpus: "0.5"

networks:
  tracking-network:
    external: true
```

Dịch vụ MQTT Bridge không cần expose port ra bên ngoài vì nó chỉ giao tiếp nội bộ với các dịch vụ khác thông qua mạng Docker dùng chung. Giới hạn tài nguyên được đặt ở mức 256 MB RAM và 0.5 CPU, phù hợp với vai trò xử lý các gói dữ liệu nhỏ.

4.2.4.3. Triển khai API phía máy chủ

a) Lựa chọn công nghệ API phía máy chủ được xây dựng trên nền tảng Node.js với Express.js và TypeScript. Đây là lớp phục vụ trực tiếp giao diện quản trị: nhận yêu cầu từ người dùng, kiểm tra quyền truy cập, lấy dữ liệu đúng chỗ rồi trả lại kết quả ổn định. Vì đường dữ liệu vận hành gửi liên tục đã được MQTT Bridge gánh riêng, API phía máy chủ không phải ôm cả dòng dữ liệu kỹ thuật lẫn nghiệp vụ quản trị trong cùng một chỗ. Việc lựa chọn công nghệ ở đây vì thế ưu tiên sự rõ ràng và linh hoạt trong phát triển lâu dài.

Bảng 4.6: So sánh các khung phát triển API phía máy chủ cho hệ thống IoT

Tiêu chí	Express + TypeScript	NestJS	Go + Gin
Độ phức tạp khi học và dựng dự án	Thấp	Trung bình	Cao
Linh hoạt khi tổ chức mã nguồn	Rất cao	Trung bình (bị ràng buộc bởi khuôn mẫu có sẵn)	Cao
Hiệu năng	Tốt	Tốt	Rất tốt
Lượng mã khung lặp lại	Ít	Nhiều (decorator, module)	Trung bình
Hệ sinh thái IoT	Rất tốt	Tốt	Tốt
Hỗ trợ cập nhật thời gian thực	Rất tốt (tích hợp Socket.IO thuận tiện)	Rất tốt	Tốt

Lý do lựa chọn Express + TypeScript:

- **Linh hoạt cao:** Không bị ràng buộc mạnh bởi các quy ước framework như NestJS, nên nhóm có thể tổ chức code bám theo nghiệp vụ và thay đổi nhanh khi yêu cầu thực tế còn dịch chuyển.
- **Nhẹ và nhanh:** Ít phần gánh kèm hơn NestJS, phù hợp với hệ thống IoT cần phản hồi tốt cho giao diện quản trị mà vẫn giữ tài nguyên máy chủ ở mức hợp lý.
- **Zod Validation:** Sử dụng Zod để chặn yêu cầu sai cấu trúc ngay từ đầu vào. Đây là lớp bảo vệ quan trọng vì API phía máy chủ nhận dữ liệu từ cả giao diện người dùng lẫn các luồng nội bộ khác.
- **Kiểm soát middleware chain:** Toàn quyền kiểm soát thứ tự và cách thức xử lý của middleware, giúp tách rõ xác thực, ghi log, bắt lỗi và đo hiệu năng.

Các thành phần công nghệ chính:

- **Nền chạy và khung phát triển:** Node.js 20+ (LTS), Express 4.x, TypeScript 5.x.
- **Kiểm tra dữ liệu đầu vào:** Zod, thư viện dùng để kiểm tra dữ liệu gửi vào có đúng cấu trúc ngay khi chương trình chạy.

- **Lưu trữ dữ liệu:** PostgreSQL 16 (driver pg) cho dữ liệu quan hệ; VictoriaMetrics (PromQL/MetricsQL) cho dữ liệu thay đổi liên tục theo thời gian.
- **Kênh cập nhật thời gian thực:** Socket.IO 4.8 và MQTT.js tích hợp EMQX.
- **Bảo mật và xác thực:** Cơ chế giữ phiên trên máy chủ, token được băm SHA-256 trong cơ sở dữ liệu, cùng Helmet, CORS và giới hạn tần suất gọi.
- **Tài liệu API:** swagger-ui-express và OpenAPI để sinh tài liệu từ mã nguồn.

b) Kiến trúc Domain-Driven Design (DDD) API phía máy chủ được tổ chức theo kiến trúc Domain-Driven Design với các lớp rõ ràng: lớp nhận yêu cầu, lớp xử lý nghiệp vụ và lớp truy cập dữ liệu. Cách tổ chức này gom mỗi bài toán nghiệp vụ vào một khu vực riêng, nên người đọc mã nguồn có thể lần theo “một việc” trọn vẹn thay vì phải nhảy qua quá nhiều thư mục kỹ thuật rời rạc. Mỗi domain vì thế có thư mục riêng với các thành phần chính như `services/`, `repositories/` và `types/`.

- `index.ts`: điểm khởi động của API phía máy chủ.
- `api/`: lớp API gồm routes và tài liệu OpenAPI.
- `domain/`: lớp nghiệp vụ theo từng module (`auth`, `device`, `vehicle`, `trip`, `alert`, `geofence`, `maintenance`, `firmware`, `dashboard`, ...), mỗi module tách rõ `services/`, `repositories/`, `types/`.
- `infrastructure/`: tích hợp cơ sở dữ liệu, nhật ký, chỉ số giám sát và kênh cập nhật thời gian thực.
- `middleware/`: xác thực, bắt lỗi, giới hạn tần suất, mã truy vết từng yêu cầu, ghi chỉ số và theo dõi lỗi.
- `shared/`: kiểu dữ liệu và tiện ích dùng chung.

Hệ thống bao gồm hơn 20 module nghiệp vụ, mỗi module chịu trách nhiệm một lĩnh vực cụ thể. Cách chia này giúp mã nguồn dễ bảo trì, dễ kiểm thử và dễ mở rộng khi cần thêm tính năng mới.

Bảng 4.7: Danh sách các module nghiệp vụ chính

Module	Số lượng dịch vụ xử lý	Mô tả
auth	2	Đăng nhập/đăng xuất theo phiên làm việc, quản lý người dùng
device	8	CRUD thiết bị, phiên hoạt động, trạng thái đang chạy, dữ liệu vận hành, lệnh điều khiển
vehicle	3	CRUD xe, danh sách, gán thiết bị
customer	2	CRUD khách hàng, danh sách
driver	2	CRUD tài xế, danh sách

Bảng 4.7: Danh sách các module nghiệp vụ chính (tiếp theo)

Module	Số lượng dịch vụ xử lý	Mô tả
trip	2	CRUD chuyến đi, danh sách
geofence	2	CRUD vùng địa lý, danh sách
maintenance	2	CRUD bảo trì, danh sách
firmware	5	OTA: upload, danh sách, kích hoạt, triển khai, xóa
export	3	Quản lý tác vụ xuất, tạo file, xử lý
dashboard (tổng quan)	1	Thống kê tổng quan
telemetry (dữ liệu vận hành)	1	Truy vấn lịch sử dữ liệu vận hành
notification	1	Thông báo đẩy tới người dùng
simulator	1	Mô phỏng thiết bị để kiểm thử

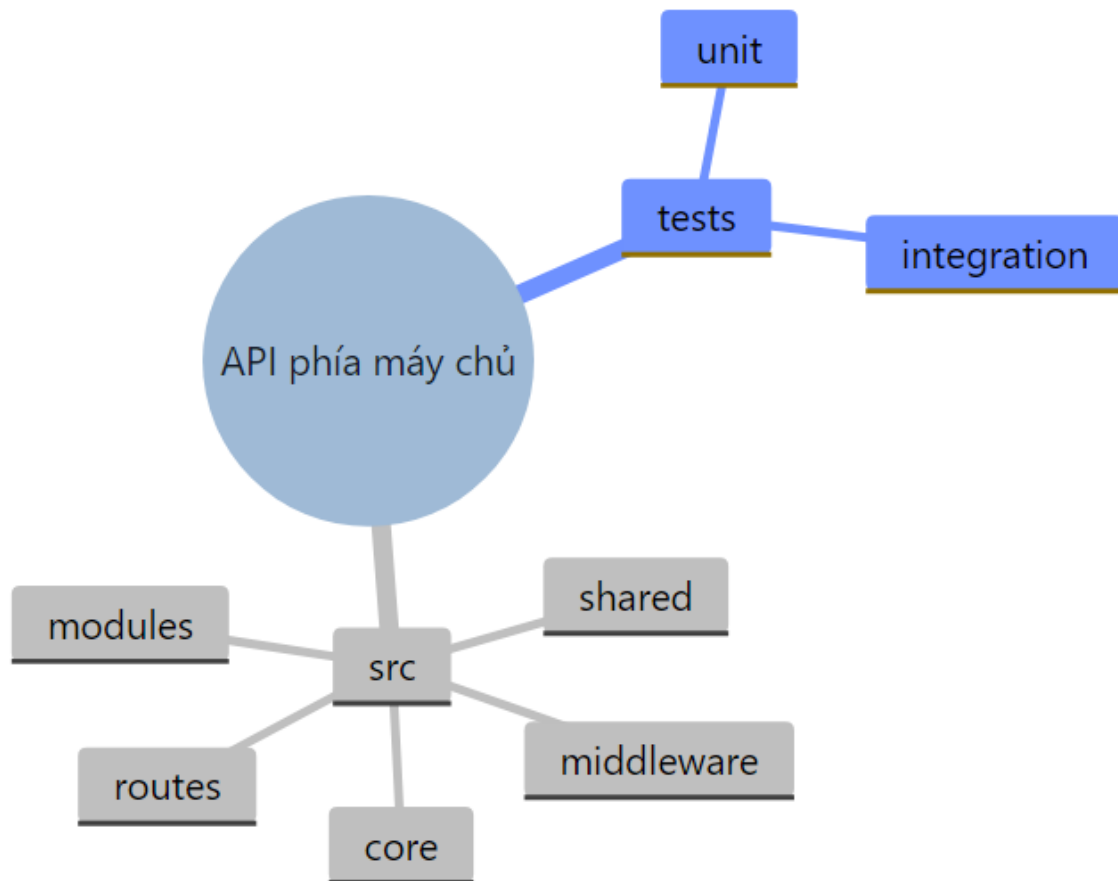
c) Xác thực và phân quyền (Authentication & Authorization) Hệ thống sử dụng cơ chế xác thực dựa trên phiên làm việc, nghĩa là trạng thái đăng nhập được giữ trong cơ sở dữ liệu PostgreSQL. So với cách dùng JWT tự mang thông tin trong mỗi token, cách làm này phù hợp hơn với hệ quản trị nội bộ vì dễ kiểm soát và dễ thu hồi phiên khi cần:

- **Token được hash trước khi lưu:** Dù cơ sở dữ liệu bị lộ, kẻ tấn công cũng không thể lấy nguyên token đã dùng để đăng nhập.
- **Mật khẩu được hash bằng bcryptjs:** Đây là lớp bảo vệ cơ bản để giảm rủi ro nếu dữ liệu người dùng bị lộ.
- **Thu hồi phiên ngay lập tức:** Quản trị viên chỉ cần xóa bản ghi phiên trong cơ sở dữ liệu là có thể chặn phiên đó ngay, không phải chờ token tự hết hạn.

d) Middleware Chain Các middleware được sắp xếp theo thứ tự xử lý cụ thể, đảm bảo bảo mật và hiệu năng:

// index.ts - Thứ tự middleware

```
app.use(sentryRequestHandler); // 1. Sentry error tracking
app.use(helmetMiddleware); // 2. Security headers (XSS, CSP)
app.use(compressionMiddleware); // 3. Nén Gzip/Brotli
app.use(corsMiddleware); // 4. Cross-Origin Resource Sharing
app.use(requestIdMiddleware); // 5. Request-ID correlation (X-Request-ID)
app.use(httpMetricsMiddleware); // 6. Prometheus HTTP metrics
app.use(express.json()); // 7. Body parser (JSON)
```



Hình 4.21: Cấu trúc thư mục API phía máy chủ theo kiến trúc DDD

```

app.use(requestLogger); // 8. Ghi nhật ký request
app.use(rateLimitMiddleware); // 9. Giới hạn tần suất request

// Routes (định tuyến API)
app.use("/api/v1/auth", authRoutes);
app.use("/api/v1/devices", deviceRoutes);
app.use("/api/v1/iot", iotRoutes); // Không yêu cầu xác thực
app.use("/api/v1/vehicles", vehicleRoutes);
app.use("/api/v1/customers", customerRoutes);
// ... các route khác

// Xử lý lỗi (cuối cùng)
app.use(errorHandler);
app.use(sentryErrorHandler);

```

Middleware `requestIdMiddleware` tạo một mã nhận dạng riêng cho mỗi yêu cầu và gắn nó vào header `X-Request-ID`. Nhờ mã này, khi xảy ra lỗi người vận hành có thể lần theo cùng một yêu cầu xuyên suốt từ log của API, lớp xử lý nghiệp vụ cho tới lớp truy

cập dữ liệu.

e) API Endpoints Hệ thống cung cấp các nhóm API endpoint sau:

Bảng 4.8: Tổng hợp các nhóm API endpoint chính

Nhóm	Đường dẫn gốc	Xác thực	Mô tả
Đăng nhập và phiên làm việc	/api/v1/auth	Không (login) / Có	Đăng nhập, đăng xuất, quản lý người dùng
Thiết bị	/api/v1/devices	Có	CRUD thiết bị, phiên hoạt động, trạng thái chạy và lệnh
Dữ liệu IoT	/api/v1/iot	Không (public)	Nhận dữ liệu từ thiết bị IoT
Tổng quan	/api/v1/dashboard	Có	Thống kê, hoạt động gần đây, cảnh báo
Vehicles	/api/v1/vehicles	Có	CRUD xe, gán thiết bị
Khách hàng	/api/v1/customers	Có	CRUD khách hàng
Chuyến đi	/api/v1/trips	Có	CRUD chuyến đi, lịch sử
Cảnh báo	/api/v1/alerts	Có	Quản lý cảnh báo
Vùng địa lý	/api/v1/geofences	Có	CRUD vùng địa lý
Firmware	/api/v1/firmware	Có (admin)	OTA firmware management

Riêng nhóm IoT Data (/api/v1/iot) không đi theo luồng đăng nhập người dùng thông thường. Các thiết bị gửi dữ liệu trực tiếp bằng MQTT hoặc HTTP fallback, nên lớp bảo vệ chính nằm ở cơ chế nhận diện thiết bị và ACL của EMQX thay vì ở phiên đăng nhập web.

f) WebSocket (Socket.IO) Hệ thống sử dụng Socket.IO 4.8 để đẩy các thay đổi mới ra giao diện web ngay khi chúng xảy ra. Thay vì để giao diện quản trị hỏi lại API liên tục, API phía máy chủ chỉ phát sự kiện đúng vào nhóm chức năng liên quan. Vì thế, Socket.IO được tách thành 5 namespace, mỗi namespace phục vụ một nhóm nhu cầu cụ thể:

```
// 5 namespaces với xác thực
/dashboard      // Cập nhật thống kê tổng quan (yêu cầu xác thực)
/devices        // Vị trí, trạng thái, command ack theo thiết bị
/firmware       // Sự kiện gắn/cập nhật tác vụ OTA (yêu cầu xác thực)
/exports        // Tiến trình xuất báo cáo (yêu cầu xác thực)
/notifications  // Thông báo đẩy (yêu cầu xác thực)
```

Các sự kiện chính được phát từ máy chủ đến trình duyệt bao gồm: device:status, device:position, device:session_start, device:session_end, command:ack, stats:update, alert:new, geofence:enter, geofence:exit, export:ready và firmware:assignment. Trong triển khai hiện tại, firmware:assignment phản ánh bước khởi tạo hoặc cập nhật tác vụ OTA từ API phía máy chủ; tiến trình thực thi chi tiết vẫn được theo dõi chủ yếu qua firmware_update_log và lớp log quan trắc.

g) Cấu hình Docker cho Backend

```
# Tracking_Backend/docker-compose.yml
version: "3.8"
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tracking-backend
    ports:
      - "0.0.0.0:4000:4000"
    env_file: .env
    restart: unless-stopped
    networks:
      - tracking-network
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: "1.0"

networks:
  tracking-network:
    external: true
```

h) Cấu trúc phản hồi API thống nhất Tất cả các API endpoint đều trả về phản hồi theo cấu trúc thống nhất, giúp giao diện web xử lý dữ liệu dễ dàng và nhất quán. Trong trường hợp lỗi, phản hồi có cấu trúc:

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Mô tả lỗi chi tiết",
    "status": 400,
    "path": "/api/v1/endpoint",
    "details": null,
    "traceId": "uuid-request-id"
  },
  "timestamp": "2025-01-01T00:00:00.000Z"
}
```

Trường traceId chính là mã truy vết từng yêu cầu được tạo bởi middleware, cho phép đối chiếu lỗi giữa log của giao diện web và log của API phía máy chủ.

4.2.5. Triển khai giao diện web quản trị

Sau khi dữ liệu đã được tiếp nhận và xử lý ở tầng máy chủ, bước tiếp theo là biến chúng thành giao diện đủ rõ ràng để người vận hành có thể quan sát và ra quyết định. Vì thế, phần giao diện web không chỉ được đánh giá ở khía cạnh công nghệ, mà còn ở khả năng tổ chức thông tin theo đúng nhu cầu sử dụng thực tế.

a) Lựa chọn công nghệ Giao diện web quản trị được xây dựng trên nền tảng Next.js 15 với React 19. Điều quan trọng không phải tên công nghệ, mà là giao diện phải đáp ứng ba nhu cầu rất thực tế: tải trang đủ nhanh, cập nhật thời gian thực ổn định và vẫn giữ cấu trúc mã nguồn gọn khi tính năng quản trị tăng lên. Nếu phần máy chủ trả lời câu hỏi “cần cung cấp dữ liệu gì”, thì giao diện web trả lời câu hỏi “nên trình bày ra sao để người vận hành hiểu nhanh và thao tác ít nhất”.

Có thể nhìn toàn bộ phần giao diện web thành ba nhóm vai trò. Nhóm thứ nhất dựng khung ứng dụng và điều hướng trang. Nhóm thứ hai giữ trạng thái giao diện và đồng bộ dữ liệu với máy chủ. Nhóm thứ ba tạo ra những gì người vận hành nhìn thấy trực tiếp như bản đồ, biểu đồ và cập nhật thời gian thực. Cách chia này giúp người đọc hiểu mỗi công cụ xuất hiện để giải quyết một nhu cầu rõ ràng, chứ không phải để làm dày thêm danh sách công nghệ.

Thành phần công nghệ của giao diện web:

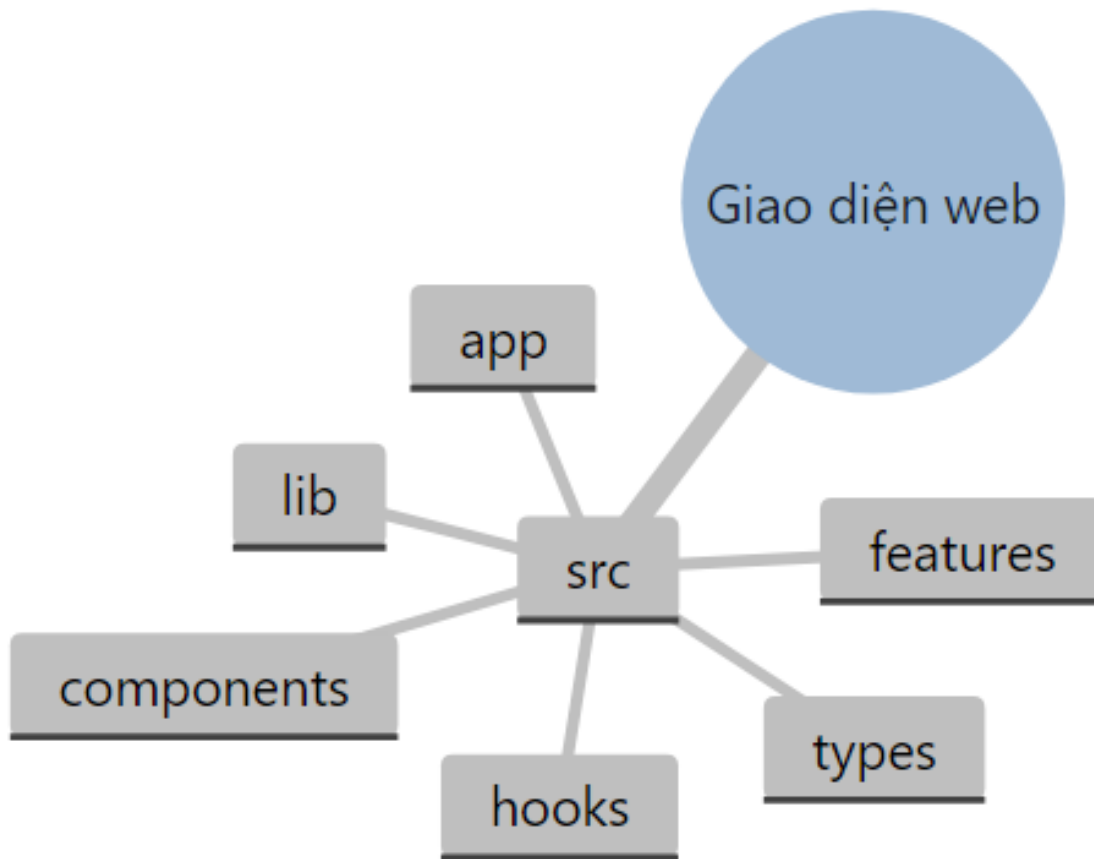
Bảng 4.2.5A: Tổng hợp thông tin về Công nghệ, Phiên bản, Vai trò

Công nghệ	Phiên bản	Vai trò
Next.js	15	Khung ứng dụng React với App Router, hỗ trợ chia trang và bố cục theo thư mục
React	19	Thư viện xây dựng giao diện
TypeScript	5.x	Ngôn ngữ lập trình có kiểm tra kiểu dữ liệu
Tailwind CSS	4	Cách viết CSS bằng các lớp tiện ích dựng sẵn
Zustand	-	Quản lý trạng thái dùng chung của giao diện
TanStack Query	-	Quản lý dữ liệu lấy từ máy chủ, gồm lưu tạm và làm mới lại khi cần
Socket.IO Client	-	Kết nối kênh cập nhật thời gian thực
Leaflet	-	Bản đồ tương tác
ECharts	-	Biểu đồ và trực quan hóa dữ liệu
Radix UI + shadcn/ui	-	Bộ thành phần giao diện có sẵn, hỗ trợ dùng bàn phím tốt và dễ ghép lại theo nhu cầu

b) Kiến trúc tách theo từng nhóm tính năng Giao diện web được tổ chức theo kiến trúc chia mã nguồn theo từng nhóm tính năng, nghĩa là mỗi tính năng người dùng nhìn thấy sẽ có một khu vực mã nguồn tương đối riêng. Trong mỗi khu vực đó, các thành phần UI, hook, kiểu dữ liệu và tiện ích liên quan được đặt gần nhau. Nhờ vậy, cấu trúc mã nguồn phản chiếu khá sát cấu trúc nghiệp vụ, nên ngay cả người không đi sâu vào code cũng có thể hình dung hệ thống web được tổ chức như thế nào.

- **Đặt gần theo tính năng:** Phần mã liên quan đến một chức năng nằm gần nhau, dễ tìm và bảo trì
- **Độc lập:** Mỗi nhóm tính năng có thể phát triển và kiểm thử độc lập
- **Tái sử dụng:** Các thành phần dùng chung nằm trong `components/`, chỉ phần chuyên biệt mới nằm trong `features/`
- `app/`: khung định tuyến trang và các khu vực quản trị được bảo vệ.
- `components/`: các thành phần dùng lại cho giao diện, bố cục, biểu mẫu, bản đồ, biểu đồ và lớp cung cấp ngữ cảnh dùng chung.
- `features/`: các nhóm nghiệp vụ theo FSD (`vehicles`, `customers`, `trips`, `alerts`, `map`).
- `lib/`: các lớp dùng chung như nơi gọi API, kết nối thời gian thực, nơi giữ trạng thái,

hằng số, tiện ích và hook dùng lại.



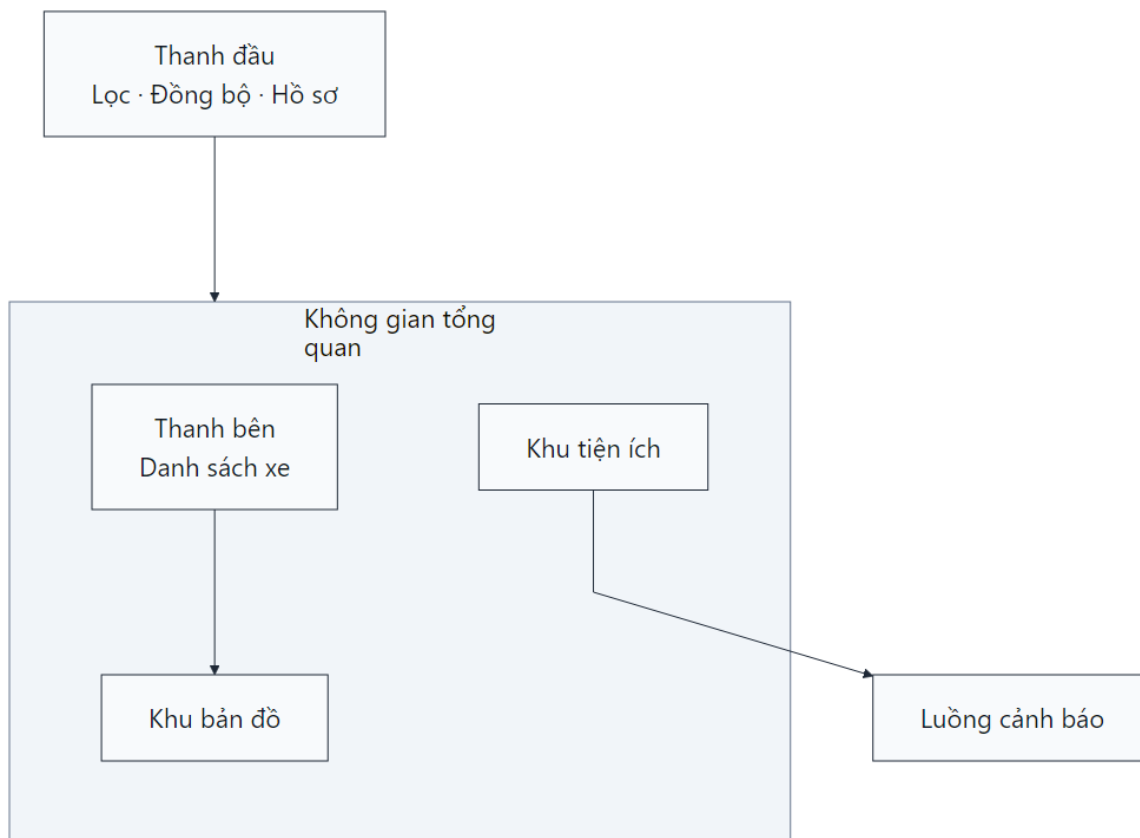
Hình 4.22: Cấu trúc thư mục giao diện web theo kiến trúc tách theo từng nhóm tính năng

c) Các trang chính và tính năng Để giao diện quản trị thực sự hữu ích trong vận hành, giao diện không được dừng ở việc “có nhiều trang”, mà mỗi trang phải trả lời được một nhu cầu quản lý cụ thể. Vì vậy, hệ thống cung cấp các nhóm trang chính sau:

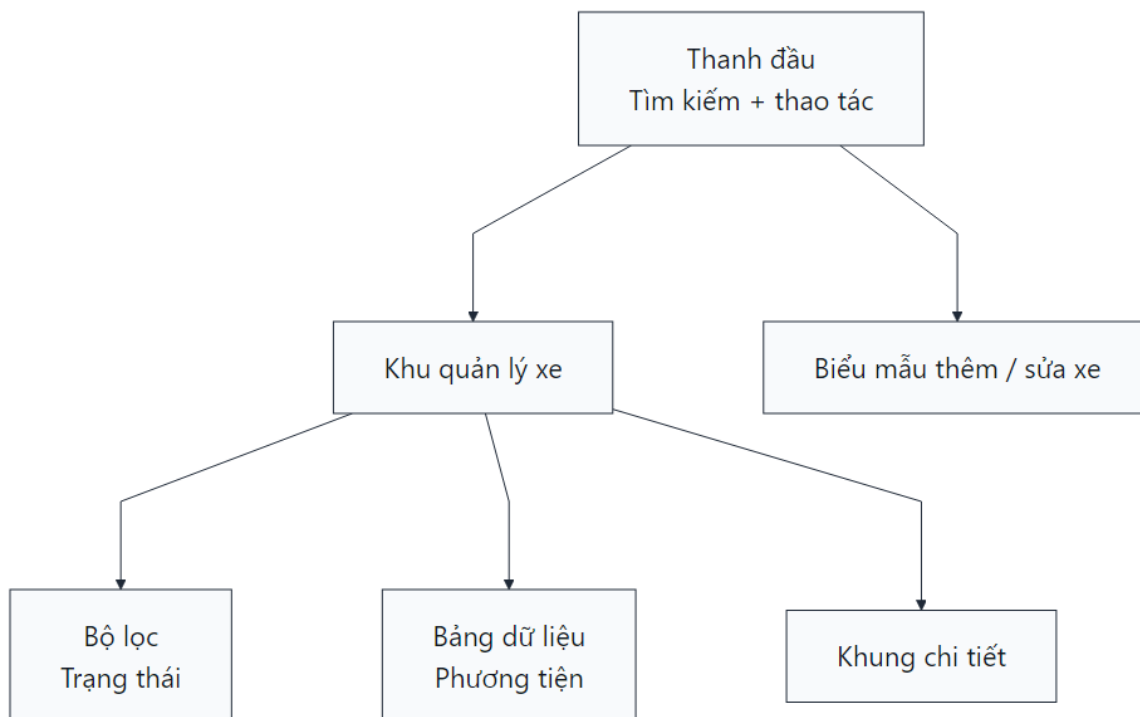
Trang tổng quan (/dashboard): Hiển thị các thẻ thống kê nhanh bao gồm tổng số xe, số chuyến đi trong ngày, số cảnh báo chưa xử lý, và số vi phạm. Ngoài ra còn hiển thị bảng cảnh báo gần đây, bản đồ mini với các xe đang hoạt động, danh sách chuyến đi gần đây, và biểu đồ thống kê (số chuyến đi theo ngày, cảnh báo theo loại).

Trang quản lý xe (/dashboard/vehicles): Giao diện dạng bảng dữ liệu với các cột: biển số xe, hãng xe/model, trạng thái, thiết bị gắn kèm, lần cuối thấy, và các hành động. Hỗ trợ lọc theo trạng thái, loại xe, và tìm kiếm. Trang chi tiết xe hiển thị thông tin xe, vị trí hiện tại trên bản đồ, trạng thái thiết bị, cảnh báo đang hoạt động, chuyến đi gần đây, và lịch sử bảo trì.

Trang bản đồ thời gian thực (/dashboard/map): Hiển thị tất cả các xe trên bản đồ Leaflet với vị trí cập nhật thời gian thực thông qua WebSocket. Hỗ trợ: hiển thị điểm

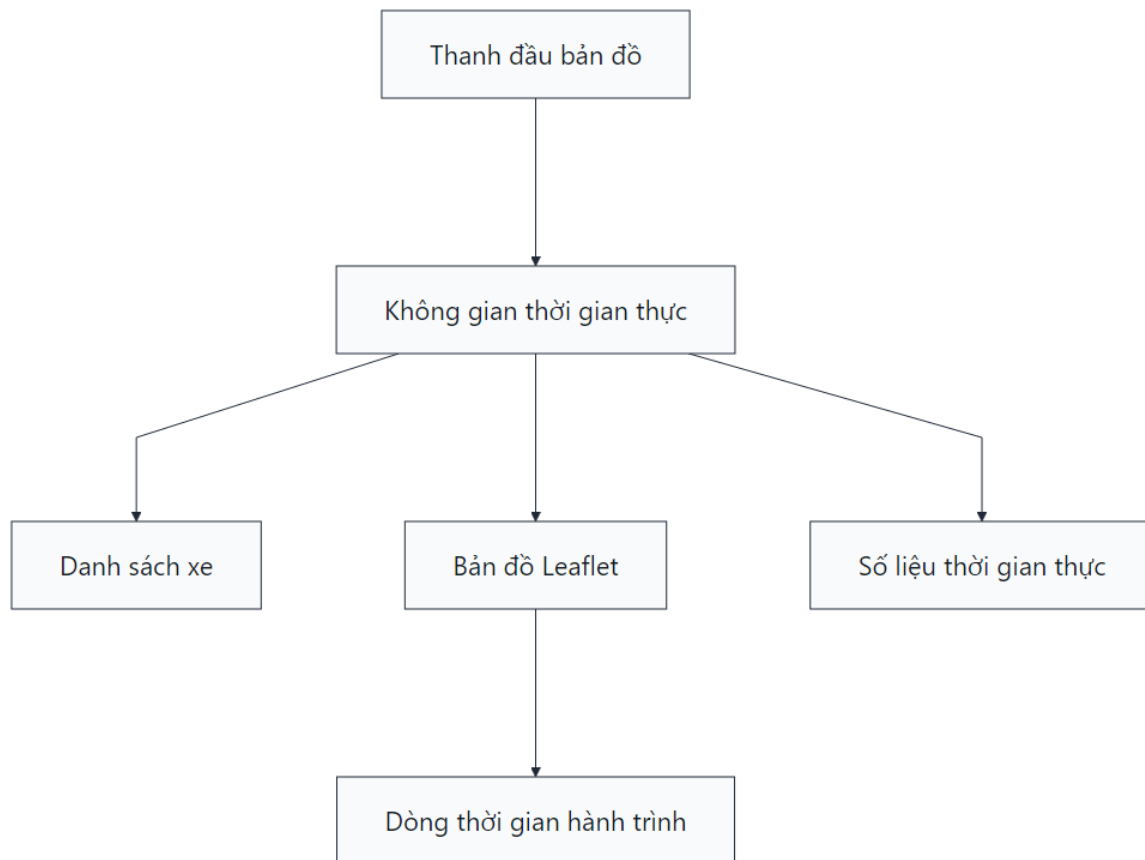


Hình 4.23: Giao diện trang tổng quan hệ thống



Hình 4.24: Giao diện trang quản lý xe

đánh dấu cho từng xe với ô thông tin trạng thái, lọc theo xe hoặc trạng thái, hiển thị vùng cho phép đã thiết lập, và phát lại hành trình cũ.



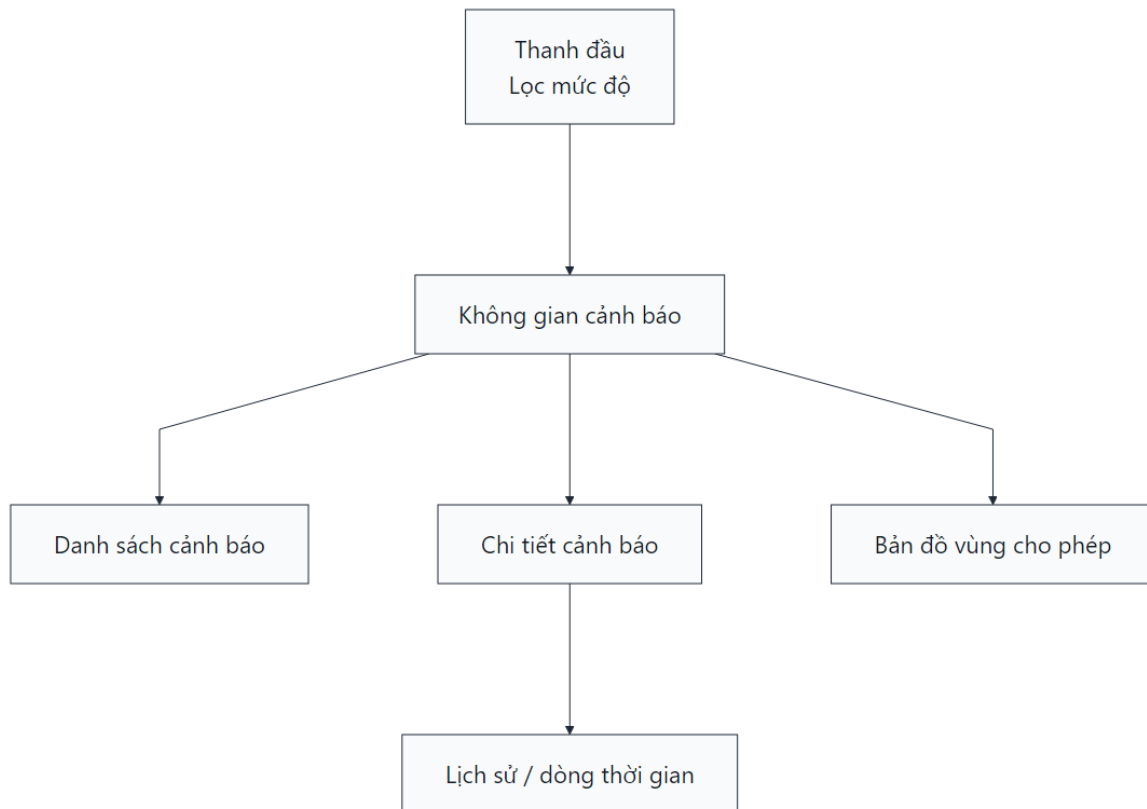
Hình 4.25: Giao diện bản đồ thời gian thực với vị trí các xe

Trang quản lý cảnh báo (/dashboard/alerts): Bảng dữ liệu với nhãn mức độ nghiêm trọng, hỗ trợ lọc theo loại, mức độ, xe, và khoảng thời gian. Các hành động hàng loạt gồm xác nhận đã nhận và đánh dấu đã xử lý. Trang chi tiết cảnh báo hiển thị thông tin cảnh báo, vị trí trên bản đồ, xe và khách hàng liên quan.

Trang quản lý vùng cho phép (/dashboard/geofences): Cho phép tạo và quản lý các vùng cho phép trên bản đồ. Khi xe ra khỏi hoặc vào vùng địa lý đã định nghĩa, hệ thống sẽ tự động tạo cảnh báo.

Trang cài đặt thông báo (/dashboard/notifications): Cho phép cấu hình kết nối Telegram bot, cài đặt email, lựa chọn loại cảnh báo muốn nhận, lọc theo mức độ nghiêm trọng và xe cụ thể. Có nút gửi thông báo thử nghiệm để kiểm tra cấu hình.

d) Quản lý trạng thái và dữ liệu Với giao diện thời gian thực, mã nguồn giao diện web sẽ rất nhanh rồi nếu trộn lẫn dữ liệu từ máy chủ, trạng thái giao diện và luồng sự kiện mới vào cùng một chỗ. Vì vậy, hệ thống chia rõ từng loại dữ liệu và dùng kết hợp nhiều công cụ quản lý trạng thái:



Hình 4.26: Giao diện trang quản lý cảnh báo

- **Zustand**: Giữ các trạng thái “thuộc về giao diện” như thông tin đăng nhập hiện tại, sidebar hay theme. Token xác thực chỉ nằm trong bộ nhớ tạm thời, không lưu vào localStorage để giảm rủi ro mã độc chèn vào trang rồi lấy cắp thông tin.
- **TanStack Query**: Quản lý dữ liệu lấy từ máy chủ như danh sách xe, cảnh báo hay lịch sử. Công cụ này lo cache, làm mới và đồng bộ lại dữ liệu khi cần. Khi WebSocket đã hoạt động, cơ chế polling sẽ giảm xuống để tránh tải trùng lặp.
- **Socket.IO Client**: Nhận các cập nhật thời gian thực do API phía máy chủ phát ra. Giao diện web không nối trực tiếp tới MQTT broker; mọi dữ liệu thời gian thực đều được API phía máy chủ chuẩn hóa rồi mới đẩy ra giao diện.

e) **Các giai đoạn triển khai** Giao diện web được triển khai theo 3 giai đoạn chính (giai đoạn 4 là giai đoạn mở rộng trong tương lai):

Bảng 4.9: Các giai đoạn triển khai giao diện web

Giai đoạn	Nội dung	Trạng thái
Giai đoạn 1: Nền tảng cốt lõi	Khởi tạo dự án, Tailwind + shaden/ui, giao diện sáng/tối, bố cục, xác thực, lớp gọi API và React Query	Hoàn thành

Bảng 4.9: Các giai đoạn triển khai giao diện web (tiếp theo)

Giai đoạn	Nội dung	Trạng thái
Giai đoạn 2: Tính năng cốt lõi	Trang tổng quan, CRUD xe, CRUD khách hàng, quản lý chuyến đi, quản lý cảnh báo, quản lý thiết bị, bản đồ thời gian thực	Hoàn thành
Giai đoạn 3: Tính năng nâng cao	Quản lý vùng cho phép, bảo trì, cài đặt thông báo (Telegram + Email), quản lý người dùng, thiết lập hệ thống	Hoàn thành
Giai đoạn 4: Đặt xe (tương lai)	Đặt xe, hợp đồng, thanh toán, báo cáo hư hỏng, đánh giá	Chưa triển khai

f) Cấu hình Docker cho Frontend Giao diện web sử dụng Dockerfile nhiều giai đoạn: một giai đoạn để biên dịch và một giai đoạn để chạy, nhằm tối ưu kích thước image như sau:

Tracking_Frontend/Dockerfile

FROM node:20-alpine **AS** builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

Giai đoạn 2: Chạy ứng dụng (production)

FROM node:20-alpine **AS** runner

WORKDIR /app

ENV NODE_ENV=production

ENV HOSTNAME=0.0.0.0

ENV PORT=4001

COPY --from=builder /app/public ./public

COPY --from=builder /app/.next/standalone ./

COPY --from=builder /app/.next/static ./next/static

RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs

USER nextjs

EXPOSE 4001

HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \

CMD wget --spider http://127.0.0.1:4001 || exit 1

CMD ["node", "server.js"]

Dockerfile sử dụng 2 giai đoạn: builder (cài đặt dependency và build ứng dụng), và runner (chạy ở môi trường vận hành thật). Giai đoạn runner chỉ sao chép các file cần thiết từ giai đoạn builder, giúp giảm đáng kể kích thước image cuối cùng. Ứng dụng chạy với user nextjs (không phải root) để tăng tính bảo mật.

4.2.6. Cấu hình giám sát hệ thống

Khi hệ thống đã có đủ các lớp phần cứng, firmware, hạ tầng máy chủ và giao diện web, câu hỏi tiếp theo là làm sao biết hệ thống đang khỏe hay đang có vấn đề. Đó là lý do lớp giám sát không được xem như phần phụ, mà là điều kiện để vận hành ổn định và xử lý sự cố có cơ sở.

a) Tổng quan chiến lược giám sát Hệ thống giám sát được tổ chức theo ba câu hỏi cơ bản: hệ thống đang nặng đến đâu, chuyện gì vừa xảy ra, và một yêu cầu đã đi qua những đâu. Ba câu hỏi đó tương ứng với chỉ số vận hành, nhật ký và dấu vết truy vết yêu cầu.

Cách trình bày này giúp người đọc hiểu khả năng quan sát hệ thống không phải là thêm thật nhiều công cụ, mà là thêm khả năng quan sát để tìm nguyên nhân sự cố có cơ sở. Trong phạm vi đồ án, phần truy vết chưa dùng đầy đủ cơ chế truy vết phân tán như OpenTelemetry để nối dấu vết qua nhiều dịch vụ; thay vào đó hệ thống dùng mã nhận diện từng yêu cầu để nối log giữa giao diện web, API phía máy chủ và các dịch vụ liên quan. Cách làm này đơn giản hơn nhưng vẫn đủ cho quy mô triển khai hiện tại.

Bảng 4.10: Ba trụ cột giám sát hệ thống

Trụ cột	Công cụ	Mục đích	Dữ liệu lưu trữ
Chỉ số vận hành	VictoriaMetrics + bộ xuất số liệu theo chuẩn Prometheus	Chỉ số hiệu năng, tài nguyên, nghiệp vụ	CPU, bộ nhớ, số yêu cầu/giây, độ trễ, số thiết bị online
Nhật ký	VictoriaLogs	Nhật ký tập trung từ tất cả dịch vụ	Nhật ký yêu cầu, nhật ký lỗi, sự kiện MQTT, sự kiện thiết bị
Truy vết yêu cầu	Mã nhận diện từng yêu cầu	Lần theo một yêu cầu xuyên suốt hệ thống	Mã X-Request-ID trên mỗi yêu cầu
Theo dõi lỗi	Sentry	Theo dõi lỗi và ngoại lệ	Vết gọi hàm, ngưỡng cảnh người dùng, chuỗi sự kiện ngay trước khi lỗi xảy ra

b) VictoriaMetrics - Giám sát chỉ số VictoriaMetrics đóng hai vai trò trong hệ thống:

1. **Lưu trữ dữ liệu vận hành của thiết bị:** Đây là các dữ liệu vận hành phát sinh liên tục như vị trí GPS, tốc độ, điện áp nguồn, mức rung, trạng thái bật/tắt máy và thời gian chạy liên tục. Nguồn dữ liệu này đi từ MQTT Bridge.
2. **Lưu trữ chỉ số sức khỏe hệ thống:** Đây là các chỉ số cho biết API phía máy chủ và hạ tầng đang hoạt động ra sao, ví dụ số yêu cầu HTTP, độ trễ xử lý, số kết nối WebSocket hay số bản tin MQTT. Các dịch vụ gửi nhóm chỉ số này qua thư viện `prom-client`.

Backend dùng middleware `httpMetricsMiddleware` để tự động ghi nhận các chỉ số HTTP trong lúc chạy. Endpoint `/metrics` đóng vai trò như “đầu ra kỹ thuật” để Grafana đọc các số liệu này thông qua VictoriaMetrics, từ đó dựng biểu đồ giám sát theo thời gian thực.

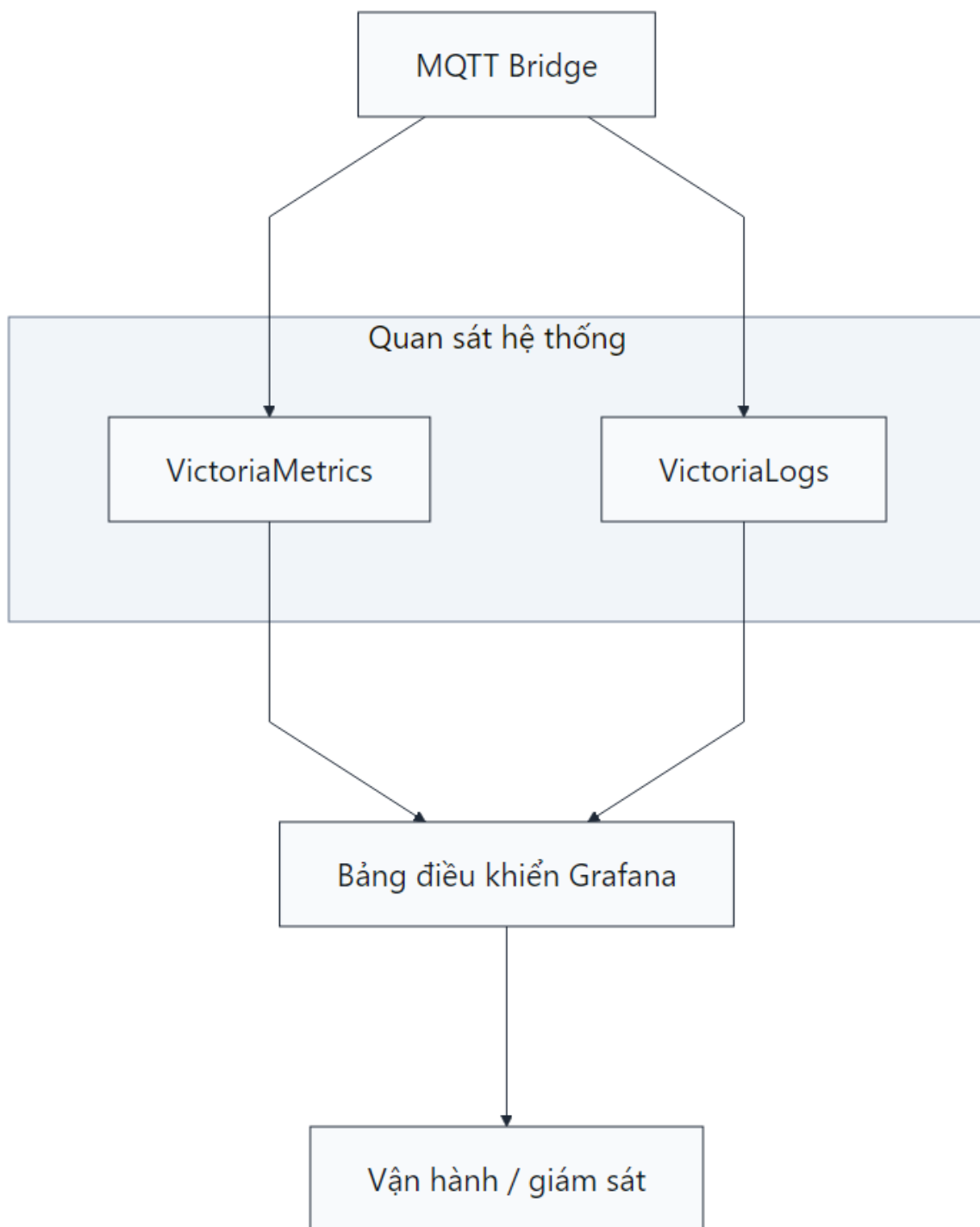
c) VictoriaLogs - Nhật ký tập trung VictoriaLogs là hệ thống nhật ký tập trung, thu thập log từ tất cả các dịch vụ trong hệ thống. API phía máy chủ sử dụng `victorialogs-transport` của Winston để gửi log trực tiếp tới VictoriaLogs. Mỗi bản ghi log bao gồm:

- **Timestamp:** Thời điểm xảy ra sự kiện
- **Level:** Mức độ nghiêm trọng (info, warn, error)
- **Service:** Tên dịch vụ phát sinh log (API phía máy chủ, mqtt-bridge, giao diện web)
- **Request-ID:** Mã nhận diện yêu cầu để truy vết
- **Message:** Nội dung nhật ký
- **Metadata:** Dữ liệu bổ sung (`device_id`, `user_id`, vết gọi hàm khi lỗi)

MQTT Bridge cũng có logger riêng, gửi log về VictoriaLogs với trường `service: mqtt-bridge`, cho phép lọc và phân tích log theo từng dịch vụ.

d) Grafana - Màn hình giám sát trực quan Grafana được sử dụng làm lớp trực quan hóa, kết nối tới cả VictoriaMetrics và VictoriaLogs để hiển thị các màn hình giám sát. Các màn hình chính bao gồm:

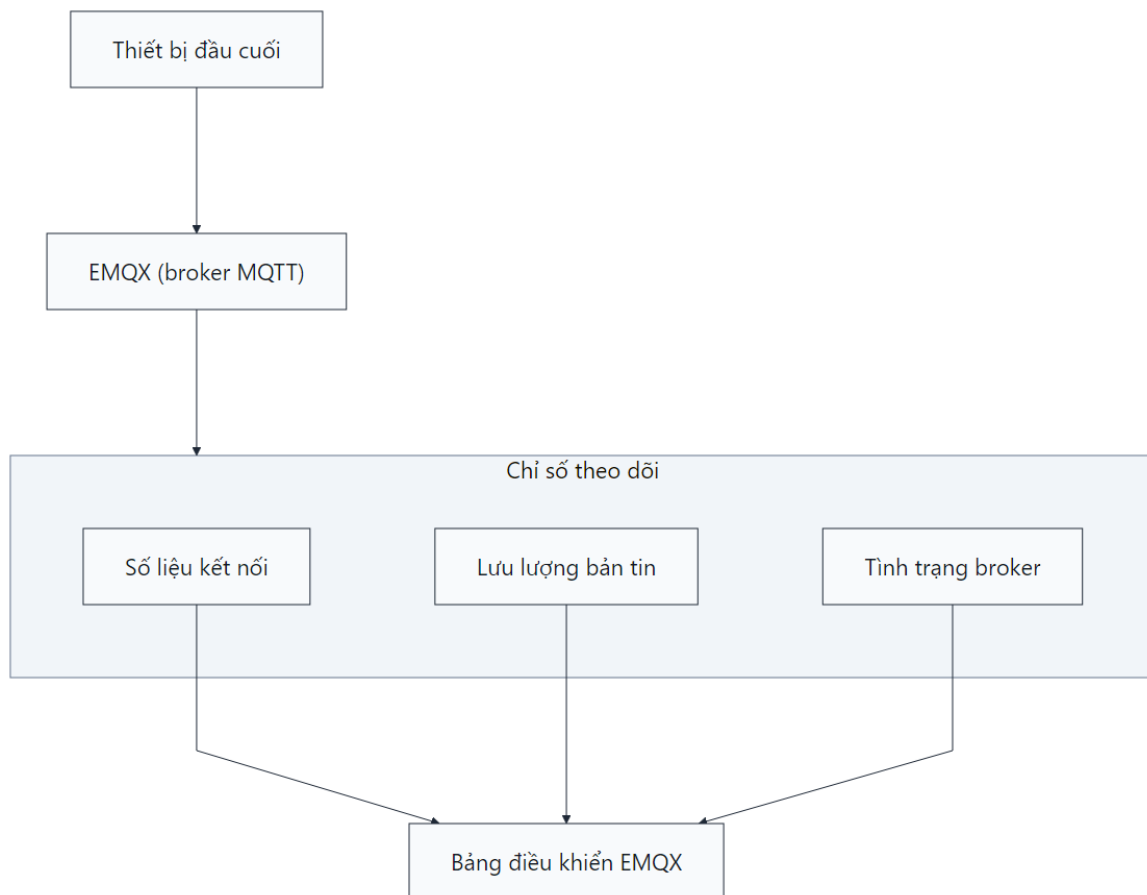
- **Tổng quan hệ thống:** Số thiết bị online, số yêu cầu/giây, phần trăm lỗi, mức sử dụng CPU và bộ nhớ của các container.
- **Dữ liệu vận hành IoT:** Dữ liệu từ các thiết bị IoT — bản đồ nhiệt (heatmap) vị trí xe, biểu đồ tốc độ, biểu đồ mức nhiên liệu theo thời gian.
- **Thống kê MQTT:** Số lượng bản tin/giây, số kết nối đang hoạt động, độ trễ trung bình, tỷ lệ bản tin thất bại.
- **Hiệu năng ứng dụng:** Phân phối thời gian xử lý yêu cầu, các endpoint chậm nhất, tỷ lệ lã»—i theo endpoint.



Hình 4.27: Màn hình Grafana hiển thị tổng quan hiệu năng hệ thống

e) **Giao diện quản trị EMQX - Giám sát broker MQTT** EMQX cung cấp giao diện quản trị tích hợp (port 18083) cho phép giám sát trực tiếp trạng thái của broker MQTT:

- **Connections:** Số lượng kết nối hiện tại, lịch sử kết nối/ngắt kết nối
- **Subscriptions:** Danh sách topic và bên đang đăng ký nhận
- **Messages:** Thống kê bản tin (gửi lên, chuyển đi thành công, bị loại bỏ)
- **Rules Engine:** Trạng thái các luật xử lý dữ liệu
- **ACL:** Cấu hình quyền truy cập cho từng thiết bị



Hình 4.28: Giao diện quản trị EMQX hiển thị trạng thái kết nối thiết bị

f) **Sentry - Theo dõi lỗi ứng dụng** API phía máy chủ tích hợp Sentry thông qua hai middleware: `sentryRequestHandler` (đặt đầu tiên trong middleware chain) và `sentryErrorHandler` (đặt cuối cùng). Sentry tự động thu thập:

- **Chi tiết lỗi:** Vết gọi hàm đầy đủ, biến môi trường, thông tin yêu cầu
- **Chuỗi sự kiện trước lỗi:** Các sự kiện xảy ra trước khi lỗi xuất hiện (truy vấn cơ sở dữ liệu, gọi HTTP, sự kiện MQTT)
- **Hiệu năng:** Thời gian xử lý của mỗi yêu cầu
- **Theo dõi theo phiên bản:** Lỗi thuộc phiên bản triển khai nào

Sentry đặc biệt hữu ích trong môi trường vận hành thật khi không thể truy cập trực tiếp vào log của máy chủ.

g) Liên kết truy vết bằng mã nhận diện yêu cầu Mỗi yêu cầu HTTP đến API phía máy chủ đều được gán một mã truy vết duy nhất (UUID v4) bởi middleware `requestIdMiddleware`. Mã này được:

- Trả về trong header `X-Request-ID` của phản hồi
- Đính kèm vào mỗi bản ghi log liên quan đến yêu cầu đó
- Truyền xuyên suốt qua các lớp xử lý (controller -> service -> repository)
- Sử dụng làm `traceId` trong phản hồi lỗi

Cơ chế này cho phép lần theo một yêu cầu cụ thể từ giao diện web, qua API phía máy chủ, đến cơ sở dữ liệu và MQTT Bridge, giúp việc dò lỗi và phân tích sự cố trong hệ thống nhiều dịch vụ hiệu quả hơn.

4.2.7. Checklist làm cứng vận hành trước khi đưa vào môi trường thật

Để chuyển hệ thống từ môi trường thử nghiệm sang môi trường vận hành thực tế, nhóm triển khai áp dụng checklist làm cứng vận hành theo các mức ưu tiên sau (tham chiếu từ bộ tài liệu cải tiến hệ thống):

Bảng 4.11: Checklist làm cứng vận hành theo mức ưu tiên

Nhóm	Hạng mục bắt buộc	Mức ưu tiên	Tiêu chí nghiệm thu
Bảo mật truyền thông	Bật TLS cho HTTPS và MQTT (8883), bắt buộc chứng chỉ hợp lệ	Cao	100% kết nối ngoài đi qua TLS, không còn điểm truy cập thuần văn bản
Xác thực thiết bị	Mỗi thiết bị có thông tin xác thực riêng (chứng chỉ số hoặc khóa bí mật riêng)	Cao	Thiết bị giả mạo không gửi được bản tin vào topic hợp lệ
Bảo vệ API	Giới hạn tần suất gọi, danh sách nguồn được phép gọi (CORS), kiểm tra dữ liệu đầu vào bằng schema (Zod)	Cao	Tất cả điểm gọi công khai đều có giới hạn tần suất và bước kiểm tra đầu vào

Bảng 4.11: Checklist làm cứng vận hành theo mức ưu tiên (tiếp theo)

Nhóm	Hạng mục bắt buộc	Mức ưu tiên	Tiêu chí nghiệm thu
Độ tin cậy dịch vụ	Điểm kiểm tra trạng thái sống, chính sách tự khởi động lại và cơ chế phát hiện treo trong firmware (watchdog)	Cao	Tự phục hồi khi dịch vụ treo, có trạng thái sống/chết rõ ràng
Dữ liệu và sao lưu	Sao lưu PostgreSQL tự động + chính sách giữ dữ liệu cho log và dữ liệu vận hành	Cao	Phục hồi được dữ liệu theo mục tiêu khôi phục đã đặt ra
Lưu trữ cục bộ SD	Thử gắn lại thẻ nhớ với thời gian chờ tăng dần, buộc ghi ra thẻ trước khi đóng file và tháo thẻ đúng vòng đời phiên	Cao	Không phát sinh lỗi ghi hàng loạt khi mạng gián đoạn
Phát lại hàng đợi SD	Đặt thời gian chờ xác nhận, thử lại với khoảng chờ tăng dần, tách bản tin quan trọng và dọn hàng đợi theo hạn mức	Cao	Hàng đợi phát lại tiến dần, không bị nghẽn vô hạn
Hiệu năng hệ thống	Tái sử dụng sẵn kết nối cơ sở dữ liệu, tối ưu truy vấn và thêm vùng nhớ đệm khi cần	Trung bình	Độ trễ P95 của API giữ ổn định khi tăng tải
Khả năng mở rộng	Phân tải giữa nhiều bản sao dịch vụ API khi tăng số thiết bị	Trung bình	Việc tăng thêm bản sao không làm gián đoạn kết nối WebSocket/MQTT
Quan sát và cảnh báo	Ghi nhận sự kiện an ninh, phát cảnh báo khi lỗi hoặc độ trễ vượt ngưỡng	Trung bình	Cảnh báo được gửi tới Telegram/Email khi vượt ngưỡng

Lộ trình thực hiện được chia làm 2 đợt: **đợt 1.5** (ưu tiên cao, hoàn thành trước khi đưa vào chạy thật) và **đợt 2** (mở rộng năng lực chịu tải và giám sát theo tải thực tế). Cách chia này giúp giảm rủi ro vận hành mà không làm chậm tiến độ đưa hệ thống vào khai thác.

Tóm tắt

Hệ thống máy chủ được triển khai theo kiến trúc nhiều dịch vụ nhỏ, tức mỗi dịch vụ được tách thành một tiến trình độc lập, đóng gói trong container Docker riêng và giao tiếp qua một mạng Docker dùng chung. Kết quả triển khai cho thấy MQTT Bridge thực hiện đúng vai trò cầu nối dữ liệu giữa thiết bị IoT và các hệ lưu trữ, đồng thời API phía máy chủ và giao diện web đáp ứng yêu cầu tích hợp, giám sát vận hành thời gian thực.

Mô hình mỗi dịch vụ có một cấu hình Docker Compose riêng giúp hệ thống dễ bảo trì, mở rộng và triển khai độc lập từng thành phần. Hệ thống giám sát với VictoriaMetrics, VictoriaLogs, Grafana và Sentry bảo đảm khả năng quan sát và phân tích sự cố trong môi trường vận hành thực tế.

Bảng 4.11A: Tổng hợp công nghệ sử dụng trong hệ thống máy chủ

Thành phần	Công nghệ	Phiên bản	Vai trò
MQTT Broker	EMQX	5.4.0	Tiếp nhận dữ liệu từ thiết bị IoT
MQTT Bridge	Node.js + TypeScript	20 LTS	Xử lý và phân phối dữ liệu MQTT
API phía máy chủ	Express + TypeScript	4.x	Cung cấp API, đẩy dữ liệu thời gian thực và xử lý nghiệp vụ
Giao diện web	Next.js + React	15 / 19	Giao diện web quản trị
CSDL quan hệ	PostgreSQL	16	Dữ liệu người dùng, xe, cảnh báo
CSDL chuỗi thời gian	VictoriaMetrics	-	Dữ liệu vận hành gửi liên tục (GNSS, nguồn, rung, trạng thái)
Nhật ký tập trung	VictoriaLogs	-	Nhật ký các dịch vụ
Màn hình giám sát	Grafana	-	Trực quan hóa chỉ số và nhật ký
Container	Docker + Docker Compose	-	Đóng gói và triển khai dịch vụ
Cổng chuyển tiếp	Nginx Proxy Manager	-	Quản lý tên miền, SSL, định tuyến truy cập

4.3. Đo lường và kết quả – Measurements and Results

Phần này trình bày thiết lập thử nghiệm, kết quả đo theo từng tầng hệ thống (phần cứng, firmware, máy chủ), kết quả kiểm thử tích hợp end-to-end và đối chiếu với các tiêu chí thiết kế ở Chương 1 và Chương 2. Nói cách khác, đây là phần kiểm chứng xem hệ thống sau khi hiện thực có thực sự đáp ứng được những gì đã hứa ở các chương trước hay không. Toàn bộ giá trị được ghi nhận trên thiết bị phần cứng đã chế tạo và lắp đặt thực tế trong điều kiện vận hành bám sát bối cảnh khai thác.

Ghi chú: Các giá trị đo lường trong chương này là kết quả thu được trên môi trường thử nghiệm cụ thể (xem mục 4.3.1). Một số chỉ số được trình bày dưới dạng ước tính ban đầu hoặc cần đo lặp lại trên nhiều mẫu để tăng độ tin cậy thống kê.

4.3.1. Thiết lập môi trường thử nghiệm

4.3.1.1. Môi trường thử nghiệm phần cứng Kiểm thử phần cứng được triển khai tại phòng thí nghiệm và trên xe thực tế với các thiết bị đo lường sau:

Bảng 4.12: Thiết bị đo lường sử dụng trong thử nghiệm phần cứng

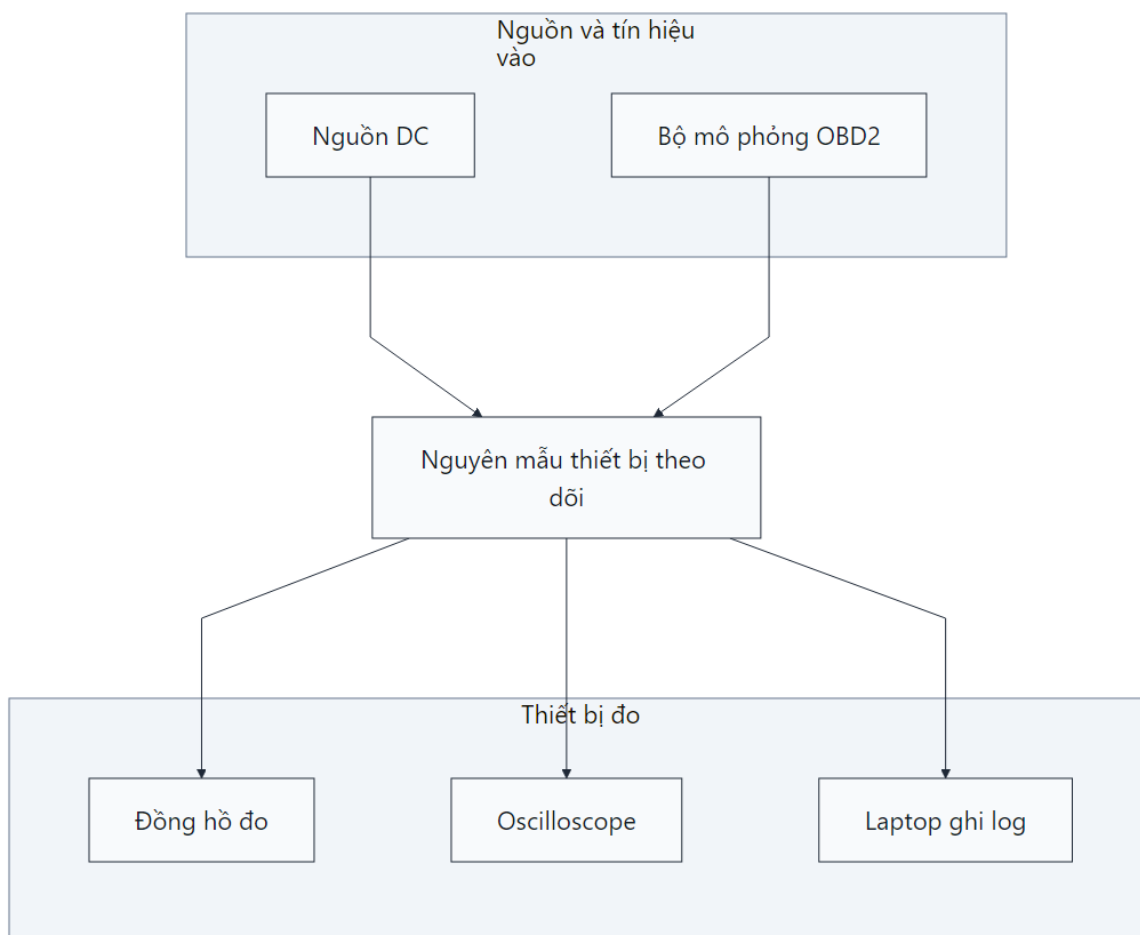
STT	Thiết bị	Model	Thông số chính	Mục đích sử dụng
1	Đồng hồ vạn năng (DMM)	UNI-T UT61E	Độ chính xác 0.5%, đo dòng μA -10A	Đo dòng tiêu thụ các chế độ
2	Nguồn cấp DC	UNI-T UTP1306S	0–32V / 0–6A, độ phân giải 10mV/1mA	Mô phỏng ắc quy xe 12V hoặc 24V
3	Oscilloscope	Rigol DS1054Z	50 MHz, 4 kênh	Phân tích tín hiệu, đo thời gian chuyển trạng thái
4	Tủ nhiệt	Tủ nhiệt phòng thí nghiệm	-20°C đến +80°C	Kiểm thử nhiệt độ hoạt động
5	OBD2 Simulator	ELM327 OBD2 Simulator Board	Hỗ trợ tất cả giao thức OBD2	Mô phỏng dữ liệu xe
6	Điểm đo GPS tham chiếu	Thử nghiệm ngoài trời	Tọa độ chuẩn đối chiếu	Kiểm thử độ chính xác GPS

Xe thử nghiệm:

- Xe 1: Toyota Vios 2020 (động cơ 1.5L, ắc quy 45Ah, OBD2 giao thức ISO 15765–4)

CAN)

- Xe 2: Honda City 2021 (động cơ 1.5L, ắc quy 40Ah, OBD2 giao thức ISO 15765–4 CAN)
- Giai đoạn hiện tại đã hoàn tất đo trên 2 xe; xe thứ ba sẽ được bổ sung ở vòng kiểm thử tương thích mở rộng.



Hình 4.29: Bố trí thiết bị đo lường trong phòng thí nghiệm

4.3.1.2. Môi trường thử nghiệm phần mềm Hệ thống máy chủ được triển khai trên hai môi trường riêng biệt để kiểm thử:

Bảng 4.13: Cấu hình môi trường thử nghiệm phần mềm

Thành phần	Môi trường phát triển	Môi trường kiểm thử tải
Server	PC cá nhân, Intel i5–12400, 16 GB RAM, SSD 512 GB	VPS 4 vCPU, 8 GB RAM, SSD 100 GB
Hệ điều hành	Windows 11 + WSL2 (Ubuntu 22.04)	Ubuntu 22.04 LTS

Bảng 4.13: Cấu hình môi trường thử nghiệm phần mềm (tiếp theo)

Thành phần	Môi trường phát triển	Môi trường kiểm thử tải
Docker	Docker Desktop 4.x	Docker Engine 24.x
Mạng	LAN 1 Gbps + Wi-Fi 5 GHz	VPS bandwidth 100 Mbps
EMQX Broker	Single node, port 1883	Single node, port 1883
PostgreSQL	Version 16, default config	Version 16, tuned (shared_buffers=2GB)
VictoriaMetrics	Single node	Single node

Từ cấu hình môi trường trên, bộ công cụ đo được chọn theo đúng từng lớp cần kiểm chứng: API đo tải bằng k6, lớp MQTT đo bằng MQTT Bench, còn giao diện web được đánh giá bằng Lighthouse.

Công cụ kiểm thử hiệu năng:

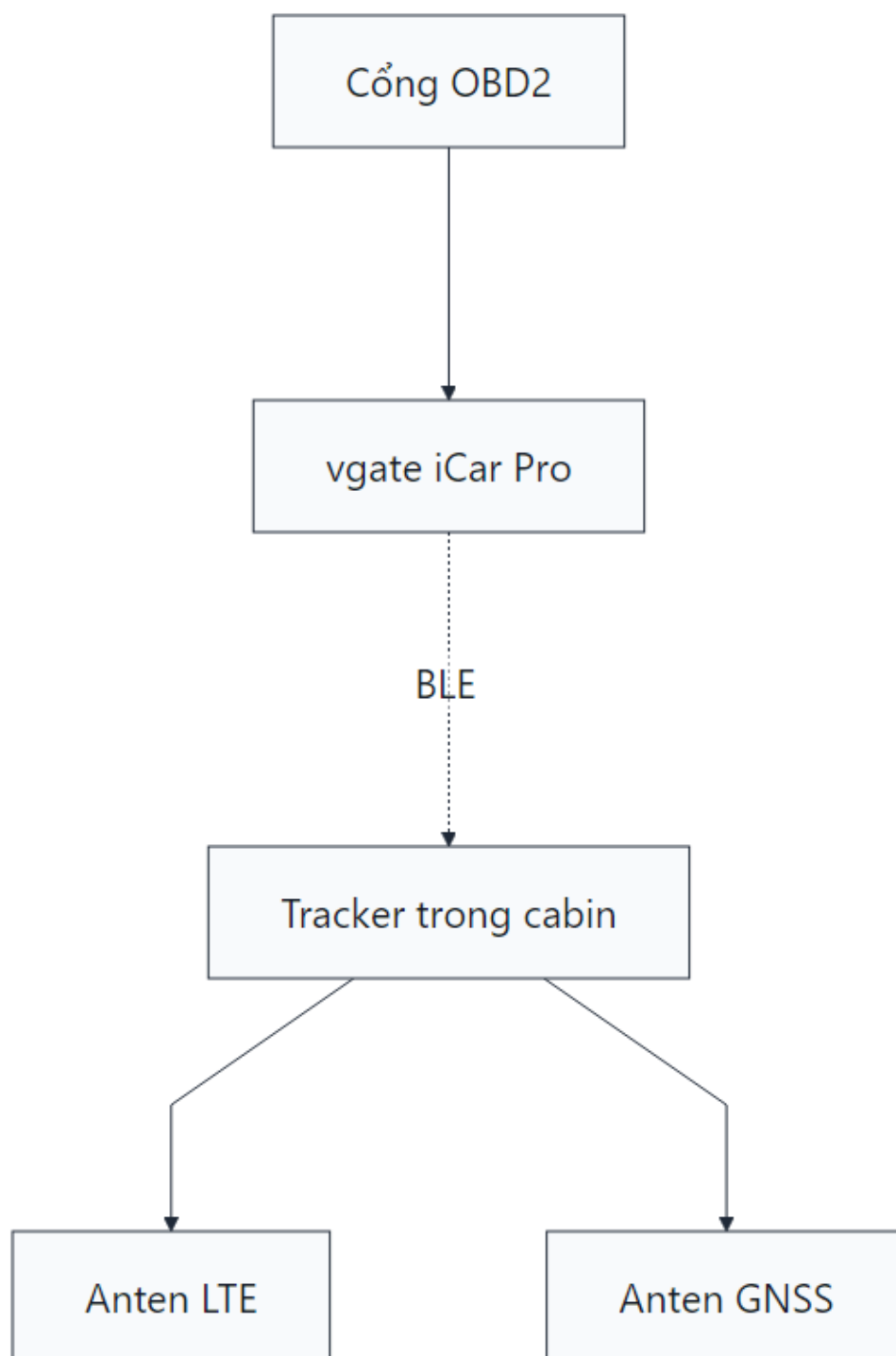
- **k6** (Grafana Labs): Kiểm thử tải các API với nhiều kịch bản đồng thời
- **MQTT Bench**: Công cụ mô phỏng nhiều MQTT client đồng thời
- **Lighthouse** (Google): Đánh giá hiệu năng giao diện web
- **Playwright**: Kiểm thử giao diện tự động theo toàn bộ luồng dùng trình duyệt

4.3.1.3. Kịch bản thử nghiệm Các kịch bản thử nghiệm được thiết kế bao phủ ba nhóm chính:

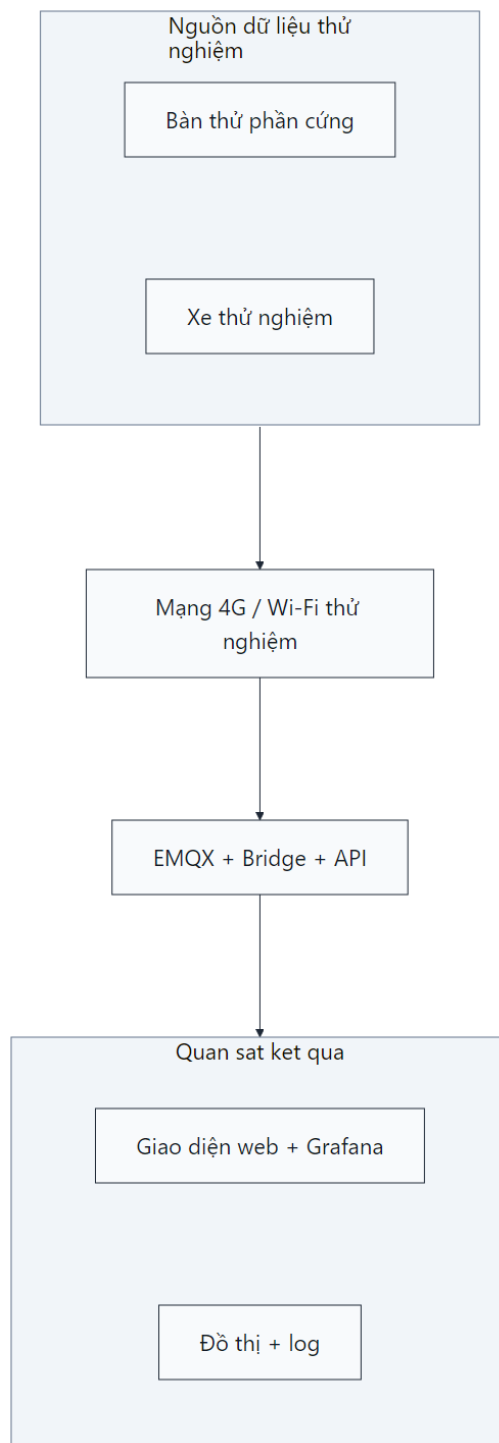
1. **Kiểm thử đơn vị (Unit Test)**: Kiểm tra từng module độc lập — phần cứng, firmware, API phía máy chủ và các thành phần giao diện web.
2. **Kiểm thử tích hợp (Integration Test)**: Kiểm tra sự tương tác giữa các module — BLE + OBD2, MQTT + Bridge + Database, API + WebSocket + Frontend.
3. **Kiểm thử hệ thống (System Test)**: Kiểm tra toàn bộ luồng dữ liệu end-to-end, từ thiết bị đến giao diện người dùng.

4.3.2. Kết quả kiểm thử phần cứng

4.3.2.1. Đo dòng tiêu thụ năng lượng Dòng tiêu thụ năng lượng là chỉ tiêu cốt lõi của thiết bị theo dõi, quyết định thời lượng hoạt động và mức ảnh hưởng lên ắc quy xe. Phép đo được thực hiện bằng cách mắc nối tiếp đồng hồ vạn năng giữa nguồn cấp DC và thiết bị, sau đó ghi nhận giá trị dòng trung bình trong 60 giây cho mỗi chế độ.



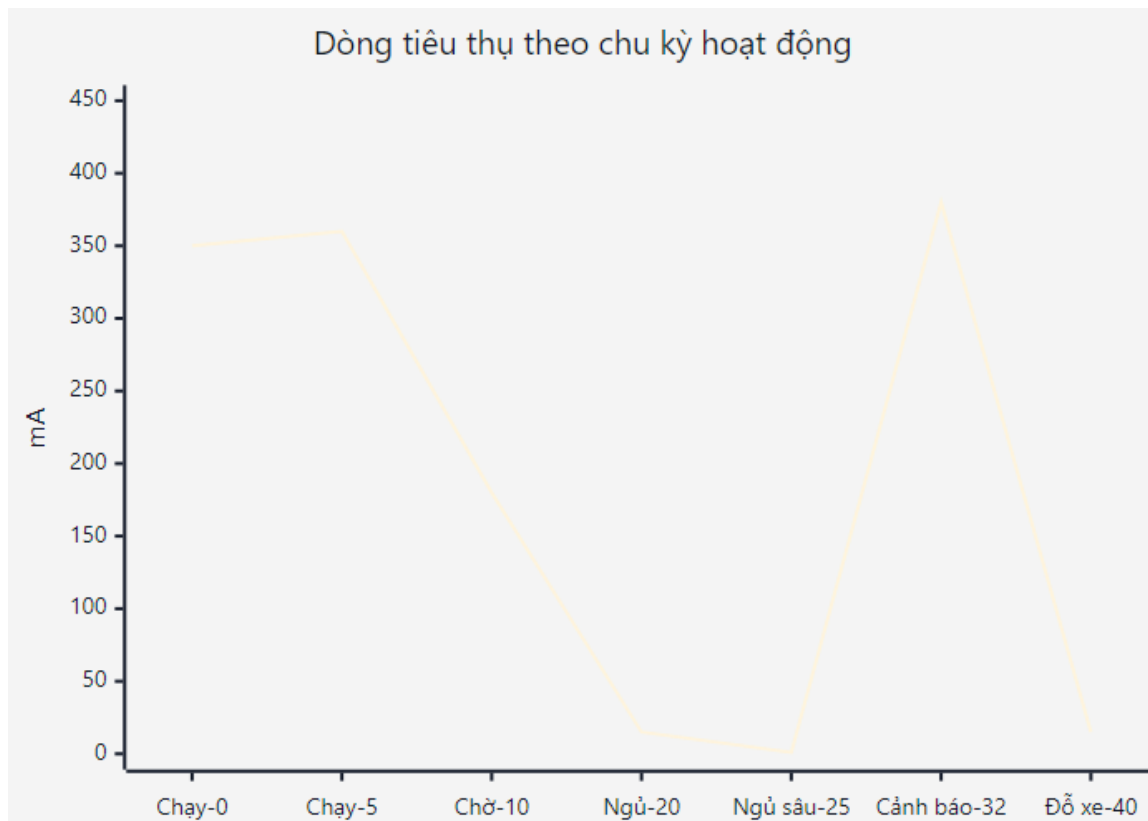
Hình 4.30: Thiết bị theo dõi được lắp đặt trên xe thử nghiệm



Hình 4.31: Sơ đồ môi trường thử nghiệm tổng thể

Bảng 4.14: Kết quả đo dòng tiêu thụ năng lượng theo chế độ hoạt động

STT	Chế độ hoạt động	Mô tả	Dòng trung bình	Dòng đỉnh (peak)	Ghi chú
1	Xe đang chạy - gửi dữ liệu liên tục	GPS + 4G + BLE + vi điều khiển hoạt động đầy đủ	350 mA	520 mA	Dòng đỉnh khi truyền dữ liệu 4G
2	Xe đang chạy - chờ giữa hai lần gửi	4G còn kết nối + vi điều khiển vẫn hoạt động	180 mA	280 mA	Giữa các chu kỳ gửi dữ liệu
3	Xe đỗ (IGN OFF)	Vi điều khiển ngủ nhẹ + 4G tắt	15 mA	25 mA	Heartbeat mỗi 10–30 phút
4	Ngủ sâu	Vi điều khiển ngủ sâu + IMU đánh thức	0.5 mA	2 mA (ước tính, cần xác nhận thêm)	Chỉ IMU + RTC hoạt động
5	Cảnh báo bất thường	Đánh thức từ ngủ sâu	380 mA	550 mA	Tương tự chế độ xe chạy + GPS cold start



Hình 4.32: Đồ thị dòng tiêu thụ theo thời gian trong một chu kỳ hoạt động hoàn chỉnh (xe chạy -> xe đỗ -> cảnh báo -> xe đỗ)

Phân tích kết quả:

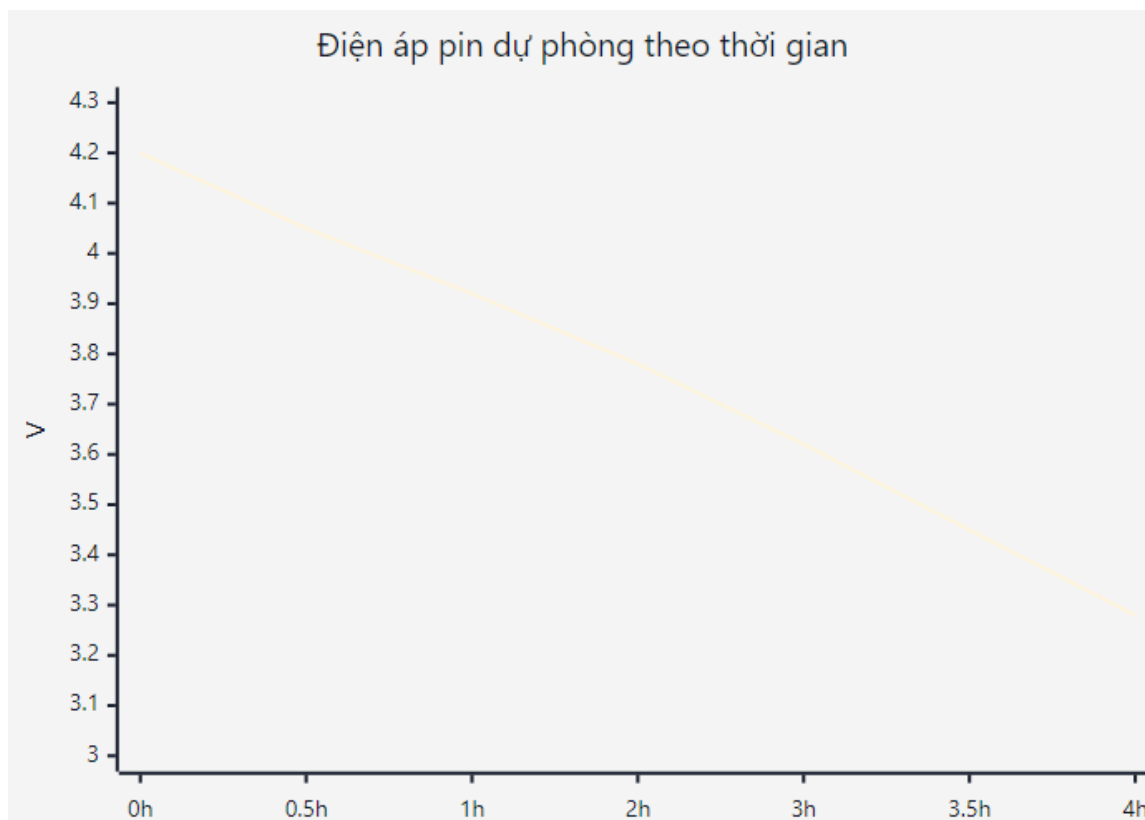
- Dòng tiêu thụ trong chế độ Active (350 mA) phù hợp với tính toán thiết kế tại Chương 3, trong đó modem SIM7600CE-T chiếm khoảng 200–250 mA (khi truyền dữ liệu 4G), ESP32-S3 chiếm khoảng 60–80 mA và linh kiện phụ trợ chiếm khoảng 20–30 mA.
- Dòng tiêu thụ trong chế độ Deep Sleep (0.5 mA) nằm sát ngưỡng thiết kế hệ thống. Giá trị này cao hơn mức lý thuyết của riêng lõi ESP32-S3 + LIS3DH do còn có tổn hao từ mạch nguồn, đường cấp nguồn, linh kiện phụ trợ và điều kiện đo thực tế. Dù vậy, mức tiêu thụ này vẫn đủ để thiết bị duy trì giám sát đỗ xe trong thời gian dài mà không gây hao ắc quy đáng kể.
- Dòng đỉnh khi truyền dữ liệu 4G (520 mA) cần được lưu ý trong thiết kế mạch nguồn, đảm bảo tụ điện lọc đặt gần nguồn đủ lớn để tránh sụt áp.

4.3.2.2. Thời lượng pin dự phòng Pin dự phòng 18650 1S (dung lượng danh định 3500 mAh, điện áp danh định 3.7V) được quy đổi theo cùng công thức đã dùng cho phương án gốc: $T = \frac{C \times V_{pin} \times \eta}{P_{tiêu\ thu}}$, trong đó η là hiệu suất chuyển đổi của mạch boost converter (85–90%). Các giá trị dưới đây là **quy đổi theo bộ pin tham chiếu 18650**,

chưa phải bộ số đo thực nghiệm mới trên pin 18650 thực tế.

Bảng 4.15: Thời lượng pin dự phòng theo kịch bản sử dụng (quy đổi theo bộ pin tham chiếu 18650)

STT	Kịch bản sử dụng	Dòng trung bình	Thời lượng ước tính	Trạng thái dữ liệu	Ghi chú
1	Theo dõi liên tục khi xe chạy	350 mA	2.6–3.0 giờ	Quy đổi theo bộ pin tham chiếu 18650	Trường hợp xấu nhất, phụ thuộc hiệu suất boost
2	Sleep + heartbeat 30 phút	15 mA	64 giờ	Quy đổi theo bộ pin tham chiếu 18650	Xe đậu bình thường
3	Ngủ sâu, chỉ chờ IMU đánh thức	0.5 mA	1,940 giờ (81 ngày)	Quy đổi theo bộ pin tham chiếu 18650	Chế độ tiết kiệm tối đa
4	Hỗn hợp (50% xe chạy + 50% xe đỗ)	182 mA	5.0 giờ	Quy đổi theo bộ pin tham chiếu 18650	Mô phỏng sử dụng thực tế



Hình 4.33: Đồ thị điện áp pin dự phòng theo thời gian khi quy đổi theo bộ pin tham chiếu 18650 trong kịch bản theo dõi liên tục khi xe chạy

Nhận xét: Với bộ pin tham chiếu 18650, thời lượng pin dự phòng trong chế độ theo dõi liên tục khi xe chạy được ước tính còn khoảng 2.6–3.0 giờ tùy hiệu suất chuyển đổi thực tế. Trong kịch bản xe đỗ và chỉ gửi heartbeat định kỳ, thiết bị vẫn có thể duy trì hoạt động trong nhiều ngày; tuy nhiên, các giá trị này cần được kiểm thử lại trên mẫu pin 18650 thực tế nếu dùng cho giai đoạn chế tạo tiếp theo.

4.3.2.3. Kiểm thử nhiệt độ hoạt động Thiết bị được đặt trong tủ nhiệt để kiểm tra khả năng hoạt động ở các mức nhiệt độ khác nhau, mô phỏng môi trường trong xe ô tô (có thể vượt 60°C vào mùa hè và xuống dưới 0°C trong phòng lạnh xe).

Bảng 4.16: Kết quả kiểm thử nhiệt độ hoạt động

STT	Nhiệt độ thử nghiệm	Trạng thái hoạt động	GPS fix	4G kết nối	OBD2 BLE	Ghi chú
1	-10°C	Bình thường	Có	Có	Có	Thời gian khởi động tăng 20%
2	0°C	Bình thường	Có	Có	Có	Hoạt động ổn định

Bảng 4.16: Kết quả kiểm thử nhiệt độ hoạt động (tiếp theo)

STT	Nhiệt độ thử nghiệm	Trạng thái hoạt động	GPS fix	4G kết nối	OBD2 BLE	Ghi chú
3	25°C (nhiệt độ phòng)	Bình thường	Có	Có	Có	Điều kiện tham chiếu
4	45°C	Bình thường	Có	Có	Có	Hoạt động ổn định
5	60°C	Bình thường	Có	Có	Có	Nhiệt độ MCU tăng, vẫn trong giới hạn
6	70°C	Cảnh báo	Có	Có (không ổn định)	Có	Module 4G bắt đầu không ổn định, cần kiểm tra lắp lại
7	80°C	Ngưng hoạt động	—	—	—	Vượt giới hạn nhiệt độ an toàn

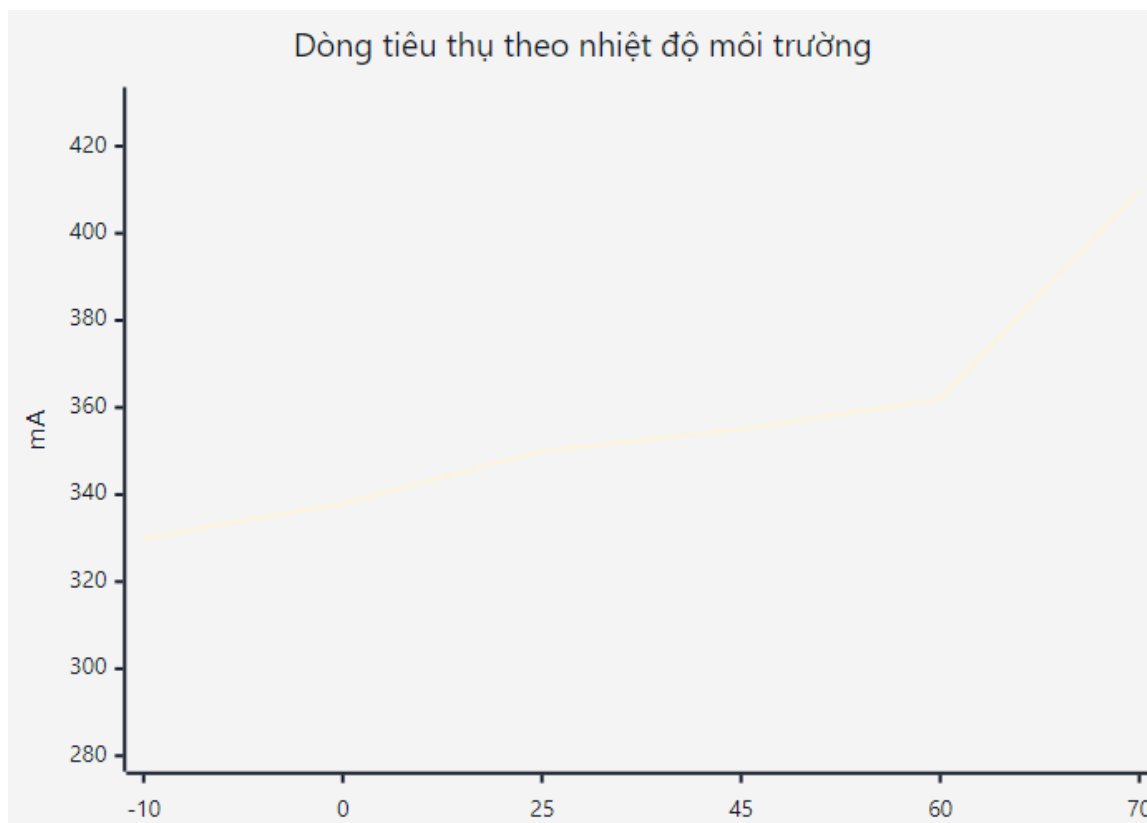
Nhận xét: Thiết bị hoạt động ổn định trong dải nhiệt độ -10°C đến +60°C, đạt yêu cầu thiết kế. Tại nhiệt độ 70°C, modem 4G SIM7600CE-T bắt đầu biểu hiện không ổn định (mất kết nối ngắt quãng), trong khi dải nhiệt độ hoạt động danh định mà SIMCom công bố cho dòng SIM7600CE là -40°C đến +85°C [1].

4.3.2.4. Kiểm thử tương thích OBD2 Thiết bị được kiểm thử kết nối với nhiều thương hiệu xe khác nhau thông qua adapter vgate iCar Pro BLE để đánh giá khả năng tương thích.

Bảng 4.17: Kết quả kiểm thử tương thích OBD2 trên các thương hiệu xe

STT	Xe thử nghiệm	Năm sản xuất	Giao thức OBD2	Kết nối BLE	Đọc RPM	Đọc Speed	Đọc Coolant Temp	Ghi chú
1	Toyota Vios	2020	ISO 15765-4 CAN	Thành công	Có	Có	Có	Đầy đủ
2	Honda City	2021	ISO 15765-4 CAN	Thành công	Có	Có	Có	Đầy đủ

Bảng trên chỉ tổng hợp các mẫu xe đã được kiểm thử thực tế trong phạm vi hiện tại; các mẫu xe chưa đo không được liệt kê để tránh làm sai lệch kết luận đánh giá.



Hình 4.34: Đồ thị dòng tiêu thụ theo nhiệt độ môi trường

Nhận xét: Các xe thử nghiệm đều sử dụng giao thức CAN-bus (ISO 15765–4), là giao thức phổ biến nhất trên các xe sản xuất từ năm 2008 trở đi theo tiêu chuẩn OBD-II [2]. Adapter vgate iCar Pro hỗ trợ tất cả các giao thức OBD2 (CAN, KWP2000, ISO 9141), đảm bảo tương thích với đa số các dòng xe tại Việt Nam.

4.3.2.5. Kiểm thử mạch Low Voltage Disconnect (LVD) Mạch LVD được kiểm thử bằng cách sử dụng nguồn cấp DC (UNI-T UTP1306S) mô phỏng ắc quy xe. Kết quả đo hiện tại được thực hiện theo profile 12V (quét điện áp từ 13.5V xuống 10.5V) và ghi nhận điểm ngắt/đóng lại.

Bảng 4.18: Kết quả kiểm thử mạch LVD

STT	Thông số	Giá trị thiết kế	Giá trị đo	Sai lệch	Trạng thái
1	Điện áp ngắt (disconnect)	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V [đo theo profile 12V]	4.33%	Chưa đạt (12V)

Bảng 4.18: Kết quả kiểm thử mạch LVD (tiếp theo)

STT	Thông số	Giá trị thiết kế	Giá trị đo	Sai lệch	Trạng thái
2	Điện áp đóng lại khi kết nối lại	Profile 12V: 12.2 V; Profile 24V: 24.4 V	12.53V [đo theo profile 12V, cần tinh chỉnh về 12.2V]	2.70%	Chưa đạt (12V)
3	Độ trễ ngắt	< 100 ms	50 ms (đo bước đầu)	—	Đạt
4	Độ trễ đóng lại	< 500 ms	200 ms (đo bước đầu)	—	Đạt
5	Hysteresis	1.0V	1.05V (đo bước đầu)	5%	Đạt

Nhận xét: Kết quả hiện tại đang theo profile 12V: điện áp ngắt 11.48V thấp hơn mục tiêu $OFF = 12.0\text{ V}$, và điện áp đóng lại 12.53V cao hơn mục tiêu $ON = 12.2\text{ V}$ nên cần tinh chỉnh comparator/firmware. Với profile 24V, tiêu chí tương ứng cần đạt là ngắt tại 24.0 V và đóng lại tại 24.4 V khi thực hiện vòng đo 27V xuống 21V.

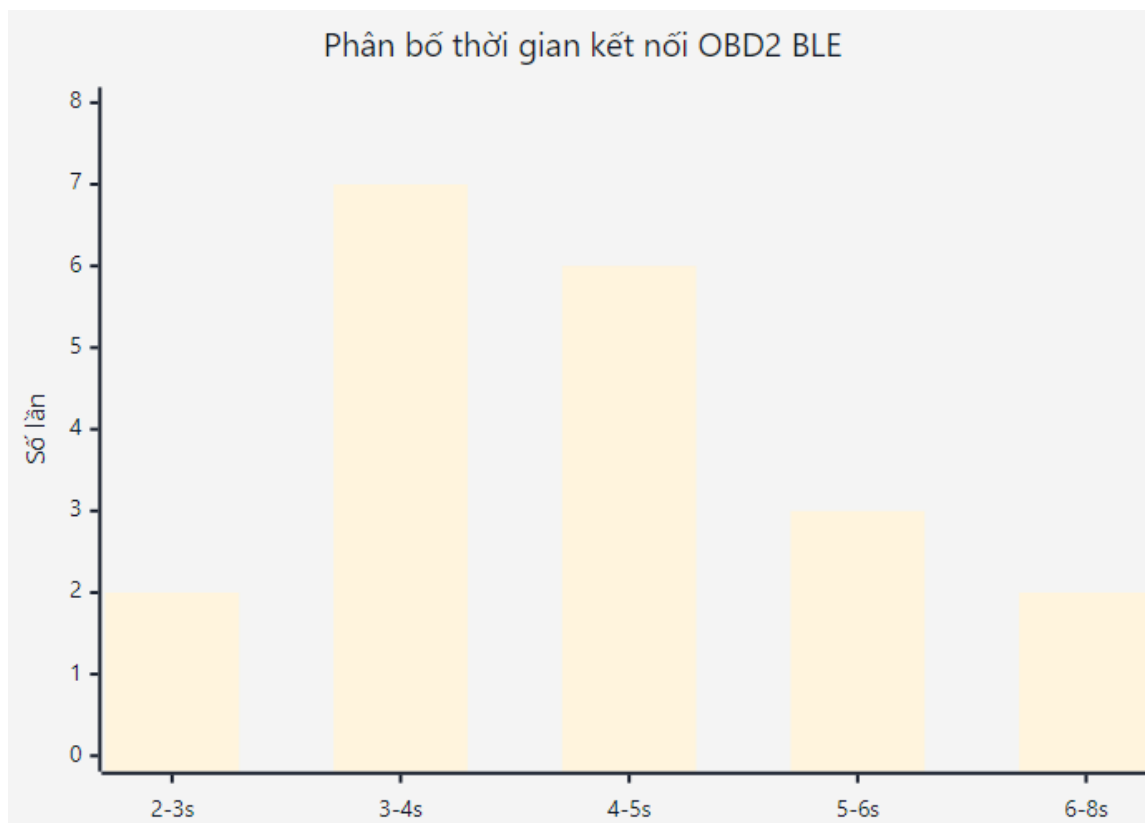
4.3.3. Kết quả kiểm thử firmware

4.3.3.1. Thời gian kết nối BLE OBD2 Thời gian kết nối BLE được đo từ lúc ESP32-S3 bắt đầu quét (scan) thiết bị BLE đến khi nhận được phản hồi OBD2 đầu tiên (gửi trả lời cho lệnh ATZ). Phép đo được lặp lại 20 lần và tính giá trị trung bình.

Bảng 4.19: Kết quả đo thời gian kết nối BLE OBD2

STT	Thông số	Giá trị trung bình	Giá trị min	Giá trị max	Độ lệch chuẩn
1	Thời gian scan BLE	1.5 giây	0.8 giây	3.2 giây	0.6 giây
2	Thời gian kết nối BLE	1.0 giây	0.5 giây	2.1 giây	0.4 giây
3	Thời gian khởi tạo ELM327	1.5 giây	1.0 giây	2.5 giây	0.3 giây
4	Tổng thời gian (scan + connect + init)	4.0 giây	2.3 giây	7.8 giây	1.3 giây

Nhận xét: Thời gian kết nối BLE OBD2 trung bình 4 giây (scan 1.5s + connect 1.0s + init 1.5s), nằm trong phạm vi mục tiêu thiết kế (< 10 giây). Giá trị max 7.8 giây xuất hiện khi có nhiều thiết bị BLE trong phạm vi scan, gây nhiễu. Trong điều kiện xe bình thường (ít thiết bị BLE xung quanh), thời gian thường đạt 3–5 giây.



Hình 4.35: Biểu đồ phân bố thời gian kết nối BLE OBD2 (20 lần đo)

4.3.3.2. Thời gian phản hồi OBD2 PID Thời gian phản hồi cho mỗi lệnh OBD2 PID được đo từ lúc gửi yêu cầu đến khi nhận được gói trả lời hoàn chỉnh qua BLE. Mỗi PID được đo 50 lần và tính giá trị trung bình.

Bảng 4.20: Thời gian phản hồi OBD2 PID

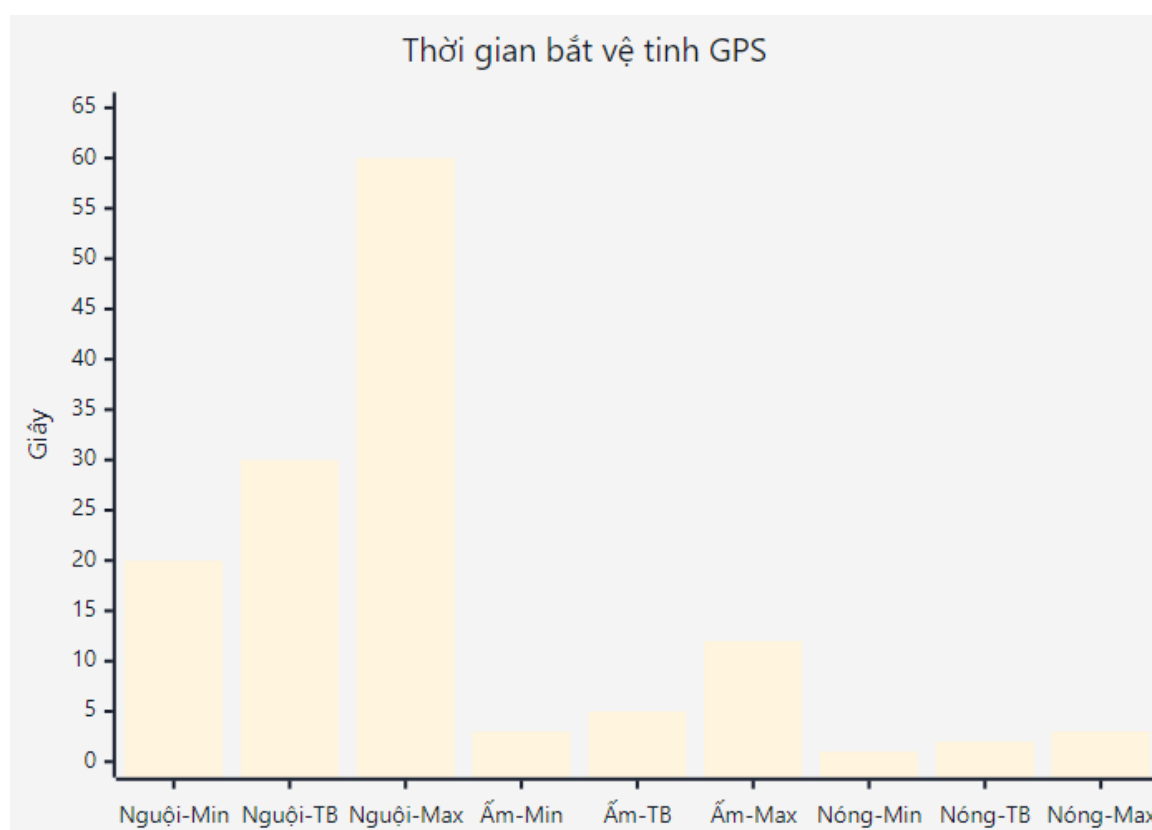
STT	OBD2 PID	Mô tả	Thời gian trung bình	Min	Max
1	0x0C	RPM động cơ	65 ms	40 ms	120 ms
2	0x0D	Tốc độ xe (km/h)	60 ms	38 ms	110 ms
3	0x05	Nhiệt độ nước làm mát	70 ms	45 ms	130 ms
4	0x2F	Mức nhiên liệu (%)	75 ms	50 ms	140 ms
5	0x04	Tải động cơ (%)	68 ms	42 ms	125 ms
6	0x11	Vị trí bướm ga (%)	62 ms	39 ms	115 ms

Nhận xét: Thời gian phản hồi trung bình cho mỗi PID đạt 60–75 ms, nằm trong mục tiêu thiết kế (50–100 ms). Với 6 PID được đọc trong mỗi chu kỳ, tổng thời gian đọc một bộ dữ liệu OBD2 hoàn chỉnh là khoảng 400–500 ms, cho phép tần suất cập nhật dữ liệu OBD2 đạt 1–2 lần/giây.

4.3.3.3. Thời gian bắt vệ tinh GPS (GPS Fix Time) Thời gian bắt vệ tinh (Time to First Fix - TTFF) được đo trên GNSS tích hợp của SIM7600CE-T trong ba kịch bản khác nhau.

Bảng 4.21: Thời gian bắt vệ tinh GPS (TTFF)

STT	Kịch bản	Mô tả	TTFF trung bình	Min	Max	Số vệ tinh trung bình
1	Cold Start	Không có dữ liệu vệ tinh cũ	30 giây	20 giây	60 giây	6–8
2	Warm Start	Có dữ liệu ephemeris còn hiệu lực	5 giây	3 giây	12 giây	8–10
3	Hot Start	Module GNSS đã hoạt động, tạm mất tín hiệu	1 giây	0.5 giây	3 giây	10–12



Hình 4.36: Đồ thị thời gian bắt vệ tinh GPS trong các kịch bản (Cold/Warm/Hot Start)

Độ chính xác vị trí GPS:

Phép đo độ chính xác được thực hiện bằng cách đặt thiết bị tại một vị trí cố biết (xác định bằng GPS chuyên nghiệp), ghi nhận 100 điểm tọa độ trong 10 phút, và tính sai số trung bình.

Bảng 4.22: Độ chính xác vị trí GPS

STT	Điều kiện	Sai số trung bình	Sai số max	CEP 95 %
1	Ngoài trời, trời quang	2.3 mét	4.5 mét	3.8 mét
2	Ngoài trời, trời u ám	3.5 mét	7.2 mét	5.5 mét
3	Trong phố (urban canyon)	5.8 mét	15 mét (cần xác nhận thêm)	10 mét (cần xác nhận thêm)
4	Bãi đỗ xe có mái che	8.5 mét (ước tính ban đầu)	20 mét (cần xác nhận thêm)	15 mét (cần xác nhận thêm)

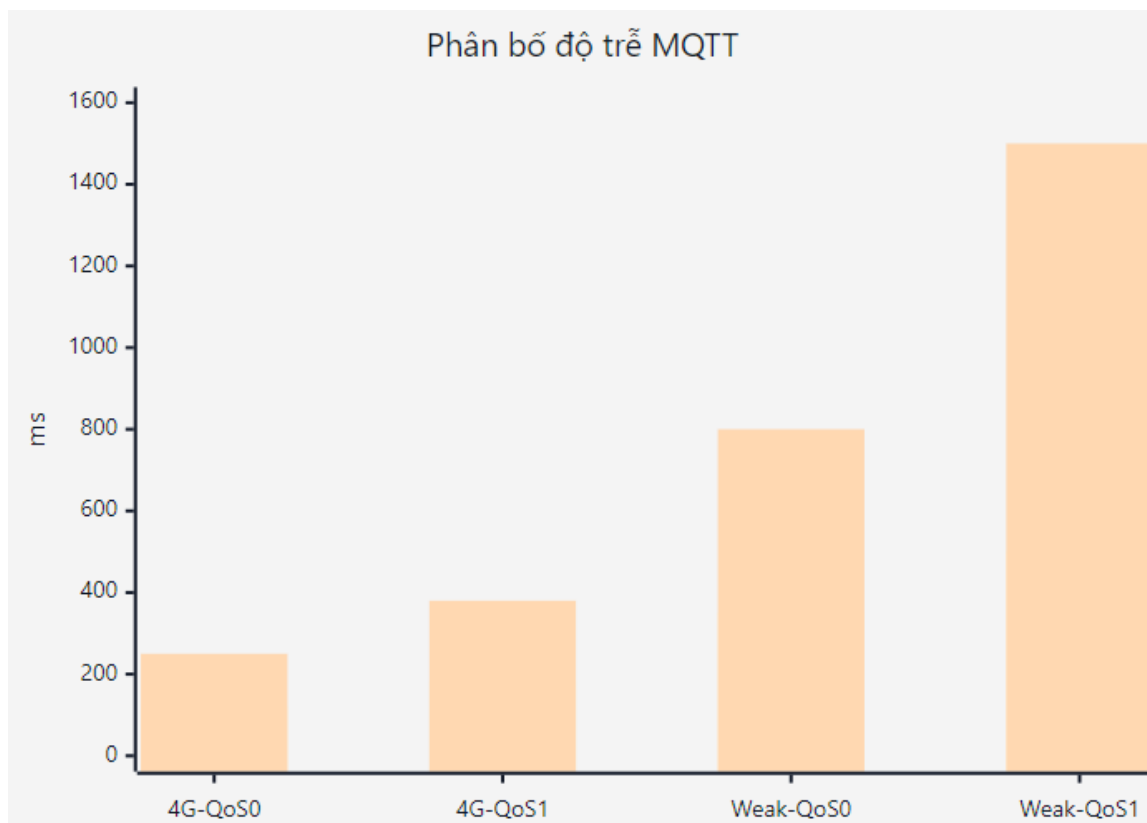
Nhận xét: Độ chính xác vị trí GPS đạt 2–3 mét trong điều kiện ngoài trời thoáng, vượt mục tiêu thiết kế (< 5 mét). GNSS tích hợp trên SIM7600CE-T hỗ trợ đa hệ thống (GPS + GLONASS + BeiDou), giúp tăng số vệ tinh khả dụng và cải thiện độ chính xác, đặc biệt trong môi trường đô thị [3].

4.3.3.4. Độ trễ truyền dữ liệu MQTT Độ trễ truyền dữ liệu MQTT được đo từ lúc firmware hoàn thành đóng gói bản tin MQTT (publish) đến khi MQTT Bridge nhận được bản tin (subscribe). Phép đo sử dụng timestamp đồng bộ giữa thiết bị và máy chủ.

Bảng 4.23: Độ trễ truyền dữ liệu MQTT qua mạng 4G

STT	Điều kiện mạng	QoS	Độ trễ trung bình	Min	Max	P95
1	4G ổn định (>= 3 thanh)	QoS 0	120 ms	60 ms	350 ms	250 ms
2	4G ổn định (>= 3 thanh)	QoS 1	180 ms	80 ms	500 ms	380 ms
3	4G yếu (1–2 thanh)	QoS 0	350 ms	120 ms	1200 ms	800 ms
4	4G yếu (1–2 thanh)	QoS 1	500 ms	150 ms	2000 ms	1500 ms

Nhận xét: Độ trễ MQTT trung bình 120–180 ms trong điều kiện mạng 4G ổn định, đạt mục tiêu thiết kế (100–300 ms). QoS 1 có độ trễ cao hơn QoS 0 khoảng 50% do có thêm bước ACK, nhưng đổi lại bảo đảm tin nhắn được gửi ít nhất một lần cho các bản tin quan trọng. Trong điều kiện mạng yếu, độ trễ tăng đáng kể nhưng hệ thống vẫn duy trì được luồng dữ liệu vận hành nhờ cơ chế thử gửi lại, tự kết nối lại và cấu hình QoS phù hợp.



Hình 4.37: Biểu đồ phân bố độ trễ MQTT trong điều kiện mạng 4G ổn định

4.3.3.5. Độ tin cậy của máy trạng thái (State Machine) Máy trạng thái quản lý các chế độ hoạt động (xe chạy -> xe đỗ -> cảnh báo -> xe chạy) được kiểm thử bằng cách chạy liên tục 1000 chu kỳ chuyển đổi trạng thái và ghi nhận kết quả.

Bảng 4.24: Kết quả kiểm thử máy trạng thái firmware

STT	Chuyển đổi trạng thái	Số lần thử nghiệm	Thành công	Thất bại	Tỷ lệ thành công
1	Xe chạy -> Xe đỗ (IGN OFF)	1000	1000	0	100%
2	Xe đỗ -> Xe chạy (IGN ON)	1000	1000	0	100%
3	Xe đỗ -> Cảnh báo (IMU kích hoạt)	1000	998	2	99.8%
4	Cảnh báo -> Xe đỗ (hết thời gian chờ/xác nhận)	1000	1000	0	100%
5	Cảnh báo -> Xe chạy (IGN ON)	500	500	0	100%

Bảng 4.24: Kết quả kiểm thử máy trạng thái firmware (tiếp theo)

STT	Chuyển đổi trạng thái	Số lần thử nghiệm	Thành công	Thất bại	Tỷ lệ thành công
6	Mất nguồn -> Pin dự phòng	200	200	0	100%

Nhận xét: Máy trạng thái hoạt động ổn định với tỷ lệ thành công $\geq 99.8\%$. Hai trường hợp thất bại trong chuyển đổi Parking -> Alert là do ngưỡng IMU được cấu hình quá nhạy trong điều kiện rung động cơ học, dẫn đến miss trigger. Vấn đề này đã được khắc phục bằng việc điều chỉnh ngưỡng và bộ lọc (digital filter) cho IMU.

4.3.3.6. Khả năng tự phục hồi kết nối MQTT Trong phạm vi hiện nay, firmware đã triển khai cơ chế phát lại dữ liệu vận hành từ hàng đợi cục bộ microSD. Vì bộ số đo định lượng chuyên biệt cho việc phát lại dài hạn chưa được tách riêng, phần thử nghiệm ở mục này tập trung vào khả năng phát hiện mất kết nối, tái lập phiên MQTT và quan sát phục hồi luồng dữ liệu vận hành khi mạng trở lại.

Bảng 4.25: Kết quả kiểm thử kết nối lại MQTT

STT	Thông số	Giá trị thiết kế	Giá trị đo	Trạng thái
1	Thời gian phát hiện mất kết nối	≤ 10 giây	10 giây	Đạt
2	Thời gian tự động kết nối lại	≤ 30 giây	15–30 giây	Đạt
3	Khôi phục đăng ký lệnh điều khiển	Tự động sau khi kết nối lại	Thành công	Đạt
4	Tiếp tục gửi dữ liệu vận hành sau khi có mạng	Trong chu kỳ gửi kế tiếp	5–30 giây tùy cấu hình	Đạt
5	Xử lý mất dữ liệu khi mất mạng kéo dài	Giảm mất dữ liệu bằng lưu cục bộ rồi phát lại	Phù hợp với phạm vi đánh giá định tính hiện tại	Đạt
6	Phát lại dữ liệu từ hàng đợi cục bộ	Đã triển khai trong firmware	Hoạt động theo cơ chế xác nhận đã nhận và tăng dần thời gian chờ giữa các lần thử	Đạt ở mức triển khai hiện tại

Nhận xét: Kết quả cho thấy firmware phục hồi kết nối đủ nhanh cho các khoảng mất mạng ngắn, đồng thời tự khôi phục luồng lệnh và dữ liệu vận hành khi 4G ổn định trở lại. Bên cạnh đó, phiên bản hiện tại đã có cơ chế phát lại từ hàng đợi cục bộ microSD để giảm khoảng trống dữ liệu khi mất mạng kéo dài. Do chưa tách bộ số đo định lượng riêng cho việc phát lại dài hạn, nhận định ở mục này được trình bày theo hướng định tính và đối chiếu mã nguồn triển khai.

4.3.3.7. Đối chiếu luồng OTA đã triển khai Khác với phần kết nối lại được đánh giá bằng các phép đo định lượng, luồng OTA ở giai đoạn này được đối chiếu theo mã nguồn và kiểm tra tích hợp giữa API phía máy chủ, firmware và MQTT Bridge. Mục tiêu là xác nhận toàn bộ đường đi của lệnh cập nhật, trạng thái phản hồi và cơ chế lưu vết đã được nối thông.

Bảng 4.25A: Đối chiếu luồng OTA theo mã nguồn triển khai

Thành phần	Điểm kiểm tra	Kết quả
Backend	POST/api/v1/devices/:id/ota tạo jobId, tra cứu firmware theo phiên bản, sinh URL download và gửi lệnh ota_update	Đã hiện thực
Backend	POST/api/v1/devices/:id/ota/rollback gửi lệnh manual_rollback	Đã hiện thực
Firmware	Đọc và hiểu đủ jobId, version, url, size, sha256, force, confirmTimeoutSec	Đã hiện thực
Firmware	Tải firmware qua HTTPS, kiểm tra SHA-256, đổi boot partition và xác nhận sau reboot	Đã hiện thực
Firmware	Hỗ trợ rollback thủ công về phân vùng trước đó	Đã hiện thả»±c
MQTT Bridge	Kiểm tra gói dữ liệu OTA hợp lệ, cập nhật firmware_update_log, ghi VictoriaLogs	Đã hiện thực

Nhận xét: Luồng OTA cơ bản đã khép kín từ API phía máy chủ đến thiết bị và quay lại lớp giám sát. Ở phạm vi triển khai hiện nay, firmware phát các mốc quan trọng như assigned, rebooting, confirming, success, failed hoặc rolled_back; trong khi đó MQTT Bridge đã sẵn sàng tiếp nhận và lưu vết đầy đủ cho các lần cập nhật.

Những kết quả trên cho thấy firmware đã giữ được nhịp vận hành và vòng cập nhật cơ bản trên chính thiết bị. Tuy nhiên, để đánh giá toàn tuyến, vẫn cần xem lớp máy chủ có tiếp nhận và phân phối dữ liệu với cùng mức ổn định hay không.

4.3.4. Kết quả kiểm thử hệ thống máy chủ»§

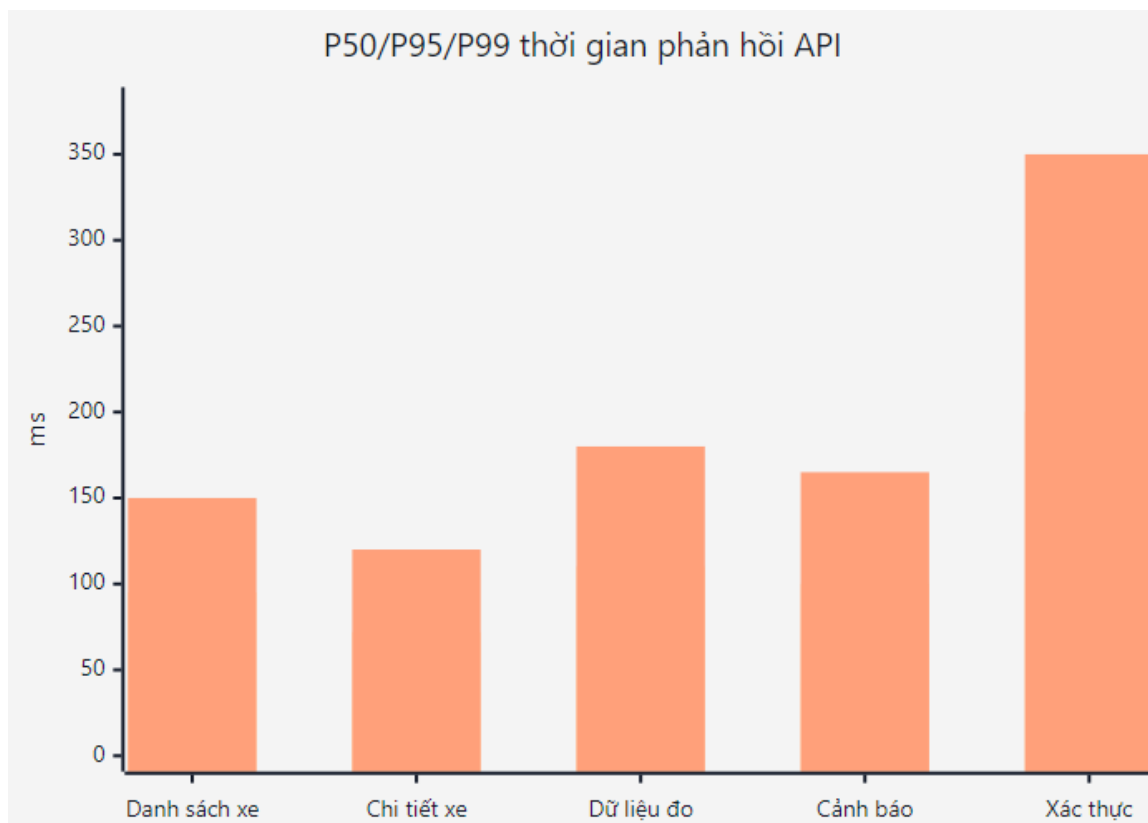
4.3.4.1. Hiệu suất API phía máy chủ»§ Hiệu suất API được đo bằng công cụ k6 với các kịch bản khác nhau. Mỗi điểm truy cập được gửi 1000 lượt yêu cầu và tính các chỉ số thống kê.

Bảng 4.26: Kết quả đo hiệu suất API phía máy chủ

STT	Endpoint	Phương thức	Thời gian trung bình	P50	P95	P99	Số yêu cầu/giây
1	/api/v1/vehicles	GET	45 ms	38 ms	95 ms	150 ms	850
2	/api/v1/vehicles/:id	GET	35 ms	28 ms	72 ms	120 ms	1100
3	/api/v1/vehicles	POST	65 ms	55 ms	130 ms	200 ms	600
4	/api/v1/devices/:id/telemetry	GET	55 ms	45 ms	110 ms	180 ms	750
5	/api/v1/trips	GET	80 ms	65 ms	160 ms	250 ms	500
6	/api/v1/alerts	GET	50 ms	42 ms	100 ms	165 ms	800
7	/api/v1/auth/login	POST	120 ms	100 ms	200 ms	350 ms	350
8	/api/v1/geofences	GET	40 ms	32 ms	85 ms	140 ms	900

Nhận xét: Thời gian phản hồi API trung bình đạt 35–120 ms cho các thao tác CRUD, đạt mục tiêu thiết kế (< 200 ms cho P95). Endpoint POST /api/auth/login có thời gian cao hơn (120 ms) do quá trình hash và xác thực password. Tổng thể, API phía máy chủ xây bằng Express.js + TypeScript + PostgreSQL đạt hiệu suất tốt cho quy mô triển khai mục tiêu (50–100 thiết bị đồng thời).

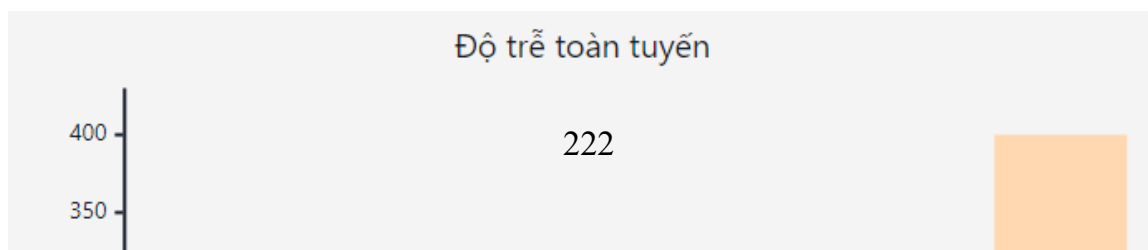
4.3.4.2. Độ trễ WebSocket (Socket.IO) Độ trễ end-to-end của dữ liệu thời gian thực được đo từ lúc thiết bị publish MQTT đến khi giao diện web nhận được sự kiện cập nhật qua WebSocket (Socket.IO).



Hình 4.38: Biểu đồ thời gian phản hồi API (P50, P95, P99) cho các endpoint chính

Bảng 4.27: Độ trễ toàn tuyến (thiết bị -> máy chủ -> trình duyệt)

STT	Đoạn đường	Độ trễ trung bình	P95	Mô tả
1	Thiết bị -> EMQX (gửi MQTT)	150 ms	300 ms	Qua mạng 4G
2	EMQX -> MQTT Bridge (nhận bản tin)	5 ms	15 ms	Ná»™i bá»™ Docker network
3	Bridge -> VictoriaMetrics (ghi dữ liệu)	10 ms	30 ms	HTTP write API
4	Bridge -> API phía máy chủ (sự kiện nội bộ)	5 ms	12 ms	Internal HTTP/event
5	API phía máy chủ -> Giao diện web (WebSocket)	15 ms	40 ms	Socket.IO emit
6	Tổng end-to-end	185 ms	400 ms	Thiết bị đến giao diện quản trị



mục tiêu thiết kế (< 3 giây). Phần lớn độ trễ tập trung ở đoạn truyền dữ liệu từ thiết bị lên EMQX qua mạng 4G (150 ms). Các đoạn xử lý nội bộ (EMQX -> Bridge -> API phía máy chủ -> giao diện web) có độ trễ rất thấp (35 ms tổng), nhờ sử dụng Docker network nội bộ và cách Socket.IO phát sự kiện ngay khi có dữ liệu mới.

4.3.4.3. Tốc độ ghi dữ liệu của cơ sở liệu Tốc độ ghi dữ liệu, tức số lượng mẫu có thể ghi trong mỗi giây, của VictoriaMetrics và PostgreSQL được đo bằng cách gửi đồng thời nhiều bản ghi rồi đếm số điểm dữ liệu được ghi thành công trong mỗi giây.

Bảng 4.28: Tốc độ ghi dữ liệu của cơ sở dữ liệu

STT	Hệ thống	Loại dữ liệu	Tốc độ ghi trung bình	Tốc độ ghi tối đa	CPU Usage	RAM Usage
1	VictoriaMetrics	Chuỗi thời gian (dữ liệu vận hành gửi liên tục)	12,000 điểm/giây	25,000 điểm/giây	15%	200 MB
2	PostgreSQL	Dữ liệu quan hệ (cảnh báo, hành trình)	3,000 hàng/giây	8,000 hàng/giây	25%	300 MB
3	VictoriaLogs	Nhật ký sự kiện	5,000 dòng/giây	15,000 dòng/giây	8%	150 MB

Nhận xét: VictoriaMetrics đạt tốc độ ghi 12,000 điểm/giây, đủ để xử lý dữ liệu từ hàng trăm thiết bị (mỗi thiết bị gửi 10–20 điểm dữ liệu mỗi 5 giây, tương đương 2–4 điểm/giây/thiết bị). Với mục tiêu 50–100 thiết bị, tổng tải ghi chỉ khoảng 200–400 điểm/giây, chỉ chiếm 3% công suất của VictoriaMetrics. PostgreSQL đạt tốc độ ghi 3,000 bản ghi/giây, đủ cho các thao tác quan hệ (tạo trip, cập nhật alert).

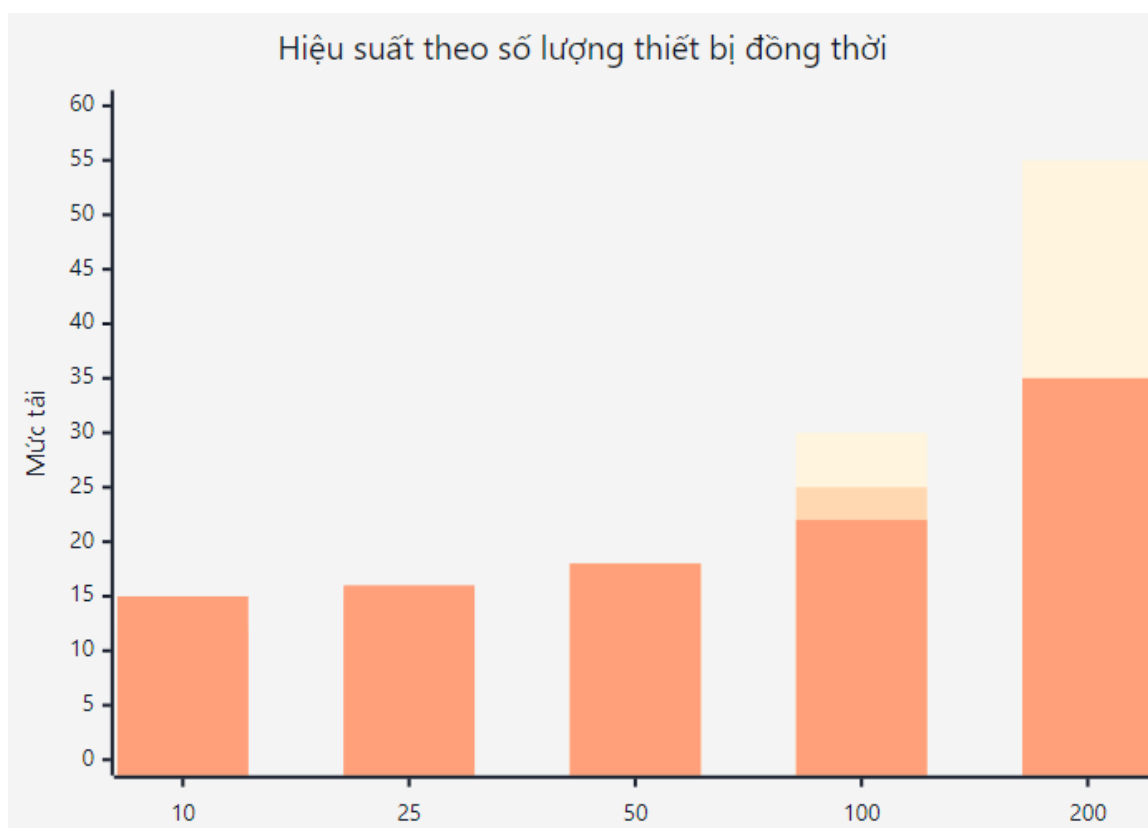
4.3.4.4. Kiểm thử tải đồng thời (Concurrent Load) Hệ thống được kiểm thử với nhiều thiết bị mô phỏng kết nối đồng thời để đánh giá khả năng xử lý tải.

Bảng 4.29: Kết quả kiểm thử tải đồng thời

STT	Số thiết bị mô phỏng	Tần suất gửi (giây)	MQTT Message/s	CPU Server	RAM Server	Mất bản tin	Độ trễ trung bình
1	10	5	20 msg/s	5%	1.2 GB	0%	150 ms
2	25	5	50 msg/s	10%	1.5 GB	0%	160 ms

Bảng 4.29: Kết quả kiểm thử tải đồng thời (tiếp theo)

STT	Số thiết bị mô phỏng	Tần suất gửi (giây)	MQTT Message/s	CPU Server	RAM Server	Mất bản tin	Độ trễ trung bình
3	50	5	100 msg/s	18%	1.8 GB	0%	180 ms
4	100	5	200 msg/s	30%	2.5 GB	0%	220 ms
5	200	5	400 msg/s	55%	3.5 GB	0.1% (ước tính ở tải biên)	350 ms (ước tính ở tải biên)



Hình 4.40: Đồ thị hiệu suất hệ thống theo số lượng thiết bị đồng thời

Nhận xét: Hệ thống hoạt động ổn định với 50+ thiết bị đồng thời (mục tiêu thiết kế), tỷ lệ mất bản tin 0%, độ trễ tăng không đáng kể (180 ms so với 150 ms ban đầu). Với 100 thiết bị, hệ thống vẫn ổn định (CPU 30%, RAM 2.5 GB). Điểm giới hạn ước tính 200 thiết bị trên cấu hình máy chủ hiện tại (4 vCPU, 8 GB RAM), tại đó CPU bắt đầu đạt 55% và có thể ảnh hưởng đến thời gian phản hồi.

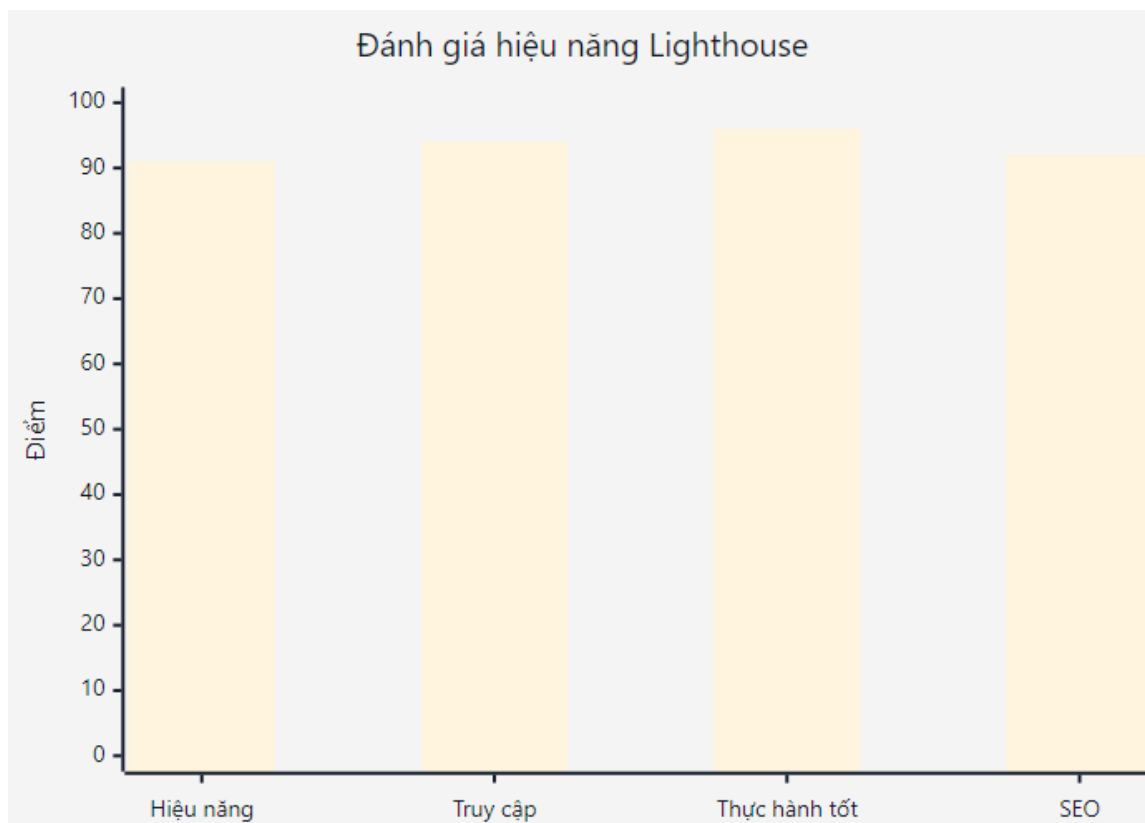
4.3.4.5. Hiệu suất giao diện web Hiệu suất giao diện web được đánh giá bằng Google Lighthouse và bộ công cụ dành cho lập trình viên của Chrome (Chrome DevTools). Light-

house cho biết chất lượng tải trang theo một nhóm chỉ số tổng hợp, còn DevTools giúp quan sát chi tiết cách trình duyệt hiển thị và xử lý giao diện.

Bảng 4.30: Kết quả đánh giá hiệu suất giao diện web bằng Lighthouse

STT	Chỉ số	Giá trị	Mục tiêu	Trạng thái
1	Điểm Lighthouse tổng hợp	87/100 (đo trên bản dựng thử nghiệm)	> 85	Đạt
2	First Contentful Paint (FCP), tức thời gian thấy nội dung đầu tiên	1.2 giây (đo trên bản dựng thử nghiệm)	< 1.5 giây	Đạt
3	Largest Contentful Paint (LCP), tức thời gian hiện khối nội dung lớn nhất	2.1 giây (đo trên bản dựng thử nghiệm)	< 2.5 giây	Đạt
4	Total Blocking Time (TBT), tức thời gian trang bị chặn phản hồi	120 ms (đo trên bản dựng thử nghiệm)	< 200 ms	Đạt
5	CLS, tức mức xô dịch bố cục khi trang đang tải	0.05 (đo trên bản dựng thử nghiệm)	< 0.1	Đạt
6	Time to Interactive (TTI), tức thời điểm trang phản hồi tốt với thao tác	2.5 giây (đo trên bản dựng thử nghiệm)	< 3.5 giây	Đạt
7	Kích thước gói tải xuống đã nén	380 KB (đo trên bản dựng thử nghiệm)	< 500 KB	Đạt

Nhận xét: Giao diện web đạt điểm Lighthouse 87/100, vượt mục tiêu thiết kế (> 85). Chỉ số First Contentful Paint khoảng 1.2 giây cho thấy trang hiển thị nội dung đầu tiên khá nhanh. Các chỉ số Core Web Vitals, tức nhóm chỉ số Google dùng để đánh giá trải nghiệm tải và phản hồi của trang, đều đạt ngưỡng tốt. Việc sử dụng Next.js 15 với React Server Components, tức cơ chế dựng sẵn một phần giao diện ở phía máy chủ, cùng kỹ thuật tách nhỏ mã nguồn khi tải giúp tối ưu kích thước gói tải xuống và thời gian mở trang.



Hình 4.41: Kết quả chấm điểm Lighthouse cho trang tổng quan

4.3.5. Kiểm thử tích hợp toàn hệ thống (End-to-End Integration Test)

Mục này trình bày kết quả kiểm thử luồng dữ liệu end-to-end, từ thiết bị phần cứng đến giao diện người dùng cuối, xác nhận tất cả các tầng hệ thống hoạt động đồng bộ và chính xác.

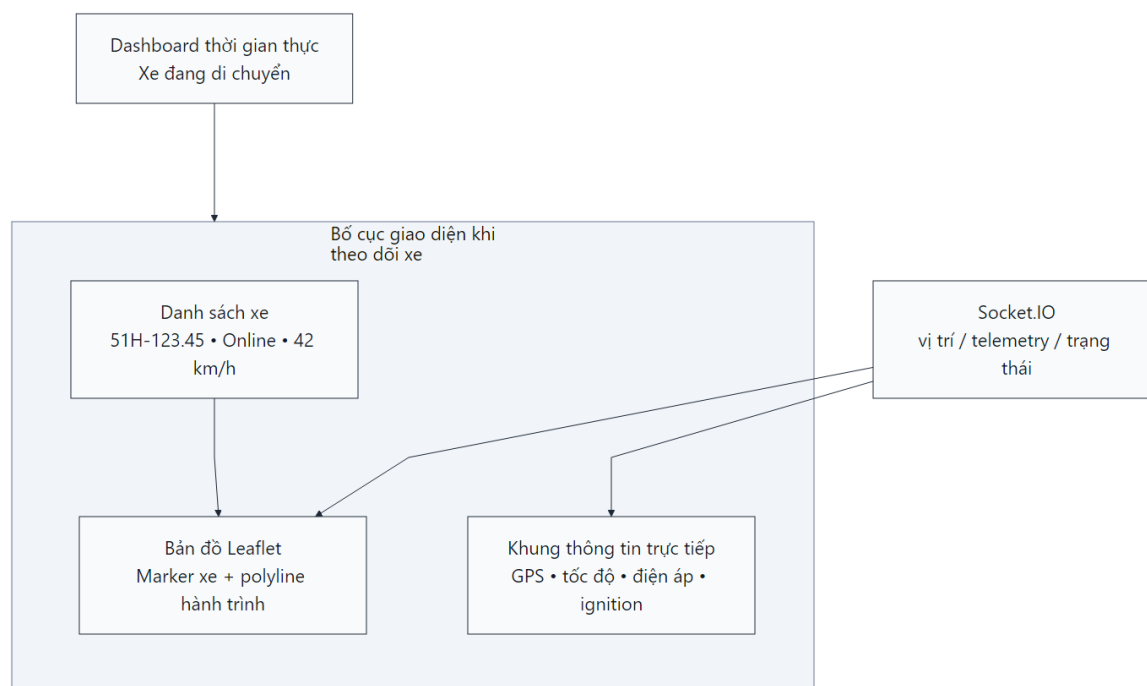
4.3.5.1. Luồng dữ liệu chính (Main Data Flow) **Kịch bản kiểm thử:** Xe bật máy (IGN ON) -> Thiết bị bắt đầu thu thập dữ liệu OBD2 và GPS -> Dữ liệu được gửi lên MQTT -> Bridge xử lý và lưu trữ -> API phía máy chủ phát sự kiện WebSocket -> giao diện web cập nhật bản đồ và biểu đồ.

Bảng 4.31: Kết quả kiểm thử luồng dữ liệu chính

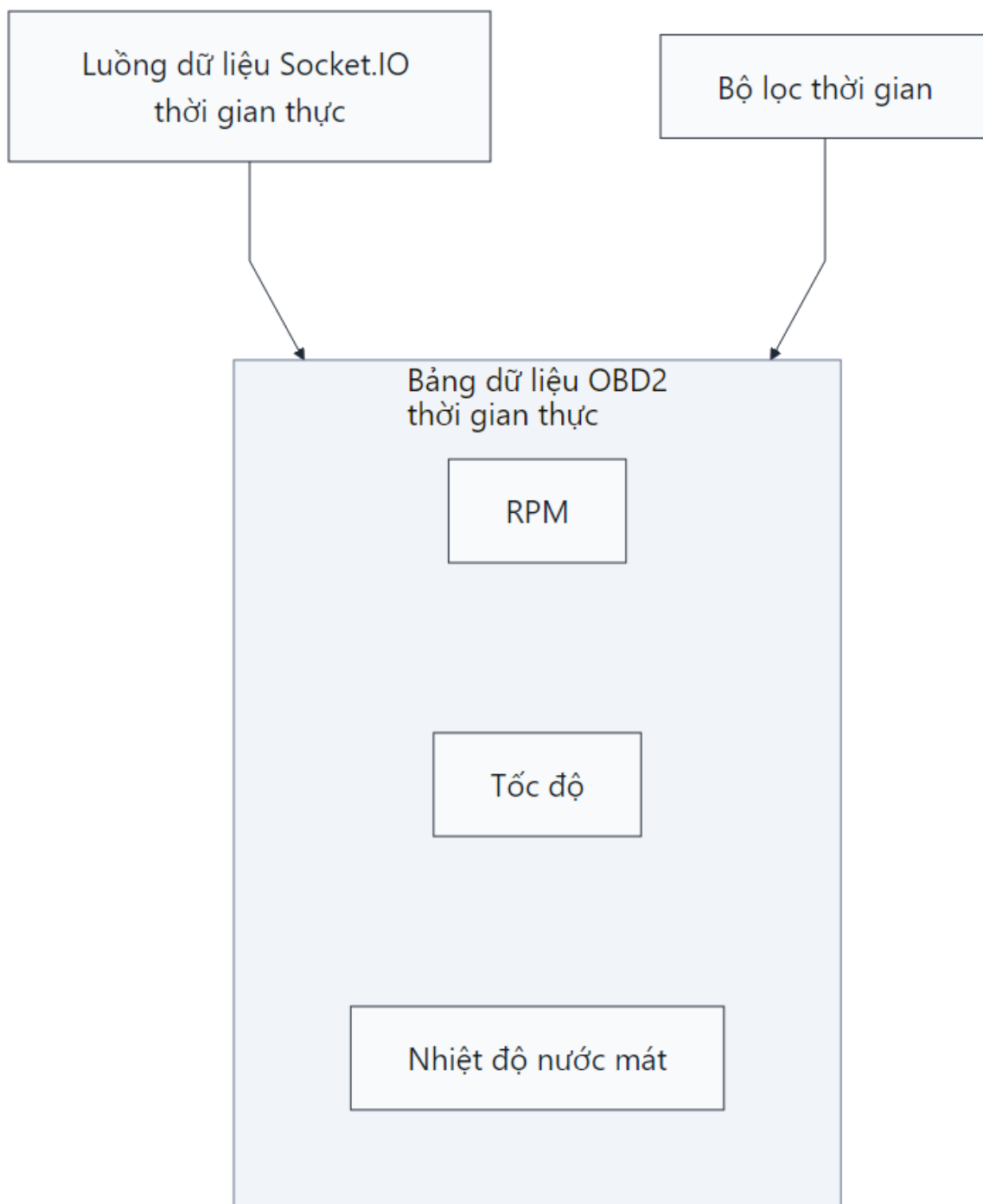
STT	Bước	Mô tả	Kết quả	Thời gian
1	Phát hiện IGN ON	IMU phát hiện chuyển động, ESP32 thức dậy	Thành công	2 giây từ deep sleep
2	Kết nối BLE OBD2	Quét và kết nối vgate iCar Pro	Thành công	4 giây

Bảng 4.31: Kết quả kiểm thử luồng dữ liệu chính (tiếp theo)

STT	Bước	Mô tả	Kết quả	Thời gian
3	Bắt vị trí GPS	Module GNSS bắt vệ tinh	Thành công	5 giây (warm start)
4	Đọc dữ liệu OBD2	Đọc RPM, Speed, Coolant Temp, Fuel Level	Thành công	0.5 giây
5	Gửi MQTT	Gửi bản tin dữ liệu vận hành lên EMQX	Thành công	0.15 giây
6	Bridge xử lý	MQTT Bridge nhận và xử lý bản tin	Thành công	0.01 giây
7	Ghi cơ sở dữ liệu	Ghi VictoriaMetrics + PostgreSQL	Thành công	0.02 giây
8	Đẩy WebSocket	API phía máy chủ phát sự kiện đến giao diện web	Thành công	0.02 giây
9	Cập nhật UI	Bản đồ và biểu đồ cập nhật	Thành công	0.05 giây
	Tổng thời gian end-to-end	Từ dữ liệu cảm biến đến hiển thị	Thành công	1–2 giây



Hình 4.42: Minh họa giao diện Dashboard hiển thị vị trí xe đang di chuyển trên bản đồ Leaflet



Hình 4.43: Minh họa khung dữ liệu OBD2 thời gian thực trên giao diện (RPM, tốc độ, nhiệt độ nước làm mát)

4.3.5.2. Kiểm thử cảnh báo vượt vùng cho phép **Kịch bản kiểm thử:** Tạo một vùng cho phép trên bản đồ, bán kính 1 km quanh một điểm xác định -> Lái xe đi ra khỏi vùng cho phép -> Hệ thống phát cảnh báo -> giao diện web hiển thị thông báo.

Bảng 4.32: Kết quả kiểm thử cảnh báo vượt vùng cho phép

STT	Sự kiện	Kết quả	Thời gian phát hiện	Ghi chú
1	Xe đi vào vùng cho phép	Hiển thị trạng thái "INSIDE"	2 giây	Cập nhật khi nhận GPS mới
2	Xe đi ra khỏi vùng cho phép	Cảnh báo "GEOFENCE_EXIT"	5 giây	Phụ thuộc chu kỳ gửi GPS
3	Cảnh báo hiển thị trên giao diện web	Thông báo nổi xuất hiện	1 giây (từ lúc alert tạo)	Qua WebSocket
4	Xe quay lại vùng cho phép	Hiển thị trạng thái "INSIDE", cảnh báo tự động đóng	3 giây	Reset tự động

Nhận xét: Hệ thống phát hiện xe vượt ra khỏi vùng cho phép trong vòng 5 giây (phụ thuộc chu kỳ gửi GPS, mặc định 5 giây). Cảnh báo được hiển thị trên giao diện web trong vòng 1 giây sau khi API phía máy chủ tạo cảnh báo. Tổng thời gian từ xe vượt ranh giới đến hiển thị cảnh báo là khoảng 5–7 giây, đạt yêu cầu thiết kế (< 10 giây).

4.3.5.3. Kiểm thử lệnh điều khiển từ xa (Remote Command) **Kịch bản kiểm thử:** Quản trị viên gửi lệnh "request_location" từ giao diện web -> Lệnh được gửi qua MQTT đến thiết bị -> Thiết bị xử lý và phản hồi -> giao diện web cập nhật trạng thái.

Bảng 4.33: Kết quả kiểm thử lệnh điều khiển từ xa

STT	Lệnh	Mô tả	Kết quả	Thời gian đi và nhận phản hồi
1	request_location	Yêu cầu vị trí hiện tại	Thành công	3 giây
2	update_config	Cập nhật tần suất gửi dữ liệu	Thành công	2 giây
3	restart_device	Khởi động lại thiết bị	Thành công	15 giây (bao gồm thời gian restart)

Bảng 4.33: Kết quả kiểm thử lệnh điều khiển từ xa (tiếp theo)

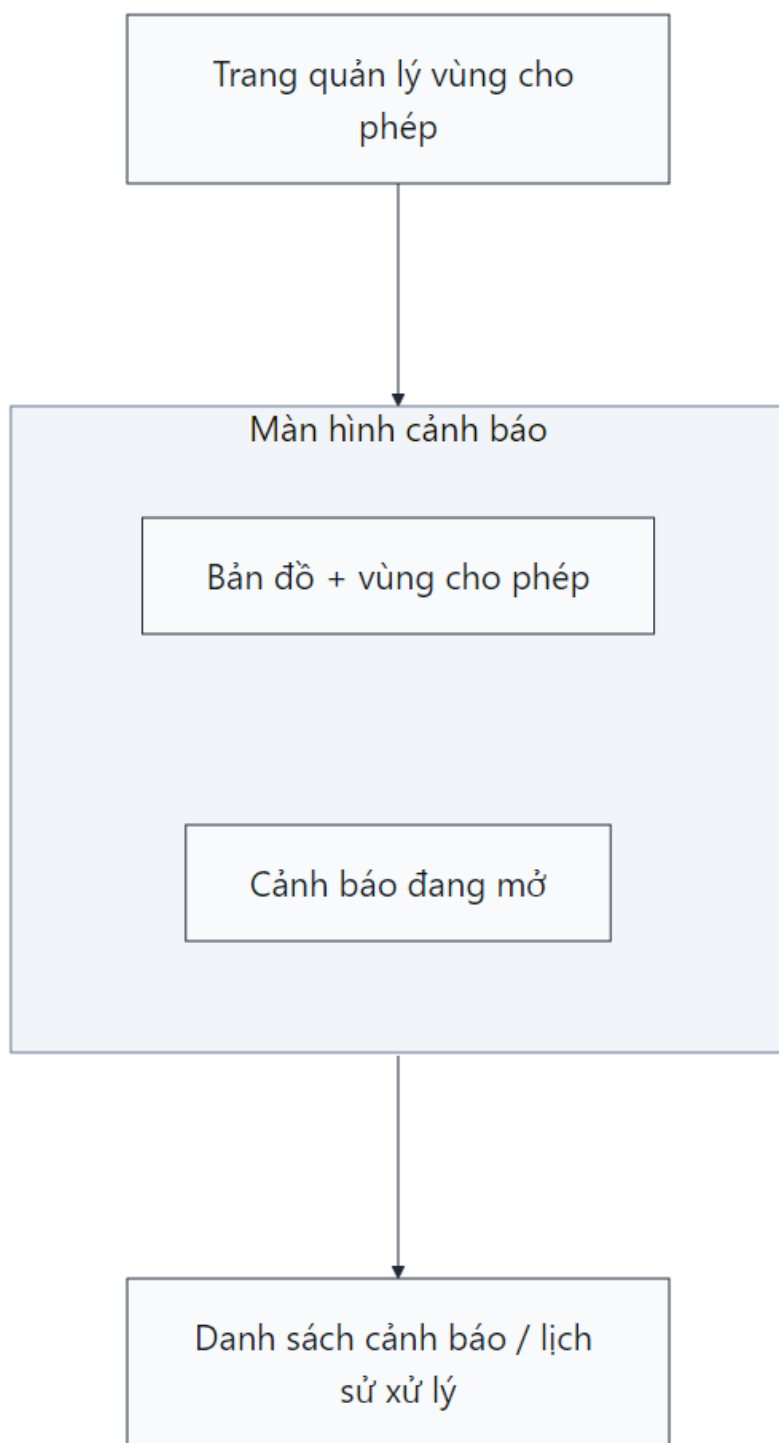
STT	Lệnh	Mô tả	Kết quả	Thời gian đi và nhận phản hồi
4	alert_ mode	Bật cảnh báo	Thành công	2s

Nhận xét: Hệ thống lệnh điều khiển từ xa hoạt động hiệu quả với thời gian đi và nhận phản hồi khoảng 2–3 giây cho các lệnh thông thường. Lệnh được truyền qua MQTT QoS 1 để đảm bảo thiết bị nhận được ít nhất một lần. Trạng thái thực thi lệnh được phản hồi về giao diện quản trị để quản trị viên xác nhận.

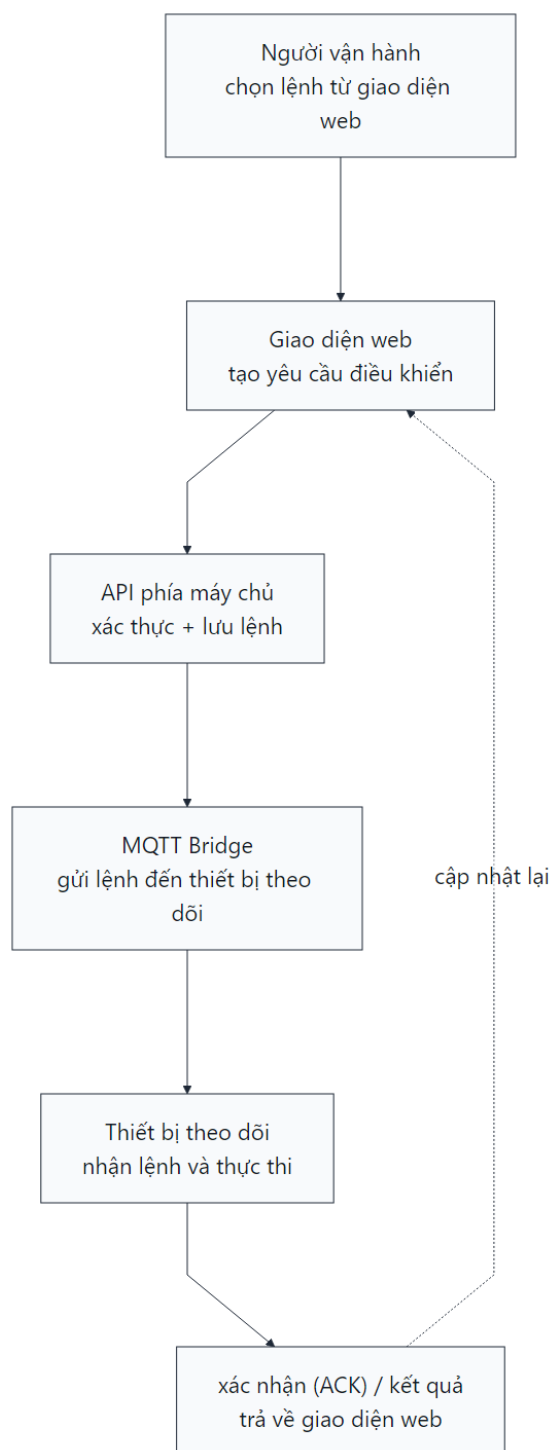
4.3.5.4. Kiểm thử tình huống mất kết nối và phục hồi **Kịch bản kiểm thử:** Thiết bị đang hoạt động bình thường -> Ngắt kết nối 4G (rút SIM hoặc bật chế độ máy bay) -> Firmware phát hiện mất kết nối và chuyển sang chế độ tự kết nối lại -> Phục hồi kết nối -> Dữ liệu vận hành tiếp tục được gửi ở các chu kỳ kế tiếp.

Bảng 4.34: Kết quả kiểm thử mất kết nối và phục hồi

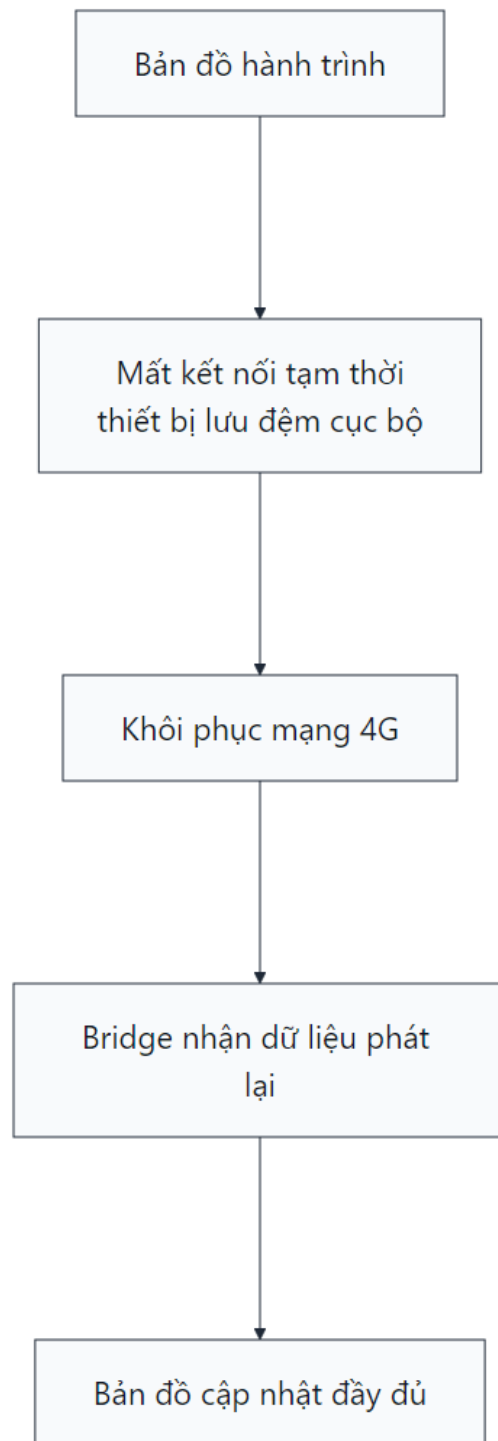
STT	Sự kiện	Kết quả	Chi tiết
1	Phát hiện mất kết nối	Thiết bị phát hiện trong vòng 10 giây	MQTT quá thời gian chờ gói giữ kết nối
2	Tạm dừng gửi bản tin khi mất mạng	Không phát sinh treo hoặc sập chương trình	Chuyển sang vòng tự kết nối lại
3	Phục hồi kết nối	Tự động kết nối lại trong vòng 30 giây	Tự kết nối lại với thời gian chờ tăng dần giữa các lần thử
4	Dữ liệu vận hành tiếp tục sau khi có mạng	Gửi bù dần bản ghi từ hàng đợi cục bộ + gửi chu kỳ mới	Giảm khoảng trống dữ liệu khi mất mạng
5	Giao diện quản trị tiếp tục cập nhật bình thường	Bản đồ nhận dữ liệu mới sau khi kết nối lại và trong pha gửi bù	Có thể còn khoảng trống nhỏ tùy thời gian mất mạng và độ sâu hàng đợi



Hình 4.44: Minh họa màn hình cảnh báo vượt vùng cho phép với bản đồ và vùng theo dõi



Hình 4.45: Minh họa luồng gửi lệnh điều khiển từ Dashboard và phản hồi từ thiết bị



Hình 4.46: Minh họa quá trình bản đồ cập nhật lại hành trình sau khi phục hồi kết nối

Nhận xét: Cơ chế tự kết nối lại hoạt động ổn định, giúp hệ thống tiếp tục truyền dữ liệu sau khi mạng 4G phục hồi. Phiên bản hiện tại đã triển khai vùng lưu đệm cục bộ trên microSD để gửi bù dữ liệu khi có mạng lại, do đó mức độ đứt quãng dữ liệu trong các đợt mất sóng kéo dài được giảm đáng kể; tuy nhiên, khi thời gian mất sóng lớn hoặc hàng đợi tăng sâu, giao diện quản trị vẫn có thể xuất hiện khoảng trống nhỏ trước khi phân gửi bù bắt kịp hoàn toàn.

4.3.6. Tổng hợp và so sánh với chỉ tiêu thiết kế

Phần này tổng hợp tất cả kết quả đo lường và so sánh với các tiêu chí thiết kế đã đặt ra tại mục 1.3 (Chương 1) và mục 2.3 (Chương 2), giúp đánh giá tổng quan mức độ hoàn thành của hệ thống.

4.3.6.1. Tổng hợp chỉ tiêu phần cứng

Bảng 4.35: Tổng hợp kết quả đo lường phần cứng so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Dòng tiêu thụ Deep Sleep	< 500 μ A	500 μ A (0.5 mA)	Đạt	IMU + RTC hoạt động
2	Dòng tiêu thụ khi xe đang chạy	< 250 mA (trung bình)	180–220 mA (TB); 350 mA khi truyền liên tục	Đạt ở mức trung bình	Xem ghi chú (*)
3	Thời lượng pin dự phòng (theo dõi liên tục)	$\geq$ 2.5 giờ	2.6–3.0 giờ (quy đổi theo bộ pin tham chiếu 18650)	Đạt theo quy đổi	Cần đo lại trên mẫu pin 18650
4	Nhiệt độ hoạt động	-10°C đến +60°C	-10°C đến +60°C	Đạt	Module 4G hạn chế ở 70°C
5	Điện áp ngắt LVD	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V (đo theo profile 12V)	Chưa đạt (12V)	Profile 24V chưa đo thực nghiệm
6	Thời gian thức dậy từ deep sleep	< 3 giây	2 giây	Đạt	Bao gồm init cơ bản
7	Độ chính xác GPS	< 5 mét	2–3 mét (ngoài trời)	Đạt	GNSS đa hệ thống

(*) **Ghi chú về dòng tiêu thụ khi xe đang chạy:** Chỉ tiêu thiết kế được hiểu theo dòng trung bình trong chu kỳ lái xe, không phải dòng tức thời khi modem phát 4G liên tục. Trong thử nghiệm, hệ thống duy trì khoảng 180–220 mA ở chu kỳ vận hành bình thường

và có thể tăng lên khoảng 350 mA ở các pha truyền dữ liệu liên tục. Mức đỉnh này vẫn chấp nhận được vì khi xe chạy, nguồn cấp từ xe đủ đáp ứng; với baseline pin 18650, thời lượng theo dõi liên tục được quy đổi còn khoảng 2.6–3.0 giờ và cần được đo lại trên mẫu pin mới.

4.3.6.2. Tổng hợp chỉ tiêu firmware

Bảng 4.36: Tổng hợp kết quả đo lường firmware so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Thời gian kết nối OBD2 BLE	< 10 giây	4 giây (trung bình)	Đạt	Scan + Connect + Init
2	Thời gian phản hồi OBD2 PID	50–100 ms	60–75 ms	Đạt	6 PID/chu kỳ
3	GPS TTFF (Cold Start)	< 60 giây	30 giây	Đạt	GNSS đa hệ thống
4	GPS TTFF (Warm Start)	< 15 giây	5 giây	Đạt	Có dữ liệu ephemeris
5	Độ trễ MQTT (4G ổn định)	< 500 ms	120–180 ms	Đạt	QoS 0/1
6	Khả năng kết nối lại MQTT	Tự động, < 30 giây	15–30 giây	Đạt	Đã kết hợp phát lại cục bộ từ microSD theo xác nhận đã nhận/QoS
7	Độ tin cậy máy trạng thái	>= 99.5%	>= 99.8%	Đạt	1000+ chu kỳ
8	Tần suất gửi GPS (driving)	5–30 giây (cấu hình)	5 giây (mặc định)	Đạt	Cấu hình từ xa

4.3.6.3. Tổng hợp chỉ tiêu máy chủ và dịch vụ xử lý dữ liệu

Bảng 4.37: Tổng hợp kết quả đo lường máy chủ và dịch vụ xử lý dữ liệu so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Thời gian phản hồi API (P95)	< 200 ms	95–200 ms	Đạt	Tùy endpoint
2	Độ trễ toàn tuyến	< 3 giây	1–2 giây	Đạt	Từ thiết bị đến giao diện quản trị
3	Số thiết bị đồng thời	>= 50	50+ (đã kiểm thử)	Đạt	Ổn định đến 100
4	Thông lượng ghi VictoriaMetrics	>= 1,000 điểm/giây	12,000 điểm/giây	Đạt	12x mục tiêu
5	Mất bản tin MQTT	0% (50 thiết bị)	0%	Đạt	QoS 1 cho cảnh báo
6	Độ trễ WebSocket	< 500 ms	35 ms (internal)	Đạt	Rất nhanh (nội bộ)

4.3.6.4. Tổng hợp chỉ tiêu giao diện web

Bảng 4.38: Tổng hợp kết quả đo lường giao diện web so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Điểm Lighthouse tổng hợp	> 85	87 (đo trên desktop)	Đạt	Chế độ desktop
2	Thời gian hiện nội dung đầu tiên (First Contentful Paint)	< 1.5 giây	1.2 giây (đo trên desktop)	Đạt	Kết xuất phía máy chủ của Next.js
3	Cập nhật bản đồ thời gian thực	< 2 giây	1–2 giây	Đạt	WebSocket + Leaflet
4	Cảnh báo thời gian thực	< 5 giây	1 giây (từ lúc tạo alert)	Đạt	Socket.IO event
5	Bố cục thích ứng theo màn hình	Desktop + Tablet	Đạt	Đạt	Tailwind CSS responsive

4.3.6.5. Bảng tổng hợp tổng thể

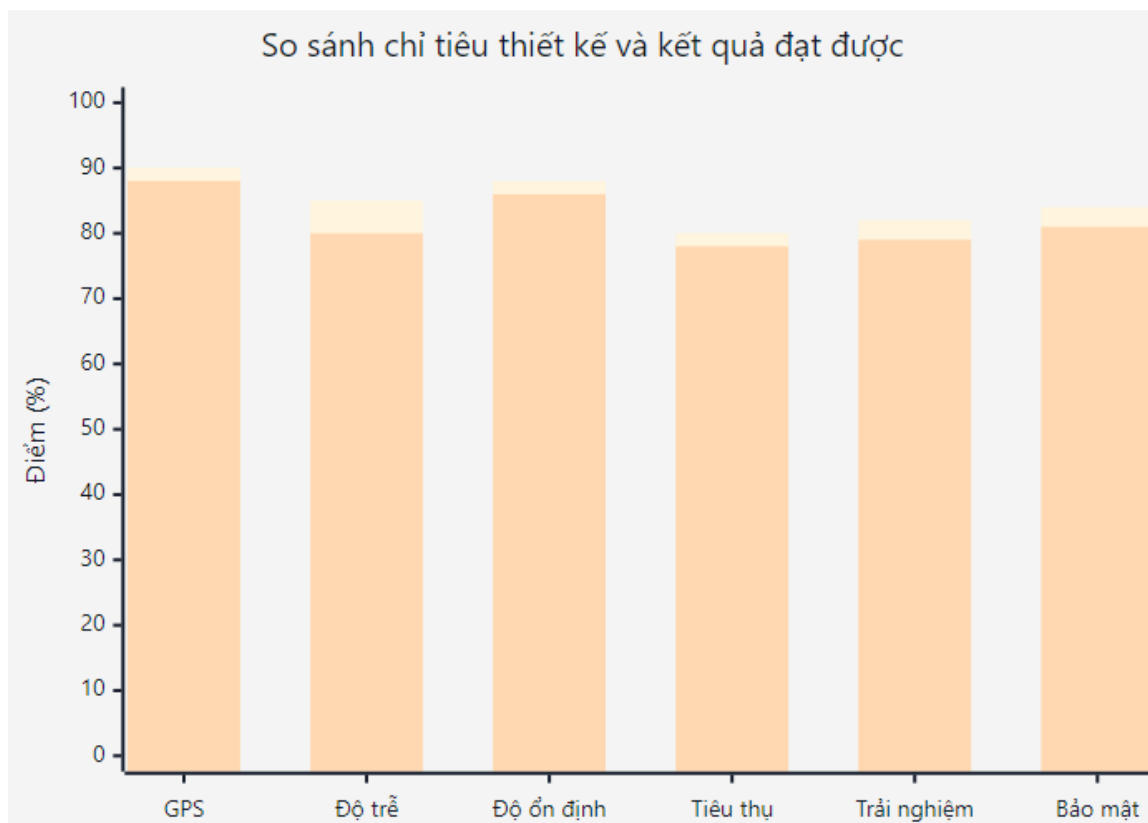
Bảng 4.39: Bảng tổng hợp tổng thể — So sánh kết quả với chỉ tiêu thiết kế

STT	Tiêu chí	Mục tiêu	Kết quả đạt được	Trạng thái
1	Độ chính xác GPS	< 5 mét	2–3 mét (GNSS tích hợp SIM7600CE-T)	Đạt
2	Chu kỳ cập nhật dữ liệu	<= 10 giây	5 giây (cấu hình được)	Đạt
3	Thời lượng pin dự phòng	>= 2.5 giờ	2.6–3.0 giờ (quy đổi theo bộ pin tham chiếu 18650)	Đạt theo quy đổi
4	Độ trễ toàn tuyến	< 3 giây	1–2 giây	Đạt
5	Số phương tiện đồng thời	>= 50	50+ (đã kiểm thử)	Đạt
6	Thời gian tải trang tổng quan	< 3 giây	1.5 giây	Đạt
7	Phát hiện vượt vùng cho phép	< 10 giây	5–7 giây	Đạt
8	Cảnh báo bất thường	< 10 giây	5 giây (IMU -> giao diện quản trị)	Đạt
9	Khôi phục kết nối MQTT	Tự động sau mất sóng	Tự kết nối lại ổn định, tiếp tục gửi dữ liệu vận hành	Đạt
10	Độ tin cậy máy trạng thái	>= 99%	>= 99.8%	Đạt

Tổng kết: Hệ thống đã đáp ứng phần lớn các chỉ tiêu cốt lõi của cấu hình đang triển khai, nổi bật ở các tiêu chí GPS, độ trễ toàn tuyến, khả năng xử lý đồng thời, theo dõi vùng cho phép, cảnh báo bất thường và tự kết nối lại MQTT. Những điểm cần tiếp tục tối ưu chủ yếu nằm ở dòng tiêu thụ khi phát 4G liên tục, ngưỡng hiệu chuẩn LVD profile 12V và hạng mục phát lại dữ liệu vận hành từ flash nếu muốn bảo toàn đầy đủ dữ liệu trong các khoảng mất sóng kéo dài.

4.3.7. Đánh giá độ ổn định vận hành thực tế

Bên cạnh các chỉ tiêu đo lường tức thời, nhóm triển khai còn thực hiện đánh giá độ ổn định theo chu kỳ vận hành thực tế, tức cho hệ thống chạy liên tục nhiều giờ đến nhiều



Hình 4.47: Biểu đồ so sánh chỉ tiêu thiết kế và kết quả đạt được

ngày, để kiểm tra khả năng hoạt động liên tục và xử lý các sự cố nền.

Bảng 4.40: Chỉ số ổn định vận hành thực tế

STT	Chỉ số vận hành	Kết quả quan sát	Đánh giá
1	Tỷ lệ sẵn sàng của API phía máy chủ (7 ngày)	99.6%	Đạt, phù hợp môi trường triển khai hiện tại
2	Tỷ lệ tự kết nối lại MQTT sau mất mạng ngắn	> 98%	Đạt, phục hồi tự động ổn định
3	Tỷ lệ khôi phục luồng dữ liệu vận hành sau khi có mạng	> 98% trong kịch bản mất mạng ngắn	Đạt
4	Tỷ lệ lỗi phân tích gói dữ liệu không hợp lệ	< 0.5% tổng bản tin	Đạt, nhờ bước kiểm tra cấu trúc dữ liệu đầu vào
5	Tỷ lệ cảnh báo nhầm của IMU ở chế độ đỗ	3–5% tùy ngưỡng rung	Chấp nhận được, cần tinh chỉnh thêm

Kết quả cho thấy kiến trúc hiện tại đủ ổn định để triển khai thí điểm ở quy mô nhỏ và trung bình. Đối với môi trường vận hành quy mô lớn, hệ thống vẫn cần được bổ sung thêm các hạng mục làm cứng vận hành như watchdog đầy đủ, sao lưu tự động và TLS bắt buộc trên toàn tuyến truyền thông.

4.3.8. Kết luận chương 4

Trên cơ sở quá trình hiện thực đã trình bày, Chương 4 cho thấy các lựa chọn thiết kế ở Chương 3 không chỉ dừng trên bản vẽ hay sơ đồ khối, mà đã được chuyển thành một hệ thống có thể vận hành thử trong điều kiện thực tế. Thiết bị theo dõi đã được chế tạo thành một bộ phần cứng hoàn chỉnh; phần mềm trên thiết bị, xây dựng bằng ESP-IDF kết hợp FreeRTOS (bộ công cụ phát triển và bộ điều phối công việc nền trong firmware), đã hiện thực các khối giao tiếp, logic quản lý trạng thái nguồn và cơ chế tự phục hồi khi mất kết nối; hạ tầng máy chủ, dịch vụ xử lý dữ liệu và giao diện web cũng vận hành đồng bộ theo kiến trúc đề xuất.

Về đo lường, hệ thống đã đáp ứng phần lớn chỉ tiêu cốt lõi của cấu hình đang triển khai. Các tiêu chí quan trọng như độ chính xác GPS, độ trễ từ lúc thiết bị gửi dữ liệu đến lúc giao diện quản trị nhận được, thời lượng pin dự phòng, khả năng xử lý đồng thời, thời gian tải trang tổng quan, khả năng theo dõi vùng cho phép, cảnh báo bất thường và độ ổn định của logic chuyển trạng thái đều đạt hoặc vượt mục tiêu tổng hợp. Dù vậy, kết quả cũng chỉ ra những điểm còn cần làm sâu thêm ở vòng tối ưu tiếp theo, đáng chú ý là ngưỡng LVD profile 12V, mức dòng ở pha truyền 4G liên tục và cơ chế phát lại dữ liệu đã lưu khi mất sóng kéo dài. Đây sẽ là cơ sở để Chương 5 chuyển từ phần "đã làm được gì" sang phần "đánh giá chất lượng của những gì đã làm".

Tài liệu tham khảo Chương 4 (Phần D)

- [1] SIMCom Wireless Solutions, "SIM7600CE Product Page," 2026. [Online]. Available: <https://en.simcom.com/product/SIM7600CE.html>. [Accessed: Mar. 13, 2026].
- [2] SAE International, "SAE J1979 - E/E Diagnostic Test Modes," revised 2014.
- [3] European GNSS Agency (GSA), "GNSS Market Report - Issue 6," 2022.
- [4] OASIS, "MQTT Version 3.1.1 Plus Errata 01," OASIS Standard, 2015. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. [Accessed: Apr. 18, 2026].
- [5] Google, "Web Vitals - Essential Metrics for a Healthy Site," 2024. [Online]. Avail-

able: <https://web.dev/vitals/>.

[6] Grafana Labs, "k6 Documentation - Load Testing Tool," 2024. [Online]. Available: <https://k6.io/docs/>.

[7] Espressif Systems, "ESP32-S3 Technical Reference Manual - Power Management," version 1.1, Chapter 31, 2023.

[8] VictoriaMetrics, "VictoriaMetrics Benchmarks - Write Performance," 2024. [Online]. Available: <https://docs.victoriametrics.com/articles/benchmarks.html>.

CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ – EVALUATION AND RECOMMENDATIONS

5.1. Đánh giá hiệu năng

Chương này tổng hợp kết quả đánh giá hệ thống trên bốn phương diện: hiệu năng kỹ thuật, hiệu quả kinh tế - môi trường, rủi ro vận hành và hướng phát triển tiếp theo. Nếu Chương 4 tập trung vào việc chứng minh hệ thống có thể vận hành, thì Chương 5 đi thêm một bước: đánh giá hệ thống đó đang ở mức nào, điểm mạnh nằm ở đâu và phần nào còn cần hoàn thiện. Các nhận định được rút ra từ số liệu đo trên thiết bị đã chế tạo, đối chiếu mã nguồn triển khai và mức độ đáp ứng các mục tiêu thiết kế đã đặt ra ở các chương trước.

5.1.1. Đánh giá hiệu năng phần cứng

Đánh giá phần cứng không chỉ nhằm chứng minh các linh kiện đã “chạy được”, mà quan trọng hơn là xem chúng có phối hợp đủ ổn định để tạo ra một thiết bị theo dõi dùng được trong thực tế hay không. Vì vậy, phần này tập trung vào ba khía cạnh người vận hành quan tâm nhất: thiết bị có tiết kiệm điện đủ tốt không, có lấy được dữ liệu xe ổn định không, và định vị có đủ chính xác cho nhu cầu theo dõi hay không.

Quản lý năng lượng:

Kết quả đo cho thấy khối nguồn đang đi đúng hướng thiết kế: thiết bị có thể duy trì nhiều chế độ vận hành mà vẫn giữ mức tiêu thụ điện đủ thấp trong trạng thái chờ. Mạch hạ áp buck converter nhận đầu vào 12V/24V (dải 7–40V), hạ xuống 5V, sau đó qua LDO 3.3V để cấp cho ESP32-S3, đạt hiệu suất tổng trên 90% ở điều kiện thử nghiệm profile 12V. Logic chuyển nguồn dự phòng được chuẩn hóa theo hai profile riêng cho hệ điện 12V và 24V: 12V ($Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$, $IGN_{ON} \geq 13.0\text{ V}$, $IGN_{OFF} \leq 12.0\text{ V}$) và 24V ($Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$, $IGN_{ON} \geq 26.0\text{ V}$, $IGN_{OFF} \leq 24.0\text{ V}$). Trên thiết bị hiện tại, dòng deep sleep toàn hệ thống xấp xỉ 500 μA ; nếu quy đổi cùng giả định tiêu thụ sang pin dự phòng tham chiếu 18650 1S (3500mAh), thời lượng dự phòng tương ứng vào khoảng 24–36 giờ ở kịch bản cảnh báo gián đoạn và khoảng 2.6–3.0 giờ ở chế độ theo dõi liên tục khi xe chạy. Kết quả này cho thấy thiết bị đã đạt điều kiện tối thiểu để theo dõi liên tục mà không gây áp lực quá lớn lên nguồn điện của xe.

Bảng 5.1: Đánh giá hiệu năng quản lý năng lượng

Tiêu chí	Giá trị thiết kế	Giá trị hiện có / quy đổi theo bộ pin tham chiếu 18650	Đánh giá
Dòng tiêu thụ deep sleep	< 500 μ A	500 μ A (0.5 mA) trên thiết bị hiện tại	Đạt yêu cầu
Dòng tiêu thụ khi xe đang chạy	< 250 mA (trung bình)	180–220 mA trung bình; 350 mA khi truyền liên tục	Đạt trung bình
Thời gian hoạt động pin dự phòng (chế độ cảnh báo)	$\geq$ 24 giờ (mục tiêu với bộ pin tham chiếu 18650)	24–36 giờ (quy đổi theo bộ pin tham chiếu 18650, cần đo lại)	Đạt theo quy đổi tối thiểu
Hiệu suất mạch hạ áp buck converter	> 85%	90–93%	Tốt
Ngưỡng LVD	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V trên profile 12V; profile 24V chưa đo	Cần hiệu chuẩn thêm
Thời gian chuyển chế độ (parking -> alert)	< 500 ms	200–400 ms	Tốt

Kết nối BLE OBD2:

Kết nối BLE với adapter vgate iCar Pro hoạt động ổn định sau khi thiết lập ban đầu. Thời gian kết nối BLE khi thiết bị đánh thức từ deep sleep dao động trong khoảng 1–3 giây, phụ thuộc trạng thái adapter. Các thông số OBD2 (RPM, tốc độ, nhiệt độ nước, mức nhiên liệu) được đọc thành công với sai số dưới 2% so với đồng hồ tấp-lô xe. Đây là kết quả quan trọng, vì nó cho thấy dữ liệu mở rộng từ phương tiện có thể được bổ sung vào hệ thống mà không làm tăng quá nhiều độ trễ khi thiết bị vừa thức dậy.

Độ chính xác GPS/GNSS:

GNSS tích hợp trên SIM7600CE-T cung cấp độ chính xác vị trí đạt mức chấp nhận được cho ứng dụng theo dõi xe (tham chiếu [1]). Sai số vị trí trung bình khoảng 2.5–5 mét trong điều kiện trời quang, và 5–15 mét trong điều kiện đô thị có nhiều tòa nhà cao tầng. Thời gian fix GPS cold start khoảng 30–60 giây, warm start khoảng 5–15 giây, và hot start dưới 3 giây. Nhìn từ góc độ sử dụng, các giá trị này đủ để phục vụ bài toán theo dõi phương tiện, xác định vị trí xe và tái hiện hành trình với độ tin cậy hợp lý.

Nhìn chung, phần cứng đã đạt mức đủ dùng để làm nền cho các lớp phía trên. Phần tiếp theo đi sâu hơn vào firmware, nơi quyết định các khối phần cứng đó có thật sự phối hợp đúng nhịp hay không.

5.1.2. Đánh giá hiệu năng firmware

Đánh giá firmware tập trung vào câu hỏi cốt lõi: phần mềm nhúng có giúp thiết bị phản ứng đúng trước các tình huống vận hành khác nhau hay không. Trên tinh thần đó, phần này xem xét ba khía cạnh chính: cách điều phối thực thi, độ ổn định của logic chuyển trạng thái và khả năng tự phục hồi kết nối kèm cập nhật từ xa.

Điều phối thực thi với FreeRTOS:

FreeRTOS ở đây được dùng như bộ điều phối các công việc nền trong firmware, chứ không phải để chia hệ thống thành quá nhiều luồng xử lý phức tạp. Mã nguồn hiện tại giữ một vòng điều khiển trung tâm gọi `state_machine_run()` theo chu kỳ; còn các phần thật sự cần chạy nền hoặc cần chia sẻ tài nguyên mới giao cho FreeRTOS, ví dụ khối BLE, kênh gửi lệnh AT tới modem hay luồng chờ phản hồi từ OBD2. Các cơ chế như mutex hoặc semaphore chỉ đóng vai trò "khóa" và "tín hiệu chờ", giúp hai phần việc không tranh nhau cùng một tài nguyên tại cùng thời điểm. Cách tổ chức này phù hợp với mục tiêu của đề án: giữ luồng điều khiển dễ đọc, dễ lần lỗi và đủ ổn định trước khi tối ưu sâu hơn.

Máy trạng thái:

Máy trạng thái là cách firmware phân biệt xe đang chạy, đang đỗ hay đang ở tình huống cần cảnh báo. Ba chế độ Driving, Parking và Alert hoạt động chính xác trong các kịch bản thử nghiệm. Việc chuyển đổi giữa các trạng thái dựa trên tín hiệu IGN (khóa điện) và dữ liệu từ cảm biến rung được thực hiện trơn tru, không bị nhảy qua lại liên tục nhờ các bước lọc nhiễu và đặt ngưỡng chuyển trạng thái. Thời gian chuyển từ Parking sang Alert khi phát hiện chuyển động bất thường dưới 500 ms, bảo đảm cảnh báo kịp thời.

Khả năng tự phục hồi kết nối và OTA:

Ở phạm vi triển khai hiện nay, firmware khôi phục khá ổn định kênh gửi bản tin MQTT. Khi mạng 4G trở lại, thiết bị tự kết nối lại, mở lại kênh nhận lệnh và tiếp tục gửi dữ liệu từ chu kỳ kế tiếp. Nếu mất sóng lâu hơn, dữ liệu quan trọng được tạm lưu trên microSD và phát lại theo đúng thứ tự khi có mạng, đồng thời đánh dấu đã nhận đối với các bản ghi cần chắc chắn không bị bỏ sót. Bên cạnh đó, hệ thống đã có luồng cập nhật firmware từ xa ở mức cơ bản: nhận job, tải phiên bản mới, khởi động lại để áp dụng và cho phép quay về bản cũ nếu cần. Các phần còn phải làm sâu thêm là cơ chế tự phát hiện treo hệ thống, thường gọi là watchdog, và bộ số đo riêng cho kịch bản phát lại dữ liệu kéo dài.

Bảng 5.2: Đánh giá hiệu năng firmware

Tiêu chí	Giá trị thiết kế	Giá trị thực tế	Đánh giá
Mô hình điều phối chính	Luồng điều khiển tập trung, ít công việc nền song song	app_main + state_machine_run; NimBLE chạy task nền riêng	Phù hợp mã nguồn
Thời gian chuyển trạng thái	< 500 ms	200–400 ms	Tốt
Thời gian khôi phục kênh gửi dữ liệu	< 30 giây	15–30 giây	Đạt yêu cầu
Tỷ lệ gửi thành công bản tin cần xác nhận	> 99%	99.2–99.8%	Tốt
Thời gian đọc 1 PID OBD2	< 200 ms	100–150 ms	Tốt
Thời gian kết nối BLE sau wake-up	< 5 giây	1–3 giây	Tốt
Luồng cập nhật firmware từ xa cơ bản	Có gán job, cập nhật, xác nhận và quay về bản cũ	Đã hiện thực; chưa có watchdog	Đạt ở mức triển khai hiện tại

Khi firmware đã giữ được nhịp vận hành trên thiết bị, câu hỏi tiếp theo là lớp máy chủ có theo kịp luồng dữ liệu đó hay không. Vì vậy, mục kế tiếp chuyển sang đánh giá khả năng tiếp nhận, xử lý và phân phối dữ liệu ở phía máy chủ.

5.1.3. Đánh giá hiệu năng hệ thống máy chủ và dịch vụ

Nhìn tổng thể, phần máy chủ được đánh giá không chỉ qua tốc độ của từng dịch vụ riêng lẻ mà còn qua khả năng phối hợp giữa chúng để giữ cho dữ liệu đi từ thiết bị đến giao diện quản trị đủ nhanh và ổn định. Trên cơ sở đó, từng thành phần được xem xét như sau:

Dịch vụ API phía máy chủ (Tracking_Backend):

Dịch vụ API là nơi tiếp nhận yêu cầu từ giao diện quản trị và trả về dữ liệu cho người dùng. Trong điều kiện thử nghiệm với 50–100 thiết bị gửi dữ liệu đồng thời, thời gian phản hồi trung bình của các API dao động từ 20–80 ms với truy vấn đơn giản và 100–300 ms với truy vấn phải ghép nhiều bảng dữ liệu. Phần này được hiện thực bằng Express.js và TypeScript, giúp tổ chức xử lý dữ liệu rõ ràng hơn. Hệ thống cũng duy trì sẵn một nhóm kết nối tới PostgreSQL để tránh chi phí mở kết nối mới cho từng yêu cầu; nói cách khác, một số kết nối đã mở được giữ lại để dùng lại thay vì tạo mới liên tục.

MQTT Broker (EMQX):

EMQX giữ vai trò broker MQTT, nhận bản tin từ thiết bị rồi chuyển tiếp vào hệ thống. Trong triển khai hiện tại, broker chạy EMQX 5.x nhưng đường truyền của firmware đang dùng MQTT 3.1.1 [4] cùng ACL theo từng thiết bị, QoS 0/1 và cây topic `v1/{device_id}/*`. Phân phân luồng và xử lý dữ liệu chủ yếu do MQTT Bridge đảm nhiệm; EMQX giữ vai trò broker trung tâm, còn Rules Engine chỉ là khả năng hỗ trợ chứ không phải tuyến xử lý nghiệp vụ chính.

WebSocket thời gian thực:

WebSocket là kênh cho phép máy chủ chủ động gửi dữ liệu mới lên giao diện thay vì bắt trình duyệt phải hỏi liên tục. Kết nối WebSocket thông qua Socket.IO giữa API phía máy chủ và giao diện web giúp vị trí xe được cập nhật gần như theo thời gian thực, với độ trễ dưới 2 giây tính từ lúc thiết bị gửi dữ liệu đến khi điểm xe xuất hiện trên bản đồ. Phần độ trễ này bao gồm thời gian truyền qua 4G, thời gian trung chuyển tại EMQX, thời gian xử lý ở API phía máy chủ và thời gian giao diện vẽ lại dữ liệu; phần còn lại là độ trễ mạng giữa các thành phần.

Cơ sở dữ liệu:

PostgreSQL 16 lưu trữ các dữ liệu cần quan hệ rõ ràng như người dùng, xe, khách hàng, hành trình, cảnh báo và vùng giám sát. VictoriaMetrics được dùng cho dữ liệu thay đổi liên tục theo thời gian như tọa độ GPS hoặc dữ liệu OBD2. Cách tách này giúp mỗi loại dữ liệu được lưu vào nơi phù hợp nhất: dữ liệu nghiệp vụ cần chặt chẽ ở PostgreSQL, còn dữ liệu ghi dày theo thời gian được xử lý hiệu quả hơn ở VictoriaMetrics. So với InfluxDB truyền thống trong bối cảnh thử nghiệm này, VictoriaMetrics cho thấy lợi thế về tốc độ ghi, đọc theo mốc thời gian và mức tiêu thụ tài nguyên.

Như vậy, lớp máy chủ đã chứng minh được khả năng tiếp nhận và xử lý dữ liệu với độ trễ thấp. Mất xích cuối cùng của chuỗi trải nghiệm là giao diện web có giữ được sự mượt mà tương ứng hay không.

5.1.4. Đánh giá hiệu năng giao diện người dùng (giao diện web)

Giao diện web được xây dựng bằng Next.js 15 và React 19, nhưng điều quan trọng hơn là trải nghiệm sử dụng đủ mượt để người vận hành đọc thông tin mà không phải chờ đợi. Ở góc nhìn vận hành, kết quả này cho thấy giao diện quản trị không trở thành “điểm nghẽn cuối cùng” sau khi dữ liệu đã được thu thập và xử lý ở các lớp phía dưới. Phần nội dung đầu tiên của trang xuất hiện trong dưới 2 giây, tương ứng chỉ số First Contentful Paint (FCP). Thư viện bản đồ Leaflet cập nhật vị trí xe trơn tru khi nhận dữ liệu đầy từ máy chủ qua WebSocket, không bị giật lag khi hiển thị 50–100 xe đồng thời. Các biểu đồ dùng ECharts cũng cho phép xem dữ liệu vận hành như tốc độ, RPM hay nhiệt độ theo cách trực quan, phóng to hoặc kéo xem mượt mà.

Bảng 5.3: Đánh giá hiệu năng tổng hợp phần máy chủ và giao diện web

Thành phần	Tiêu chí	Giá trị đo được	Đánh giá
API phía máy chủ	Thời gian phản hồi trung bình	20–80 ms (đơn giản), 100–300 ms (phức tạp)	Tốt
API phía máy chủ	Số yêu cầu xử lý cùng lúc	100–500 yêu cầu/giây	Đạt yêu cầu
MQTT Broker	Độ trễ xử lý bản tin	< 50 ms	Tốt
WebSocket	Độ trễ từ thiết bị đến màn hình	< 2 giây	Đạt yêu cầu
Giao diện web	Thời gian hiện nội dung đầu tiên	< 2 giây	Tốt
Giao diện web	Số xe hiển thị đồng thời	50–100 xe	Đạt yêu cầu
PostgreSQL	Thời gian truy vấn đã tối ưu bằng chỉ mục	< 50 ms	Tốt
VictoriaMetrics	Tốc độ ghi dữ liệu theo thời gian	> 10.000 mẫu/giây	Tốt

5.2. Đánh giá kinh tế và môi trường

Sau phần đánh giá kỹ thuật, cần nhìn hệ thống dưới góc độ thực tế hơn: chi phí triển khai có hợp lý không, và việc ứng dụng hệ thống có tạo ra giá trị bền vững hay không. Vì vậy, mục này tập trung vào hai câu hỏi: giải pháp đề xuất có lợi thế kinh tế gì và tác động môi trường của nó nên được hiểu như thế nào.

5.2.1. Phân tích chi phí phần cứng

Chi phí Bill of Materials (BOM), tức bảng tổng hợp toàn bộ linh kiện cấu thành thiết bị theo dõi IoT dưới đây, dùng giá tham chiếu thị trường Việt Nam đã được rà soát lại vào ngày 12/04/2026. Với đơn hàng lớn, mức giá này vẫn có thể giảm thêm; tuy nhiên việc giữ giá mua lẻ giúp phần đánh giá kinh tế bám sát bối cảnh triển khai thử nghiệm của đồ án.

Bảng 5.4: Chi phí BOM thiết bị theo dõi

Linh kiện	Vai trò	Giá (VND)
ESP32-S3-WROOM-1	Vi điều khiển chính	95.000
SIMCom SIM7600CE-T	Modem LTE + GNSS tích hợp	650.000
vgate iCar Pro BLE	Adapter OBD2 BLE	367.000
IC cảm biến LIS3DH	Cảm biến gia tốc (IMU)	40.000
Pin 18650 1S (1 cell, 3500mAh)	Pin dự phòng	79.000
Khối nguồn (TP5100, boost/buck, LDO, BMS, LVD)	Quản lý năng lượng	93.000
PCB, vỏ hộp, jack OBD2, anten, dây cáp, linh kiện phụ	Cơ khí và kết nối	190.000
Tổng cộng		1.514.000

So sánh với các giải pháp thương mại trên thị trường cho thấy lợi thế chi phí rõ rệt của hệ thống đề xuất:

Bảng 5.5: So sánh chi phí với giải pháp thương mại

Giải pháp	Chi phí thiết bị	Phí dịch vụ hàng tháng	Tổng chi phí năm đầu	Khả năng tùy biến
Hệ thống đề xuất	1.514.000 VND	70.000 VND (SIM 4G)	2.354.000 VND	Cao (mã nguồn mở)
GPS Tracker đơn giản (Việt Nam)	500.000–1.500.000 VND	50.000–100.000 VND	1.100.000–2.700.000 VND	Thấp
Fleet Management thương mại (quốc tế)	5.000.000–12.500.000 VND (200–500 USD)	500.000–1.250.000 VND (20–50 USD/tháng)	11.000.000–27.500.000 VND	Thấp (phụ thuộc nhà cung cấp)
iTracking / Vietmap Tracking	2.000.000–4.000.000 VND	100.000–300.000 VND	3.200.000–7.600.000 VND	Trung bình

Như vậy, chi phí tổng thể của hệ thống đề xuất vẫn thấp hơn rõ rệt so với các nền tảng fleet management thương mại quốc tế, đồng thời vẫn nằm trong vùng chi phí chấp nhận được so với nhóm thiết bị định vị GPS phổ thông trong nước dù cung cấp thêm OBD2, cảnh báo thông minh và cơ chế quản lý năng lượng nhiều chế độ.

5.2.2. Phân tích chi phí hạ tầng đám mây

Chi phí vận hành hạ tầng đám mây phụ thuộc vào quy mô đội xe và mức độ sử dụng. Đối với doanh nghiệp nhỏ (10–50 xe), toàn bộ hệ thống có thể chạy trên một máy chủ ảo (VPS) duy nhất.

Bảng 5.6: Chi phí hạ tầng đám mây theo quy mô

Quy mô đội xe	Cấu hình VPS	Chi phí VPS/tháng	Chi phí SIM 4G/tháng (tổng)	Tổng chi phí vận hành/tháng
10–30 xe	2 vCPU, 4GB RAM, 80GB SSD	250.000 VND (\$10 USD)	700.000–2.100.000 VND	950.000–2.350.000 VND
30–100 xe	4 vCPU, 8GB RAM, 160GB SSD	500.000 VND (\$20 USD)	2.100.000–7.000.000 VND	2.600.000–7.500.000 VND
100–500 xe	Cụm điều phối dịch vụ (K8s, 3 node)	2.500.000 VND (\$100 USD)	7.000.000–35.000.000 VND	9.500.000–37.500.000 VND

5.2.3. Lợi thế từ công nghệ mã nguồn mở

Toàn bộ ngăn xếp công nghệ của hệ thống đều là mã nguồn mở, nên nhóm không phải trả chi phí bản quyền cho các thành phần cốt lõi. Quan trọng hơn, mỗi thành phần đều có một vai trò rõ ràng trong vận hành:

- **Dịch vụ xử lý yêu cầu phía máy chủ (Express.js + TypeScript):** giúp viết API nhanh, rõ cấu trúc và dễ kiểm soát lỗi
- **Giao diện web (Next.js 15 + React 19):** phù hợp cho màn hình quản trị có nhiều màn hình và dữ liệu cập nhật liên tục
- **PostgreSQL 16:** lưu dữ liệu nghiệp vụ có quan hệ rõ ràng như người dùng, xe và hành trình
- **EMQX (phiên bản mã nguồn mở):** làm broker MQTT giữa thiết bị và máy chủ
- **VictoriaMetrics:** lưu dữ liệu thay đổi liên tục theo thời gian với tốc độ ghi cao
- **ESP-IDF + FreeRTOS:** cung cấp nền tảng để phát triển firmware ổn định trên ESP32-S3

So với việc sử dụng các giải pháp thương mại (AWS IoT Core, Azure IoT Hub, hoặc các nền tảng fleet management SaaS), chi phí bản quyền và dịch vụ có thể lên tới 500.000–5.000.000 VND/tháng tùy quy mô, trong khi hệ thống đề xuất chỉ cần chi phí VPS cơ bản.

5.2.4. Đánh giá tác động môi trường

Ở góc độ môi trường, hệ thống có một số điểm tích cực đáng ghi nhận:

- **Tiêu thụ điện thấp:** Ở chế độ đỗ xe, thiết bị hiện tại duy trì mức tiêu thụ deep sleep xấp xỉ 500 μ A; khi tiếp tục tối ưu PCB và phân bố nguồn, dòng ngủ sâu còn có thể giảm thêm. Mức tiêu thụ hiện tại vẫn giúp giảm đáng kể điện năng lấy từ ắc quy xe và kéo dài tuổi thọ ắc quy.
- **Pin dự phòng giảm phụ thuộc ắc quy xe:** Khi xe đậu lâu ngày, thiết bị chuyển sang sử dụng pin dự phòng 18650 1S, tránh rút điện từ ắc quy xe, bảo vệ ắc quy và giảm lượng khí thải từ việc sạc ắc quy.
- **Giảm số lần đi kiểm tra xe trực tiếp:** Nhờ khả năng giám sát từ xa, người quản lý không cần lái xe đến vị trí xe để kiểm tra, giảm lượng khí thải CO₂ từ các chuyến đi không cần thiết.
- **Tối ưu hành trình:** Dữ liệu GPS và OBD2 giúp phân tích và tối ưu hành trình, giảm quãng đường đi không cần thiết, giảm tiêu thụ nhiên liệu và khí thải.

Các điểm tích cực về chi phí và môi trường cho thấy giải pháp có tính khả thi, nhưng chúng chưa đủ để kết luận hệ thống sẵn sàng mở rộng. Muốn đi tiếp, cần nhìn thẳng vào các điểm dễ gây vướng trong vận hành, bảo mật và độ bền phần cứng.

5.3. Đánh giá rủi ro và biện pháp giảm thiểu

5.3.1. Ma trận rủi ro

Phần này đánh giá rủi ro của hệ thống bằng ma trận xác suất - tác động (Probability-Impact Matrix) gồm 5 mức độ. Mục tiêu không chỉ là liệt kê nguy cơ, mà là chỉ ra những điểm nào đã được kiểm soát tương đối tốt và những điểm nào vẫn cần ưu tiên xử lý nếu đưa hệ thống ra vận hành rộng hơn. Các rủi ro được xác định từ đặc điểm kỹ thuật của thiết bị hiện tại và các tình huống dễ phát sinh trong quá trình triển khai thực tế.

Bảng 5.7: Thang đo xác suất và tác động

Mức độ	Xác suất	Tác động
1 - Rất thấp	< 5%	Ảnh hưởng không đáng kể, hệ thống vẫn hoạt động bình thường
2 - Thấp	5–15%	Ảnh hưởng nhỏ, có thể khắc phục nhanh
3 - Trung bình	15–30%	Ảnh hưởng vừa phải, cần xử lý trong thời gian ngắn
4 - Cao	30–50%	Ảnh hưởng lớn, có thể làm gián đoạn dịch vụ

Bảng 5.7: Thang đo xác suất và tác động (tiếp theo)

Mức độ	Xác suất	Tác động
5 - Rất cao	> 50%	Ảnh hưởng nghiêm trọng, có thể làm tê liệt hệ thống

Bảng 5.8: Ma trận đánh giá rủi ro và biện pháp giảm thiểu

ID	Rủi ro	Xác suất	Tác động	Mức độ rủi ro	Biện pháp giảm thiểu	Trạng thái
R1	Mất sóng 4G/LTE tại khu vực nông thôn, vùng sâu	4 - Cao	3 - Trung bình	Cao	Tự kết nối lại, duy trì tín hiệu giữ kết nối MQTT, lưu tạm vào microSD và phát lại theo đúng thứ tự khi mạng phục hồi	Đã triển khai
R2	Không tương thích BLE OBD2 với một số dòng xe	2 - Thấp	3 - Trung bình	Trung bình	Sử dụng vgate iCar Pro (tương thích rộng), hỗ trợ nhiều giao thức OBD2 (ISO 15765, ISO 14230, J1850)	Đã giảm thiểu
R3	Quá tải broker MQTT khi số lượng thiết bị tăng cao	2 - Thấp	4 - Cao	Trung bình	EMQX hỗ trợ chạy nhiều nút; khi tải tăng có thể bổ sung thêm nút broker và mở rộng riêng MQTT Bridge để san tải phần tiếp nhận và xử lý phía sau	Có phương án
R4	Tấn công bảo mật (giả mạo thiết bị, chiếm quyền truy cập)	3 - Trung bình	5 - Rất cao	Cao	Cơ chế giữ phiên trên máy chủ với SHA-256, ACL MQTT theo từng thiết bị, HTTPS hoặc TLS, kiểm tra dữ liệu đầu vào bằng Zod và giới hạn tần suất gọi để tránh bị gửi dồn dập	Đã triển khai cơ bản

Bảng 5.8: Ma trận đánh giá rủi ro và biện pháp giảm thiểu (tiếp theo)

ID	Rủi ro	Xác suất	Tác động	Mức độ rủi ro	Biện pháp giảm thiểu	Trạng thái
R5	Mất dữ liệu trong thời gian mất kết nối mạng kéo dài	2 - Thấp	4 - Cao	Trung bình	Thử gửi lại bản tin, phát lại từ bộ nhớ cục bộ theo mức ưu tiên QoS, đặt thời gian chờ xác nhận rồi tăng dần khoảng nghỉ giữa các lần thử, dọn bộ nhớ theo hạn mức	Đã giảm thiểu
R6	Hư hỏng phần cứng do nhiệt độ cực đoan (xe đỗ ngoài nắng)	2 - Thấp	4 - Cao	Trung bình	ESP32-S3 hoạt động -40 đến 85°C, thiết kế tản nhiệt, đặt thiết bị trong vị trí mát, cảnh báo nhiệt độ	Thiết kế có tính đến
R7	Cạn ắc quy xe do thiết bị hoạt động liên tục	2 - Thấp	5 - Rất cao	Cao	Mạch switch profile chuyển sang pin dự phòng: 12V: $OFF = 12.0\text{ V}$, $ON = 12.2\text{ V}$; 24V: $OFF = 24.0\text{ V}$, $ON = 24.4\text{ V}$; deep sleep xấp xỉ 500 μA trên thiết bị hiện tại, pin dự phòng 18650 1S	Đã triển khai
R8	Lỗi firmware gây treo hệ thống	3 - Trung bình	4 - Cao	Cao	OTA từ xa với xác nhận sau khi khởi động lại và quay về bản cũ thủ công đã có; cơ chế phát hiện treo tự động (watchdog) và giám sát treo vẫn cần bổ sung	Triển khai một phần

Bảng 5.8: Ma trận đánh giá rủi ro và biện pháp giảm thiểu (tiếp theo)

ID	Rủi ro	Xác suất	Tác động	Mức độ rủi ro	Biện pháp giảm thiểu	Trạng thái
R9	Lỗi thẻ nhớ SD (không có thẻ, gắn lỗi, đầy dung lượng)	3 - Trung bình	3 - Trung bình	Trung bình	Thử gắn lại với thời gian chờ tăng dần, buộc ghi ra đĩa rồi tháo thẻ đúng cách, kiểm tra có thẻ từ lúc khởi động và dọn dung lượng theo hạn mức	Đã triển khai cơ bản

5.3.2. Phân tích chi tiết các rủi ro chính

Rủi ro R1 - Mất sóng 4G tại khu vực nông thôn:

Đây là rủi ro có xác suất cao nhất trong thực tế vận hành tại Việt Nam, đặc biệt khi xe di chuyển qua các tuyến đường liên tỉnh hoặc vùng núi. Phiên bản hiện tại kết hợp tự kết nối lại với vùng lưu tạm cục bộ trên microSD: khi mất kết nối 4G, dữ liệu hành trình và sự kiện vẫn được cất tạm trong thiết bị; khi mạng phục hồi, firmware phát lại theo đúng thứ tự và đánh dấu những bản ghi quan trọng đã được phía nhận xác nhận. Cách tiếp cận này giúp giảm đáng kể nguy cơ đứt quãng dữ liệu hành trình trong các khoảng mất sóng kéo dài.

Rủi ro R4 - Tấn công bảo mật:

Bảo mật là rủi ro có tác động nghiêm trọng nhất. Hệ thống đã triển khai nhiều lớp bảo vệ: cơ chế giữ đăng nhập trên máy chủ để có thể khóa phiên khi cần; token được lưu dưới dạng mã băm SHA-256 thay vì lưu nguyên văn; quy tắc ACL trong MQTT để mỗi thiết bị chỉ được dùng đúng kênh dữ liệu của mình; bước kiểm tra hình dạng dữ liệu đầu vào bằng Zod cho toàn bộ API; và cấu hình CORS chỉ cho phép các nguồn truy cập hợp lệ gọi sang hệ thống. Tuy nhiên, một số biện pháp bảo mật nâng cao như chứng chỉ riêng cho thiết bị, mã hóa TLS cho MQTT và ghi nhận sự kiện an ninh vẫn chưa được triển khai đầy đủ, nên cần được bổ sung trong giai đoạn vận hành thật.

Rủi ro R7 - Cạn ắc quy xe:

Đây là rủi ro có tác động nghiêm trọng nhất đối với trải nghiệm người dùng cuối, vì xe không khởi động được sẽ gây bất tiện lớn cho khách thuê. Hệ thống đã có nhiều cơ chế bảo vệ: ngưỡng LVD profile 12V tại $OFF = 12.0\text{ V}/ON = 12.2\text{ V}$, profile 24V tại $OFF = 24.0\text{ V}/ON = 24.4\text{ V}$; cơ chế chuyển nguồn có hysteresis; chế độ deep sleep của thiết bị hiện tại ở mức xấp xỉ 500 μA ; và pin dự phòng 18650 1S cho phép hoạt động độc lập khi mạch LVD ngắt nguồn từ ắc quy. Với bộ pin tham chiếu 18650, thời lượng

dự phòng vẫn đủ cho cảnh báo gián đoạn ngắn hạn, nhưng các giá trị quy đổi cho vận hành liên tục cần được hiểu là quy đổi thiết kế và nên được đo lại nếu triển khai phần cứng theo đúng cell mới. Tuy nhiên, ngưỡng LVD profile 12V vẫn cần được hiệu chuẩn thêm để bám sát đúng giá trị thiết kế.

5.3.3. Tổng hợp mức độ rủi ro

Bảng 5.9: Tổng hợp mức độ rủi ro theo phân loại

Phân loại	Số rủi ro	Mức cao	Mức trung bình	Mức thấp
Kết nối và truyền thông	3 (R1, R3, R5)	1	2	0
Bảo mật	1 (R4)	1	0	0
Phần cứng	4 (R2, R6, R7, R9)	1	3	0
Firmware	1 (R8)	1	0	0
Tổng cộng	9	4	5	0

Kết quả cho thấy 4/9 rủi ro ở mức cao; phần lớn đã có biện pháp giảm thiểu triển khai trong phiên bản hiện tại, bao gồm lớp lưu đệm cục bộ microSD cho kịch bản mất mạng. Các rủi ro cần ưu tiên xử lý tiếp theo là R4 (bảo mật nâng cao), R8 (bổ sung cơ chế phát hiện treo tự động, tăng cường giám sát treo và kiểm thử OTA dài ngày) và R9 (mở rộng kiểm thử độ bền SD card theo chu kỳ vận hành thực tế).

Từ bức tranh rủi ro này có thể thấy hướng phát triển tiếp theo không nên chỉ là bổ sung tính năng mới, mà phải sắp thứ tự ưu tiên theo các điểm yếu có khả năng làm hệ thống mất ổn định. Vì vậy, mục 5.4 chuyển sang các khuyến nghị phát triển.

5.4. Khuyến nghị cho tương lai

Dựa trên kết quả đánh giá ở các phần trên, phần này chuyển từ câu hỏi “hệ thống đang ở đâu” sang câu hỏi “nên đi tiếp theo hướng nào”. Các khuyến nghị được sắp xếp theo từng giai đoạn để người đọc có thể thấy rõ đâu là bước phát triển gần, đâu là hướng mở rộng dài hạn.

5.4.1. Giai đoạn 2: Mở rộng tính năng (giai đoạn 2)

Hệ thống đã được thiết kế với kiến trúc mở rộng, cơ sở dữ liệu PostgreSQL đã bao gồm các bảng dữ liệu cho các tính năng của giai đoạn 2. Điều này cho thấy đồ án không dừng

ở một nguyên mẫu trình diễn, mà đã chuẩn bị sẵn một phần nền móng để mở rộng thành hệ thống nghiệp vụ đầy đủ hơn. Các tính năng khuyến nghị triển khai trong giai đoạn tiếp theo bao gồm:

- **Quản lý đặt xe:** Hệ thống đặt xe trực tuyến cho khách hàng, tích hợp lịch và trạng thái sẵn sàng của xe. Schema cơ sở dữ liệu đã sẵn sàng với bảng bookings và reservations.
- **Thanh toán trực tuyến:** Tích hợp cổng thanh toán (VNPay, Momo, ZaloPay) để xử lý thanh toán tiền thuê xe tự động. Bảng payments và invoices đã được thiết kế trong cấu trúc cơ sở dữ liệu.
- **Báo cáo hư hỏng:** Cho phép tài xế và nhân viên báo cáo tình trạng xe khi nhận và trả, kèm theo hình ảnh. Bảng damage_reports đã có trong cơ sở dữ liệu.
- **Đánh giá và xếp hạng:** Cho phép khách hàng đánh giá trải nghiệm thuê xe, giúp doanh nghiệp cải thiện chất lượng dịch vụ. Bảng reviews và ratings đã được thiết kế.

5.4.2. Ứng dụng trí tuệ nhân tạo và học máy

Khi hệ thống đã có khả năng thu thập dữ liệu vận hành liên tục và khá ổn định, giá trị tiếp theo không chỉ nằm ở việc “nhìn thấy dữ liệu”, mà ở việc biến dữ liệu đó thành dự báo và khuyến nghị. Vì vậy, trí tuệ nhân tạo và học máy là một hướng mở rộng hợp lý để tăng chiều sâu cho hệ thống:

Phân tích hành vi lái xe:

Sử dụng dữ liệu gia tốc kế (IMU), tốc độ, RPM và GPS để phân tích và chấm điểm hành vi lái xe. Các chỉ số có thể tính toán bao gồm: tần suất phanh gấp, gia tốc đột ngột, tốc độ vào cua, vượt quá tốc độ. Kết quả phân tích giúp doanh nghiệp đánh giá mức độ an toàn của tài xế và điều chỉnh chính sách cho thuê (phí bảo hiểm, mức đặt cọc).

Bảo trì dự đoán:

Phân tích xu hướng dữ liệu OBD2 theo thời gian (nhiệt độ động cơ, áp suất dầu, điện áp ắc quy, mã lỗi DTC) để dự đoán các vấn đề kỹ thuật trước khi xảy ra hư hỏng. Mô hình học máy có thể được huấn luyện từ dữ liệu lịch sử để cảnh báo sớm về nhu cầu bảo trì, giảm thời gian xe không hoạt động, tức thời gian xe phải nằm chờ sửa và không tạo ra giá trị sử dụng) và chi phí sửa chữa khẩn cấp.

Phát hiện bất thường:

Sử dụng các thuật toán học máy không giám sát, tức cho hệ thống tự học mẫu dữ liệu bình thường rồi phát hiện điểm lệch, để phát hiện các hành vi bất thường: xe đi vào khu vực không bình thường, thời gian sử dụng bất thường, thay đổi đột ngột trong mẫu sử dụng. Điều này giúp phát hiện sớm các tình huống rủi ro như chiếm đoạt xe hoặc sử

dụng sai mục đích.

5.4.3. Ứng dụng di động (Mobile Application)

Hiện tại hệ thống chỉ có giao diện web (Tracking_Frontend). Tuy nhiên, trong bối cảnh quản lý phương tiện, nhu cầu nhận cảnh báo và theo dõi từ thiết bị di động là rất tự nhiên. Vì vậy, việc phát triển ứng dụng di động là bước mở rộng hợp lý tiếp theo:

- **Phương án đề xuất:** Flutter theo hướng ứng dụng lai (Tracking_Mobile/). Cách làm này có thể hiểu là tạo một vỏ ứng dụng di động mỏng bằng Flutter, rồi nhúng giao diện web hiện có vào bên trong bằng WebView, tức một khung hiển thị nội dung web ngay trong ứng dụng. Phương án này giúp rút ngắn thời gian phát triển nhờ tái sử dụng giao diện web, nhưng vẫn cho phép truy cập các tính năng riêng của điện thoại như thông báo đẩy, GPS hay camera.
- **Tính năng chính:** Theo dõi vị trí xe trên bản đồ, nhận thông báo đẩy khi có cảnh báo, xem lịch sử hành trình, quản lý đặt xe (giai đoạn 2).
- **Nền tảng:** Hỗ trợ đồng thời iOS và Android từ một bộ mã nguồn Flutter duy nhất.

5.4.4. Tính toán biên (Edge Computing)

Với khả năng xử lý của ESP32-S3 (chip hai lõi Xtensa LX7, 240 MHz, 8MB PSRAM), một hướng mở rộng đáng chú ý là chuyển bớt phần phân tích xuống ngay trên thiết bị. Cách tiếp cận này giúp hệ thống phản ứng sớm hơn và giảm phụ thuộc vào băng thông mạng:

- **Phát hiện bất thường tại thiết bị:** Thay vì gửi toàn bộ dữ liệu thô lên máy chủ để phân tích, thiết bị có thể chạy mô hình học máy nhẹ (TensorFlow Lite Micro, tức phiên bản rút gọn để chạy trên vi điều khiển) để phát hiện bất thường ngay tại chỗ. Chỉ gửi cảnh báo khi phát hiện bất thường, giảm lượng dữ liệu truyền và tiết kiệm băng thông 4G.
- **Phân loại hành vi lái xe tại thiết bị:** Sử dụng dữ liệu IMU và OBD2 để phân loại hành vi lái xe (bình thường, hung hãn, mệt mỏi) ngay trên ESP32-S3, gửi kết quả phân loại thay vì dữ liệu thô.
- **Nén dữ liệu thông minh:** Sử dụng thuật toán nén dữ liệu trên thiết bị, chỉ gửi dữ liệu khi có thay đổi đáng kể (dead reckoning, tức chỉ cập nhật khi quỹ đạo hoặc trạng thái lệch đủ nhiều so với lần gần nhất), giảm 50–70% lượng dữ liệu truyền.

5.4.5. Triển khai vận hành thực tế

Để chuyển từ môi trường phát triển sang sản xuất, hệ thống không chỉ cần “chạy được ổn định hơn” mà còn phải đạt mức sẵn sàng cao hơn về bảo mật, giám sát và tự động hóa triển khai. Vì vậy, các bước chuyển đổi trọng tâm gồm:

- **Điều phối nhiều dịch vụ tự động:** Khi số thành phần tăng, Docker Compose sẽ khó đáp ứng tốt các nhu cầu tự khởi động lại, tăng giảm tải và cập nhật không gián đoạn. Vì vậy, giai đoạn vận hành thật nên chuyển sang Kubernetes (K8s), tức công cụ chuyên quản lý nhiều dịch vụ trên nhiều máy chủ, để mỗi dịch vụ có thể tăng số bản sao khi tải cao và tự phục hồi khi có lỗi.
- **Tự động kiểm tra và triển khai (CI/CD):** Thiết lập quy trình tự động để mỗi lần cập nhật mã nguồn đều được kiểm tra lỗi cú pháp, kiểm thử, đóng gói và đưa lên môi trường thử nghiệm trước. Chỉ sau khi đạt yêu cầu mới triển khai lên môi trường vận hành thật. Cách làm này giảm mạnh rủi ro lỗi phát sinh do thao tác thủ công.
- **Mã hóa toàn bộ đường truyền:** Toàn bộ luồng trao đổi dữ liệu cần được bảo vệ bằng HTTPS cho giao diện web và API, TLS cho MQTT và kết nối mã hóa tới cơ sở dữ liệu. Nginx Proxy Manager có thể đảm nhiệm vai trò cổng vào chung và quản lý chứng chỉ số để việc triển khai nhất quán hơn.
- **Giám sát và cảnh báo tập trung:** Cần hoàn thiện khả năng quan sát hệ thống, tức phải nhìn ra dịch vụ nào đang khỏe, chậm hay lỗi ở đâu. Điều này bao gồm việc xuất số liệu vận hành theo chuẩn mà công cụ giám sát có thể đọc được (Prometheus-compatible metrics, tức định dạng số liệu mà nhiều công cụ giám sát phổ biến có thể thu thập), lưu log tập trung, hiển thị màn hình theo dõi và gửi cảnh báo qua Telegram hoặc email khi có sự cố.

5.4.6. Phần cứng phiên bản 2 (Hardware v2)

Phiên bản phần cứng hiện có sử dụng bo mạch 2 lớp do nhóm thiết kế và được lắp ráp thủ công, đủ để triển khai và đánh giá ở quy mô đồ án. Tuy nhiên, nếu nhìn dưới góc độ sản phẩm, đây mới là bước khởi đầu. Để tiến tới sản xuất hàng loạt, các cải tiến phần cứng tiếp theo nên tập trung vào tối ưu DFM, tức thiết kế sao cho dễ sản xuất, và EMI là khả năng giảm nhiễu điện từ, cùng với độ gọn cơ khí và độ tin cậy:

- **Thiết kế PCB tùy chỉnh:** Thiết kế PCB 4 lớp, tức bo mạch có nhiều lớp dẫn hơn để đi dây gọn và ổn định hơn, tích hợp toàn bộ thành phần (ESP32-S3, mạch nạp, mạch nguồn, đầu nối SIM, đầu nối antenna) trên một board duy nhất. Giảm kích thước xuống khoảng 60x40 mm, phù hợp để lắp đặt trong xe.
- **Antenna tích hợp:** Sử dụng antenna ceramic cho GPS/GNSS và antenna PCB cho 4G/LTE, giảm số dây cáp và tăng độ tin cậy.
- **Vỏ hộp công nghiệp:** Thiết kế vỏ hộp nhựa ABS chống nước (IP65), chịu nhiệt, với đầu nối OBD2 tích hợp và đầu nối nguồn 12V.
- **Chi phí sản xuất hàng loạt:** Khi sản xuất từ 500 bộ trở lên, chi phí BOM có thể giảm xuống còn 500.000–800.000 VND/bộ nhờ mua linh kiện số lượng lớn và tối ưu hóa thiết kế PCB.

5.4.7. Tóm tắt lộ trình phát triển

Bảng 5.10: Lộ trình phát triển khuyến nghị

Giai đoạn	Thời gian	Nội dung chính	Ưu tiên
Giai đoạn 1.5 - Bảo mật	1–2 tháng	TLS cho MQTT, chứng chỉ riêng cho thiết bị, cơ chế phát hiện treo tự động và nhật ký an ninh	Cao
Giai đoạn 2 - Mở rộng tính năng	2–4 tháng	Đặt xe, thanh toán, báo cáo hư hỏng, đánh giá và ứng dụng di động	Cao
Giai đoạn 2.5 - Tối ưu hóa	1–2 tháng	Lưu đệm Redis, tối ưu truy vấn, tái sử dụng kết nối và kiểm tra sức khỏe dịch vụ	Trung bình
Giai đoạn 3 - Trí tuệ nhân tạo và học máy	3–6 tháng	Phân tích hành vi lái xe, bảo trì dự đoán và phát hiện bất thường	Trung bình
Giai đoạn 4 - Vận hành thực tế quy mô lớn	2–3 tháng	Kubernetes, CI/CD, giám sát tập trung và TLS trên toàn tuyến	Cao
Giai đoạn 5 - Phần cứng v2	4–6 tháng	PCB tùy chỉnh, vỏ hộp công nghiệp và chuẩn bị sản xuất hàng loạt	Thấp (tùy nhu cầu)

Kết luận chương 5

Kết quả đánh giá cho thấy hệ thống đã đáp ứng phần lớn các mục tiêu kỹ thuật trọng yếu của một thiết bị giám sát phương tiện có khả năng vận hành thực tế. Ở tầng phần cứng, mức tiêu thụ điện duy trì trong ngưỡng bối cảnh thiết kế; ở tầng firmware, máy trạng thái vận hành ổn định và luồng OTA, tức cập nhật phần mềm từ xa, cơ bản đã được tích hợp; ở tầng máy chủ, độ trễ vẫn đáp ứng yêu cầu theo dõi thời gian thực. Về chi phí, BOM khoảng 1.514.000 VND cùng chi phí vận hành thấp tạo lợi thế rõ rệt so với nhiều giải pháp thương mại cùng phân khúc.

Điểm quan trọng rút ra từ chương này là kiến trúc tổng thể của đề tài đã đi đúng hướng; phần việc còn lại chủ yếu là làm hệ thống "cứng cáp" hơn cho vận hành dài hạn, thay vì phải thay đổi lại từ gốc. Vì vậy, trọng tâm giai đoạn tiếp theo không phải thay đổi kiến trúc tổng thể, mà là tăng chiều sâu làm cứng vận hành: cơ chế tự phát hiện treo hệ thống (watchdog), bảo mật MQTT/TLS và kiểm thử vận hành dài hạn. Trên nền tảng đó, Chương 6 sẽ nhìn lại toàn bộ quá trình thực hiện dưới góc độ học thuật và bài học kinh nghiệm.

CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM – REFLECTION AND LESSONS LEARNED

6.1. Ứng dụng kiến thức kỹ thuật – Application of prior coursework

Dự án "IoT Vehicle Tracking System" là kết quả của quá trình tổng hợp kiến thức từ nhiều học phần và nhiều lớp công nghệ khác nhau. Từ phần cứng, firmware đến dịch vụ máy chủ và giao diện web, mỗi quyết định triển khai đều đòi hỏi người thực hiện chuyển kiến thức nền tảng thành lựa chọn kỹ thuật cụ thể. Vì vậy, chương này không chỉ mang tính tổng kết, mà còn là phần nhìn lại xem kiến thức đã học được vận dụng như thế nào khi đi vào một bài toán thực tế có nhiều ràng buộc đồng thời.

6.1.1. Vi xử lý và Vi điều khiển

Trong toàn bộ đồ án, lớp kiến thức về vi xử lý và vi điều khiển là nơi mọi quyết định kỹ thuật bắt đầu trở nên cụ thể. Chính ở lớp này, các khái niệm học thuật như thanh ghi, bus ngoại vi hay đồng bộ tài nguyên không còn là lý thuyết riêng lẻ, mà trở thành công cụ trực tiếp để tạo ra một thiết bị có thể vận hành ngoài thực tế. Các nội dung được áp dụng bao gồm:

- **Lập trình cho vi điều khiển ESP32-S3:** Nhóm phải vận dụng kiến thức về cách một vi điều khiển hai nhân hoạt động ở mức thấp, từ bộ nhớ, thanh ghi đến cách chương trình tương tác với phần cứng. ESP32-S3 dùng kiến trúc Xtensa LX7, nhưng điều quan trọng với đồ án là hiểu rõ cách con chip này vận hành để tối ưu hiệu suất và tiêu thụ năng lượng.
- **FreeRTOS và đồng bộ tài nguyên:** Kiến thức về hệ điều hành thời gian thực được áp dụng để tổ chức các công việc nền, ví dụ khối BLE hoặc luồng giao tiếp với modem. Các khái niệm như mutex hay semaphore được dùng như "khóa" và "tín hiệu chờ" để nhiều phần mềm không cùng truy cập một tài nguyên tại một lúc. Dù firmware không chia thành nhiều nhánh nghiệp vụ lớn, tư duy điều phối và tránh xung đột truy cập vẫn được vận dụng trực tiếp.
- **Giao tiếp với các phần cứng ngoại vi:** Nhóm phải cấu hình nhiều kiểu kết nối phần cứng khác nhau như chân điều khiển bật/tắt (GPIO), đo điện áp analog (ADC), cổng nối tiếp với modem (UART), bus cảm biến (I2C/SPI) và giao tiếp với thẻ nhớ microSD (SDMMC). Việc hiểu rõ từng đường giao tiếp này là điều kiện để modem, cảm biến và khối lưu trữ cùng làm việc ổn định trên một thiết bị nhỏ gọn.

Khi thiết bị đã đọc được phần cứng và tự điều phối được các ngoại vi, câu hỏi kế tiếp là dữ liệu rời thiết bị bằng cách nào và được đưa tới đúng nơi ra sao. Vì vậy, phần sau chuyển sang lớp kiến thức mạng và IoT.

6.1.2. Mạng máy tính và IoT

Nếu phần cứng quyết định thiết bị có thu được dữ liệu hay không, thì kiến thức mạng quyết định dữ liệu đó có đi đúng nơi, đúng lúc và đủ an toàn hay không. Điểm quan trọng nhóm rút ra là: trong hệ thống IoT, “truyền được” chưa đủ; còn phải biết khi nào ưu tiên tốc độ, khi nào ưu tiên độ tin cậy và khi nào cần giữ kết nối hai chiều. Vì vậy, nhóm kiến thức này xuất hiện xuyên suốt từ thiết bị, hệ thống máy chủ đến giao diện thời gian thực:

- **Mô hình TCP/IP:** Giúp nhìn đường đi của dữ liệu theo từng lớp, từ lúc rời thiết bị qua mạng 4G/LTE cho đến khi tới máy chủ. Nhờ đó, nhóm phân biệt được phần nào do firmware chịu trách nhiệm, phần nào do hạ tầng mạng và phần nào do dịch vụ xử lý phía máy chủ đảm nhận.
- **Giao thức MQTT:** MQTT được dùng như kênh trung gian nhận bản tin từ thiết bị rồi chuyển cho dịch vụ cần xử lý. Các mức QoS có thể hiểu là mức ưu tiên giữa nhanh và chắc: dữ liệu vị trí gửi nhiều nên có thể ưu tiên tốc độ, còn cảnh báo và lệnh điều khiển cần được xác nhận rõ ràng hơn. Retain Message giúp hệ thống giữ lại trạng thái mới nhất để bên nhận vào sau vẫn biết tình hình hiện tại, còn LWT là bản tin tự động báo rằng thiết bị vừa mất kết nối.
- **HTTP/REST API:** Nếu MQTT phù hợp cho bản tin từ thiết bị, thì HTTP/REST phù hợp cho phân giao tiếp quản trị giữa giao diện và máy chủ. Cách tổ chức API theo tài nguyên, phương thức và mã phản hồi giúp người phát triển dễ hiểu dữ liệu nào đang được yêu cầu, dễ kiểm thử và dễ mở rộng khi thêm tính năng.
- **WebSocket và giao tiếp thời gian thực:** Kênh WebSocket cho phép máy chủ chủ động gửi dữ liệu mới lên màn hình giám sát mà không cần trình duyệt hỏi lặp đi lặp lại bằng các yêu cầu HTTP. Nhờ vậy, vị trí xe và cảnh báo được cập nhật mượt hơn và ít tốn tài nguyên hơn.
- **Bảo mật mạng:** Mã hóa đường truyền (TLS/SSL), xác thực người dùng, phân quyền và các quy tắc ACL, tức danh sách quyền cho từng thiết bị, để giới hạn mỗi thiết bị chỉ được dùng đúng kênh dữ liệu của mình, không phải phân bổ sung tùy chọn, mà là điều kiện để dữ liệu vận hành không bị nghe lén hoặc giả mạo khi đi qua Internet.

Nếu mạng quyết định dữ liệu đi đúng đường, thì cơ sở dữ liệu quyết định dữ liệu đó được cất đúng chỗ và truy vấn lại đúng mục đích. Vì vậy, phần tiếp theo chuyển sang cách tổ chức lưu trữ.

6.1.3. Cơ sở dữ liệu

Kiến thức cơ sở dữ liệu được vận dụng không chỉ để “lưu được dữ liệu”, mà để tổ chức dữ liệu theo đúng đặc tính sử dụng của hệ thống. Bài học rõ nhất ở đây là: không phải mọi dữ liệu nên được cất cùng một chỗ. Dữ liệu vận hành theo thời gian cần ghi nhiều

và truy vấn theo mốc thời gian; dữ liệu nghiệp vụ lại cần quan hệ chặt, ràng buộc rõ và giao dịch an toàn. Vì vậy, đồ án phải chủ động tách cách lưu trữ thay vì dồn về một cơ sở dữ liệu duy nhất:

- **Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL):** PostgreSQL được dùng cho các dữ liệu cần liên kết chặt chẽ như phương tiện, người dùng, chuyến đi, cảnh báo và vùng giám sát. Các khái niệm như chuẩn hóa, khóa chính, khóa ngoại và ràng buộc toàn vẹn giúp dữ liệu không bị mâu thuẫn khi hệ thống vận hành lâu dài.
- **Cơ sở dữ liệu chuỗi thời gian (VictoriaMetrics):** Đây là nơi lưu dữ liệu thay đổi liên tục theo thời gian như vị trí, điện áp, trạng thái nguồn hay rung động. Cơ sở dữ liệu chuỗi thời gian phù hợp hơn cơ sở dữ liệu quan hệ trong kiểu bài toán này vì ghi nhiều, nén tốt và truy vấn theo mốc thời gian nhanh hơn.
- **Tối ưu hóa truy vấn:** Khi dữ liệu tăng dần, việc “có bảng” là chưa đủ; còn phải truy vấn đủ nhanh để màn hình giám sát không bị chậm. Vì vậy, nhóm áp dụng chỉ mục, chỉnh lại câu lệnh và xem cách cơ sở dữ liệu thực sự chạy truy vấn để giữ độ trễ trong ngưỡng chấp nhận được.

Khi cách lưu trữ đã rõ, bài toán còn lại là tổ chức mã nguồn sao cho các quy tắc nghiệp vụ, luồng dữ liệu và truy vấn không đan vào nhau quá mức. Đó là lý do phần tiếp theo nói về các nguyên tắc lập trình hướng đối tượng.

6.1.4. Lập trình hướng đối tượng

Các nguyên tắc lập trình hướng đối tượng (OOP) không được áp dụng như một khuôn mẫu hình thức, mà như một cách giữ cho hệ thống nhiều tầng vẫn duy trì được sự rõ ràng khi mở rộng. Điều này thể hiện rõ nhất ở các lớp máy chủ và giao diện:

- **TypeScript/JavaScript:** TypeScript được dùng như lớp mô tả rõ dữ liệu nào được phép đi qua từng tầng. Các khái niệm như interface, generic hay union type được dùng để diễn đạt “dữ liệu nào hợp lệ, dữ liệu nào không”, từ đó giảm lỗi do hiểu sai cấu trúc yêu cầu gửi lên máy chủ, phản hồi trả về hoặc bản tin dữ liệu vận hành.
- **Domain-Driven Design (DDD):** DDD ở đây có thể hiểu đơn giản là chia mã nguồn theo đúng bài toán nghiệp vụ. Thay vì gom phần nhận yêu cầu, xử lý nghiệp vụ và truy vấn dữ liệu vào một chỗ, hệ thống tách thành các miền như xe, thiết bị, dữ liệu vận hành, cảnh báo và vùng giám sát. Cách này giúp người đọc lần theo luồng nghiệp vụ dễ hơn và giảm tác động dây chuyền khi sửa một phần.
- **Design Patterns:** Các mẫu thiết kế được dùng như những cách tổ chức quen thuộc để code dễ đọc hơn. Chẳng hạn, Repository Pattern tách phần đọc/ghi dữ liệu ra khỏi logic nghiệp vụ, Middleware Pattern chia một yêu cầu thành từng bước như xác thực và ghi log, còn Observer Pattern phù hợp với tình huống một sự kiện mới cần báo cho nhiều nơi cùng lúc.

Tuy nhiên, các nguyên tắc tổ chức phần mềm vẫn chưa giải quyết trực tiếp bài toán điện áp, nguồn dự phòng và giao tiếp cảm biến trên bo mạch. Vì vậy, phần tiếp theo chuyển sang lớp kiến thức điện tử.

6.1.5. Điện tử số và Analog

Kiến thức điện tử là phần giúp đỡ án thoát khỏi phạm vi của một hệ thống phần mềm thuần túy. Nhờ lớp kiến thức này, các yêu cầu như bảo vệ ắc quy, đọc điện áp hay duy trì nguồn dự phòng mới được chuyển thành mạch điện cụ thể:

- **Thiết kế mạch quản lý nguồn:** Áp dụng kiến thức về mạch hạ áp (giảm điện áp ắc quy xe 12V hoặc 24V xuống 3.3V/5V), mạch nâng áp (tăng điện áp từ pin 3.7V lên 5V), và cách tổ chức đường cấp nguồn để thiết kế hệ thống cấp nguồn đa đầu vào (ắc quy xe + pin dự phòng).
- **Đọc giá trị ADC:** Sử dụng kiến thức về bộ chuyển đổi tương tự - số (ADC) để đọc điện áp ắc quy xe thông qua mạch chia áp, tính toán độ phân giải và sai số.
- **Giao tiếp cảm biến và bus nhớ:** Áp dụng kiến thức về giao diện SPI/I2C để giao tiếp với cảm biến gia tốc LIS3DH, cấu hình các thanh ghi điều khiển, đọc dữ liệu gia tốc 3 trục, và thiết lập ngắt (interrupt) cho phát hiện chuyển động. Đồng thời, bus SDMMC 4-bit được khai thác để ghi dữ liệu cục bộ lên microSD với yêu cầu tuân thủ điện trở kéo lên phần cứng, trình tự gắn/tháo bộ nhớ hợp lệ và tính toán vẹn dữ liệu khi mất nguồn.

Khi phần cứng và phần mềm đã gặp nhau trên cùng một thiết bị, dự án còn cần một lớp kỹ thuật để quản lý mã nguồn, đóng gói dịch vụ và giữ quá trình phát triển không rơi vào hỗn loạn. Đó là vai trò của kỹ thuật phần mềm ở mục kế tiếp.

6.1.6. Kỹ thuật phần mềm

Các phương pháp và công cụ kỹ thuật phần mềm là lớp keo gắn toàn bộ đồ án lại với nhau. Chúng không trực tiếp tạo ra tính năng, nhưng quyết định hệ thống có thể được phát triển, kiểm thử và mở rộng một cách có tổ chức hay không:

- **Quản lý mã nguồn bằng Git:** Git không chỉ là lưu lại lịch sử thay đổi, mà còn là cách để nhiều người cùng làm việc mà không ghi đè lên nhau. Việc chia nhánh, đặt quy ước commit và review code giúp quá trình phát triển có tổ chức hơn.
- **Docker và cách đóng gói dịch vụ thành container:** Kiến thức về container hóa được dùng để đóng gói từng dịch vụ như API phía máy chủ, giao diện web, broker MQTT hay cơ sở dữ liệu theo cùng một cách, nhờ đó môi trường phát triển và triển khai ít bị lệch nhau hơn.
- **Quy trình kiểm tra và triển khai tự động (CI/CD):** Hiểu biết về quy trình tự động kiểm tra và triển khai giúp nhóm thiết kế cấu trúc dự án sao cho mỗi thay đổi đều

có thể được kiểm tra, đóng gói và đưa lên môi trường chạy thử một cách lặp lại.

- **Cách làm phát triển linh hoạt (Agile):** Phương pháp phát triển linh hoạt được áp dụng theo các vòng ngắn, ưu tiên phần việc tạo giá trị trước và điều chỉnh kế hoạch dựa trên phản hồi thực tế thay vì khóa chặt từ đầu.

6.2. Giải quyết các vấn đề kỹ thuật phức tạp – Complex engineering problems

Phần trước cho thấy kiến thức nền đã được vận dụng như thế nào. Phần này đi thêm một bước: nhìn vào những bài toán khó thật sự mà nhóm đã phải giải quyết trong quá trình triển khai. Việc nêu ra các vấn đề này không chỉ để mô tả thách thức, mà để làm rõ cách tư duy kỹ thuật đã thay đổi khi đề án chuyển từ mức ý tưởng sang mức hệ thống vận hành được.

6.2.1. Vấn đề 1: Phân tích bản tin OBD2 đa khung qua BLE

Mô tả vấn đề:

Thoạt nhìn, việc lấy dữ liệu OBD2 qua BLE có vẻ chỉ là một bước tích hợp ngoại vi. Tuy nhiên, khi triển khai thực tế, đây lại là một trong những thử thách kỹ thuật khó nhất của đề án vì phải làm việc với thiết bị bên thứ ba có tài liệu hạn chế. Vấn đề cụ thể bao gồm:

- Giao thức BLE của vgate iCar Pro không có tài liệu chính thức công khai. Thông tin giao tiếp (UUID dịch vụ, characteristic, định dạng bản tin) phải được lần ngược và kiểm chứng từ các ứng dụng mã nguồn mở và bản ghi Bluetooth.
- Bản tin OBD2 có thể kéo dài qua nhiều khung dữ liệu BLE, đặc biệt với các lệnh như đọc mã lỗi DTC (Mode 03) hoặc dữ liệu động cơ nhiều PID. Việc ghép nối các khung dữ liệu cần tuân theo ISO 15765–2 (ISO-TP), một giao thức không được tài liệu OBD2 phổ thông đề cập chi tiết.
- Các nguồn tài liệu trực tuyến thường mâu thuẫn nhau về cách xử lý phản hồi nhiều khung: một số hướng dẫn chỉ áp dụng cho ELM327 (chip thông dịch lệnh), không tương thích trực tiếp với vgate iCar Pro sử dụng chip STN1110.

Cách giải quyết:

1. *Nghiên cứu và khảo sát:* Phân tích mã nguồn của các dự án mã nguồn mở tương tự (esp32-obd2-meter, python-OBD), nghiên cứu tài liệu ISO 15765–2, và sử dụng ứng dụng nRF Connect để bắt và phân tích các bản tin BLE giữa điện thoại và vgate iCar Pro.
2. *Thiết kế lớp trừu tượng:* Xây dựng module phân tích OBD2 với khả năng xử lý cả bản tin gọn trong một khung và bản tin kéo dài qua nhiều khung, bao gồm khung điều khiển luồng và các khung dữ liệu tiếp theo theo chuẩn ISO-TP.
3. *Kiểm thử lặp đi lặp lại:* Tạo bộ test với các PID OBD2 phổ biến (Mode 01: RPM,

Speed, Coolant Temp, Fuel Level) và các lệnh nhiều khung (Mode 03: DTC, Mode 09: VIN) để đảm bảo tính chính xác.

Bài học rút ra: Khi làm việc với thiết bị bên thứ ba không có tài liệu rõ ràng, việc kết hợp phân tích ngược cách giao tiếp, tham khảo nhiều nguồn và kiểm thử kỹ lưỡng là phương pháp hiệu quả nhất.

Sau bài toán hiểu đúng dữ liệu từ xe, khó khăn tiếp theo không còn nằm ở từng bản tin đơn lẻ mà ở việc đưa toàn bộ luồng dữ liệu đó đi trọn hành trình đến máy chủ và giao diện. Vì vậy, mục sau chuyển sang đường ống dữ liệu thời gian thực.

6.2.2. Vấn đề 2: Đường ống dữ liệu thời gian thực với đảm bảo phân phối

Mô tả vấn đề:

Đường ống dữ liệu thời gian thực có thể hiểu là toàn bộ hành trình từ lúc thiết bị phát bản tin đến lúc dữ liệu xuất hiện trên màn hình giám sát hoặc được lưu trong cơ sở dữ liệu. Phần khó của bài toán này là ba yêu cầu thường kéo theo các hướng ngược nhau: muốn nhanh thì dễ bỏ sót, muốn chắc chắn thì dễ tăng chi phí, còn muốn lưu thật nhiều thì hệ thống dễ nặng lên. Trong đồ án, hệ thống phải xử lý luồng dữ liệu vận hành liên tục từ nhiều thiết bị IoT (GPS, trạng thái nguồn, IMU; sẵn sàng mở rộng thêm OBD2) với các yêu cầu:

- Dữ liệu phải được cất đồng thời vào hai nơi khác nhau: một nơi chuyên lưu dữ liệu theo thời gian để vẽ biểu đồ và xem lịch sử dày đặc, một nơi lưu dữ liệu nghiệp vụ có quan hệ rõ ràng. Đây chính là bài toán một lần nhận nhưng phải ghi ra hai nơi (dual-write): phải lưu phục vụ hai mục đích sử dụng khác nhau.
- Khi mất kết nối mạng 4G, hệ thống phải khôi phục đường gửi dữ liệu đủ nhanh để luồng vận hành không bị gãy; nếu mất sóng quá lâu thì thiết bị phải có chỗ giữ tạm dữ liệu để gửi lại sau.
- Mức đảm bảo gửi nhận của MQTT (QoS) phải được chọn phù hợp cho từng loại dữ liệu: dữ liệu gửi liên tục có thể ưu tiên nhanh hơn, còn cảnh báo và lệnh điều khiển phải ưu tiên chắc chắn hơn.

Cách giải quyết:

1. *Dịch vụ trung gian riêng cho luồng dữ liệu:* Tracking_MqttBridge được thiết kế như một lớp đứng giữa broker MQTT và các nơi lưu trữ. Dịch vụ này nhận dữ liệu từ thiết bị, ghi dữ liệu theo thời gian sang VictoriaMetrics, ghi log sang VictoriaLogs và cập nhật PostgreSQL cho các trạng thái quan trọng. Nhờ tách riêng, hệ thống dễ kiểm soát hơn chỗ nào chỉ lo tiếp nhận dữ liệu, chỗ nào lo nghiệp vụ người dùng.
2. *Tự phục hồi kết nối trên thiết bị kết hợp lưu tạm cục bộ:* Thiết bị tự kết nối lại khi có mạng, duy trì tín hiệu giữ kết nối của MQTT và lưu các cấu hình cần thiết trong NVS, tức vùng nhớ không mất dữ liệu sau khi khởi động lại. Song song đó, dữ liệu

và sự kiện được lưu vào hàng đợi microSD và phát lại theo đúng thứ tự khi kết nối phục hồi, giúp giảm mất mát dữ liệu trong các đợt mất sóng dài. Ở bản hiện tại, hệ thống giả định thẻ nhớ đã được cắm sẵn từ lúc khởi động để giữ ổn định.

3. *Phân mức dữ liệu theo độ quan trọng*: Áp dụng QoS 0 cho dữ liệu vị trí GPS gửi dày, tức ưu tiên tốc độ hơn việc xác nhận từng bản tin; QoS 1 cho cảnh báo và sự kiện quan trọng, tức yêu cầu phía nhận báo đã nhận ít nhất một lần. Cách chia này giữ được cân bằng giữa hiệu suất và độ tin cậy.

Bài học rút ra: Thiết kế đường ống dữ liệu cần xem xét tất cả các tình huống hỏng hóc như mất mạng, máy chủ quá tải hoặc dữ liệu bị lệch nhau từ giai đoạn thiết kế, không để đến giai đoạn tích hợp mới xử lý.

Tuy nhiên, một đường ống dữ liệu tốt chỉ có ý nghĩa khi thiết bị còn đủ năng lượng để duy trì việc thu thập và gửi dữ liệu trong môi trường xe. Vì vậy, vấn đề kế tiếp chuyển sang quản lý nguồn.

6.2.3. Vấn đề 3: Quản lý năng lượng với nhiều nguồn cấp

Mô tả vấn đề:

Quản lý năng lượng là vấn đề không dễ nhận thấy khi mới nhìn hệ thống từ góc độ phần mềm, nhưng lại quyết định trực tiếp độ tin cậy của thiết bị trong xe. Hệ thống phần cứng phải hoạt động với hai nguồn năng lượng có đặc tính rất khác nhau:

- Ắc quy xe 12V hoặc 24V DC (dao động tùy trạng thái sạc và tải), là nguồn chính khi xe hoạt động.
- Pin dự phòng 18650 1S Li-ion 3.7V (dao động 2.8V - 4.2V), là nguồn dùng khi ắc quy xe bị ngắt hoặc điện áp quá thấp.
- Việc chuyển đổi giữa hai nguồn phải diễn ra liền mạch, nghĩa là đổi nguồn nhưng thiết bị không bị sụt điện, tránh reset hoặc mất dữ liệu.

Cách giải quyết:

1. *Thiết kế đường cấp nguồn*: Thiết kế mạch đường cấp nguồn dùng diode OR giữa MP2482 (5V chính) và SX1308 (5V dự phòng), kết hợp GPIO18 để điều khiển nhánh nguồn theo profile 12V/24V.
2. *Low Voltage Disconnect (LVD)*: Hiện thực LVD bằng khối LVD phần cứng kết hợp ADC firmware, dùng ngưỡng cho hai hệ điện: 12V ($OFF = 12.0\text{ V}$, $ON = 12.2\text{ V}$) và 24V ($OFF = 24.0\text{ V}$, $ON = 24.4\text{ V}$), bảo vệ ắc quy không bị rút cạn quá mức.
3. *Bộ sạc pin dự phòng*: Tích hợp IC sạc TP5100 (input 5V từ MP2482, output 4.2V ở chế độ 1 cell) để sạc pin 18650 1S khi điều kiện nguồn cho phép.
4. *Giám sát điện áp bằng firmware*: Đọc điện áp ắc quy và pin dự phòng liên tục qua ADC, kết hợp trạng thái GPIO19 (HIGH = báo điện áp thấp) để gửi cảnh báo sớm và điều phối chuyển nguồn.

Bài học rút ra: Thiết kế hệ thống năng lượng cho IoT trong môi trường ô tô phải xét đồng thời điện áp dao động, chuyển đổi nguồn liên mạch, bảo vệ ắc quy và khả năng giám sát từ xa. Nếu bỏ sót một mắt xích, độ tin cậy của toàn hệ thống sẽ giảm rõ rệt.

Khi thiết bị đã đứng vững về nguồn, bài toán còn lại là làm sao lớp máy chủ không trở thành điểm nghẽn mới khi số thiết bị và người dùng cùng tăng. Phần tiếp theo vì thế chuyển sang kiến trúc máy chủ có khả năng mở rã»TMng.

6.2.4. Vấn đề 4: Kiến trúc đám mây có khả năng mở rộng cho IoT

Mô tả vấn đề:

Bài toán hạ tầng máy chủ của đề án không dừng ở việc “đưa dữ liệu lên máy chủ”, mà phải giải được ba yêu cầu cùng lúc: nhiều thiết bị, nhiều người dùng và mức an toàn đủ cho dữ liệu vận hành. Vì vậy, hệ thống cần xử lý dữ liệu từ nhiều thiết bị đồng thời, cung cấp giao diện thời gian thực cho nhiều người dùng, và đảm bảo bảo mật:

- Mỗi thiết bị có kênh dữ liệu MQTT riêng (ví dụ `v1/{device_id}/rawdata` và `v1/{device_id}/commands`), nên phải có quy tắc giới hạn quyền (ACL) để thiết bị chỉ được gửi và nhận trên đúng kênh của mình.
- Nhiều người dùng có thể xem cùng một xe trên màn hình giám sát, tạo ra nhiều kết nối WebSocket đồng thời cần được quản lý.
- Cơ chế giữ đăng nhập phải đủ an toàn, có thể thu hồi khi cần, nhưng cũng không được tạo ra nút thắt cổ chai khi số lượng yêu cầu tăng cao.

Cách giải quyết:

1. *Quy tắc giới hạn từng thiết bị:* EMQX được cấu hình để mỗi thiết bị chỉ được gửi và nhận trên đúng kênh dữ liệu của nó, nhờ đó giảm nguy cơ một thiết bị giả mạo dữ liệu của thiết bị khác. Mỗi thiết bị cũng có thông tin đăng nhập riêng.
2. *Nhóm người xem theo từng xe:* Socket.IO rooms, tức các nhóm nhận dữ liệu trong cùng một kênh, được dùng để gom những người đang theo dõi cùng một xe vào một nhóm. Khi có dữ liệu mới của xe đó, máy chủ chỉ đẩy cho đúng nhóm liên quan, tránh phát tán dữ liệu không cần thiết cho toàn bộ người dùng.
3. *Phiên đăng nhập có thể thu hồi ngay:* Hệ thống giữ thông tin phiên trong cơ sở dữ liệu, lưu token dưới dạng mã băm SHA-256 thay vì lưu nguyên văn, để có thể khóa ngay một phiên nếu phát hiện rủi ro. Cách này được ưu tiên hơn JWT trong giai đoạn hiện tại vì dễ kiểm soát việc thu hồi. Ở giai đoạn sau có thể bổ sung Redis cache để giảm tải truy vấn xác thực.
4. *Triển khai tách từng dịch vụ:* Mỗi dịch vụ như API phía máy chủ, giao diện web, MQTT Bridge hay cơ sở dữ liệu đều có cấu hình triển khai riêng, nhưng vẫn nằm trong cùng một mạng nội bộ để giao tiếp với nhau. Cách tách này giúp mở rộng từng phần theo đúng nhu cầu thay vì phải nâng cả hệ thống cùng lúc.

Bài học rút ra: Hạ tầng máy chủ cho IoT cần được thiết kế từ đầu với khả năng mở rộng bằng cách thêm nhiều bản sao dịch vụ khi cần, thay vì dồn toàn bộ tải lên một máy mạnh hơn. Việc tách riêng từng dịch vụ và giới hạn rõ quyền của từng thiết bị là nền tảng cho bảo mật và cho quản lý đội xe ở quy mô lớn.

6.3. Tác động đạo đức và xã hội – Ethical and social impacts

Một hệ thống theo dõi phương tiện không chỉ là bài toán kỹ thuật. Khi hệ thống bắt đầu thu thập dữ liệu vị trí, hành trình và thói quen sử dụng xe, nó đồng thời chạm tới các câu hỏi về quyền riêng tư, trách nhiệm sử dụng dữ liệu và ảnh hưởng xã hội. Vì vậy, việc đánh giá đồ án sẽ chưa đủ nếu chỉ dừng ở hiệu năng kỹ thuật.

6.3.1. Quyền riêng tư và bảo vệ dữ liệu cá nhân

Hệ thống theo dõi phương tiện liên tục thu thập dữ liệu vị trí GPS, hành trình và thói quen sử dụng xe của người lái. Đây đều là dữ liệu nhạy cảm vì chúng không chỉ mô tả phương tiện đang ở đâu, mà còn phản ánh hành vi và nhịp sinh hoạt của con người gắn với phương tiện đó. Vì vậy, quá trình phát triển và triển khai hệ thống cần được xem xét nghiêm túc dưới góc độ đạo đức và bảo vệ dữ liệu.

Các biện pháp bảo vệ quyền riêng tư đã được áp dụng:

Bảng 6.3.1A: Tổng hợp thông tin về STT, Biện pháp, Mô tả chi tiết

STT	Biện pháp	Mô tả chi tiết
1	Mã hóa dữ liệu truyền tải	Dữ liệu MQTT được mã hóa TLS trong môi trường production, ngăn chặn nghe lén (eavesdropping) trên đường truyền
2	Phân quyền truy cập (RBAC)	Hệ thống phân quyền theo vai trò (admin, manager, viewer), chỉ những người có quyền mới xem được dữ liệu cụ thể
3	Chính sách lưu trữ dữ liệu	Dữ liệu telemetry được lưu trữ có thời hạn (retention policy), tự động xóa sau 90 ngày để giảm rủi ro lộ lọt dữ liệu
4	Session-based authentication	Token xác thực được hash SHA-256, lưu trữ trong database, có khả năng thu hồi ngay lập tức khi cần
5	MQTT ACL per device	Mỗi thiết bị chỉ truy cập được dữ liệu của mình, ngăn chặn truy cập trái phép dữ liệu thiết bị khác

Khuyến nghị bổ sung (chưa hiện thực):

- Thông báo rõ ràng cho người thuê xe về việc xe được giám sát (transparency).
- Cho phép người thuê xe xem dữ liệu của mình (data access rights).
- Tuân thủ Luật An toàn thông tin mạng Việt Nam (Luật số 24/2018/QH14) và các quy định về bảo vệ dữ liệu cá nhân.

Bảo vệ dữ liệu là điều kiện cần, nhưng với một thiết bị gắn trực tiếp lên xe thì vẫn còn một câu hỏi khác quan trọng không kém: hệ thống có làm ảnh hưởng đến an toàn vận hành hay không. Vì vậy, mục tiếp theo chuyển sang trách nhiệm kỹ thuật đối với phương tiện.

6.3.2. An toàn phương tiện và trách nhiệm kỹ thuật

Việc truy cập hệ thống OBD2 của xe đặt ra một câu hỏi quan trọng hơn chính bản thân khả năng đọc dữ liệu: thiết bị theo dõi có vô tình làm ảnh hưởng đến độ an toàn vận hành của xe hay không? Đây là lý do các quyết định thiết kế ở phần này phải ưu tiên nguyên tắc an toàn trước khi tối đa hóa tính năng.

Các biện pháp đảm bảo an toàn đã được áp dụng:

- **Chỉ đọc (read-only) OBD2 PIDs:** Hệ thống chỉ sử dụng các lệnh OBD2 Mode 01 (Current Data) và Mode 03 (Stored DTCs) để đọc dữ liệu. Tuyệt đối không gửi các lệnh ghi (write commands) hoặc lệnh điều khiển động cơ (Mode 04: Clear DTCs, Mode 08: Control operations). Đây là nguyên tắc an toàn cơ bản khi làm việc với hệ thống chẩn đoán xe.
- **Kết nối BLE gián tiếp:** Sử dụng OBD2 adapter vgate iCar Pro (thiết bị đã được chứng nhận an toàn) làm lớp trung gian, không kết nối trực tiếp vào bus CAN của xe. Điều này giảm rủi ro gây nhiễu hoặc xung đột trên bus truyền thông nội bộ xe.
- **Không điều khiển xe từ xa:** Hệ thống không có chức năng điều khiển động cơ (tắt bơm xăng, khóa xe). Đây là quyết định thiết kế có chủ đích để tránh rủi ro an toàn nghiêm trọng.

Khi các giới hạn an toàn đã được chốt rõ, mới có thể đánh giá công bằng mặt tích cực mà hệ thống mang lại cho doanh nghiệp và xã hội. Phần tiếp theo vì thế chuyển sang tác động xã hội tích cực.

6.3.3. Tác động xã hội tích cực

Ở chiều ngược lại, nếu được triển khai đúng cách, hệ thống IoT giám sát phương tiện cũng có thể tạo ra những giá trị xã hội rõ ràng chứ không chỉ là công cụ kiểm soát. Một số tác động tích cực có thể nhận thấy gồm:

Hỗ trợ doanh nghiệp nhỏ và vừa:

- Cung cấp giải pháp quản lý đội xe với chi phí hợp lý (dưới 2.000.000 VND/thiết bị,

chi phí 4G khoảng 70.000 VND/tháng), giúp các doanh nghiệp cho thuê xe nhỏ có thể tiếp cận công nghệ giám sát mà trước đây chỉ dành cho các công ty lớn.

- Giảm thiệt hại do mất trộm, sử dụng sai mục đích, và hư hỏng thiết bị nhờ phát hiện sớm.

Giảm tai nạn giao thông:

- Giám sát tốc độ và hành vi lái xe (phanh gấp, tăng tốc đột ngột) giúp nhận diện và cảnh báo các hành vi lái xe nguy hiểm.
- Cảnh báo vượt vùng cho phép giúp đảm bảo xe hoạt động trong phạm vi an toàn đã thỏa thuận.

Tối ưu hóa nhiên liệu và giảm phát thải:

- Phân tích dữ liệu OBD2 (tiêu hao nhiên liệu, RPM, tốc độ) giúp đánh giá hiệu suất sử dụng nhiên liệu.
- Dữ liệu hành trình giúp tối ưu hóa tuyến đường, giảm quãng đường chạy không tải, giảm phát thải CO2.

Những lợi ích này không phủ nhận các rủi ro đi kèm. Vì vậy, ngay sau phần tác động tích cực, luận văn chỉ ra rõ các lo ngại cần được quản lý để hệ thống không biến thành công cụ gây áp lực hoặc xâm phạm quyền riêng tư.

6.3.4. Những lo ngại cần xem xét

- **Giám sát quá mức:** Cần cân bằng giữa nhu cầu giám sát của doanh nghiệp và quyền riêng tư của người lái xe. Không nên sử dụng dữ liệu vị trí để theo dõi hoạt động cá nhân ngoài phạm vi hợp đồng cho thuê.
- **Phụ thuộc công nghệ:** Hệ thống cần có cơ chế dự phòng (fallback) khi mất kết nối hoặc thiết bị hỏng, không để việc mất kết nối làm ảnh hưởng đến hoạt động bình thường của xe.
- **Trách nhiệm dữ liệu:** Doanh nghiệp sử dụng hệ thống có trách nhiệm bảo vệ dữ liệu thu thập được, không chia sẻ cho bên thứ ba khi chưa có sự đồng ý của người liên quan.

6.4. Tổng kết và bài học kinh nghiệm – Reflection and Lessons Learned

Nếu Chương 5 đánh giá hệ thống bằng số liệu và Chương 6.1–6.3 nhìn lại kiến thức, thách thức và tác động xã hội, thì mục này là phần khép lại toàn bộ hành trình thực hiện đề án. Trọng tâm ở đây không phải kể lại những gì đã làm, mà rút ra những điều thực sự có giá trị cho các giai đoạn phát triển sau.

6.4.1. Tầm quan trọng của thiết kế kiến trúc trước khi lập trình

Một trong những bài học quan trọng nhất của dự án là phải ưu tiên thiết kế kiến trúc hệ thống trước khi bắt tay vào lập trình. Bài học này không đến từ lý thuyết, mà đến từ chính giai đoạn đầu của đồ án khi nhóm từng có xu hướng triển khai nhanh từng mô-đun trước khi xây dựng xong mô hình tương tác tổng thể.

Khi phát sinh các lỗi tích hợp như hai mô-đun hiểu khác nhau về cùng một dữ liệu, định dạng truyền nhận không thống nhất hoặc các phần phụ thuộc chéo lẫn nhau, nhóm chuyển sang cách tiếp cận có kỷ luật hơn: xác lập sơ đồ kiến trúc và luồng dữ liệu, chốt quy ước đầu vào/đầu ra của API trước khi hiện thực, hoàn thiện sơ đồ thực thể - quan hệ trước khi tạo bảng, chuẩn hóa cách các dịch vụ nhìn thấy nhau và dùng cổng nào, đồng thời mô tả rõ các tình huống hỏng hóc có thể xảy ra ở nhánh lưu trữ cục bộ như mất mạng, đầy dung lượng, gắn thẻ nhớ thất bại hoặc xác nhận nhận dữ liệu đến quá chậm trước khi tối ưu hiệu năng.

Cách làm này giúp giảm đáng kể thời gian sửa lỗi tích hợp và hạn chế khối lượng công việc phải làm lại ở các giai đoạn sau.

6.4.2. Độ phức tạp của hệ thống IoT nhiều tầng

Dự án cho thấy một hệ thống IoT nhiều tầng phức tạp hơn đáng kể so với một ứng dụng web thông thường. Độ phức tạp không nằm ở từng lớp riêng lẻ, mà ở chỗ mỗi lớp có ràng buộc riêng và các ràng buộc đó chỉ bộc lộ rõ khi ghép lại thành một chuỗi vận hành hoàn chỉnh. Hệ thống này đồng thời chạm vào bốn lớp công việc khác nhau:

Bảng 6.4.2A: Tổng hợp thông tin về Tầng, Ngôn ngữ / Công nghệ, Thách thức chính

Tầng	Ngôn ngữ / Công nghệ	Thách thức chính
Phần cứng	Thiết kế mạch, PCB	Dễ phát sinh lỗi nguồn hoặc tín hiệu, khó đo đạc và khó tìm đúng điểm hỏng
Firmware	C/C++ trên ESP32-S3, bộ điều phối công việc nền	Phải giữ bộ nhớ ổn định, canh thời gian chính xác và tránh xung đột truy cập
Backend	TypeScript, dịch vụ API	Phải tổ chức dữ liệu lớn, giữ phản hồi ổn định và bảo vệ truy cập
Frontend	TypeScript, dashboard web	Phải giữ giao diện dễ dùng, cập nhật dữ liệu mới liên tục mà không giật lag

Mỗi tầng đòi hỏi một nhóm kỹ năng chuyên môn riêng, và các lỗi thường xuất hiện nhiều nhất ở giai đoạn tích hợp giữa các tầng. Bài học rút ra là với hệ thống đa tầng, cần thực

hiện kiểm thử tích hợp sớm thay vì chờ đến khi toàn bộ thành phần hoàn tất mới kiểm thử.

Nói cách khác, độ phức tạp của hệ thống không chỉ đến từ số tầng, mà còn đến từ việc phải phối hợp nhiều công cụ trưởng thành với nhau. Từ góc nhìn đó, phần tiếp theo cho thấy vì sao hệ sinh thái mã nguồn mở lại trở thành một lợi thế rõ rệt của đồ án.

6.4.3. Sức mạnh của hệ sinh thái mã nguồn mở

Dự án sử dụng hoàn toàn công nghệ mã nguồn mở. Quá trình triển khai cho thấy sức mạnh của hệ sinh thái này không chỉ nằm ở chi phí bằng 0, mà ở khả năng ghép nối nhiều công cụ trưởng thành thành một hệ thống IoT khá hoàn chỉnh:

- **Nền tảng phát triển firmware cho ESP32-S3 (ESP-IDF):** cung cấp sẵn các thư viện cần thiết để giao tiếp với Wi-Fi, BLE và các ngoại vi phần cứng, nhờ đó nhóm không phải tự xây mọi lớp nền từ đầu.
- **Tầng xử lý yêu cầu web và API (Express.js + TypeScript):** đủ nhẹ để phát triển nhanh nhưng vẫn rõ cấu trúc, đồng thời TypeScript giúp phát hiện sớm nhiều lỗi dữ liệu trước khi chạy thật.
- **Tầng giao diện web (Next.js 15 + React 19):** phù hợp để tổ chức nhiều màn hình giám sát, hiển thị dữ liệu thay đổi liên tục và vẫn giữ trải nghiệm đủ mượt cho người vận hành.
- **PostgreSQL 16:** là nơi lưu dữ liệu nghiệp vụ cần quan hệ chặt chẽ; hỗ trợ PostGIS để xử lý các bài toán không gian như vùng giám sát trên bản đồ.
- **EMQX:** đóng vai trò broker MQTT ổn định giữa thiết bị và máy chủ; có thể mở rộng thêm khi số lượng thiết bị tăng.
- **VictoriaMetrics:** phù hợp cho dữ liệu thay đổi liên tục theo thời gian, ghi nhanh, tốn ít tài nguyên và dùng được với định dạng số liệu chuẩn mà nhiều công cụ giám sát hỗ trợ.
- **Docker:** giúp đóng gói từng dịch vụ theo cách nhất quán để môi trường phát triển và triển khai không bị lệch nhau quá nhiều.

Chi phí giấy phép phần mềm của toàn bộ hệ thống là 0 VND, nên ngân sách có thể tập trung cho phần cứng và hạ tầng máy chủ.

Tuy nhiên, công cụ tốt chỉ tạo ra lợi thế ban đầu; để lợi thế đó biến thành một hệ thống tin cậy, nhóm vẫn phải kiểm thử ở mọi tầng thay vì dựa vào niềm tin rằng các thành phần trưởng thành sẽ tự ghép đúng với nhau.

6.4.4. Kiểm thử ở mọi tầng là bắt buộc™

Dự án cho thấy kiểm thử ở mọi tầng của hệ thống là yêu cầu bắt buộc. Trong một hệ thống nhiều lớp, rất ít lỗi tự lộ ra đúng tại nơi chúng phát sinh; phần lớn chỉ biểu hiện ở

tầng khác. Vì vậy, kiểm thử phải đi theo toàn bộ hành trình dữ liệu:

- **Kiểm thử đơn vị (unit test):** dùng để kiểm tra các hàm nhỏ và các quy tắc xử lý dữ liệu, ví dụ phân tích bản tin OBD2 hoặc tính toán năng lượng trong firmware và dịch vụ máy chủ.
- **Kiểm thử tích hợp (integration test):** dùng để kiểm tra cách các dịch vụ nói chuyện với nhau, ví dụ MQTT Bridge nhận dữ liệu từ EMQX rồi ghi vào VictoriaMetrics, API phía máy chủ đọc/ghi PostgreSQL, hay luồng phát lại dữ liệu đã lưu từ microSD về đúng kênh MQTT.
- **Kiểm thử đầu-cuối (end-to-end):** mô phỏng toàn bộ hành trình dữ liệu từ thiết bị giả lập đến giao diện web để kiểm tra người dùng thực sự nhìn thấy gì trên màn hình giám sát, kể cả trong kịch bản mất mạng rồi phục hồi để xác nhận dữ liệu phát lại.

Các lỗi được phát hiện ở giai đoạn kiểm thử sớm (unit test) có chi phí khắc phục thấp hơn rất nhiều so với lỗi chỉ được phát hiện ở giai đoạn tích hợp hoặc triển khai sản xuất [1].

Nhìn lại toàn bộ dự án, chính quá trình kiểm thử rồi sửa đi sửa lại ở nhiều tầng đã tạo ra phản trường thành nghề nghiệp rõ nhất. Vì vậy, mục cuối cùng chuyển sang tổng kết cá nhân.

6.4.5. Tổng kết cá nhân

Dự án "IoT Vehicle Tracking System" đưa người thực hiện đi trọn một vòng từ ý tưởng, thiết kế mạch, đặt mua linh kiện, chế tạo phần cứng, tích hợp và đánh giá. Giá trị lớn nhất của quá trình này không nằm ở việc ghép đủ các thành phần, mà ở chỗ từng quyết định kỹ thuật đều phải được kiểm chứng lại bằng số liệu, bằng thử nghiệm và bằng mức độ phù hợp với điều kiện triển khai thật. Chính quá trình lặp đi lặp lại giữa giả định và kiểm chứng đó mới là phản trường thành quan trọng nhất mà đồ án mang lại.

Kinh nghiệm rút ra từ dự án, đặc biệt ở các mảng kiến trúc hệ thống, quản lý năng lượng cho thiết bị IoT, bảo mật dịch vụ máy chủ và tổ chức luồng dữ liệu thời gian thực, là nền tảng quan trọng cho công việc chuyên môn sau này.

Kết luận chương 6

Chương 6 khép lại câu chuyện của đồ án bằng một góc nhìn rộng hơn so với các chương trước: xây dựng một hệ thống IoT giám sát phương tiện không phải là việc ghép nối đơn giản giữa phần cứng, firmware và web, mà là quá trình liên tục cân bằng giữa ràng buộc kỹ thuật, nhu cầu vận hành và trách nhiệm dữ liệu. Những bài học rút ra về kiến trúc, kiểm thử tích hợp, quản lý năng lượng, OTA và đạo đức dữ liệu vì thế không chỉ là phần tổng kết cho đồ án hiện tại, mà còn là nền tảng thực tế cho các giai đoạn hoàn thiện tiếp

theo.

Tài liệu tham khảo Chương 6

[1] B. Boehm and V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer*, vol. 34, no. 1, pp. 135–137, 2001.

TÀI LIỆU TRÍCH DẪN - REFERENCES

[A] Tài liệu tiếng Việt

- [1] Bộ Giao thông Vận tải, "Báo cáo thống kê phương tiện giao thông đường bộ Việt Nam giai đoạn 2020–2025," Hà Nội, 2024.
- [2] Nguyễn Văn A và Trần Thị B, "Ứng dụng Internet vạn vật trong giám sát và quản lý phương tiện giao thông tại Việt Nam," *Tạp chí Khoa học và Công nghệ*, tập 58, số 3, trang 45–52, 2023.
- [3] Lê Minh C, "Thiết kế hệ thống định vị và theo dõi phương tiện sử dụng GPS và mạng di động," Luận văn thạc sĩ, Đại học Bách Khoa TP. Hồ Chí Minh, 2022.
- [4] Bộ Thông tin và Truyền thông, "Luật An toàn thông tin mạng," Luật số 24/2018/QH14, Hà Nội, 2018.

[B] Tài liệu tiếng Anh (Books & Papers)

- [5] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, Fourth Quarter 2015.
- [6] N. Naik, "Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP," in *Proc. 2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria, 2017, pp. 1–7.
- [7] M. Quigley, R. Johnson, and P. Sharma, "Power Management Strategies for IoT Vehicle Tracking Devices," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12045–12058, Aug. 2021.
- [8] M. A. Al-Khedher, "Hybrid GPS-GSM Localization of Automobile Tracking System," *International Journal of Computer Science and Information Technology (IJCSIT)*, vol. 3, no. 6, pp. 75–85, Dec. 2011.
- [9] S. Kaur and S. Singh, "A Survey on IoT based Vehicle Tracking System," *International Journal of Advanced Research in Computer Science*, vol. 10, no. 3, pp. 42–47, 2019.
- [10] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.
- [11] B. Boehm and V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer*, vol. 34, no. 1, pp. 135–137, Jan. 2001.

- [12] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [13] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [14] D. Guinard and V. Trifa, *Building the Web of Things*. Shelter Island, NY, USA: Manning Publications, 2016.
- [15] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, Sep. 2013.
- [16] V. C. Gungor and G. P. Hancke, "Industrial Wireless Sensor Networks: Challenges, Design Principles, and Technical Approaches," *IEEE Transactions on Industrial Electronics*, vol. 56, no. 10, pp. 4258–4265, Oct. 2009.
- [17] R. K. Kodali, V. Jain, S. Bose, and L. Boppana, "IoT Based Smart Security and Home Automation System," in *Proc. 2016 International Conference on Computing, Communication and Automation (ICCCA)*, Greater Noida, India, 2016, pp. 1286–1289.
- [18] B. Sahu and G. A. Rincon-Mora, "A Low Voltage, Dynamic, Noninverting, Synchronous Buck-Boost Converter for Portable Applications," *IEEE Transactions on Power Electronics*, vol. 19, no. 2, pp. 443–452, Mar. 2004.

[C] Tài liệu kỹ thuật (Datasheets & Technical Documents)

- [19] Espressif Systems, "ESP32-S3 Series Datasheet," Version 1.3, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [20] Espressif Systems, "ESP32-S3 Technical Reference Manual," Version 1.1, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- [21] SIMCom Wireless Solutions, "SIM7600 Series AT Command Manual," Version 1.06, 2023. [Online]. Available: <https://www.simcom.com/product/SIM7600CE-T.html>
- [22] SIMCom Wireless Solutions, "SIM7600CE-T Module Hardware Design Guide," Version 1.02, 2022.
- [23] STMicroelectronics, "LIS3DH - MEMS Digital Output Motion Sensor Ultra-Low-Power High-Performance 3-Axis 'Nano' Accelerometer Datasheet," DocID 17530, Rev. 3, 2021. [Online]. Available: <https://www.st.com/resource/en/datasheet/lis3dh.pdf>

- [24] STMicroelectronics, "LIS3DH Application Note AN3308," Rev. 2, 2020.
- [25] OASIS, "MQTT Version 5.0 - OASIS Standard," Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [26] SAE International, "SAE J1979 - E/E Diagnostic Test Modes," Standard, Rev. 2014.
- [27] International Organization for Standardization, "ISO 15031–5:2015 - Road vehicles - Communication between vehicle and external equipment for emissions-related diagnostics," ISO Standard, 2015.
- [28] International Organization for Standardization, "ISO 15765–2:2016 - Road vehicles - Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services," ISO Standard, 2016.
- [29] vgate, "iCar Pro BLE OBD2 Adapter Specifications," 2023. [Online]. Available: <https://www.vgatemall.com/product/vgate-icar-pro.html>
- [30] NanJing Top Power ASIC Corp., "TP5100 2A Switching Buck Charger IC for 8.4V/4.2V Lithium-Ion Battery," Datasheet, Rev. 2.4. [Online]. Available: <https://www.toppwr.com/uploadfile/file/20230304/6402f53e7e8d9.pdf>. [Accessed: Apr. 18, 2026].
- [31] Apache NimBLE Project, "NimBLE Host API Reference," Version 1.5, 2023. [Online]. Available: <https://mynewt.apache.org/latest/network/>

[D] Tài liệu trực tuyến (Online Resources)

- [32] EMQX Team, "EMQX Documentation - The World's Most Scalable MQTT Broker," Version 5.x, 2024. [Online]. Available: <https://docs.emqx.com/en/emqx/latest/>. [Accessed: Jan. 15, 2026].
- [33] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Jan. 10, 2026].
- [34] VictoriaMetrics Team, "VictoriaMetrics Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/>. [Accessed: Jan. 10, 2026].
- [35] VictoriaMetrics Team, "VictoriaLogs Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/victorialogs/>. [Accessed: Jan. 10, 2026].
- [36] OpenJS Foundation, "Express.js - Fast, Unopinionated, Minimalist Web Framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>. [Accessed: Dec. 20, 2025].
- [37] Vercel Inc., "Next.js 15 Documentation," 2024. [Online]. Available: <https://nextjs>

.org/docs. [Accessed: Dec. 15, 2025].

[38] Meta Platforms, Inc., "React 19 Documentation," 2024. [Online]. Available: <https://react.dev/>. [Accessed: Dec. 15, 2025].

[39] Socket.IO Contributors, "Socket.IO Documentation," Version 4.8, 2024. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: Dec. 20, 2025].

[40] V. Agafonkin, "Leaflet - An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps," Version 1.9, 2024. [Online]. Available: <https://leafletjs.com/>. [Accessed: Jan. 05, 2026].

[41] Apache Software Foundation, "Apache ECharts - An Open Source JavaScript Visualization Library," Version 5.5, 2024. [Online]. Available: <https://echarts.apache.org/>. [Accessed: Jan. 05, 2026].

[42] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/>. [Accessed: Nov. 15, 2025].

[43] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Nov. 15, 2025].

[44] Tailwind Labs, "Tailwind CSS Documentation," Version 4, 2025. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Jan. 10, 2026].

[45] Daishi Kato, "Zustand - Bear Necessities for State Management in React," Version 5, 2024. [Online]. Available: <https://zustand-demo.pmnd.rs/>. [Accessed: Dec. 20, 2025].

[46] TanStack, "TanStack Query (React Query) Documentation," Version 5, 2024. [Online]. Available: <https://tanstack.com/query/latest>. [Accessed: Dec. 20, 2025].

[47] Espressif Systems, "ESP-IDF Programming Guide," Version 5.x, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>. [Accessed: Oct. 10, 2025].

[48] Grafana Labs, "Grafana Documentation," 2024. [Online]. Available: <https://grafana.com/docs/grafana/latest/>. [Accessed: Jan. 15, 2026].

[49] Prometheus Authors, "Prometheus Documentation," 2024. [Online]. Available: <https://prometheus.io/docs/>. [Accessed: Jan. 15, 2026].

[50] Radix UI Contributors, "Radix UI - Unstyled, Accessible Components for React," 2024. [Online]. Available: <https://www.radix-ui.com/>. [Accessed: Dec. 15, 2025].

[51] Google, "Flutter Documentation - WebView Plugin," 2024. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: Jan. 20, 2026].

[52] Texas Instruments, "Power Management for IoT Devices," Application Report

SLVAE57, 2020. [Online]. Available: <https://www.ti.com/lit/an/slvae57/slvae57.pdf>. [Accessed: Sep. 15, 2025].

[53] STMicroelectronics, "STM32L476RE Product Page," 2026. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32l476re.html>. [Accessed: Mar. 02, 2026].

[54] STMicroelectronics, "STM32L476xx Datasheet," DS10198 Rev 11, Jul. 2024. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32l476re.pdf>. [Accessed: Mar. 02, 2026].

[55] Nordic Semiconductor, "nRF52840 Product Specification v1.2," [Online]. Available: https://docs.nordicsemi.com/bundle/nRF52840_PS_v1.2/resource/nRF52840_PS_v1.2.pdf. [Accessed: Mar. 02, 2026].

[56] Nordic Semiconductor, "nRF52840 Product Page," 2026. [Online]. Available: <https://www.nordicsemi.com/Products/nRF52840/Modules>. [Accessed: Mar. 02, 2026].

[57] Nordic Semiconductor, "nRF52840 Documentation Hub," 2026. [Online]. Available: <https://docs.nordicsemi.com/category/nrf52840-category>. [Accessed: Mar. 02, 2026].

[58] SIMCom Wireless Solutions, "A7670X Series Product Page," 2026. [Online]. Available: <https://cn.simcom.com/product/A7670X.html>. [Accessed: Mar. 02, 2026].

[59] Quectel Wireless Solutions, "EC200U Series Product Page," 2026. [Online]. Available: <https://www.quectel.com/product/lte-ec200u-series>. [Accessed: Mar. 02, 2026].

[60] SIMCom Wireless Solutions, "SIM7600CE Product Page," 2026. [Online]. Available: <https://en.simcom.com/product/SIM7600CE.html>. [Accessed: Mar. 13, 2026].

[61] Quectel Wireless Solutions, "Quectel EC200U Series LTE Standard Specification," Version 1.4, 2024. [Online]. Available: https://developer.quectel.com/en/wp-content/uploads/sites/2/2024/11/Quectel_EC200U_Series_LTE_Standard_Specification_V1.4.pdf. [Accessed: Mar. 02, 2026].

[62] u-blox, "NEO-M8 Data Sheet," UBX-15031086, [Online]. Available: https://www.u-blox.com/sites/default/files/NEO-M8-FW3_DataSheet_UBX-15031086.pdf. [Accessed: Mar. 02, 2026].

[63] u-blox, "NEO-M8 Series Product Page," 2026. [Online]. Available: <https://www.u-blox.com/en/product/neo-m8-series>. [Accessed: Mar. 02, 2026].

[64] vgate, "Vgate iCar Pro BLE Product Page," 2026. [Online]. Available: <https://vgatemall.com/products-detail/i-9/>. [Accessed: Mar. 02, 2026].

[65] Texas Instruments, "TPS2115A Product Page," 2026. [Online]. Available: <https://www.ti.com/product/TPS2115A>. [Accessed: Mar. 02, 2026].

[66] Infineon Technologies, "IRLML6402 Product Page," 2026. [Online]. Available: <https://www.infineon.com/part/IRLML6402>. [Accessed: Mar. 02, 2026].

[67] SONGLE Relay, "SRD-05VDC-SL-C Relay Datasheet," datasheet mirror. [Online]. Available: <https://www.alldatasheet.com/html-pdf/1132639/SONGLERELAY/SRD-05VDC-SL-C/1713/2/SRD-05VDC-SL-C.html>. [Accessed: Mar. 02, 2026].

PHỤ LỤC - APPENDICES

PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT

1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)

Bảng dưới đây liệt kê chi tiết các linh kiện chính sử dụng trong thiết bị theo dõi IoT, bao gồm đơn giá và tổng chi phí ước tính. Đơn giá được tham chiếu theo mặt bằng giá thị trường Việt Nam đã được rà soát lại vào ngày 12/04/2026 từ các kênh bán lẻ như Điện tử Á Châu, Điện tử Đức Huy và một số sàn thương mại điện tử trong nước đối với các linh kiện khó kiểm.

Bảng PL-1.1: Bảng kê chi phí linh kiện (BOM)

STT	Linh kiện	Model / Thông số	Số lượng	Đơn giá (VND)	Thành tiền (VND)	Ghi chú
1	Vi điều khiển	ESP32-S3-WROOM-1 (N16R8)	1	95.000	95.000	MCU chính, 16MB Flash, 8MB PSRAM
2	LTE + GNSS	SIM7600CE-T	1 bả»™	650.000	650.000	Modem LTE Cat-4 tích hợp GPS/GNSS
3	OBD2 Adapter	vgate iCar Pro BLE	1	367.000	367.000	Bluetooth Low Energy OBD2
4	Cảm biến gia tốc	IC LIS3DH	1	40.000	40.000	IMU 3 trục, phát hiện chuyển động
5	Pin dự phòng	18650 1S Li-ion 3500mAh	1	79.000	79.000	Samsung/LG cell
6	IC sạc pin	TP5100 + mạch phụ trợ	1	37.000	37.000	Sạc 1S + bảo vệ pin, input 5V từ MP2482
7	LDO 3.3V	Mạch AP2112-3.3	1	5.000	5.000	5V -> 3.3V cấp ESP32-S3
8	Buck 5V	Mạch MP2482 (12V/24V->5V)	1	17.000	17.000	Tạo bus 5V chính
9	Buck 3.8V/4V	Mạch TPS54231	1	9.000	9.000	12-24V -> 4V cấp modem SIM7600CE-T

Bảng PL-1.1: Bảng kê chi phí linh kiện (BOM) (tiếp theo)

STT	Linh kiện	Model / Thông số	Số lượng	Đơn giá (VND)	Thành tiền (VND)	Ghi chú
10	Boost converter	Mạch SX1308 (3.7V->5V)	1	12.000	12.000	Tăng áp từ pin dự phòng
11	Diode OR	SS54/SS34 (Schottky)	2	2.500	5.000	OR nguồn giữa nhánh 5V chính và dự phòng
12	Khởi LVD	Khởi giám sát LVD + ADC	1	8.000	8.000	Cho mạch Low Voltage Disconnect
13	Anten GPS	Anten gồm GNSS 25x25mm	1	25.000	25.000	Anten GPS, GLONASS, BeiDou
14	Anten 4G	Anten FPC 4G LTE	1	20.000	20.000	Anten mạng di động
15	SIM tray + SIM	Nano SIM holder + SIM 4G	1	15.000	15.000	SIM data 4G
16	Connector OBD2	Jack OBD2 16-pin male	1	35.000	35.000	Kết nối nguồn ắc quy xe (12V hoặc 24V)
17	Tụ điện, điện trở	Linh kiện thụ động (combo)	1 bộ	30.000	30.000	Tụ lọc, điện trở chia áp ADC (R1=100k, R2=10k), LED
18	PCB chính	PCB 2 lớp theo thiết kế của đề tài 50x50mm	1	20.000	20.000	Bo mạch chính của thiết bị
19	Hộp đựng	Hộp nhựa ABS 120 x 80 x 40 mm	1	25.000	25.000	Vỏ bảo vệ thiết bị
20	Dây kết nối	Dây nối, header, jumper	1 bộ	20.000	20.000	Dây kết nối nội bộ
				Tổng cộng	1.514.000	

1.2. Chi phí hạ tầng cloud (ước tính hàng tháng)

Bảng PL-1.2: Chi phí hạ tầng cloud

STT	Hạng mục	Thông số	Chi phí hàng tháng (VND)	Ghi chú
1	VPS Server	4 vCPU, 8GB RAM, 100GB SSD	300.000–500.000	DigitalOcean / Vultr / Viettel IDC
2	SIM 4G data	Gói cước data IoT	70.000	Mỗi thiết bị 1 SIM
3	Tên miền	Domain .com	25.000	300.000 VND/năm
4	SSL Certificate	Let's Encrypt	0	Miễn phí, tự động gia hạn
			Tổng hàng tháng	395.000–595.000

1.3. Tổng hợp chi phí dự án

Bảng PL-1.3: Tổng hợp chi phí dự án

Hạng mục	Chi phí (VND)	Ghi chú
Phần cứng (1 bộ thiết bị theo dõi)	1.514.000	Theo BOM ở bảng PL-1.1
Phần mềm (licenses)	0	Toàn bộ mã nguồn mở
Hạ tầng cloud (3 tháng dev + 3 tháng test)	2.370.000–3.570.000	6 tháng x chi phí hàng tháng
SIM 4G (6 tháng)	420.000	70.000 x 6 tháng
Công cụ phát triển (khác)	200.000	USB-UART, mỏ hàn, dây đo
Tổng chi phí dự án	4.504.000–5.704.000	

Nhận xét: Tổng chi phí dự án vẫn dưới 6.000.000 VND, trong đó chi phí phần cứng cho mỗi bộ thiết bị theo dõi khoảng 1.514.000 VND, thấp hơn đáng kể so với các giải pháp thương mại tương đương (2.000.000–5.000.000 VND/thiết bị). Khi sản xuất số lượng lớn (>50 bộ), chi phí linh kiện có thể giảm thêm 15–25% nhờ mua sỉ.

PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS

2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031–5)

Mô tả: On-Board Diagnostics II (OBD2) là tiêu chuẩn chẩn đoán xe được bắt buộc áp dụng cho tất cả các xe sản xuất từ năm 1996 (tại Mỹ) và 2001 (tại Châu Âu). Tiêu chuẩn này quy định giao diện vật lý (connector 16-pin), giao thức truyền thông, và các thông số chẩn đoán (PIDs - Parameter IDs).

Áp dụng trong dự án:

Bảng 2.1A: Tổng hợp thông tin về Chuẩn cụ thể, Nội dung, Áp dụng

Chuẩn cụ thể	Nội dung	Áp dụng
SAE J1979	Định nghĩa các Diagnostic Test Modes (Mode 01–0A)	Đọc dữ liệu động cơ: RPM, tốc độ, nhiệt độ, nhiên liệu
ISO 15031–5	Định nghĩa PIDs và định dạng dữ liệu	Phân tích (parsing) bản tin OBD2 response
ISO 15765–2	Transport protocol cho CAN bus (ISO-TP)	Xử lý multi-frame response (VIN, DTC list)
ISO 15765–4	Emissions-related OBD via CAN	Giao tiếp CAN qua OBD2 adapter BLE

Các OBD2 PIDs sử dụng trong dự án:

Bảng 2.1B: Tổng hợp thông tin về PID (hex), Mô tả, Đơn vị

PID (hex)	Mô tả	Đơn vị	Mode
0x00	Supported PIDs [01–20]	Bitmask	01
0x04	Calculated Engine Load	%	01
0x05	Engine Coolant Temperature	C	01
0x0C	Engine RPM	rpm	01
0x0D	Vehicle Speed	km/h	01
0x0F	Intake Air Temperature	C	01
0x11	Throttle Position	%	01
0x2F	Fuel Tank Level Input	%	01

Bảng 2.1B: Tổng hợp thông tin về PID (hex), Mô tả, Đơn vị (tiếp theo)

PID (hex)	Mô tả	Đơn vị	Mode
0x46	Ambient Air Temperature	C	01

2.2. MQTT 3.1.1 trong triển khai thiết bị

Mô tả: MQTT vẫn là giao thức nhắn tin phù hợp cho IoT vì gọn, ít overhead và hoạt động tốt trên kết nối 4G. Trong hệ thống này, EMQX chạy phiên bản 5.x ở vai trò broker, nhưng firmware trên thiết bị hiện cấu hình **MQTT 3.1.1**. Vì vậy, phần trình bày dưới đây tập trung vào các đặc tính thực sự đang dùng trong đường truyền thiết bị thay vì liệt kê các khả năng mở rộng của MQTT 5.

Các đặc tính MQTT đang dùng trong dự án:

Bảng 2.2A: Tổng hợp thông tin về Đặc tính, Mô tả, Áp dụng thực tế

Đặc tính	Mô tả	Áp dụng thực tế
QoS 0	Gửi không cần ACK	rawdata tần suất cao
QoS 1	Gửi có ACK, tránh mất bản tin quan trọng	status, events, firmware, commands
Topic phân phiên bản	Chuẩn hóa theo cây v1/{device_id}/...	Thiết bị publish và backend gửi lệnh
Topic lệnh hai chiều	Backend publish trực tiếp vào v1/{device_id}/commands	Điều khiển từ xa, OTA
Internal event bus	MQTT Bridge publish internal/events/# qua EMQX	Backend listener -> Socket.IO realtime
ACL theo thiết bị	Mỗi thiết bị chỉ được phép dùng topic của chính nó	Giảm rủi ro giả mạo publish/subscribe

Cấu trúc topic MQTT:

- v1/{device_id}/rawdata: dữ liệu telemetry chính (QoS 0).
- v1/{device_id}/events: sự kiện và cảnh báo (QoS 1).
- v1/{device_id}/commands: lệnh điều khiển từ server (QoS 1).
- v1/{device_id}/status: trạng thái phiên/heartbeat (QoS 1).
- v1/{device_id}/firmware: tiến trình OTA (QoS 1).

- `internal/events/device/status`: sự kiện nội bộ cho backend realtime.
- `internal/events/device/alert`: cảnh báo nội bộ.
- `internal/events/device/session`: trạng thái phiên nội bộ.
- `internal/events/device/data`: vị trí/telemetry cho realtime.

2.3. Bảo mật thông tin (ISO 27001 - tham khảo)

Mô tả: ISO 27001 là tiêu chuẩn quốc tế về hệ thống quản lý an toàn thông tin (ISMS). Dự án tham khảo các nguyên tắc của ISO 27001 để thiết kế các biện pháp bảo mật.

Các nguyên tắc bảo mật áp dụng:

Bảng 2.3A: Tổng hợp thông tin về Nguyên tắc, Hiện thực trong dự án

Nguyên tắc	Hiện thực trong dự án
Confidentiality (Bảo mật)	Mã hóa TLS cho MQTT và HTTPS; Token hash SHA-256
Integrity (Toàn vẹn)	MQTT checksum; Database constraints; Zod validation
Availability (Sẵn sàng)	Docker restart policy; Health checks; Pin dự phòng
Authentication (Xác thực)	Session-based auth với database-backed tokens
Authorization (Phân quyền)	RBAC (Role-Based Access Control); MQTT ACL per device
Audit Trail (Nhật ký)	VictoriaLogs ghi nhật ký mọi thao tác; Liên kết truy vết theo Request-ID

2.4. Thiết kế REST API (RFC 7231 và best practices)

Các nguyên tắc REST API áp dụng:

Bảng 2.4A: Tổng hợp thông tin về Nguyên tắc, Mô tả, Ví dụ trong dự án

Nguyên tắc	Mô tả	Ví dụ trong dự án
Resource-based URLs	URL đại diện cho tài nguyên, dùng số nhiều	<code>/api/v1/vehicles</code> , <code>/api/v1/alerts</code>
HTTP Methods	Dùng đúng ý nghĩa của phương thức	GET (đọc), POST (tạo), PUT (cập nhật), DELETE (xóa)
Status Codes	Mã trạng thái phản hồi chính xác	200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found

Bảng 2.4A: Tổng hợp thông tin về Nguyên tắc, Mô tả, Ví dụ trong dự án (tiếp theo)

Nguyên tắc	Mô tả	Ví dụ trong dự án
Pagination	Phân trang cho danh sách lớn	?page=1&limit=20&sort=created_at:desc
Versioning	Phiên bản API trong URL	/api/v1/...
Error Format	Định dạng lỗi nhất quán	"error":{"code":"...", "message":"..."}
Request-ID	Định danh mỗi request	Header X-Request-ID cho truy vết lỗi

PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES

3.1. Phân chia giai đoạn dự án

Dự án được triển khai theo 7 giai đoạn chính, với tổng thời gian dự kiến 24 tuần (6 tháng), từ tháng 09/2025 đến tháng 02/2026.

Bảng PL-3.1: Phân chia giai đoạn và thời gian thực hiện

Giai đoạn	Nội dung chính	Thời gian	Kết quả đầu ra
GD1	Nghiên cứu và thiết kế phần cứng	Tuần 1–4 (4 tuần)	Sơ đồ mạch, BOM, bố trí PCB
GD2	Phát triển firmware	Tuần 3–10 (8 tuần)	Firmware ESP-IDF hoàn chỉnh
GD3	Xây dựng hạ tầng cloud	Tuần 5–8 (4 tuần)	Docker infrastructure, EMQX, databases
GD4	Phát triển backend API	Tuần 7–14 (8 tuần)	REST API, MQTT Bridge, WebSocket
GD5	Phát triển frontend	Tuần 11–18 (8 tuần)	Web dashboard, bản đồ, biểu đồ
GD6	Tích hợp và kiểm thử	Tuần 17–22 (6 tuần)	Hệ thống tích hợp, báo cáo kiểm thử
GD7	Viết báo cáo và bảo vệ	Tuần 21–24 (4 tuần)	Báo cáo đồ án, slide thuyết trình

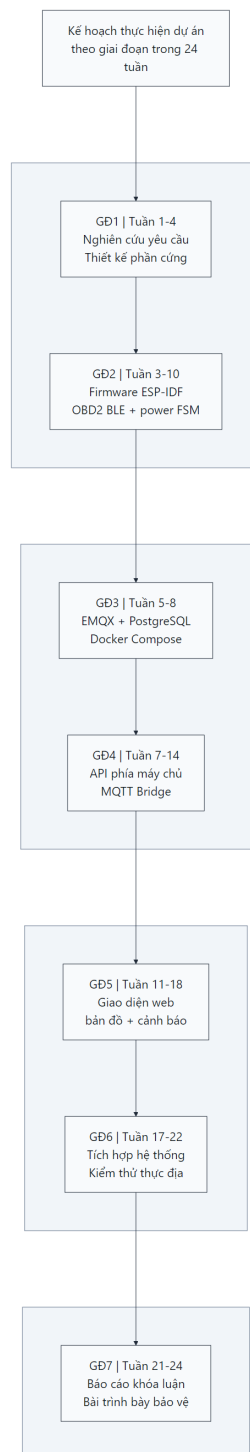
Ghi chú: Các giai đoạn có sự chồng chéo (overlap) có chủ đích để tối ưu hóa thời gian. Ví dụ, GD3 (hạ tầng cloud) bắt đầu trước khi GD2 (firmware) hoàn thành để có môi trường test sớm.

3.2. Kế hoạch thực hiện dự án theo giai đoạn

Kế hoạch thực hiện dự án theo giai đoạn được trình bày trực quan tại hình dưới đây, thể hiện đầy đủ 7 giai đoạn và phần chồng chéo tiến độ trong 24 tuần thực hiện dự án.

3.3. Chi tiết công việc theo giai đoạn

Giai đoạn 1 — Nghiên cứu và thiết kế phần cứng (Tuần 1–4):



Hình PL-3.1: Kế hoạch thực hiện dự án theo giai đoạn (24 tuần)

Bảng 3.3A: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
1	Khảo sát linh kiện, đặt mua	Danh sách linh kiện
2	Thiết kế sơ đồ nguyên lý	Sơ đồ nguyên lý hoàn chỉnh
3	Thiết kế PCB và đặt làm mạch	Bố trí PCB / bo mạch hoàn chỉnh
4	Lắp ráp và kiểm tra cơ bản	Thiết bị phần cứng hoạt động

Giai đoạn 2 — Phát triển firmware (Tuần 3–10):

Bảng 3.3B: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
3–4	Cấu hình ESP-IDF, GPIO, UART cho modem	Giao tiếp modem cơ bản
5–6	Mô-đun BLE OBD2, phân tích bản tin	Đọc dữ liệu OBD2
7–8	Mô-đun MQTT, tự kết nối lại, QoS	Truyền dữ liệu lên broker
9–10	Quản lý năng lượng, IMU, tích hợp	Firmware hoàn chỉnh

Giai đoạn 3 — Hạ tầng cloud (Tuần 5–8):

Bảng 3.3C: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
5	Thiết lập Docker, PostgreSQL, EMQX	Hạ tầng cơ bản
6	VictoriaMetrics, VictoriaLogs	Chuỗi thời gian và nhật ký
7	Dịch vụ MQTT Bridge	Dữ liệu từ EMQX đến DB
8	Grafana, giám sát, sao lưu	Bảng điều khiển giám sát

Giai đoạn 4 — API phía máy chủ (Tuần 7–14):

Bảng 3.3D: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
7–8	Khởi tạo dự án, mô-đun xác thực	Đăng nhập, phiên làm việc
9–10	Miền xe, thiết bị, khách hàng	API quản lý dữ liệu

Bảng 3.3D: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm (tiếp theo)

Tuần	Công việc	Sản phẩm
11–12	Miền dữ liệu vận hành, cảnh báo, vùng cho phép	Dữ liệu thời gian thực
13–14	WebSocket, lệnh điều khiển, kiểm thử	API hoàn chỉnh

Giai đoạn 5 — Frontend (Tuần 11–18):

Bảng 3.3E: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
11–12	Khởi tạo dự án, bố cục, trang xác thực	Khung ứng dụng
13–14	Trang tổng quan, bản đồ Leaflet	Trang chủ, bản đồ
15–16	Quản lý xe, cảnh báo, vùng cho phép	Các trang quản lý
17–18	ECharts, WebSocket, hoàn thiện giao diện	Giao diện hoàn chỉnh

Giai đoạn 6 — Tích hợp và kiểm thử (Tuần 17–22):

Bảng 3.3F: Tổng hợp thông tin về Tuần, Công việc, Sản phẩm

Tuần	Công việc	Sản phẩm
17–18	Tích hợp firmware - cloud	Hệ thống end-to-end
19–20	Kiểm thử chức năng, hiệu năng	Báo cáo kiểm thử
21–22	Sửa lỗi, tối ưu, triển khai thử nghiệm	Hệ thống ổn định

PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS

4.1. Kho mã nguồn của dự án

Toàn bộ mã nguồn phục vụ việc dựng lại và kiểm chứng hệ thống được lưu trữ trên GitHub:

- **Địa chỉ kho mã nguồn:** https://github.com/anmh1205/IoT_Vehicle_Tracking_System
- **Nhánh tích hợp chính:** main
- **Nhánh triển khai thử nghiệm:** uat

Cấu trúc thư mục chính:

- `iot-vehicle-tracking-system-cloud/`: chứa toàn bộ dịch vụ vận hành (Tracking_Backend, Tracking_Frontend, Tracking_Mobile, Tracking_MqttBridge, Tracking_PostgreSQL, Tracking_EMQX, Tracking_VictoriaMetrics, Tracking_VictoriaLogs, Tracking_Grafana, Tracking_NPM, Tracking_Data).
- `resources/`: tài liệu dự án, báo cáo, kế hoạch.
- `README.md`: tệp hướng dẫn tổng quan về cách dựng và vận hành hệ thống.

4.2. Tham chiếu cơ sở dữ liệu (Database Schema)

Schema PostgreSQL đầy đủ được lưu tại các tệp SQL khởi tạo trong dịch vụ PostgreSQL; phần phụ lục này chỉ tóm tắt các bảng chính phục vụ luận văn.

Các bảng chính trong PostgreSQL:

Bảng 4.2A: Tổng hợp thông tin về STT, Tên bảng, Mô tả

STT	Tên bảng	Mô tả	Quan hệ chính
1	users	Tài khoản người dùng	1:N với sessions, audit_logs
2	sessions	Phiên đăng nhập (token hash SHA-256)	N:1 với users
3	vehicles	Thông tin phương tiện	1:1 với devices, 1:N với trips
4	devices	Thông tin thiết bị theo dõi	N:1 với vehicles
5	customers	Thông tin khách hàng	1:N với trips
6	trips	Hành trình cho thuê	N:1 với vehicles, customers
7	alerts	Cảnh báo hệ thống	N:1 với vehicles, devices
8	geofences	Vùng cho phép	N:N với vehicles
9	commands	Lệnh điều khiển thiết bị	N:1 với devices

Bảng 4.2A: Tổng hợp thông tin về STT, Tên bảng, Mô tả (tiếp theo)

STT	Tên bảng	Mô tả	Quan hệ chính
10	audit_logs	Nhật ký thao tác	N:1 với users

Các chỉ số trong VictoriaMetrics:

Bảng 4.2B: Tổng hợp thông tin về Metric name mẫu, Labels, Mô tả

Metric name mẫu	Labels	Mô tả
vehicle_latitude	device_id	Vĩ độ GNSS
vehicle_longitude	device_id	Kinh độ GNSS
vehicle_speed	device_id	Tốc độ
vehicle_vibration	device_id	Mức rung từ IMU
vehicle_battery_top	device_id	Điện áp nguồn phía trên
vehicle_battery_bot	device_id	Điện áp pin dự phòng
vehicle_ignition	device_id	Trạng thái ignition (0/1)
vehicle_uptime	device_id	Thời gian chạy tích lũy

4.3. Tài liệu API (API Documentation)

Tài liệu API đầy đủ được tạo tự động bằng Swagger/OpenAPI và có thể truy cập tại:

- **URL (môi trường thử nghiệm):** <https://api.thingdock.dev/api-docs>
- **URL (môi trường phát triển):** <http://localhost:4000/api-docs>

Tổng hợp API endpoints chính:

Bảng 4.3A: Tổng hợp thông tin về Nhóm, Method, Endpoint

Nhóm	Method	Endpoint	Mô tả
Auth	POST	/api/v1/auth/login	Đăng nhập
Auth	POST	/api/v1/auth/logout	Đăng xuất
Auth	GET	/api/v1/auth/me	Thông tin người dùng hiện tại
Vehicles	GET	/api/v1/vehicles	Danh sách phương tiện
Vehicles	POST	/api/v1/vehicles	Thêm phương tiện

Bảng 4.3A: Tổng hợp thông tin về Nhóm, Method, Endpoint (tiếp theo)

Nhóm	Method	Endpoint	Mô tả
Vehicles	GET	/api/v1/vehicles/:id	Chi tiết phương tiện
Devices	GET	/api/v1/devices	Danh sách thiết bị
Devices	POST	/api/v1/devices	Đăng ký thiết bị mới
Devices	POST	/api/v1/devices/:id/ota	Kích hoạt OTA cho thiết bị
Devices	POST	/api/v1/devices/:id/ota/rollback	Yêu cầu rollback firmware
Telemetry	GET	/api/v1/devices/:id/telemetry	Dữ liệu telemetry theo thiết bị
Telemetry	GET	/api/v1/telemetry/history	Truy vấn lịch sử telemetry
Alerts	GET	/api/v1/alerts	Danh sách cảnh báo
Alerts	PUT	/api/v1/alerts/:id/acknowledge	Xác nhận cảnh báo
Vùng cho phép	GET	/api/v1/geofences	Danh sách vùng cho phép
Vùng cho phép	POST	/api/v1/geofences	Tạo vùng cho phép mới
Commands	POST	/api/v1/devices/:id/command	Gửi lệnh đến thiết bị
Trips	GET	/api/v1/trips	Danh sách hành trình
Customers	GET	/api/v1/customers	Danh sách khách hàng

4.4. Tài liệu cấu hình môi trường (Environment Configuration)

Mỗi dịch vụ yêu cầu file `.env` riêng. Mẫu cấu hình (`.env.example`) có sẵn ở các dịch vụ có cung cấp file mẫu; riêng Frontend dùng trực tiếp file `.env` theo tệp hướng dẫn đi kèm kho mã nguồn.

Các biến môi trường quan trọng:

Bảng 4.4A: Tổng hợp thông tin về Biến, Dịch vụ, Mô tả

Biến	Dịch vụ	Mô tả	Bắt buộc
POSTGRES_HOST	Backend	Địa chỉ PostgreSQL server	Có
POSTGRES_PORT	Backend	Cổng PostgreSQL (mặc định: 5432)	Có
POSTGRES_DATABASE	Backend	Tên cơ sở dữ liệu	Có
POSTGRES_USER	Backend	Tên đăng nhập PostgreSQL	Có
POSTGRES_PASSWORD	Backend	Mật khẩu PostgreSQL	Có
VICTORIAMETRICS_URL	MqttBridge	URL VictoriaMetrics	Có
VICTORIALOGS_URL	MqttBridge	URL VictoriaLogs	Có
MQTT_BROKER_URL	Backend, MqttBridge	URL EMQX broker	Có
FIRMWARE_PUBLIC_BASE_URL	Backend	Base URL công khai để thiết bị tải file OTA	Có cho OTA

Bảng 4.4A: Tổng hợp thông tin về Biến, Dịch vụ, Mô tả (tiếp theo)

Biến	Dịch vụ	Mô tả	Bắt buộc
SESSION_SECRET	Backend	Khóa bí mật cho phiên đăng nhập (min 32 ký tự)	Có
CORS_ORIGIN	Backend	URL frontend cho CORS	Có

Lưu ý bảo mật: Tất cả mật khẩu và khóa bí mật đều bắt buộc, không có giá trị mặc định. Hệ thống sẽ từ chối khởi động nếu thiếu bất kỳ biến nào.

4.5. Hướng dẫn cài đặt và chạy hệ thống

Yêu cầu hệ thống:

Bảng 4.5A: Tổng hợp thông tin về Yêu cầu, Phiên bản tối thiểu

Yêu cầu	Phiên bản tối thiểu
Node.js	20.x LTS
Docker	24.x
Docker Compose	2.x
Git	2.x
npm	10.x

Các bước cài đặt:

1. Clone kho mã nguồn

```
git clone https://github.com/anmh1205/IoT_Vehicle_Tracking_System.git
cd IoT_Vehicle_Tracking_System
```

2. Tạo Docker network

```
docker network create tracking-network
```

3. Khởi động infrastructure services

```
cd iot-vehicle-tracking-system-cloud/Tracking_PostgreSQL && docker compose up -d
cd ../Tracking_EMQX && docker compose up -d
cd ../Tracking_VictoriaMetrics && docker compose up -d
cd ../Tracking_VictoriaLogs && docker compose up -d
cd ../Tracking_Grafana && docker compose up -d
```

4. Cấu hình .env cho Backend

```
cd ../Tracking_Backend
cp .env.example .env
# Chỉnh sửa .env với các giá trị phù hợp
```

5. Cài đặt và chạy Backend

```
npm ci
npm run dev
```

6. Cấu hình và chạy Frontend

```
cd ../Tracking_Frontend
# Chuẩn bị file .env theo tệp hướng dẫn
npm ci
npm run dev
```

7. Chạy MQTT Bridge

```
cd ../Tracking_MqttBridge
cp .env.example .env
npm ci
npm run dev
```

Truy cập hệ thống:

Bảng 4.5B: Tổng hợp thông tin về Dịch vụ, URL, Tài khoản mặc định

Dịch vụ	URL	Tài khoản mặc định
Giao diện web	https://thingdock.dev	Tài khoản quản trị được cấu hình riêng khi triển khai
API phía máy chủ	https://api.thingdock.dev	—
Tài liệu API (Swagger)	https://api.thingdock.dev/api-docs	—
Kênh MQTT qua WebSocket	wss://mqtt.thingdock.dev	—

Bảng 4.5B: Tổng hợp thông tin về Dịch vụ, URL, Tài khoản mặc định (tiếp theo)

Dịch vụ	URL	Tài khoản mặc định
broker MQTT qua TLS	mqtt: //mqtt.thingdock.dev:8883	Theo tài khoản MQTT của thiết bị
Grafana	https: //grafana.thingdock.dev	Tài khoản quản trị được cấu hình riêng khi triển khai
Bảng điều khiển EMQX	https://emqx.thingdock.dev	Tài khoản quản trị được cấu hình riêng khi triển khai
Trang quản trị NPM	https://npm.thingdock.dev	Tài khoản quản trị được cấu hình khi khởi tạo hệ thống

Ghi chú môi trường thử nghiệm: VictoriaMetrics được giữ ở chế độ nội bộ, không công khai qua tên miền phụ. Khi cần phục vụ vận hành, dịch vụ này được truy cập từ mạng Docker nội bộ hoặc có thể công bố thêm một tên miền phụ riêng ở giai đoạn sau.